Arquitectura hexagonal y su distribución en microservicios

De un monolito hexagonal en Java 21 y Spring Boot a tres servicios con GuildBoard

Fábrica de Cursos
10 de octubre de 2026

Índice general

Cómo usar este libro **x**

I Bases **1**

1. Repaso I: Java 21 moderno y SOLID **2**

1.1. Cómo usar este repaso . . . 2
1.2. Comprobar el entorno y crear el esqueleto Gradle . . . 2
1.3. Records y valores inmutables . . . 4
1.3.1. Validar en el constructor compacto . . . 5
1.3.2. Inmutabilidad superficial . . . 5
1.4. Sealed y pattern matching para modelar estados . . . 6
1.5. SOLID: los cinco principios . . . 7
1.5.1. SRP, OCP, LSP e ISP con ejemplos del gremio . . . 8
1.5.2. DIP en detalle: la base de los puertos . . . 8
1.6. En el proyecto: H0, el esqueleto de GuildBoard . . . 10
1.7. Ejercicios . . . 10
1.8. Resumen . . . 14
Soluciones de los ejercicios . . . 14

2. Repaso II: DDD básico **22**

2.1. Por qué DDD antes de la arquitectura . . . 22
2.2. Lenguaje ubicuo y glosario de GuildBoard . . . 22
2.3. Entidades y objetos de valor . . . 23
2.3.1. Objeto de valor . . . 23
2.3.2. Entidad . . . 24
2.4. Agregados y repositorios . . . 25
2.5. Dominio anémico frente a rico . . . 26
2.5.1. Una regla que ningún agregado puede garantizar solo . . . 27
2.6. Bounded contexts: solo la idea . . . 27
2.7. En el proyecto: hito H0, el documento de dominio . . . 28
2.8. Ejercicios . . . 28
2.9. Resumen . . . 32
Soluciones de los ejercicios . . . 32

3. La arquitectura hexagonal y su estructura de carpetas **42**
3.1. El problema: la lógica enredada con el exterior . . . 42
3.2. Hexagonal, onion y Clean: la regla de dependencia . . . 43
3.3. Dónde pone cada fuente las carpetas . . . 44
3.3.1. La estructura que adopta el curso, y en qué se diferencia de Buckpal . . . 45
3.4. Verificación de estructura 1: el árbol inicial . . . 46
3.5. Lombok: instalación y uso . . . 48
3.6. El dominio mínimo y los dos puertos . . . 49
3.7. JUnit mínimo . . . 52
3.8. Comprobar la regla de dependencia con ArchUnit . . . 54
3.9. En el proyecto: hito H1, parte 1 . . . 56
3.10. Ejercicios . . . 57
3.11. Resumen . . . 60
Soluciones de los ejercicios . . . 61

4. Casos de uso y adaptadores: el primer endpoint **73**
4.1. El caso de uso como frontera de entrada . . . 73
4.1.1. El comando . . . 74
4.1.2. El servicio de aplicación . . . 74
4.2. Repaso mínimo de Spring: el contenedor de beans . . . 75
4.3. Añadir Spring Boot al proyecto . . . 77
4.3.1. Initializr: de dónde salen los nombres . . . 77
4.3.2. Configuración con `application.yml` . . . 79
4.4. El adaptador web: controlador y DTO . . . 80
4.5. El adaptador en memoria . . . 82
4.6. La raíz de composición . . . 83
4.6.1. Por qué el núcleo no lleva `@Service` . . . 84
4.7. Ejecutar y probar con curl . . . 85
4.8. Estructura: sin verificación propia . . . 86
4.9. En el proyecto: hito H1 completo . . . 87
4.10. Ejercicios . . . 87
4.11. Resumen . . . 91
Soluciones de los ejercicios . . . 91

II Hexagonal en un monolito **108**

5. Un dominio rico: invariantes y errores de dominio **109**
5.1. De entidad con setters a agregado con comportamiento . . . 109
5.2. Rank y Reward como objetos de valor . . . 110

5.3. Errores de dominio como jerarquía sellada . . . 111
5.4. Las reglas de contexto: rango y límite de misiones . . . 114
5.5. Los casos de uso de H2 y sus endpoints . . . 115
5.6. Probar el dominio sin Spring . . . 120
5.7. Lo que este diseño no garantiza . . . 122
5.7.1. El reparto de reglas que se hereda . . . 122
5.8. Estructura: sin cambios en el árbol . . . 123
5.9. En el proyecto: hito H2 . . . 123
5.9.1. Probar el hito con curl . . . 124
5.10. Ejercicios . . . 125
5.11. Resumen . . . 128
Soluciones de los ejercicios . . . 129

6. Los límites: DTO, mapeo manual, errores HTTP y más puertos de salida 166
6.1. Por qué no devolver el dominio por la API . . . 166
6.2. Verificación de estructura 2: los paquetes nuevos . . . 167
6.3. DTO y mapeadores a mano . . . 169
6.4. Validar en el borde y proteger invariantes en el dominio . . . 171
6.5. Errores de dominio a HTTP . . . 173
6.6. El tiempo como dependencia: Clock y caducidad . . . 174
6.7. NotificationSender: un puerto de salida sin base de datos . . . 177
6.8. En el proyecto: hito H3, parte 1 . . . 179
6.8.1. Probar el hito con curl . . . 179
6.9. Ejercicios . . . 180
6.10. Resumen . . . 183
Soluciones de los ejercicios . . . 184

7. Hacer cumplir la estructura: Gradle multimódulo 217
7.1. Por qué una convención no basta . . . 217
7.2. Verificación de estructura 3: el árbol multimódulo . . . 218
7.2.1. El árbol anterior . . . 218
7.2.2. El árbol nuevo . . . 219
7.2.3. El grafo de dependencias . . . 219
7.2.4. Cambios respecto al árbol anterior . . . 220
7.2.5. Comprobaciones . . . 220
7.3. Principios de componentes: qué se agrupa y quién depende de quién . . . 221
7.4. Módulos Gradle: `java-library`, api e implementation . . . 221
7.4.1. Declarar los módulos . . . 221
7.4.2. El plugin java-library y las dos formas de depender . . . 222

7.4.3. Lombok y el resto de configuración, módulo a módulo . . . 222
7.5. Cortar el monolito en cuatro módulos . . . 223
7.5.1. Mover las carpetas . . . 224
7.5.2. El settings.gradle y el build.gradle raíz . . . 224
7.5.3. El build.gradle de cada módulo . . . 224
7.5.4. Qué ve cada módulo . . . 226
7.5.5. ArchitectureTest se muda a bootstrap . . . 226
7.5.6. Qué errores aparecen ahora si alguien rompe la regla . . . 226
7.6. Un módulo por adaptador frente a un módulo de adaptadores . . . 228
7.7. Lo que el compilador no ve: el aislamiento entre adaptadores . . . 229
7.8. En el proyecto: hito H3, parte 2 . . . 229
7.8.1. Probar el hito . . . 229
7.9. Ejercicios . . . 230
7.10. Resumen . . . 234
Soluciones de los ejercicios . . . 235

8. Cambiar de adaptador: de memoria a PostgreSQL 246
8.1. Qué cambia y qué no al cambiar de base de datos . . . 246
8.2. Verificación de estructura 4: el árbol con persistencia . . . 247
8.2.1. El árbol anterior . . . 247
8.2.2. El árbol nuevo . . . 248
8.2.3. Cambios respecto al árbol anterior . . . 249
8.2.4. Comprobaciones . . . 249
8.3. PostgreSQL con Docker Compose . . . 249
8.3.1. El archivo compose.yaml . . . 250
8.3.2. Los comandos que usarás . . . 251
8.4. La entidad JPA no es el dominio . . . 252
8.4.1. Por qué no se anota el dominio . . . 253
8.4.2. La entidad de misión . . . 253
8.4.3. Del dominio a la entidad, y de vuelta . . . 255
8.5. El adaptador JPA y el puerto de salida . . . 257
8.5.1. El repositorio de Spring Data . . . 257
8.5.2. El adaptador . . . 257
8.5.3. Por qué `@Transactional` va aquí . . . 258
8.6. Migraciones con Flyway . . . 259
8.7. Elegir el adaptador por perfil . . . 260
8.7.1. El problema: configuración automática sin base de datos . . . 260
8.7.2. Perfiles de Spring . . . 261
8.7.3. Apagar la autoconfiguración de datos en el modo memoria . . . 261

8.7.4. Las dos clases de configuración . . . 262
8.7.5. Arrancar de las dos formas . . . 264
8.8. ¿Sigue valiendo un solo módulo de adaptadores? . . . 265
8.9. En el proyecto: hito H4 . . . 266
8.9.1. Probar el hito . . . 266
8.10. Ejercicios . . . 268
8.11. Resumen . . . 271
Soluciones de los ejercicios . . . 272

III Del hexágono a los microservicios 296

9. De un monolito a servicios: fronteras y primer corte 297
9.1. Qué es un microservicio y qué no . . . 297
9.2. Dónde cortar: los contextos de GuildBoard . . . 298
9.3. Verificación de estructura 5: el árbol por servicios . . . 299
9.3.1. El árbol anterior . . . 299
9.3.2. El árbol nuevo . . . 300
9.3.3. Cambios respecto al árbol anterior . . . 301
9.3.4. Los nombres de proyecto Gradle . . . 301
9.3.5. Comprobaciones . . . 302
9.4. Una base de datos por servicio . . . 302
9.5. El contrato entre servicios . . . 303
9.6. RestClient como adaptador de salida . . . 305
9.7. El corte, paso a paso . . . 306
9.7.1. Paso 1: mover el monolito a Quests . . . 306
9.7.2. Paso 2: repartir los archivos de aventureros . . . 306
9.7.3. Paso 3: renombrar los paquetes . . . 307
9.7.4. Paso 4: Gradle . . . 308
9.7.5. Paso 5: un ArchitectureTest por servicio . . . 309
9.7.6. Paso 6: Quests deja de conocer los aventureros . . . 312
9.8. En el proyecto: hito H5 (parte 1) . . . 317
9.8.1. Probar el hito . . . 317
9.9. Ejercicios . . . 319
9.10. Resumen . . . 322
Soluciones de los ejercicios . . . 323

10. Tercer servicio y plantilla de servicio: Treasury 342
10.1. Qué hace Treasury y por qué es un servicio aparte . . . 342
10.2. Verificación de estructura 6: tres servicios, un mismo molde . . . 343

10.2.1. El árbol anterior . . . 343
10.2.2. El árbol nuevo . . . 344
10.2.3. Cambios respecto al árbol anterior . . . 345
10.2.4. Comprobaciones . . . 345
10.3. Reproducir la estructura de un servicio . . . 345
10.4. El libro de cuentas . . . 347
10.4.1. El dominio . . . 348
10.4.2. La aplicación: puertos y casos de uso . . . 350
10.5. Idempotencia: pagar una misión una sola vez . . . 353
10.6. Persistencia de Treasury: memoria y JPA . . . 355
10.7. El adaptador REST provisional . . . 358
10.8. Pruebas que vigilan los servicios . . . 361
10.8.1. java.nio.file mínimo . . . 361
10.8.2. La raíz gana el plugin java y JUnit . . . 361
10.8.3. StructureTest . . . 363
10.8.4. Independencia entre servicios con slices . . . 364
10.8.5. El ArchitectureTest de Treasury . . . 365
10.9. En el proyecto: hito H5 (parte 2) . . . 365
10.9.1. Probar el hito . . . 365
10.10Ejercicios . . . 367
10.11Resumen . . . 370
Soluciones de los ejercicios . . . 371

11.Mensajería: eventos entre servicios con RabbitMQ 397
11.1. Por qué un evento y no una llamada . . . 397
11.2. Verificación de estructura 7: los paquetes de mensajería . . . 398
11.2.1. El árbol anterior . . . 398
11.2.2. El árbol nuevo . . . 399
11.2.3. Cambios respecto al árbol anterior . . . 400
11.2.4. Comprobaciones . . . 401
11.3. RabbitMQ en Compose: exchange, colas y enlaces . . . 401
11.4. El núcleo sigue sin conocer el broker . . . 404
11.4.1. Publicar: un puerto de salida y RabbitTemplate . . . 404
11.4.2. Declarar exchange, colas y enlaces: Declarables . . . 406
11.4.3. Consumir: un adaptador de entrada con @RabbitListener . . . 407
11.5. El evento como mensaje JSON . . . 408
11.5.1. El conversor: qué se verificó y qué no . . . 409
11.6. Mensajes duplicados: consumidor idempotente . . . 411
11.7. Mensajes que fallan: reintentos y cola de mensajes muertos . . . 412

11.8. Lo que esto no resuelve: el evento perdido . . . 413
11.9. Por qué RabbitMQ y no Kafka (contexto) . . . 415
11.10En el proyecto: hito H6 (parte 1) . . . 415
11.10.1Probar el hito . . . 416
11.11Ejercicios . . . 417
11.12Resumen . . . 420
Soluciones de los ejercicios . . . 421

12.Resiliencia básica: timeouts, reintentos y circuit breaker 443
12.1. Qué falla en una llamada de red . . . 443
12.2. Este capítulo no cambia la estructura . . . 444
12.3. Timeouts: siempre . . . 445
12.4. Reintentos: solo en operaciones seguras . . . 446
12.5. Circuit breaker: cerrado, abierto y semiabierto . . . 447
12.6. Resilience4j en el adaptador de salida, nunca en el dominio . . . 447
12.6.1. La configuración: un mismo YAML para las dos formas . . . 449
12.6.2. Con anotaciones . . . 450
12.6.3. Con la API programática . . . 450
12.7. Cuando no hay respuesta: ¿se rechaza o se degrada? . . . 453
12.8. Por qué no `@Retryable` de Spring Framework 7 . . . 453
12.9. En el proyecto: hito H6 (parte 2) . . . 454
12.9.1. Probar el hito . . . 454
12.10Ejercicios . . . 455
12.11Resumen . . . 458
Soluciones de los ejercicios . . . 459

13.API gateway y configuración centralizada 485
13.1. Qué problema resuelve el gateway (y cuáles crea) . . . 485
13.2. Dos proyectos sin hexágono: el límite parcial . . . 486
13.3. Verificación de estructura 8: infraestructura en la raíz . . . 487
13.3.1. El árbol anterior . . . 487
13.3.2. El árbol nuevo . . . 487
13.3.3. Cambios respecto al árbol anterior . . . 488
13.3.4. Comprobaciones . . . 489
13.4. Rutas, predicados y filtros . . . 489
13.5. Config Server: la configuración fuera del jar . . . 491
13.6. Los servicios como clientes de configuración . . . 494
13.6.1. Qué se mueve y qué no . . . 496
13.6.2. Orden de arranque y lo que no hace esta configuración . . . 496

13.7. Crear el gateway . . . 497
13.8. En el proyecto: hito H7 (parte 1) . . . 499
13.8.1. Probar el hito . . . 499
13.9. Ejercicios . . . 500
13.10Resumen . . . 504
Soluciones de los ejercicios . . . 505

14.Seguridad: JWT validado en cada servicio 518
14.1. Verificación de estructura 9: el emisor y el paquete de seguridad . . . 519
14.1.1. El árbol anterior . . . 519
14.1.2. El árbol nuevo . . . 519
14.1.3. Cambios respecto al árbol anterior . . . 520
14.1.4. Comprobaciones . . . 521
14.2. Qué es un JWT: cabecera, claims y firma . . . 521
14.2.1. Decodificar con la shell . . . 521
14.2.2. Qué comprueba un servicio al recibir un token . . . 522
14.3. El emisor de tokens de desarrollo . . . 523
14.4. Resource server: cada servicio valida el token . . . 528
14.4.1. Qué verá el cliente . . . 531
14.5. Roles: de la claim a las autoridades . . . 531
14.6. Dónde autorizar: el borde, no el dominio . . . 532
14.7. Pasar el token de Quests a Adventurers . . . 533
14.7.1. Un interceptor del RestClient . . . 534
14.7.2. El gateway y los mensajes . . . 535
14.8. En el proyecto: hito H7 (parte 2) . . . 535
14.8.1. Probar el hito . . . 536
14.8.2. Lo que este capítulo no resuelve . . . 538
14.9. Ejercicios . . . 538
14.10Resumen . . . 542
Soluciones de los ejercicios . . . 542

15.Observabilidad y todo el sistema en Docker Compose 563
15.1. Verificación de estructura 10: el árbol final . . . 564
15.1.1. El árbol anterior . . . 564
15.1.2. El árbol final . . . 564
15.1.3. Cambios respecto al árbol anterior . . . 565
15.1.4. Comprobaciones . . . 566
15.2. Tres señales: métricas, trazas y logs . . . 566
15.2.1. Micrometer: una fachada para medir y trazar . . . 567

15.3. Actuator: salud y métricas de cada servicio . . . 567
15.3.1. Health: ¿estás vivo? . . . 568
15.3.2. El conflicto con la seguridad . . . 568
15.3.3. Metrics: números de lo que ha pasado . . . 569
15.4. Trazas entre servicios . . . 570
15.4.1. Qué se instala y por qué Brave con Zipkin . . . 570
15.4.2. Los logs, con el identificador . . . 571
15.4.3. La trampa del RestClient . . . 572
15.4.4. La mensajería . . . 574
15.5. Empaquetar cada servicio: el Dockerfile . . . 574
15.6. Compose completo: red, nombres de servicio y orden de arranque . . . 576
15.6.1. Una red, un DNS . . . 576
15.6.2. El perfil docker . . . 576
15.6.3. Dos variables de entorno por cliente . . . 578
15.6.4. El Config Server dentro de un contenedor . . . 579
15.6.5. Healthchecks y orden de arranque . . . 579
15.7. En el proyecto: hito H8 . . . 581
15.7.1. Probar el hito . . . 581
15.7.2. Lo que este capítulo no resuelve . . . 583
15.7.3. Qué se verificó y qué no (9 de octubre de 2026) . . . 583
15.8. Cierre del curso: qué cambiaría en un proyecto real . . . 584
15.8.1. Lo que no se movió . . . 584
15.8.2. Las diez verificaciones de estructura, de un vistazo . . . 585
15.8.3. Qué cambiaría en un proyecto real . . . 585
15.9. Ejercicios . . . 586
15.10Resumen . . . 591
Soluciones de los ejercicios . . . 592

Resumen final **619**

Referencias **620**

Cómo usar este libro

A quién va dirigido

Este libro es para ti si ya programas en Java 21 y has usado Spring Boot, aunque no digas que los dominas: con unas bases basta. No hace falta saber DDD, SOLID ni microservicios; los dos primeros capítulos repasan Java moderno, SOLID y DDD básico, y de ahí en adelante se explica todo lo demás. Doy por supuesto que trabajas en Linux con NeoVim (o el editor que prefieras), y que tienes instalados JDK 21, Gradle, Git y Docker, y que ya sabes usar Docker con soltura. El objetivo es práctico: poder diseñar y mantener servicios reales con esta arquitectura.

Qué vas a aprender

Aprenderás qué es la arquitectura hexagonal (puertos y adaptadores), cómo se organiza un proyecto de calidad con ella y cómo se reparte después en microservicios que se hablan por REST y por mensajería. El libro tiene tres partes y quince capítulos:

- **Parte I, Bases** (capítulos 1 a 4): repaso de Java 21 y SOLID, repaso de DDD básico, la arquitectura hexagonal con su estructura de carpetas y el primer caso de uso con su endpoint.
- **Parte II, Arquitectura hexagonal en un monolito** (capítulos 5 a 8): un dominio rico con invariantes, los límites (DTO, mapeo manual y errores HTTP), cómo hacer cumplir la estructura con Gradle multimódulo y el cambio de la persistencia en memoria a PostgreSQL.
- **Parte III, Del hexágono a los microservicios** (capítulos 9 a 15): fronteras y primer corte, un tercer servicio y una plantilla de servicio, mensajería con RabbitMQ, resiliencia básica, API gateway y configuración centralizada, seguridad con JWT en cada servicio, y observabilidad con todo el sistema en Docker Compose.

Quedan fuera Kubernetes y la nube. En el capítulo 11 solo se compara RabbitMQ con Kafka como lectura de contexto, sin ejercicio. Tampoco verás sagas, patrón *outbox* ni consistencia distribuida avanzada: el proyecto se diseña para evitarlas, y cuando una limitación te afecte, el libro lo dice.

El proyecto del curso

Construirás *GuildBoard*, el tablón de misiones de un gremio de aventureros, expuesto como API REST. El maestro del gremio publica misiones, los aventureros las aceptan, las completan y cobran la recompensa; con ello ganan experiencia y suben de rango. Las reglas son sencillas y serias: una misión pasa por los estados publicada, aceptada y completada; un aventurero de rango bajo no puede aceptar misiones de rango alto; y nadie lleva más de tres misiones activas. Esas reglas dan al dominio invariantes reales, y eso hace necesarias las ideas del curso.

El proyecto crece en hitos:

- **H0**: esqueleto Gradle y repositorio Git, y el documento de dominio (capítulos 1 y 2).
- **H1**: monolito hexagonal mínimo con publicar misión, adaptador REST y repositorio en memoria (capítulos 3 y 4).
- **H2**: dominio completo con aventureros, rango y recompensa, aceptar y completar misiones (capítulo 5).
- **H3**: puertos de salida adicionales, mapeos a mano y módulos Gradle (capítulos 6 y 7).
- **H4**: cambio a PostgreSQL sin tocar el dominio (capítulo 8).
- **H5**: división en servicios Quests, Adventurers y Treasury (capítulos 9 y 10).
- **H6**: eventos por mensajería y consulta REST con resiliencia (capítulos 11 y 12).
- **H7**: gateway, configuración centralizada y JWT en cada servicio (capítulos 13 y 14).
- **H8**: observabilidad y todo el sistema levantado con Docker Compose (capítulo 15).

Cada capítulo parte del estado en que dejó el proyecto el anterior, y cada hito tiene un ejercicio de tipo *proyecto* que lo avanza.

Verificaciones de estructura

La estructura de carpetas del proyecto cambia a lo largo del libro. Cada vez que cambia, el capítulo incluye una sección titulada **Verificación de estructura**. Hay diez, en los capítulos 3, 6, 7, 8, 9, 10, 11, 13, 14 y 15.

Cada verificación muestra tres cosas: el árbol de carpetas del proyecto tal como queda (con lo nuevo y lo movido marcado), una lista de una línea por cambio y una lista corta de comprobaciones. Te sirve para que tu proyecto coincida con el árbol antes de seguir y no arrastres errores de estructura a los capítulos siguientes.

Esta es la lista de comprobación:

1. ¿Pasa `./gradlew test`? Incluye la prueba de arquitectura.
2. ¿Cada adaptador está en `in` o en `out` según quién lo invoca?
3. ¿Los nombres de las carpetas dicen qué hace el sistema?
4. ¿Las carpetas de tu proyecto coinciden con el árbol del capítulo?

La estructura se comprueba en tres niveles. El primero es el compilador: desde el capítulo 7, los módulos de Gradle impiden que una capa importe lo que no debe. El segundo son las pruebas de arquitectura con ArchUnit, que leen el bytecode y fallan con la clase y la línea de la violación (`ArchitectureTest` desde el capítulo 3, más `StructureTest` cuando el capítulo lo indica). El tercero es esta lista, reducida a lo que ninguna herramienta ve: dónde va cada adaptador y si los nombres cuentan lo que hace el sistema.

El código Java de este libro está revisado, no ejecutado

Presta atención a este aviso. Todo el código Java, las pruebas de arquitectura con ArchUnit y la configuración de Spring Boot 4 que aparecen en el libro **se han revisado a mano, pero no se han compilado ni ejecutado**. En particular, ArchUnit 1.5.1 con JUnit 6.0.3 no se ha ejecutado. Se ha comprobado que los imports estén completos, que los nombres y las firmas coincidan entre capítulos y que las versiones sean las del curso, pero eso no sustituye a

ejecutarlo. Si un ejemplo no compila o una configuración falla, no pienses que has hecho algo mal: comprueba primero la documentación oficial de la versión que uses (las referencias están en cada capítulo) y toma el libro como guía de diseño más que como código garantizado. Lo que sí está garantizado es que el LaTeX compila.

Los ejercicios

Cada capítulo termina con ejercicios. Todos se pueden resolver con lo que has visto hasta ese punto del libro: si un ejercicio necesita una herramienta o una dependencia, el libro te habrá explicado antes cómo instalarla y usarla. Cada ejercicio indica su dificultad:

Dificultad	Qué significa
★☆☆☆☆	Reproducir: aplicar lo que acabas de ver siguiendo los pasos que se indican.
★★☆☆☆	Aplicar con guía: usar un concepto nuevo en un caso parecido, con pistas.
★★★☆☆	Aplicar sin guía: resolver un problema nuevo combinando lo visto en el capítulo.
★★★★☆	Integrar: combinar conceptos de varios capítulos y tomar decisiones de diseño.
★★★★★	Diseñar: problema abierto con varias soluciones válidas; se da una de referencia y criterios para valorar la tuya.

y su tipo:

Tipo	En qué consiste
Guiado	Pasos numerados que te llevan de la mano.
Práctica	Resolver un problema concreto con lo aprendido.
Proyecto	Avanzar el proyecto del curso un hito más.
Diseño	Decidir cómo estructurar algo y justificarlo.
Depuración	Encontrar y corregir el fallo de un código dado.
Lectura de código	Leer código y responder preguntas sobre él.
Test	Escribir o completar pruebas automáticas.

Las soluciones completas están al final de cada capítulo, en la sección *Soluciones de los ejercicios*. Cada enunciado enlaza con su solución y cada solución con su enunciado. Intenta resolver el ejercicio antes de mirarla: la solución es más útil cuando la comparas con la tuya.

Qué necesitas instalar para empezar

Para el capítulo 1 solo necesitas lo que ya tienes: JDK 21 (comprueba `java -version`), Git y Gradle, además de un terminal y tu editor. En ese capítulo crearás el wrapper de Gradle del proyecto. El resto se instala en el capítulo donde se usa por primera vez: Spring Boot en el

capítulo 4, el soporte para pruebas en el 5, PostgreSQL con Docker Compose en el 8, RabbitMQ en el 11, y así sucesivamente. Docker lo usarás desde el capítulo 8.

Referencias

El libro se apoya en libros, artículos y documentación que puedes consultar para profundizar. Aparecen citados entre corchetes, como [1], y la lista completa está al final, en *Referencias*.

Parte I

Bases

CAPÍTULO 1

Repaso I: Java 21 moderno y SOLID

Objetivos del capítulo

- Escribir objetos de valor con *records*, validando en el constructor compacto.
- Modelar un conjunto cerrado de estados con interfaces selladas y recorrerlo con `switch` y *pattern matching*.
- Reconocer los cinco principios SOLID en código real y aplicar la inversión de dependencias (DIP).
- Dejar listo el esqueleto Gradle y el repositorio Git de GuildBoard (hito H0).

Una arquitectura hexagonal no es más que unas pocas ideas de diseño aplicadas con disciplina, y esas ideas se escriben con las herramientas del lenguaje. Los *records* te dan objetos de valor sin ceremonia, las interfaces selladas te dejan decir «estos son todos los casos posibles» y el compilador lo comprueba, y SOLID, sobre todo su última letra, es la razón de ser de los puertos y los adaptadores del capítulo 3. Este capítulo repasa esas piezas con ejemplos del gremio y deja creado el esqueleto vacío del proyecto.

1.1 Cómo usar este repaso

Doy por sabido que escribes clases e interfaces, que usas genéricos y colecciones, que has levantado alguna aplicación con Spring Boot y que sabes qué es un archivo de construcción. No lo doy por sabido todo lo demás: lo que veas aquí se explica desde cero, aunque sea breve. Para Java me apoyo en la lista de novedades de JDK 21 [34] y, para el diseño de objetos inmutables, en *Effective Java* [3].

Los ejemplos y ejercicios de código de este capítulo y del siguiente se ejecutan con `java Archivo.java`: desde Java 11, el comando `java` compila y ejecuta en memoria un único archivo `.java`. Así no hace falta un proyecto para probar una idea, y, sobre todo, **no se fija todavía ninguna estructura de paquetes**: el árbol de carpetas del código aparece en la verificación de estructura 1 del capítulo 3, y no antes.

Un archivo, una clase con `main` la primera

En el modo de archivo único, Java ejecuta la *primera* clase que aparece en el archivo, y esa clase debe tener `main`. Por eso, en todos los ejemplos ejecutables, la clase con `main` va arriba y los tipos que usa (records, interfaces) van debajo, en el mismo archivo. Es una comodidad del modo rápido, no una forma de organizar código real.

1.2 Comprobar el entorno y crear el esqueleto Gradle

El proyecto del curso se llama *GuildBoard* y vive en su propia carpeta, `guildboard/`, con su repositorio Git. En este capítulo solo creas el esqueleto: ni una línea de Java.

☞ JDK 21, Git y el wrapper de Gradle

JDK 21, Git y Gradle ya están instalados según tu entorno; solo se comprueba. La versión de Java debe empezar por 21:

```
$ java -version
openjdk version "21.0.5" 2024-10-15 LTS
$ git --version
git version 2.43.0
```

(El número menor y la fecha pueden ser otros; lo importante es el `21`.) Spring Boot 4.1 funciona con Java 17 a 26 y con Gradle 8.14 o posterior, incluida la serie 9 [54]. El curso usa **Gradle 9.8.1, o cualquier otra 9.x** que tengas a mano; ninguna decisión del libro depende del número de versión exacto.
El *wrapper* es un pequeño script (`gradlew`) que descarga y usa la versión de Gradle que el proyecto declara, de modo que quien clone el repositorio no necesita instalar Gradle ni coincidir contigo. Lo generas una sola vez con tu Gradle del sistema, dentro de la carpeta del proyecto:

```
$ gradle wrapper --gradle-version 9.8.1
```

A partir de ahí usarás siempre `./gradlew` y no el `gradle` del sistema. Si tu Gradle del sistema es tan antiguo que no arranca con JDK 21, actualízalo antes (con tu gestor de JDK) o, si no, descarga una versión reciente solo para ejecutar este comando.

El wrapper deja cuatro archivos en el proyecto, y *todas se versionan con Git*:

- `gradlew` y `gradlew.bat`: los scripts para Linux/macOS y Windows.
- `gradle/wrapper/gradle-wrapper.properties`: la versión de Gradle que se descarga.
- `gradle/wrapper/gradle-wrapper.jar`: el pequeño programa que hace la descarga.

Gradle necesita al menos dos archivos más. El primero, `settings.gradle`, nombra el proyecto y marca la raíz del build:

`settings.gradle`

```
rootProject.name = 'guildboard'
```

El segundo, `build.gradle`, dice qué se construye. Está escrito en el DSL Groovy de Gradle [13]: cada bloque (`plugins`, `java`, `repositories`) es una llamada a un método que recibe un bloque de configuración. Por ahora basta con un plugin de Java y la versión del lenguaje:

`build.gradle`

```
plugins {
    id 'java'
}

group = 'com.guildboard'
version = '0.0.1-SNAPSHOT'

java {
```

```
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}
```

El `toolchain` exige que el código se compile con JDK 21, venga de donde venga el Gradle que lo lanza. `mavenCentral()` es el repositorio del que se descargarán las dependencias cuando las haya; hoy no hay ninguna. Este archivo irá creciendo: el capítulo 3 le añade Lombok y una tarea propia, y el capítulo 4 el plugin de Spring Boot.

Falta decirle a Git qué no debe guardar. El `.gitignore` deja fuera lo que Gradle y los editores generan, pero *no* el `gradle-wrapper.jar`:

`.gitignore`

```
# Gradle
.gradle/
build/

# Editores e IDE (NeoVim con jdtls, IntelliJ)
.idea/
*.iml
.classpath
.project
.settings/
/bin/
/out/

# Java y sistema
*.class
*.log
.DS_Store
```

JAVA_HOME apunta a otro JDK

El fallo más habitual del entorno: tienes JDK 21 instalado, pero la variable `JAVA_HOME` (o el `java` del `PATH`) apunta a otra versión. Gradle usa ese JDK para arrancar, y `./gradlew -v` muestra una línea `JVM` que no empieza por 21. Corrige la variable antes de seguir; el ejercicio 1.1 te enseña a comprobarlo.

1.3 Records y valores inmutables

Un *record* es una clase pensada para transportar datos inmutables. Declaras sus *componentes* entre paréntesis y Java genera el resto: campos `final`, constructor, accesores, `equals`, `hashCode` y `toString`. Dos records con los mismos componentes son iguales: igualdad por valor, no por identidad.

Un record mínimo

```
public record Coordinates(int x, int y) { }

// Coordinates a = new Coordinates(2, 3);
// a.x()                  -> 2          (el accesor se llama como el componente
    ↪ )
// a.equals(new Coordinates(2, 3))  -> true
// a.toString()           -> Coordinates[x=2, y=3]
```

Para qué sirven en este curso: los *objetos de valor* del dominio (una recompensa, un rango, un identificador) y los comandos y DTO de los límites son datos que no cambian y se comparan por contenido. Un record los escribe en una línea, y la inmutabilidad es justo lo que recomienda la literatura para este tipo de clases [3].

1.3.1 Validar en el constructor compacto

Un objeto de valor no debería poder existir en un estado inválido. El *constructor compacto* es el sitio para comprobarlo: no repite la lista de parámetros y se ejecuta antes de asignar los campos. Ahí también se normalizan valores (reasignando el parámetro).

QuestTitle.java

```
public record QuestTitle(String value) {

    public QuestTitle {
        if (value == null || value.isBlank()) {
            throw new IllegalArgumentException("title must not be blank");
        }
        value = value.strip();          // normaliza antes de asignar el campo
    }

    public int length() {
        return value.length();
    }
}
```

`QuestTitle` es solo ilustrativo: en el proyecto el título seguirá siendo un `String title`, tal como lo usan los capítulos siguientes.

Un record puede tener métodos, métodos estáticos e implementar interfaces. No puede heredar de otra clase ni declarar campos de instancia extra: todo su estado son sus componentes.

1.3.2 Inmutabilidad superficial

El error típico con los records es creer que son inmutables «hasta el fondo». Los campos son `final`, pero si un componente es mutable, su contenido puede cambiar desde fuera:

Party.java

```
import java.util.List;

public record Party(String name, List<String> members) {

```

```
    public Party {
        members = List.copyOf(members);   // copia defensiva e inmutable
    }
}
```

Sin la línea del `List.copyOf`, quien creó el record con una `ArrayList` podría seguir añadiendo miembros a la lista «inmutable». Con ella, el record guarda su propia copia, que además no admite modificaciones. Regla práctica: si un componente es una colección o un objeto mutable, cópialo en el constructor compacto.

Como no hay *setters*, «modificar» un record es crear otro. Cuando hace falta, se escribe a mano un método que devuelva la copia cambiada:

```
public record Quest(String title, int gold) {
    public Quest withGold(int newGold) {
        return new Quest(title, newGold);
    }
}
```

(Este `Quest` es solo ilustrativo; no es el del proyecto, que llega en el capítulo 3.)

1.4 Sealed y pattern matching para modelar estados

Muchas veces un concepto tiene un número *cerrado* de variantes, y cada variante lleva sus propios datos: una misión publicada no tiene aventurero; una aceptada, sí; una completada, además, tiene un pago. Una *interfaz sellada* (`sealed`) declara con `permits` cuáles son las únicas implementaciones permitidas. Combinada con records, modela esto con precisión:

StateDemo.java

```
public class StateDemo {

    public static void main(String[] args) {
        QuestState[] states = {
            new Published(),
            new Accepted("Lyra"),
            new Completed("Lyra", 120)
        };
        for (QuestState state : states) {
            System.out.println(describe(state));
        }
    }

    static String describe(QuestState state) {
        return switch (state) {
            case Published p -> "Disponible en el tabl^^c3^^b3n";
            case Accepted(String adventurer) ->
                "En curso, a cargo de " + adventurer;
            case Completed(String adventurer, int gold) ->
                adventurer + " cobr^^c3^^b3 " + gold + " de oro";
        };
    }
}
```

```
24
25 sealed interface QuestState permits Published, Accepted, Completed { }
26
27 record Published() implements QuestState { }
28
29 record Accepted(String adventurer) implements QuestState { }
30
31 record Completed(String adventurer, int gold) implements QuestState { }
```

Ejecútalo con `java StateDemo.java`. Salida:

```
Disponible en el tabl^^c3^^b3n
En curso, a cargo de Lyra
Lyra cobr^^c3^^b3 120 de oro
```

Tres ideas hay aquí.

1. **Exhaustividad.** El `switch` sobre un tipo sellado no necesita `default`: el compilador sabe que solo hay tres casos y, si falta uno, no compila. Si más adelante añades una cuarta variante, *todos* los `switch` que no la cubran dejan de compilar y te señalan dónde revisar.
2. ***Record patterns.*** `case Accepted(String adventurer)` comprueba el tipo y, a la vez, extrae los componentes del record en variables. Es lo que antes requería un `instanceof`, una conversión y varias llamadas a accesores.
3. ***Guards.*** Un caso puede llevar una condición con `when`: `case Completed(String a, int g) when g == 0 -> "gratis"`. Los casos se evalúan en orden, así que el caso con condición debe ir *antes* que el caso general del mismo tipo; si no, el compilador protesta porque el segundo quedaría «dominado» por el primero.

Un `default` apaga la red de seguridad

Si añades `default -> ...` a un `switch` sobre un tipo sellado, el compilador deja de avisarte cuando aparezca una variante nueva: caerá en el `default` sin que lo notes. Para los tipos sellados, mejor no escribir `default`. En Java 21, además, el comodín `_` para patrones sin nombre solo existe como vista previa; usa un nombre normal, aunque no lo uses.

¿Por qué sellado y no un `enum`? Un `enum` sirve cuando las variantes no llevan datos distintos. Cuando cada variante lleva los suyos, como arriba, la interfaz sellada con records lo expresa sin campos opcionales ni `null`. Volverás a usar este patrón para los errores de dominio del capítulo 5.

1.5 SOLID: los cinco principios

SOLID agrupa cinco principios de diseño orientado a objetos que Robert C. Martin dedica los capítulos 7 a 11 de *Clean Architecture* [28] («SRP», «OCP», «LSP», «ISP» y «DIP», en ese orden). No son leyes: son heurísticas para que el código se pueda cambiar sin romper lo que ya funciona. Esta es la versión corta, con la pregunta que conviene hacerse y el síntoma de que se incumple.

Principio	Pregunta	Síntoma de que se incumple
SRP, responsabilidad única	¿Cuántos motivos distintos hay para cambiar esta clase?	Una clase que valida, calcula, guarda en disco y escribe en consola.
OCP, abierto/cerrado	¿Añadir un caso nuevo obliga a editar código que ya funciona?	Una cadena de `if/else` sobre un tipo que crece con cada novedad.
LSP, sustitución de Liskov	¿Puedo usar una subclase donde se espera la superclase sin sorpresas?	Una subclase que lanza `UnsupportedOperationException` en un método heredado.
ISP, segregación de interfaces	¿Obligo a los clientes a depender de métodos que no usan?	Una interfaz enorme que casi todos implementan a medias.
DIP, inversión de dependencias	¿Depende la lógica importante de detalles concretos?	La lógica hace `new` de una clase de acceso a ficheros o a base de datos.

1.5.1 SRP, OCP, LSP e ISP con ejemplos del gremio

SRP. Una clase que a la vez comprueba que el título no está vacío, calcula la recompensa, la guarda en un fichero y avisa por consola cambia si cambia la regla de validación, el formato del fichero o el canal de aviso. Tres motivos de cambio, tres responsabilidades. La salida es separar: cada pieza tiene un motivo. El ejercicio 1.4 lo practica.

OCP. Si la recompensa se calcula con una cadena de `if/else` que compara el tipo con cadenas de texto, cada tipo de misión nuevo obliga a editar ese método. Si el dato vive en el propio tipo (por ejemplo, un `enum` cuyas constantes llevan su recompensa) o se inyecta una política, un tipo nuevo es *código nuevo*, no código modificado.

LSP. Imagina `class ExpiredQuest extends Quest` cuyo `accept()` lanza una excepción porque «una misión caducada no se acepta». Cualquier código que reciba un `Quest` y llame a `accept()` se rompe al recibir esa subclase: no es sustituible. La salida habitual es no usar herencia para representar estados, sino un componente de estado (como el tipo sellado de la sección anterior).

ISP. Una interfaz `QuestOperations` con `publish`, `accept`, `complete` y `delete` obliga a un cliente que solo consulta a conocer, y a las pruebas a simular, operaciones que no usa. Mejor interfaces pequeñas, una por motivo de uso. En este curso será natural: cada caso de uso tendrá su propia interfaz.

1.5.2 DIP en detalle: la base de los puertos

El principio de inversión de dependencias dice dos cosas: los módulos de alto nivel no deben depender de los de bajo nivel (ambos deben depender de abstracciones), y las abstracciones no deben depender de los detalles. En lenguaje de proyecto: la lógica importante, la que explica qué hace el sistema, no debe conocer cómo se guarda un dato ni cómo se envía un aviso.

Este código viola el principio: la lógica de publicar una misión crea directamente el detalle de almacenamiento.

Antes: la política depende del detalle

```
public class PublishQuest {

    // Detalle concreto, creado dentro de la l^^c3^^b3gica
    private final FileQuestStore store = new FileQuestStore("quests.txt");

    public void publish(String title) {
```

```
        store.append(title);
    }
}
```

Para probar `PublishQuest` necesitas un fichero real, y para cambiar a una base de datos hay que editar la clase. La inversión consiste en que la lógica *declare lo que necesita* como una interfaz y reciba una implementación desde fuera:

Después: el detalle depende de la abstracción

```
public interface QuestStore {
    void save(String title);
}

public class PublishQuest {

    private final QuestStore store;      // solo conoce la interfaz

    public PublishQuest(QuestStore store) {
        this.store = store;
    }

    public void publish(String title) {
        store.save(title);
    }
}

public class InMemoryQuestStore implements QuestStore {

    private final java.util.List<String> titles = new java.util.ArrayList<>();

    @Override
    public void save(String title) {
        titles.add(title);
    }
}
```

(Si pones estas clases en un mismo archivo para ejecutarlas, recuerda: sin `public` salvo la primera, y la que tenga `main`, arriba.)

La flecha de dependencia cambia de sentido. Antes, la lógica apuntaba al detalle. Ahora ambos apuntan a la interfaz, que pertenece al lado de la lógica:



Quien decide qué implementación se entrega es otra pieza, la que construye los objetos y los une: el capítulo 4 la llama raíz de composición. El capítulo 3 les pondrá nombre: la interfaz que la lógica posee es un *puerto* y la implementación que la satisface, un *adaptador*. Si entiendes bien esta sección, la arquitectura hexagonal es esta misma idea aplicada a todo el sistema.

Nota

La elección del nombre y del lugar de la interfaz importa: pertenece al lado de quien la *necesita* (aquí, la lógica de publicar), no al de quien la implementa. Una interfaz `FileQuestStoreInterface` ubicada junto al fichero sería una abstracción de nombre, pero seguiría arrastrando el detalle hacia la lógica.

1.6 En el proyecto: H0, el esqueleto de GuildBoard

Hito H0: esqueleto Gradle y repositorio

El primer hito no contiene código: es el punto de partida, con el repositorio Git y el build que más adelante construirá la aplicación. Al terminar el ejercicio 1.5 la carpeta `guildboard/` tiene este aspecto:

```
guildboard/
|-- .git/
|-- .gitignore
|-- settings.gradle
|-- build.gradle
|-- gradlew
|-- gradlew.bat
`-- gradle/
    `-- wrapper/
        |-- gradle-wrapper.jar
        `-- gradle-wrapper.properties
```

Este árbol **no fija una estructura de paquetes**: no hay `src/`. La disposición del código (qué carpetas, qué va dentro de cada una, qué puede depender de qué) aparece en la verificación de estructura 1 del capítulo 3. Hasta entonces, todo el código de los capítulos 1 y 2 se ejecuta con `java Archivo.java`, fuera del proyecto.

Desde este capítulo, cada capítulo con proyecto termina con un *commit*. Antes de confirmar, `git status` te dice qué ha cambiado y `git diff -stat` te lo resume por archivo, con el número de líneas añadidas y quitadas. Es una buena costumbre: si el resumen toca archivos que no esperabas, lo ves antes de guardarlo.

1.7 Ejercicios

Ejercicio 1.1: Comprueba el entorno y crea un wrapper de prueba ★☆☆☆☆

Comprueba que tu entorno cumple lo que pide el curso y genera un wrapper de Gradle en una carpeta de prueba (el del proyecto real lo harás en el ejercicio 1.5).

1. Ejecuta `java -version` y comprueba que la versión empieza por 21.
2. Ejecuta `git -version` y `gradle -version`.
3. Crea la carpeta `prueba-gradle`, entra en ella y crea un `settings.gradle` vacío (evita que Gradle busque un build en las carpetas superiores).
4. Genera el wrapper con `gradle wrapper -gradle-version 9.8.1` (o la 9.x que pre-

fieras).

5. Ejecuta `./gradlew -v`.

Está bien si `./gradlew -v` muestra una línea `Gradle 9....` y una línea `JVM` que empieza por 21.

Guiado ▪ solución en la pág. 14

Ejercicio 1.2: Reward como record ★☆☆☆☆

Escribe el record `Reward` con un único componente `int gold`. Su constructor compacto debe lanzar `IllegalArgumentException` con el mensaje `"gold must not be negative: "` seguido del valor recibido si `gold` es negativo. Guarda el record dentro del archivo `RewardDemo.java`, debajo de esta clase, que ya tienes escrita:

RewardDemo.java

```
public class RewardDemo {

    public static void main(String[] args) {
        Reward a = new Reward(50);
        Reward b = new Reward(50);
        System.out.println(a);
        System.out.println(a.equals(b));
        System.out.println(a.gold());
        try {
            new Reward(-1);
        } catch (IllegalArgumentException e) {
            System.out.println("rechazado: " + e.getMessage());
        }
    }
}
```

Está bien si `java RewardDemo.java` imprime, en este orden: `Reward[gold=50]`, `true`, `50` y `rechazado: gold must not be negative: -1`.

Práctica ▪ solución en la pág. 15

Ejercicio 1.3: Predice el switch ★★☆☆☆

Lee este programa, que no hay que escribir, solo analizar:

SwitchReading.java

```java
public class SwitchReading {

    public static void main(String[] args) {
        System.out.println(label(new Offered()));
        System.out.println(label(new Taken("Mira", 2)));
        System.out.println(label(new Taken("Mira", 5)));
        System.out.println(label(new Settled(0)));
    }

    static String label(Contract contract) {
        return switch (contract) {
            case Offered o -> "libre";
            case Taken(String who, int days) when days > 3 ->
                who + " va tarde";
            case Taken(String who, int days) ->
                who + " trabaja " + days + " d^^c3^^adas";
            case Settled(int score) -> "cerrado con " + score;
        };
    }
}

sealed interface Contract permits Offered, Taken, Settled { }

record Offered() implements Contract { }

record Taken(String adventurer, int daysLeft) implements Contract { }

record Settled(int score) implements Contract { }
```

Responde:

1. ¿Qué imprime el programa, línea a línea?
2. ¿Qué pasa si intercambias el orden de los dos casos `Taken`?
3. Añades `Cancelled(String reason)` como nueva variante (en `permits` y como record) sin tocar el `switch`. ¿Qué dice el compilador?
4. Si en ese caso añades un `default -> "otro"` al `switch`, ¿qué ocurre con el aviso del apartado anterior y por qué es peor?

Lectura de código ▪ solución en la pág. 16

Ejercicio 1.4: Diagnostica y refactoriza QuestManager ★★★☆☆

Esta clase publica una misión. Funciona, pero incumple varios principios SOLID:

QuestManager.java

```java
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class QuestManager {

    private final List<String> titles = new ArrayList<>();

    public void publish(String title, String type) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("title must not be blank");
        }
        int gold;
        if (type.equals("HUNT")) {
            gold = 100;
        } else if (type.equals("ESCORT")) {
            gold = 60;
        } else {
            gold = 20;
        }
        titles.add(title);
        try (FileWriter out = new FileWriter("quests.txt", true)) {
            out.write(title + ";" + gold + "\n");
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        System.out.println("Nueva misi^^c3^^b3n: " + title + " (" + gold +
    ↪ " de oro)");
    }
}
```

1. Indica qué principios incumple (SRP, OCP, DIP), señalando el fragmento responsable, y cuáles no están en juego (LSP, ISP) y por qué.
2. Refactoriza aplicando DIP: la clase que publica solo debe depender de interfaces que ella declara y recibir sus colaboradores por el constructor. Mueve también el cálculo de la recompensa fuera del `if/else`.
3. Escribe un `main` que la use con implementaciones en memoria y por consola.

Está bien si la clase refactorizada no hace `new` de ninguna clase concreta de almacenamiento ni de aviso, la clase `QuestManager` no importa nada de `java.io`, y tu `main` publica dos misiones y muestra por consola los avisos y lo guardado.

Pista: *te hacen falta dos interfaces pequeñas (una para guardar y otra para avisar), un tipo para la clase de misión que lleve su recompensa y un registro de lo publicado.*

Práctica ▪ solución en la pág. 16

Ejercicio 1.5: Hito H0: el esqueleto guildboard ★★☆☆☆

Crea el esqueleto del proyecto y haz el primer commit.

1. Crea la carpeta `guildboard` y entra en ella.

2. Escribe `settings.gradle` y `build.gradle` como en el capítulo (proyecto `guildboard`, plugin `java`, `toolchain` con Java 21).
3. Genera el wrapper de Gradle (9.8.1 o la 9.x que uses).
4. Escribe el `.gitignore` del capítulo.
5. Inicializa Git con la rama `main`, añade todo y haz el primer commit.

Está bien si `./gradlew build` termina con `BUILD SUCCESSFUL` sin que exista ningún código Java, si `git status` no muestra nada pendiente tras el commit, y si `git ls-files` incluye `gradle/wrapper/gradle-wrapper.jar` pero no `build/` ni `.gradle/`.

Proyecto ▪ solución en la pág. 20

1.8 Resumen

Resumen

- Un record es una clase de datos inmutable con igualdad por valor; el constructor compacto valida y normaliza, y los componentes mutables se copian.
- Una interfaz sellada con records modela un conjunto cerrado de variantes con datos propios; un `switch` sobre ella es exhaustivo y no necesita `default`.
- Los *record patterns* extraen componentes y `when` añade condiciones; el caso con condición va antes que el general.
- SOLID: un motivo de cambio por clase (SRP), extender sin editar (OCP), subclases sustituibles (LSP), interfaces pequeñas (ISP) y lógica que depende de abstracciones (DIP).
- DIP invierte la flecha: la interfaz pertenece al lado de la lógica y el detalle la implementa; es la base de los puertos y adaptadores.
- GuildBoard (H0) es un esqueleto Gradle con wrapper, `.gitignore` y repositorio Git, sin código ni estructura de paquetes.

Soluciones de los ejercicios

Solución del ejercicio 1.1 enunciado en la pág. 10

```
$ java -version
openjdk version "21.0.5" 2024-10-15 LTS
$ git --version
git version 2.43.0
$ gradle --version
$ mkdir prueba-gradle
$ cd prueba-gradle
$ touch settings.gradle
$ gradle wrapper --gradle-version 9.8.1
$ ./gradlew -v
```

En la salida de `./gradlew -v`, entre otras líneas, deben aparecer:

```
Gradle 9.8.1
JVM:          21.0.5 (Eclipse Adoptium 21.0.5+11-LTS)
```

(La versión de Gradle será la 9.x que hayas pedido; el detalle de la línea `JVM` cambia con tu distribución.) El primer `./gradlew` descarga la distribución de Gradle, así que tarda unos segundos y necesita red.
Error típico: la línea `JVM` muestra, por ejemplo, 17: tu `JAVA_HOME` apunta a otro JDK. Comprueba `echo $JAVA_HOME` y `which java`; apunta la variable al JDK 21 (por ejemplo, `export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64`, con la ruta de tu instalación) y repite `./gradlew -v`. Otro tropiezo: ejecutar `gradle wrapper` sin `settings.gradle` en una carpeta dentro de otro proyecto Gradle, que acaba generando el wrapper en el proyecto equivocado.

Solución del ejercicio 1.2

enunciado en la pág. 11

RewardDemo.java

```java
public class RewardDemo {

    public static void main(String[] args) {
        Reward a = new Reward(50);
        Reward b = new Reward(50);
        System.out.println(a);
        System.out.println(a.equals(b));
        System.out.println(a.gold());
        try {
            new Reward(-1);
        } catch (IllegalArgumentException e) {
            System.out.println("rechazado: " + e.getMessage());
        }
    }
}

record Reward(int gold) {

    Reward {
        if (gold < 0) {
            throw new IllegalArgumentException(
                "gold must not be negative: " + gold);
        }
    }
}
```

La salida es:

```
Reward[gold=50]
true
50
rechazado: gold must not be negative: -1
```

El constructor compacto (sin lista de parámetros) se ejecuta antes de asignar el campo, así que un `Reward` negativo no llega a existir. `equals` y `toString` los genera el record: de ahí

el `true` y el formato `Reward[gold=50]`.
Errores típicos: escribir `this.gold = gold;` dentro del constructor compacto (es un error: la asignación es automática al final) y olvidar que el record va *debajo* de la clase con `main`, no encima.

Solución del ejercicio 1.3

enunciado en la pág. 11

1. La salida es:

   ```
   libre
   Mira trabaja 2 d^^c3^^adas
   Mira va tarde
   cerrado con 0
   ```

   Con `Taken("Mira", 2)` la condición `days > 3` es falsa y se aplica el segundo caso `Taken`; con 5 es verdadera y se aplica el primero.

2. No compila: el caso general `Taken(String who, int days)` aparece antes que el que tiene condición y lo *domina* (siempre se cumpliría primero, así que el segundo nunca se alcanzaría). El compilador lo rechaza con un error del tipo «this case label is dominated by a preceding case label».

3. No compila: el `switch` deja de ser exhaustivo, porque `Cancelled` no tiene ningún caso. El error dice que la expresión `switch` no cubre todos los valores posibles de entrada. Es útil: el compilador te indica cada `switch` que hay que revisar.

4. Con el `default`, el aviso desaparece y el programa compila, pero cualquier `Cancelled` cae en `"otro"` sin que nadie decida qué debe hacerse con él. Un error que el compilador habría detectado pasa a ser un comportamiento silencioso en ejecución.

Error típico: pensar que el orden de los casos no importa. En un `switch` con patrones sí importa: el primero que encaja gana, y el compilador solo avisa cuando uno queda inalcanzable.

Solución del ejercicio 1.4

enunciado en la pág. 12

Diagnóstico.

- **SRP:** la clase valida el título, calcula la recompensa, guarda en memoria y en un fichero, y escribe por consola. Hay al menos tres motivos de cambio distintos: la regla de negocio, el formato de almacenamiento y el canal de aviso.
- **OCP:** la cadena `if/else` sobre `type`. Un tipo de misión nuevo obliga a editar `publish`.
- **DIP:** el método crea un `FileWriter` y usa `System.out`: la lógica de publicar depende de detalles de bajo nivel y no se puede probar sin tocar el disco.
- **LSP e ISP no están en juego:** no hay herencia ni interfaces (LSP habla de sustituir subtipos; ISP, de interfaces demasiado grandes).

Refactorización. Todo en un solo archivo, con `main` arriba; el listado va en tres trozos que se pegan en orden:

QuestRefactor.java (1/3)

```java
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class QuestRefactor {

    public static void main(String[] args) {
        InMemoryQuestStore store = new InMemoryQuestStore();
        QuestManager manager = new QuestManager(store, new ConsoleNotifier()
    ↪ );

        manager.publish("Cazar al lobo", QuestType.HUNT);
        manager.publish("Escoltar la caravana", QuestType.ESCORT);

        System.out.println(store.all());
    }
}

enum QuestType {
    HUNT(100),
    ESCORT(60),
    DELIVERY(20);

    private final int gold;

    QuestType(int gold) {
        this.gold = gold;
    }

    int gold() {
        return gold;
    }
}

record PublishedQuest(String title, int gold) { }

interface QuestStore {
    void save(PublishedQuest quest);
}

interface Notifier {
    void published(PublishedQuest quest);
}
```

Continuación del listado anterior:

QuestRefactor.java (2/3)

```java
class QuestManager {

    private final QuestStore store;
    private final Notifier notifier;

    QuestManager(QuestStore store, Notifier notifier) {
        this.store = store;
        this.notifier = notifier;
    }

    void publish(String title, QuestType type) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("title must not be blank");
        }
        PublishedQuest quest = new PublishedQuest(title, type.gold());
        store.save(quest);
        notifier.published(quest);
    }
}
```

Continuación del listado anterior:

QuestRefactor.java (3/3)

```java
class InMemoryQuestStore implements QuestStore {

    private final List<PublishedQuest> saved = new ArrayList<>();

    @Override
    public void save(PublishedQuest quest) {
        saved.add(quest);
    }

    List<PublishedQuest> all() {
        return List.copyOf(saved);
    }
}

class FileQuestStore implements QuestStore {

    private final String path;

    FileQuestStore(String path) {
        this.path = path;
    }

    @Override
    public void save(PublishedQuest quest) {
        try (FileWriter out = new FileWriter(path, true)) {
            out.write(quest.title() + ";" + quest.gold() + "\n");
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
}

class ConsoleNotifier implements Notifier {

    @Override
    public void published(PublishedQuest quest) {
        System.out.println("Nueva misi^^c3^^b3n: " + quest.title()
            + " (" + quest.gold() + " de oro)");
    }
}
```

Los `import java.io` están en el mismo archivo que `QuestManager`, porque el ejercicio se resuelve en un único archivo; lo que pedía el criterio es que la *clase* `QuestManager` no use nada de `java.io`, y así es: solo `FileQuestStore` lo usa.
La salida de `java QuestRefactor.java` es:

```
Nueva misi^^c3^^b3n: Cazar al lobo (100 de oro)
Nueva misi^^c3^^b3n: Escoltar la caravana (60 de oro)
[PublishedQuest[title=Cazar al lobo, gold=100], PublishedQuest[title=Escoltar
    ↪ la caravana, gold=60]]
```

Por qué funciona. `QuestManager` solo conoce `QuestStore` y `Notifier`, interfaces que él

necesita, y las recibe por el constructor (DIP). Guardar en memoria, en fichero o por consola son detalles que se enchufan desde `main`: cambiar `new InMemoryQuestStore()` por `new FileQuestStore("quests.txt")` no toca `QuestManager` y es el mismo caso de uso. La clase conserva un único motivo de cambio, la orquestación de «publicar» (SRP). La recompensa vive en el `enum QuestType`: un tipo nuevo es una constante nueva, no una edición de `publish` (OCP).
Errores típicos: dejar un `new FileQuestStore(...)` dentro de `QuestManager` (se sigue incumpliendo DIP aunque exista la interfaz) y poner las interfaces junto a las implementaciones en lugar de pensarlas desde lo que necesita el gestor. También es habitual crear una única interfaz grande con «guardar y avisar»: incumple ISP; son dos necesidades distintas.

Solución del ejercicio 1.5

enunciado en la pág. 13

```
$ mkdir guildboard
$ cd guildboard
```

Crea los archivos `settings.gradle`, `build.gradle` y `.gitignore` exactamente como en el capítulo:

`settings.gradle`

```
rootProject.name = 'guildboard'
```

`build.gradle`

```
plugins {
    id 'java'
}

group = 'com.guildboard'
version = '0.0.1-SNAPSHOT'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}
```

.gitignore

```
# Gradle
.gradle/
build/

# Editores e IDE (NeoVim con jdtls, IntelliJ)
.idea/
*.iml
.classpath
.project
.settings/
/bin/
/out/

# Java y sistema
*.class
*.log
.DS_Store
```

Después, el wrapper, la comprobación y el commit:

```
$ gradle wrapper --gradle-version 9.8.1
$ ./gradlew build
BUILD SUCCESSFUL in 2s
$ git init -b main
$ git add .
$ git status
$ git commit -m "Esqueleto Gradle de GuildBoard (H0)"
$ git status
nothing to commit, working tree clean
$ git ls-files
.gitignore
build.gradle
gradle/wrapper/gradle-wrapper.jar
gradle/wrapper/gradle-wrapper.properties
gradlew
gradlew.bat
settings.gradle
```

(El tiempo de `BUILD SUCCESSFUL` varía, y los mensajes intermedios del build cambian con la versión de Gradle.) `./gradlew build` pasa en vacío porque no hay código que compilar ni pruebas que ejecutar: comprueba que el toolchain encuentra JDK 21 y que el wrapper funciona. `build/` y `.gradle/`, creados por esa ejecución, están en el `.gitignore` y por eso `git ls-files` no los lista.

Este es el estado del que parte el capítulo siguiente: un esqueleto sin `src/`, a la espera de la estructura del capítulo 3 (verificación de estructura 1).

Errores típicos: ignorar el `gradle-wrapper.jar` con una regla `*.jar` en el `.gitignore` (quien clone el repositorio no podrá usar `./gradlew`); olvidar el `settings.gradle`, con lo que el proyecto se llama como la carpeta y Gradle puede buscar un build en carpetas superiores; y ejecutar `git init` antes de tener el `.gitignore`, con lo que acabas añadiendo `build/` al primer commit.

CAPÍTULO 2

Repaso II: DDD básico

Objetivos del capítulo

- Usar el lenguaje ubicuo: un glosario compartido que decide los nombres del código.
- Distinguir entidad y objeto de valor, y reconocer un agregado, su raíz y su repositorio.
- Explicar por qué un dominio anémico deja las reglas sin dueño y escribir uno rico.
- Redactar `docs/dominio.md`, el documento de dominio de GuildBoard (hito H0).

Antes de dibujar puertos y adaptadores hay que saber qué va dentro del hexágono. Un hexágono lleno de clases con datos y de servicios con toda la lógica protege muy poco: las reglas del gremio estarían repartidas por cualquier sitio. El *diseño guiado por el dominio* (DDD, por *Domain-Driven Design*) aporta el vocabulario para modelar ese interior: un lenguaje común con el negocio, tipos con identidad y tipos sin ella, y unidades de consistencia llamadas agregados. Este capítulo repasa lo mínimo y lo aplica a GuildBoard. No escribe todavía el dominio en el proyecto: deja por escrito cómo es, y eso decide mucho de lo que vendrá en los capítulos 3 a 5 y 9.

2.1 Por qué DDD antes de la arquitectura

La arquitectura hexagonal dice *dónde* vive el dominio y *qué* puede conocer. DDD dice *cómo* se modela. Sin lo segundo, lo primero se queda en carpetas bien puestas con clases anémicas dentro. La obra de referencia es el libro de Evans [7]; para este repaso te basta su resumen de patrones, escrito por el propio autor y de acceso libre [8], y una presentación moderna con la mirada puesta en la arquitectura, la de Khononov [25].

DDD tiene dos mitades. La *estratégica* se ocupa de dividir un sistema grande en partes con sentido (contextos, equipos, relaciones entre ellos); la *táctica* se ocupa de cómo se escribe el modelo dentro de una parte (entidades, objetos de valor, agregados). Este capítulo es sobre todo táctico y solo asoma a la mitad estratégica en la última sección. Todo lo que ves aquí son ideas, no una biblioteca: no añade ninguna dependencia al proyecto.

Los ejemplos de código de este capítulo se ejecutan con `java Archivo.java`, como en el capítulo anterior, y por la misma razón la clase con `main` va siempre la primera del archivo.

2.2 Lenguaje ubicuo y glosario de GuildBoard

El *lenguaje ubicuo* es el vocabulario que comparten las personas del negocio y las que programan, y que aparece *tal cual* en el código. Si el maestro del gremio habla de «aceptar una misión», el método se llama `accept`, no `assign` ni `setOwner`. Si «rango» y «nivel» significan lo mismo, uno de los dos sobra. Un glosario escrito evita la deriva: cada término tiene una definición y un nombre de código.

La convención de este libro: el texto va en español y los identificadores, en inglés. El glosario

guarda la correspondencia, y es lo primero que escribirás en `docs/dominio.md`. Este es el vocabulario de GuildBoard:

Término	Identificador	Definición
Misión	`Quest`	Encargo del gremio con título, recompensa y rango mínimo.
Aventurero	`Adventurer`	Miembro del gremio que acepta misiones.
Rango	`Rank`	Nivel del aventurero: `NOVICE`, `ADEPT` o `VETERAN`, de menor a mayor.
Recompensa	`Reward`	Oro que cobra quien completa una misión.
Experiencia	`experience`	Puntos acumulados por un aventurero; con ellos sube de rango.
Maestro del gremio	`GUILD_MASTER`	Quien publica las misiones.
Publicar	`publish`	Poner una misión en el tablón.
Aceptar	`accept`	Un aventurero se compromete con una misión publicada.
Completar	`complete`	Se da por cumplida una misión aceptada.
Misión activa	—	Misión `ACCEPTED` (aceptada y no completada) de un aventurero.

Y una regla que cierra el vocabulario: **la experiencia que da una misión es igual a su recompensa en oro** (1 de oro, 1 punto de experiencia). Las reglas de negocio forman parte del glosario: «la misión te da experiencia» es ambiguo hasta que dice cuánta.

Qué estados tiene una misión

Una misión pasa por `PUBLISHED`, `ACCEPTED` y `COMPLETED`. Antes de publicarla existe como borrador, un cuarto estado, `DRAFT`, que no está en el tablón: es lo que hace posible que `publish` sea una transición con reglas (solo se publica un borrador) y no un simple constructor.

2.3 Entidades y objetos de valor

2.3.1 Objeto de valor

Un *objeto de valor* se define por sus datos, no por quién es: dos recompensas de 50 de oro son la misma recompensa y se pueden intercambiar. No tiene identidad, es inmutable y se compara por valor. En Java son *records* (capítulo 1): `Reward` y también los identificadores, que son valores aunque identifiquen a una entidad.

`Identificadores y rango`

```java
public record QuestId(String value) { }

public record AdventurerId(String value) { }

public enum Rank {
    NOVICE, ADEPT, VETERAN;     // de menor a mayor: el orden importa

    public boolean canTake(Rank required) {
        return this.compareTo(required) >= 0;
    }
}
```

`Rank` es un `enum` porque son tres valores fijos y sin datos propios. Su método `canTake` pone la regla «un rango alcanza al exigido» junto al tipo, en lugar de repetir una comparación de números en cada sitio: un `NOVICE` no puede con una misión de `ADEPT`, y no se pueden crear rangos nuevos desde fuera. Es un objeto de valor con comportamiento, y es lo habitual. Que los valores sean inmutables y se comparen por contenido es, además, la recomendación general para clases pequeñas de datos [3]. (Un título también podría ser un `QuestTitle`, como el record del capítulo anterior, pero en GuildBoard `Quest` usa un simple `String title`; `QuestTitle` fue solo un ejemplo y no lo buscarás en el proyecto.)

2.3.2 Entidad

Una *entidad* tiene identidad: la misión `q-1` sigue siendo la misma misión aunque cambie su estado o su aventurero. Dos misiones con el mismo título y la misma recompensa *no* son la misma misión, y la misma misión en dos momentos distintos sí lo es. Por eso:

- su igualdad (`equals`, `hashCode`) se basa *solo* en el identificador;
- su estado cambia a lo largo de su vida, mediante métodos con nombre del negocio, no mediante *setters*;
- el identificador no cambia nunca.

Este es un extracto de `Quest`, con los campos, la igualdad por identificador y la transición `publish`. El resto de transiciones las escribirás tú en el ejercicio 2.4:

Quest.java (extracto)

```
public final class Quest {

    private final QuestId id;
    private final String title;
    private final Reward reward;
    private QuestStatus status;
    private AdventurerId assignedTo;        // null hasta que alguien acepta

    // constructor privado y f^^c3^^a1brica est^^c3^^a1tica: el borrador nace
    ↪ en DRAFT

    public void publish() {
        if (status != QuestStatus.DRAFT) {
            throw new IllegalStateException("quest " + id.value()
                + " is " + status + ", expected DRAFT");
        }
        status = QuestStatus.PUBLISHED;
    }

    @Override
    public boolean equals(Object other) {
        return other instanceof Quest quest && id.equals(quest.id);
    }

    @Override
    public int hashCode() {
        return id.hashCode();
    }
}
```

Tres detalles. El estado no se asigna desde fuera: no hay `setStatus`, solo `publish`, que *comprueba antes* que la transición es válida. Una regla que debe cumplirse siempre, como esa, se llama *invariante*; ponerla dentro de la entidad hace que no se pueda saltar. El capítulo 5 las trabaja con errores tipados; aquí basta una excepción con un mensaje del negocio. El objeto `Reward` es inmutable, así que `Quest` puede guardarlo sin copiarlo. Y la misión guarda solo el *identificador* del aventurero (`AdventurerId`), no el objeto `Adventurer` entero; la sección siguiente explica por qué.

Igualdad por todos los campos en una entidad

Un *record* (o una clase con `@Data`, o con un `equals` escrito sobre todos los campos) compara por todos sus campos: dos misiones con el mismo título y recompensa pero distinto id serían «iguales», y, si la entidad es mutable, su `hashCode` cambiaría al cambiar su estado y una misión guardada en un `HashSet` «desaparecería» al aceptarla. Un record además es inmutable, así que no admite las transiciones. Una entidad se escribe como clase normal, con `equals` y `hashCode` sobre el identificador.

2.4 Agregados y repositorios

Las entidades no viven aisladas: una misión tiene recompensa, un aventurero tiene rango. Un *agregado* es un grupo de objetos que se trata como una unidad cuando cambia, con una entidad que lo representa, la *raíz del agregado*. Evans [8] lo resume así: lo de fuera solo puede tener referencias a la raíz, y las reglas que afectan a todo el grupo las garantiza la raíz.

En GuildBoard hay dos agregados, y cada uno tiene una sola entidad:

- `Quest`, con `Reward` como valor, un `String title` y el id del aventurero asignado.
- `Adventurer`, con su `Rank` y su experiencia.

Reglas de diseño que usará el curso, y de las que dependen los capítulos siguientes:

1. **Un agregado se referencia desde otro por identificador.** `Quest` guarda un id (`AdventurerId`), no un aventurero entero. Así no se carga un `Adventurer` para leer una misión y, más adelante, cada agregado puede vivir en un servicio distinto sin cambiar el modelo.
2. **Cada operación modifica como máximo un agregado.** Aceptar una misión modifica solo `Quest`; dar experiencia modifica solo `Adventurer`. Es una decisión de este curso, coherente con la idea de que cada agregado protege sus propias reglas.
3. **Hay un repositorio por raíz.** Un *repositorio* ofrece el acceso a los agregados como si fueran una colección, en el vocabulario del dominio, sin que este sepa dónde se guardan.

Un repositorio, solo la idea

```
import java.util.Optional;

public interface QuestRepository {

    Optional<Quest> findById(QuestId id);

    void save(Quest quest);
}
```

Es una *interfaz* que pertenece al lado del dominio: habla de `Quest` y de `QuestId`, no de tablas ni de JSON. Quién la implementa (una memoria, una base de datos) es un detalle. Es el DIP del capítulo anterior aplicado al almacenamiento, y en el capítulo 3 pasará a llamarse *puerto de salida*. Por ahora ni siquiera decimos en qué paquete vive: no fijamos estructura hasta la verificación de estructura 1, en el capítulo 3.

2.5 Dominio anémico frente a rico

Un modelo *anémico* tiene clases que solo guardan datos (campos con *getters* y *setters*) y servicios que contienen toda la lógica. Fowler lo describe como un antipatrón [9]: el modelo parece de objetos, pero se programa de forma procedimental, y las reglas del negocio no tienen dueño. Un modelo *rico* pone el comportamiento en las entidades y valores que tienen los datos.

Un aventurero anémico y su servicio:

Versión anémica

```
public class Adventurer {
    private String id;
    private Rank rank;
    private int experience;
    // getters y setters p^^c3^^blicos de los tres campos
}

public class AdventurerService {

    public void giveExperience(Adventurer adventurer, int points) {
        adventurer.setExperience(adventurer.getExperience() + points);
    }
}
```

El problema no está en `giveExperience`, está en todo lo demás: cualquier parte del programa puede llamar a `setExperience(-500)` o a `setRank(Rank.VETERAN)` sin pasar por el servicio. Ninguna regla protege el dato porque la regla no está *en* el dato. La versión rica:

Versión rica

```
public final class Adventurer {

    private final AdventurerId id;
    private Rank rank;
    private int experience;

    public void gainExperience(int points) {
        if (points <= 0) {
            throw new IllegalArgumentException(
                "points must be positive: " + points);
        }
        experience += points;       // la subida de rango llega en el cap. 5
    }

    public int experience() {
        return experience;
    }
}
```

No hay `setExperience`: la única forma de cambiar la experiencia es `gainExperience`, que rechaza los puntos no positivos. Cuando el cap. 5 añada los umbrales de rango, irán *dentro* de ese método y nadie podrá ascender a un aventurero sin ganar la experiencia.

No todo servicio sobra: un servicio de aplicación coordina (carga la misión, llama a su método, la guarda), pero no decide. La regla de oro: si una regla habla solo de los datos de una entidad, vive en la entidad. El ejercicio 2.2 te hace buscar las invariantes rotas en un caso anémico.

Rico no es «meter todo en la entidad»

Una entidad que consulta una base de datos, envía correos o lee el reloj ha dejado de ser dominio. El dominio rico contiene las *reglas*; lo que está fuera de él (guardar, avisar, el reloj) entra por puertos desde los capítulos siguientes.

2.5.1 Una regla que ningún agregado puede garantizar solo

Un caso incómodo, y real: «nadie lleva más de 3 misiones activas». ¿De quién es esta regla? Hay tres candidatos.

- **`Adventurer` guarda sus misiones activas.** Entonces aceptar modifica dos agregados (la misión y el aventurero), lo que rompe la regla de «uno por operación», y cada aventurero cargaría toda su lista.
- **`Quest.accept` recibe los datos que necesita.** `Quest` posee la transición `accept`: comprueba su estado, que el rango del aventurero alcance el de la misión y que el recuento de activas sea menor que 3. El rango y el recuento *llegan como datos* que lee quien orquesta (en el capítulo 5, el caso de uso, que se define en el capítulo 3): `accept` los recibe como parámetros.
- **Un servicio de dominio.** Sirve para reglas que no encajan en ninguna entidad, pero aquí la transición sí es de la misión.

El curso elige la segunda. Pero hay que ser honesto: `Quest.accept` puede comprobar que el recuento recibido es menor que 3, y no puede garantizar que ese recuento siga siendo cierto cuando guarda. Entre contar y guardar, otra aceptación puede colarse. Ningún agregado por sí solo asegura una regla que cruza muchas misiones; hace falta una restricción en la base de datos, un bloqueo u otro mecanismo. Lo reconoceremos en el capítulo 5 y no lo resolveremos por completo: es un límite del diseño, no un olvido.

2.6 Bounded contexts: solo la idea

En un sistema grande, la misma palabra no significa lo mismo en todas partes. «Aventurero» es, para quien publica misiones, poco más que un identificador y un rango; para quien lleva la contabilidad, es el titular de una cuenta; para quien gestiona el gremio, una ficha con experiencia y datos. Un *contexto delimitado* (*bounded context*) es una frontera dentro de la cual cada término del lenguaje ubicuo tiene un único significado; Fowler lo explica con un ejemplo corto [10]. Cada contexto tiene su modelo propio.

Para GuildBoard, el análisis anterior sugiere tres *candidatos*:

- **Quests**: misiones y sus transiciones.
- **Adventurers**: aventureros, rangos y experiencia.
- **Treasury**: el pago de las recompensas.

Mientras GuildBoard sea un monolito, los tres viven en un mismo programa, y eso está bien: los contextos son una idea de modelo, no una obligación de despliegue. En el capítulo 9 los cortarás en servicios, y entonces cada uno tendrá su propia idea de lo que es un aventurero. Lo que decidas hoy sobre dónde pasan las fronteras (por ejemplo, que `Quest` guarde un `AdventurerId`) hará ese corte barato.

2.7 En el proyecto: hito H0, el documento de dominio

Hito 0: docs/dominio.md

Este capítulo no escribe código del proyecto: escribe el *documento de dominio*. `docs/dominio.md` recoge:

1. el glosario (términos, identificadores y la regla «experiencia = oro»);
2. las reglas de negocio, numeradas;
3. los agregados y su reparto de responsabilidades;
4. la decisión sobre «máximo 3 misiones activas»;
5. los contextos candidatos.

Lo redactas tú en el ejercicio 2.5. En los capítulos siguientes, cada vez que dudes de dónde va una regla, abre ese archivo.

La estructura del proyecto no cambia

Este capítulo no toca la estructura: lo único que añade al repositorio es `docs/dominio.md`, y la carpeta `docs/` ya figura en el árbol que se propone en el capítulo 3. No hay `src/`, ni paquetes, ni cambios en `build.gradle`. El árbol completo figura en la verificación de estructura 1 del capítulo 3.

2.8 Ejercicios

Ejercicio 2.1: Clasifica los términos del gremio ★☆☆☆☆

Clasifica estos doce términos como *entidad*, *objeto de valor* o *dato derivado* (algo que no se guarda, se calcula), y señala cuáles son *raíz de agregado*: `Quest`, `Adventurer`, `Rank`, `Reward`, el título de la misión (`title`, un `String`), `QuestId`, `AdventurerId`, la experiencia acumulada de un aventurero, la fecha límite de una misión (`Deadline`), un pago al aventurero (`Payment`), el estado de la misión (`QuestStatus`) y las «misiones activas de un aventurero».

1. Para cada término pregúntate: ¿tiene identidad y un ciclo de vida, es decir, sigue siendo «el mismo» aunque cambien sus datos?
2. Si dos instancias con los mismos datos fueran intercambiables, es un objeto de valor.
3. Si no se guarda en ningún sitio porque se obtiene contando o calculando, es un dato derivado.
4. Entre las entidades, marca raíz de agregado las que dos reglas de este capítulo (referencia por id, una operación un agregado) obligan a tratar como unidad.

Está bien si tu tabla tiene doce filas, cada una con su clasificación y una razón de una línea, y si exactamente dos términos, `Quest` y `Adventurer`, están marcados como raíz de agregado.

Marca `Payment` como entidad, pero no como raíz: todavía no existe ningún agregado de pagos.

Guiado ▪ solución en la pág. 33

Ejercicio 2.2: Qué se puede romper en este Quest ★★☆☆☆

Lee esta clase y el servicio que la usa. No hay que ejecutar nada:

QuestAnemic.java

```java
public class Quest {
    private String status;          // "PUBLISHED", "ACCEPTED", "COMPLETED"
    private String assignedTo;
    private int gold;
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
    public String getAssignedTo() { return assignedTo; }
    public void setAssignedTo(String id) { this.assignedTo = id; }
    public int getGold() { return gold; }
    public void setGold(int gold) { this.gold = gold; }
}

public class QuestService {
    public void accept(Quest quest, String adventurerId, int active) {
        if (active >= 3) {
            throw new IllegalStateException("too many active quests");
        }
        quest.setStatus("ACCEPTED");
        quest.setAssignedTo(adventurerId);
    }
    public void complete(Quest quest) {
        quest.setStatus("COMPLETED");
    }
}
```

1. Enumera al menos cinco situaciones inválidas que este código permite (por ejemplo, una concreta con `accept` y otra con `complete`).
2. Para cada una, di si la causa es que el servicio no comprueba algo o que el *setter* permite saltarse el servicio.
3. Esboza qué métodos tendría un `Quest` rico para que ninguna fuese posible (basta con las firmas y una línea de explicación).

Está bien si identificas las dos familias de fallos (reglas ausentes en el servicio y datos desprotegidos) y propones métodos del negocio en lugar de *setters*.

Lectura de código ▪ solución en la pág. 33

Ejercicio 2.3: Reward con plus ★★☆☆☆

Escribe el record `Reward` con un componente `int gold`:

- su constructor compacto lanza `IllegalArgumentException` si `gold` es negativo;
- una constante estática `ZERO` (recompensa de 0 de oro);
- un método `plus(Reward other)` que devuelve una *nueva* recompensa con la suma.

Guárdalo en `RewardPlusDemo.java`, debajo de esta clase:

`RewardPlusDemo.java`

```
public class RewardPlusDemo {

    public static void main(String[] args) {
        Reward base = new Reward(40);
        Reward total = base.plus(new Reward(60));
        System.out.println(total);
        System.out.println(total.equals(new Reward(100)));
        System.out.println(base);
        System.out.println(Reward.ZERO.plus(base).equals(base));
    }
}
```

Está bien si `java RewardPlusDemo.java` imprime `Reward[gold=100]`, `true`, `Reward[gold=40]` y `true`: la suma devuelve otra recompensa y no modifica la original.

Pista: *una constante en un record se declara con* `public static final Reward ZERO = new Reward(0);` *dentro del cuerpo, y el resultado de* `plus` *es un* `new Reward(...)`*.*

Práctica ▪ solución en la pág. 34

Ejercicio 2.4: Las transiciones de Quest ★★★☆☆

Escribe la entidad `Quest` y el enum `QuestStatus` (`DRAFT`, `PUBLISHED`, `ACCEPTED`, `COMPLETED`). Parte del archivo `QuestFlow.java`, cuya clase con `main` y los records de apoyo ya tienes:

QuestFlow.java

```java
public class QuestFlow {
    public static void main(String[] args) {
        Quest quest = Quest.draft(new QuestId("q-1"), "Escoltar la caravana
    ↪ ",
            new Reward(60));
        System.out.println(quest.status());
        quest.publish();
        quest.accept(new AdventurerId("a-1"));
        System.out.println(quest.status() + " "
            + quest.assignedTo().orElseThrow().value());
        tryAccept(quest);
        quest.complete();
        System.out.println(quest.status());
        System.out.println(quest.equals(Quest.draft(new QuestId("q-1"),
            "Otro t^^c3^^adtulo", new Reward(1))));
    }

    static void tryAccept(Quest quest) {
        try {
            quest.accept(new AdventurerId("a-2"));
        } catch (IllegalStateException e) {
            System.out.println("rechazado: " + e.getMessage());
        }
    }
}

record QuestId(String value) { }
record AdventurerId(String value) { }
record Reward(int gold) { }
```

Reglas: `draft(id, title, reward)` crea la misión en `DRAFT` y rechaza un título en blanco con `IllegalArgumentException`; `publish` solo desde `DRAFT`, `accept` solo desde `PUBLISHED`, `complete` solo desde `ACCEPTED`. Una transición inválida lanza `IllegalStateException` con el mensaje `quest <id> is <estado actual>, expected <estado requerido>`. La igualdad de `Quest` se basa solo en su id.
Está bien si el programa imprime `DRAFT`, `ACCEPTED a-1`, `rechazado: quest q-1 is ACCEPTED, expected PUBLISHED`, `COMPLETED` y `true`, y si `Quest` no tiene ningún *setter*.

Pista: *`assignedTo()` devuelve `Optional<AdventurerId>` con `Optional.ofNullable` sobre el campo, que es `null` mientras nadie acepta.*

Práctica ▪ solución en la pág. 36

Ejercicio 2.5: Hito H0: redacta docs/dominio.md ★★★☆☆

En la carpeta `guildboard` (la del capítulo 1), crea la carpeta `docs` y escribe `docs/dominio.md`, en Markdown, con estas cinco secciones:

1. **Glosario**: los términos del capítulo con su identificador en inglés y la regla «experiencia de una misión = su recompensa en oro».
2. **Reglas de negocio**: numeradas, cada una en una frase (estados, rango, límite de 3 misiones activas, experiencia).
3. **Agregados**: `Quest` y `Adventurer`, qué contiene cada uno y qué transiciones posee

cada uno.

4. **El límite de 3 misiones activas**: a quién pertenece y qué datos llegan de fuera, con las alternativas descartadas y lo que la solución no garantiza.
5. **Contextos candidatos**: Quests, Adventurers y Treasury, con una frase sobre qué significa «aventurero» en cada uno.

Después, haz un commit con el mensaje `docs: dominio de GuildBoard (H0)`.
Está bien si: la sección de agregados dice que `Quest` posee `publish`, `accept` y `complete`; el rango del aventurero y el recuento de misiones activas se describen como datos que llegan a `accept`; la experiencia sale de la recompensa; el documento reconoce que ningún agregado garantiza solo el límite; y `git status` no muestra nada pendiente.

Proyecto ▪ solución en la pág. 38

2.9 Resumen

Resumen

- El lenguaje ubicuo es un glosario compartido cuyos términos son los nombres del código; el texto va en español y los identificadores en inglés.
- Un objeto de valor se define por sus datos, es inmutable y se compara por valor (record o enum); una entidad tiene identidad, su igualdad se basa en el id y cambia mediante métodos del negocio.
- Un agregado es una unidad de consistencia con una raíz; se referencia por id y cada operación modifica como máximo uno; hay un repositorio por raíz, definido en el lenguaje del dominio.
- Un modelo anémico deja las reglas sin dueño; uno rico las pone en las entidades y los valores, sin *setters* que las salten.
- En GuildBoard hay dos agregados: `Quest` (posee `publish`, `accept` y `complete`) y `Adventurer`. El rango y el recuento llegan como datos a `accept`, y ningún agregado garantiza solo el límite de 3 misiones activas.
- Un contexto delimitado es una frontera donde cada término tiene un significado; Quests, Adventurers y Treasury son candidatos, y se cortan en servicios en el capítulo 9.
- H0 queda completo con `docs/dominio.md`; la estructura del proyecto no cambia.

Soluciones de los ejercicios

Solución del ejercicio 2.1 enunciado en la pág. 28

Término	Clasificación	Razón
`Quest`	Entidad, raíz	Identidad (`QuestId`) y ciclo de vida: sigue siendo la misma al cambiar de estado.
`Adventurer`	Entidad, raíz	Identidad propia; su rango y su experiencia cambian.
`Rank`	Valor	Tres constantes sin identidad; dos `ADEPT` son intercambiables.
`Reward`	Valor	Una cantidad de oro: 50 de oro es 50 de oro.
Título (`String`)	Valor	Un texto comparado por contenido; `Quest` lo guarda como `String title`.
`QuestId`	Valor	Identifica a una entidad, pero él mismo es solo un valor.
`AdventurerId`	Valor	Igual que el anterior; es lo que `Quest` guarda del aventurero.
Experiencia acumulada	Valor	Un número dentro de `Adventurer`; no tiene vida propia.
`Deadline`	Valor	Un instante; dos fechas iguales son la misma.
`Payment`	Entidad	Cada pago se distingue de otro igual y se puede consultar; no es raíz porque en el monolito no hay agregado de pagos.
`QuestStatus`	Valor	Un enum de estados; no tiene identidad.
Misiones activas	Dato derivado	No se guarda: se cuenta sobre las misiones aceptadas del aventurero.

Por qué sale así. Las entidades son las que responden «¿cuál?»: `Quest`, `Adventurer` y `Payment`. Todo lo demás se define por sus datos. Los dos agregados del monolito son `Quest` y `Adventurer`; sus valores (`Reward`, `Rank`, los ids) viven dentro de ellos. `Payment` es entidad, pero no raíz: solo lo será cuando exista un servicio futuro del curso. Por tanto, la respuesta es única: dos raíces.
Errores típicos: clasificar `QuestId` como entidad (identifica a una entidad, pero no tiene identidad propia) y tratar «misiones activas» como un campo de `Adventurer`: si fuera un campo, aceptar una misión modificaría dos agregados.

Solución del ejercicio 2.2 enunciado en la pág. 29

1. Situaciones inválidas que el código permite:
   - Aceptar una misión ya `COMPLETED` (o ya aceptada por otro): `accept` no mira el estado y pisa el aventurero.
   - Completar una misión que sigue `PUBLISHED` y nunca se aceptó, con `assignedTo` a `null`.
   - Aceptar sin comprobar el rango: el servicio no recibe el rango y nadie lo compara con el exigido.
   - `setStatus("DONE")` o `setStatus(null)`: el estado es un `String` libre.
   - `setGold(-10)`, o cambiar la recompensa de una misión ya completada.
   - `setAssignedTo(null)` en una misión `ACCEPTED`, que la deja aceptada sin dueño.
   - Llamar a `setStatus("ACCEPTED")` directamente, sin pasar por `accept` y sin comprobar el límite de 3.
2. Reglas ausentes en el servicio: las dos primeras situaciones y la del rango (`accept` y `complete` no validan estado ni rango). Datos desprotegidos: las demás, porque el

setter permite hacer cualquier cosa sin pasar por el servicio, así que aunque arreglaras el servicio, el resto del código podría saltárselo.

3. Un `Quest` rico:

   - `static Quest draft(QuestId, String, Reward)`: crea el borrador y valida el título.
   - `void publish()`: solo desde `DRAFT`.
   - `void accept(AdventurerId)`: solo desde `PUBLISHED`.
   - `void complete()`: solo desde `ACCEPTED`.
   - `QuestStatus status()` y `Optional<AdventurerId> assignedTo()`: solo lectura.

   Los cuerpos son lo que escribes en el ejercicio 2.4. Sin *setters*, el estado es un enum, y la recompensa se fija al crear y no cambia; el rango y el límite de 3 entran como datos de `accept` en el capítulo 5.

Errores típicos: proponer solo «validar en el servicio» (no basta: el *setter* sigue ahí) o dejar los *setters* y añadir comprobaciones dentro de ellos (devuelve la lógica a un sitio sin un nombre del negocio).

Solución del ejercicio 2.3

enunciado en la pág. 29

RewardPlusDemo.java

```java
public class RewardPlusDemo {

    public static void main(String[] args) {
        Reward base = new Reward(40);
        Reward total = base.plus(new Reward(60));
        System.out.println(total);
        System.out.println(total.equals(new Reward(100)));
        System.out.println(base);
        System.out.println(Reward.ZERO.plus(base).equals(base));
    }
}

record Reward(int gold) {

    public static final Reward ZERO = new Reward(0);

    Reward {
        if (gold < 0) {
            throw new IllegalArgumentException(
                "gold must not be negative: " + gold);
        }
    }

    public Reward plus(Reward other) {
        return new Reward(gold + other.gold);
    }
}
```

La salida es:

```
Reward[gold=100]
true
Reward[gold=40]
true
```

`plus` no modifica `this`: construye otro `Reward`, que además pasa por el constructor compacto y valida el resultado. La comparación por valor la da el record: dos recompensas de 100 son iguales aunque sean objetos distintos.
Errores típicos: escribir `this.gold += other.gold` (no compila: el campo es `final`) y declarar `ZERO` sin `static final` o fuera del cuerpo del record.

Solución del ejercicio 2.4 enunciado en la pág. 30

QuestFlow.java (1/2)

```
import java.util.Optional;

public class QuestFlow {
    public static void main(String[] args) {
        Quest quest = Quest.draft(new QuestId("q-1"), "Escoltar la caravana
    ↪ ",
            new Reward(60));
        System.out.println(quest.status());
        quest.publish();
        quest.accept(new AdventurerId("a-1"));
        System.out.println(quest.status() + " "
            + quest.assignedTo().orElseThrow().value());
        tryAccept(quest);
        quest.complete();
        System.out.println(quest.status());
        System.out.println(quest.equals(Quest.draft(new QuestId("q-1"),
            "Otro t^^c3^^adtulo", new Reward(1))));
    }

    static void tryAccept(Quest quest) {
        try {
            quest.accept(new AdventurerId("a-2"));
        } catch (IllegalStateException e) {
            System.out.println("rechazado: " + e.getMessage());
        }
    }
}

record QuestId(String value) { }
record AdventurerId(String value) { }
record Reward(int gold) { }

enum QuestStatus { DRAFT, PUBLISHED, ACCEPTED, COMPLETED }

final class Quest {

    private final QuestId id;
    private final String title;
    private final Reward reward;
    private QuestStatus status;
    private AdventurerId assignedTo;

    private Quest(QuestId id, String title, Reward reward) {
        this.id = id;
        this.title = title;
        this.reward = reward;
        this.status = QuestStatus.DRAFT;
    }
```

Continuación del listado anterior:

QuestFlow.java (2/2)

```
    static Quest draft(QuestId id, String title, Reward reward) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("title must not be blank");
        }
        return new Quest(id, title.strip(), reward);
    }

    void publish() {
        requireStatus(QuestStatus.DRAFT);
        status = QuestStatus.PUBLISHED;
    }

    void accept(AdventurerId adventurerId) {
        requireStatus(QuestStatus.PUBLISHED);
        assignedTo = adventurerId;
        status = QuestStatus.ACCEPTED;
    }

    void complete() {
        requireStatus(QuestStatus.ACCEPTED);
        status = QuestStatus.COMPLETED;
    }

    QuestStatus status() {
        return status;
    }

    Optional<AdventurerId> assignedTo() {
        return Optional.ofNullable(assignedTo);
    }

    private void requireStatus(QuestStatus required) {
        if (status != required) {
            throw new IllegalStateException("quest " + id.value()
                + " is " + status + ", expected " + required);
        }
    }

    @Override
    public boolean equals(Object other) {
        return other instanceof Quest quest && id.equals(quest.id);
    }

    @Override
    public int hashCode() {
        return id.hashCode();
    }
}
```

La salida de `java QuestFlow.java` es:

```
DRAFT
ACCEPTED a-1
rechazado: quest q-1 is ACCEPTED, expected PUBLISHED
COMPLETED
true
```

Cada transición empieza con `requireStatus`, que concentra la invariante «solo desde el estado correcto» en un único sitio; el estado solo cambia dentro de la clase, porque no hay *setters*. La fábrica `draft` valida el título antes de crear nada, y `equals` mira solo el id: por eso las dos misiones del final son la misma aunque su título y su recompensa difieran. Los campos `title` y `reward` se guardan para que existan, aunque el programa de prueba no los lea; el capítulo 5 los usa (la recompensa, como experiencia).
Errores típicos: olvidar `import java.util.Optional;`, añadir un `setStatus` «para los tests» (abre la puerta que se quería cerrar) y escribir `equals` comparando todos los campos, con lo que la misma misión deja de ser igual a sí misma tras cambiar de estado.

Solución del ejercicio 2.5

enunciado en la pág. 31

Comandos:

```
$ cd guildboard
$ mkdir docs
$ # escribe aqu^^c3^^ad docs/dominio.md con el contenido de abajo
$ git add docs/dominio.md
$ git commit -m "docs: dominio de GuildBoard (H0)"
$ git status
```

El archivo `docs/dominio.md` (el contenido completo):

docs/dominio.md (1/2)

```
# Dominio de GuildBoard

GuildBoard es el tabl^^c3^^b3n de misiones de un gremio: el maestro publica
misiones, los aventureros las aceptan y las completan, y con ello cobran
y ganan experiencia. Texto en espa^^c3^^b1ol; identificadores en ingl^^c3^^
    ↪ a9s.

## 1. Glosario

- Misi^^c3^^b3n (`Quest`): encargo con t^^c3^^adtulo, recompensa y rango m
    ↪ ^^c3^^adnimo.
- Aventurero (`Adventurer`): miembro del gremio que acepta misiones.
- Rango (`Rank`): `NOVICE` < `ADEPT` < `VETERAN`.
- Recompensa (`Reward`): oro que cobra quien completa una misi^^c3^^b3n.
- Experiencia (`experience`): puntos acumulados por un aventurero.
- Maestro del gremio (`GUILD_MASTER`): quien publica las misiones.
- Publicar (`publish`): poner una misi^^c3^^b3n borrador en el tabl^^c3^^
    ↪ b3n.
- Aceptar (`accept`): un aventurero se compromete con una misi^^c3^^b3n
  publicada.
- Completar (`complete`): se da por cumplida una misi^^c3^^b3n aceptada.
- Misi^^c3^^b3n activa: misi^^c3^^b3n `ACCEPTED` de un aventurero (no
    ↪ completada).
- Regla de experiencia: la experiencia que da una misi^^c3^^b3n es igual a
    ↪ su
  recompensa en oro (1 de oro = 1 punto de experiencia).

## 2. Reglas de negocio

R1. Una misi^^c3^^b3n nace como `DRAFT` (borrador) y pasa por `PUBLISHED`,
    `ACCEPTED` y `COMPLETED`, en ese orden y sin saltos.
R2. Solo se publica un borrador, solo se acepta una misi^^c3^^b3n `
    ↪ PUBLISHED`
    y solo se completa una `ACCEPTED`.
R3. Un aventurero solo acepta misiones cuyo rango m^^c3^^adnimo no supere
    ↪ el
    suyo (`Rank.canTake`).
R4. Nadie lleva m^^c3^^a1s de 3 misiones activas.
R5. Completar una misi^^c3^^b3n da al aventurero experiencia igual a su
    recompensa en oro.
R6. La experiencia solo aumenta; los puntos siempre son positivos.
R7. La recompensa no es negativa y el t^^c3^^adtulo no est^^c3^^a1 en
    ↪ blanco.

## 3. Agregados
```

Continuación del listado anterior:

docs/dominio.md (2/2)

```
- `Quest` (ra^^c3^^adz). Contiene `QuestId`, t^^c3^^adtulo, `Reward`, rango m^^c3^^adnimo,
  estado y `assignedTo` (solo el `AdventurerId`, nunca el aventurero).
  Posee las transiciones `publish`, `accept` y `complete`.
- `Adventurer` (ra^^c3^^adz). Contiene `AdventurerId`, `Rank` y experiencia.
  Posee `gainExperience`, que suma puntos y gestiona la subida de rango.

Un agregado se referencia desde otro por identificador, y cada operaci^^c3^^b3n
modifica como m^^c3^^a1ximo un agregado: aceptar modifica solo `Quest`; dar
experiencia modifica solo `Adventurer`. Hay un repositorio por ra^^c3^^adz.

## 4. El l^^c3^^admite de 3 misiones activas (R4)

Pertenece a `Quest.accept`, que posee la transici^^c3^^b3n. El rango del
aventurero y el n^^c3^^bamero de sus misiones activas son datos que llegan de
fuera: los lee quien orquesta (el caso de uso) y los pasa a `accept`,
que comprueba estado, rango y que el n^^c3^^bamero sea menor que 3.

Alternativas descartadas:
- `Adventurer` con la lista de sus misiones activas: aceptar tocar^^c3^^ada
  dos agregados, y cada aventurero cargar^^c3^^ada su lista completa.
- Un servicio de dominio: la transici^^c3^^b3n es de la misi^^c3^^b3n, no hay una
  regla que no encaje en ninguna entidad.

Lo que no garantiza: `accept` solo ve el n^^c3^^bamero que le pasan. Entre
contar y guardar otra aceptaci^^c3^^b3n puede colarse, y ning^^c3^^ban agregado, por
s^^c3^^ad solo, asegura una regla que cruza muchas misiones. Har^^c3^^a1 falta una
restricci^^c3^^b3n en base de datos, un bloqueo u otro mecanismo; este
documento lo reconoce como l^^c3^^admite del dise^^c3^^b1o.

## 5. Contextos candidatos

- Quests: misiones y sus transiciones. Un aventurero es solo un id y
  un rango, lo justo para decidir si puede aceptar.
- Adventurers: aventureros, rangos y experiencia. Un aventurero es una
  ficha completa con su progreso.
- Treasury: pago de recompensas. Un aventurero es el titular de una
  cuenta a la que se abona el oro.

Hoy los tres conviven en un monolito; los contextos son una idea de
modelo, no de despliegue.
```

Por qué es la respuesta correcta. Recoge cada elemento que pedía el ejercicio. La decisión del límite sigue el reparto del capítulo: `Quest` posee `accept` y el rango y el recuento llegan como datos; el documento descarta las otras dos opciones con su razón y admite lo que no resuelve. La regla R5 hace explícito que la experiencia sale del oro de la recompensa. **Errores típicos:** meter en el documento código o decisiones de estructura (paquetes, carpetas) que aún no se

han tomado, repartir R4 entre dos agregados sin decidir quién la comprueba, y olvidar que `docs/` no existe hasta que la creas, por lo que `git add` falla.

CAPÍTULO 3

La arquitectura hexagonal y su estructura de carpetas

Objetivos del capítulo

- Explicar qué son los puertos y los adaptadores (primarios y secundarios) y qué problema resuelven.
- Enunciar la regla de dependencia y atribuirla bien: qué dijo Cockburn y qué unificó Martin.
- Verificar la estructura de carpetas del proyecto con una lista de comprobación y compararla con la de Buckpal.
- Instalar Lombok, JUnit y ArchUnit y escribir una prueba de arquitectura que comprueba la regla de dependencia (hito H1, parte 1).

Ya sabes qué reglas tiene GuildBoard y cómo se escriben sus objetos. Ahora toca fijar *dónde* vive cada clase y *qué puede conocer* cada carpeta, y esa estructura es la que, dentro de un año, deja cambiar la base de datos sin tocar las reglas o convierte cada cambio en una cirugía. Este capítulo explica la idea (puertos y adaptadores), la regla que la sostiene (la regla de dependencia), fija el primer árbol de carpetas del proyecto y lo verifica con una lista de comprobaciones. Después instala Lombok, escribe el dominio mínimo con sus dos primeros puertos y una prueba de arquitectura, escrita con ArchUnit, que vigila la regla cada vez que ejecutas `./gradlew test`.

3.1 El problema: la lógica enredada con el exterior

Cockburn describió el patrón en 2005 [4] a partir de un vicio muy común: la lógica de negocio se cuela en el código de la interfaz de usuario. Las consecuencias que enumera son tres: el sistema no se puede probar con pruebas automáticas porque parte de la lógica depende de detalles visuales; no se puede pasar de un uso manual a un proceso por lotes; y no se puede dejar que otro programa maneje la aplicación. Y hay un problema simétrico «al otro lado»: cuando la lógica queda atada a una base de datos concreta, si el servidor cae o se cambia, nadie puede trabajar.

Su diagnóstico es que ambos problemas son el mismo error: el enredo entre la lógica de negocio y la conversación con entidades externas. La asimetría que hay que explotar no es izquierda frente a derecha, sino **dentro frente a fuera**: lo de dentro no se filtra a lo de fuera. La solución son los *puertos y adaptadores*:

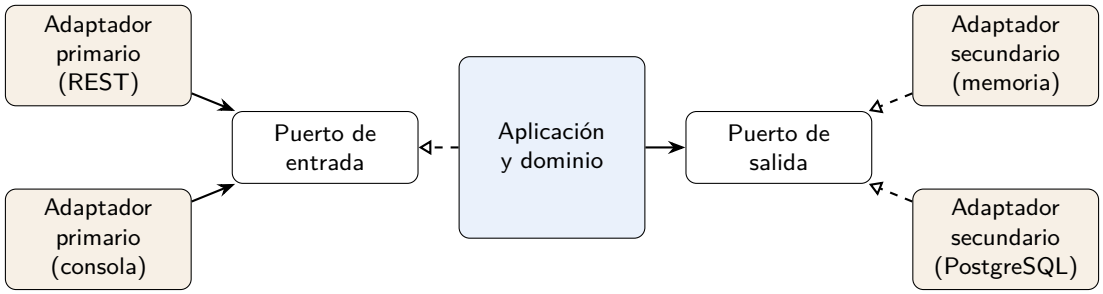
- Un *puerto* es la interfaz por la que la aplicación conversa con el exterior, definida según el propósito de la conversación, no según la tecnología.
- Un *adaptador* es la pieza que traduce entre ese puerto y un dispositivo concreto: una pantalla, un cliente HTTP, una base de datos, un script de pruebas.

Cockburn distingue dos sabores. Los adaptadores **primarios** (también llamados *driving*, «los que

conducen») son los que *llaman* a la aplicación: un controlador REST, una línea de comandos, un temporizador, una prueba. Los **secundarios** (*driven*, «los conducidos») son los que la aplicación *llama*: la base de datos, el envío de correos. Para esos dos sentidos hay dos tipos de puerto:

- **Puerto de entrada**: lo que la aplicación *ofrece*. En GuildBoard, los casos de uso, como `PublishQuestUseCase`. Lo *implementa* la propia aplicación y lo *usa* un adaptador primario.
- **Puerto de salida**: lo que la aplicación *necesita*. En GuildBoard, `QuestRepository`. Lo *declara* la aplicación y lo *implementa* un adaptador secundario (la memoria hoy, PostgreSQL en el capítulo 8).

Es el DIP del capítulo 1 aplicado en los dos extremos: la interfaz siempre pertenece al lado de la aplicación.



Flecha continua: «usa» o «conoce». Flecha discontinua: «implementa». Mira dónde terminan las flechas: ninguna sale del centro hacia un adaptador. La aplicación solo conoce los dos puertos; los adaptadores conocen los puertos y no se conocen entre sí.

Por qué un hexágono

Cockburn lo aclara él mismo: el hexágono no es un hexágono porque el seis importe, sino para dar sitio en el dibujo a los puertos y adaptadores que haga falta, sin la imagen unidimensional de una pila de capas. Que haya seis lados no significa que haya seis puertos.

La misma idea explicada con Java y Spring, centrada en casos de uso, puertos y adaptadores, está en el artículo de Hombergs [18] (en este curso solo se usa para los conceptos, no para su estructura de carpetas), y un resumen independiente de cuándo aplicarla es la guía de AWS [1]. Un aviso honesto desde ya: el patrón no es gratis. Tendrás más interfaces, más clases y una pieza que ensambla todo. Lo que compras con ese coste es poder probar y cambiar los bordes sin tocar el centro.

3.2 Hexagonal, onion y Clean: la regla de dependencia

Hexagonal no es la única propuesta con esta forma. Palermo presentó la arquitectura *onion* en 2008 [35]: el modelo de dominio en el centro, capas alrededor y una regla fundamental: el código puede depender de capas más centrales, pero no de las más externas. Martin, en 2012, juntó esas ideas (hexagonal, onion, *screaming* y otras) en un solo diagrama de círculos concéntricos y las resumió en lo que llama **la regla de dependencia** [27]:

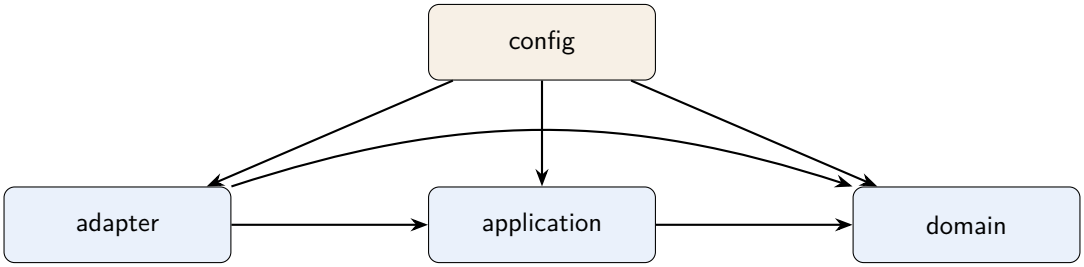
> *las dependencias del código fuente solo pueden apuntar hacia dentro.* Nada de un círculo interior puede conocer nada de uno exterior; en particular, el nombre de algo declarado fuera no puede aparecer en el código de dentro.

(Es una paráfrasis, no una cita literal.) El mismo artículo explica cómo se cumple cuando el control tiene que ir hacia fuera: el caso de uso llama a una *interfaz* que está dentro y es otra pieza, de fuera, la que la implementa. Es exactamente la inversión de dependencias del capítulo anterior. Y pide que por los límites crucen estructuras de datos simples y no entidades ni filas de base de datos, una idea que volverá en el capítulo 6. Martin desarrolla el tema en su libro, en los capítulos 17 «Boundaries: Drawing Lines», 18 «Boundary Anatomy» y 22 «The Clean Architecture» [28].

Quién dijo qué

Cockburn habla de dentro y fuera, de puertos y adaptadores; no formula la «regla de dependencia». Palermo enuncia algo muy parecido para su cebolla. La *síntesis* (que hexagonal, onion y Clean comparten una misma regla y un mismo diagrama) es de Martin. En este libro, cuando se diga «regla de dependencia», se atribuye a Martin; cuando se diga «puertos y adaptadores», a Cockburn.

Traducida a las carpetas de GuildBoard, la regla da este grafo. Cada flecha es un `import`:



- `domain` no importa nada del proyecto ni de ningún framework.
- `application` importa `domain`, y nada más del proyecto.
- `adapter` importa `application` y, si hace falta, `domain`.
- `config` es la pieza que lo ensambla todo y por eso lo conoce todo; nadie la importa. El capítulo 4 la construye.

Como el grafo de dependencias tiene solo una dirección, una dependencia en la dirección contraria es una infracción, y se puede detectar de forma automática. Eso es lo que hará la prueba de la sección 3.8.

Dos matices que conviene fijar. Primero, un puerto de salida vive en `application` aunque lo implemente algo de `adapter`: por eso el adaptador depende de la aplicación y no al revés. Segundo, la regla habla de dependencias de código fuente, no del flujo de ejecución: en tiempo de ejecución, una petición entra por el adaptador, atraviesa la aplicación y vuelve a salir por otro adaptador; son las flechas de `import` las que no cambian.

3.3 Dónde pone cada fuente las carpetas

Ninguna fuente fija «la» estructura. Lo que sigue es lo que cada una dice, y lo que esa fuente *no* dice.

Cockburn no prescribe carpetas: lo que importa es que haya dentro y fuera, puertos y adaptadores [4].

Buckpal. Hombergs desarrolla en su libro [19] una estructura de paquetes concreta, y su repositorio de ejemplo, *buckpal* [20], es el que sirve de comparación. Para escribir este capítulo

se ha leído su árbol de archivos en GitHub (rama `master`, consultado el 8 de octubre de 2026; el último *commit* es de julio de 2023). Debajo del paquete base `io.reflectoring.buckpal` hay:

Paquetes de Buckpal (verificado)

```
BuckPalApplication.java
BuckPalConfiguration.java
adapter/
|-- in/web/                (SendMoneyController)
`-- out/persistence/       (AccountPersistenceAdapter, ...)
application/
|-- domain/
|   |-- model/             (Account, Activity, Money, ...)
|   `-- service/           (SendMoneyService, ...)
`-- port/
    |-- in/                (SendMoneyUseCase, SendMoneyCommand, ...)
    `-- out/               (LoadAccountPort, UpdateAccountStatePort, ...)
common/                    (UseCase, WebAdapter, PersistenceAdapter, ...)
```

Dos observaciones sobre ese árbol. La primera: **agrupa por capas, no por funcionalidad**: no hay un paquete intermedio del tipo `account/` que reúna puertos, adaptadores y dominio de una misma funcionalidad. Es un proyecto de un solo contexto, como el nuestro en el monolito. La segunda: es un módulo Gradle único, con Spring Boot 3.1 y Java 17, y la regla de dependencia la vigila una prueba de ArchUnit (hay un `DependencyRuleTests` y un paquete `archunit` en sus pruebas). Nada de eso es referencia de versiones para este curso.

Martin aporta un argumento, no un árbol. En el capítulo 21 «Screaming Architecture» sostiene que la estructura de un sistema debería comunicar su intención (qué hace), no el framework que usa [28]. Un árbol por capas (`domain`, `application`, `adapter`) lo cumple a medias: dice cómo está construido, no que trata de misiones de un gremio. Es una limitación del monolito que el libro reconoce, y que el capítulo 9 resolverá subiendo un nivel: el primer nivel de carpetas pasará a ser `services/quests`, `services/adventurers`, `services/treasury`. El ensamblado fuera del núcleo se apoya en el capítulo 26 «The Main Component» del mismo libro.

3.3.1 La estructura que adopta el curso, y en qué se diferencia de Buckpal

El curso adopta una adaptación propia. Las diferencias con Buckpal son deliberadas:

Tema	Buckpal	El curso y su motivo
El dominio	Dentro: `application/domain/model` y `service`.	`domain` es hermano de `application`. Coincide con los círculos de Martin y permitirá, en el capítulo 7, dos módulos Gradle sin mover paquetes.
El ensamblado	`BuckPalConfiguration` en el paquete raíz; los servicios llevan `@UseCase`, que es un `@Component` de Spring con otro nombre.	Un paquete `config` aparte, y los servicios de aplicación sin ninguna anotación de Spring: se registran allí. `application` no depende de Spring. Coste: más código de composición.
Paquete `common`	Existe, con anotaciones propias y validación.	No hay. Lo compartido que haga falta entra por `domain` o por `application`.
Transacciones	`@Transactional` en el servicio de aplicación.	Solo en el adaptador de persistencia (capítulo 8).
Vigilar la regla	Prueba de ArchUnit (`DependencyRuleTests`) [21].	También una prueba de ArchUnit, `ArchitectureTest`, con reglas escritas a mano. No es una diferencia de fondo.

Y una diferencia que podrías esperar y **no existe**: ambos organizan por capas, sin nivel de funcionalidad. Cuando en el capítulo 9 el proyecto se corte en servicios, el nivel `services/<servicio>/` cumplirá el papel de esa agrupación por contexto.

Los nombres `port/in` y `port/out` y la separación entre `adapter/in` y `adapter/out` son los de Buckpal [20, 19]; `domain` como hermano de `application` sigue los círculos de Martin [27], y `config` como raíz de composición se apoya en el capítulo 26 de su libro [28]. El resto (por ejemplo, que el ensamblado se llame `config`) es una convención de este curso, no de una fuente.

3.4 Verificación de estructura 1: el árbol inicial

Nota

Este capítulo fija la estructura de carpetas. Comprueba que tu proyecto coincide con el árbol de abajo antes de seguir.

El árbol anterior, el que dejó el capítulo 2, no tiene `src/` ni estructura de paquetes: solo `.git/`, `.gitignore`, `settings.gradle`, `build.gradle`, el *wrapper* de Gradle y `docs/dominio.md`. Lo nuevo va marcado con `[N]` y lo modificado con `[M]`:

Árbol T1: lo que existe al terminar el capítulo 3

```
guildboard/
|-- .git/
|-- .gitignore
|-- settings.gradle
|-- build.gradle                    [M] Lombok, JUnit y ArchUnit
|-- gradlew
|-- gradlew.bat
|-- docs/
|   `-- dominio.md
|-- gradle/
|   `-- wrapper/
`-- src/
    |-- main/java/com/guildboard/   [N]
    |   |-- domain/                 [N] entidades, valores; sin frameworks
    |   `-- application/            [N]
    |       `-- port/
    |           |-- in/             [N] casos de uso (interfaces)
    |           `-- out/            [N] puertos de salida (interfaces)
    `-- test/java/com/guildboard/   [N]
        `-- ArchitectureTest.java   [N] prueba de la regla de dependencia
```

El árbol incluye además estas carpetas *previstas*. No se crean todavía: las crean los capítulos 4 y 5, que por eso no cambian la estructura.

Carpetas previstas (se crean después)

```
src/main/java/com/guildboard/
|-- GuildBoardApplication.java      (cap. 4)
|-- application/
|   `-- service/                    (cap. 4) implementan port/in
|-- adapter/
|   |-- in/web/                     (cap. 4) controladores REST
```

```
|   `-- out/memory/                (cap. 4) repositorios en memoria
`-- config/                        (cap. 4) ra^^c3^^adz de composici^^c3^^b3n
src/main/resources/application.yml (cap. 4)
src/test/java/com/guildboard/      (cap. 5) pruebas del dominio; la carpeta ya
                                   existe por ArchitectureTest
```

Un paquete temporal `adapter/in/cli` aparecerá en un ejercicio del capítulo 4 y se borrará: no forma parte del árbol.

Cambios respecto al árbol anterior:

- `src/main/java/com/guildboard/domain/`: entidades y valores del dominio.
- `src/main/java/com/guildboard/application/port/in/`: casos de uso.
- `src/main/java/com/guildboard/application/port/out/`: puertos de salida.
- `src/test/java/com/guildboard/ArchitectureTest.java`: la prueba de arquitectura.
- `build.gradle`: se añaden Lombok, JUnit y ArchUnit.
- `.git` y `.gitignore`: sin cambios.

Un `.gitignore` que se traga `port/out`

El `.gitignore` del capítulo 1 ya ancla `/bin/` y `/out/` (la carpeta de salida de IntelliJ) a la raíz. Si estuvieran escritos como `out/`, sin barra inicial, esa regla valdría en *cualquier* nivel, y Git ignoraría sin avisar la carpeta `application/port/out` y todo lo que haya dentro; con `bin/` pasa lo mismo con cualquier paquete llamado `bin`. Por eso las dos reglas están ancladas a la raíz del repositorio:

.gitignore (fragmento, ya en el capítulo 1)

```
/bin/
/out/
```

No hay que tocarlas. Si alguna vez dudas de por qué Git no ve un archivo, `git check-ignore -v ruta/del/archivo` te dice qué regla lo ignora.

Comprobaciones sobre *tu* proyecto:

- [ ] `./gradlew test` pasa (cuando termines el capítulo). Incluye la prueba de arquitectura, que ya vigila que `domain` no conozca las capas exteriores y que `application` dependa solo de `domain`.
- [ ] Cada adaptador está en `in` o en `out`: si *llama* a la aplicación va en `in` (el controlador); si la aplicación lo *llama*, va en `out` (el repositorio). Todavía no hay adaptadores, y esto es lo que ninguna prueba automática decide por ti.
- [ ] Los nombres de las carpetas dicen qué hace el sistema o qué papel juega cada pieza (`port/in`, `port/out`), no con qué tecnología está hecho.
- [ ] Las carpetas de tu proyecto coinciden con el árbol T1.

Si algo no coincide con el árbol, corrígelo antes de seguir. El ejercicio 3.2 compara esta estructura con la de Buckpal.

3.5 Lombok: instalación y uso

Lombok es una biblioteca que genera código repetitivo (*getters*, constructores, `equals`) cuando compila, a partir de anotaciones. Una advertencia sincera: con *records* (capítulo 1) la mitad de su utilidad ya viene en el lenguaje. Este libro la usa porque el briefing pide Lombok y porque es habitual en proyectos reales, pero solo donde aporta algo: en clases con identidad y estado, no en objetos de valor ni DTO.

☞ Lombok 1.18.48 en Gradle

La documentación de Lombok para Gradle [37] usa dos configuraciones, y hacen falta las dos. En `build.gradle`, añade después del bloque `repositories`:

`build.gradle (añadido)`

```
dependencies {
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

- `compileOnly`: la biblioteca está disponible para compilar, pero no viaja con la aplicación. Las anotaciones de Lombok solo se leen al compilar; en ejecución no hacen falta.
- `annotationProcessor`: el procesador que genera el código. Gradle no lo descubre solo desde el classpath: sin esta línea las anotaciones se ignoran y los métodos generados no existen.

Si algún día una clase de *pruebas* usara Lombok, harían falta además `testCompileOnly` y `testAnnotationProcessor`; este curso no lo hace. Se comprueba al final de la sección siguiente, cuando haya código que lo use. Si tu editor (por ejemplo, NeoVim con jdtls) subraya en rojo `getTitle()` aunque Gradle compile bien, es que el editor no ejecuta el procesador: configúralo según la documentación de tu cliente de jdtls. El curso se valida con `./gradlew`, no con el editor.

Dónde se usará Lombok a lo largo del curso:

- `@Getter` y `@EqualsAndHashCode` (solo con el identificador) en `Quest` y `Adventurer` (`domain`): las entidades con identidad.
- `@RequiredArgsConstructor` en los servicios de aplicación y los adaptadores (desde el capítulo 4): genera el constructor con los campos `final`, que es la inyección por constructor sin escribirla.
- `@Slf4j` en el adaptador de notificaciones (capítulo 6).
- `@Getter`, `@Setter` y `@NoArgsConstructor` en las entidades JPA (capítulo 8), que viven en el adaptador, no en el dominio.
- En ningún sitio: objetos de valor y DTO, que son *records*.

Lombok y la regla de dependencia. Lombok es una dependencia de *compilación*, no de ejecución, y no es un framework que dé forma a la lógica. Por eso el curso la permite en `domain` y en `application`: la regla de `application` de la sección 3.8 incluye `lombok` entre los paquetes permitidos. Si el dominio dependiera de una anotación de Spring, sí cambiaría de dueño.

`@EqualsAndHashCode` sin límites

Si escribes `@EqualsAndHashCode` a secas, Lombok incluye *todos* los campos, y una entidad cambiaría de `hashCode` al cambiar de estado: el problema del `HashSet` del capítulo 2. Con `onlyExplicitlyIncluded = true` solo cuenta lo que marques con `@EqualsAndHashCode.Include`: el identificador.

3.6 El dominio mínimo y los dos puertos

Un hexágono vacío no se puede comprobar, así que este capítulo escribe lo justo: la entidad `Quest` con sus valores, un puerto de entrada y un puerto de salida. Los paquetes Java siguen a las carpetas: un archivo en `src/main/java/com/guildboard/domain/` declara `package com.guildboard.domain;`. El código de los ejercicios del capítulo 2 pasa al proyecto con tres cambios: todo es `public` y lleva paquete, `Quest` usa Lombok y `Quest` gana el rango mínimo que ya mencionaba `docs/dominio.md`.

Empezamos por los valores. Primero los dos identificadores, `QuestId` y `AdventurerId`, cada uno en su archivo de `domain`:

domain/QuestId.java

```
1 package com.guildboard.domain;
2
3 public record QuestId(String value) { }
```

domain/AdventurerId.java

```
1 package com.guildboard.domain;
2
3 public record AdventurerId(String value) { }
```

El rango, la recompensa y los estados:

domain/Rank.java

```
1 package com.guildboard.domain;
2
3 public enum Rank {
4     NOVICE, ADEPT, VETERAN;
5
6     public boolean canTake(Rank required) {
7         return this.compareTo(required) >= 0;
8     }
9 }
```

domain/Reward.java

```
1 package com.guildboard.domain;
2
3 public record Reward(int gold) {
4
5     public static final Reward ZERO = new Reward(0);
```

```
6
7       public Reward {
8           if (gold < 0) {
9               throw new IllegalArgumentException(
10                  "gold must not be negative: " + gold);
11          }
12      }
13
14      public Reward plus(Reward other) {
15          return new Reward(gold + other.gold);
16      }
17  }
```

domain/QuestStatus.java

```
1  package com.guildboard.domain;
2
3  public enum QuestStatus { DRAFT, PUBLISHED, ACCEPTED, COMPLETED }
```

La entidad. Es el `Quest` del capítulo anterior con las anotaciones de Lombok, el rango mínimo y las comprobaciones de nulos. La versión mínima de `accept(AdventurerId)` se queda como en el capítulo 2: el rango y el límite de misiones activas llegan en el capítulo 5. Con Lombok, los accesores dejan de ser `status()` y `assignedTo()`, como en el capítulo 2, y pasan a ser `getStatus()` y `getAssignedTo()`.

domain/Quest.java

```
1  package com.guildboard.domain;
2
3  import java.util.Objects;
4  import java.util.Optional;
5  import lombok.EqualsAndHashCode;
6  import lombok.Getter;
7
8  @Getter
9  @EqualsAndHashCode(onlyExplicitlyIncluded = true)
10 public final class Quest {
11
12     @EqualsAndHashCode.Include
13     private final QuestId id;
14     private final String title;
15     private final Reward reward;
16     private final Rank requiredRank;
17     private QuestStatus status;
18     private AdventurerId assignedTo;        // null hasta aceptar
19
20     private Quest(QuestId id, String title, Reward reward,
21                   Rank requiredRank) {
22         this.id = id;
23         this.title = title;
24         this.reward = reward;
25         this.requiredRank = requiredRank;
26         this.status = QuestStatus.DRAFT;
27     }
28
```

```
29      public static Quest draft(QuestId id, String title, Reward reward,
30                                Rank requiredRank) {
31          Objects.requireNonNull(id, "id");
32          Objects.requireNonNull(reward, "reward");
33          Objects.requireNonNull(requiredRank, "requiredRank");
34          if (title == null || title.isBlank()) {
35              throw new IllegalArgumentException("title must not be blank");
36          }
37          return new Quest(id, title.strip(), reward, requiredRank);
38      }
39
40      public void publish() {
41          requireStatus(QuestStatus.DRAFT);
42          status = QuestStatus.PUBLISHED;
43      }
44
45      public void accept(AdventurerId adventurerId) {
46          requireStatus(QuestStatus.PUBLISHED);
47          assignedTo = Objects.requireNonNull(adventurerId, "adventurerId");
48          status = QuestStatus.ACCEPTED;
49      }
50
51      public void complete() {
52          requireStatus(QuestStatus.ACCEPTED);
53          status = QuestStatus.COMPLETED;
54      }
55
56      // Escrito a mano: Lombok no genera un getter si ya existe uno con
57      // ese nombre, y as^^c3^^ad se devuelve un Optional en lugar de null.
58      public Optional<AdventurerId> getAssignedTo() {
59          return Optional.ofNullable(assignedTo);
60      }
61
62      private void requireStatus(QuestStatus required) {
63          if (status != required) {
64              throw new IllegalStateException("quest " + id.value()
65                  + " is " + status + ", expected " + required);
66          }
67      }
68  }
```

Tres cosas que mirar. `@Getter` sobre la clase genera `getId()`, `getTitle()`, `getReward()`, `getRequiredRank()` y `getStatus()`; para `assignedTo` se escribe el getter a mano y Lombok lo respeta. No hay `setStatus`: las anotaciones de Lombok no rompen la regla de que el estado solo cambia con métodos del negocio. Y `@EqualsAndHashCode` solo cuenta el `id`, que es lo que pide una entidad.

Y ahora los dos puertos, cada uno en su carpeta. El de entrada, que es el caso de uso que la aplicación ofrece:

application/port/in/PublishQuestUseCase.java

```
1  package com.guildboard.application.port.in;
2
3  import com.guildboard.domain.Quest;
4  import com.guildboard.domain.Rank;
```

```
import com.guildboard.domain.Reward;

public interface PublishQuestUseCase {

    Quest publish(String title, Reward reward, Rank requiredRank);
}
```

Los tres parámetros sueltos son *provisionales*: el capítulo 4 los sustituye por un objeto de entrada propio, el comando. Y el de salida, que es lo que la aplicación necesita para guardar y leer misiones (es el `QuestRepository` del capítulo 2):

application/port/out/QuestRepository.java

```
package com.guildboard.application.port.out;

import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.util.Optional;

public interface QuestRepository {

    Optional<Quest> findById(QuestId id);

    void save(Quest quest);
}
```

Fíjate en los `import`: los dos puertos importan `domain` y nada más. Las interfaces no tienen todavía implementación ni usuario; los adaptadores llegan en el capítulo 4. Cuando tengas todo escrito, `./gradlew compileJava` debe terminar con `BUILD SUCCESSFUL`. Si falla con «cannot find symbol» sobre un método como `getId()`, falta una de las dos líneas de Lombok.

3.7 JUnit mínimo

Una prueba automática es código que ejecuta otro código y falla si el resultado no es el esperado. JUnit es el marco de pruebas habitual de Java [24]: encuentra los métodos marcados como pruebas, los ejecuta y cuenta cuáles pasan. Aquí solo necesitas lo mínimo para ejecutar la prueba de arquitectura de la sección siguiente; los capítulos posteriores lo ampliarán. `build.gradle` sigue escrito en Groovy DSL [13]: no hace falta nada más de ese lenguaje que copiar estas líneas.

☞ JUnit 6.0.3 en Gradle

La documentación de JUnit para Gradle [23] y el manual de Gradle [16] piden tres cosas. Amplía el bloque `dependencies` de `build.gradle` y añade el bloque `tasks.named` al final:

build.gradle (ampliado)

```
dependencies {
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'

    testImplementation platform('org.junit:junit-bom:6.0.3')
    testImplementation 'org.junit.jupiter:junit-jupiter'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

tasks.named('test', Test) {
    useJUnitPlatform()
}
```

- `platform('org.junit:junit-bom:6.0.3')` importa una lista de versiones (un *BOM*): fija la versión de todos los artefactos de JUnit a la vez, y por eso las otras dos líneas no llevan versión. `testImplementation` pone la biblioteca solo en el código de pruebas.
- `org.junit.jupiter:junit-jupiter` trae lo que se usa al escribir pruebas (la anotación `@Test` y las aserciones) y el motor que las ejecuta.
- `junit-platform-launcher` con `testRuntimeOnly`: el lanzador que Gradle usa para arrancar las pruebas. JUnit y Gradle recomiendan declararlo explícitamente en vez de dejar que Gradle lo busque.
- `useJUnitPlatform()` le dice a la tarea `test` que use JUnit Platform; sin esa línea Gradle no ejecuta las pruebas de JUnit 6.

Las clases de prueba viven en `src/test/java`, con los mismos paquetes que el código que prueban. La versión 6.0.3 es una elección del curso: es la que gestiona Spring Boot 4.1.1, que entra en el capítulo 4.

Una prueba es un método sin parámetros marcado con `@Test`. Esta, provisional, comprueba la regla de rangos del dominio (`src/test/java/com/guildboard/domain/RankTest.java`):

test/.../domain/RankTest.java (provisional)

```
package com.guildboard.domain;

import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertTrue;

import org.junit.jupiter.api.Test;

class RankTest {

    @Test
    void adeptCanTakeNoviceQuests() {
        assertTrue(Rank.ADEPT.canTake(Rank.NOVICE));
    }

    @Test
    void noviceCannotTakeVeteranQuests() {
```

```
17          assertFalse(Rank.NOVICE.canTake(Rank.VETERAN));
18      }
19  }
```

`assertTrue` y `assertFalse` fallan, lanzando una excepción, si la condición no es la esperada; una prueba que termina sin excepción pasa. Se ejecutan todas con:

```
$ ./gradlew test

BUILD SUCCESSFUL in 3s
```

(La duración y el resto de la salida cambian; Gradle no lista las pruebas que pasan.) Para ver un fallo, cambia `assertFalse` por `assertTrue` en la segunda prueba y vuelve a ejecutar: `BUILD FAILED`, con el nombre de la prueba y la condición que no se cumplió. Repón el código. Después **borra `RankTest.java`**: no forma parte del árbol T1 (las pruebas del dominio llegan en el capítulo 5).

Una prueba que no se ejecuta

Si olvidas `useJUnitPlatform()`, `./gradlew test` termina con `BUILD SUCCESSFUL` sin haber ejecutado nada de JUnit, y no te enteras. Si una prueba nueva no aparece, empieza por ahí.

3.8 Comprobar la regla de dependencia con ArchUnit

ArchUnit es una biblioteca que lee el código *compilado* (el bytecode de los `.class`) y comprueba reglas sobre sus dependencias, que se escriben en Java dentro de una prueba normal [66, 67]. No lee texto: si una clase de `domain` usa una de `application`, lo detecta aunque el código escriba el nombre completo y no haya `import`. Hombergs vigila la regla de dependencia de Buckpal con esta misma biblioteca [21]; sus versiones son antiguas y aquí no se copian.

☞ ArchUnit 1.5.1 en Gradle

Añade una línea más al bloque `dependencies` de `build.gradle`, junto a las de JUnit:

`build.gradle (línea añadida)`

```
1       testImplementation 'com.tngtech.archunit:archunit:1.5.1'
```

La 1.5.1 es la versión que fija el curso [65]. El artefacto es el *base*, `archunit`, que no depende de JUnit: se usa desde un `@Test` normal. Existe también un artefacto específico para JUnit 6, `archunit-junit6` [29], con anotaciones propias; el curso no lo usa porque no se ha verificado su compatibilidad con JUnit 6.0.3.

Lo que este capítulo no ha verificado

El código Java de este libro no se ha compilado ni ejecutado. Eso incluye la combinación de versiones de JUnit y ArchUnit y la API de las reglas, que se ha tomado de la guía de usuario de ArchUnit [66] y de su documentación Javadoc [64], no de una ejecución. Tampoco se ha comprobado si el lanzador `junit-platform-launcher` hace falta con estas versiones; se declara por recomendación de JUnit y Gradle. Si algo no compila en tu máquina, la guía de

ArchUnit es la referencia.

Una regla de ArchUnit se lee casi como una frase en inglés. Estas son las piezas que usa el curso:

- `ClassFileImporter` importa las clases compiladas de un paquete y sus subpaquetes (`importPackages`) y devuelve un `JavaClasses`. Si pasas la opción `DoNotIncludeTests`, las clases de prueba quedan fuera.
- `classes()` y `noClasses()` empiezan una regla. `that()` elige a qué clases se aplica (y `and()` añade otra condición); `should()` dice lo que se exige.
- `resideInAPackage` y `resideInAnyPackage` comparan con un patrón de paquete. `..` significa «cualquier número de subpaquetes»: `..domain..` es cualquier paquete que contenga un tramo `domain`.
- `dependOnClassesThat` es «usa alguna clase que…»; `onlyDependOnClassesThat` es «solo usa clases que…».
- `check(clases)` evalúa la regla y lanza una excepción, con el detalle de cada infracción, si no se cumple. Dentro de un `@Test`, eso es una prueba fallida.

La prueba del capítulo tiene dos reglas, una por capa. La de `domain` dice que nada de ahí usa las capas de fuera ni un framework; la de `application` dice que solo usa su propio paquete, `domain`, Java estándar y Lombok:

test/.../ArchitectureTest.java

```
1  package com.guildboard;
2
3  import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
4  import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;
5
6  import com.tngtech.archunit.core.domain.JavaClasses;
7  import com.tngtech.archunit.core.importer.ClassFileImporter;
8  import com.tngtech.archunit.core.importer.ImportOption;
9  import org.junit.jupiter.api.Test;
10
11 public class ArchitectureTest {
12
13     private static final String BASE_PACKAGE = "com.guildboard";
14
15     private static final JavaClasses CLASSES = new ClassFileImporter()
16         .withImportOption(new ImportOption.DoNotIncludeTests())
17         .importPackages(BASE_PACKAGE);
18
19     @Test
20     void domainDoesNotDependOnOuterLayers() {
21         noClasses().that().resideInAPackage("..domain..")
22             .should().dependOnClassesThat().resideInAnyPackage(
23                 "..application..", "..adapter..", "..config..",
24                 "org.springframework..", "jakarta..")
25             .check(CLASSES);
26     }
27
28     @Test
29     void applicationOnlyDependsOnDomain() {
30         classes().that().resideInAPackage("..application..")
31             .should().onlyDependOnClassesThat().resideInAnyPackage(
```

```
32                  "..application..", "..domain..", "java..", "lombok..")
33              .check(CLASSES);
34      }
35  }
```

La regla de `domain` enumera lo prohibido; la de `application` enumera lo permitido. `java..` deja pasar la biblioteca estándar (`Optional`, `Objects`) y `lombok..` permite Lombok, que es una dependencia de compilación. El `CLASSES` se calcula una vez y las dos reglas lo comparten.

Con el dominio y los puertos de este capítulo escritos, todo pasa:

```
$ ./gradlew test

BUILD SUCCESSFUL in 4s
```

Hay un detalle de ArchUnit que conviene saber: si una regla no encuentra *ninguna* clase a la que aplicarse (porque el paquete está mal escrito o aún no hay clases), falla en lugar de pasar en silencio. Es lo que quieres: una prueba que no vigila nada no sirve. Por eso el capítulo escribe antes el dominio mínimo.

Cuando una regla se rompe, el mensaje dice cuál, qué clase y en qué línea. Si, por ejemplo, `Quest` llamara a `QuestRepository`, saldría algo así (abreviado; el texto exacto varía):

```
FAILED
java.lang.AssertionError: Architecture Violation [Priority: MEDIUM] -
Rule 'no classes that reside in a package '..domain..' should depend
on classes that reside in any package ['..application..', ...]'
was violated (1 times):
Method <com.guildboard.domain.Quest.publish(...)> calls method
<com.guildboard.application.port.out.QuestRepository.findById(...)>
in (Quest.java:36)
```

Se lee de fuera adentro: la regla, la clase infractora y la línea del archivo. El ejercicio 3.3 te hace provocarlo y leerlo.

Qué no comprueba. ArchUnit ve dependencias entre clases, no intenciones. No sabe si un adaptador está bien en `in` o en `out` (por eso queda en la lista de la verificación de estructura), no mira las dependencias declaradas en `build.gradle` y solo comprueba lo que tú escribes en las reglas. Tampoco decide si los nombres de las carpetas son buenos. ArchUnit ofrece además atajos que describen una arquitectura entera, `onionArchitecture()` y `layeredArchitecture()` [66]; este curso no los usa y prefiere reglas explícitas, que se leen sin conocer el atajo. En los capítulos 7 y 10 la misma prueba gana reglas nuevas.

3.9 En el proyecto: hito H1, parte 1

Hito 1 (parte 1): el hexágono mínimo y su vigilante

Al terminar el ejercicio 3.5 el proyecto tiene el árbol T1 de la verificación de estructura 1, con estos archivos dentro:

Archivos de GuildBoard tras el capítulo 3

```
src/main/java/com/guildboard/
|-- domain/
|   |-- AdventurerId.java, Quest.java, QuestId.java
|   `-- QuestStatus.java, Rank.java, Reward.java
`-- application/port/
    |-- in/PublishQuestUseCase.java
    `-- out/QuestRepository.java
src/test/java/com/guildboard/ArchitectureTest.java
```

Todavía no hay Spring, ni adaptadores, ni forma de arrancar nada: el capítulo 4 añade el primer caso de uso, el primer controlador y el repositorio en memoria (H1 completo). El árbol no cambia en el capítulo 4 ni en el 5.

3.10 Ejercicios

Los ejercicios 3.3 y 3.4 trabajan sobre el proyecto: haz antes el ejercicio de proyecto 3.5, o ten escrito el código del capítulo.

Ejercicio 3.1: Clasifica quince clases del hexágono ★☆☆☆☆

Clasifica estas quince clases de GuildBoard (algunas llegarán en capítulos posteriores) en una de estas categorías: *dominio*, *puerto de entrada*, *puerto de salida*, *adaptador primario*, *adaptador secundario* u *otra cosa* (si es así, di cuál):

1. `Quest`: entidad con identidad y estado.
2. `Reward`: record con una cantidad de oro.
3. `Adventurer`: entidad con rango y experiencia.
4. `PublishQuestUseCase`: interfaz con un método `publish`.
5. `ExpireQuestsUseCase`: interfaz con un método `expireOld`.
6. `QuestRepository`: interfaz con `findById` y `save`.
7. `NotificationSender`: interfaz con un método `send`.
8. `QuestController`: clase con `@RestController` que recibe peticiones HTTP y llama a `PublishQuestUseCase`.
9. `QuestCli`: programa de consola que lee argumentos y llama a `PublishQuestUseCase`.
10. `NightlyExpiryJob`: tarea programada que cada noche llama a `ExpireQuestsUseCase`.
11. `InMemoryQuestRepository`: implementa `QuestRepository` con un mapa.
12. `JpaQuestRepository`: implementa `QuestRepository` con una base de datos.
13. `EmailNotificationSender`: implementa `NotificationSender` enviando correos.
14. `PublishQuestService`: implementa `PublishQuestUseCase` usando `QuestRepository`.
15. `BeanConfig`: clase que crea los objetos anteriores y los une.

1. Para cada clase, pregúntate: ¿la llama el exterior, o la llama la aplicación? ¿La declara o la implementa la aplicación?
2. Escribe la tabla con la categoría y una razón de una línea.
3. Para las clases de dominio y de puertos, y para `QuestController` e `InMemoryQuestRepository`, di en qué carpeta del árbol T1 (o de las previstas) vive cada una.

Está bien si tu tabla tiene quince filas, hay tres de dominio, dos puertos de entrada, dos de salida, tres adaptadores primarios, tres secundarios y dos clases de *otra cosa*, y cada razón menciona quién llama a quién.

Lectura de código ▪ solución en la pág. 61

Ejercicio 3.2: Compara tu árbol con el de Buckpal ★★☆☆☆

Este es el árbol de Buckpal de la sección 3.3 (paquetes bajo `io.reflectoring.buckpal`). Compáralo con el árbol T1 y las carpetas previstas de la verificación de estructura 1:

Buckpal

```
BuckPalApplication.java, BuckPalConfiguration.java
adapter/in/web/, adapter/out/persistence/
application/domain/model/, application/domain/service/
application/port/in/, application/port/out/
common/
```

Además, el servicio `SendMoneyService` de Buckpal lleva las anotaciones `@UseCase` y `@Transactional`; `UseCase` es una anotación del paquete `common` que es un `@Component` de Spring, y `@Transactional` es de `jakarta.transaction`.

1. Nombra **tres diferencias** entre los dos árboles y, para cada una, el motivo que da el curso y el coste que tiene.
2. ¿Agrupa Buckpal por funcionalidad (por ejemplo, con un nivel `account/`)? Responde solo con lo que muestra el árbol.
3. Aplica a `SendMoneyService`, tal como las formula el curso, las reglas de `ArchitectureTest`, sobre todo `applicationOnlyDependsOnDomain`: ¿pasaría?

Está bien si das tres diferencias reales (no inventadas), cada una con su motivo y su coste, si no afirmas que Buckpal agrupe por funcionalidad, y si en el tercer apartado distingues «no pasa la comprobación del curso» de «está mal».

Lectura de código ▪ solución en la pág. 61

Ejercicio 3.3: Una misión que consulta el repositorio ★★☆☆☆

Un compañero quiere que no se publique una misión que ya está guardada, y lo ha resuelto dentro de la entidad. Ha cambiado en `domain/Quest.java` el método `publish` y ha añadido un `import`:

domain/Quest.java (cambios)

```
import com.guildboard.application.port.out.QuestRepository;

    public void publish(QuestRepository repository) {
        if (repository.findById(id).isPresent()) {
            throw new IllegalStateException(
                "quest " + id.value() + " is already stored");
        }
        requireStatus(QuestStatus.DRAFT);
        status = QuestStatus.PUBLISHED;
    }
```

(El `import` va con los demás, arriba; el método sustituye a `publish()`. El resto de la clase no cambia.)

1. Aplica el cambio a `Quest.java` y ejecuta `./gradlew test`.
2. Lee el mensaje: ¿qué regla falla, qué clase y en qué línea? ¿Por qué falla solo esa regla y no `applicationOnlyDependsOnDomain`?
3. Arregla `Quest` para que `./gradlew test` vuelva a pasar y `publish()` recupere su forma original.
4. Explica en dos frases quién debería hacer ahora la consulta al repositorio.

Está bien si la prueba falla primero con la regla de `domain` y señala `Quest`, pasa después de tu cambio, y `Quest` no menciona nada de `application`. El código compila con la infracción incluida: es la prueba, no el compilador, la que protege la regla.

Pista: *el dominio no puede conocer a quien lo guarda; si hace falta saber algo de fuera, lo sabe otra pieza y el dominio recibe, si acaso, el resultado ya calculado.*

Depuración ▪ solución en la pág. 62

Ejercicio 3.4: Sin JDBC en application y nombres de casos de uso ★★★☆☆

Añade a `ArchitectureTest` dos pruebas más:

1. `applicationDoesNotUseJdbc`: ninguna clase de `application` usa el paquete `java.sql` (ni sus subpaquetes). La regla `applicationOnlyDependsOnDomain` no lo cubre, porque permite todo `java..`.
2. `useCaseInterfacesEndWithUseCase`: las *interfaces* de `application.port.in` tienen un nombre que acaba en `UseCase`.

Pruébalas rompiéndolas una a una. Para la primera, crea `application/TempDeadline.java` con una clase que tenga un campo `java.sql.Timestamp`. Para la segunda, crea en `application/port/in` una interfaz vacía llamada `ListQuests`. Comprueba que cada una falla con la regla que toca y borra los dos archivos después.
Está bien si con el proyecto limpio las cuatro pruebas pasan, y cada archivo de prueba hace fallar solo su regla (un `import` que no se usa en el código no cuenta: ArchUnit mira el bytecode).

Pista: *la primera regla es parecida a `domainDoesNotDependOnOuterLayers`, con `resideInAPackage("java.sql..")`; la segunda usa `classes()`, `that().resideInAPackage(...).and().areInterfaces()` y `should().haveSimpleNameEndingWith("UseCase")`.*

Práctica ▪ solución en la pág. 63

Ejercicio 3.5: Hito H1 (parte 1): el hexágono mínimo ★★★☆☆

Parte del proyecto tal como lo dejó el capítulo 2: el esqueleto de H0 más `docs/dominio.md`. Construye el árbol T1 de la verificación de estructura 1.

1. Añade a `build.gradle` Lombok, JUnit y ArchUnit, y el bloque `tasks.named('test', Test)`. El `.gitignore` del capítulo 1 ya ancla `/bin/` y `/out/`; no lo toques.
2. Crea los seis archivos de `domain` y los dos puertos, con sus paquetes.
3. Crea `src/test/java/com/guildboard/ArchitectureTest.java` con las dos pruebas de la sección 3.8.
4. Ejecuta `./gradlew build`.
5. Haz un *commit* con el mensaje `H1a: hexágono mínimo (dominio, puertos, prueba de arquitectura)`.

Está bien si: `./gradlew build` termina con `BUILD SUCCESSFUL` (compila con Lombok y ejecuta `ArchitectureTest`); `git status` no muestra nada pendiente; `git ls-files` incluye `src/main/java/com/guildboard/application/port/out/QuestRepository.java`; y, si haces que `Quest` use `QuestRepository`, `./gradlew test` falla (revierte el cambio después).

Proyecto ▪ solución en la pág. 65

3.11 Resumen

Resumen

- Los puertos son interfaces que la aplicación define; los adaptadores primarios la llaman (REST, consola, tareas), los secundarios son llamados por ella (repositorios, avisos). Cockburn lo formuló como dentro frente a fuera.
- La regla de dependencia (Martin, síntesis de hexagonal, onion y Clean): las dependencias de código solo apuntan hacia dentro. `domain` no usa nada de fuera; `application`, solo `domain`; `adapter`, `application` y `domain`; `config` lo conoce todo.
- Buckpal agrupa por capas, como el curso. Las diferencias del curso son: `domain` hermano de `application`, `config` aparte con servicios sin anotaciones, sin `common` y transacciones solo en el adaptador de persistencia. Ambos vigilan la regla con ArchUnit.
- La verificación de estructura 1 muestra el árbol T1 (`domain`, `application/port/in`, `application/port/out`, `ArchitectureTest`) y las carpetas previstas de los capítulos 4 y 5; se comprueba con cuatro puntos.
- Lombok entra con `compileOnly` y `annotationProcessor`; se usa en clases con identidad y estado y se permite en `domain` y `application` por ser dependencia de compilación.
- JUnit 6.0.3 se instala con su BOM, `junit-jupiter`, el lanzador y `useJUnitPlatform()`; `./gradlew test` ejecuta las pruebas. ArchUnit 1.5.1 lee el bytecode, y una prueba con reglas explícitas hace fallar la compilación si una capa usa lo que no debe; no sustituye a la verificación de estructura.
- El `.gitignore` del capítulo 1 ancla `/bin/` y `/out/`; sin la barra inicial, `out/` ignoraría `port/out`.

Soluciones de los ejercicios

Solución del ejercicio 3.1

enunciado en la pág. 57

Clase	Categoría	Razón
`Quest`	Dominio	Entidad con reglas propias; no conoce nada del exterior.
`Reward`	Dominio	Objeto de valor.
`Adventurer`	Dominio	Entidad con rango y experiencia.
`PublishQuestUseCase`	Puerto de entrada	Lo que la aplicación ofrece; lo usa un adaptador primario.
`ExpireQuestsUseCase`	Puerto de entrada	Igual, para la caducidad.
`QuestRepository`	Puerto de salida	Lo que la aplicación necesita para guardar; lo declara ella.
`NotificationSender`	Puerto de salida	Lo que la aplicación necesita para avisar.
`QuestController`	Adaptador primario	Recibe HTTP y llama al caso de uso: llama a la aplicación.
`QuestCli`	Adaptador primario	Una consola que llama al mismo caso de uso.
`NightlyExpiryJob`	Adaptador primario	Un temporizador que llama a la aplicación; no es una persona pero sí quien la conduce.
`InMemoryQuestRepository`	Adaptador secundario	Implementa un puerto de salida con un mapa.
`JpaQuestRepository`	Adaptador secundario	Implementa el mismo puerto con una base de datos.
`EmailNotificationSender`	Adaptador secundario	Implementa `NotificationSender`.
`PublishQuestService`	Otra cosa: servicio de aplicación	Implementa el puerto de entrada y usa uno de salida; está dentro del hexágono.
`BeanConfig`	Otra cosa: raíz de composición	Crea y une los objetos; no es primario ni secundario.

Carpetas: `Quest`, `Reward` y `Adventurer` están en `domain`; los dos casos de uso, en `application/port/in`; `QuestRepository` y `NotificationSender`, en `application/port/out`. `QuestController` está en `adapter/in/web` e `InMemoryQuestRepository` en `adapter/out/memory`, ambas carpetas previstas del capítulo 4.

Errores típicos: llamar «puerto» a un servicio o a un adaptador por llevar el nombre de un caso de uso; clasificar `NightlyExpiryJob` como secundario porque «no es un usuario» (lo que cuenta es que *llama* a la aplicación); confundir `JpaQuestRepository` con un puerto porque su nombre acaba en `Repository` (el puerto es la interfaz; la clase la implementa).

Solución del ejercicio 3.2

enunciado en la pág. 58

Tres diferencias (a elegir; hay cuatro válidas):

1. **El dominio es hermano de `application`** (curso: `domain` y `application`; Buckpal: `application/domain/model`). Motivo: sigue los círculos de Martin, con el dominio en el centro, y permitirá en el capítulo 7 dos módulos Gradle sin mover paquetes. Coste: un nivel más que explicar, y el dominio y los servicios, que en Buckpal conviven, se separan.
2. **El ensamblado vive en `config` y los servicios no llevan anotaciones** (Buckpal: `BuckPalConfiguration` en el paquete raíz y `@UseCase`/`@Transactional` en los ser-

vicios). Motivo: que `application` no dependa de Spring ni de `jakarta`. Coste: más código de composición, porque cada servicio hay que registrarlo a mano.

3. **No hay `common`** (Buckpal tiene `common` con anotaciones propias y validación). Motivo: un paquete «común» acaba siendo un cajón de sastre del que dependen todas las capas; lo que haga falta entra por `domain` o por `application`. Coste: a veces habrá que duplicar un valor pequeño.

4. **Transacciones**: Buckpal las pone en el servicio; el curso, solo en el adaptador de persistencia (capítulo 8), con la regla de que cada caso de uso modifica como máximo un agregado. Coste: la carrera entre contar y guardar del capítulo 5.

Segunda pregunta: no. El árbol muestra `adapter`, `application` y `common` directamente bajo el paquete base, sin ningún nivel intermedio por funcionalidad: Buckpal agrupa por capas, igual que el curso. No es una diferencia, aunque hubiera sido razonable esperarla.
Tercera pregunta: con las reglas del curso, `SendMoneyService` no pasaría `applicationOnlyDependsOnDomain`. Este servicio, en `application`, usa `@UseCase` (que es un `@Component` de Spring) y `@Transactional` (de `jakarta`), de modo que la capa de aplicación conoce a Spring y a `jakarta` y no depende solo del dominio. Eso no es un error de Buckpal: es otro equilibrio, con menos código de ensamblado a cambio de acoplar la aplicación al framework. La prueba del curso marcaría las dos anotaciones, y también la dependencia con `common`, porque ArchUnit mira los paquetes reales de las clases y `common` no está en la lista de paquetes permitidos de la regla.
Errores típicos: afirmar que Buckpal agrupa por funcionalidad, o inventar una diferencia que el árbol no muestra (por ejemplo, decir que el curso usa módulos Gradle: aún no); y dar diferencias sin motivo ni coste.

Solución del ejercicio 3.3

enunciado en la pág. 58

Con el cambio, `./gradlew test` falla. El mensaje (abreviado) es:

```
> Task :test FAILED

ArchitectureTest > domainDoesNotDependOnOuterLayers() FAILED
    java.lang.AssertionError: Architecture Violation [Priority: MEDIUM] -
    Rule 'no classes that reside in a package '..domain..' should depend
    on classes that reside in any package ['..application..', ...]'
    was violated (2 times):
    Method <com.guildboard.domain.Quest.publish(
        com.guildboard.application.port.out.QuestRepository)>
        has parameter of type <...QuestRepository> in (Quest.java:...)
    Method <com.guildboard.domain.Quest.publish(...)> calls method
        <...QuestRepository.findById(...)> in (Quest.java:...)
```

(La forma exacta y los números de línea cambian.) Falla la regla `domainDoesNotDependOnOuterLayers`: una clase de `domain` (`Quest`) usa una de `application` (`QuestRepository`), en el parámetro del método y en la llamada a `findById`. No falla `applicationOnlyDependsOnDomain`, porque esa regla mira lo que usa `application`, y `QuestRepository` solo usa `Quest`, `QuestId` y `Optional`, que están permitidos.
La corrección es devolver `Quest` a su forma del capítulo: se quita el `import` de `QuestRepository` y el método vuelve a ser

domain/Quest.java (método publish, restaurado)

```
    public void publish() {
        requireStatus(QuestStatus.DRAFT);
        status = QuestStatus.PUBLISHED;
    }
```

El resto de `Quest.java` es exactamente el listado de la sección 3.6. Con eso, `./gradlew test` pasa. La consulta («¿ya está guardada?») la hará el caso de uso de la capa `application`: él sí puede llamar a `QuestRepository`, preguntar si existe y, solo después, llamar a `quest.publish()`. El dominio no sabe que existe un repositorio; el que coordina es quien lo conoce, y el capítulo 4 construye ese servicio.

Errores típicos: «arreglarlo» quitando `..application..` de la lista de paquetes de la regla, que es apagar el detector en lugar de arreglar el diseño; mover `QuestRepository` a `domain` para que ya no sea de `application` (se rompe la idea de que los puertos pertenecen a la aplicación y, además, se hace lo mismo con otro nombre); o tocar solo el `import` y dejar el parámetro del método.

Solución del ejercicio 3.4 enunciado en la pág. 59

El archivo `ArchitectureTest.java` completo, con las dos pruebas del capítulo y las dos nuevas:

test/.../ArchitectureTest.java

```java
package com.guildboard;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;

import com.tngtech.archunit.core.domain.JavaClasses;
import com.tngtech.archunit.core.importer.ClassFileImporter;
import com.tngtech.archunit.core.importer.ImportOption;
import org.junit.jupiter.api.Test;

public class ArchitectureTest {

    private static final String BASE_PACKAGE = "com.guildboard";

    private static final JavaClasses CLASSES = new ClassFileImporter()
        .withImportOption(new ImportOption.DoNotIncludeTests())
        .importPackages(BASE_PACKAGE);

    @Test
    void domainDoesNotDependOnOuterLayers() {
        noClasses().that().resideInAPackage("..domain..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "..application..", "..adapter..", "..config..",
                "org.springframework..", "jakarta..")
            .check(CLASSES);
    }

    @Test
    void applicationOnlyDependsOnDomain() {
        classes().that().resideInAPackage("..application..")
            .should().onlyDependOnClassesThat().resideInAnyPackage(
                "..application..", "..domain..", "java..", "lombok..")
            .check(CLASSES);
    }

    @Test
    void applicationDoesNotUseJdbc() {
        noClasses().that().resideInAPackage("..application..")
            .should().dependOnClassesThat().resideInAPackage("java.sql..")
            .check(CLASSES);
    }

    @Test
    void useCaseInterfacesEndWithUseCase() {
        classes().that().resideInAPackage("..application.port.in..")
            .and().areInterfaces()
            .should().haveSimpleNameEndingWith("UseCase")
            .check(CLASSES);
    }
}
```

La primera regla nueva es necesaria porque `applicationOnlyDependsOnDomain` permite todo `java..`, y `java.sql` está dentro. La segunda se restringe a interfaces porque `port/in` puede acabar guardando también otras clases (por ejemplo, los comandos del capítulo 4). Para provocar la primera, el archivo temporal es

application/TempDeadline.java

```
package com.guildboard.application;

import java.sql.Timestamp;

public class TempDeadline {

    private Timestamp value;
}
```

y la prueba falla con una salida parecida a (abreviada)

```
ArchitectureTest > applicationDoesNotUseJdbc() FAILED
    Rule 'no classes that reside in a package '..application..' should
    depend on classes that reside in a package 'java.sql..'' was
    violated (1 times):
    Field <com.guildboard.application.TempDeadline.value> has type
    <java.sql.Timestamp> in (TempDeadline.java:...)
```

Para la segunda, el archivo temporal es

application/port/in/ListQuests.java

```
package com.guildboard.application.port.in;

public interface ListQuests {
}
```

y falla `useCaseInterfacesEndWithUseCase`, con un mensaje que nombra `ListQuests` y dice que su nombre no acaba en `UseCase`. Borra los dos archivos y las cuatro pruebas vuelven a pasar. Si solo añadieras el `import java.sql.Timestamp;` sin usarlo, no fallaría: un `import` que no se usa no llega al bytecode.

Errores típicos: escribir `"java.sql"` sin los dos puntos finales (solo cubriría ese paquete, no los subpaquetes); usar `noClasses()` en la segunda regla (se pediría que ninguna clase tenga ese nombre); olvidar `areInterfaces()`, con lo que cualquier clase de `port/in` debería acabar en `UseCase`; y escribir el `import` estático de `classes` o de `noClasses` de otra clase de ArchUnit.

Solución del ejercicio 3.5

enunciado en la pág. 60

Comandos (desde la carpeta `guildboard`):

```
$ mkdir -p src/main/java/com/guildboard/domain
$ mkdir -p src/main/java/com/guildboard/application/port/in
$ mkdir -p src/main/java/com/guildboard/application/port/out
$ mkdir -p src/test/java/com/guildboard
$ ./gradlew build
$ git status
$ git add .
$ git commit -m "H1a: hex^^c3^^a1gono m^^c3^^adnimo (dominio, puertos, prueba
    ↪ de arquitectura)"
```

Archivos, uno por uno. Primero el `.gitignore`, que no cambia respecto al capítulo 1 (se reproduce para que veas las reglas ancladas `/bin/` y `/out/`):

.gitignore

```
# Gradle
.gradle/
build/

# Editores e IDE (NeoVim con jdtls, IntelliJ)
.idea/
*.iml
.classpath
.project
.settings/
/bin/
/out/

# Java y sistema
*.class
*.log
.DS_Store
```

`build.gradle` completo:

build.gradle

```
plugins {
    id 'java'
}

group = 'com.guildboard'
version = '0.0.1-SNAPSHOT'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'

    testImplementation platform('org.junit:junit-bom:6.0.3')
    testImplementation 'org.junit.jupiter:junit-jupiter'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}

tasks.named('test', Test) {
    useJUnitPlatform()
}
```

La prueba de arquitectura, en `src/test/java/com/guildboard/ArchitectureTest.java` (las dos pruebas de la sección 3.8; el ejercicio 3.4 añade otras dos y cualquiera de las dos versiones sirve para el hito):

test/.../ArchitectureTest.java

```java
package com.guildboard;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;

import com.tngtech.archunit.core.domain.JavaClasses;
import com.tngtech.archunit.core.importer.ClassFileImporter;
import com.tngtech.archunit.core.importer.ImportOption;
import org.junit.jupiter.api.Test;

public class ArchitectureTest {

    private static final String BASE_PACKAGE = "com.guildboard";

    private static final JavaClasses CLASSES = new ClassFileImporter()
        .withImportOption(new ImportOption.DoNotIncludeTests())
        .importPackages(BASE_PACKAGE);

    @Test
    void domainDoesNotDependOnOuterLayers() {
        noClasses().that().resideInAPackage("..domain..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "..application..", "..adapter..", "..config..",
                "org.springframework..", "jakarta..")
            .check(CLASSES);
    }

    @Test
    void applicationOnlyDependsOnDomain() {
        classes().that().resideInAPackage("..application..")
            .should().onlyDependOnClassesThat().resideInAnyPackage(
                "..application..", "..domain..", "java..", "lombok..")
            .check(CLASSES);
    }
}
```

El dominio, en `src/main/java/com/guildboard/domain/`:

domain/QuestId.java

```java
package com.guildboard.domain;

public record QuestId(String value) { }
```

domain/AdventurerId.java

```java
package com.guildboard.domain;

public record AdventurerId(String value) { }
```

domain/Rank.java

```java
package com.guildboard.domain;

public enum Rank {
    NOVICE, ADEPT, VETERAN;

    public boolean canTake(Rank required) {
        return this.compareTo(required) >= 0;
    }
}
```

domain/Reward.java

```java
package com.guildboard.domain;

public record Reward(int gold) {

    public static final Reward ZERO = new Reward(0);

    public Reward {
        if (gold < 0) {
            throw new IllegalArgumentException(
                "gold must not be negative: " + gold);
        }
    }

    public Reward plus(Reward other) {
        return new Reward(gold + other.gold);
    }
}
```

domain/QuestStatus.java

```java
package com.guildboard.domain;

public enum QuestStatus { DRAFT, PUBLISHED, ACCEPTED, COMPLETED }
```

domain/Quest.java (1/2)

```java
package com.guildboard.domain;

import java.util.Objects;
import java.util.Optional;
import lombok.EqualsAndHashCode;
import lombok.Getter;

@Getter
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
public final class Quest {

    @EqualsAndHashCode.Include
    private final QuestId id;
    private final String title;
    private final Reward reward;
    private final Rank requiredRank;
    private QuestStatus status;
    private AdventurerId assignedTo;        // null hasta aceptar

    private Quest(QuestId id, String title, Reward reward,
                  Rank requiredRank) {
        this.id = id;
        this.title = title;
        this.reward = reward;
        this.requiredRank = requiredRank;
        this.status = QuestStatus.DRAFT;
    }

    public static Quest draft(QuestId id, String title, Reward reward,
                              Rank requiredRank) {
        Objects.requireNonNull(id, "id");
        Objects.requireNonNull(reward, "reward");
        Objects.requireNonNull(requiredRank, "requiredRank");
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("title must not be blank");
        }
        return new Quest(id, title.strip(), reward, requiredRank);
    }
```

Continuación del listado anterior:

domain/Quest.java (2/2)

```
    public void publish() {
        requireStatus(QuestStatus.DRAFT);
        status = QuestStatus.PUBLISHED;
    }

    public void accept(AdventurerId adventurerId) {
        requireStatus(QuestStatus.PUBLISHED);
        assignedTo = Objects.requireNonNull(adventurerId, "adventurerId");
        status = QuestStatus.ACCEPTED;
    }

    public void complete() {
        requireStatus(QuestStatus.ACCEPTED);
        status = QuestStatus.COMPLETED;
    }

    // Escrito a mano: Lombok no genera un getter si ya existe uno con
    // ese nombre, y as^^c3^^ad se devuelve un Optional en lugar de null.
    public Optional<AdventurerId> getAssignedTo() {
        return Optional.ofNullable(assignedTo);
    }

    private void requireStatus(QuestStatus required) {
        if (status != required) {
            throw new IllegalStateException("quest " + id.value()
                + " is " + status + ", expected " + required);
        }
    }
}
```

Los puertos:

application/port/in/PublishQuestUseCase.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.Quest;
import com.guildboard.domain.Rank;
import com.guildboard.domain.Reward;

public interface PublishQuestUseCase {

    Quest publish(String title, Reward reward, Rank requiredRank);
}
```

application/port/out/QuestRepository.java

```java
package com.guildboard.application.port.out;

import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.util.Optional;

public interface QuestRepository {

    Optional<Quest> findById(QuestId id);

    void save(Quest quest);
}
```

El proyecto queda en el árbol T1: `domain` con seis archivos, `application/port/in` y `application/port/out` con uno cada uno, y `ArchitectureTest` en `src/test/java`. `./gradlew build` compila con Lombok, ejecuta `ArchitectureTest` (a través de `test`, que `check` incluye) y termina con `BUILD SUCCESSFUL`. Las clases de los puertos solo usan `domain`, y las de `domain` solo usan Java y Lombok, por eso las dos reglas pasan.

Errores típicos: un `package` que no coincide con la carpeta (Gradle compila, pero otros archivos no encuentran la clase); olvidar `annotationProcessor` (falla con «cannot find symbol» en `getId()`); olvidar `useJUnitPlatform()` (no se ejecuta ninguna prueba y nadie avisa); un paquete base mal escrito en `BASE_PACKAGE` (la selección queda vacía y ArchUnit falla); y reescribir el `.gitignore` con `out/` sin anclar: `git status` no mostraría `QuestRepository.java` (Git lo ignora por estar en una carpeta `out`; compruébalo con `git check-ignore -v`).

CAPÍTULO 4

Casos de uso y adaptadores: el primer endpoint

Objetivos del capítulo

- Escribir un caso de uso con su comando y un servicio de aplicación que no conoce Spring.
- Repasar el contenedor de beans de Spring y montar la raíz de composición fuera del núcleo.
- Añadir Spring Boot 4.1.1 al proyecto con Gradle y configurarlo con `application.yml`.
- Conectar un adaptador web y un repositorio en memoria, arrancar GuildBoard y publicar una misión con `curl` (hito H1, parte 2: H1 completo).

En el capítulo anterior construiste el centro del hexágono: un dominio mínimo, un puerto de entrada y un puerto de salida, y una tarea que vigila la regla de dependencia. Todo eso es correcto y no hace nada: no hay quien llame al puerto de entrada ni quien implemente el de salida. Este capítulo pone las tres piezas que faltan para que GuildBoard *arranque*: el servicio que implementa el caso de uso, un adaptador primario (un controlador REST) y un adaptador secundario (un repositorio en memoria). Y una cuarta, que es la que más cuesta ver la primera vez: la pieza que los une sin que el núcleo sepa que Spring existe.

Es el primer capítulo con Spring, así que incluye un repaso mínimo del contenedor de beans y la instalación del plugin de Spring Boot para Gradle. El árbol de carpetas **no cambia**: las carpetas que se crean hoy son las que figuran en el árbol de la verificación de estructura 1 del capítulo 3, y la sección 4.8 lo comprueba.

4.1 El caso de uso como frontera de entrada

Un *caso de uso* es una operación que la aplicación ofrece, expresada con el vocabulario del negocio: «publicar una misión», «aceptar una misión». Martin lo sitúa en la capa de reglas de negocio *específicas de la aplicación*, distintas de las reglas de la entidad, que valen en cualquier aplicación (capítulo 20, «Business Rules», de [28]). Hombergs lo traduce a Java con Spring con una interfaz por caso de uso, una clase que la implementa y un objeto de entrada propio [19]. Eso es lo que vas a escribir.

Tres piezas, todas dentro del hexágono:

- El **puerto de entrada**: la interfaz (`PublishQuestUseCase`). Ya existe desde el capítulo anterior.
- El **comando**: el objeto que lleva los datos de entrada del caso de uso. Es un *record* sin lógica: lo que el exterior pide.
- El **servicio de aplicación**: la clase que implementa el puerto y *orquesta*: crea la entidad, le pide que cambie de estado, la guarda usando el puerto de salida.

4.1.1 El comando

El puerto del capítulo 3 recibía tres parámetros sueltos: `publish(String title, Reward reward, Rank requiredRank)`. Funciona, pero cada dato nuevo cambia la firma de la interfaz y todos sus usuarios, y es fácil cambiar dos parámetros del mismo tipo de sitio. Un comando agrupa los datos en un *record* con nombre propio. Créalo en la carpeta de los puertos de entrada, junto al caso de uso al que pertenece (no hace falta una carpeta nueva):

application/port/in/PublishQuestCommand.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.Rank;
import com.guildboard.domain.Reward;

public record PublishQuestCommand(String title, Reward reward,
                                  Rank requiredRank) {
}
```

Dos decisiones. El comando usa los objetos de valor del dominio (`Reward`, `Rank`) y no `int` ni `String`: el exterior, el controlador, ya los habrá construido, y así un oro negativo ni siquiera llega al caso de uso. Y no valida nada por sí mismo: las reglas ya viven en `Quest.draft`, que lanza una excepción si el título está en blanco. Duplicarlas en el comando sería repetir la regla en dos sitios.

Actualiza el puerto para que reciba el comando (la versión del capítulo 3 era provisional, y esta ya no cambia la carpeta ni el nombre del archivo):

application/port/in/PublishQuestUseCase.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.Quest;

public interface PublishQuestUseCase {

    Quest publish(PublishQuestCommand command);
}
```

4.1.2 El servicio de aplicación

El servicio implementa el puerto. Va en `application/service`, una de las carpetas previstas:

application/service/PublishQuestService.java

```
package com.guildboard.application.service;

import com.guildboard.application.port.in.PublishQuestCommand;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.util.UUID;
import lombok.RequiredArgsConstructor;
```

```
10
11 @RequiredArgsConstructor
12 public class PublishQuestService implements PublishQuestUseCase {
13
14     private final QuestRepository quests;
15
16     @Override
17     public Quest publish(PublishQuestCommand command) {
18         Quest quest = Quest.draft(
19             new QuestId(UUID.randomUUID().toString()),
20             command.title(), command.reward(), command.requiredRank());
21         quest.publish();
22         quests.save(quest);
23         return quest;
24     }
25 }
```

Léelo como una receta de tres verbos: *crear* la misión en borrador, *publicarla* y *guardarla*. Todas las reglas están en `Quest`: que el título no esté en blanco, que `publish` solo valga desde el borrador. El servicio no decide nada de negocio; si lo hiciera, estarías de vuelta en el dominio anémico del capítulo 2. Tres detalles:

- `@RequiredArgsConstructor` (Lombok, capítulo 3) genera un constructor público con un parámetro por cada campo `final`; aquí, uno solo, el `QuestRepository`. Es exactamente el constructor que escribirías a mano.
- El servicio recibe el *puerto* `QuestRepository`, nunca una implementación. No sabe si detrás hay un mapa o una base de datos.
- El identificador se genera aquí con `UUID`, que es de la biblioteca estándar de Java. Podría ser un puerto de salida más (un «generador de ids»); para una sola llamada sería sobrediseño, y el capítulo 5 lo dejará como está.

Comprueba que el núcleo sigue compilando y limpio. Todavía no hay Spring en el proyecto, y no hace falta:

```
$ ./gradlew test
```

Debe terminar con `BUILD SUCCESSFUL`: compila y pasa `ArchitectureTest`, la prueba de arquitectura del capítulo 3. El servicio importa `lombok`, `java.util` y las demás capas interiores; ni una línea de `org.springframework`.

4.2 Repaso mínimo de Spring: el contenedor de beans

Partes de unas bases de Spring, y todo lo que sigue descansa en tres ideas. Ninguno de los ejemplos de esta sección entra en el proyecto todavía: no puedes ejecutarlos hasta la sección siguiente.

Inversión de control y bean. En un programa sin contenedor, la clase que necesita un colaborador lo construye con `new`. Con Spring, la construcción pasa a otro: el *contenedor* (el `ApplicationContext`) crea objetos y se los entrega a quien los necesita. A cada objeto que el contenedor gestiona se le llama *bean*. Es el principio de inversión de dependencias del capítulo 1 llevado a la construcción de objetos.

Inyección por constructor. Una clase declara lo que necesita como parámetros de su constructor, y el contenedor busca, para cada parámetro, un bean de ese tipo. Es la forma que usa todo este libro, por tres motivos: los campos son `final`, el objeto nunca existe a medias, y puedes construirlo con `new` en una prueba, sin Spring. Cuando una clase tiene un solo constructor, Spring lo usa sin que se lo indiques con `@Autowired`.

Cómo se registra un bean. Hay dos caminos:

- **Una clase de configuración**: una clase con `@Configuration` cuyos métodos con `@Bean` dicen cómo construir cada bean. El método *es* la receta: su tipo de retorno es el tipo del bean, su nombre es el nombre del bean y sus parámetros son las dependencias que el contenedor inyecta.
- **El escaneo**: el contenedor recorre un paquete y sus subpaquetes y registra por sí mismo las clases marcadas con anotaciones como `@Component` o `@RestController`.

Un ejemplo ilustrativo (la clase `Deadlines` es inventada; el reloj reaparece en el capítulo 6):

Ejemplo ilustrativo (no entra en el proyecto)

```
package com.guildboard.demo;

import java.time.Clock;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class DemoConfig {

    @Bean
    public Clock clock() {
        return Clock.systemUTC();
    }

    @Bean
    public Deadlines deadlines(Clock clock) {
        return new Deadlines(clock);
    }
}
```

El contenedor llama primero a `clock()`, guarda el resultado y lo pasa al parámetro de `deadlines(Clock)`. Por defecto cada bean es un *singleton*: hay una sola instancia por contenedor, que comparten todos los que la piden.

La anotación de arranque. Una aplicación Spring Boot tiene una clase con `@SpringBootApplication` y un método `main` que llama a `SpringApplication.run`. Esa anotación junta tres cosas: declara la clase como configuración, activa la autoconfiguración (Boot adivina qué configurar según las dependencias que hay en el proyecto) y activa el escaneo *del paquete de esa clase y de todos los que cuelgan de él* [52]. De ahí una regla práctica: la clase principal va en el paquete raíz (`com.guildboard`), o el escaneo no alcanzará a `adapter` ni a `config`.

Una tarea puntual al arrancar. Spring Boot tiene la interfaz `CommandLineRunner`, con un método `void run(String... args)`. Un bean que la implementa ejecuta su método una vez, cuando el contexto está listo y justo antes de que `SpringApplication.run` devuelva el control. Sirve para tareas de arranque. La usarás solo en el ejercicio 4.4, como segundo adaptador de entrada.

Dos errores clásicos del contenedor

Un método `@Bean` dentro de una clase sin `@Configuration` (ni `@Component`) no se lee nunca: el bean «no existe» aunque lo veas escrito. Y dos beans del mismo tipo hacen fallar el arranque, porque Spring no sabe cuál inyectar (salvo que el nombre del parámetro coincida con el de uno de los beans: entonces desempata por el nombre). El ejercicio 4.3 te pone delante de ambos mensajes.

4.3 Añadir Spring Boot al proyecto

Hasta ahora el proyecto solo tenía Lombok. Spring Boot se añade con dos piezas de Gradle: un *plugin* (que añade tareas como `bootRun` y gestiona versiones) y una *dependencia* (el starter web, que trae el servidor y el soporte REST).

4.3.1 Initializr: de dónde salen los nombres

Escribir a mano un nombre de dependencia de Boot 4 sin comprobarlo es una buena forma de perder media hora, porque Boot 4 reorganizó sus starters [50]. La herramienta de referencia es Spring Initializr (`start.spring.io`), el generador oficial de proyectos, cuyos metadatos listan los identificadores de las dependencias [62]. Para escribir este capítulo se generó con Initializr un proyecto de prueba (Gradle con `build.gradle` en Groovy DSL, Java 21, Boot 4.1.1, y las dependencias «Spring Web» y «Lombok», el 8 de octubre de 2026) y se copiaron los nombres que produce. Tú no necesitas generar nada: ya tienes un proyecto. Pero mira qué le pone Initializr a su `build.gradle`: un plugin `org.springframework.boot` versión 4.1.1, un plugin `io.spring.dependency-management` versión 1.1.7 y la dependencia `spring-boot-starter-webmvc`.

`spring-boot-starter-web` está en desuso

Casi todos los tutoriales dicen `spring-boot-starter-web`. En Boot 4 sigue existiendo, pero su propia descripción en Maven Central avisa de que está en desuso en favor de `spring-boot-starter-webmvc`. En este libro usamos el segundo nombre.

☞ Spring Boot 4.1.1 con Gradle

Edita `build.gradle`. Cuatro cambios sobre el del capítulo 3: dos líneas nuevas en `plugins`, una nueva en `dependencies` y una que se quita (la versión de JUnit, que ahora pone Boot). El resto (grupo, toolchain de Java 21, Lombok, ArchUnit y la configuración de `test`) se queda como estaba.

build.gradle

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.1'
    id 'io.spring.dependency-management' version '1.1.7'
}

group = 'com.guildboard'
version = '0.0.1-SNAPSHOT'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'

    testImplementation 'org.junit.jupiter:junit-jupiter'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}

tasks.named('test', Test) {
    useJUnitPlatform()
}
```

Fíjate en lo que falta: ya no hay `platform('org.junit:junit-bom:6.0.3')`. El BOM de Boot 4.1.1 gestiona también JUnit, y su versión es la 6.0.3, la misma que fijabas a mano [47]. Por eso `junit-jupiter` y `junit-platform-launcher` se escriben ya sin versión y sin BOM propio. ArchUnit no está en esa lista, así que su versión (`1.5.1`) sigue explícita.
Qué hace cada pieza [51]:

- `org.springframework.boot`: añade las tareas `bootRun` (arranca la aplicación) y `bootJar` (la empaqueta en un jar ejecutable) y compila con la opción `-parameters`, de modo que Spring puede leer el nombre de los parámetros de tus métodos.
- `io.spring.dependency-management`: importa la lista de versiones compatibles de Boot. Por eso el starter se escribe *sin versión*: la versión la pone el plugin.
- `spring-boot-starter-webmvc`: Spring MVC (el soporte de controladores REST), el servidor Tomcat embebido y la conversión de objetos a JSON. En Boot 4 esa conversión usa Jackson 3, que Spring configura por ti: no importarás nada de Jackson.

Una sutileza con Lombok: la lista de versiones de Boot 4.1.1 incluye su propia versión de Lombok (la 1.18.46), pero una versión escrita en la propia dependencia tiene prioridad sobre la lista, según la documentación del plugin de gestión de dependencias. Por eso las líneas del capítulo 3, con la 1.18.48, siguen mandando.

Comprueba la instalación. Todavía no hay clase principal, así que `build` y `bootRun` fallarían (el empaquetado necesita saber cuál es); compila y pasa las pruebas, y lista las tareas:

```
$ ./gradlew test
$ ./gradlew tasks --group application
```

La primera debe terminar con `BUILD SUCCESSFUL` (la primera vez descarga Spring, y tarda). `ArchitectureTest` sigue pasando: el núcleo no ha cambiado. Ahora, además, puede fallar de verdad si un `import` de Spring se cuela en `domain` o `application`, porque Spring ya está en el classpath. La segunda debe listar `bootRun`. Si Gradle dice que no encuentra el plugin, revisa la comilla y la versión: `'4.1.1'`.

4.3.2 Configuración con `application.yml`

Spring Boot lee al arrancar, si existe, el archivo `src/main/resources/application.yml` y vuelca su contenido en el entorno de la aplicación [52]. YAML anida con sangrías de espacios (nunca tabuladores): cada nivel de sangría es un punto en el nombre de la propiedad. Crea la carpeta y el archivo:

`src/main/resources/application.yml`

```yaml
spring:
  application:
    name: guildboard

server:
  port: 8080
```

Se lee como las propiedades `spring.application.name` y `server.port`. La segunda es del propio Boot (el puerto del servidor, 8080 si no dices nada); la escribimos para que tengas a la vista dónde se cambia. La primera pone nombre a la aplicación y volverá a ser útil cuando haya varios servicios.

Para leer un valor de configuración en tu código está `@Value`. Se pone en un parámetro del constructor y lleva entre llaves precedidas de `$` el nombre de la propiedad; después de dos puntos puedes dar un valor por defecto:

`Ejemplo ilustrativo (no entra en el proyecto)`

```java
package com.guildboard.demo;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class StartupInfo {

    private final String appName;
    private final int port;

    public StartupInfo(
            @Value("${spring.application.name}") String appName,
```

```
            @Value("${server.port:8080}") int port) {
        this.appName = appName;
        this.port = port;
    }
}
```

Spring convierte el texto al tipo del parámetro (aquí, a `int`). Si la propiedad no existe y no hay valor por defecto, la aplicación no arranca, con un mensaje que empieza por «Could not resolve placeholder». Una regla de diseño: `@Value` es Spring, así que solo se usa en el borde (adaptadores y `config`). Un valor externo llega al núcleo como un argumento normal, por constructor o por parámetro, nunca porque `application` lea la configuración. Lo practicarás en el ejercicio 4.4.

4.4 El adaptador web: controlador y DTO

El **adaptador primario** web traduce entre HTTP y el caso de uso. Recibe una petición, la convierte en un comando, llama al puerto de entrada y convierte el resultado en una respuesta. Nada más. Martin lo llama un *objeto humilde* (capítulo 23, «Presenters and Humble Objects» [28]): la parte difícil de probar se deja tan fina que casi no tiene lógica, y la lógica vive en un sitio fácil de probar, el núcleo.

Un *DTO* (*data transfer object*) es un objeto cuyo único trabajo es llevar datos a través de un límite. Aquí hay dos, ambos *records*, que viven junto al controlador porque son el formato de la API, no del dominio:

adapter/in/web/PublishQuestRequest.java

```
package com.guildboard.adapter.in.web;

import com.guildboard.domain.Rank;

public record PublishQuestRequest(String title, int reward,
                                  Rank requiredRank) {
}
```

adapter/in/web/QuestResponse.java

```
package com.guildboard.adapter.in.web;

import com.guildboard.domain.Quest;
import com.guildboard.domain.Rank;
import com.guildboard.domain.QuestStatus;

public record QuestResponse(String id, String title, int reward,
                            Rank requiredRank, QuestStatus status) {

    public static QuestResponse from(Quest quest) {
        return new QuestResponse(
            quest.getId().value(),
            quest.getTitle(),
            quest.getReward().gold(),
            quest.getRequiredRank(),
            quest.getStatus());
    }
```

```
}
```

¿Por qué no devolver el `Quest` directamente? Con `QuestResponse` la API expone exactamente los campos que decides (por ejemplo, el `id` sale como texto y no como `{"value": "abc"}`), y el día que cambie el dominio, el formato de la API no cambia solo. Es el consejo de Martin de que por los límites crucen estructuras de datos simples. Hoy es un atajo: la conversión está dentro del propio record (`from`) y los DTO usan los enums del dominio. En el capítulo 6 se separan en sus propios paquetes, con mapeadores a mano, y se explica con calma por qué es necesario.

Ahora el controlador. Spring MVC usa estas anotaciones:

- `@RestController`: la clase atiende peticiones HTTP, y lo que devuelven sus métodos se escribe en la respuesta (como JSON), no se interpreta como el nombre de una vista. El escaneo la registra sola.
- `@RequestMapping("/quests")` en la clase: prefijo de todas las rutas.
- `@PostMapping`: el método atiende `POST` a esa ruta. Su hermana `@GetMapping("/{id}")` atiende `GET` y, con `@PathVariable` en un parámetro, recibe el trozo de la ruta que va entre llaves.
- `@RequestBody`: Spring convierte el JSON del cuerpo en el parámetro, que aquí es un record.
- `ResponseEntity`: respuesta con estado, cabeceras y cuerpo. Para un alta se devuelve `201 Created` con una cabecera `Location` con la dirección del recurso nuevo.

adapter/in/web/QuestController.java

```
package com.guildboard.adapter.in.web;

import com.guildboard.application.port.in.PublishQuestCommand;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.domain.Quest;
import com.guildboard.domain.Reward;
import java.net.URI;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/quests")
@RequiredArgsConstructor
public class QuestController {

    private final PublishQuestUseCase publishQuest;

    @PostMapping
    public ResponseEntity<QuestResponse> publish(
            @RequestBody PublishQuestRequest request) {
        PublishQuestCommand command = new PublishQuestCommand(
            request.title(), new Reward(request.reward()),
            request.requiredRank());
        Quest quest = publishQuest.publish(command);
```

```
        return ResponseEntity
            .created(URI.create("/quests/" + quest.getId().value()))
            .body(QuestResponse.from(quest));
    }
}
```

Fíjate en qué recibe el controlador: el *puerto* `PublishQuestUseCase`, no el servicio. El controlador no sabe que existe una clase llamada `PublishQuestService`. Y mira de qué se compone: tres líneas de traducción (de entrada, la llamada, de salida). Ninguna regla de negocio.

¿Y si los datos son inválidos?

Por ahora, mal. Un título en blanco o una recompensa negativa hacen que el dominio lance `IllegalArgumentException`, que Spring convierte en un `500` genérico. Un `requiredRank` que no sea ninguno de los tres rangos sí da un `400`, porque falla la conversión del JSON antes de llegar a tu código. El capítulo 6 traduce los errores del dominio a respuestas HTTP con sentido. Hoy el objetivo es el camino feliz.

4.5 El adaptador en memoria

El **adaptador secundario** implementa el puerto de salida. Para el primer hito basta un mapa: nada de bases de datos. Va en `adapter/out/memory`.

adapter/out/memory/InMemoryQuestRepository.java

```
package com.guildboard.adapter.out.memory;

import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

public class InMemoryQuestRepository implements QuestRepository {

    private final Map<QuestId, Quest> quests = new ConcurrentHashMap<>();

    @Override
    public Optional<Quest> findById(QuestId id) {
        return Optional.ofNullable(quests.get(id));
    }

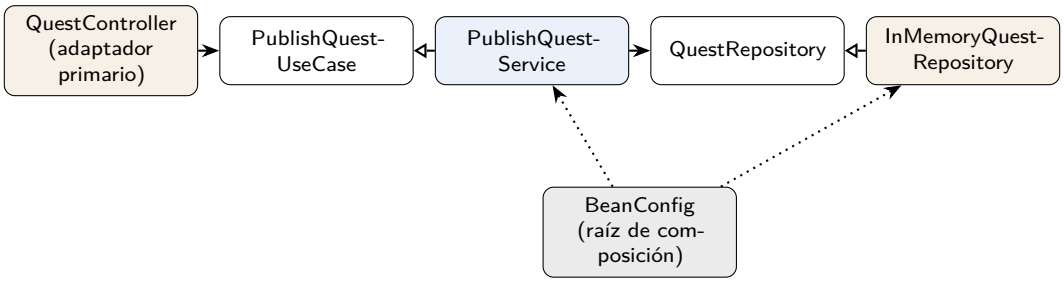
    @Override
    public void save(Quest quest) {
        quests.put(quest.getId(), quest);
    }
}
```

Tres notas. `QuestId` es un *record*, así que ya tiene `equals` y `hashCode` por valor y sirve de clave del mapa. Se usa `ConcurrentHashMap` porque Tomcat atiende cada petición en un hilo y dos peticiones simultáneas tocarían el mapa a la vez. Y una limitación que conviene tener presente: el mapa guarda la *misma instancia* de `Quest` que le dieron, así que quien la modifique después

modifica lo «guardado». Con una base de datos no pasa (cada lectura crea un objeto nuevo). El capítulo 8 cambia este adaptador por uno de PostgreSQL sin tocar una línea de `application` ni de `domain`, y esa es la prueba de que el puerto está bien puesto. Los datos viven solo mientras vive la aplicación.

4.6 La raíz de composición

Hay un problema pendiente. El controlador pide un `PublishQuestUseCase`, el servicio pide un `QuestRepository`, y alguien tiene que escribir los `new` que los unen. A esa pieza se la llama *raíz de composición*: el único lugar que conoce todas las clases concretas y las ensambla. Martin la describe con el *Main component* (capítulo 26, «The Main Component» [28]), el componente más sucio y más externo, del que todo lo demás ignora que existe. En GuildBoard son dos clases, ambas fuera del núcleo.



Flecha continua: «usa». Discontinua: «implementa». De puntos: «crea con `new`». El controlador lo crea el escaneo de Spring, no `BeanConfig`. Ninguna clase del núcleo (las dos del centro y el puerto de salida) apunta a un adaptador ni a la raíz de composición.

La configuración es una clase con `@Configuration` en `config`:

config/BeanConfig.java

```
1  package com.guildboard.config;
2
3  import com.guildboard.adapter.out.memory.InMemoryQuestRepository;
4  import com.guildboard.application.port.in.PublishQuestUseCase;
5  import com.guildboard.application.port.out.QuestRepository;
6  import com.guildboard.application.service.PublishQuestService;
7  import org.springframework.context.annotation.Bean;
8  import org.springframework.context.annotation.Configuration;
9
10 @Configuration
11 public class BeanConfig {
12
13     @Bean
14     public QuestRepository questRepository() {
15         return new InMemoryQuestRepository();
16     }
17
18     @Bean
19     public PublishQuestUseCase publishQuestUseCase(
20             QuestRepository questRepository) {
21         return new PublishQuestService(questRepository);
22     }
23 }
```

Los métodos devuelven la *interfaz* (`QuestRepository`), no la clase concreta: el resto de la aplicación pide el puerto y no sabe nada más. Es también el sitio donde se cambiará el adaptador. En el capítulo 8, cambiar de memoria a PostgreSQL será cambiar una línea de esta clase.

Y la clase principal, en el paquete raíz para que el escaneo llegue a todo:

GuildBoardApplication.java

```java
package com.guildboard;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class GuildBoardApplication {

    public static void main(String[] args) {
        SpringApplication.run(GuildBoardApplication.class, args);
    }
}
```

4.6.1 Por qué el núcleo no lleva `@Service`

Seguramente esperabas ver `@Service` sobre `PublishQuestService`, que es lo habitual en Spring, y es lo que hace Buckpal con su anotación `@UseCase` (ejercicio 3.2). El curso lo evita por tres motivos:

1. **La regla de dependencia.** La anotación `@Service` es de Spring (su paquete es `stereotype`, dentro de `org.springframework`). Ponerla en `application` haría que la capa interior importara el framework, y `ArchitectureTest` fallaría. Pruébalo en un momento: añade `@Service` y su `import` a `PublishQuestService`, ejecuta `./gradlew test` y quítalo después.
2. **Se puede construir sin Spring.** Con un `new` y el repositorio en memoria como argumento basta, en una prueba o en un programa de consola. Una clase con anotaciones de Spring sigue pudiéndose construir así; lo que cambia es que su código ya *depende* de Spring para compilar.
3. **El ensamblado está en un solo sitio.** Quien quiera saber qué implementación se usa lee `BeanConfig`, no busca anotaciones por todas las carpetas.

El coste es real: cada servicio nuevo exige un método `@Bean`. Si lo olvidas, la aplicación no arranca (lo verás en el ejercicio 4.3). El curso lo acepta: prefiere un error ruidoso al arrancar a un núcleo acoplado.

Una decisión de reparto: el controlador sí lleva anotación de Spring (`@RestController`), porque es un adaptador, y los adaptadores *son* la parte que habla con el framework. El repositorio en memoria no lleva ninguna y se crea en `BeanConfig`, así su cambio por otro adaptador queda visible en una sola clase.

4.7 Ejecutar y probar con curl

Con todo escrito, el proyecto ya empaqueta. Primero, que la construcción completa pase (compila, ejecuta las pruebas, incluida `ArchitectureTest`, y empaqueta el jar):

```
$ ./gradlew build
```

Después, arranca la aplicación con la tarea del plugin:

```
$ ./gradlew bootRun

  :: Spring Boot ::                (v4.1.1)

... Starting GuildBoardApplication using Java 21 ...
... Tomcat initialized with port 8080 (http)
... Tomcat started on port 8080 (http) with context path '/'
... Started GuildBoardApplication in 1.6 seconds (process running for 1.9)
<==========---> 80% EXECUTING [12s]
```

(Salida abreviada; los tiempos cambian.) Gradle se queda en «EXECUTING» mientras la aplicación corre: es lo normal, no se ha colgado. Se detiene con `Ctrl+C`. Si ves «Port 8080 was already in use», otra cosa ocupa el puerto: arranca en otro y usa ese puerto en los comandos de abajo.

```
$ ./gradlew bootRun --args='--server.port=9090'
```

La opción `--args` de Gradle pasa argumentos a la aplicación [51], y una propiedad dada en la línea de comandos gana a la de `application.yml` [52].

Para probar la API se usa `curl`, el cliente HTTP de línea de comandos. Ya debería estar instalado: comprueba que responde con `curl -V`. En otra terminal, publica una misión. Hay que decir que el cuerpo es JSON, o Spring responde `415`:

```
$ curl -i -X POST http://localhost:8080/quests \
    -H "Content-Type: application/json" \
    -d '{"title":"Limpiar el s^^c3^^b3tano","reward":150,"requiredRank":"NOVICE"}
    ↪ '
HTTP/1.1 201
Location: /quests/6b8f0c2e-5a1d-4c7e-9a3b-2d1e7f4a9c10
Content-Type: application/json

{"id":"6b8f0c2e-5a1d-4c7e-9a3b-2d1e7f4a9c10","title":"Limpiar el s^^c3^^b3tano","
    ↪ reward":150,"requiredRank":"NOVICE","status":"PUBLISHED"}
```

(Salida abreviada; el `id` será otro.) La opción `-i` imprime las cabeceras; `-X POST` fija el método; `-H` añade una cabecera; `-d` pone el cuerpo. Mira lo que ha pasado: `201`, una cabecera `Location` y una misión en estado `PUBLISHED`, que es el estado final de la operación aunque por dentro haya pasado por el borrador. Si has hecho el ejercicio 4.1, puedes recuperarla con el `id` de la respuesta: `curl -i http://localhost:8080/quests/<id>`.

Resumen de lo que debes ver con distintas peticiones:

Petición	Respuesta
`POST` correcto	`201` con `Location` y el cuerpo
`POST` sin la cabecera de JSON	`415` (tipo no soportado)
`POST` con un rango que no existe	`400` (JSON no convertible)
`POST` con título en blanco	`500` (error del dominio sin traducir; capítulo 6)
`GET` de un `id` que no existe (ejercicio 4.1)	`404`

Y recuerda que el repositorio es un mapa: al parar y volver a arrancar, las misiones han desaparecido.

4.8 Estructura: sin verificación propia

Este capítulo **no cambia la estructura** del proyecto. En el árbol de la verificación de estructura 1 (capítulo 3) figuran el árbol T1 y las carpetas *previstas* de los capítulos 4 y 5. Hoy se crean las cuatro carpetas y los dos archivos previstos, y se rellena `port/in`; no aparece ninguna carpeta que no estuviera ya en esa lista, y por eso este capítulo no tiene verificación de estructura nueva. Lo nuevo, marcado con `[N]`:

Árbol tras el capítulo 4 (sin cambios de estructura)

```
guildboard/
|-- build.gradle                    [M] plugins de Spring Boot, starter web
|-- docs/dominio.md
`-- src/
    |-- test/java/com/guildboard/ArchitectureTest.java   (sin cambios)
    `-- main/
        |-- resources/
        |   `-- application.yml     [N] (prevista)
        `-- java/com/guildboard/
            |-- GuildBoardApplication.java   [N] (prevista)
            |-- domain/             (sin cambios)
            |-- application/
            |   |-- port/in/        [M] PublishQuestCommand [N],
    ↪ PublishQuestUseCase
            |   |-- port/out/       (sin cambios)
            |   `-- service/        [N] PublishQuestService (prevista)
            |-- adapter/
            |   |-- in/web/         [N] controlador y DTO (prevista)
            |   `-- out/memory/     [N] repositorio en memoria (prevista)
            `-- config/             [N] BeanConfig (prevista)
```

(Se omiten `.git`, `.gitignore` y el wrapper. La carpeta `src/test` existe desde el capítulo 3 por la prueba de arquitectura, que no cambia.) El comando nuevo, `PublishQuestCommand`, vive en `port/in`: es parte del puerto de entrada, no justifica una carpeta. Puedes comprobarlo con Git: `git status` debe mostrar archivos nuevos solo dentro de esas carpetas, y `build.gradle`, `PublishQuestUseCase.java` modificados.

4.9 En el proyecto: hito H1 completo

Hito 1 (parte 2): GuildBoard arranca y publica una misión

Al terminar el ejercicio 4.5 el hito H1 está completo: un monolito hexagonal mínimo, con persistencia en memoria, que arranca y atiende `POST /quests` y `GET /quests/{id}`. El proyecto queda así:

Archivos de GuildBoard tras el capítulo 4

```
build.gradle
src/test/java/com/guildboard/ArchitectureTest.java
src/main/resources/application.yml
src/main/java/com/guildboard/
|-- GuildBoardApplication.java
|-- domain/            (los seis archivos del cap^^c3^^adtulo 3)
|-- application/
|   |-- port/in/       PublishQuestCommand, PublishQuestUseCase,
|   |                  GetQuestUseCase
|   |-- port/out/      QuestRepository
|   `-- service/       PublishQuestService, GetQuestService
|-- adapter/
|   |-- in/web/        QuestController, PublishQuestRequest, QuestResponse
|   `-- out/memory/    InMemoryQuestRepository
`-- config/            BeanConfig
```

El dominio y los dos puertos del capítulo 3 siguen igual salvo el comando. El capítulo 5 enriquece el dominio (reglas de rango y límite de misiones) sin tocar la estructura. Fíjate en el reparto de Spring: solo `GuildBoardApplication`, `BeanConfig` y `QuestController` importan `org.springframework`.

4.10 Ejercicios

Los ejercicios parten del código del capítulo. Hay dos formas de recorrerlos. Si ya has escrito el código siguiendo el texto, hazlos en el orden en que aparecen: los que modifican el proyecto (4.1 y 4.3) terminan con un *commit*, de modo que cada ejercicio empieza con el árbol de trabajo limpio (`git status` sin nada pendiente) y ves bien qué cambia en él. Si todavía no lo has escrito, haz primero el ejercicio de proyecto 4.5, que reúne todo el código e incluye el `GET`: en ese caso ya tienes hecho el ejercicio 4.1, y puedes pasar directamente a los demás.

Ejercicio 4.1: Añade un GET para recuperar una misión ★★☆☆☆

Añade al proyecto un segundo caso de uso, `GetQuestUseCase`, y su endpoint `GET /quests/{id}`. Sigue estos pasos:

1. En `application/port/in`, crea la interfaz `GetQuestUseCase` con un método `Optional<Quest> get(QuestId id)`.
2. En `application/service`, crea `GetQuestService`, que la implementa con el puerto de salida `QuestRepository` (un solo campo `final` y `@RequiredArgsConstructor`).
3. En `QuestController`, añade el campo `GetQuestUseCase getQuest` y un método `get` con `@GetMapping("/{id}")` y `@PathVariable String id`. Si hay misión, responde

200 con su `QuestResponse`; si no, 404 sin cuerpo.

4. En `BeanConfig`, registra el *bean* `GetQuestUseCase`.
5. Arranca con `./gradlew bootRun`, publica una misión con `curl` y recupérala por su `id`.
6. Haz un *commit* con el mensaje `Añade GET /quests/{id}`.

Está bien si: `GET /quests/<id>` de una misión publicada responde `200` con el mismo cuerpo que devolvió el `POST`; con un `id` inventado responde `404`; `./gradlew build` termina con `BUILD SUCCESSFUL`; y `GetQuestService` no importa nada de Spring.

Pista: *para el 404 sin cuerpo,* `ResponseEntity.notFound().build()`*. Y recuerda que un* `Optional` *se transforma con* `map` *y se completa con* `orElseGet`*.*

Guiado ▪ solución en la pág. 91

Ejercicio 4.2: Sigue un POST por las clases ★★☆☆☆

Sigue la petición `POST /quests` desde que llega hasta que se devuelve la respuesta. Participan once clases de GuildBoard:

1. `QuestController`
2. `PublishQuestRequest`
3. `PublishQuestCommand`
4. `PublishQuestUseCase`
5. `PublishQuestService`
6. `Quest`
7. `QuestRepository`
8. `InMemoryQuestRepository`
9. `QuestResponse`
10. `BeanConfig`
11. `GuildBoardApplication`

1. Para cada una, escribe en qué carpeta (bajo `com/guildboard`) está y qué papel tiene: dominio, puerto de entrada, puerto de salida, servicio de aplicación, comando, adaptador primario, adaptador secundario, DTO o raíz de composición.
2. Marca cuáles importan `org.springframework`.
3. Escribe el orden en que se ejecutan las clases 1, 5, 6 y 8 durante la petición, y di cuál de las once ejecutó el `new` de `PublishQuestService` y cuándo.
4. En la cadena de llamadas, hay dos saltos en los que el `import` apunta en sentido contrario a la llamada. ¿Cuáles son y por qué ocurre?

Está bien si tienes once filas con carpeta y papel correctos, solo tres clases importan Spring, y los dos saltos son los de las interfaces (puerto de entrada y puerto de salida).

Lectura de código ▪ solución en la pág. 95

Ejercicio 4.3: La aplicación no arranca ★★★☆☆

Un compañero ha dejado esta clase de configuración y ahora la aplicación no arranca. Sustituye `config/BeanConfig.java` por ella (necesitas haber hecho el ejercicio 4.1):

`config/BeanConfig.java`

```java
package com.guildboard.config;

import com.guildboard.adapter.out.memory.InMemoryQuestRepository;
import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.application.service.GetQuestService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public QuestRepository questRepository() {
        return new InMemoryQuestRepository();
    }

    @Bean
    public GetQuestUseCase getQuestUseCase(QuestRepository repository) {
        return new GetQuestService(repository);
    }
}
```

1. Ejecuta `./gradlew bootRun`. Copia las dos frases del mensaje (*Description* y *Action*) que dicen qué clase pide qué tipo.
2. Arregla `BeanConfig` para que arranque y `POST /quests` responda `201`.
3. Explica por qué la solución *no* es ponerle `@Service` a `PublishQuestService`, y qué prueba fallaría con `./gradlew test` si lo hicieras.
4. Para ver el error contrario, declara además en `BeanConfig` un segundo bean de tipo `QuestRepository` (otro `InMemoryQuestRepository`, con otro nombre de método). Arranca y anota qué mensaje da y qué parte te dice cuál es el problema. Después, quítalo.
5. Haz un *commit* con el mensaje `Corrige BeanConfig`.

Está bien si: la aplicación arranca, el `POST` responde `201`, `PublishQuestService` no ha cambiado ni lleva anotaciones, y `./gradlew build` termina con `BUILD SUCCESSFUL`.

Pista: *el mensaje nombra el constructor y el tipo que no encuentra: busca en `BeanConfig` qué método debería devolver ese tipo.*

Depuración ▪ solución en la pág. 95

Ejercicio 4.4: Un segundo adaptador de entrada (temporal) ★★★☆☆

Demuestra que el núcleo no sabe quién lo llama. Crea un **segundo adaptador primario**, un `CommandLineRunner`, que al arrancar publique una misión llamando al *mismo* puerto `PublishQuestUseCase`.

1. Empieza con el árbol de trabajo limpio.
2. Crea `PublishQuestRunner` en el paquete `com.guildboard.adapter.in.cli` (carpeta `src/main/java/com/guildboard/adapter/in/cli`). Es un **paquete temporal**: no figura en el árbol de la verificación de estructura 1 y lo borrarás al final.
3. El título de la misión viene de la propiedad `guildboard.cli.title`, con «Misión de prueba» por defecto (con `@Value`). La recompensa es 10 de oro y el rango, `NOVICE`.
4. El runner imprime en la consola «Misión publicada: » seguido del `id`.
5. Arranca, copia el `id` de la consola y recupera la misión por la API con `curl`. Comprueba con `git status` que solo hay archivos nuevos en `adapter/in/cli`.
6. Borra la carpeta `adapter/in/cli`, comprueba que `git status` está limpio y que la aplicación vuelve a arrancar sin publicar nada.

Está bien si: la misión publicada al arrancar se recupera por `GET`; no has modificado `application`, `domain`, `BeanConfig` ni el controlador; y tras el borrado el proyecto vuelve a su estado anterior.

Pista: *una clase con `@Component` que implemente `CommandLineRunner` la encuentra el escaneo sola, y `@Value` va en el parámetro del constructor.*

Práctica ▪ solución en la pág. 97

Ejercicio 4.5: Hito H1 (parte 2): GuildBoard arranca ★★★☆☆

Parte del proyecto tal como lo dejó el capítulo 3: el árbol T1 con el dominio, los dos puertos y `ArchitectureTest`. Implementa `PublishQuest` de punta a punta y recupera la misión por `GET`.

1. Añade Spring Boot a `build.gradle` y crea `application.yml`.
2. Crea el comando, cambia el puerto de entrada y escribe el servicio.
3. Crea el repositorio en memoria.
4. Crea `PublishQuestRequest`, `QuestResponse` y el controlador.
5. Crea `BeanConfig` y `GuildBoardApplication`.
6. Añade también `GetQuestUseCase`, `GetQuestService` y el `GET /quests/{id}` (ejercicio 4.1); se quedan en el proyecto. Si ya hiciste ese ejercicio, lo tienes hecho: comprueba que está y sigue.
7. Ejecuta `./gradlew test` y `./gradlew build`, arranca con `./gradlew bootRun` y pruébalo con `curl`.
8. Haz un *commit* con el mensaje `H1b: primer endpoint (caso de uso, adaptador web, repositorio en memoria)`.

Está bien si: `./gradlew build` termina con `BUILD SUCCESSFUL`; `POST /quests` responde `201` con `Location` y una misión en estado `PUBLISHED`; `GET` de ese `id` responde `200` y de uno inventado `404`; `git status` no muestra nada pendiente; y solo tres clases importan `org.springframework`.

Proyecto ▪ solución en la pág. 99

4.11 Resumen

Resumen

- Un caso de uso son tres piezas: un puerto de entrada (interfaz), un comando (record con los datos de entrada) y un servicio de aplicación que lo implementa y orquesta sin contener las reglas, que viven en el dominio.
- Spring crea objetos (*beans*) y los inyecta por constructor. Se registran con métodos `@Bean` en una clase `@Configuration` o por escaneo de `@Component` y similares; `@SpringBootApplication` escanea su paquete y los de debajo, así que la clase principal va en el paquete raíz.
- Spring Boot 4.1.1 con Gradle: plugin `org.springframework.boot` y `io.spring.dependency-management`, y el starter `spring-boot-starter-webmvc` (el nombre `spring-boot-starter-web` está en desuso). Los nombres se comprueban con Initializr.
- Con el plugin de Boot, su BOM gestiona JUnit 6.0.3: se quita la versión explícita de JUnit y se mantiene la de ArchUnit. `ArchitectureTest` sigue pasando y ahora puede fallar por un `import` de Spring en `domain` o `application`.
- `application.yml` configura la aplicación (`server.port`); `@Value("${clave:defecto}")` lee un valor, y solo se usa en adaptadores y `config`, nunca en el núcleo.
- El adaptador web es un objeto humilde: convierte JSON en comando, llama al puerto y convierte el resultado en respuesta. El adaptador en memoria implementa el puerto de salida con un mapa.
- La raíz de composición (`BeanConfig` y la clase principal) es el único sitio que conoce las clases concretas. El núcleo no lleva `@Service`: así respeta la regla de dependencia y se construye sin Spring. El coste es registrar cada servicio a mano.
- Se arranca con `./gradlew bootRun` y se prueba con `curl`. La estructura no ha cambiado: se han rellenado las carpetas previstas que figuran en el árbol de la verificación de estructura 1. H1 está completo.

Soluciones de los ejercicios

Solución del ejercicio 4.1 enunciado en la pág. 87

Los cuatro cambios, archivo por archivo. El puerto, nuevo:

application/port/in/GetQuestUseCase.java

```java
package com.guildboard.application.port.in;

import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.util.Optional;

public interface GetQuestUseCase {

    Optional<Quest> get(QuestId id);
}
```

El servicio, nuevo:

application/service/GetQuestService.java

```java
package com.guildboard.application.service;

import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.util.Optional;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class GetQuestService implements GetQuestUseCase {

    private final QuestRepository quests;

    @Override
    public Optional<Quest> get(QuestId id) {
        return quests.findById(id);
    }
}
```

El controlador completo, con el método nuevo:

adapter/in/web/QuestController.java

```java
package com.guildboard.adapter.in.web;

import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.in.PublishQuestCommand;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import com.guildboard.domain.Reward;
import java.net.URI;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/quests")
@RequiredArgsConstructor
public class QuestController {

    private final PublishQuestUseCase publishQuest;
    private final GetQuestUseCase getQuest;

    @PostMapping
    public ResponseEntity<QuestResponse> publish(
            @RequestBody PublishQuestRequest request) {
        PublishQuestCommand command = new PublishQuestCommand(
            request.title(), new Reward(request.reward()),
            request.requiredRank());
        Quest quest = publishQuest.publish(command);
        return ResponseEntity
            .created(URI.create("/quests/" + quest.getId().value()))
            .body(QuestResponse.from(quest));
    }

    @GetMapping("/{id}")
    public ResponseEntity<QuestResponse> get(@PathVariable String id) {
        return getQuest.get(new QuestId(id))
            .map(QuestResponse::from)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }
}
```

Y la configuración completa:

config/BeanConfig.java

```java
package com.guildboard.config;

import com.guildboard.adapter.out.memory.InMemoryQuestRepository;
import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.application.service.GetQuestService;
import com.guildboard.application.service.PublishQuestService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public QuestRepository questRepository() {
        return new InMemoryQuestRepository();
    }

    @Bean
    public PublishQuestUseCase publishQuestUseCase(
            QuestRepository questRepository) {
        return new PublishQuestService(questRepository);
    }

    @Bean
    public GetQuestUseCase getQuestUseCase(
            QuestRepository questRepository) {
        return new GetQuestService(questRepository);
    }
}
```

Por qué funciona: el controlador pide los dos puertos por constructor (el orden de los campos fija el orden de los parámetros que genera Lombok), y `BeanConfig` construye cada servicio con el mismo repositorio, que el contenedor guarda como *singleton*: por eso lo que publica un `POST` lo ve el `GET`. El `map(QuestResponse::from).map(ResponseEntity::ok)` transforma el `Optional` de la misión, primero en uno de `QuestResponse` y luego en uno de respuesta HTTP, y `orElseGet` da el `404` cuando está vacío. Prueba:

```
$ curl -i http://localhost:8080/quests/6b8f0c2e-5a1d-4c7e-9a3b-2d1e7f4a9c10
HTTP/1.1 200
Content-Type: application/json

{"id":"6b8f0c2e-5a1d-4c7e-9a3b-2d1e7f4a9c10","title":"Limpiar el s^^c3^^b3tano
    ↪ ","reward":150,"requiredRank":"NOVICE","status":"PUBLISHED"}
$ curl -i http://localhost:8080/quests/no-existe
HTTP/1.1 404
```

(Salida abreviada; el `id` será otro.)

Errores típicos: olvidar el método `@Bean` (la aplicación no arranca: ejercicio 4.3); devolver el `Optional` directamente desde el controlador en lugar de convertirlo en una respues-

ta; poner a `GetQuestService` un `@Service` (rompe la regla de dependencia); y escribir `@PathVariable("id")` con un nombre distinto del de la ruta, que da un `500` al arrancar o al primer `GET`.

Solución del ejercicio 4.2 enunciado en la pág. 88

Clase	Carpeta	Papel (¿Spring?)
`QuestController`	`adapter/in/web`	Adaptador primario (sí)
`PublishQuestRequest`	`adapter/in/web`	DTO de entrada (no)
`PublishQuestCommand`	`application/port/in`	Comando (no)
`PublishQuestUseCase`	`application/port/in`	Puerto de entrada (no)
`PublishQuestService`	`application/service`	Servicio de aplicación (no)
`Quest`	`domain`	Entidad (no)
`QuestRepository`	`application/port/out`	Puerto de salida (no)
`InMemoryQuestRepository`	`adapter/out/memory`	Adaptador secundario (no)
`QuestResponse`	`adapter/in/web`	DTO de salida (no)
`BeanConfig`	`config`	Raíz de composición (sí)
`GuildBoardApplication`	raíz (`com/guildboard`)	Arranque y escaneo (sí)

Solo tres clases importan Spring: el controlador (anotaciones de MVC y `ResponseEntity`), la configuración y la clase principal. Todo lo demás se podría compilar sin Spring.
Orden en la petición: primero el controlador (1) recibe la petición y llama al puerto (4), que en ejecución es el servicio (5). El servicio llama a `Quest` (6), para crear y publicar la misión, y después guarda con el puerto de salida (7), que en ejecución es el repositorio en memoria (8). El `new` de `PublishQuestService` no lo ejecuta ninguna de esas: lo ejecutó `BeanConfig` (10), *una sola vez, al arrancar*, no en cada petición.
Los dos saltos con el `import` al revés: de `PublishQuestUseCase` a `PublishQuestService` (la llamada va hacia el servicio, pero es el servicio el que importa la interfaz) y de `QuestRepository` a `InMemoryQuestRepository` (la llamada va al adaptador, pero es el adaptador el que importa el puerto). Es la inversión de dependencias: el control sale hacia fuera, las dependencias del código apuntan hacia dentro.
Errores típicos: decir que el controlador llama a `PublishQuestService` (llama al puerto); ubicar el comando en un paquete de `adapter`; creer que `BeanConfig` se ejecuta con cada petición (corre al arrancar); y contar como «con Spring» las clases que solo se usan junto a Spring pero no lo importan, como los DTO.

Solución del ejercicio 4.3 enunciado en la pág. 89

El mensaje, abreviado:

```
APPLICATION FAILED TO START

Description:

Parameter 0 of constructor in com.guildboard.adapter.in.web.QuestController
required a bean of type
'com.guildboard.application.port.in.PublishQuestUseCase' that could not be
    ↪ found.

Action:

Consider defining a bean of type
'com.guildboard.application.port.in.PublishQuestUseCase' in your configuration
    ↪ .
```

(Aquí se han partido líneas para que quepan.) El constructor generado por Lombok tiene `PublishQuestUseCase` como parámetro 0 y nadie fabrica un bean de ese tipo: a `BeanConfig` le falta el método. La corrección es añadirlo; la clase completa queda así:

config/BeanConfig.java

```java
package com.guildboard.config;

import com.guildboard.adapter.out.memory.InMemoryQuestRepository;
import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.application.service.GetQuestService;
import com.guildboard.application.service.PublishQuestService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public QuestRepository questRepository() {
        return new InMemoryQuestRepository();
    }

    @Bean
    public PublishQuestUseCase publishQuestUseCase(
            QuestRepository questRepository) {
        return new PublishQuestService(questRepository);
    }

    @Bean
    public GetQuestUseCase getQuestUseCase(
            QuestRepository repository) {
        return new GetQuestService(repository);
    }
}
```

El método recibe el repositorio por parámetro, y el contenedor le da el bean `questRepository`; construye el servicio con él y lo registra como `PublishQuestUseCase`. Por qué no ponerle `@Service` al servicio: sería arreglar el síntoma con una dependencia del núcleo hacia Spring. Si lo haces, `./gradlew test` falla en `ArchitectureTest`, en la prueba `applicationOnlyDependsOnDomain`, con un mensaje de ArchUnit que nombra la clase, la anotación y el archivo con su línea, parecido a

```
Rule 'classes that reside in a package '..application..' should only depend on
classes that reside in any package ['..application..', '..domain..', 'java..',
'lombok..']' was violated (1 times):
Class <com.guildboard.application.service.PublishQuestService> is annotated
    ↪ with
<org.springframework.stereotype.Service> in (PublishQuestService.java:0)
```

(El texto exacto depende de la versión de ArchUnit; lo importante es que dice qué clase rompe qué regla. Es el mensaje que ya leíste en el capítulo 3; con las reglas que añadiste en el ejercicio 3.4 el planteamiento es el mismo.) Y aunque la aplicación arrancara, habrías roto la decisión del curso de que `application` no dependa del framework.

El error contrario, con dos beans de tipo `QuestRepository`: Spring no sabe cuál inyectar en `getQuestUseCase(QuestRepository repository)` y el arranque falla con un mensaje que dice que ese parámetro «required a single bean, but 2 were found», con una lista de los dos beans y el método que define cada uno. (En `publishQuestUseCase` no falla, porque su parámetro se llama `questRepository`, igual que uno de los beans, y Spring desempata por el nombre; ese parecido es una casualidad que no conviene usar como diseño.) Aquí hay que decidir: normalmente sobra uno; si de verdad hacen falta dos, se distingue por nombre o con `@Primary` o `@Qualifier` (que no necesitas en este curso). Quita el segundo y vuelve a arrancar.

Errores típicos: arreglarlo con `@Service`; olvidar `@Configuration` en la clase, con lo que ningún método `@Bean` se lee (el mensaje es el mismo que el de arriba, y la clase parece correcta); y mover `BeanConfig` fuera del paquete `com.guildboard`, donde el escaneo no la encuentra.

Solución del ejercicio 4.4

enunciado en la pág. 89

Crea la carpeta y el archivo:

```
$ mkdir -p src/main/java/com/guildboard/adapter/in/cli
```

adapter/in/cli/PublishQuestRunner.java (temporal)

```java
package com.guildboard.adapter.in.cli;

import com.guildboard.application.port.in.PublishQuestCommand;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.domain.Quest;
import com.guildboard.domain.Rank;
import com.guildboard.domain.Reward;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class PublishQuestRunner implements CommandLineRunner {

    private final PublishQuestUseCase publishQuest;
    private final String title;

    public PublishQuestRunner(
            PublishQuestUseCase publishQuest,
            @Value("${guildboard.cli.title:Misi^^c3^^b3n de prueba}")
            String title) {
        this.publishQuest = publishQuest;
        this.title = title;
    }

    @Override
    public void run(String... args) {
        PublishQuestCommand command = new PublishQuestCommand(
            title, new Reward(10), Rank.NOVICE);
        Quest quest = publishQuest.publish(command);
        System.out.println("Misi^^c3^^b3n publicada: " + quest.getId().
    ↪ value());
    }
}
```

Se escribe el constructor a mano porque `@RequiredArgsConstructor` no copia el `@Value` del parámetro. Arranca y busca la línea en la consola:

```
$ ./gradlew bootRun
(registro de arranque omitido)
Misi^^c3^^b3n publicada: 0e7c1a52-3b9d-4f60-8a14-5c2d9e6b7f31
$ curl -i http://localhost:8080/quests/0e7c1a52-3b9d-4f60-8a14-5c2d9e6b7f31
HTTP/1.1 200
(cabecera y cuerpo omitidos)
$ git status --short
?? src/main/java/com/guildboard/adapter/in/cli/
```

(Salida abreviada.) El runner no necesitó tocar `BeanConfig`: es un `@Component` y el escaneo del paquete `com.guildboard` lo encuentra. La misión publicada al arrancar es la misma que sirve el controlador porque hay un solo repositorio singleton. Es la demostración de la tesis del capítulo 3: dos adaptadores primarios, uno HTTP y otro de arranque, llaman al mismo puerto y el núcleo no se entera. Para cambiar el título sin tocar código, pásalo como

propiedad:

```
$ ./gradlew bootRun --args='--guildboard.cli.title=Otra'
```

Al terminar, borra el paquete temporal y comprueba:

```
$ rm -r src/main/java/com/guildboard/adapter/in/cli
$ git status --short
$ ./gradlew bootRun
```

La primera orden de `git` no muestra nada, y la aplicación arranca sin la línea «Misión publicada». Como el paquete nunca se confirmó en Git, no hay nada que deshacer; si lo hubieras confirmado, un `git rm -r` del mismo directorio y un nuevo *commit* lo quitan. La estructura del árbol de la verificación de estructura 1 no ha cambiado.

Errores típicos: dejar el paquete en el proyecto (aparecería como carpeta nueva fuera del árbol de la verificación de estructura 1); poner `@Value` en un campo `final` con `@RequiredArgsConstructor`; llamar a `PublishQuestService` en lugar del puerto; olvidar el valor por defecto, con lo que falla el arranque si falta la propiedad.

Solución del ejercicio 4.5

enunciado en la pág. 90

Desde la carpeta `guildboard`, crea las carpetas nuevas:

```
$ mkdir -p src/main/resources
$ mkdir -p src/main/java/com/guildboard/application/service
$ mkdir -p src/main/java/com/guildboard/adapter/in/web
$ mkdir -p src/main/java/com/guildboard/adapter/out/memory
$ mkdir -p src/main/java/com/guildboard/config
```

Los archivos, en el orden del enunciado. Construcción:

build.gradle

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.1'
    id 'io.spring.dependency-management' version '1.1.7'
}

group = 'com.guildboard'
version = '0.0.1-SNAPSHOT'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'

    testImplementation 'org.junit.jupiter:junit-jupiter'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}

tasks.named('test', Test) {
    useJUnitPlatform()
}
```

src/main/resources/application.yml

```
spring:
  application:
    name: guildboard

server:
  port: 8080
```

Comando, puertos de entrada y servicios (`application`):

application/port/in/PublishQuestCommand.java

```java
package com.guildboard.application.port.in;

import com.guildboard.domain.Rank;
import com.guildboard.domain.Reward;

public record PublishQuestCommand(String title, Reward reward,
                                  Rank requiredRank) {
}
```

application/port/in/PublishQuestUseCase.java

```java
package com.guildboard.application.port.in;

import com.guildboard.domain.Quest;

public interface PublishQuestUseCase {

    Quest publish(PublishQuestCommand command);
}
```

application/port/in/GetQuestUseCase.java

```java
package com.guildboard.application.port.in;

import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.util.Optional;

public interface GetQuestUseCase {

    Optional<Quest> get(QuestId id);
}
```

application/service/PublishQuestService.java

```java
package com.guildboard.application.service;

import com.guildboard.application.port.in.PublishQuestCommand;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.util.UUID;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class PublishQuestService implements PublishQuestUseCase {

    private final QuestRepository quests;

    @Override
    public Quest publish(PublishQuestCommand command) {
        Quest quest = Quest.draft(
            new QuestId(UUID.randomUUID().toString()),
            command.title(), command.reward(), command.requiredRank());
        quest.publish();
        quests.save(quest);
        return quest;
    }
}
```

application/service/GetQuestService.java

```java
package com.guildboard.application.service;

import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.util.Optional;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class GetQuestService implements GetQuestUseCase {

    private final QuestRepository quests;

    @Override
    public Optional<Quest> get(QuestId id) {
        return quests.findById(id);
    }
}
```

Adaptadores:

adapter/out/memory/InMemoryQuestRepository.java

```java
package com.guildboard.adapter.out.memory;

import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

public class InMemoryQuestRepository implements QuestRepository {

    private final Map<QuestId, Quest> quests = new ConcurrentHashMap<>();

    @Override
    public Optional<Quest> findById(QuestId id) {
        return Optional.ofNullable(quests.get(id));
    }

    @Override
    public void save(Quest quest) {
        quests.put(quest.getId(), quest);
    }
}
```

adapter/in/web/PublishQuestRequest.java

```java
package com.guildboard.adapter.in.web;

import com.guildboard.domain.Rank;

public record PublishQuestRequest(String title, int reward,
                                  Rank requiredRank) {
}
```

adapter/in/web/QuestResponse.java

```
package com.guildboard.adapter.in.web;

import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestStatus;
import com.guildboard.domain.Rank;

public record QuestResponse(String id, String title, int reward,
                            Rank requiredRank, QuestStatus status) {

    public static QuestResponse from(Quest quest) {
        return new QuestResponse(
            quest.getId().value(),
            quest.getTitle(),
            quest.getReward().gold(),
            quest.getRequiredRank(),
            quest.getStatus());
    }
}
```

adapter/in/web/QuestController.java

```java
package com.guildboard.adapter.in.web;

import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.in.PublishQuestCommand;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import com.guildboard.domain.Reward;
import java.net.URI;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/quests")
@RequiredArgsConstructor
public class QuestController {

    private final PublishQuestUseCase publishQuest;
    private final GetQuestUseCase getQuest;

    @PostMapping
    public ResponseEntity<QuestResponse> publish(
            @RequestBody PublishQuestRequest request) {
        PublishQuestCommand command = new PublishQuestCommand(
            request.title(), new Reward(request.reward()),
            request.requiredRank());
        Quest quest = publishQuest.publish(command);
        return ResponseEntity
            .created(URI.create("/quests/" + quest.getId().value()))
            .body(QuestResponse.from(quest));
    }

    @GetMapping("/{id}")
    public ResponseEntity<QuestResponse> get(@PathVariable String id) {
        return getQuest.get(new QuestId(id))
            .map(QuestResponse::from)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }
}
```

Raíz de composición y arranque:

config/BeanConfig.java

```java
package com.guildboard.config;

import com.guildboard.adapter.out.memory.InMemoryQuestRepository;
import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.application.service.GetQuestService;
import com.guildboard.application.service.PublishQuestService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public QuestRepository questRepository() {
        return new InMemoryQuestRepository();
    }

    @Bean
    public PublishQuestUseCase publishQuestUseCase(
            QuestRepository questRepository) {
        return new PublishQuestService(questRepository);
    }

    @Bean
    public GetQuestUseCase getQuestUseCase(
            QuestRepository questRepository) {
        return new GetQuestService(questRepository);
    }
}
```

GuildBoardApplication.java

```java
package com.guildboard;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class GuildBoardApplication {

    public static void main(String[] args) {
        SpringApplication.run(GuildBoardApplication.class, args);
    }
}
```

Construye, prueba y confirma:

```
$ ./gradlew build
$ ./gradlew bootRun
$ curl -i -X POST http://localhost:8080/quests \
    -H "Content-Type: application/json" \
    -d '{"title":"Limpiar el s^^c3^^b3tano","reward":150,"requiredRank":"
    ↪ NOVICE"}'
$ curl -i http://localhost:8080/quests/<id>
$ git add .
$ git commit -m "H1b: primer endpoint (caso de uso, adaptador web, repositorio
    ↪  en memoria)"
```

La respuesta del `POST` es `201` con `Location` y la misión en `PUBLISHED`; el `GET` de ese `id`, `200`, y el de uno inventado, `404`. Por qué funciona: el escaneo de `@SpringBootApplication` encuentra `QuestController` y `BeanConfig`; `BeanConfig` crea el repositorio y los dos servicios con `new`; el controlador recibe los puertos por constructor. Solo estas tres clases (la principal, la configuración y el controlador) importan Spring, y `ArchitectureTest` lo vigila para `domain` y `application`. El hito H1 queda completo, y la estructura es la del árbol T1 más las carpetas previstas ya creadas.

Errores típicos: escribir `spring-boot-starter-web` (en desuso en Boot 4) o no poner el plugin de gestión de dependencias y obtener un error de versión en el starter; dejar la versión explícita del BOM de JUnit al lado del plugin de Boot (dos fuentes de versión para lo mismo); ejecutar `./gradlew build` sin la clase principal (el empaquetado no encuentra qué arrancar); poner la clase principal en un subpaquete y que el escaneo no llegue a `config` ni a `adapter`; no configurar el `Content-Type` en `curl` y recibir `415`; y olvidar el `git add` de las carpetas nuevas.

Parte II

Hexagonal en un monolito

CAPÍTULO 5

Un dominio rico: invariantes y errores de dominio

Objetivos del capítulo

- Distinguir las invariantes de un objeto de las reglas que dependen de datos de fuera, y decidir dónde vive cada una.
- Sustituir las excepciones genéricas por errores de dominio tipados, con una jerarquía sellada y un `switch` exhaustivo.
- Escribir los casos de uso de H2 (registrar, aceptar, completar y dar experiencia) modificando un solo agregado en cada uno.
- Probar el dominio sin Spring con JUnit y AssertJ, y reconocer lo que este diseño *no* garantiza.

Al terminar el capítulo 4 GuildBoard arranca y publica misiones, pero cualquiera podría aceptar cualquier misión, sin mirar su rango y sin límite. Las reglas R3 y R4 de `docs/dominio.md` (el rango y las tres misiones activas) existen en un documento y no en el código. Mientras el código no las haga cumplir, no son reglas: son deseos. Este capítulo las mete en el dominio, hace que sus violaciones sean errores con nombre y tipo que el borde podrá traducir (capítulo 6), y añade al proyecto los aventureros y los casos de uso que los usan (hito H2).

Hay además una parte incómoda, y el capítulo la trata con franqueza: hay reglas que un solo agregado no puede garantizar. La sección final dice cuáles y qué no resuelve el curso.

5.1 De entidad con setters a agregado con comportamiento

Una *invariante* es una condición que tiene que ser cierta siempre que puedas observar el objeto: «el título de una misión no está en blanco», «una misión `COMPLETED` pasó antes por `ACCEPTED`». Evans las sitúa en el corazón de los agregados: la raíz del agregado es quien responde de que sus invariantes se cumplan en cada cambio [7]. Martin lo dice con otras palabras: las reglas críticas del negocio pertenecen a las entidades, y son las que valdrían aunque no existiera la aplicación (capítulo 20, «Business Rules», de [28]). Lo contrario es el modelo anémico que ya conoces [9]: datos con *setters* y un servicio que decide.

Repasa las reglas R1 a R7 del documento de dominio. Hay tres clases de invariante, según lo que necesitan saber:

- **De valor**: solo dependen de los datos del propio objeto y se comprueban al construirlo. «La recompensa no es negativa» (R7) vive en el constructor de `Reward`.
- **De transición**: dependen del estado actual. «Solo se acepta una misión publicada» (R2) vive en el método que cambia el estado.
- **De contexto**: dependen de datos que el objeto no tiene. «El aventurero tiene el rango suficiente» (R3) y «no lleva más de 3 misiones activas» (R4) necesitan el rango y el

recuento de un aventurero.

Las dos primeras clases ya las cumple el código del capítulo anterior. La tercera es el problema del capítulo, y la sección 5.4 decide qué hacer con ella.

Regla	Clase de invariante	Quién la hace cumplir
R1, R2	transición	`Quest`: cada método comprueba el estado antes de cambiarlo
R3	contexto (rango)	`Quest`, con el rango llegando como dato
R4	contexto (recuento)	`Quest`, con el recuento llegando como dato; no la garantiza del todo
R5	entre agregados	el caso de uso, que llama a dos casos de uso
R6	valor y transición	`Adventurer`, en `gainExperience`
R7	valor	`Reward` y `Quest.draft`

5.2 Rank y Reward como objetos de valor

Los dos valores ya existen desde el capítulo 3, pero merece la pena releerlos con los ojos de este capítulo.

`Reward` es un *record*: inmutable, con igualdad por valor, y con la invariante en el constructor compacto. No hay forma de tener un `Reward` de oro negativo, así que quien recibe uno no tiene que comprobarlo. Su método `plus` devuelve un valor nuevo en lugar de modificar el actual. Es la regla que Bloch da para las clases de valor: minimizar la mutabilidad, de modo que un objeto tenga un único estado, el de su construcción [3].

`Rank` es un *enum* cuyo orden de declaración *es* la regla de negocio: `canTake` usa `compareTo`, que compara posiciones. Por eso `NOVICE < ADEPT < VETERAN` y por eso no se reordenan las constantes sin repasar todo lo que usa `canTake`. Es una trampa pequeña y real: un enum reordenado compila perfectamente.

Ejemplo ilustrativo (no entra en el proyecto)

```
Rank.ADEPT.canTake(Rank.NOVICE);     // true:  ADEPT >= NOVICE
Rank.NOVICE.canTake(Rank.ADEPT);     // false: NOVICE < ADEPT
Rank.VETERAN.canTake(Rank.VETERAN);  // true:  el mismo rango basta
```

El aventurero necesita un tercer valor: la experiencia, que sube el rango. El curso fija estos umbrales (son del juego de GuildBoard, no de ningún estándar):

Rango	Experiencia mínima
`NOVICE`	0
`ADEPT`	100
`VETERAN`	300

Y el agregado `Adventurer`. Empieza en `NOVICE` con 0 de experiencia, no tiene *setters* y su única forma de ganar experiencia es `gainExperience`, que rechaza los puntos que no sean positivos (R6). En esta primera versión la experiencia sube y el rango no se mueve; hacer que ascienda según la tabla es el ejercicio 5.3.

domain/Adventurer.java

```java
package com.guildboard.domain;

import java.util.Objects;
import lombok.EqualsAndHashCode;
import lombok.Getter;

@Getter
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
public final class Adventurer {

    @EqualsAndHashCode.Include
    private final AdventurerId id;
    private final String name;
    private Rank rank;
    private int experience;

    private Adventurer(AdventurerId id, String name) {
        this.id = id;
        this.name = name;
        this.rank = Rank.NOVICE;
        this.experience = 0;
    }

    public static Adventurer register(AdventurerId id, String name) {
        Objects.requireNonNull(id, "id");
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name must not be blank");
        }
        return new Adventurer(id, name.strip());
    }

    public void gainExperience(int points) {
        if (points <= 0) {
            throw new IllegalArgumentException(
                "points must be positive: " + points);
        }
        experience += points;
    }
}
```

Con Lombok, `@Getter` da `getId()`, `getName()`, `getRank()` y `getExperience()`. Fíjate en la distinción que usa el curso: un valor mal formado (un nombre en blanco, unos puntos negativos) lanza `IllegalArgumentException`, la excepción estándar para argumentos inválidos, como ya hacían `Quest.draft` y `Reward`. Una *regla de negocio* violada con datos bien formados (aceptar una misión que está por encima de tu rango) merece un error propio. La sección siguiente lo define.

5.3 Errores de dominio como jerarquía sellada

En el capítulo 3, una transición inválida lanzaba `IllegalStateException` con un mensaje en texto. Es suficiente para un programador que lee la traza y inútil para quien tiene que decidir qué responder: ¿cuál de las dos situaciones «no encontrada» o «ya aceptada» merece un `404` y

cuál un 409? Comparar el texto del mensaje sería frágil. La alternativa es que el error *sea un tipo*.

Con las interfaces selladas del capítulo 1 se declara una jerarquía cerrada: la lista completa de lo que puede salir mal en el dominio, cada caso con los datos que lo explican. Java 21 hace definitivos los *record patterns* y el *pattern matching* en `switch` [34]: con una jerarquía sellada, el compilador sabe cuándo un `switch` cubre todos los casos.

domain/DomainError.java

```
package com.guildboard.domain;

public sealed interface DomainError {

    record QuestNotFound(QuestId questId) implements DomainError { }

    record AdventurerNotFound(AdventurerId adventurerId)
            implements DomainError { }

    record InvalidTransition(QuestId questId, QuestStatus actual,
                             QuestStatus expected)
            implements DomainError { }

    record RankTooLow(Rank adventurerRank, Rank requiredRank)
            implements DomainError { }

    record TooManyActiveQuests(int active, int limit)
            implements DomainError { }

    default String code() {
        return switch (this) {
            case QuestNotFound e -> "QUEST_NOT_FOUND";
            case AdventurerNotFound e -> "ADVENTURER_NOT_FOUND";
            case InvalidTransition e -> "INVALID_TRANSITION";
            case RankTooLow e -> "RANK_TOO_LOW";
            case TooManyActiveQuests e -> "TOO_MANY_ACTIVE_QUESTS";
        };
    }
}
```

Cada variante es un *record* anidado dentro de la propia interfaz. Si una interfaz sellada no escribe `permits`, los permitidos son los que están en el mismo archivo: aquí, los cinco *records*. El método `code()` da a cada error un identificador estable, pensado para registros y clientes. Su `switch` no tiene `default`, y es lo importante: si mañana alguien añade una variante y olvida este método, el código *no compila*. Esa es la ventaja de lo sellado frente a una jerarquía abierta de excepciones.

Una excepción de Java tiene que ser una clase que herede de `Throwable`, y un *record* no puede. Por eso el error viaja dentro de una única excepción, que lo lleva como dato y lo expone:

domain/DomainException.java

```
package com.guildboard.domain;

public final class DomainException extends RuntimeException {

```

```
    private final DomainError error;

    public DomainException(DomainError error) {
        super(describe(error));
        this.error = error;
    }

    public DomainError getError() {
        return error;
    }

    private static String describe(DomainError error) {
        return switch (error) {
            case DomainError.QuestNotFound(QuestId id) ->
                "quest not found: " + id.value();
            case DomainError.AdventurerNotFound(AdventurerId id) ->
                "adventurer not found: " + id.value();
            case DomainError.InvalidTransition(QuestId id,
                    QuestStatus actual, QuestStatus expected) ->
                "quest " + id.value() + " is " + actual
                    + ", expected " + expected;
            case DomainError.RankTooLow(Rank have, Rank need) ->
                "rank " + have + " cannot take a quest that requires "
                    + need;
            case DomainError.TooManyActiveQuests(int active, int limit) ->
                "adventurer has " + active
                    + " active quests, the limit is " + limit;
        };
    }
}
```

Es `RuntimeException` (no comprobada): Bloch reserva las excepciones comprobadas para condiciones que el llamante puede recuperar, y aquí el llamante inmediato (un servicio de aplicación) no puede arreglar nada; quien decide qué hacer es el borde [3]. El constructor usa `describe` para fabricar el mensaje: el segundo `switch` exhaustivo. Los patrones de la forma `DomainError.RankTooLow(Rank have, Rank need)` deconstruyen el *record* y dan nombre a sus componentes.

Dos usos de las excepciones, entonces, y no se mezclan:

Situación	Excepción	Ejemplo
Dato mal formado	`IllegalArgumentException`	título en blanco, oro negativo
Regla de negocio violada	`DomainException`	rango insuficiente, transición no permitida
Invariante rota por un fallo del programa	`IllegalStateException`	una misión aceptada sin aventurero

Esta es una decisión del curso, no una ley de Java. Por ahora la traducción a HTTP no existe: un `DomainException` que llegue al controlador acaba como un `500`, igual que el título en blanco del capítulo 4. El capítulo 4 lo avisó y el capítulo 6 lo arregla.

Ahora puedes cambiar los métodos de `Quest`. Los `IllegalStateException` de `requireStatus` pasan a ser un error tipado, y es lo único que cambia en `publish` y `complete`:

domain/Quest.java (fragmento: el método requireStatus)

```
private void requireStatus(QuestStatus expected) {
    if (status != expected) {
        throw new DomainException(
            new DomainError.InvalidTransition(id, status, expected));
    }
}
```

(El parámetro se llama `expected` y no `required` para no chocar con el campo `requiredRank` al leerlo.)

5.4 Las reglas de contexto: rango y límite de misiones

Falta decidir dónde viven R3 y R4. `docs/dominio.md` ya lo dejó escrito: la transición de aceptar es de la misión, y `Quest` posee `accept`. Lo que no tiene es el rango del aventurero ni cuántas misiones lleva. La solución es que *lleguen como datos*: quien orquesta el caso de uso los lee y los pasa.

¿Y por qué no pasar el `Adventurer` entero o el repositorio? Porque entonces `Quest` conocería a otro agregado y a la persistencia. Con un `Rank` y un `int` sigue siendo una clase que se prueba con `new`, sin nada más. Tampoco se guarda la lista de misiones dentro de `Adventurer`: aceptar tocaría dos agregados y cada aventurero cargaría su lista completa. Es la recomendación habitual para dimensionar un agregado: lo más pequeño que permita proteger las invariantes que necesitan consistencia inmediata [25]. Hombergs defiende lo mismo en su libro: agregados pequeños, para que cada caso de uso toque pocos objetos [19].

El método, con las comprobaciones *antes* de cualquier cambio:

domain/Quest.java (fragmento: constante y accept)

```
public static final int MAX_ACTIVE_QUESTS = 3;

public void accept(AdventurerId adventurerId, Rank adventurerRank,
                   int activeQuests) {
    Objects.requireNonNull(adventurerId, "adventurerId");
    Objects.requireNonNull(adventurerRank, "adventurerRank");
    if (activeQuests < 0) {
        throw new IllegalArgumentException(
            "activeQuests must not be negative: " + activeQuests);
    }
    requireStatus(QuestStatus.PUBLISHED);
    if (!adventurerRank.canTake(requiredRank)) {
        throw new DomainException(
            new DomainError.RankTooLow(adventurerRank, requiredRank));
    }
    if (activeQuests >= MAX_ACTIVE_QUESTS) {
        throw new DomainException(new DomainError.TooManyActiveQuests(
            activeQuests, MAX_ACTIVE_QUESTS));
    }
    assignedTo = adventurerId;
    status = QuestStatus.ACCEPTED;
}
```

Esta es la firma que fija el plan y que usarán los capítulos siguientes: `accept(AdventurerId, Rank, int)`. Tres cosas.

- **El orden de las comprobaciones.** Estado, rango, límite, y solo entonces se asigna el aventurero y se cambia el estado. Si cualquiera falla, la misión queda *exactamente* como estaba. Una invariante que se rompe y se repara a medias es peor que una que no se comprueba.
- **Los datos que llegan se validan.** Un recuento negativo es un fallo del programa, no una regla de negocio, y por eso es `IllegalArgumentException`.
- **La constante.** `MAX_ACTIVE_QUESTS` es una regla del negocio y vive en el dominio. Lombok no genera accesores de campos `static`, así que `@Getter` no la toca.

`Quest` no puede saber si el recuento es verdad

`accept` se fía del `int` que recibe. Si quien llama pasa un 0 cuando el aventurero lleva cinco, la misión se acepta. El límite solo está garantizado *si* el recuento es cierto, y garantizar eso no está en manos del agregado. La sección 5.7 vuelve sobre ello.

5.5 Los casos de uso de H2 y sus endpoints

H2 añade cuatro casos de uso de escritura (más adelante verás también `GetAdventurerUseCase`, una consulta que no modifica nada). Cada uno sigue el patrón del capítulo 4 (puerto de entrada, comando, servicio sin Spring) y cumple una regla de diseño del plan: **modifica como máximo un agregado**. Puede *leer* datos de otro para decidir, pero solo cambia uno.

Caso de uso	Modifica	Lee o llama
`RegisterAdventurer`	`Adventurer`	nada
`AcceptQuest`	`Quest`	lee el `Adventurer` y cuenta sus misiones activas
`GrantExperience`	`Adventurer`	nada
`CompleteQuest`	`Quest`	llama a `GrantExperience` por su puerto

Antes, el puerto de salida de las misiones necesita saber contar, y hace falta un repositorio para los aventureros:

application/port/out/QuestRepository.java

```
1  package com.guildboard.application.port.out;
2
3  import com.guildboard.domain.AdventurerId;
4  import com.guildboard.domain.Quest;
5  import com.guildboard.domain.QuestId;
6  import java.util.Optional;
7
8  public interface QuestRepository {
9
10     Optional<Quest> findById(QuestId id);
11
12     void save(Quest quest);
13
14     int countActiveBy(AdventurerId adventurerId);
15 }
```

Una misión *activa* es una misión `ACCEPTED` de ese aventurero (`docs/dominio.md`). Ese cálculo pertenece al repositorio, que es quien tiene todas las misiones. El repositorio de aventureros es el espejo del de misiones:

application/port/out/AdventurerRepository.java

```
package com.guildboard.application.port.out;

import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;
import java.util.Optional;

public interface AdventurerRepository {

    Optional<Adventurer> findById(AdventurerId id);

    void save(Adventurer adventurer);
}
```

Aceptar una misión. El comando y el puerto:

application/port/in/AcceptQuestCommand.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.QuestId;

public record AcceptQuestCommand(QuestId questId,
                                 AdventurerId adventurerId) {
}
```

application/port/in/AcceptQuestUseCase.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.Quest;

public interface AcceptQuestUseCase {

    Quest accept(AcceptQuestCommand command);
}
```

Y el servicio, que es una receta: leer la misión, leer el aventurero, contar, pedir a la misión que acepte, guardar. Observa qué *no* hace: no compara rangos ni cuenta hasta tres. Eso es `Quest`.

application/service/AcceptQuestService.java

```
package com.guildboard.application.service;

import com.guildboard.application.port.in.AcceptQuestCommand;
import com.guildboard.application.port.in.AcceptQuestUseCase;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.Adventurer;
```

```
import com.guildboard.domain.DomainError;
import com.guildboard.domain.DomainException;
import com.guildboard.domain.Quest;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class AcceptQuestService implements AcceptQuestUseCase {

    private final QuestRepository quests;
    private final AdventurerRepository adventurers;

    @Override
    public Quest accept(AcceptQuestCommand command) {
        Quest quest = quests.findById(command.questId())
            .orElseThrow(() -> new DomainException(
                new DomainError.QuestNotFound(command.questId())));
        Adventurer adventurer = adventurers.findById(command.adventurerId())
            .orElseThrow(() -> new DomainException(
                new DomainError.AdventurerNotFound(
                    command.adventurerId())));
        int active = quests.countActiveBy(adventurer.getId());
        quest.accept(adventurer.getId(), adventurer.getRank(), active);
        quests.save(quest);
        return quest;
    }
}
```

Dar experiencia es el caso de uso más pequeño, y es el que usa el siguiente:

application/port/in/GrantExperienceCommand.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.AdventurerId;

public record GrantExperienceCommand(AdventurerId adventurerId,
                                     int points) {
}
```

application/port/in/GrantExperienceUseCase.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.Adventurer;

public interface GrantExperienceUseCase {

    Adventurer grant(GrantExperienceCommand command);
}
```

Su servicio, `GrantExperienceService`, lee el aventurero, llama a `gainExperience` y lo guarda; está entero en la solución del ejercicio 5.5. Completar una misión es el caso de uso que *llama a otro*. La regla R5 («la experiencia de una misión es su recompensa en oro») no pertenece a `Quest` ni a `Adventurer`: nace de la relación entre los dos, y la coordina el servicio, llamando al otro caso de uso por su *puerto*, no a la clase concreta:

application/service/CompleteQuestService.java

```java
package com.guildboard.application.service;

import com.guildboard.application.port.in.CompleteQuestCommand;
import com.guildboard.application.port.in.CompleteQuestUseCase;
import com.guildboard.application.port.in.GrantExperienceCommand;
import com.guildboard.application.port.in.GrantExperienceUseCase;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.DomainError;
import com.guildboard.domain.DomainException;
import com.guildboard.domain.Quest;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class CompleteQuestService implements CompleteQuestUseCase {

    private final QuestRepository quests;
    private final GrantExperienceUseCase grantExperience;

    @Override
    public Quest complete(CompleteQuestCommand command) {
        Quest quest = quests.findById(command.questId())
            .orElseThrow(() -> new DomainException(
                new DomainError.QuestNotFound(command.questId())));
        quest.complete();
        AdventurerId assignee = quest.getAssignedTo().orElseThrow(
            () -> new IllegalStateException("accepted quest without "
                + "assignee: " + quest.getId().value()));
        quests.save(quest);
        int points = quest.getReward().gold();
        if (points > 0) {
            grantExperience.grant(
                new GrantExperienceCommand(assignee, points));
        }
        return quest;
    }
}
```

Dos detalles. Primero, el orden: `quest.complete()` va antes de leer al aventurero asignado, porque es el que rechaza las misiones que no están `ACCEPTED` (en una misión sin aceptar `getAssignedTo()` está vacío, y queremos el error de transición, no un fallo extraño). Segundo, la guarda `points > 0`: una misión puede tener recompensa cero, y `gainExperience` rechaza los puntos que no son positivos; sin esa guarda, completar una misión gratuita fallaría *después* de haberla guardado. Y fíjate en que ese caso de uso llama a otro y guarda dos agregados distintos en dos pasos. La sección 5.7 explica qué cuesta eso.

El comando y el puerto de `CompleteQuest`, el de registrar y el de consultar un aventurero siguen el mismo patrón y están en las soluciones. Como siempre, el núcleo no lleva `@Service`: las piezas nuevas se registran en la raíz de composición. Los métodos `@Bean` que se añaden a `BeanConfig`:

config/BeanConfig.java (fragmento: beans nuevos)

```
@Bean
public AdventurerRepository adventurerRepository() {
    return new InMemoryAdventurerRepository();
}

@Bean
public GrantExperienceUseCase grantExperienceUseCase(
        AdventurerRepository adventurerRepository) {
    return new GrantExperienceService(adventurerRepository);
}

@Bean
public AcceptQuestUseCase acceptQuestUseCase(
        QuestRepository questRepository,
        AdventurerRepository adventurerRepository) {
    return new AcceptQuestService(questRepository, adventurerRepository);
}

@Bean
public CompleteQuestUseCase completeQuestUseCase(
        QuestRepository questRepository,
        GrantExperienceUseCase grantExperienceUseCase) {
    return new CompleteQuestService(questRepository,
                                    grantExperienceUseCase);
}
```

Observa cómo el contenedor resuelve `CompleteQuest`: su segundo parámetro es el *puerto* `GrantExperienceUseCase`, y el contenedor entrega el bean que ya construyó con ese tipo. Los beans de registrar y consultar aventureros están en la solución completa.

El adaptador web. El endpoint de aceptar es `POST /quests/{id}/accept`, con el aventurero en el cuerpo. Un `record` para la petición y un método nuevo en el controlador de la misión, que sigue siendo un objeto humilde:

adapter/in/web/AcceptQuestRequest.java

```
package com.guildboard.adapter.in.web;

public record AcceptQuestRequest(String adventurerId) {
}
```

adapter/in/web/QuestController.java (fragmento: aceptar)

```
@PostMapping("/{id}/accept")
public QuestResponse accept(@PathVariable String id,
                            @RequestBody AcceptQuestRequest request) {
    Quest quest = acceptQuest.accept(new AcceptQuestCommand(
        new QuestId(id), new AdventurerId(request.adventurerId())));
    return QuestResponse.from(quest);
}
```

El controlador gana un campo `AcceptQuestUseCase acceptQuest`. Devolver el *record* directamente da un 200 con el cuerpo en JSON. Los endpoints de completar misión y de registrar

aventureros siguen el mismo molde y los escribes en el ejercicio de proyecto. `QuestResponse` gana un campo `assignedTo`, el aventurero que aceptó (o `null`), para poder ver el resultado.

5.6 Probar el dominio sin Spring

Un dominio rico se prueba bien porque es Java normal: se construyen los objetos con `new` (o con sus métodos estáticos), se llama a un método y se mira el resultado. No hace falta arrancar nada. Esa es la prueba de que la regla de dependencia hizo su trabajo. Para escribirlas necesitas un marco de pruebas y una librería de aserciones.

☞ AssertJ con el starter de pruebas de Boot

JUnit ya lo tienes desde el capítulo 3: escribiste `@Test` y ejecutaste las pruebas con Gradle para `ArchitectureTest`. Lo que falta son las aserciones. Spring Boot agrupa las dependencias de prueba en un único starter, `spring-boot-starter-test`. Su descriptor en Maven Central para la versión 4.1.1 incluye `junit-jupiter` y `assertj-core` (las aserciones)[52], entre otras herramientas que no usaremos. Comprobado el 8 de octubre de 2026 en Maven Central y en Spring Initializr: un proyecto generado sin dependencias web pide justo esa dependencia, y uno con `Spring Web` pide `spring-boot-starter-webmvc-test`, que añade herramientas de prueba de la capa web que este curso no enseña. Para el dominio basta el starter general.

El starter sustituye a la línea explícita de `junit-jupiter` del capítulo 3, porque ya la trae. Se conservan el lanzador y `archunit`, con su versión. Edita el bloque `dependencies` de `build.gradle`; el resto del archivo queda como estaba.

`build.gradle (solo el bloque dependencies)`

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'

    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

`useJUnitPlatform()` sigue en el bloque `test`, sin tocar. Con el cambio, `ArchitectureTest` debe seguir pasando: es la comprobación de que el starter no ha roto nada. La carpeta `src/test/java` ya existe por esa prueba; solo crea el subpaquete de las del dominio:

```
$ mkdir -p src/test/java/com/guildboard/domain
```

Las pruebas del dominio van en el subpaquete `domain` de `src/test/java/com/guildboard/`, que replica el del código que prueban. Cada una es independiente: JUnit crea una instancia nueva de la clase para cada método.

Lo mínimo de AssertJ: se empieza con `assertThat(valor)` y se encadena lo que debe cumplir, y se lee casi como una frase. Para probar que algo lanza un error está `assertThatThrownBy`, que recibe una expresión que debe fallar. Un ejemplo con el ciclo de vida de la misión:

src/test/java/com/guildboard/domain/QuestTransitionTest.java

```java
package com.guildboard.domain;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

import org.junit.jupiter.api.Test;

class QuestTransitionTest {

    private static final QuestId ID = new QuestId("q-1");
    private static final AdventurerId HERO = new AdventurerId("a-1");

    private static Quest draft() {
        return Quest.draft(ID, "Limpiar el s^^c3^^b3tano", new Reward(150),
                           Rank.NOVICE);
    }

    @Test
    void publishedQuestIsAcceptedByQualifiedAdventurer() {
        Quest quest = draft();
        quest.publish();

        quest.accept(HERO, Rank.NOVICE, 0);

        assertThat(quest.getStatus()).isEqualTo(QuestStatus.ACCEPTED);
        assertThat(quest.getAssignedTo()).contains(HERO);
    }

    @Test
    void cannotPublishTwice() {
        Quest quest = draft();
        quest.publish();

        assertThatThrownBy(quest::publish)
            .isInstanceOfSatisfying(DomainException.class, e ->
                assertThat(e.getError()).isEqualTo(
                    new DomainError.InvalidTransition(
                        ID, QuestStatus.PUBLISHED, QuestStatus.DRAFT)));
    }
}
```

Dos aserciones merecen explicación. `contains(HERO)` sobre un `Optional` comprueba que está presente y que vale eso. Y `isInstanceOfSatisfying` comprueba que la excepción es de ese tipo y, además, ejecuta una comprobación sobre ella: aquí, que el error tipado lleva los datos esperados. Comparar los errores con `isEqualTo` funciona porque son *records*, y los *records* tienen igualdad por valor. Por eso la prueba no necesita mirar el texto del mensaje. Se ejecuta junto con `ArchitectureTest`, como en el capítulo 3: las dos pruebas nuevas deben pasar, y si alguna falla Gradle indica cuál y dónde está el informe en HTML (`build/reports/tests/test/index.html`).

Las pruebas no son el protagonista

En este libro las pruebas son una herramienta para ver que una regla se cumple, no un tema en sí. Solo vas a escribirlas de clases sueltas, sin arrancar Spring. Las pruebas de la web o de la base de datos no se enseñan.

5.7 Lo que este diseño no garantiza

Hay que decir en voz alta lo que el diseño no resuelve. Son dos cosas, y el curso las reconoce y las deja así.

La carrera entre contar y guardar. El límite de tres misiones activas se comprueba con un recuento que `AcceptQuestService` lee de `countActiveBy`, antes de que la misión se guarde. Contar y guardar son dos operaciones separadas: el proyecto no las envuelve en una transacción (`@Transactional`, cuando exista, estará solo en el adaptador de persistencia del capítulo 8, y cada llamada al repositorio es una unidad suelta). Dos peticiones simultáneas pueden intercalarse así:

Paso	Petición A (misión 3)	Petición B (misión 4)
1	cuenta: 2 activas	
2		cuenta: 2 activas
3	`accept` con 2: correcto; guarda	
4		`accept` con 2: correcto; guarda

El aventurero termina con cuatro misiones activas, y ninguna de las dos peticiones ha hecho nada «mal»: cada una vio un recuento verdadero en su momento. Con un solo hilo no se ve nunca. Con el repositorio en memoria del capítulo 4 es posible en teoría (Tomcat atiende cada petición en un hilo) y improbable en una prueba manual; con PostgreSQL en el capítulo 8 es perfectamente posible. Arreglarlo exigiría un bloqueo (por ejemplo, de la fila del aventurero mientras se cuenta y se guarda) o una restricción en la base de datos que lo impidiera, o repartir la decisión entre servicios con *sagas* [12]. Ninguna entra en el curso. Lo que sí hace el capítulo es que el riesgo conste y no quede escondido.

El paso a medias entre completar y dar experiencia. `CompleteQuestService` guarda la misión como `COMPLETED` y después llama a `GrantExperienceUseCase`. Si entre las dos cosas falla algo (la aplicación se cae, o el segundo guardado lanza una excepción), la misión queda completada y el aventurero sin su experiencia. Reintentar `complete` no sirve: la misión ya no está `ACCEPTED` y el segundo intento sería rechazado como transición inválida. Invertir el orden solo cambia la inconsistencia: experiencia dada y misión sin completar. Dos agregados, dos guardados, ninguna transacción que los ate; no se resuelve en el curso. Más adelante, al separar el sistema en servicios, esta vía desaparece y la experiencia pasa a viajar como un evento, con sus propios riesgos que se explican entonces.

Las dos cosas tienen la misma raíz: un agregado es una frontera de consistencia y lo que cruza fronteras es consistencia *eventual* ([7], [25]). Dos agregados que cambian en una misma operación no los protege ningún agregado.

5.7.1 El reparto de reglas que se hereda

Lo que queda en cada sitio al terminar H2 importa para el capítulo 9, cuando las misiones y los aventureros dejen de vivir en el mismo proceso:

- **En `Quest`**: estado, rango (con el rango como dato), límite (con el recuento como dato). No sabe nada de aventureros.
- **En `Adventurer`**: experiencia y rango. No sabe nada de misiones.
- **En los servicios**: leer el rango, contar y orquestar. Nada más.

Como `accept` ya recibe datos y no objetos, el día que el rango viva en otro servicio solo cambia *de dónde* sale ese `Rank`, no `Quest`.

5.8 Estructura: sin cambios en el árbol

Este capítulo **no cambia la estructura**. Todo el código nuevo cae en carpetas que ya existen. `src/test/java/com/guildboard/` ya existe desde la verificación de estructura 1 (capítulo 3), por `ArchitectureTest`; las pruebas del dominio se organizan en un subpaquete `domain` que replica el del código. Lo nuevo, marcado con `[N]`, y lo modificado, con `[M]`:

Árbol tras el capítulo 5 (sin cambios de estructura)

```
guildboard/
|-- build.gradle                    [M] starter de pruebas
|-- docs/dominio.md
`-- src/
    |-- main/java/com/guildboard/
    |   |-- domain/                 [N] Adventurer, DomainError,
    |   |                           DomainException; [M] Quest
    |   |-- application/
    |   |   |-- port/in/            [N] cuatro casos de uso y sus comandos
    |   |   |-- port/out/           [N] AdventurerRepository;
    |   |   |                       [M] QuestRepository
    |   |   `-- service/            [N] cinco servicios
    |   |-- adapter/
    |   |   |-- in/web/             [N] AdventurerController y DTO;
    |   |   |                       [M] QuestController, QuestResponse
    |   |   `-- out/memory/         [N] InMemoryAdventurerRepository;
    |   |                           [M] InMemoryQuestRepository
    |   `-- config/                 [M] BeanConfig
    `-- test/java/com/guildboard/
        |-- ArchitectureTest.java
        `-- domain/                 [N] pruebas del dominio
```

Compruébalo con Git: `git status` debe mostrar archivos nuevos solo en esas carpetas (y en `src/test`), y ningún directorio fuera de la lista.

5.9 En el proyecto: hito H2

Hito 2: un dominio con reglas

Al terminar el ejercicio 5.5, GuildBoard tiene aventureros con rango y experiencia, y las reglas de rango y de límite viven en el dominio. Puedes registrar un aventurero, publicar una misión y aceptarla (con rechazo si el rango no alcanza o si ya lleva tres), y completarla para darle experiencia según su recompensa. Los errores de dominio son tipos, aunque la web aún los muestra como `500` hasta el capítulo 6. Hay además pruebas del dominio con

JUnit y AssertJ. El proyecto queda con estos archivos nuevos:

Archivos nuevos de GuildBoard tras el capítulo 5

```
domain/        Adventurer, DomainError, DomainException
port/in/       RegisterAdventurer{Command,UseCase}, GetAdventurerUseCase,
               AcceptQuest{Command,UseCase},
               CompleteQuest{Command,UseCase},
               GrantExperience{Command,UseCase}
port/out/      AdventurerRepository
service/       RegisterAdventurerService, GetAdventurerService,
               AcceptQuestService, CompleteQuestService,
               GrantExperienceService
adapter/in/web/    AdventurerController, RegisterAdventurerRequest,
                   AdventurerResponse, AcceptQuestRequest
adapter/out/memory/  InMemoryAdventurerRepository
src/test/java/com/guildboard/domain/   QuestTransitionTest, QuestAcceptTest
```

5.9.1 Probar el hito con curl

Este es el escenario con el que comprobarás el ejercicio de proyecto. Arranca con `./gradlew bootRun` y, en otra terminal, registra a Mara. Los identificadores son de ejemplo (usa los que devuelva tu aplicación) y las respuestas se muestran partidas en varias líneas, aunque la aplicación las devuelve en una.

```
$ curl -s -X POST http://localhost:8080/adventurers \
    -H "Content-Type: application/json" -d '{"name":"Mara"}'
{"id":"7d1c4e52-0b8a-4f3e-9c61-5a2b8e0d7f14","name":"Mara",
 "rank":"NOVICE","experience":0}
$ MARA=7d1c4e52-0b8a-4f3e-9c61-5a2b8e0d7f14
```

Publica una misión de rango `ADEPT` y otra de rango `NOVICE` con 150 de oro, y guarda sus ids en variables de la shell:

```
$ curl -s -X POST http://localhost:8080/quests \
    -H "Content-Type: application/json" \
    -d '{"title":"Domar al drag^^c3^^b3n","reward":500,"requiredRank":"ADEPT"}'
{"id":"c91e7a30-4d5b-4c28-8a0f-6e3b2d1f9a45",
 "title":"Domar al drag^^c3^^b3n","reward":500,"requiredRank":"ADEPT",
 "status":"PUBLISHED","assignedTo":null}
$ DRAGON=c91e7a30-4d5b-4c28-8a0f-6e3b2d1f9a45
$ curl -s -X POST http://localhost:8080/quests \
    -H "Content-Type: application/json" \
    -d '{"title":"Limpiar el s^^c3^^b3tano","reward":150,"requiredRank":"NOVICE"}
    ↪ '
{"id":"b2a6f0d4-8c3e-4a17-9e55-1d7c3b6a9f02",
 "title":"Limpiar el s^^c3^^b3tano","reward":150,"requiredRank":"NOVICE",
 "status":"PUBLISHED","assignedTo":null}
$ CELLAR=b2a6f0d4-8c3e-4a17-9e55-1d7c3b6a9f02
```

Mara es `NOVICE`, así que el dragón la rechaza; el sótano, no. Al completarlo, sus 150 puntos de experiencia la hacen `ADEPT`:

```
$ curl -i -X POST http://localhost:8080/quests/$DRAGON/accept \
    -H "Content-Type: application/json" \
    -d '{"adventurerId":"'$MARA'"}'
HTTP/1.1 500
$ curl -s -X POST http://localhost:8080/quests/$CELLAR/accept \
    -H "Content-Type: application/json" \
    -d '{"adventurerId":"'$MARA'"}'
{"id":"b2a6f0d4-8c3e-4a17-9e55-1d7c3b6a9f02",
 "title":"Limpiar el s^^c3^^b3tano","reward":150,"requiredRank":"NOVICE",
 "status":"ACCEPTED","assignedTo":"7d1c4e52-0b8a-4f3e-9c61-5a2b8e0d7f14"}
$ curl -s -X POST http://localhost:8080/quests/$CELLAR/complete
{"id":"b2a6f0d4-8c3e-4a17-9e55-1d7c3b6a9f02",
 "title":"Limpiar el s^^c3^^b3tano","reward":150,"requiredRank":"NOVICE",
 "status":"COMPLETED","assignedTo":"7d1c4e52-0b8a-4f3e-9c61-5a2b8e0d7f14"}
$ curl -s http://localhost:8080/adventurers/$MARA
{"id":"7d1c4e52-0b8a-4f3e-9c61-5a2b8e0d7f14","name":"Mara",
 "rank":"ADEPT","experience":150}
```

La consola de la aplicación muestra en el rechazo la excepción `DomainException: rank NOVICE cannot take a quest that requires ADEPT`. Para el límite, publica cuatro misiones `NOVICE` y acepta tres con Mara (`200` las tres); la cuarta da `500` y en la consola `DomainException: adventurer has 3 active quests, the limit is 3`. Un `GET` a esa misión sigue mostrándola `PUBLISHED` y sin aventurero.

5.10 Ejercicios

Los ejercicios parten del código del capítulo. Los dos que modifican el proyecto (5.3 y 5.4) se quedan en él: si haces antes el ejercicio de proyecto 5.5, que reúne todo el código, ya los tienes hechos, y la solución del proyecto incluye sus cambios. Empieza cada ejercicio con el árbol de trabajo limpio (`git status` sin nada pendiente) y termina con un *commit*.

Ejercicio 5.1: Invariantes y errores de Quest.accept ★★☆☆☆

Lee este método, que es el `accept` del capítulo, y responde sin ejecutar nada. El método `requireStatus` lanza `DomainException` con el error `InvalidTransition`.

Quest.accept

```
public void accept(AdventurerId adventurerId, Rank adventurerRank,
                   int activeQuests) {
    Objects.requireNonNull(adventurerId, "adventurerId");
    Objects.requireNonNull(adventurerRank, "adventurerRank");
    if (activeQuests < 0) {
        throw new IllegalArgumentException("activeQuests < 0");
    }
    requireStatus(QuestStatus.PUBLISHED);
    if (!adventurerRank.canTake(requiredRank)) {
        throw new DomainException(
            new DomainError.RankTooLow(adventurerRank, requiredRank));
    }
    if (activeQuests >= MAX_ACTIVE_QUESTS) {
        throw new DomainException(new DomainError.TooManyActiveQuests(
            activeQuests, MAX_ACTIVE_QUESTS));
    }
    assignedTo = adventurerId;
    status = QuestStatus.ACCEPTED;
}
```

1. Nombra las tres invariantes de negocio que comprueba el método y di qué variante de `DomainError` lanza cada una.
2. ¿Qué clase de invariante es cada una (de valor, de transición o de contexto)?
3. Una misión `PUBLISHED` de rango `ADEPT` recibe `accept(a1, Rank.NOVICE, 5)`. ¿Qué error sale, el del rango o el del límite? ¿Y en qué estado queda la misión?
4. ¿Por qué `activeQuests < 0` lanza `IllegalArgumentException` y no un `DomainException`?
5. ¿Hay alguna regla de las tres que `Quest` no pueda garantizar por sí sola? ¿Cuál y por qué?

Está bien si identificas las tres invariantes (estado `PUBLISHED`, rango suficiente y menos de 3 activas) con sus variantes `InvalidTransition`, `RankTooLow` y `TooManyActiveQuests`; dices que sale el error del rango y que la misión no cambia; y señalas el límite de misiones como la regla que depende de un dato de fuera.

Lectura de código ▪ solución en la pág. 129

Ejercicio 5.2: Pruebas de rango y de límite ★★☆☆☆

Escribe las pruebas de `Quest.accept` para las dos reglas de contexto. Parte del proyecto con AssertJ instalado (sección 5.6). Usas JUnit (capítulo 3) y AssertJ (este capítulo).

1. Crea `QuestAcceptTest` en `src/test/java/com/guildboard/domain/`.
2. Prueba 1: un aventurero `NOVICE` no puede aceptar una misión de rango `ADEPT`. Comprueba el error exacto (`RankTooLow`) y que la misión sigue `PUBLISHED` y sin aventurero asignado.
3. Prueba 2: con 3 misiones activas, no puede aceptar la cuarta. Comprueba el error (`TooManyActiveQuests`) y que la misión no cambia.
4. Prueba 3: con 2 activas sí puede, y la misión queda `ACCEPTED` con ese aventurero.

5. Ejecuta `./gradlew test` y haz un *commit*.

Está bien si las tres pruebas pasan, ninguna arranca Spring, y si rompes a propósito `MAX_ACTIVE_QUESTS` (por ejemplo, lo pones a 4) la prueba 2 falla.

Pista: *reutiliza el patrón de `QuestTransitionTest`: un método auxiliar que devuelva una misión ya publicada y `isInstanceOfSatisfying` para comparar el error.*

Test ▪ solución en la pág. 130

Ejercicio 5.3: Subir de rango con la experiencia ★★★☆☆

Completa `Adventurer.gainExperience` para que el rango suba según la tabla de umbrales del capítulo (`ADEPT` desde 100 puntos y `VETERAN` desde 300).

1. Representa los umbrales como una tabla (un `Map` de `Rank` a `int`), no como números sueltos dentro de `if`.
2. `gainExperience` pasa a devolver el `Rank` resultante.
3. El rango nunca baja, y una sola llamada puede saltar dos rangos (de 0 a 350 puntos, de `NOVICE` a `VETERAN`).
4. Los puntos no positivos siguen lanzando `IllegalArgumentException`.
5. Comprueba los casos de la tabla de abajo con una prueba JUnit y haz un *commit*.

Experiencia antes	Puntos	Rango devuelto
0	99	`NOVICE`
0	100	`ADEPT`
99	1	`ADEPT`
0	350	`VETERAN`

Está bien si los cuatro casos dan lo indicado, `GrantExperienceService` sigue compilando sin cambios y `./gradlew build` termina con `BUILD SUCCESSFUL`.

Pista: *recorre `Rank.values()`, que devuelve las constantes en el orden de declaración, y quédate con la última cuyo umbral hayas alcanzado.*

Práctica ▪ solución en la pág. 135

Ejercicio 5.4: Un error nuevo: misión no publicada ★★★☆☆

Hoy, aceptar una misión que no está publicada lanza `InvalidTransition`, el mismo error genérico de cualquier transición. Un cliente de la API querría distinguirlo. Añade el error `QuestNotPublished(QuestId questId, QuestStatus actual)`.

1. Declara la variante en `DomainError` *sin tocar nada más*.
2. Compila con `./gradlew compileJava`. Copia el error del compilador y di qué dos sitios señala.
3. Completa los dos `switch`. Elige un código y un mensaje con sentido.
4. Cambia `Quest.accept` para que, si la misión no está `PUBLISHED`, lance el error nuevo en lugar de `InvalidTransition`. `publish` y `complete` no cambian.
5. Añade una prueba en `QuestAcceptTest` (o en un archivo nuevo) que compruebe el error al aceptar una misión en borrador. Haz un *commit*.

Está bien si el compilador marca exactamente los dos `switch` antes de que los completes,

no has usado ningún `default`, la prueba nueva pasa y las anteriores siguen pasando.

Pista: *el mensaje del compilador dice «the switch expression does not cover all possible input values» (o similar) y la línea del* `switch`*.*

Práctica ▪ solución en la pág. 138

Ejercicio 5.5: Hito H2: aventureros, aceptar y completar ★★★★☆

Parte del proyecto tal como lo dejó el capítulo 4 (H1 completo). Implementa H2.

1. Instala el starter de pruebas, que añade AssertJ (sección 5.6).
2. Dominio: `Adventurer`, `DomainError`, `DomainException` y los cambios en `Quest` (incluidos los de los ejercicios 5.3 y 5.4).
3. Aplicación: los puertos de salida, los cuatro casos de uso de H2 con sus comandos y servicios, más `GetAdventurerUseCase` para consultar un aventurero (decisión del curso: sin ella no podrías ver su experiencia).
4. Adaptadores: repositorio de aventureros en memoria, `countActiveBy` en el de misiones, `AdventurerController`, y los endpoints `POST /quests/{id}/accept` y `POST /quests/{id}/complete`.
5. Registra todo en `BeanConfig`.
6. Pruébalo con `curl`: un `NOVICE` rechazado en una misión `ADEPT`, cuarta misión rechazada y una misión completada que sube el rango.
7. Ejecuta `./gradlew build` y haz un *commit* con el mensaje `H2: dominio rico (invariantes, errores tipados, casos de uso)`.

Está bien si: `./gradlew build` termina con `BUILD SUCCESSFUL` (compila y pasan las pruebas, incluida `ArchitectureTest`); `domain` y `application` no importan Spring; ningún servicio modifica más de un agregado ni lleva `@Service`; las firmas de `Quest.accept(AdventurerId, Rank, int)` y `GrantExperienceUseCase.grant(GrantExperienceCommand)` son las del capítulo; y el escenario de la sección 5.9.1 da los resultados indicados (un aventurero registrado por la API, que acepta y completa una misión, sube de rango).

Pista: *construye de dentro afuera: dominio, puertos, servicios, adaptadores y, al final,* `BeanConfig`*. Compila después de cada capa.*

Proyecto ▪ solución en la pág. 142

5.11 Resumen

Resumen

- Una invariante es una condición siempre cierta. Las hay de valor (en el constructor), de transición (en el método que cambia el estado) y de contexto (necesitan datos de fuera). Los agregados ricos las hacen cumplir; los servicios no deciden reglas.
- `Rank` y `Reward` son objetos de valor inmutables; el orden de las constantes de `Rank` es una regla de negocio. `Adventurer` no tiene *setters*: su experiencia solo cambia con `gainExperience`.
- Los errores de dominio son una jerarquía sellada (`DomainError`) que viaja dentro de

una `DomainException`. Los `switch` sobre ella son exhaustivos y sin `default`: añadir una variante rompe la compilación hasta que se trata en todos los sitios.

- Datos mal formados lanzan `IllegalArgumentException`; reglas de negocio violadas, `DomainException`; invariantes rotas por un fallo del programa, `IllegalStateException`.
- `Quest.accept(AdventurerId, Rank, int)` comprueba estado, rango y límite antes de cambiar nada. El rango y el recuento llegan como datos, así que `Quest` no conoce otros agregados.
- Cada caso de uso modifica como máximo un agregado. `CompleteQuest` llama a `GrantExperience` por su puerto. El núcleo sigue sin `@Service`.
- Con `spring-boot-starter-test` se prueba el dominio con JUnit (ya conocido) y AssertJ, sin Spring. El starter sustituye a la línea de `junit-jupiter` y se declara con `testImplementation`; `archunit` se conserva.
- No se garantiza que contar y guardar sean atómicos (dos aceptaciones simultáneas pueden pasar de 3 activas) ni que completar y dar experiencia lo sean (puede quedar una misión completada sin experiencia). El curso lo reconoce y no lo resuelve.
- La estructura no ha cambiado: se rellenan las carpetas previstas; `src/test/java/com/guildboard/` ya existía por `ArchitectureTest`. H2 está completo.

Soluciones de los ejercicios

Solución del ejercicio 5.1 enunciado en la pág. 125

1 y 2. Las tres invariantes de negocio, con su error y su clase:

Invariante	Error	Clase
Solo se acepta una misión `PUBLISHED`	`InvalidTransition`	de transición
El rango alcanza el exigido	`RankTooLow`	de contexto (rango)
Menos de 3 misiones activas	`TooManyActiveQuests`	de contexto (recuento)

3. Sale `RankTooLow(NOVICE, ADEPT)`: el rango se comprueba antes que el límite, así que el `5` de las misiones activas ni se mira. La misión queda como estaba (`PUBLISHED`, sin aventurero), porque las asignaciones van después de todas las comprobaciones.
4. Un recuento negativo no es una situación del negocio, es un error de quien llama: es un argumento mal formado, y para eso Java ya tiene `IllegalArgumentException`. Los `DomainException` se reservan para reglas del negocio con datos bien formados.
5. El límite de 3 misiones activas. `Quest` se fía del número que recibe; no puede saber si es verdad ni si otra petición está aceptando a la vez otra misión del mismo aventurero. El rango también llega como dato, pero tiene una sola fuente (el `Adventurer`), y el riesgo de que cambie entre medias es menor.
Errores típicos: responder que sale el error del límite (el orden de las comprobaciones lo decide el código, no la gravedad); decir que la misión queda `ACCEPTED` «porque pasó por las asignaciones» (no llega a ellas); y creer que `Quest` puede garantizar el recuento solo porque lo comprueba.

Solución del ejercicio 5.2 enunciado en la pág. 126

Código que se prueba: el dominio del capítulo. Los tres archivos de dominio (son los de las secciones anteriores):

domain/DomainError.java

```
package com.guildboard.domain;

public sealed interface DomainError {

    record QuestNotFound(QuestId questId) implements DomainError { }

    record AdventurerNotFound(AdventurerId adventurerId)
            implements DomainError { }

    record InvalidTransition(QuestId questId, QuestStatus actual,
                             QuestStatus expected)
            implements DomainError { }

    record RankTooLow(Rank adventurerRank, Rank requiredRank)
            implements DomainError { }

    record TooManyActiveQuests(int active, int limit)
            implements DomainError { }

    default String code() {
        return switch (this) {
            case QuestNotFound e -> "QUEST_NOT_FOUND";
            case AdventurerNotFound e -> "ADVENTURER_NOT_FOUND";
            case InvalidTransition e -> "INVALID_TRANSITION";
            case RankTooLow e -> "RANK_TOO_LOW";
            case TooManyActiveQuests e -> "TOO_MANY_ACTIVE_QUESTS";
        };
    }
}
```

domain/DomainException.java

```
package com.guildboard.domain;

public final class DomainException extends RuntimeException {

    private final DomainError error;

    public DomainException(DomainError error) {
        super(describe(error));
        this.error = error;
    }

    public DomainError getError() {
        return error;
    }

    private static String describe(DomainError error) {
        return switch (error) {
            case DomainError.QuestNotFound(QuestId id) ->
                "quest not found: " + id.value();
            case DomainError.AdventurerNotFound(AdventurerId id) ->
                "adventurer not found: " + id.value();
            case DomainError.InvalidTransition(QuestId id,
                    QuestStatus actual, QuestStatus expected) ->
                "quest " + id.value() + " is " + actual
                    + ", expected " + expected;
            case DomainError.RankTooLow(Rank have, Rank need) ->
                "rank " + have + " cannot take a quest that requires "
                    + need;
            case DomainError.TooManyActiveQuests(int active, int limit) ->
                "adventurer has " + active
                    + " active quests, the limit is " + limit;
        };
    }
}
```

domain/Quest.java (1/2)

```java
package com.guildboard.domain;

import java.util.Objects;
import java.util.Optional;
import lombok.EqualsAndHashCode;
import lombok.Getter;

@Getter
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
public final class Quest {

    public static final int MAX_ACTIVE_QUESTS = 3;

    @EqualsAndHashCode.Include
    private final QuestId id;
    private final String title;
    private final Reward reward;
    private final Rank requiredRank;
    private QuestStatus status;
    private AdventurerId assignedTo;        // null hasta aceptar

    private Quest(QuestId id, String title, Reward reward,
                  Rank requiredRank) {
        this.id = id;
        this.title = title;
        this.reward = reward;
        this.requiredRank = requiredRank;
        this.status = QuestStatus.DRAFT;
    }

    public static Quest draft(QuestId id, String title, Reward reward,
                              Rank requiredRank) {
        Objects.requireNonNull(id, "id");
        Objects.requireNonNull(reward, "reward");
        Objects.requireNonNull(requiredRank, "requiredRank");
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("title must not be blank");
        }
        return new Quest(id, title.strip(), reward, requiredRank);
    }
```

Continuación del listado anterior:

domain/Quest.java (2/2)

```
    public void publish() {
        requireStatus(QuestStatus.DRAFT);
        status = QuestStatus.PUBLISHED;
    }

    public void accept(AdventurerId adventurerId, Rank adventurerRank,
                       int activeQuests) {
        Objects.requireNonNull(adventurerId, "adventurerId");
        Objects.requireNonNull(adventurerRank, "adventurerRank");
        if (activeQuests < 0) {
            throw new IllegalArgumentException(
                "activeQuests must not be negative: " + activeQuests);
        }
        requireStatus(QuestStatus.PUBLISHED);
        if (!adventurerRank.canTake(requiredRank)) {
            throw new DomainException(
                new DomainError.RankTooLow(adventurerRank, requiredRank));
        }
        if (activeQuests >= MAX_ACTIVE_QUESTS) {
            throw new DomainException(new DomainError.TooManyActiveQuests(
                activeQuests, MAX_ACTIVE_QUESTS));
        }
        assignedTo = adventurerId;
        status = QuestStatus.ACCEPTED;
    }

    public void complete() {
        requireStatus(QuestStatus.ACCEPTED);
        status = QuestStatus.COMPLETED;
    }

    // Escrito a mano: Lombok no genera un getter si ya existe uno con
    // ese nombre, y as^^c3^^ad se devuelve un Optional en lugar de null.
    public Optional<AdventurerId> getAssignedTo() {
        return Optional.ofNullable(assignedTo);
    }

    private void requireStatus(QuestStatus expected) {
        if (status != expected) {
            throw new DomainException(
                new DomainError.InvalidTransition(id, status, expected));
        }
    }
}
```

La prueba:

src/test/java/com/guildboard/domain/QuestAcceptTest.java (1/2)

```java
package com.guildboard.domain;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

import org.junit.jupiter.api.Test;

class QuestAcceptTest {

    private static final AdventurerId HERO = new AdventurerId("a-1");

    private static Quest publishedQuest(Rank requiredRank) {
        Quest quest = Quest.draft(new QuestId("q-1"), "Limpiar el s^^c3^^
    ↪ b3tano",
                                  new Reward(150), requiredRank);
        quest.publish();
        return quest;
    }

    @Test
    void rejectsAdventurerWhoseRankIsTooLow() {
        Quest quest = publishedQuest(Rank.ADEPT);

        assertThatThrownBy(() -> quest.accept(HERO, Rank.NOVICE, 0))
            .isInstanceOfSatisfying(DomainException.class, e ->
                assertThat(e.getError()).isEqualTo(
                    new DomainError.RankTooLow(Rank.NOVICE, Rank.ADEPT)));
        assertThat(quest.getStatus()).isEqualTo(QuestStatus.PUBLISHED);
        assertThat(quest.getAssignedTo()).isEmpty();
    }
```

Continuación del listado anterior:

src/test/java/com/guildboard/domain/QuestAcceptTest.java (2/2)

```
    @Test
    void rejectsFourthActiveQuest() {
        Quest quest = publishedQuest(Rank.NOVICE);

        assertThatThrownBy(() -> quest.accept(HERO, Rank.VETERAN, 3))
            .isInstanceOfSatisfying(DomainException.class, e ->
                assertThat(e.getError()).isEqualTo(
                    new DomainError.TooManyActiveQuests(3, 3)));
        assertThat(quest.getStatus()).isEqualTo(QuestStatus.PUBLISHED);
        assertThat(quest.getAssignedTo()).isEmpty();
    }

    @Test
    void acceptsWhenAdventurerHasTwoActiveQuests() {
        Quest quest = publishedQuest(Rank.NOVICE);

        quest.accept(HERO, Rank.NOVICE, 2);

        assertThat(quest.getStatus()).isEqualTo(QuestStatus.ACCEPTED);
        assertThat(quest.getAssignedTo()).contains(HERO);
    }
}
```

Por qué funciona: las tres pruebas construyen la misión con los métodos del dominio y llaman a `accept` con los datos que fabrican cada caso; no hay Spring ni repositorios. En las dos de rechazo se comprueba el error *exacto* (los *records* se comparan por valor), y que la misión quedó intacta, que es la promesa de comprobar antes de modificar. Para la prueba 2 el rango es `VETERAN`: así solo falla el límite y no el rango. Si cambias `MAX_ACTIVE_QUESTS` a 4, `accept(HERO, VETERAN, 3)` ya no se rechaza y la prueba 2 falla.
Errores típicos: usar en la prueba del límite un rango bajo (entonces sale `RankTooLow` y la prueba comprueba lo que no quería); olvidar la importación `static` de `assertThat`; poner la prueba en un paquete distinto y no poder ver clases del dominio; y olvidar `testRuntimeOnly` del lanzador o `useJUnitPlatform()`, con lo que Gradle no encuentra pruebas o no las ejecuta.

Solución del ejercicio 5.3

enunciado en la pág. 127

La versión completa de `Adventurer`, con la tabla de umbrales y el método que devuelve el rango:

domain/Adventurer.java (1/2)

```java
package com.guildboard.domain;

import java.util.Map;
import java.util.Objects;
import lombok.EqualsAndHashCode;
import lombok.Getter;

@Getter
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
public final class Adventurer {

    private static final Map<Rank, Integer> THRESHOLDS = Map.of(
        Rank.NOVICE, 0,
        Rank.ADEPT, 100,
        Rank.VETERAN, 300);

    @EqualsAndHashCode.Include
    private final AdventurerId id;
    private final String name;
    private Rank rank;
    private int experience;

    private Adventurer(AdventurerId id, String name) {
        this.id = id;
        this.name = name;
        this.rank = Rank.NOVICE;
        this.experience = 0;
    }
```

Continuación del listado anterior:

domain/Adventurer.java (2/2)

```
    public static Adventurer register(AdventurerId id, String name) {
        Objects.requireNonNull(id, "id");
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name must not be blank");
        }
        return new Adventurer(id, name.strip());
    }

    public Rank gainExperience(int points) {
        if (points <= 0) {
            throw new IllegalArgumentException(
                "points must be positive: " + points);
        }
        experience += points;
        Rank reached = rankFor(experience);
        if (reached.compareTo(rank) > 0) {
            rank = reached;
        }
        return rank;
    }

    private static Rank rankFor(int experience) {
        Rank result = Rank.NOVICE;
        for (Rank candidate : Rank.values()) {
            if (experience >= THRESHOLDS.get(candidate)) {
                result = candidate;
            }
        }
        return result;
    }
}
```

Por qué funciona: `Rank.values()` recorre `NOVICE`, `ADEPT`, `VETERAN` en orden, y `rankFor` se queda con el último cuyo umbral ya se alcanzó; por eso 350 puntos dan `VETERAN` directamente. El rango solo se asigna si `reached` es mayor, aunque como la experiencia nunca baja, el rango calculado tampoco; la comparación deja la regla «nunca baja» escrita y no la deduce el lector. `GrantExperienceService` llama a `gainExperience` y descarta el valor devuelto, así que sigue compilando sin cambios (un método puede ignorar el resultado de otro). Una prueba rápida con los cuatro casos:

src/test/java/com/guildboard/domain/AdventurerTest.java

```java
package com.guildboard.domain;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.Test;

class AdventurerTest {

    private static Adventurer newcomer() {
        return Adventurer.register(new AdventurerId("a-1"), "Mara");
    }

    @Test
    void staysNoviceBelowFirstThreshold() {
        assertThat(newcomer().gainExperience(99)).isEqualTo(Rank.NOVICE);
    }

    @Test
    void becomesAdeptAtExactThreshold() {
        assertThat(newcomer().gainExperience(100)).isEqualTo(Rank.ADEPT);
    }

    @Test
    void accumulatesAcrossCalls() {
        Adventurer adventurer = newcomer();
        adventurer.gainExperience(99);
        assertThat(adventurer.gainExperience(1)).isEqualTo(Rank.ADEPT);
    }

    @Test
    void canJumpTwoRanksInOneCall() {
        assertThat(newcomer().gainExperience(350))
            .isEqualTo(Rank.VETERAN);
    }
}
```

Errores típicos: usar `>` en lugar de `>=` (con 100 puntos justos seguiría siendo `NOVICE`); escribir el rango con una cadena de `if` que se salta el salto doble; olvidar que el `Map` necesita todas las constantes (`THRESHOLDS.get` lanzaría `NullPointerException` con un rango sin umbral si añades un cuarto rango); y reasignar el rango sin comprobar que sube.

Solución del ejercicio 5.4

enunciado en la pág. 127

1. Con la variante añadida y nada más, `DomainError.java` queda así (los dos `switch` aún sin tocar):

domain/DomainError.java

```java
package com.guildboard.domain;

public sealed interface DomainError {

    record QuestNotFound(QuestId questId) implements DomainError { }

    record AdventurerNotFound(AdventurerId adventurerId)
            implements DomainError { }

    record InvalidTransition(QuestId questId, QuestStatus actual,
                             QuestStatus expected)
            implements DomainError { }

    record QuestNotPublished(QuestId questId, QuestStatus actual)
            implements DomainError { }

    record RankTooLow(Rank adventurerRank, Rank requiredRank)
            implements DomainError { }

    record TooManyActiveQuests(int active, int limit)
            implements DomainError { }

    default String code() {
        return switch (this) {
            case QuestNotFound e -> "QUEST_NOT_FOUND";
            case AdventurerNotFound e -> "ADVENTURER_NOT_FOUND";
            case InvalidTransition e -> "INVALID_TRANSITION";
            case QuestNotPublished e -> "QUEST_NOT_PUBLISHED";
            case RankTooLow e -> "RANK_TOO_LOW";
            case TooManyActiveQuests e -> "TOO_MANY_ACTIVE_QUESTS";
        };
    }
}
```

2. Antes de completar `code()` y `describe`, `./gradlew compileJava` falla señalando `DomainError.java` (el método `code`) y `DomainException.java` (el método `describe`), con un mensaje parecido a «the switch expression does not cover all possible input values». Son los dos únicos sitios que tratan todas las variantes.
3. Se completa el de `DomainException`:

domain/DomainException.java

```java
package com.guildboard.domain;

public final class DomainException extends RuntimeException {

    private final DomainError error;

    public DomainException(DomainError error) {
        super(describe(error));
        this.error = error;
    }

    public DomainError getError() {
        return error;
    }

    private static String describe(DomainError error) {
        return switch (error) {
            case DomainError.QuestNotFound(QuestId id) ->
                "quest not found: " + id.value();
            case DomainError.AdventurerNotFound(AdventurerId id) ->
                "adventurer not found: " + id.value();
            case DomainError.InvalidTransition(QuestId id,
                    QuestStatus actual, QuestStatus expected) ->
                "quest " + id.value() + " is " + actual
                    + ", expected " + expected;
            case DomainError.QuestNotPublished(QuestId id,
                    QuestStatus actual) ->
                "quest " + id.value() + " is not published, it is "
                    + actual;
            case DomainError.RankTooLow(Rank have, Rank need) ->
                "rank " + have + " cannot take a quest that requires "
                    + need;
            case DomainError.TooManyActiveQuests(int active, int limit) ->
                "adventurer has " + active
                    + " active quests, the limit is " + limit;
        };
    }
}
```

4. En `Quest`, solo cambia `accept`: la comprobación del estado deja de usar `requireStatus`. El método entero, que sustituye al anterior:

domain/Quest.java (método accept)

```
public void accept(AdventurerId adventurerId, Rank adventurerRank,
                   int activeQuests) {
    Objects.requireNonNull(adventurerId, "adventurerId");
    Objects.requireNonNull(adventurerRank, "adventurerRank");
    if (activeQuests < 0) {
        throw new IllegalArgumentException(
            "activeQuests must not be negative: " + activeQuests);
    }
    if (status != QuestStatus.PUBLISHED) {
        throw new DomainException(
            new DomainError.QuestNotPublished(id, status));
    }
    if (!adventurerRank.canTake(requiredRank)) {
        throw new DomainException(
            new DomainError.RankTooLow(adventurerRank, requiredRank));
    }
    if (activeQuests >= MAX_ACTIVE_QUESTS) {
        throw new DomainException(new DomainError.TooManyActiveQuests(
            activeQuests, MAX_ACTIVE_QUESTS));
    }
    assignedTo = adventurerId;
    status = QuestStatus.ACCEPTED;
}
```

5. La prueba, en un archivo nuevo para no tocar el anterior:

src/test/java/com/guildboard/domain/QuestNotPublishedTest.java

```
package com.guildboard.domain;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

import org.junit.jupiter.api.Test;

class QuestNotPublishedTest {

    @Test
    void draftQuestCannotBeAccepted() {
        QuestId id = new QuestId("q-1");
        Quest quest = Quest.draft(id, "Limpiar el s^^c3^^b3tano",
                                  new Reward(150), Rank.NOVICE);

        assertThatThrownBy(() -> quest.accept(
                new AdventurerId("a-1"), Rank.NOVICE, 0))
            .isInstanceOfSatisfying(DomainException.class, e ->
                assertThat(e.getError()).isEqualTo(
                    new DomainError.QuestNotPublished(
                        id, QuestStatus.DRAFT)));
        assertThat(quest.getStatus()).isEqualTo(QuestStatus.DRAFT);
    }
}
```

Por qué funciona: al ser sellada la interfaz, el compilador conoce todas las variantes, y por eso señala los `switch` sin `default`. Con un `default` el compilador se habría callado y la variante nueva habría caído en un código genérico sin que nadie lo notara. Las pruebas anteriores siguen pasando porque ninguna aceptaba una misión no publicada.
Errores típicos: «arreglar» el error del compilador añadiendo un `default`; completar `describe` y olvidarte de `code()` (el compilador avisa, pero solo si no hay `default`); y cambiar `requireStatus` en vez de `accept`, con lo que `publish` y `complete` también cambiarían de error.

Solución del ejercicio 5.5

enunciado en la pág. 128

Solución de referencia. Estado final de todo lo que cambia el capítulo, con los cambios de los ejercicios 5.3 y 5.4 incluidos. Los archivos que no aparecen (`QuestId`, `AdventurerId`, `Rank`, `Reward`, `QuestStatus`, `PublishQuest*`, `GetQuest*`, `PublishQuestRequest` y `GuildBoardApplication`) no cambian respecto al capítulo 4.
Gradle.

build.gradle

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.1'
    id 'io.spring.dependency-management' version '1.1.7'
}

group = 'com.guildboard'
version = '0.0.1-SNAPSHOT'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}

tasks.named('test', Test) {
    useJUnitPlatform()
}
```

Dominio.

domain/DomainError.java

```java
package com.guildboard.domain;

public sealed interface DomainError {

    record QuestNotFound(QuestId questId) implements DomainError { }

    record AdventurerNotFound(AdventurerId adventurerId)
            implements DomainError { }

    record InvalidTransition(QuestId questId, QuestStatus actual,
                             QuestStatus expected)
            implements DomainError { }

    record QuestNotPublished(QuestId questId, QuestStatus actual)
            implements DomainError { }

    record RankTooLow(Rank adventurerRank, Rank requiredRank)
            implements DomainError { }

    record TooManyActiveQuests(int active, int limit)
            implements DomainError { }

    default String code() {
        return switch (this) {
            case QuestNotFound e -> "QUEST_NOT_FOUND";
            case AdventurerNotFound e -> "ADVENTURER_NOT_FOUND";
            case InvalidTransition e -> "INVALID_TRANSITION";
            case QuestNotPublished e -> "QUEST_NOT_PUBLISHED";
            case RankTooLow e -> "RANK_TOO_LOW";
            case TooManyActiveQuests e -> "TOO_MANY_ACTIVE_QUESTS";
        };
    }
}
```

domain/DomainException.java

```
package com.guildboard.domain;

public final class DomainException extends RuntimeException {

    private final DomainError error;

    public DomainException(DomainError error) {
        super(describe(error));
        this.error = error;
    }

    public DomainError getError() {
        return error;
    }

    private static String describe(DomainError error) {
        return switch (error) {
            case DomainError.QuestNotFound(QuestId id) ->
                "quest not found: " + id.value();
            case DomainError.AdventurerNotFound(AdventurerId id) ->
                "adventurer not found: " + id.value();
            case DomainError.InvalidTransition(QuestId id,
                    QuestStatus actual, QuestStatus expected) ->
                "quest " + id.value() + " is " + actual
                    + ", expected " + expected;
            case DomainError.QuestNotPublished(QuestId id,
                    QuestStatus actual) ->
                "quest " + id.value() + " is not published, it is "
                    + actual;
            case DomainError.RankTooLow(Rank have, Rank need) ->
                "rank " + have + " cannot take a quest that requires "
                    + need;
            case DomainError.TooManyActiveQuests(int active, int limit) ->
                "adventurer has " + active
                    + " active quests, the limit is " + limit;
        };
    }
}
```

domain/Adventurer.java (1/2)

```java
package com.guildboard.domain;

import java.util.Map;
import java.util.Objects;
import lombok.EqualsAndHashCode;
import lombok.Getter;

@Getter
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
public final class Adventurer {

    private static final Map<Rank, Integer> THRESHOLDS = Map.of(
        Rank.NOVICE, 0,
        Rank.ADEPT, 100,
        Rank.VETERAN, 300);

    @EqualsAndHashCode.Include
    private final AdventurerId id;
    private final String name;
    private Rank rank;
    private int experience;

    private Adventurer(AdventurerId id, String name) {
        this.id = id;
        this.name = name;
        this.rank = Rank.NOVICE;
        this.experience = 0;
    }
```

Continuación del listado anterior:

domain/Adventurer.java (2/2)

```
    public static Adventurer register(AdventurerId id, String name) {
        Objects.requireNonNull(id, "id");
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("name must not be blank");
        }
        return new Adventurer(id, name.strip());
    }

    public Rank gainExperience(int points) {
        if (points <= 0) {
            throw new IllegalArgumentException(
                "points must be positive: " + points);
        }
        experience += points;
        Rank reached = rankFor(experience);
        if (reached.compareTo(rank) > 0) {
            rank = reached;
        }
        return rank;
    }

    private static Rank rankFor(int experience) {
        Rank result = Rank.NOVICE;
        for (Rank candidate : Rank.values()) {
            if (experience >= THRESHOLDS.get(candidate)) {
                result = candidate;
            }
        }
        return result;
    }
}
```

domain/Quest.java (1/2)

```java
package com.guildboard.domain;

import java.util.Objects;
import java.util.Optional;
import lombok.EqualsAndHashCode;
import lombok.Getter;

@Getter
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
public final class Quest {

    public static final int MAX_ACTIVE_QUESTS = 3;

    @EqualsAndHashCode.Include
    private final QuestId id;
    private final String title;
    private final Reward reward;
    private final Rank requiredRank;
    private QuestStatus status;
    private AdventurerId assignedTo;        // null hasta aceptar

    private Quest(QuestId id, String title, Reward reward,
                  Rank requiredRank) {
        this.id = id;
        this.title = title;
        this.reward = reward;
        this.requiredRank = requiredRank;
        this.status = QuestStatus.DRAFT;
    }

    public static Quest draft(QuestId id, String title, Reward reward,
                              Rank requiredRank) {
        Objects.requireNonNull(id, "id");
        Objects.requireNonNull(reward, "reward");
        Objects.requireNonNull(requiredRank, "requiredRank");
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("title must not be blank");
        }
        return new Quest(id, title.strip(), reward, requiredRank);
    }

    public void publish() {
        requireStatus(QuestStatus.DRAFT);
        status = QuestStatus.PUBLISHED;
    }
```

Continuación del listado anterior:

domain/Quest.java (2/2)

```
    public void accept(AdventurerId adventurerId, Rank adventurerRank,
                       int activeQuests) {
        Objects.requireNonNull(adventurerId, "adventurerId");
        Objects.requireNonNull(adventurerRank, "adventurerRank");
        if (activeQuests < 0) {
            throw new IllegalArgumentException(
                "activeQuests must not be negative: " + activeQuests);
        }
        if (status != QuestStatus.PUBLISHED) {
            throw new DomainException(
                new DomainError.QuestNotPublished(id, status));
        }
        if (!adventurerRank.canTake(requiredRank)) {
            throw new DomainException(
                new DomainError.RankTooLow(adventurerRank, requiredRank));
        }
        if (activeQuests >= MAX_ACTIVE_QUESTS) {
            throw new DomainException(new DomainError.TooManyActiveQuests(
                activeQuests, MAX_ACTIVE_QUESTS));
        }
        assignedTo = adventurerId;
        status = QuestStatus.ACCEPTED;
    }

    public void complete() {
        requireStatus(QuestStatus.ACCEPTED);
        status = QuestStatus.COMPLETED;
    }

    // Escrito a mano: Lombok no genera un getter si ya existe uno con
    // ese nombre, y as^^c3^^ad se devuelve un Optional en lugar de null.
    public Optional<AdventurerId> getAssignedTo() {
        return Optional.ofNullable(assignedTo);
    }

    private void requireStatus(QuestStatus expected) {
        if (status != expected) {
            throw new DomainException(
                new DomainError.InvalidTransition(id, status, expected));
        }
    }
}
```

Aplicación: puertos de entrada.

application/port/in/RegisterAdventurerCommand.java

```
package com.guildboard.application.port.in;

public record RegisterAdventurerCommand(String name) {
}
```

application/port/in/RegisterAdventurerUseCase.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.Adventurer;

public interface RegisterAdventurerUseCase {

    Adventurer register(RegisterAdventurerCommand command);
}
```

application/port/in/GetAdventurerUseCase.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;
import java.util.Optional;

public interface GetAdventurerUseCase {

    Optional<Adventurer> get(AdventurerId id);
}
```

application/port/in/AcceptQuestCommand.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.QuestId;

public record AcceptQuestCommand(QuestId questId,
                                 AdventurerId adventurerId) {
}
```

application/port/in/AcceptQuestUseCase.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.Quest;

public interface AcceptQuestUseCase {

    Quest accept(AcceptQuestCommand command);
}
```

application/port/in/CompleteQuestCommand.java

```java
package com.guildboard.application.port.in;

import com.guildboard.domain.QuestId;

public record CompleteQuestCommand(QuestId questId) {
}
```

application/port/in/CompleteQuestUseCase.java

```java
package com.guildboard.application.port.in;

import com.guildboard.domain.Quest;

public interface CompleteQuestUseCase {

    Quest complete(CompleteQuestCommand command);
}
```

application/port/in/GrantExperienceCommand.java

```java
package com.guildboard.application.port.in;

import com.guildboard.domain.AdventurerId;

public record GrantExperienceCommand(AdventurerId adventurerId,
                                     int points) {
}
```

application/port/in/GrantExperienceUseCase.java

```java
package com.guildboard.application.port.in;

import com.guildboard.domain.Adventurer;

public interface GrantExperienceUseCase {

    Adventurer grant(GrantExperienceCommand command);
}
```

Aplicación: puertos de salida.

application/port/out/AdventurerRepository.java

```java
package com.guildboard.application.port.out;

import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;
import java.util.Optional;

public interface AdventurerRepository {

    Optional<Adventurer> findById(AdventurerId id);

    void save(Adventurer adventurer);
}
```

application/port/out/QuestRepository.java

```java
package com.guildboard.application.port.out;

import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.util.Optional;

public interface QuestRepository {

    Optional<Quest> findById(QuestId id);

    void save(Quest quest);

    int countActiveBy(AdventurerId adventurerId);
}
```

Aplicación: servicios.

application/service/RegisterAdventurerService.java

```java
package com.guildboard.application.service;

import com.guildboard.application.port.in.RegisterAdventurerCommand;
import com.guildboard.application.port.in.RegisterAdventurerUseCase;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;
import java.util.UUID;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class RegisterAdventurerService
        implements RegisterAdventurerUseCase {

    private final AdventurerRepository adventurers;

    @Override
    public Adventurer register(RegisterAdventurerCommand command) {
        Adventurer adventurer = Adventurer.register(
            new AdventurerId(UUID.randomUUID().toString()),
            command.name());
        adventurers.save(adventurer);
        return adventurer;
    }
}
```

application/service/GetAdventurerService.java

```java
package com.guildboard.application.service;

import com.guildboard.application.port.in.GetAdventurerUseCase;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;
import java.util.Optional;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class GetAdventurerService implements GetAdventurerUseCase {

    private final AdventurerRepository adventurers;

    @Override
    public Optional<Adventurer> get(AdventurerId id) {
        return adventurers.findById(id);
    }
}
```

application/service/AcceptQuestService.java

```java
package com.guildboard.application.service;

import com.guildboard.application.port.in.AcceptQuestCommand;
import com.guildboard.application.port.in.AcceptQuestUseCase;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.Adventurer;
import com.guildboard.domain.DomainError;
import com.guildboard.domain.DomainException;
import com.guildboard.domain.Quest;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class AcceptQuestService implements AcceptQuestUseCase {

    private final QuestRepository quests;
    private final AdventurerRepository adventurers;

    @Override
    public Quest accept(AcceptQuestCommand command) {
        Quest quest = quests.findById(command.questId())
            .orElseThrow(() -> new DomainException(
                new DomainError.QuestNotFound(command.questId())));
        Adventurer adventurer = adventurers.findById(command.adventurerId())
            .orElseThrow(() -> new DomainException(
                new DomainError.AdventurerNotFound(
                    command.adventurerId())));
        int active = quests.countActiveBy(adventurer.getId());
        quest.accept(adventurer.getId(), adventurer.getRank(), active);
        quests.save(quest);
        return quest;
    }
}
```

application/service/GrantExperienceService.java

```java
package com.guildboard.application.service;

import com.guildboard.application.port.in.GrantExperienceCommand;
import com.guildboard.application.port.in.GrantExperienceUseCase;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.domain.Adventurer;
import com.guildboard.domain.DomainError;
import com.guildboard.domain.DomainException;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class GrantExperienceService implements GrantExperienceUseCase {

    private final AdventurerRepository adventurers;

    @Override
    public Adventurer grant(GrantExperienceCommand command) {
        Adventurer adventurer = adventurers.findById(command.adventurerId())
            .orElseThrow(() -> new DomainException(
                new DomainError.AdventurerNotFound(
                    command.adventurerId())));
        adventurer.gainExperience(command.points());
        adventurers.save(adventurer);
        return adventurer;
    }
}
```

application/service/CompleteQuestService.java

```java
package com.guildboard.application.service;

import com.guildboard.application.port.in.CompleteQuestCommand;
import com.guildboard.application.port.in.CompleteQuestUseCase;
import com.guildboard.application.port.in.GrantExperienceCommand;
import com.guildboard.application.port.in.GrantExperienceUseCase;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.DomainError;
import com.guildboard.domain.DomainException;
import com.guildboard.domain.Quest;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class CompleteQuestService implements CompleteQuestUseCase {

    private final QuestRepository quests;
    private final GrantExperienceUseCase grantExperience;

    @Override
    public Quest complete(CompleteQuestCommand command) {
        Quest quest = quests.findById(command.questId())
            .orElseThrow(() -> new DomainException(
                new DomainError.QuestNotFound(command.questId())));
        quest.complete();
        AdventurerId assignee = quest.getAssignedTo().orElseThrow(
            () -> new IllegalStateException("accepted quest without "
                + "assignee: " + quest.getId().value()));
        quests.save(quest);
        int points = quest.getReward().gold();
        if (points > 0) {
            grantExperience.grant(
                new GrantExperienceCommand(assignee, points));
        }
        return quest;
    }
}
```

Adaptadores de salida.

adapter/out/memory/InMemoryAdventurerRepository.java

```java
package com.guildboard.adapter.out.memory;

import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

public class InMemoryAdventurerRepository implements AdventurerRepository {

    private final Map<AdventurerId, Adventurer> adventurers =
        new ConcurrentHashMap<>();

    @Override
    public Optional<Adventurer> findById(AdventurerId id) {
        return Optional.ofNullable(adventurers.get(id));
    }

    @Override
    public void save(Adventurer adventurer) {
        adventurers.put(adventurer.getId(), adventurer);
    }
}
```

adapter/out/memory/InMemoryQuestRepository.java

```java
package com.guildboard.adapter.out.memory;

import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import com.guildboard.domain.QuestStatus;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

public class InMemoryQuestRepository implements QuestRepository {

    private final Map<QuestId, Quest> quests = new ConcurrentHashMap<>();

    @Override
    public Optional<Quest> findById(QuestId id) {
        return Optional.ofNullable(quests.get(id));
    }

    @Override
    public void save(Quest quest) {
        quests.put(quest.getId(), quest);
    }

    @Override
    public int countActiveBy(AdventurerId adventurerId) {
        return (int) quests.values().stream()
            .filter(quest -> quest.getStatus() == QuestStatus.ACCEPTED)
            .filter(quest -> quest.getAssignedTo()
                .filter(adventurerId::equals).isPresent())
            .count();
    }
}
```

Adaptadores de entrada.

adapter/in/web/QuestResponse.java

```java
package com.guildboard.adapter.in.web;

import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestStatus;
import com.guildboard.domain.Rank;

public record QuestResponse(String id, String title, int reward,
                            Rank requiredRank, QuestStatus status,
                            String assignedTo) {

    public static QuestResponse from(Quest quest) {
        return new QuestResponse(
            quest.getId().value(),
            quest.getTitle(),
            quest.getReward().gold(),
            quest.getRequiredRank(),
            quest.getStatus(),
            quest.getAssignedTo().map(AdventurerId::value).orElse(null));
    }
}
```

adapter/in/web/AcceptQuestRequest.java

```java
package com.guildboard.adapter.in.web;

public record AcceptQuestRequest(String adventurerId) {
}
```

adapter/in/web/QuestController.java (1/2)

```
package com.guildboard.adapter.in.web;

import com.guildboard.application.port.in.AcceptQuestCommand;
import com.guildboard.application.port.in.AcceptQuestUseCase;
import com.guildboard.application.port.in.CompleteQuestCommand;
import com.guildboard.application.port.in.CompleteQuestUseCase;
import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.in.PublishQuestCommand;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import com.guildboard.domain.Reward;
import java.net.URI;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

Continuación del listado anterior:

adapter/in/web/QuestController.java (2/2)

```java
@RestController
@RequestMapping("/quests")
@RequiredArgsConstructor
public class QuestController {

    private final PublishQuestUseCase publishQuest;
    private final GetQuestUseCase getQuest;
    private final AcceptQuestUseCase acceptQuest;
    private final CompleteQuestUseCase completeQuest;

    @PostMapping
    public ResponseEntity<QuestResponse> publish(
            @RequestBody PublishQuestRequest request) {
        PublishQuestCommand command = new PublishQuestCommand(
            request.title(), new Reward(request.reward()),
            request.requiredRank());
        Quest quest = publishQuest.publish(command);
        return ResponseEntity
            .created(URI.create("/quests/" + quest.getId().value()))
            .body(QuestResponse.from(quest));
    }

    @GetMapping("/{id}")
    public ResponseEntity<QuestResponse> get(@PathVariable String id) {
        return getQuest.get(new QuestId(id))
            .map(QuestResponse::from)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping("/{id}/accept")
    public QuestResponse accept(@PathVariable String id,
                                @RequestBody AcceptQuestRequest request) {
        Quest quest = acceptQuest.accept(new AcceptQuestCommand(
            new QuestId(id), new AdventurerId(request.adventurerId())));
        return QuestResponse.from(quest);
    }

    @PostMapping("/{id}/complete")
    public QuestResponse complete(@PathVariable String id) {
        Quest quest = completeQuest.complete(
            new CompleteQuestCommand(new QuestId(id)));
        return QuestResponse.from(quest);
    }
}
```

adapter/in/web/RegisterAdventurerRequest.java

```java
package com.guildboard.adapter.in.web;

public record RegisterAdventurerRequest(String name) {
}
```

adapter/in/web/AdventurerResponse.java

```
package com.guildboard.adapter.in.web;

import com.guildboard.domain.Adventurer;
import com.guildboard.domain.Rank;

public record AdventurerResponse(String id, String name, Rank rank,
                                 int experience) {

    public static AdventurerResponse from(Adventurer adventurer) {
        return new AdventurerResponse(
            adventurer.getId().value(),
            adventurer.getName(),
            adventurer.getRank(),
            adventurer.getExperience());
    }
}
```

adapter/in/web/AdventurerController.java

```java
package com.guildboard.adapter.in.web;

import com.guildboard.application.port.in.GetAdventurerUseCase;
import com.guildboard.application.port.in.RegisterAdventurerCommand;
import com.guildboard.application.port.in.RegisterAdventurerUseCase;
import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;
import java.net.URI;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/adventurers")
@RequiredArgsConstructor
public class AdventurerController {

    private final RegisterAdventurerUseCase registerAdventurer;
    private final GetAdventurerUseCase getAdventurer;

    @PostMapping
    public ResponseEntity<AdventurerResponse> register(
            @RequestBody RegisterAdventurerRequest request) {
        Adventurer adventurer = registerAdventurer.register(
            new RegisterAdventurerCommand(request.name()));
        return ResponseEntity
            .created(URI.create(
                "/adventurers/" + adventurer.getId().value()))
            .body(AdventurerResponse.from(adventurer));
    }

    @GetMapping("/{id}")
    public ResponseEntity<AdventurerResponse> get(
            @PathVariable String id) {
        return getAdventurer.get(new AdventurerId(id))
            .map(AdventurerResponse::from)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }
}
```

Raíz de composición.

config/BeanConfig.java (1/2)

```java
package com.guildboard.config;

import com.guildboard.adapter.out.memory.InMemoryAdventurerRepository;
import com.guildboard.adapter.out.memory.InMemoryQuestRepository;
import com.guildboard.application.port.in.AcceptQuestUseCase;
import com.guildboard.application.port.in.CompleteQuestUseCase;
import com.guildboard.application.port.in.GetAdventurerUseCase;
import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.in.GrantExperienceUseCase;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.application.port.in.RegisterAdventurerUseCase;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.application.service.AcceptQuestService;
import com.guildboard.application.service.CompleteQuestService;
import com.guildboard.application.service.GetAdventurerService;
import com.guildboard.application.service.GetQuestService;
import com.guildboard.application.service.GrantExperienceService;
import com.guildboard.application.service.PublishQuestService;
import com.guildboard.application.service.RegisterAdventurerService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public QuestRepository questRepository() {
        return new InMemoryQuestRepository();
    }

    @Bean
    public AdventurerRepository adventurerRepository() {
        return new InMemoryAdventurerRepository();
    }

    @Bean
    public PublishQuestUseCase publishQuestUseCase(
            QuestRepository questRepository) {
        return new PublishQuestService(questRepository);
    }
```

Continuación del listado anterior:

config/BeanConfig.java (2/2)

```
    @Bean
    public GetQuestUseCase getQuestUseCase(
            QuestRepository questRepository) {
        return new GetQuestService(questRepository);
    }

    @Bean
    public RegisterAdventurerUseCase registerAdventurerUseCase(
            AdventurerRepository adventurerRepository) {
        return new RegisterAdventurerService(adventurerRepository);
    }

    @Bean
    public GetAdventurerUseCase getAdventurerUseCase(
            AdventurerRepository adventurerRepository) {
        return new GetAdventurerService(adventurerRepository);
    }

    @Bean
    public GrantExperienceUseCase grantExperienceUseCase(
            AdventurerRepository adventurerRepository) {
        return new GrantExperienceService(adventurerRepository);
    }

    @Bean
    public AcceptQuestUseCase acceptQuestUseCase(
            QuestRepository questRepository,
            AdventurerRepository adventurerRepository) {
        return new AcceptQuestService(questRepository,
                                      adventurerRepository);
    }

    @Bean
    public CompleteQuestUseCase completeQuestUseCase(
            QuestRepository questRepository,
            GrantExperienceUseCase grantExperienceUseCase) {
        return new CompleteQuestService(questRepository,
                                        grantExperienceUseCase);
    }
}
```

Pruebas. Quedan las del capítulo: `QuestTransitionTest` (sección 5.6), `QuestAcceptTest` (ejercicio 5.2), `AdventurerTest` (ejercicio 5.3) y `QuestNotPublishedTest` (ejercicio 5.4). Una de ellas, `QuestTransitionTest`, no cambia con el error nuevo porque ninguna de sus pruebas acepta una misión en borrador.

Escenario de comprobación. Es el de la sección 5.9.1: los resultados que allí se muestran son los que debe dar esta solución.

Por qué funciona: cada servicio lee con los puertos de salida y delega la decisión en el agregado. Los rechazos llegan como `DomainException`, que Spring convierte en `500` porque aún no hay traducción (capítulo 6); lo que prueba que la regla funciona es que el estado no cambia. `completeQuest` llama al *puerto* `GrantExperienceUseCase`, y el contenedor se lo da en la configuración. Solo `GuildBoardApplication`, `BeanConfig` y los dos controladores

importan Spring.

Criterios para valorar otras soluciones. Es válida si (a) `Quest` sigue sin importar ni conocer `Adventurer` y los repositorios; (b) cada servicio modifica un solo agregado y `AcceptQuest` comprueba rango y límite *solo* a través de `Quest.accept`; (c) `CompleteQuest` no duplica la lógica de subida de rango; (d) los errores de reglas de negocio son `DomainError`, no cadenas ni `IllegalStateException`; (e) no hay `default` en los `switch` sobre `DomainError`; y (f) la instalación de pruebas y las pruebas pasan con `./gradlew build`.

Alternativa válida. `GrantExperienceUseCase` puede devolver `void` o el `Rank` resultante en lugar del `Adventurer`, y los errores de «no encontrado» pueden modelarse devolviendo `Optional` desde los servicios y dejando que el controlador responda `404`. La segunda es más cómoda ahora y pierde la ventaja que da el error tipado cuando el borde traduzca los errores en el capítulo 6; por eso la solución de referencia usa `DomainException`.

Errores típicos: poner el mapa de umbrales como números mágicos dentro de un `if`; olvidar la guarda `points > 0` en `CompleteQuestService`; cambiar la firma de `QuestRepository` y olvidar `countActiveBy` en el repositorio en memoria (no compila); registrar `CompleteQuest` sin tener el bean de `GrantExperienceUseCase` (la aplicación no arranca: «No qualifying bean»); no contar solo las misiones `ACCEPTED` del aventurero (contar las completadas haría que nadie pudiera aceptar una cuarta misión jamás); y poner `@Service` en los servicios, con lo que `ArchitectureTest` falla.

CAPÍTULO 6

Los límites: DTO, mapeo manual, errores HTTP y más puertos de salida

Objetivos del capítulo

- Mantener modelos distintos en cada lado de un límite (web y dominio) y convertir entre ellos con mapeadores escritos a mano.
- Validar la forma de una petición en el borde y explicar por qué eso no sustituye a las invariantes del dominio.
- Traducir los errores de dominio a respuestas HTTP con `@RestControllerAdvice` y `ProblemDetail`.
- Sacar del núcleo el reloj (`java.time.Clock`) y las notificaciones (`NotificationSender`), y comprobar los paquetes nuevos en la verificación de estructura 2 (hito H3, parte 1).

Al terminar el capítulo 5 GuildBoard tiene reglas de negocio de verdad, pero su cara hacia fuera sigue siendo la del capítulo 4: los DTO viven junto a los controladores, algunos usan los `enum` del dominio, cada rechazo de una regla es un `500`, y nada impide que alguien envíe un título vacío y reciba otro `500`. Hay además dos dependencias escondidas que el núcleo todavía no tiene y que van a hacer falta: el *tiempo* (una misión puede caducar) y el *aviso* a un aventurero cuando completa algo.

Todas son versiones del mismo problema: lo que cruza un límite. El límite web (HTTP en un lado, casos de uso en el otro) pide un modelo propio, un traductor de errores y una validación de la forma de los datos. Los límites hacia otros sistemas (el reloj, un canal de avisos) piden un puerto de salida. Este capítulo resuelve las cuatro cosas, añade algunos paquetes al árbol y, por eso, incluye la **verificación de estructura 2**.

6.1 Por qué no devolver el dominio por la API

Hasta ahora el controlador de aventureros ya devuelve un *record* propio (`AdventurerResponse`) y no un `Adventurer`. Pero esa separación era una costumbre, no una decisión explicada: los DTO tienen un método `from` que conoce al dominio, y algunos campos son `enum` del dominio. Para ver qué se pierde al saltársela, imagina que `QuestController.get` devolviera el `Quest` tal cual. Jackson escribe en JSON lo que ve en los *getters* públicos, más o menos así:

Lo que saldría de devolver Quest directamente (aproximado)

```
{"id":{"value":"c91e7a30-4d5b-4c28-8a0f-6e3b2d1f9a45"},
 "title":"Domar al drag^^c3^^b3n","reward":{"gold":500},
 "requiredRank":"ADEPT","status":"PUBLISHED",
 "assignedTo":null,"expiresAt":null}
```

No es lo que querías publicar (el identificador sale envuelto en un objeto, la recompensa también)

y, peor, es un contrato accidental. Cuatro cosas salen mal cuando el modelo del dominio es, a la vez, el modelo de la API:

- **El contrato queda atado al dominio.** Renombrar `requiredRank` o una constante del `enum` rompe a todos los clientes. El dominio debería poder cambiar de forma sin que cambie la API.
- **Todo *getter* es público para siempre.** Añadir un *getter* al agregado, por ejemplo uno de uso interno, lo publica sin que nadie lo decida. Con un DTO, cada campo expuesto es una línea escrita a propósito.
- **El dominio se llena de anotaciones del borde.** Para ajustar el JSON o validar los datos tendrías que poner anotaciones de Jackson o de validación dentro de `domain`, y eso viola la regla de dependencia del capítulo 3: `ArchitectureTest` falla si `domain` depende de `jakarta` o de `org.springframework` (Jackson no está en esa lista, pero el efecto sería el mismo: el dominio quedaría atado a una librería del borde).
- **Las dos formas necesitan cambiar por razones distintas.** La API cambia cuando un cliente lo pide; el dominio, cuando cambia el negocio. Un solo modelo para ambas cosas obliga a coordinar cambios que no tienen nada que ver.

Martin lo resume en una regla para los límites: lo que cruza debe ser una estructura de datos simple, no una entidad ni una fila de base de datos [27]. El controlador de la web es, además, el *objeto humilde* del capítulo 23 de [28]: lo que no puedes probar sin un servidor se deja tan fino como sea posible, y la traducción pasa a una clase sencilla que sí puedes probar.

De ahí salen **tres modelos distintos**, uno por lado de cada límite:

Modelo	Dónde vive	Cambia cuando	Se escribe como
Web (DTO)	`adapter/in/web/dto`	lo pide un cliente de la API	*records* de campos simples
Dominio	`domain`	cambia el negocio	clases con reglas
Persistencia	el adaptador de salida de la base (capítulo 8)	cambia el esquema de la base	entidades JPA

El tercero no existe todavía: el repositorio en memoria guarda el propio `Quest`. Aparecerá cuando haya una base de datos, y para entonces ya sabrás traducir entre modelos: es la misma técnica que verás hoy.

Hay un coste, y conviene decirlo: cada modelo extra es código que escribir y mantener, y en una API pequeña de mantenimiento de datos copiar los mismos tres campos tres veces es ceremonia pura. El libro de Hombergs compara varias estrategias de mapeo, desde no mapear nada hasta mapear completamente en cada límite, y presenta el coste como una elección consciente [19]. GuildBoard elige separar los modelos en los límites donde el dominio tiene reglas propias, que es el caso de la web.

6.2 Verificación de estructura 2: los paquetes nuevos

Nota

Este capítulo cambia la estructura de carpetas. Comprueba que tu proyecto coincide con el árbol de abajo antes de seguir.

El árbol anterior es el que dejó el capítulo 5, con `adapter/in/web` (controladores y DTO juntos) y `adapter/out/memory`. Lo nuevo va marcado con `[N]` y lo modificado con `[M]`:

Árbol tras el capítulo 6

```
guildboard/
|-- build.gradle            [M] + spring-boot-starter-validation
|-- docs/dominio.md
`-- src/
    |-- main/java/com/guildboard/
    |   |-- GuildBoardApplication.java
    |   |-- domain/         [M] Quest, DomainError, DomainException
    |   |-- application/
    |   |   |-- port/in/    [M] PublishQuestCommand
    |   |   |-- port/out/   [N] NotificationSender
    |   |   `-- service/    [M] PublishQuest, AcceptQuest, CompleteQuest
    |   |-- adapter/
    |   |   |-- in/web/     [M] QuestController, AdventurerController
    |   |   |   |-- dto/    [N] los cinco DTO (salen de web/)
    |   |   |   |-- mapper/ [N] QuestWebMapper, AdventurerWebMapper
    |   |   |   `-- error/  [N] DomainExceptionHandler
    |   |   `-- out/
    |   |       |-- memory/
    |   |       `-- notification/ [N] LogNotificationSender
    |   `-- config/         [M] BeanConfig (+ Clock, NotificationSender)
    `-- test/java/com/guildboard/
        |-- ArchitectureTest.java
        `-- domain/         [N] QuestExpirationTest
```

Cambios respecto al árbol anterior:

- `adapter/in/web/dto`: los *records* de petición y respuesta.
- `adapter/in/web/mapper`: mapeadores estáticos entre DTO, comandos y dominio.
- `adapter/in/web/error`: traducción de errores de dominio a HTTP.
- `adapter/out/notification`: adaptador de `NotificationSender`.
- `application/port/out`: el puerto `NotificationSender`.
- `config/BeanConfig`: registra `Clock` y `NotificationSender`.
- `Clock` no crea paquete: es `java.time.Clock`, del JDK (la sección 6.6 lo explica).

`ArchitectureTest` no cambia: los paquetes nuevos cuelgan de `adapter` (`adapter.in.web.dto`, `adapter.in.web.mapper`, `adapter.in.web.error` y `adapter.out.notification`) y de `application.port.out`, así que ninguna de sus reglas se rompe.

Comprobaciones sobre *tu* proyecto:

- [ ] `./gradlew test` pasa (cuando termines el capítulo): incluye `ArchitectureTest`, que vigila que `domain` y `application` no dependen de lo exterior. `Quest` usa `Instant` y `Clock`, del JDK, y la validación (`jakarta`) va solo en `dto`.
- [ ] Cada adaptador está en `in` o en `out`: `dto`, `mapper` y `error` en `in/web` (los llama HTTP); `notification` en `out` (lo llama la aplicación).
- [ ] Las carpetas de tu proyecto coinciden con el árbol.

Si algo no coincide con el árbol, corrígelo antes de seguir.

6.3 DTO y mapeadores a mano

Empieza por el lado fácil, los aventureros. Los dos *records* se mueven a `adapter/in/web/dto` (solo cambia la línea `package`, y desaparece el método `from`, porque ya no es el DTO quien conoce al dominio). Además, los campos que eran `enum` del dominio pasan a ser `String`: el contrato de la API son textos, no tipos de Java.

adapter/in/web/dto/RegisterAdventurerRequest.java

```
package com.guildboard.adapter.in.web.dto;

public record RegisterAdventurerRequest(String name) {
}
```

adapter/in/web/dto/AdventurerResponse.java

```
package com.guildboard.adapter.in.web.dto;

public record AdventurerResponse(String id, String name, String rank,
                                 int experience) {
}
```

La traducción vive en un *mapeador* (*mapper*): una clase con métodos estáticos que recibe un modelo y devuelve el otro.

adapter/in/web/mapper/AdventurerWebMapper.java

```
package com.guildboard.adapter.in.web.mapper;

import com.guildboard.adapter.in.web.dto.AdventurerResponse;
import com.guildboard.adapter.in.web.dto.RegisterAdventurerRequest;
import com.guildboard.application.port.in.RegisterAdventurerCommand;
import com.guildboard.domain.Adventurer;

public final class AdventurerWebMapper {

    private AdventurerWebMapper() {
    }

    public static RegisterAdventurerCommand toCommand(
            RegisterAdventurerRequest request) {
        return new RegisterAdventurerCommand(request.name());
    }

    public static AdventurerResponse toResponse(Adventurer adventurer) {
        return new AdventurerResponse(
            adventurer.getId().value(),
            adventurer.getName(),
            adventurer.getRank().name(),
            adventurer.getExperience());
    }
}
```

Cuatro reglas para un mapeador de este tipo:

- **Estático y puro**: sin estado, sin Spring, sin entrada/salida. Se prueba con `new` y una llamada, como el dominio. El constructor privado impide crear instancias inútiles.
- **Un solo sitio**: todo lo que sabe cómo se ve un `Adventurer` en la API está aquí. Si el JSON cambia, se toca esta clase.
- **Sin lógica de negocio**: solo copia y convierte formatos. Si un mapeador toma una decisión (por ejemplo, un cálculo de rango), esa decisión está en el sitio equivocado.
- **Sin librería**: el briefing del curso excluye herramientas como MapStruct, y con cuatro o cinco campos escribir el mapeo a mano es corto y claro. Además tiene una ventaja: si añades un campo a un *record*, la llamada al constructor deja de compilar y el compilador te dice dónde falta rellenarlo. Con un mapeo por reflexión el olvido aparece en ejecución.

El controlador queda más humilde que antes: recibe el DTO, lo convierte con el mapeador, llama al puerto y convierte el resultado.

adapter/in/web/AdventurerController.java

```
package com.guildboard.adapter.in.web;

import com.guildboard.adapter.in.web.dto.AdventurerResponse;
import com.guildboard.adapter.in.web.dto.RegisterAdventurerRequest;
import com.guildboard.adapter.in.web.mapper.AdventurerWebMapper;
import com.guildboard.application.port.in.GetAdventurerUseCase;
import com.guildboard.application.port.in.RegisterAdventurerUseCase;
import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;
import jakarta.validation.Valid;
import java.net.URI;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/adventurers")
@RequiredArgsConstructor
public class AdventurerController {

    private final RegisterAdventurerUseCase registerAdventurer;
    private final GetAdventurerUseCase getAdventurer;

    @PostMapping
    public ResponseEntity<AdventurerResponse> register(
            @Valid @RequestBody RegisterAdventurerRequest request) {
        Adventurer adventurer = registerAdventurer.register(
            AdventurerWebMapper.toCommand(request));
        return ResponseEntity
            .created(URI.create(
                "/adventurers/" + adventurer.getId().value()))
            .body(AdventurerWebMapper.toResponse(adventurer));
    }

```

```
40      @GetMapping("/{id}")
41      public ResponseEntity<AdventurerResponse> get(
42              @PathVariable String id) {
43          return getAdventurer.get(new AdventurerId(id))
44              .map(AdventurerWebMapper::toResponse)
45              .map(ResponseEntity::ok)
46              .orElseGet(() -> ResponseEntity.notFound().build());
47      }
48  }
```

La anotación `@Valid` y el import de `jakarta.validation.Valid` son nuevos y **todavía no compilan**: pertenecen a la dependencia de validación, que la sección siguiente explica e instala. Hasta entonces, deja el controlador sin esa línea de *import* y sin `@Valid`, o espera a instalar el starter antes de compilar. Con las misiones haces lo mismo en el ejercicio 6.1: sus tres DTO se mueven a `dto` y se escribe `QuestWebMapper`.

Dos cambios más de contrato en los DTO de misión, que están en ese ejercicio. La petición de publicar recibe el rango como `String` (el mapeador lo convierte a `Rank`) y gana un campo opcional, `validForMinutes`: cuánto tiempo seguirá vigente la misión (sección 6.6). La respuesta gana `expiresAt`, la fecha de caducidad en texto ISO-8601 o `null`.

Una API que sigue igual por fuera

Los `enum` se escriben como su nombre, así que el JSON de las respuestas es el mismo que el del capítulo 5, salvo el campo nuevo `expiresAt`. Cambia lo que hay *dentro*: ahora puedes renombrar una constante del dominio y decidir, en el mapeador, si la API cambia o no.

6.4 Validar en el borde y proteger invariantes en el dominio

Un título en blanco ya no se acepta: `Quest.draft` lanza `IllegalArgumentException`. Entonces, ¿para qué validar también en la web? Porque son dos controles con trabajos distintos, y los dos hacen falta:

	Validación en el borde	**Invariante del dominio**
Qué comprueba	la *forma* de lo que llega: campos obligatorios, tamaños, formatos	lo que es *verdad* del negocio, llegue por donde llegue
Quién la hace cumplir	el adaptador web, con anotaciones sobre el DTO	el agregado, en su constructor o su método
Cuántos errores da	todos a la vez, uno por campo	el primero que encuentra
Puede ser más estricta	sí: es una política de *esta* API	no: solo la regla del negocio
Si falta	el cliente recibe errores pobres o un `500`	el dominio acepta datos imposibles desde otro adaptador

La validación del borde protege al cliente de la API (le da todos los errores a la vez y con el nombre del campo). La invariante protege al *dominio*: el día que exista una segunda puerta de entrada (un `CommandLineRunner`, un mensaje de una cola), esa puerta no pasa por el controlador y nadie habrá validado nada. Por eso ninguna sustituye a la otra. Y por eso pueden no coincidir: el dominio admite misiones con recompensa cero (regla R7 de `docs/dominio.md`: no negativa), mientras que la API puede decidir que *publicar* una misión gratuita no se permite. Esa es una política de la API, no del negocio.

☞ Bean Validation con spring-boot-starter-validation

Java tiene una especificación de validación por anotaciones, *Bean Validation* (paquete `jakarta.validation`); Spring Boot la integra con un starter que trae su implementación de referencia, Hibernate Validator. Es la primera dependencia nueva del capítulo y la única. Se comprobó el 8 de octubre de 2026 con Spring Initializr (un proyecto de Boot 4.1.1 con *Spring Web* y *Validation* pide exactamente `spring-boot-starter-validation`) y en Maven Central, donde existe la versión 4.1.1 y su descriptor depende de `spring-boot-validation`, que a su vez trae `hibernate-validator` y `tomcat-embed-el`. Boot lo documenta en su referencia [52]. Añade una línea al bloque `dependencies` de `build.gradle`; la versión la pone la gestión de dependencias de Boot, igual que en el resto de starters:

build.gradle (fragmento)

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

Comprueba que Gradle lo resuelve:

```
$ ./gradlew dependencies --configuration compileClasspath | grep valid
```

Debe aparecer `spring-boot-starter-validation` y, dentro, `jakarta.validation-api`.

Lo básico, en cuatro anotaciones de `jakarta.validation.constraints`: `@NotBlank` (texto no nulo y con algún carácter que no sea espacio), `@NotNull`, `@Positive` (número mayor que cero) y `@Size` (longitud mínima y máxima). Se ponen sobre los componentes del *record*. Y la validación solo se ejecuta si el parámetro del controlador lleva `@Valid` (ya lo escribiste en el controlador de aventureros):

adapter/in/web/dto/RegisterAdventurerRequest.java

```
package com.guildboard.adapter.in.web.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record RegisterAdventurerRequest(
        @NotBlank @Size(max = 50) String name) {
}
```

Con esto, `POST /adventurers` con `{"name":""}` ya no llega a `RegisterAdventurerService`: Spring lanza `MethodArgumentNotValidException` antes de entrar en el método. Si no tratas esa excepción, Spring responde `400` con el cuerpo de error por defecto de Boot, que no dice qué campo falló. El formato propio de esa respuesta lo unificas en el ejercicio 6.2.

Validar no es proteger

Un error frecuente es anotar el DTO con `@NotBlank` y quitar la comprobación del dominio «porque ya está validado». El controlador es solo una de las puertas posibles. Si el dominio no comprueba, cualquier otro adaptador puede crear un aventurero sin nombre. Mantén las dos.

6.5 Errores de dominio a HTTP

El problema que dejó el capítulo 5: un `DomainException` que llega al controlador acaba en un `500`, como si el servidor se hubiera roto. No se ha roto: el cliente intentó algo que el negocio no permite, y quiere saberlo con un código que pueda interpretar.

Dónde se traduce. No en el dominio ni en la aplicación: saben de reglas, no de HTTP, y `DomainError` no contiene ningún código de estado a propósito. Se traduce en el borde, en el paquete `adapter/in/web/error`. Lo único que el dominio ofrece es `code()`, un texto estable (`RANK_TOO_LOW`) que el cliente puede usar sin depender del mensaje.

Con qué herramientas. Spring MVC tiene dos piezas:

- `@RestControllerAdvice`: una clase cuyos métodos se aplican a todos los controladores. Es un componente más, y el escaneo la encuentra (está bajo `com.guildboard`), así que no hay que registrarla en `BeanConfig`.
- `@ExceptionHandler(Tipo.class)`: un método de esa clase que atiende las excepciones de ese tipo y construye la respuesta.

Para el cuerpo de la respuesta Spring tiene `ProblemDetail`, la representación del formato estándar *Problem Details for HTTP APIs* (RFC 9457); la referencia de Boot describe cómo encaja con el manejo de errores [52]. Es un JSON con `type`, `title`, `status`, `detail` e `instance`, al que puedes añadir campos propios con `setProperty`. Un método que devuelve un `ProblemDetail` responde con ese estado y con el tipo de contenido `application/problem+json`.

Esta es la primera versión, la más sencilla que resuelve el problema: cualquier regla de negocio violada es un `422` (la petición está bien formada, pero el negocio no la acepta) y los datos mal formados son un `400`.

adapter/in/web/error/DomainExceptionHandler.java (versión 1)

```
1  package com.guildboard.adapter.in.web.error;
2
3  import com.guildboard.domain.DomainException;
4  import org.springframework.http.HttpStatus;
5  import org.springframework.http.HttpStatusCode;
6  import org.springframework.http.ProblemDetail;
7  import org.springframework.web.bind.annotation.ExceptionHandler;
8  import org.springframework.web.bind.annotation.RestControllerAdvice;
9
10 @RestControllerAdvice
11 public class DomainExceptionHandler {
12
13     @ExceptionHandler(DomainException.class)
14     public ProblemDetail handleDomain(DomainException exception) {
15         ProblemDetail problem = ProblemDetail.forStatusAndDetail(
16             HttpStatusCode.valueOf(422), exception.getMessage());
17         problem.setTitle("Business rule violated");
```

```
18          problem.setProperty("code", exception.getError().code());
19          return problem;
20      }
21
22      @ExceptionHandler(IllegalArgumentException.class)
23      public ProblemDetail handleMalformed(
24              IllegalArgumentException exception) {
25          return ProblemDetail.forStatusAndDetail(
26              HttpStatus.BAD_REQUEST, exception.getMessage());
27      }
28  }
```

Se usa `HttpStatusCode.valueOf(422)` y no una constante de `HttpStatus` porque el nombre de la constante del 422 ha cambiado entre versiones de Spring; el número no.

Mira lo que ya has ganado: una regla rota es un `422` con un JSON que tiene el mensaje y un `code`, y un título en blanco que llega al dominio es un `400`. Pero un `422` para «esa misión no existe» es tosco, y un cliente querría distinguir «no existe» (404), «la misión está en otro estado» (409) y «tú no cumples la regla» (422). Los criterios del curso:

Estado	Cuándo	Ejemplo
404	el recurso que nombra la petición no existe	misión o aventurero inexistentes
409	choca con el *estado actual* del recurso, que puede cambiar	completar una misión que no está aceptada
422	la petición es válida y el recurso existe, pero el negocio rechaza al solicitante	un rango insuficiente

Aplicar esa tabla a las siete variantes de `DomainError` es el ejercicio 6.3.

Un aviso sobre el manejador de `IllegalArgumentException`: es útil porque el dominio la usa para los datos mal formados (capítulo 5), pero captura *cualquier* `IllegalArgumentException` que se escape de un controlador, incluso la que lance una librería por un fallo del programa. Si la usas, el mensaje que se expone al cliente es el de la excepción: no pongas en ella nada que no quieras publicar.

6.6 El tiempo como dependencia: Clock y caducidad

El hito H3 pide que una misión pueda **caducar**: se publica con un plazo, y pasado el plazo ya no se puede aceptar. La tentación es llamar a `Instant.now()` dentro del código que decide. Es una dependencia escondida: el resultado depende de cuándo se ejecute, y para probarlo tendrías que esperar (o publicar misiones con plazos de milisegundos y cruzar los dedos).

La solución es la misma que con los repositorios, aplicada al tiempo: que el reloj sea una dependencia que *entra* desde fuera. El JDK ya trae el tipo: `java.time.Clock`, una clase con la que se pregunta la hora (`clock.instant()`) y que tiene dos fábricas útiles: `Clock.systemUTC()` (el reloj real) y `Clock.fixed(instante, zona)` (un reloj parado, para pruebas). Es un puerto de salida que ya existe, sin escribir una interfaz nueva: por eso no crea paquete. El coste es que la capa de aplicación importa `java.time.Clock`; es del JDK y no de un *framework*, así que `ArchitectureTest` lo permite (la regla de `application` admite `java..`). La alternativa, una interfaz propia (`TimeProvider`) con una implementación, evita esa importación a cambio de más código; para un servicio de la complejidad de GuildBoard no compensa, y la sustitución de `Clock` en pruebas es igual de fácil.

Recuerda el ejemplo ilustrativo del capítulo 4: el bean del reloj se declara en la configuración,

config/BeanConfig.java (fragmento: el reloj)

```
@Bean
public Clock clock() {
    return Clock.systemUTC();
}
```

y llega a los servicios como un parámetro más. Falta decidir quién conoce la fecha límite. Siguiendo el criterio del capítulo 5, `Quest` guarda un dato (`expiresAt`, un `Instant`) y recibe la *hora actual* como argumento, igual que recibía el rango y el recuento: el agregado no guarda un reloj ni busca la hora por su cuenta.

domain/Quest.java (fragmento: caducidad)

```
private final Instant expiresAt;        // null si no caduca

public static Quest draft(QuestId id, String title, Reward reward,
                          Rank requiredRank) {
    return draft(id, title, reward, requiredRank, null);
}

public static Quest draft(QuestId id, String title, Reward reward,
                          Rank requiredRank, Instant expiresAt) {
    Objects.requireNonNull(id, "id");
    Objects.requireNonNull(reward, "reward");
    Objects.requireNonNull(requiredRank, "requiredRank");
    if (title == null || title.isBlank()) {
        throw new IllegalArgumentException("title must not be blank");
    }
    return new Quest(id, title.strip(), reward, requiredRank, expiresAt);
}

public boolean isExpired(Clock clock) {
    Objects.requireNonNull(clock, "clock");
    return expiresAt != null && !clock.instant().isBefore(expiresAt);
}

public void requireNotExpired(Clock clock) {
    if (isExpired(clock)) {
        throw new DomainException(
            new DomainError.QuestExpired(id, expiresAt));
    }
}

public Optional<Instant> getExpiresAt() {
    return Optional.ofNullable(expiresAt);
}
```

Tres detalles. El `draft` de cuatro parámetros sigue existiendo y delega en el nuevo con `null`, así que las pruebas y el código del capítulo 5 no cambian. `isExpired` usa `!isBefore`: una misión caduca *en* el instante de su fecha límite, no un instante después. Y el `getExpiresAt` escrito a mano devuelve un `Optional` como `getAssignedTo`: Lombok no genera un *getter* si ya existe uno con ese nombre.

Aceptar una misión caducada es una regla de negocio violada, así que merece su error con nombre. Es la séptima variante de `DomainError`, y como el compilador obliga, hay que tratarla en los dos `switch` que ya existen (`code()` y `describe`), sin `default`:

domain/DomainError.java (fragmento: variante nueva)

```
record QuestExpired(QuestId questId, Instant expiresAt)
        implements DomainError { }

// dentro de code():
case QuestExpired e -> "QUEST_EXPIRED";
```

domain/DomainException.java (fragmento: describe)

```
case DomainError.QuestExpired(QuestId id, Instant expiresAt) ->
    "quest " + id.value() + " expired at " + expiresAt;
```

(Ambos archivos necesitan `import java.time.Instant;`. Están completos en la solución del ejercicio 6.5.)

Ahora los servicios. `PublishQuestCommand` gana la duración opcional (`Duration validFor`, `null` si la misión no caduca) y `PublishQuestService` calcula la fecha límite con el reloj:

application/port/in/PublishQuestCommand.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.Rank;
import com.guildboard.domain.Reward;
import java.time.Duration;

public record PublishQuestCommand(String title, Reward reward,
                                  Rank requiredRank, Duration validFor) {
}
```

application/service/PublishQuestService.java

```
package com.guildboard.application.service;

import com.guildboard.application.port.in.PublishQuestCommand;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.time.Clock;
import java.time.Instant;
import java.util.UUID;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class PublishQuestService implements PublishQuestUseCase {

    private final QuestRepository quests;
    private final Clock clock;

```

```
    @Override
    public Quest publish(PublishQuestCommand command) {
        Instant expiresAt = command.validFor() == null
            ? null
            : clock.instant().plus(command.validFor());
        Quest quest = Quest.draft(
            new QuestId(UUID.randomUUID().toString()),
            command.title(), command.reward(), command.requiredRank(),
            expiresAt);
        quest.publish();
        quests.save(quest);
        return quest;
    }
}
```

El servicio de aceptar recibe el reloj y llama a `requireNotExpired` justo antes de `accept`; el resto del método no cambia. Está entero en la solución del ejercicio de proyecto. Observa el compromiso: la comprobación de caducidad y la de `accept` son dos llamadas, y quien las olvide una no se entera. La alternativa, cambiar la firma a `accept(AdventurerId, Rank, int, Clock)`, mete la regla dentro de una sola llamada a costa de tocar la firma del capítulo anterior y todas las pruebas que la usan. El curso mantiene la firma; si crece el número de reglas de contexto, agruparlas en un objeto es lo siguiente a considerar.

application/service/AcceptQuestService.java (fragmento)

```
Adventurer adventurer = adventurers.findById(command.adventurerId())
    .orElseThrow(() -> new DomainException(
        new DomainError.AdventurerNotFound(command.adventurerId())));
quest.requireNotExpired(clock);
int active = quests.countActiveBy(adventurer.getId());
quest.accept(adventurer.getId(), adventurer.getRank(), active);
```

Y ya puedes probar la caducidad sin esperar: con `Clock.fixed` eliges la hora. Lo harás en el ejercicio 6.4.

6.7 NotificationSender: un puerto de salida sin base de datos

Un puerto de salida no tiene por qué guardar datos. Es cualquier cosa que la aplicación necesita de fuera y no debe saber cómo se hace. Cuando un aventurero completa una misión, queremos avisarle. Hoy ese aviso será una línea de *log*; mañana, un correo o un mensaje: el caso de uso no debe cambiar por ello.

El puerto se nombra por su papel (como `QuestRepository`) y habla el vocabulario del dominio:

application/port/out/NotificationSender.java

```
package com.guildboard.application.port.out;

import com.guildboard.domain.AdventurerId;

public interface NotificationSender {

    void send(AdventurerId recipient, String message);
```

```
}
```

El adaptador va en `adapter/out/notification`. Usa `@Slf4j` de Lombok, que genera un *logger* llamado `log`; es el uso de Lombok que anunciaba el capítulo 3. La clase no importa Spring: la registra `BeanConfig`.

`adapter/out/notification/LogNotificationSender.java`

```
package com.guildboard.adapter.out.notification;

import com.guildboard.application.port.out.NotificationSender;
import com.guildboard.domain.AdventurerId;
import lombok.extern.slf4j.Slf4j;

@Slf4j
public class LogNotificationSender implements NotificationSender {

    @Override
    public void send(AdventurerId recipient, String message) {
        log.info("notification to {}: {}", recipient.value(), message);
    }
}
```

`CompleteQuestService` gana la dependencia y envía el aviso al final:

`application/service/CompleteQuestService.java (fragmento)`

```
private final QuestRepository quests;
private final GrantExperienceUseCase grantExperience;
private final NotificationSender notifications;

// al final de complete, antes de devolver la misi^^c3^^b3n:
notifications.send(assignee,
    "quest completed: " + quest.getTitle());
return quest;
```

Y `BeanConfig` añade el bean (`new LogNotificationSender()`, con el tipo del puerto como retorno) y se lo pasa al servicio. El día que haya un adaptador de correo, cambiará esa línea de la configuración y nada más.

Si el aviso falla, ¿falla completar?

Con este código, una excepción de `send` se propaga y el cliente recibe un error *después* de que la misión ya se guardó como completada y la experiencia se dio: otro caso de operación a medias, como los del capítulo 5. El adaptador de *log* no falla, así que hoy no importa. Un adaptador de correo real tendría que decidir qué hacer; el capítulo 11 muestra una forma, publicar un evento y desacoplar el aviso.

6.8 En el proyecto: hito H3, parte 1

Hito 3 (parte 1): fronteras limpias

Al terminar el ejercicio 6.5, la API de GuildBoard tiene su propio modelo (DTO en `dto`, mapeadores en `mapper`), valida la forma de las peticiones, responde con `404`, `409`, `422` o `400` y un `ProblemDetail` en lugar de `500`, y las misiones pueden caducar con un `Clock` inyectado. Completar una misión envía un aviso por `NotificationSender`. Archivos nuevos:

Archivos nuevos de GuildBoard tras el capítulo 6

```
adapter/in/web/dto/       PublishQuestRequest, AcceptQuestRequest,
                          RegisterAdventurerRequest, QuestResponse,
                          AdventurerResponse
adapter/in/web/mapper/    QuestWebMapper, AdventurerWebMapper
adapter/in/web/error/     DomainExceptionHandler
adapter/out/notification/ LogNotificationSender
application/port/out/     NotificationSender
test (domain)/            QuestExpirationTest
```

Y se modifican `Quest`, `DomainError`, `DomainException`, `PublishQuestCommand`, los servicios de publicar, aceptar y completar, los dos controladores, `BeanConfig` y `build.gradle`.

6.8.1 Probar el hito con curl

Arranca con `./gradlew bootRun`. Los identificadores son de ejemplo, las respuestas se muestran partidas en varias líneas y el orden de los campos de un `ProblemDetail` puede variar. Primero, los rechazos de la web: un nombre en blanco, y una recompensa de cero (el dominio la admitiría, la API no):

```
$ curl -s -X POST http://localhost:8080/adventurers \
    -H "Content-Type: application/json" -d '{"name":""}'
{"type":"about:blank","title":"Bad Request","status":400,
 "detail":"request body is not valid","instance":"/adventurers",
 "errors":["name: must not be blank"]}
$ curl -s -o /dev/null -w "%{http_code}\n" -X POST \
    http://localhost:8080/quests -H "Content-Type: application/json" \
    -d '{"title":"Gratis","reward":0,"requiredRank":"NOVICE"}'
400
```

(El texto exacto de los mensajes de validación depende del idioma de tu sistema.) Un rango desconocido llega al mapeador, que lanza `IllegalArgumentException`, y el manejador lo convierte en `400`:

```
$ curl -s -X POST http://localhost:8080/quests \
    -H "Content-Type: application/json" \
    -d '{"title":"X","reward":10,"requiredRank":"HERO"}'
{"type":"about:blank","title":"Bad Request","status":400,
 "detail":"unknown rank: HERO","instance":"/quests"}
```

Ahora el escenario del capítulo 5 con las respuestas nuevas. Registra a Mara (`NOVICE`), publica

un dragón de rango `ADEPT` y un sótano de rango `NOVICE` que caduca en una hora, y guarda los ids en variables de la shell (`MARA`, `DRAGON`, `CELLAR`) como entonces. La respuesta del sótano muestra la fecha límite:

```
$ curl -s -X POST http://localhost:8080/quests \
    -H "Content-Type: application/json" \
    -d '{"title":"Limpiar el s^^c3^^b3tano","reward":150,
         "requiredRank":"NOVICE","validForMinutes":60}'
{"id":"b2a6f0d4-8c3e-4a17-9e55-1d7c3b6a9f02",
 "title":"Limpiar el s^^c3^^b3tano","reward":150,"requiredRank":"NOVICE",
 "status":"PUBLISHED","assignedTo":null,
 "expiresAt":"2026-10-08T11:30:00Z"}
```

Los rechazos de negocio ya no son `500` (tras el ejercicio 6.3): Mara no alcanza el rango del dragón, y una misión inexistente da `404`:

```
$ curl -s -X POST http://localhost:8080/quests/$DRAGON/accept \
    -H "Content-Type: application/json" \
    -d '{"adventurerId":"'$MARA'"}'
{"type":"about:blank","title":"Business rule violated","status":422,
 "detail":"rank NOVICE cannot take a quest that requires ADEPT",
 "instance":"/quests/c91e7a30-4d5b-4c28-8a0f-6e3b2d1f9a45/accept",
 "code":"RANK_TOO_LOW"}
$ curl -s -o /dev/null -w "%{http_code}\n" -X POST \
    http://localhost:8080/quests/no-existe/complete
404
```

Acepta y completa el sótano. La respuesta es la de siempre, con `expiresAt`; y en la consola de la aplicación aparece una línea de log del adaptador de notificaciones con el texto `notification to <id de Mara>: quest completed: Limpiar el sótano`. Para ver una caducidad real, publica otra misión con `"validForMinutes":1`, espera algo más de un minuto y acéptala: responde `409` con `"code":"QUEST_EXPIRED"`. (Si no quieres esperar, para eso está el ejercicio 6.4.)

6.9 Ejercicios

Los ejercicios parten del código del capítulo (el árbol de la verificación de estructura 2 figura arriba, y las secciones anteriores te han dado los cambios de `Quest`, de los servicios y del resto). El ejercicio de proyecto 6.5 reúne todo el código, y su solución incluye los cambios de los demás. Empieza cada ejercicio con el árbol de trabajo limpio (`git status` sin nada pendiente) y termina con un *commit*.

Ejercicio 6.1: El mapeador de misiones ★★☆☆☆

Haz con las misiones lo que la sección 6.3 hizo con los aventureros. Los DTO de misión quedan así (sin método `from`); `validForMinutes` es un `Integer` porque puede faltar:

dto: PublishQuestRequest y QuestResponse

```
public record PublishQuestRequest(String title, int reward,
                                  String requiredRank,
                                  Integer validForMinutes) { }

public record QuestResponse(String id, String title, int reward,
                            String requiredRank, String status,
                            String assignedTo, String expiresAt) { }
```

1. Crea el paquete `adapter.in.web.dto`. Mueve a él `PublishQuestRequest`, `AcceptQuestRequest` y `QuestResponse` (cambian la línea `package` y los imports del controlador), y deja `PublishQuestRequest` y `QuestResponse` como arriba.
2. Crea `QuestWebMapper` en `adapter.in.web.mapper`: clase `final`, constructor privado y métodos estáticos, como `AdventurerWebMapper`.
3. `toResponse(Quest)`: copia los campos; los `enum` con `name()`; `assignedTo` y `expiresAt` en texto, o `null` si no hay.
4. `toCommand(PublishQuestRequest)`: devuelve el `PublishQuestCommand` con la recompensa como `Reward`, el rango como `Rank` (un texto que no sea un rango lanza `IllegalArgumentException` con el mensaje `unknown rank: <texto>`) y `validForMinutes` como `Duration` o `null`.
5. En `QuestController`, sustituye cada `QuestResponse.from(...)` por el mapeador y usa `toCommand` al publicar.
6. Por escrito: nombra tres cosas que saldrían mal si el controlador devolviera el `Quest` del dominio en lugar de `QuestResponse`.

Está bien si: `./gradlew test` termina con `BUILD SUCCESSFUL`; `QuestWebMapper` no importa nada de `org.springframework`; ningún método público del controlador devuelve un tipo de `com.guildboard.domain`; publicar una misión con `curl` devuelve el mismo JSON de antes más `"expiresAt":null`; y tu lista tiene tres cosas distintas.

Pista: *`getAssignedTo()` y `getExpiresAt()` devuelven un `Optional`: `map` y `orElse(null)` lo convierten en texto o `null`. Para la fecha, `Instant::toString`.*

Guiado ▪ solución en la pág. 184

Ejercicio 6.2: Validar la publicación de una misión ★★★☆☆

Con el starter de validación instalado (sección 6.4), protege `POST /quests` y `POST /quests/{id}/accept`.

1. Anota `PublishQuestRequest`: título no vacío y de 80 caracteres como máximo; recompensa mayor que cero; rango no vacío; y, si viene, `validForMinutes` mayor que cero.
2. Anota `AcceptQuestRequest`: `adventurerId` no vacío.
3. Pon `@Valid` en los parámetros `@RequestBody` de esos dos métodos del controlador.
4. Añade a `DomainExceptionHandler` un método para `MethodArgumentNotValidException` que responda `400` con un `ProblemDetail` y una propiedad `errors`: una lista de textos `campo: mensaje`.

5. Comprueba con `curl` tres peticiones: título en blanco, recompensa 0 y una petición válida.
6. Por escrito: ¿por qué la API rechaza `"reward":0` si `new Reward(0)` es válido en el dominio?

Está bien si: el título en blanco y la recompensa 0 dan `400` con `errors` nombrando el campo; la petición válida sigue dando `201`; no has tocado nada de `domain`; y tu respuesta distingue una política de la API de una regla del negocio.

Pista: *`exception.getBindingResult().getFieldErrors()` devuelve objetos con `getField()` y `getDefaultMessage()`.*

Práctica ▪ solución en la pág. 188

Ejercicio 6.3: Un estado HTTP por cada error de dominio ★★★☆☆

La primera versión de `DomainExceptionHandler` responde `422` a todo. Afínala con los criterios de la sección 6.5 (404, 409 y 422).

1. Escribe en el manejador un método privado y estático `statusFor(DomainError)` que devuelva un `HttpStatusCode` con un `switch` sobre las siete variantes, sin `default`.
2. Haz que `handleDomain` use ese método y conserve la propiedad `code`.
3. Comenta en una línea por qué elegiste cada estado.
4. Comprueba con `curl`: aceptar una misión que no existe, completar una misión que solo está publicada, y un `NOVICE` que acepta una misión `ADEPT`.
5. Quita una variante del `switch` y anota el mensaje del compilador; luego devuélvela.

Está bien si: el `switch` es exhaustivo y no tiene `default`; las tres peticiones dan `404`, `409` y `422`, respectivamente, con el `code` de cada error en el cuerpo; y el compilador rechaza el `switch` cuando falta una variante.

Pista: *la regla práctica es: ¿falla porque el recurso no existe, porque su estado actual no lo permite o porque el solicitante no cumple una regla?*

Práctica ▪ solución en la pág. 191

Ejercicio 6.4: Probar la caducidad sin esperar ★★★☆☆

Usa JUnit (capítulo 3) y AssertJ (capítulo 5). Escribe las pruebas de la caducidad de `Quest`. Ninguna debe esperar ni leer la hora real.

1. Crea `QuestExpirationTest` en `src/test/java/com/guildboard/domain/`, con una constante para el plazo `2026-10-10T12:00:00Z` y un método auxiliar `clockAt(String)` que devuelva un `Clock.fixed` en UTC.
2. Prueba 1: un segundo antes del plazo, `isExpired` es falso.
3. Prueba 2: exactamente en el plazo, es verdadero.
4. Prueba 3: un segundo después del plazo, es verdadero.
5. Prueba 4: una misión sin plazo (el `draft` de cuatro parámetros) no caduca nunca, aunque el reloj marque el año 2999.
6. Prueba 5: `requireNotExpired` pasado el plazo lanza `DomainException` cuyo error es `QuestExpired` con el identificador de la misión y el plazo.
7. Ejecuta `./gradlew test` y haz un *commit*.

Está bien si: las cinco pruebas pasan; ninguna usa `Thread.sleep`, `Instant.now()` ni arranca Spring; y si cambias en `Quest.isExpired` la condición `!isBefore` por `isAfter`, la prueba 2 falla.

Pista: *`Clock.fixed(instante, ZoneOffset.UTC)` devuelve un reloj que siempre marca ese instante.*

Test ▪ solución en la pág. 193

Ejercicio 6.5: Hito H3 (parte 1): fronteras limpias ★★★★☆

Parte del proyecto tal como lo dejó el capítulo 5 (H2 completo). Implementa la primera mitad de H3.

1. Añade `spring-boot-starter-validation` a `build.gradle`.
2. Dominio: `QuestExpired` en `DomainError` y `DomainException`, y los cambios de `Quest` (`expiresAt`, `isExpired`, `requireNotExpired`).
3. Aplicación: `PublishQuestCommand` con `validFor`, `NotificationSender`, y los servicios de publicar, aceptar y completar con el reloj y el puerto de avisos.
4. Adaptadores: los cinco DTO en `dto`, los dos mapeadores en `mapper`, `DomainExceptionHandler` en `error` (con los estados del ejercicio 6.3 y la validación del ejercicio 6.2), los controladores actualizados y `LogNotificationSender` en `adapter/out/notification`.
5. Registra el reloj, el adaptador de avisos y los servicios cambiados en `BeanConfig`.
6. Añade `QuestExpirationTest` (ejercicio 6.4).
7. Comprueba el escenario de la sección 6.8.1, comprueba el árbol con la lista de la verificación de estructura 2 y haz un *commit* con el mensaje `H3a: DTO, mapeadores, errores HTTP, Clock y NotificationSender`.

Está bien si: `./gradlew build` termina con `BUILD SUCCESSFUL` (compila y pasan las pruebas, `ArchitectureTest` incluida); `domain` y `application` no importan Spring, `jakarta` ni `dto`; no hay `Instant.now()` ni `Clock.systemUTC()` fuera de `BeanConfig`; ningún método público de un controlador devuelve un tipo del dominio; el escenario da `400`, `404`, `409` o `422` donde debe y nunca `500`; y el árbol de tu proyecto coincide con el de la verificación de estructura 2.

Pista: *construye de dentro afuera: dominio, puertos, servicios, DTO, mapeadores, controladores y, al final, `BeanConfig`. Compila después de cada capa.*

Proyecto ▪ solución en la pág. 196

6.10 Resumen

Resumen

- Un límite pide su propio modelo. Devolver el dominio por la API ata el contrato al dominio, publica cada *getter*, obliga a poner anotaciones del borde en `domain` y mezcla cambios por razones distintas.
- Los DTO son *records* de campos simples en `adapter/in/web/dto`; los `enum` del dominio salen como `String`. Los mapeadores (`adapter/in/web/mapper`) son clases estáticas y puras, sin Spring ni lógica de negocio, escritas a mano para que el compilador avise de

los campos que faltan.

- Validar en el borde (`spring-boot-starter-validation`, `@Valid`, `@NotBlank`, `@Positive`...) protege al cliente y da todos los errores a la vez; las invariantes del dominio protegen al dominio de cualquier puerta. Una no sustituye a la otra y pueden no coincidir.
- `@RestControllerAdvice` con `@ExceptionHandler` y `ProblemDetail` traduce errores a respuestas en `adapter/in/web/error`. El dominio solo aporta `code()`. Criterio: 404 si no existe, 409 si choca con el estado actual, 422 si el negocio rechaza al solicitante, 400 si los datos están mal formados.
- El tiempo es una dependencia: `java.time.Clock` entra desde fuera (bean en `BeanConfig`, `Clock.fixed` en pruebas). `Quest` guarda `expiresAt` y recibe el reloj como argumento en `isExpired(Clock)`; no crea paquete.
- Un puerto de salida no tiene por qué guardar datos: `NotificationSender` se implementa hoy con un adaptador de *log* (`adapter/out/notification`) y mañana con otro, sin tocar el caso de uso.
- La verificación de estructura 2 recoge los paquetes nuevos: `dto`, `mapper`, `error` y `notification`; la capa de aplicación sigue dependiendo solo de `domain` y del JDK. H3 queda a medias: el capítulo 7 corta el monolito en módulos.

Soluciones de los ejercicios

Solución del ejercicio 6.1

enunciado en la pág. 180

Los tres DTO de misión, ya en `adapter/in/web/dto`:

`adapter/in/web/dto/PublishQuestRequest.java`

```java
package com.guildboard.adapter.in.web.dto;

public record PublishQuestRequest(String title, int reward,
                                  String requiredRank,
                                  Integer validForMinutes) {
}
```

`adapter/in/web/dto/AcceptQuestRequest.java`

```java
package com.guildboard.adapter.in.web.dto;

public record AcceptQuestRequest(String adventurerId) {
}
```

adapter/in/web/dto/QuestResponse.java

```java
package com.guildboard.adapter.in.web.dto;

public record QuestResponse(String id, String title, int reward,
                            String requiredRank, String status,
                            String assignedTo, String expiresAt) {
}
```

El mapeador:

adapter/in/web/mapper/QuestWebMapper.java

```java
package com.guildboard.adapter.in.web.mapper;

import com.guildboard.adapter.in.web.dto.PublishQuestRequest;
import com.guildboard.adapter.in.web.dto.QuestResponse;
import com.guildboard.application.port.in.PublishQuestCommand;
import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.Quest;
import com.guildboard.domain.Rank;
import com.guildboard.domain.Reward;
import java.time.Duration;
import java.time.Instant;

public final class QuestWebMapper {

    private QuestWebMapper() {
    }

    public static PublishQuestCommand toCommand(
            PublishQuestRequest request) {
        Duration validFor = request.validForMinutes() == null
            ? null
            : Duration.ofMinutes(request.validForMinutes());
        return new PublishQuestCommand(
            request.title(),
            new Reward(request.reward()),
            parseRank(request.requiredRank()),
            validFor);
    }

    public static QuestResponse toResponse(Quest quest) {
        return new QuestResponse(
            quest.getId().value(),
            quest.getTitle(),
            quest.getReward().gold(),
            quest.getRequiredRank().name(),
            quest.getStatus().name(),
            quest.getAssignedTo().map(AdventurerId::value).orElse(null),
            quest.getExpiresAt().map(Instant::toString).orElse(null));
    }

    private static Rank parseRank(String value) {
        if (value != null) {
            for (Rank rank : Rank.values()) {
                if (rank.name().equals(value)) {
                    return rank;
                }
            }
        }
        throw new IllegalArgumentException("unknown rank: " + value);
    }
}
```

Y el controlador, que no devuelve ningún tipo del dominio:

adapter/in/web/QuestController.java (1/2)

```java
package com.guildboard.adapter.in.web;

import com.guildboard.adapter.in.web.dto.AcceptQuestRequest;
import com.guildboard.adapter.in.web.dto.PublishQuestRequest;
import com.guildboard.adapter.in.web.dto.QuestResponse;
import com.guildboard.adapter.in.web.mapper.QuestWebMapper;
import com.guildboard.application.port.in.AcceptQuestCommand;
import com.guildboard.application.port.in.AcceptQuestUseCase;
import com.guildboard.application.port.in.CompleteQuestCommand;
import com.guildboard.application.port.in.CompleteQuestUseCase;
import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.net.URI;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/quests")
@RequiredArgsConstructor
public class QuestController {

    private final PublishQuestUseCase publishQuest;
    private final GetQuestUseCase getQuest;
    private final AcceptQuestUseCase acceptQuest;
    private final CompleteQuestUseCase completeQuest;

    @PostMapping
    public ResponseEntity<QuestResponse> publish(
            @RequestBody PublishQuestRequest request) {
        Quest quest = publishQuest.publish(
            QuestWebMapper.toCommand(request));
        return ResponseEntity
            .created(URI.create("/quests/" + quest.getId().value()))
            .body(QuestWebMapper.toResponse(quest));
    }

    @GetMapping("/{id}")
    public ResponseEntity<QuestResponse> get(@PathVariable String id) {
        return getQuest.get(new QuestId(id))
            .map(QuestWebMapper::toResponse)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }
```

adapter/in/web/QuestController.java (2/2: sigue dentro de la clase)

```
    @PostMapping("/{id}/accept")
    public QuestResponse accept(@PathVariable String id,
                                @RequestBody AcceptQuestRequest request) {
        Quest quest = acceptQuest.accept(new AcceptQuestCommand(
            new QuestId(id), new AdventurerId(request.adventurerId())));
        return QuestWebMapper.toResponse(quest);
    }

    @PostMapping("/{id}/complete")
    public QuestResponse complete(@PathVariable String id) {
        Quest quest = completeQuest.complete(
            new CompleteQuestCommand(new QuestId(id)));
        return QuestWebMapper.toResponse(quest);
    }
}
```

Por qué funciona. El mapeador es la única clase que conoce a la vez el JSON de la API y el `Quest`; el controlador solo encadena traducción, llamada y traducción. Como `QuestResponse` es un *record*, si mañana añades un campo, `toResponse` deja de compilar hasta que lo rellenes. El método privado `parseRank` da un mensaje propio en lugar del de `Rank.valueOf`, que incluiría el nombre completo de la clase del dominio.
Tres cosas que saldrían mal al devolver el `Quest` (vale cualquiera de estas, o similares): (1) el identificador y la recompensa saldrían como objetos `{"value":"abc"}` y `{"gold":500}`, y el cliente quedaría atado a esa forma; (2) cada *getter* del agregado se publicaría, también los que se añadan mañana para uso interno; (3) renombrar una constante de `Rank` o `QuestStatus` cambiaría la API sin que nadie lo decidiera; (4) harían falta anotaciones de Jackson o de validación en `domain`, y las de `jakarta` las prohíbe `ArchitectureTest`.
Errores típicos: anotar el mapeador con `@Component` (no hace falta y lo ata a Spring); meter una decisión de negocio en el mapeador; olvidar actualizar los *imports* del controlador tras mover los DTO; y dejar un `QuestResponse.from` sin borrar mientras el *record* ya no lo tiene.

Solución del ejercicio 6.2

enunciado en la pág. 181

Los DTO anotados:

adapter/in/web/dto/PublishQuestRequest.java

```
package com.guildboard.adapter.in.web.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;

public record PublishQuestRequest(
        @NotBlank @Size(max = 80) String title,
        @Positive int reward,
        @NotBlank String requiredRank,
        @Positive Integer validForMinutes) {
}
```

adapter/in/web/dto/AcceptQuestRequest.java

```java
package com.guildboard.adapter.in.web.dto;

import jakarta.validation.constraints.NotBlank;

public record AcceptQuestRequest(@NotBlank String adventurerId) {
}
```

En `QuestController`, `import jakarta.validation.Valid;` y `@Valid` en los dos parámetros:

adapter/in/web/QuestController.java (los dos métodos que cambian)

```java
@PostMapping
public ResponseEntity<QuestResponse> publish(
        @Valid @RequestBody PublishQuestRequest request) {
    Quest quest = publishQuest.publish(QuestWebMapper.toCommand(request));
    return ResponseEntity
        .created(URI.create("/quests/" + quest.getId().value()))
        .body(QuestWebMapper.toResponse(quest));
}

@PostMapping("/{id}/accept")
public QuestResponse accept(
        @PathVariable String id,
        @Valid @RequestBody AcceptQuestRequest request) {
    Quest quest = acceptQuest.accept(new AcceptQuestCommand(
        new QuestId(id), new AdventurerId(request.adventurerId())));
    return QuestWebMapper.toResponse(quest);
}
```

Y el manejador, con el método nuevo:

adapter/in/web/error/DomainExceptionHandler.java

```java
package com.guildboard.adapter.in.web.error;

import com.guildboard.domain.DomainException;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class DomainExceptionHandler {

    @ExceptionHandler(DomainException.class)
    public ProblemDetail handleDomain(DomainException exception) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatusCode.valueOf(422), exception.getMessage());
        problem.setTitle("Business rule violated");
        problem.setProperty("code", exception.getError().code());
        return problem;
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ProblemDetail handleMalformed(
            IllegalArgumentException exception) {
        return ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, exception.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail handleInvalidBody(
            MethodArgumentNotValidException exception) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, "request body is not valid");
        List<String> errors = exception.getBindingResult()
            .getFieldErrors().stream()
            .map(error -> error.getField() + ": "
                + error.getDefaultMessage())
            .toList();
        problem.setProperty("errors", errors);
        return problem;
    }
}
```

Por qué funciona. Con `@Valid`, Spring valida el objeto después de leer el JSON y antes de entrar en el método; si hay violaciones lanza `MethodArgumentNotValidException`, que el manejador convierte en un `400` con la lista de todos los campos que fallan. En un record las anotaciones del componente se aplican al campo, al *getter* y al parámetro del constructor, y Hibernate Validator las lee. `@Positive` sobre un `Integer` `null` se considera válido: por eso `validForMinutes` puede faltar sin error, y solo se valida cuando viene.

Respuesta 6. `reward` mayor que cero es una política de *esta* API (publicar misiones gratuitas no se permite). La regla del negocio, R7, solo exige que la recompensa no sea

negativa, y `Reward` sigue aceptando cero, porque otra puerta de entrada futura podría necesitarlo. Cada control tiene su nivel: el borde puede ser más estricto que el dominio, nunca más laxo.
Errores típicos: olvidar el `@Valid` del parámetro (las anotaciones se ignoran en silencio); poner `@NotNull` en un `String` esperando rechazar la cadena vacía (eso es `@NotBlank`); y dos manejadores para la misma excepción en dos clases distintas con `@RestControllerAdvice`, lo que hace que gane uno y no el que esperabas.

Solución del ejercicio 6.3 enunciado en la pág. 182

El manejador con un estado por variante:

adapter/in/web/error/DomainExceptionHandler.java (sin el método de validación)

```java
package com.guildboard.adapter.in.web.error;

import com.guildboard.domain.DomainError;
import com.guildboard.domain.DomainException;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class DomainExceptionHandler {

    @ExceptionHandler(DomainException.class)
    public ProblemDetail handleDomain(DomainException exception) {
        DomainError error = exception.getError();
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            statusFor(error), exception.getMessage());
        problem.setTitle("Business rule violated");
        problem.setProperty("code", error.code());
        return problem;
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ProblemDetail handleMalformed(
            IllegalArgumentException exception) {
        return ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, exception.getMessage());
    }

    private static HttpStatusCode statusFor(DomainError error) {
        return switch (error) {
            case DomainError.QuestNotFound e -> HttpStatus.NOT_FOUND;
            case DomainError.AdventurerNotFound e -> HttpStatus.NOT_FOUND;
            case DomainError.InvalidTransition e -> HttpStatus.CONFLICT;
            case DomainError.QuestNotPublished e -> HttpStatus.CONFLICT;
            case DomainError.QuestExpired e -> HttpStatus.CONFLICT;
            case DomainError.RankTooLow e -> HttpStatusCode.valueOf(422);
            case DomainError.TooManyActiveQuests e ->
                HttpStatusCode.valueOf(422);
        };
    }
}
```

(El método de validación del ejercicio anterior se conserva tal cual; está en la versión final del manejador, en la solución del ejercicio de proyecto.)

Error	Estado	Por qué
`QuestNotFound`, `AdventurerNotFound`	404	el recurso que nombra la petición no existe
`InvalidTransition`	409	la misión no está en el estado que exige la operación
`QuestNotPublished`	409	igual: el estado de la misión no permite aceptarla
`QuestExpired`	409	el plazo de la misión ya pasó; es parte de su estado actual
`RankTooLow`	422	la petición es válida y la misión existe, pero el aventurero no cumple la regla
`TooManyActiveQuests`	422	ídem: depende de la situación del aventurero, no de la misión

Las comprobaciones con `curl` dan: aceptar una misión inexistente, `404` con `"code":"QUEST_NOT_FOUND"`; completar una misión `PUBLISHED`, `409` con `"code":"INVALID_TRANSITION"`; y el `NOVICE` sobre la misión `ADEPT`, `422` con `"code":"RANK_TOO_LOW"`. Si quitas la línea de `QuestExpired`, el compilador dice algo como «the switch expression does not cover all possible input values», y apunta al `switch`.

Por qué funciona. Es el mismo mecanismo del capítulo 5: sin `default`, un error nuevo en el dominio no compila hasta que alguien decide qué estado le corresponde. El dominio sigue sin saber nada de HTTP.

Alternativa válida. `QuestExpired` como `410 Gone` (el recurso existió y ya no está disponible), y `TooManyActiveQuests` como `409` (el límite depende de otras misiones del aventurero, que cambian). Lo que no es válido es usar `403` para el rango: se reserva para la autorización, que llega en el capítulo 14.

Errores típicos: usar `default` en el `switch` «por si acaso» (se pierde la ventaja de la exhaustividad); devolver `404` para `QuestNotPublished` (la misión existe); y poner el estado dentro del dominio, por ejemplo como un campo de `DomainError`.

Solución del ejercicio 6.4

enunciado en la pág. 182

Las pruebas completas:

src/test/java/com/guildboard/domain/QuestExpirationTest.java (1/2)

```java
package com.guildboard.domain;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

import java.time.Clock;
import java.time.Instant;
import java.time.ZoneOffset;
import org.junit.jupiter.api.Test;

class QuestExpirationTest {

    private static final Instant DEADLINE =
        Instant.parse("2026-10-10T12:00:00Z");
    private static final QuestId ID = new QuestId("q-1");

    private static Clock clockAt(String instant) {
        return Clock.fixed(Instant.parse(instant), ZoneOffset.UTC);
    }

    private static Quest publishedQuest(Instant expiresAt) {
        Quest quest = Quest.draft(ID, "Clean the cellar", new Reward(50),
                                  Rank.NOVICE, expiresAt);
        quest.publish();
        return quest;
    }

    @Test
    void isNotExpiredOneSecondBeforeTheDeadline() {
        Quest quest = publishedQuest(DEADLINE);

        assertThat(quest.isExpired(clockAt("2026-10-10T11:59:59Z")))
            .isFalse();
    }

    @Test
    void isExpiredExactlyAtTheDeadline() {
        Quest quest = publishedQuest(DEADLINE);

        assertThat(quest.isExpired(clockAt("2026-10-10T12:00:00Z")))
            .isTrue();
    }

    @Test
    void isExpiredOneSecondAfterTheDeadline() {
        Quest quest = publishedQuest(DEADLINE);

        assertThat(quest.isExpired(clockAt("2026-10-10T12:00:01Z")))
            .isTrue();
    }
```

QuestExpirationTest.java (2/2: sigue dentro de la clase)

```
    @Test
    void aQuestWithoutDeadlineNeverExpires() {
        Quest quest = Quest.draft(ID, "Clean the cellar", new Reward(50),
                                  Rank.NOVICE);
        quest.publish();

        assertThat(quest.isExpired(clockAt("2999-01-01T00:00:00Z")))
            .isFalse();
    }

    @Test
    void requireNotExpiredFailsWithQuestExpiredAfterTheDeadline() {
        Quest quest = publishedQuest(DEADLINE);
        Clock late = clockAt("2026-10-10T12:00:01Z");

        assertThatThrownBy(() -> quest.requireNotExpired(late))
            .isInstanceOfSatisfying(DomainException.class, e ->
                assertThat(e.getError()).isEqualTo(
                    new DomainError.QuestExpired(ID, DEADLINE)));
    }
}
```

El código que prueban está en `Quest` y `DomainError`; son estos miembros (los archivos completos están en la solución del ejercicio 6.5). Primero la variante del error y el campo, el constructor privado y los accesores de `Quest`:

domain/DomainError.java (la variante que comprueban)

```
record QuestExpired(QuestId questId, Instant expiresAt)
        implements DomainError { }
```

domain/Quest.java (campo, constructor y getExpiresAt)

```
private final Instant expiresAt;        // null si no caduca

private Quest(QuestId id, String title, Reward reward,
              Rank requiredRank, Instant expiresAt) {
    this.id = id;
    this.title = title;
    this.reward = reward;
    this.requiredRank = requiredRank;
    this.expiresAt = expiresAt;
    this.status = QuestStatus.DRAFT;
}

public Optional<Instant> getExpiresAt() {
    return Optional.ofNullable(expiresAt);
}
```

Y las dos fábricas `draft` (la de cuatro parámetros delega en la de cinco con `null`) y los dos métodos de caducidad:

domain/Quest.java (lo que prueban estas pruebas)

```java
public static Quest draft(QuestId id, String title, Reward reward,
                          Rank requiredRank) {
    return draft(id, title, reward, requiredRank, null);
}

public static Quest draft(QuestId id, String title, Reward reward,
                          Rank requiredRank, Instant expiresAt) {
    Objects.requireNonNull(id, "id");
    Objects.requireNonNull(reward, "reward");
    Objects.requireNonNull(requiredRank, "requiredRank");
    if (title == null || title.isBlank()) {
        throw new IllegalArgumentException("title must not be blank");
    }
    return new Quest(id, title.strip(), reward, requiredRank, expiresAt);
}

public boolean isExpired(Clock clock) {
    Objects.requireNonNull(clock, "clock");
    return expiresAt != null && !clock.instant().isBefore(expiresAt);
}

public void requireNotExpired(Clock clock) {
    if (isExpired(clock)) {
        throw new DomainException(
            new DomainError.QuestExpired(id, expiresAt));
    }
}
```

Por qué funciona. La hora entra como argumento: cada prueba construye el reloj que necesita con `Clock.fixed`, y la misma misión da respuestas distintas con relojes distintos, sin esperar. El caso del plazo exacto es el que fija la decisión de diseño (caduca *en* el instante, no después), y por eso cambiar `!isBefore` por `isAfter` lo rompe. El error se compara entero porque `QuestExpired` es un *record*: la igualdad compara el identificador y el plazo. **Errores típicos:** probar con `Instant.now()` más unos segundos (la prueba depende de la hora y se rompe en máquinas lentas); olvidar el caso del límite exacto, que es donde se equivocan los `<` y los `<=`; y usar `Thread.sleep`, que vuelve la prueba lenta e inestable.

Solución del ejercicio 6.5

enunciado en la pág. 183

Solución de referencia. Estado final de todo lo que cambia el capítulo, con los cambios de los ejercicios 6.1, 6.2 y 6.3 incluidos. No cambian respecto al capítulo 5: `QuestId`, `AdventurerId`, `Rank`, `Reward`, `QuestStatus`, `Adventurer`, los demás puertos y servicios, los repositorios en memoria, `GuildBoardApplication`, `application.yml` y las pruebas del dominio. `QuestExpirationTest` es el del ejercicio 6.4.

Gradle. Solo cambia el bloque `dependencies`:

build.gradle (bloque dependencies)

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

Dominio.

domain/DomainError.java

```java
package com.guildboard.domain;

import java.time.Instant;

public sealed interface DomainError {

    record QuestNotFound(QuestId questId) implements DomainError { }

    record AdventurerNotFound(AdventurerId adventurerId)
            implements DomainError { }

    record InvalidTransition(QuestId questId, QuestStatus actual,
                             QuestStatus expected)
            implements DomainError { }

    record QuestNotPublished(QuestId questId, QuestStatus actual)
            implements DomainError { }

    record QuestExpired(QuestId questId, Instant expiresAt)
            implements DomainError { }

    record RankTooLow(Rank adventurerRank, Rank requiredRank)
            implements DomainError { }

    record TooManyActiveQuests(int active, int limit)
            implements DomainError { }

    default String code() {
        return switch (this) {
            case QuestNotFound e -> "QUEST_NOT_FOUND";
            case AdventurerNotFound e -> "ADVENTURER_NOT_FOUND";
            case InvalidTransition e -> "INVALID_TRANSITION";
            case QuestNotPublished e -> "QUEST_NOT_PUBLISHED";
            case QuestExpired e -> "QUEST_EXPIRED";
            case RankTooLow e -> "RANK_TOO_LOW";
            case TooManyActiveQuests e -> "TOO_MANY_ACTIVE_QUESTS";
        };
    }
}
```

domain/DomainException.java

```java
package com.guildboard.domain;

import java.time.Instant;

public final class DomainException extends RuntimeException {

    private final DomainError error;

    public DomainException(DomainError error) {
        super(describe(error));
        this.error = error;
    }

    public DomainError getError() {
        return error;
    }

    private static String describe(DomainError error) {
        return switch (error) {
            case DomainError.QuestNotFound(QuestId id) ->
                "quest not found: " + id.value();
            case DomainError.AdventurerNotFound(AdventurerId id) ->
                "adventurer not found: " + id.value();
            case DomainError.InvalidTransition(QuestId id,
                    QuestStatus actual, QuestStatus expected) ->
                "quest " + id.value() + " is " + actual
                    + ", expected " + expected;
            case DomainError.QuestNotPublished(QuestId id,
                    QuestStatus actual) ->
                "quest " + id.value() + " is not published, it is "
                    + actual;
            case DomainError.QuestExpired(QuestId id,
                    Instant expiresAt) ->
                "quest " + id.value() + " expired at " + expiresAt;
            case DomainError.RankTooLow(Rank have, Rank need) ->
                "rank " + have + " cannot take a quest that requires "
                    + need;
            case DomainError.TooManyActiveQuests(int active, int limit) ->
                "adventurer has " + active
                    + " active quests, the limit is " + limit;
        };
    }
}
```

domain/Quest.java (1/3)

```java
package com.guildboard.domain;

import java.time.Clock;
import java.time.Instant;
import java.util.Objects;
import java.util.Optional;
import lombok.EqualsAndHashCode;
import lombok.Getter;

@Getter
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
public final class Quest {

    public static final int MAX_ACTIVE_QUESTS = 3;

    @EqualsAndHashCode.Include
    private final QuestId id;
    private final String title;
    private final Reward reward;
    private final Rank requiredRank;
    private final Instant expiresAt;        // null si no caduca
    private QuestStatus status;
    private AdventurerId assignedTo;        // null hasta aceptar

    private Quest(QuestId id, String title, Reward reward,
                  Rank requiredRank, Instant expiresAt) {
        this.id = id;
        this.title = title;
        this.reward = reward;
        this.requiredRank = requiredRank;
        this.expiresAt = expiresAt;
        this.status = QuestStatus.DRAFT;
    }

    public static Quest draft(QuestId id, String title, Reward reward,
                              Rank requiredRank) {
        return draft(id, title, reward, requiredRank, null);
    }

    public static Quest draft(QuestId id, String title, Reward reward,
                              Rank requiredRank, Instant expiresAt) {
        Objects.requireNonNull(id, "id");
        Objects.requireNonNull(reward, "reward");
        Objects.requireNonNull(requiredRank, "requiredRank");
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("title must not be blank");
        }
        return new Quest(id, title.strip(), reward, requiredRank,
                         expiresAt);
    }
```

domain/Quest.java (2/3: sigue dentro de la clase)

```
    public void publish() {
        requireStatus(QuestStatus.DRAFT);
        status = QuestStatus.PUBLISHED;
    }

    public void accept(AdventurerId adventurerId, Rank adventurerRank,
                       int activeQuests) {
        Objects.requireNonNull(adventurerId, "adventurerId");
        Objects.requireNonNull(adventurerRank, "adventurerRank");
        if (activeQuests < 0) {
            throw new IllegalArgumentException(
                "activeQuests must not be negative: " + activeQuests);
        }
        if (status != QuestStatus.PUBLISHED) {
            throw new DomainException(
                new DomainError.QuestNotPublished(id, status));
        }
        if (!adventurerRank.canTake(requiredRank)) {
            throw new DomainException(
                new DomainError.RankTooLow(adventurerRank, requiredRank));
        }
        if (activeQuests >= MAX_ACTIVE_QUESTS) {
            throw new DomainException(new DomainError.TooManyActiveQuests(
                activeQuests, MAX_ACTIVE_QUESTS));
        }
        assignedTo = adventurerId;
        status = QuestStatus.ACCEPTED;
    }

    public void complete() {
        requireStatus(QuestStatus.ACCEPTED);
        status = QuestStatus.COMPLETED;
    }
```

domain/Quest.java (3/3: sigue dentro de la clase)

```
    public boolean isExpired(Clock clock) {
        Objects.requireNonNull(clock, "clock");
        return expiresAt != null && !clock.instant().isBefore(expiresAt);
    }

    public void requireNotExpired(Clock clock) {
        if (isExpired(clock)) {
            throw new DomainException(
                new DomainError.QuestExpired(id, expiresAt));
        }
    }

    // Escritos a mano: Lombok no genera un getter si ya existe uno con
    // ese nombre, y as^^c3^^ad se devuelve un Optional en lugar de null.
    public Optional<AdventurerId> getAssignedTo() {
        return Optional.ofNullable(assignedTo);
    }

    public Optional<Instant> getExpiresAt() {
        return Optional.ofNullable(expiresAt);
    }

    private void requireStatus(QuestStatus expected) {
        if (status != expected) {
            throw new DomainException(
                new DomainError.InvalidTransition(id, status, expected));
        }
    }
}
```

Aplicación. El comando y el servicio de publicar, el puerto de avisos y los servicios de aceptar y completar.

application/port/in/PublishQuestCommand.java

```
package com.guildboard.application.port.in;

import com.guildboard.domain.Rank;
import com.guildboard.domain.Reward;
import java.time.Duration;

public record PublishQuestCommand(String title, Reward reward,
                                  Rank requiredRank, Duration validFor) {
}
```

application/port/out/NotificationSender.java

```java
package com.guildboard.application.port.out;

import com.guildboard.domain.AdventurerId;

public interface NotificationSender {

    void send(AdventurerId recipient, String message);
}
```

application/service/PublishQuestService.java

```java
package com.guildboard.application.service;

import com.guildboard.application.port.in.PublishQuestCommand;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import java.time.Clock;
import java.time.Instant;
import java.util.UUID;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class PublishQuestService implements PublishQuestUseCase {

    private final QuestRepository quests;
    private final Clock clock;

    @Override
    public Quest publish(PublishQuestCommand command) {
        Instant expiresAt = command.validFor() == null
            ? null
            : clock.instant().plus(command.validFor());
        Quest quest = Quest.draft(
            new QuestId(UUID.randomUUID().toString()),
            command.title(), command.reward(), command.requiredRank(),
            expiresAt);
        quest.publish();
        quests.save(quest);
        return quest;
    }
}
```

application/service/AcceptQuestService.java

```java
package com.guildboard.application.service;

import com.guildboard.application.port.in.AcceptQuestCommand;
import com.guildboard.application.port.in.AcceptQuestUseCase;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.Adventurer;
import com.guildboard.domain.DomainError;
import com.guildboard.domain.DomainException;
import com.guildboard.domain.Quest;
import java.time.Clock;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class AcceptQuestService implements AcceptQuestUseCase {

    private final QuestRepository quests;
    private final AdventurerRepository adventurers;
    private final Clock clock;

    @Override
    public Quest accept(AcceptQuestCommand command) {
        Quest quest = quests.findById(command.questId())
            .orElseThrow(() -> new DomainException(
                new DomainError.QuestNotFound(command.questId())));
        Adventurer adventurer = adventurers.findById(command.adventurerId())
            .orElseThrow(() -> new DomainException(
                new DomainError.AdventurerNotFound(
                    command.adventurerId())));
        quest.requireNotExpired(clock);
        int active = quests.countActiveBy(adventurer.getId());
        quest.accept(adventurer.getId(), adventurer.getRank(), active);
        quests.save(quest);
        return quest;
    }
}
```

application/service/CompleteQuestService.java

```java
package com.guildboard.application.service;

import com.guildboard.application.port.in.CompleteQuestCommand;
import com.guildboard.application.port.in.CompleteQuestUseCase;
import com.guildboard.application.port.in.GrantExperienceCommand;
import com.guildboard.application.port.in.GrantExperienceUseCase;
import com.guildboard.application.port.out.NotificationSender;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.DomainError;
import com.guildboard.domain.DomainException;
import com.guildboard.domain.Quest;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class CompleteQuestService implements CompleteQuestUseCase {

    private final QuestRepository quests;
    private final GrantExperienceUseCase grantExperience;
    private final NotificationSender notifications;

    @Override
    public Quest complete(CompleteQuestCommand command) {
        Quest quest = quests.findById(command.questId())
            .orElseThrow(() -> new DomainException(
                new DomainError.QuestNotFound(command.questId())));
        quest.complete();
        AdventurerId assignee = quest.getAssignedTo().orElseThrow(
            () -> new IllegalStateException("accepted quest without "
                + "assignee: " + quest.getId().value()));
        quests.save(quest);
        int points = quest.getReward().gold();
        if (points > 0) {
            grantExperience.grant(
                new GrantExperienceCommand(assignee, points));
        }
        notifications.send(assignee,
            "quest completed: " + quest.getTitle());
        return quest;
    }
}
```

Adaptadores de entrada: DTO. Los cinco DTO, con las anotaciones de validación.

adapter/in/web/dto/RegisterAdventurerRequest.java

```java
package com.guildboard.adapter.in.web.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record RegisterAdventurerRequest(
        @NotBlank @Size(max = 50) String name) {
}
```

adapter/in/web/dto/AdventurerResponse.java

```java
package com.guildboard.adapter.in.web.dto;

public record AdventurerResponse(String id, String name, String rank,
                                 int experience) {
}
```

adapter/in/web/dto/QuestResponse.java

```java
package com.guildboard.adapter.in.web.dto;

public record QuestResponse(String id, String title, int reward,
                            String requiredRank, String status,
                            String assignedTo, String expiresAt) {
}
```

adapter/in/web/dto/PublishQuestRequest.java

```java
package com.guildboard.adapter.in.web.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;

public record PublishQuestRequest(
        @NotBlank @Size(max = 80) String title,
        @Positive int reward,
        @NotBlank String requiredRank,
        @Positive Integer validForMinutes) {
}
```

adapter/in/web/dto/AcceptQuestRequest.java

```java
package com.guildboard.adapter.in.web.dto;

import jakarta.validation.constraints.NotBlank;

public record AcceptQuestRequest(@NotBlank String adventurerId) {
}
```

Adaptadores de entrada: mapeadores.

adapter/in/web/mapper/AdventurerWebMapper.java

```java
package com.guildboard.adapter.in.web.mapper;

import com.guildboard.adapter.in.web.dto.AdventurerResponse;
import com.guildboard.adapter.in.web.dto.RegisterAdventurerRequest;
import com.guildboard.application.port.in.RegisterAdventurerCommand;
import com.guildboard.domain.Adventurer;

public final class AdventurerWebMapper {

    private AdventurerWebMapper() {
    }

    public static RegisterAdventurerCommand toCommand(
            RegisterAdventurerRequest request) {
        return new RegisterAdventurerCommand(request.name());
    }

    public static AdventurerResponse toResponse(Adventurer adventurer) {
        return new AdventurerResponse(
            adventurer.getId().value(),
            adventurer.getName(),
            adventurer.getRank().name(),
            adventurer.getExperience());
    }
}
```

adapter/in/web/mapper/QuestWebMapper.java

```java
package com.guildboard.adapter.in.web.mapper;

import com.guildboard.adapter.in.web.dto.PublishQuestRequest;
import com.guildboard.adapter.in.web.dto.QuestResponse;
import com.guildboard.application.port.in.PublishQuestCommand;
import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.Quest;
import com.guildboard.domain.Rank;
import com.guildboard.domain.Reward;
import java.time.Duration;
import java.time.Instant;

public final class QuestWebMapper {

    private QuestWebMapper() {
    }

    public static PublishQuestCommand toCommand(
            PublishQuestRequest request) {
        Duration validFor = request.validForMinutes() == null
            ? null
            : Duration.ofMinutes(request.validForMinutes());
        return new PublishQuestCommand(
            request.title(),
            new Reward(request.reward()),
            parseRank(request.requiredRank()),
            validFor);
    }

    public static QuestResponse toResponse(Quest quest) {
        return new QuestResponse(
            quest.getId().value(),
            quest.getTitle(),
            quest.getReward().gold(),
            quest.getRequiredRank().name(),
            quest.getStatus().name(),
            quest.getAssignedTo().map(AdventurerId::value).orElse(null),
            quest.getExpiresAt().map(Instant::toString).orElse(null));
    }

    private static Rank parseRank(String value) {
        if (value != null) {
            for (Rank rank : Rank.values()) {
                if (rank.name().equals(value)) {
                    return rank;
                }
            }
        }
        throw new IllegalArgumentException("unknown rank: " + value);
    }
}
```

Adaptadores de entrada: errores y controladores.

adapter/in/web/AdventurerController.java

```java
package com.guildboard.adapter.in.web;

import com.guildboard.adapter.in.web.dto.AdventurerResponse;
import com.guildboard.adapter.in.web.dto.RegisterAdventurerRequest;
import com.guildboard.adapter.in.web.mapper.AdventurerWebMapper;
import com.guildboard.application.port.in.GetAdventurerUseCase;
import com.guildboard.application.port.in.RegisterAdventurerUseCase;
import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;
import jakarta.validation.Valid;
import java.net.URI;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/adventurers")
@RequiredArgsConstructor
public class AdventurerController {

    private final RegisterAdventurerUseCase registerAdventurer;
    private final GetAdventurerUseCase getAdventurer;

    @PostMapping
    public ResponseEntity<AdventurerResponse> register(
            @Valid @RequestBody RegisterAdventurerRequest request) {
        Adventurer adventurer = registerAdventurer.register(
            AdventurerWebMapper.toCommand(request));
        return ResponseEntity
            .created(URI.create(
                "/adventurers/" + adventurer.getId().value()))
            .body(AdventurerWebMapper.toResponse(adventurer));
    }

    @GetMapping("/{id}")
    public ResponseEntity<AdventurerResponse> get(
            @PathVariable String id) {
        return getAdventurer.get(new AdventurerId(id))
            .map(AdventurerWebMapper::toResponse)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }
}
```

adapter/in/web/error/DomainExceptionHandler.java (1/2)

```java
package com.guildboard.adapter.in.web.error;

import com.guildboard.domain.DomainError;
import com.guildboard.domain.DomainException;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class DomainExceptionHandler {

    @ExceptionHandler(DomainException.class)
    public ProblemDetail handleDomain(DomainException exception) {
        DomainError error = exception.getError();
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            statusFor(error), exception.getMessage());
        problem.setTitle("Business rule violated");
        problem.setProperty("code", error.code());
        return problem;
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ProblemDetail handleMalformed(
            IllegalArgumentException exception) {
        return ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, exception.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail handleInvalidBody(
            MethodArgumentNotValidException exception) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, "request body is not valid");
        List<String> errors = exception.getBindingResult()
            .getFieldErrors().stream()
            .map(error -> error.getField() + ": "
                + error.getDefaultMessage())
            .toList();
        problem.setProperty("errors", errors);
        return problem;
    }
```

adapter/in/web/error/DomainExceptionHandler.java (2/2: sigue dentro de la clase)

```
    private static HttpStatusCode statusFor(DomainError error) {
        return switch (error) {
            case DomainError.QuestNotFound e -> HttpStatus.NOT_FOUND;
            case DomainError.AdventurerNotFound e -> HttpStatus.NOT_FOUND;
            case DomainError.InvalidTransition e -> HttpStatus.CONFLICT;
            case DomainError.QuestNotPublished e -> HttpStatus.CONFLICT;
            case DomainError.QuestExpired e -> HttpStatus.CONFLICT;
            case DomainError.RankTooLow e -> HttpStatusCode.valueOf(422);
            case DomainError.TooManyActiveQuests e ->
                HttpStatusCode.valueOf(422);
        };
    }
}
```

adapter/in/web/QuestController.java (1/2)

```java
package com.guildboard.adapter.in.web;

import com.guildboard.adapter.in.web.dto.AcceptQuestRequest;
import com.guildboard.adapter.in.web.dto.PublishQuestRequest;
import com.guildboard.adapter.in.web.dto.QuestResponse;
import com.guildboard.adapter.in.web.mapper.QuestWebMapper;
import com.guildboard.application.port.in.AcceptQuestCommand;
import com.guildboard.application.port.in.AcceptQuestUseCase;
import com.guildboard.application.port.in.CompleteQuestCommand;
import com.guildboard.application.port.in.CompleteQuestUseCase;
import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import jakarta.validation.Valid;
import java.net.URI;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/quests")
@RequiredArgsConstructor
public class QuestController {

    private final PublishQuestUseCase publishQuest;
    private final GetQuestUseCase getQuest;
    private final AcceptQuestUseCase acceptQuest;
    private final CompleteQuestUseCase completeQuest;

    @PostMapping
    public ResponseEntity<QuestResponse> publish(
            @Valid @RequestBody PublishQuestRequest request) {
        Quest quest = publishQuest.publish(
            QuestWebMapper.toCommand(request));
        return ResponseEntity
            .created(URI.create("/quests/" + quest.getId().value()))
            .body(QuestWebMapper.toResponse(quest));
    }

    @GetMapping("/{id}")
    public ResponseEntity<QuestResponse> get(@PathVariable String id) {
        return getQuest.get(new QuestId(id))
            .map(QuestWebMapper::toResponse)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }
```

adapter/in/web/QuestController.java (2/2: sigue dentro de la clase)

```
    @PostMapping("/{id}/accept")
    public QuestResponse accept(
            @PathVariable String id,
            @Valid @RequestBody AcceptQuestRequest request) {
        Quest quest = acceptQuest.accept(new AcceptQuestCommand(
            new QuestId(id), new AdventurerId(request.adventurerId())));
        return QuestWebMapper.toResponse(quest);
    }

    @PostMapping("/{id}/complete")
    public QuestResponse complete(@PathVariable String id) {
        Quest quest = completeQuest.complete(
            new CompleteQuestCommand(new QuestId(id)));
        return QuestWebMapper.toResponse(quest);
    }
}
```

Adaptador de salida: avisos.

adapter/out/notification/LogNotificationSender.java

```
package com.guildboard.adapter.out.notification;

import com.guildboard.application.port.out.NotificationSender;
import com.guildboard.domain.AdventurerId;
import lombok.extern.slf4j.Slf4j;

@Slf4j
public class LogNotificationSender implements NotificationSender {

    @Override
    public void send(AdventurerId recipient, String message) {
        log.info("notification to {}: {}", recipient.value(), message);
    }
}
```

Raíz de composición. `BeanConfig` registra el reloj y el adaptador de avisos, y pasa el reloj y el puerto a los servicios que los usan. Va en dos trozos, que forman un solo archivo.

config/BeanConfig.java (1/2)

```java
package com.guildboard.config;

import com.guildboard.adapter.out.memory.InMemoryAdventurerRepository;
import com.guildboard.adapter.out.memory.InMemoryQuestRepository;
import com.guildboard.adapter.out.notification.LogNotificationSender;
import com.guildboard.application.port.in.AcceptQuestUseCase;
import com.guildboard.application.port.in.CompleteQuestUseCase;
import com.guildboard.application.port.in.GetAdventurerUseCase;
import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.in.GrantExperienceUseCase;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.application.port.in.RegisterAdventurerUseCase;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.application.port.out.NotificationSender;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.application.service.AcceptQuestService;
import com.guildboard.application.service.CompleteQuestService;
import com.guildboard.application.service.GetAdventurerService;
import com.guildboard.application.service.GetQuestService;
import com.guildboard.application.service.GrantExperienceService;
import com.guildboard.application.service.PublishQuestService;
import com.guildboard.application.service.RegisterAdventurerService;
import java.time.Clock;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public Clock clock() {
        return Clock.systemUTC();
    }

    @Bean
    public NotificationSender notificationSender() {
        return new LogNotificationSender();
    }

    @Bean
    public QuestRepository questRepository() {
        return new InMemoryQuestRepository();
    }

    @Bean
    public AdventurerRepository adventurerRepository() {
        return new InMemoryAdventurerRepository();
    }
```

config/BeanConfig.java (2/2: sigue dentro de la clase)

```
    @Bean
    public PublishQuestUseCase publishQuestUseCase(
            QuestRepository questRepository, Clock clock) {
        return new PublishQuestService(questRepository, clock);
    }

    @Bean
    public GetQuestUseCase getQuestUseCase(
            QuestRepository questRepository) {
        return new GetQuestService(questRepository);
    }

    @Bean
    public RegisterAdventurerUseCase registerAdventurerUseCase(
            AdventurerRepository adventurerRepository) {
        return new RegisterAdventurerService(adventurerRepository);
    }

    @Bean
    public GetAdventurerUseCase getAdventurerUseCase(
            AdventurerRepository adventurerRepository) {
        return new GetAdventurerService(adventurerRepository);
    }

    @Bean
    public GrantExperienceUseCase grantExperienceUseCase(
            AdventurerRepository adventurerRepository) {
        return new GrantExperienceService(adventurerRepository);
    }

    @Bean
    public AcceptQuestUseCase acceptQuestUseCase(
            QuestRepository questRepository,
            AdventurerRepository adventurerRepository, Clock clock) {
        return new AcceptQuestService(questRepository,
                                      adventurerRepository, clock);
    }

    @Bean
    public CompleteQuestUseCase completeQuestUseCase(
            QuestRepository questRepository,
            GrantExperienceUseCase grantExperienceUseCase,
            NotificationSender notificationSender) {
        return new CompleteQuestService(questRepository,
                                        grantExperienceUseCase,
                                        notificationSender);
    }
}
```

Por qué funciona. El controlador solo traduce: el DTO entra, el mapeador lo convierte en un comando, el puerto de entrada hace el trabajo y el mapeador devuelve otro DTO. Los errores del dominio suben como excepciones hasta `DomainExceptionHandler`, que es el único sitio donde se decide su estado HTTP. El reloj y el aviso entran por constructor y se eligen

en `BeanConfig`; ni `domain` ni `application` saben que existe `Clock.systemUTC()` ni un *log*. `ArchitectureTest` pasa con este código: el dominio solo importa `java.time`, la aplicación solo importa dominio y JDK, y `jakarta` aparece solo en `dto` y en los controladores.
Criterios para valorar otras soluciones. Es válida si (a) los modelos de la API no se usan fuera de `adapter/in/web` y los comandos no llevan anotaciones; (b) el controlador no contiene reglas ni conversiones de formato; (c) los errores del dominio se traducen en un único lugar y con `switch` exhaustivo; (d) la validación del borde no sustituye a ninguna invariante del dominio; (e) no hay llamadas a la hora real fuera de la configuración; y (f) `./gradlew test` pasa y las comprobaciones de la verificación de estructura 2 se cumplen.
Alternativa válida. Mapeadores como *beans* de Spring (con `@Component`) inyectados en el controlador, si algún día necesitan colaboradores (por ejemplo, el reloj para calcular la fecha límite en la respuesta); a cambio, se prueban con algo más de ceremonia. Otra: una interfaz propia `TimeProvider` en lugar de `java.time.Clock`.
Errores típicos: registrar el manejador de errores en `BeanConfig` (el escaneo ya lo encuentra, y registrarlo dos veces duplica el manejador); dejar el método `from` en un DTO y que el DTO vuelva a conocer al dominio; meter la recompensa positiva en `Reward` «para que sea coherente» (rompe la regla R7 del documento de dominio); olvidar el bean de `Clock` (la aplicación no arranca: «No qualifying bean of type java.time.Clock»); y actualizar los servicios pero no sus llamadas a constructor en `BeanConfig` (no compila).

CAPÍTULO 7

Hacer cumplir la estructura: Gradle multimódulo

Objetivos del capítulo

- Explicar por qué una convención (una lista, una prueba que alguien puede no ejecutar) no basta para sostener la regla de dependencia y qué añade el compilador.
- Declarar módulos Gradle con `java-library` y elegir entre `api` e `implementation` según lo que ve cada módulo.
- Cortar GuildBoard en `domain`, `application`, `adapters` y `bootstrap`, comprobar el árbol nuevo en la verificación de estructura 3 y comparar un módulo de adaptadores con uno por adaptador.
- Escribir las reglas de ArchUnit que vigilan lo que el compilador ya no puede ver, el aislamiento entre adaptadores de entrada y de salida, en un `ArchitectureTest` dentro de `bootstrap` (hito H3, parte 2).

Hasta ahora la regla de dependencia de GuildBoard se sostiene con tres cosas: tu atención en cada verificación, la lista de cuatro preguntas y una prueba de ArchUnit que analiza el bytecode. Funciona mientras todos estén atentos, y justo ahí está el problema: una arquitectura que depende de que nadie se despiste dura hasta el primer día de prisa. Este capítulo lleva la regla al sitio donde no se puede ignorar, el compilador, cortando el proyecto en cuatro módulos Gradle. Como cambia la organización del repositorio, incluye la verificación de estructura 3 y un ejercicio de comparación de árboles.

7.1 Por qué una convención no basta

Tomemos la última defensa que construiste, `ArchitectureTest` (capítulo 3). Es buena: ArchUnit lee el bytecode, así que esta línea de `PublishQuestService`, sin ningún `import`, también la caza [66]:

Un fragmento que ArchUnit detecta, si alguien ejecuta la prueba

```
@Override
public Quest publish(PublishQuestCommand command) {
    org.springframework.util.Assert.notNull(command, "command");
    // ... el resto del m^^c3^^a9todo
}
```

La debilidad es otra: la prueba solo corre cuando alguien ejecuta `test` o `build` (no en el editor, y se puede saltar con `-x test`), y cualquiera puede editar la propia prueba o su paquete base. Son debilidades de una *convención*: algo que la herramienta no te impide romper.

Los tres niveles con los que el curso vigila la estructura, de más débil a más fuerte, quedan así:

1. **Revisión humana** con la lista de cuatro preguntas, en cada verificación. Cubre todo, pero solo cuando alguien mira.
2. **Pruebas de arquitectura con ArchUnit** (`ArchitectureTest`). Se ejecutan solas en cada `./gradlew build` y entienden Java, pero se pueden saltar o editar.
3. **El compilador**, desde este capítulo. Si el módulo `application` no tiene Spring en su *classpath* (la lista de librerías con la que se compila), una línea que lo use no compila, escrita como se escriba. Y no se puede saltar: sin compilar no hay programa.

El nivel 3 no sustituye a los otros dos. El compilador es tan fuerte como el `build.gradle` que le dice qué ve cada módulo, y ese archivo también lo puede editar cualquiera; por eso la prueba y la revisión siguen. Pero es la primera defensa que convierte «no deberías» en «no puedes».

Dos palabras de vocabulario antes de seguir. Un **módulo Gradle** (en la documentación de Gradle, un *subproyecto*) es una carpeta con su propio `build.gradle` y su propio `src`, que se compila por separado a su propio *jar* y que declara de qué otros módulos depende [14]. Y el **proyecto raíz** es el de la carpeta de `settings.gradle`, que agrupa a los módulos.

7.2 Verificación de estructura 3: el árbol multimódulo

Nota

Este capítulo cambia la estructura de carpetas: de un solo módulo a cuatro. Comprueba que tu proyecto coincide con el árbol de abajo antes de seguir. Los nombres de las dependencias (`api`, `implementation`) se explican en la sección 7.4; aquí basta con leerlos como «puede ver» y «puede ver, pero no lo enseña a quien dependa de él».

7.2.1 El árbol anterior

Así dejó el proyecto el capítulo 6 (árbol T1 más los paquetes de la verificación 2). Un solo módulo, con todo bajo un `src`:

Árbol anterior (T1, tras el capítulo 6)

```
guildboard/
|-- .git/, .gitignore
|-- settings.gradle
|-- build.gradle                el proyecto entero: plugins, Lombok, Spring
|-- gradlew, gradlew.bat
|-- docs/dominio.md
|-- gradle/wrapper/
`-- src/
    |-- main/
    |   |-- resources/application.yml
    |   `-- java/com/guildboard/
    |       |-- GuildBoardApplication.java
    |       |-- domain/
    |       |-- application/     port/in, port/out, service
    |       |-- adapter/
    |       |   |-- in/web/      controladores + dto, mapper, error
    |       |   `-- out/         memory, notification
    |       `-- config/          BeanConfig
    `-- test/java/com/guildboard/
```

```
        |-- ArchitectureTest.java
        `-- domain/
```

7.2.2 El árbol nuevo

Lo nuevo va marcado con `[N]`, lo modificado con `[M]` y lo que solo cambia de sitio, sin tocar su contenido, con `[=]`:

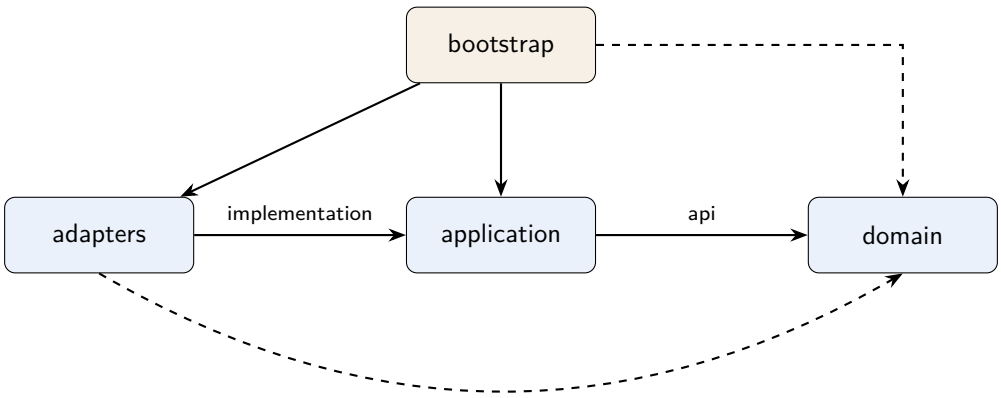
Árbol nuevo (T2, tras el capítulo 7)

```
guildboard/
|-- .git/, .gitignore
|-- settings.gradle                [M] include de los cuatro m^^c3^^b3dulos
|-- build.gradle                   [M] solo configuraci^^c3^^b3n com^^c3^^ban
|-- gradlew, gradlew.bat
|-- docs/dominio.md
|-- gradle/wrapper/
|-- domain/                        [N] java-library; solo Lombok
|   |-- build.gradle               [N]
|   `-- src/
|       |-- main/java/com/guildboard/domain/        [=]
|       `-- test/java/com/guildboard/domain/        [=]
|-- application/                   [N] api domain; Lombok
|   |-- build.gradle               [N]
|   `-- src/main/java/com/guildboard/application/   [=]
|                                  port/in, port/out, service
|-- adapters/                      [N] application; Spring web y validaci^^c3^^
    ↪ b3n
|   |-- build.gradle               [N]
|   `-- src/main/java/com/guildboard/adapter/       [=]
|                                  in/web/{dto,mapper,error}, out/{memory,
|                                  notification}
`-- bootstrap/                     [N] plugin Spring Boot; ensambla; ArchUnit
    |-- build.gradle               [N]
    `-- src/
        |-- main/
        |   |-- java/com/guildboard/
        |   |   |-- GuildBoardApplication.java      [=]
        |   |   `-- config/BeanConfig.java          [=]
        |   `-- resources/application.yml           [=]
        `-- test/java/com/guildboard/
            `-- ArchitectureTest.java               [M] viene de la ra^^c3^^adz
```

Tres cosas conviene ver de un vistazo. Primera: **no cambia ningún paquete Java** ni una línea de código. Lo que cambia es la carpeta física en la que vive cada paquete. Segunda: desaparece el `src` de la raíz, porque el proyecto raíz ya no compila nada (`ArchitectureTest` se va a `bootstrap`). Tercera: `domain` y `application` siguen siendo hermanos, como en el capítulo 3; eso permite este corte sin mover paquetes.

7.2.3 El grafo de dependencias

Cada flecha es una dependencia declarada en un `build.gradle`: «el origen puede ver las clases públicas del destino». Las flechas discontinuas no se declaran, se heredan:



`adapters` ve `domain` sin declararlo porque `application` lo *exporta* con `api`; lo mismo le pasa a `bootstrap`. Y nadie apunta a `adapters` salvo `bootstrap`: ni `domain` ni `application` pueden ver un adaptador, porque su `build.gradle` no lo declara y Gradle, además, no admite ciclos entre módulos.

7.2.4 Cambios respecto al árbol anterior

- `settings.gradle`: incluye los cuatro módulos.
- `build.gradle` raíz: queda solo con la configuración común, incluida la de JUnit.
- `ArchitectureTest.java`: pasa de `src/test` de la raíz a `bootstrap` y gana dos reglas (sección 7.7).
- `domain/`: nuevo módulo con el paquete `domain` y sus pruebas; solo Lombok.
- `application/`: nuevo módulo con `api project(':domain')` y Lombok.
- `adapters/`: nuevo módulo con Spring web y validación; ve `application`.
- `bootstrap/`: nuevo módulo con el plugin de Spring Boot, `GuildBoardApplication`, `BeanConfig`, `application.yml` y ArchUnit.
- Desaparece el `src` de la raíz.

7.2.5 Comprobaciones

- [ ] `domain/build.gradle` declara solo Lombok.
- [ ] `application/build.gradle` declara `api project(':domain')` y Lombok; ni Spring ni `adapters`.
- [ ] Cada adaptador sigue en `in` o en `out`: los paquetes no se han tocado.
- [ ] Las carpetas de tu proyecto coinciden con el árbol.
- [ ] `./gradlew test` pasa, `ArchitectureTest` incluido, y `./gradlew build` también (se comprueba al final de la sección 7.5).

Si algo no coincide con el árbol, corrígelo antes de seguir.

7.3 Principios de componentes: qué se agrupa y quién depende de quién

Cortar en módulos obliga a responder una pregunta que hasta ahora se resolvía con carpetas: qué clases van juntas. Martin llama *componente* a la unidad de despliegue (un *jar* en Java) y dedica a esa pregunta los capítulos 12, 13 y 14 de [28]: «Components», «Component Cohesion» y «Component Coupling». Aquí, un módulo Gradle es un componente. El libro resume la cohesión en tres principios y el acoplamiento en otros tres; estas son las ideas, con mis palabras y aplicadas a GuildBoard:

Sigla	Idea	En GuildBoard
REP	Lo que se reutiliza y lo que se publica es lo mismo; las clases de un componente deben poder presentarse como un conjunto con sentido.	`domain` es un conjunto con sentido propio: el modelo del negocio.
CCP	Agrupa lo que cambia junto y por la misma razón.	DTO, mapeadores y controladores cambian cuando cambia la API: van juntos en `adapters`.
CRP	No obligues a quien usa un componente a depender de lo que no usa.	Un futuro proceso por lotes que use el dominio no debería arrastrar Spring: por eso `domain` no comparte módulo con `adapters`.
ADP	Sin ciclos entre componentes.	El grafo del árbol es acíclico; Gradle se niega a construir uno con ciclos.
SDP	Depende en la dirección de la estabilidad.	Todos apuntan a `domain`, lo que menos debe cambiar; solo el ensamblado apunta a `adapters`, lo que más cambia.
SAP	Un componente estable debe ser abstracto.	Las interfaces (puertos) viven en `application`: lo que los adaptadores implementan es abstracto.

Tres matices honestos. Primero, los principios de cohesión tiran en direcciones opuestas: CCP y REP empujan a componentes grandes, CRP a pequeños, y el diseño es buscar el equilibrio, no cumplirlos todos; Martin lo presenta como una tensión entre ellos. Segundo, el ejemplo de CCP es mejorable: el repositorio en memoria cambia por razones distintas a la API web, y CCP sugeriría separarlo; si no lo hacemos es por coste, y la sección 7.6 lo discute. Tercero, SAP no se cumple a rajatabla (`domain` es estable y está hecho de clases concretas); es una orientación para mirar, no una condena.

Antes de este capítulo, GuildBoard tenía lo que el capítulo 24 de [28] llama un límite *parcial*: las capas estaban separadas por paquetes pero se compilaban y desplegaban juntas, sin pagar el precio de componentes independientes. Es una opción legítima, y el capítulo 25 del mismo libro trata cuándo merece la pena cada tipo de límite. Este capítulo da el paso siguiente porque el límite que más nos importa, `domain` frente a Spring, ya justifica el coste de hacerlo cumplir con el compilador.

7.4 Módulos Gradle: `java-library`, api e implementation

7.4.1 Declarar los módulos

Un proyecto multimódulo empieza en `settings.gradle`, que lista los módulos con `include`. Cada nombre es una carpeta de la raíz con su propio `build.gradle` [14]:

settings.gradle

```
rootProject.name = 'guildboard'

include 'domain', 'application', 'adapters', 'bootstrap'
```

Gradle identifica cada proyecto por su *ruta* (`:domain`, `:application`...), que debe ser única. Hoy los cuatro nombres lo son; en el capítulo 9 habrá varios servicios y tocará decidir cómo se nombran.

7.4.2 El plugin java-library y las dos formas de depender

El plugin `java` que usabas añade las configuraciones `implementation` y `compileOnly`, entre otras. `java-library` añade una más, `api`, y con ella la diferencia que sostiene este capítulo [17]:

- `api`: la dependencia forma parte de la API del módulo. Se *expone*: aparece en el *classpath* de compilación de quien dependa de este módulo.
- `implementation`: la dependencia es un detalle interno. Quien dependa de este módulo **no la ve** al compilar (sí la tendrá en ejecución, porque las clases hacen falta para funcionar).

La regla práctica: usa `implementation` por defecto y `api` solo cuando un tipo de esa dependencia aparece en la firma pública de algo que otros módulos usan. Un ejemplo de GuildBoard: `PublishQuestUseCase` es una interfaz de `application` cuyo método devuelve un `Quest`, una clase de `domain`. Quien llame al caso de uso necesita ver `Quest`; luego `application` declara `api project(':domain')`. En cambio, los controladores de `adapters` usan tipos de Spring en sus firmas, pero ningún otro módulo los llama: Spring es un detalle interno y va con `implementation`.

7.4.3 Lombok y el resto de configuración, módulo a módulo

Dos detalles que sorprenden la primera vez. Uno: `compileOnly` y `annotationProcessor` **no se heredan** entre módulos, de modo que cada módulo que compile código con anotaciones de Lombok (`@Getter`, `@RequiredArgsConstructor`, `@Slf4j`) declara sus dos líneas de Lombok [37]. Hoy lo usan `domain`, `application` y `adapters`. Otro: las versiones de Spring no pueden venir del plugin de Boot en las bibliotecas, porque ese plugin va solo en `bootstrap`. Para que los módulos de biblioteca sigan sin escribir versiones, el proyecto raíz importa el catálogo de versiones de Boot (su *BOM*) en todos los subproyectos, con el truco que describe la guía del plugin [51]: declarar el plugin en la raíz con `apply false` (se carga pero no se aplica) y usar la constante `SpringBootPlugin.BOM_COORDINATES`.

Así queda el `build.gradle` de la raíz. Ya no compila nada: es la configuración común de los subproyectos.

build.gradle (raíz)

```
plugins {
    id 'org.springframework.boot' version '4.1.1' apply false
}

subprojects {
    apply plugin: 'java-library'
    apply plugin: 'io.spring.dependency-management'
```

```
8
9       group = 'com.guildboard'
10      version = '0.0.1-SNAPSHOT'
11
12      java {
13          toolchain {
14              languageVersion = JavaLanguageVersion.of(21)
15          }
16      }
17
18      repositories {
19          mavenCentral()
20      }
21
22      dependencyManagement {
23          imports {
24              mavenBom(org.springframework.boot.gradle.plugin
25                       .SpringBootPlugin.BOM_COORDINATES)
26          }
27      }
28
29      dependencies {
30          testImplementation 'org.springframework.boot:spring-boot-starter-test'
31          testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
32      }
33
34      tasks.named('test') {
35          useJUnitPlatform()
36      }
37  }
```

`subprojects` ejecuta su bloque una vez por cada módulo, como si lo hubieras copiado en sus cuatro `build.gradle`. Aquí vive también la configuración común de pruebas (el starter de test del capítulo 5, el lanzador de JUnit y `useJUnitPlatform()`), para que ningún módulo la repita. ArchUnit, en cambio, no es común: solo lo declara `bootstrap`. La documentación de Gradle lo considera aceptable pero no lo recomienda para builds grandes: prefiere un *plugin de convenciones* en `buildSrc` [15]. Para cuatro módulos, `subprojects` es más corto, y el plan del curso evita `buildSrc` para no sumar otro concepto; si el build crece, es el paso siguiente.

Sin versión en el plugin de bootstrap

Cuando el plugin de Boot ya está cargado por el proyecto raíz, el módulo `bootstrap` lo pide sin versión (`id 'org.springframework.boot'`). Si le pusieras una versión, Gradle se quejaría de que el plugin ya está en el *classpath*. Esa regla es la práctica habitual de Gradle y no la he encontrado escrita literalmente en las páginas citadas, pero sí la técnica de declarar en la raíz con `apply false`; si tu Gradle dijera otra cosa, el mensaje te dirá qué quitar.

7.5 Cortar el monolito en cuatro módulos

Con el árbol comprobado, a mover. Haz el corte en este orden, siempre con `git mv` para que Git conserve el historial de cada archivo. Empieza con el árbol de trabajo limpio (`git status` sin nada pendiente).

7.5.1 Mover las carpetas

Se crean las carpetas padre y se mueve cada paquete a su módulo. No se edita ningún archivo Java:

```
$ mkdir -p domain/src/main/java/com/guildboard \
           domain/src/test/java/com/guildboard \
           application/src/main/java/com/guildboard \
           adapters/src/main/java/com/guildboard \
           bootstrap/src/main/java/com/guildboard \
           bootstrap/src/main/resources \
           bootstrap/src/test/java/com/guildboard
$ S=src/main/java/com/guildboard
$ git mv $S/domain domain/$S/domain
$ git mv src/test/java/com/guildboard/domain \
         domain/src/test/java/com/guildboard/domain
$ git mv src/test/java/com/guildboard/ArchitectureTest.java \
         bootstrap/src/test/java/com/guildboard/ArchitectureTest.java
$ git mv $S/application application/$S/application
$ git mv $S/adapter adapters/$S/adapter
$ git mv $S/config bootstrap/$S/config
$ git mv $S/GuildBoardApplication.java \
         bootstrap/$S/GuildBoardApplication.java
$ git mv src/main/resources/application.yml \
         bootstrap/src/main/resources/application.yml
$ find src -type d -empty -delete
```

(`S` es una variable de la *shell* para no repetir la ruta; si tu *shell* no la conserva entre líneas, escribe la ruta completa.) `ArchitectureTest` se va a `bootstrap` por una razón que se explica más abajo. El último comando borra las carpetas vacías que quedan en el `src` de la raíz. Si alguna no se borra, es que contiene algo que no esperabas: míralo antes de forzar nada.

El `.gitignore` del capítulo 1 no necesita cambios: `build/` y `.gradle/` están sin anclar y valen en cualquier nivel, así que ignoran las carpetas `build` de los cuatro módulos; y `/bin/` y `/out/` siguen anclados a la raíz, de modo que no tocan a `port/out`.

7.5.2 El settings.gradle y el build.gradle raíz

Los dos archivos de la sección 7.4. El de `settings` añade la línea `include`; el raíz se reemplaza entero por el de arriba, porque ya no define dependencias ni compila: su contenido pasa a los módulos.

7.5.3 El build.gradle de cada módulo

domain: no depende de nada del proyecto ni de Spring. Solo Lombok. Sus pruebas usan el starter de test, que ahora llega desde `subprojects` (en `testImplementation`, sin llegar al código de producción ni a los demás módulos):

domain/build.gradle

```
dependencies {
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
```

```
}
```

application: ve `domain` y lo expone con `api`:

application/build.gradle

```
dependencies {
    api project(':domain')

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

adapters: ve `application` (y, a través de su `api`, `domain`), y es el único módulo de biblioteca con Spring y con validación:

adapters/build.gradle

```
dependencies {
    implementation project(':application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-validation'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

bootstrap: el único con el plugin de Boot. Es la raíz de composición: nombra a `adapters` y a `application` para crear sus objetos en `BeanConfig`, y necesita Spring para la clase principal. No usa Lombok. Es también el único módulo con ArchUnit (versión explícita, porque Boot no la gestiona): ve en su *classpath* los cuatro módulos, y por eso vigila el conjunto.

bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencies {
    implementation project(':adapters')
    implementation project(':application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

Observa lo que *no* aparece. `bootstrap` no declara `domain`: le llega por el `api` de `application`. Y no declara el starter de validación: los DTO que lo usan están en `adapters`, que lo tiene como `implementation`, y en ejecución estará disponible igualmente. Para que la aplicación arranque basta con que cada clase tenga en tiempo de ejecución lo que necesita; para que compile, lo que ve en tiempo de compilación.

Por qué el escaneo sigue encontrando los controladores

`@SpringBootApplication` escanea el paquete `com.guildboard` y todos los que cuelgan de él *en el classpath de ejecución*, no en una carpeta. Los controladores están ahora en el *jar* de `adapters`, que `bootstrap` declara con `implementation` y por tanto está en ese *classpath*. El escaneo los encuentra igual que antes.

7.5.4 Qué ve cada módulo

Resumen de lo que cada módulo puede nombrar al compilar. «Sí» quiere decir que el `import` compila:

Ve desde...	`domain`	`application`	`adapters`	`bootstrap`
Clases de `domain`	sí	sí	sí (por el `api`)	sí (por el `api`)
Clases de `application`	no	sí	sí	sí
Clases de `adapters`	no	no	sí	sí
Clases de `bootstrap`	no	no	no	sí
Spring (web, validación)	no	no	sí	sí (web)

Léelo por columnas, que es como lo hace el compilador: `domain` solo se ve a sí mismo, `application` ve hasta el dominio, y así hacia fuera. Es la regla de dependencia del capítulo 3 escrita como una matriz triangular.

7.5.5 ArchitectureTest se muda a bootstrap

El `ArchitectureTest` del capítulo 3 vivía en el `src/test` del único módulo. Ahora el proyecto raíz no compila nada, y esa prueba tiene que ir a un módulo que *vea* todo lo que vigila. ArchUnit analiza las clases que encuentra en el *classpath* de la prueba dentro del paquete base [66]. El *classpath* de prueba de `bootstrap` incluye `adapters`, `application` y `domain`; el de `domain`, solo `domain`. Por eso la prueba va a `bootstrap/src/test/java/com/guildboard/` (el `git mv` de arriba) y `archunit` solo se declara en `bootstrap/build.gradle`.

El archivo no cambia salvo por lo que se añade en la sección 7.7: sigue con `BASE_PACKAGE = "com.guildboard"` como constante, que es el único dato del que depende. Los paquetes Java no se han movido, así que las dos reglas del capítulo 3 siguen valiendo tal cual.

Una consecuencia: las reglas de `domain` y `application` son ahora *redundantes* con el compilador, que ya impide que usen Spring o un adaptador. Se quedan como red de seguridad. No cubren el JDK (`java.sql` sigue pasando: la regla de `domain` no lo prohíbe y la de `application` permite todo `java..`; por eso el capítulo 3 propone ampliarlas), pero sí el día que alguien añada Spring a `domain/build.gradle` (entonces el compilador deja de protegerte y la prueba falla en cuanto se use una clase de Spring).

7.5.6 Qué errores aparecen ahora si alguien rompe la regla

Esta es la parte que justifica el capítulo. Prueba cada experimento y deshazlo con `git restore <archivo>`, que devuelve el archivo a su última versión confirmada (o borrando el archivo nuevo). Los mensajes están abreviados; el texto exacto de `javac` y de Gradle puede variar con la versión, pero la idea es la misma.

Experimento 1: Spring en la aplicación. Añade esta línea a `PublishQuestService`, bajo la línea `package`, y compila:

PublishQuestService.java (línea añadida)

```
import org.springframework.stereotype.Service;
```

```
$ ./gradlew :application:compileJava
> Task :application:compileJava FAILED
.../service/PublishQuestService.java:4: error: package
org.springframework.stereotype does not exist
import org.springframework.stereotype.Service;
```

Con el monolito del capítulo 6 esto compilaba (y solo lo cazaba la prueba de ArchUnit). Ahora ni siquiera llega a compilar. Escribirlo con el nombre completo, sin `import`, da el mismo error de compilación.

Experimento 2: el dominio mira hacia fuera. Añade a `Quest` esta línea y compila:

Quest.java (línea añadida)

```
import com.guildboard.application.port.in.PublishQuestUseCase;
```

```
$ ./gradlew :domain:compileJava
> Task :domain:compileJava FAILED
.../domain/Quest.java:5: error: package
com.guildboard.application.port.in does not exist
```

Experimento 3: la aplicación importa un adaptador. Añade a `AcceptQuestService` esta línea y compila:

AcceptQuestService.java (línea añadida)

```
import com.guildboard.adapter.out.memory.InMemoryQuestRepository;
```

```
$ ./gradlew :application:compileJava
> Task :application:compileJava FAILED
.../service/AcceptQuestService.java:3: error: package
com.guildboard.adapter.out.memory does not exist
```

Experimento 4: un ciclo. Añade a `application/build.gradle` la línea `implementation project(':adapters')` y ejecuta `./gradlew build`. Gradle no llega ni a compilar: se queja de una dependencia circular entre tareas, porque para construir `application` haría falta `adapters`, y para construir `adapters` hace falta `application`. Es ADP de la sección 7.3, impuesto por la herramienta.

Lo que sigue sin fallar. Haz que un controlador de `adapter/in/web` use de verdad (campo, parámetro o llamada) una clase de `adapter/out`; un `import` sin uso no deja rastro en el bytecode. Compila sin quejas, porque `in` y `out` viven en el mismo módulo. Y también compila si alguien le da a `domain` un starter de Spring en su `build.gradle`. Lo primero lo cubre la sección 7.7;

lo segundo, la regla de `domain` de `ArchitectureTest`, que cae en cuanto se use una clase de Spring, y la revisión del `build.gradle` en cada verificación.

Revisado, no ejecutado

Como en todo el libro, el código Java y los `build.gradle` de este capítulo están revisados a mano contra la documentación citada, pero no compilados. Lo que sí se contrastó el 8 de octubre de 2026: la diferencia `api`/`implementation` y el plugin `java-library` (documentación de Gradle [17]), el uso de `BOM_COORDINATES` y de `apply false` (guía del plugin de Spring Boot 4.1.1 [51]), el patrón de proyecto raíz que aplica `java-library` y la gestión de dependencias a todos los subproyectos y deja el plugin de Boot en un solo módulo (el repositorio archivado [68], que usa ese mismo patrón). No pude comprobar por ejecución los mensajes exactos de error que se muestran, ni que ArchUnit vea desde `bootstrap` las clases de los otros tres módulos: es lo esperable por cómo Gradle arma el *classpath* de prueba con `implementation`, pero no lo ejecuté.

7.6 Un módulo por adaptador frente a un módulo de adaptadores

Queda una pregunta de diseño: ¿por qué `adapters` es un solo módulo? Podría haber uno por adaptador: `adapters-web`, `adapters-memory`, `adapters-notification`... El proyecto de ejemplo de Trzpiot lo hace: tiene los módulos `core`, `rest`, `app`, `database-h2` y `database-neo4j`, y cambiar de base de datos es cambiar una línea de `app` (el repositorio está archivado desde febrero de 2025 [68]; lo uso como contraste, no como modelo, y fíjate en que su `core` declara `spring-boot-starter`: no es un núcleo libre de frameworks como el nuestro). Buckpal, en cambio, es un solo módulo con todo separado por paquetes [20].

	A: un módulo `adapters`	**B: un módulo por adaptador**
Módulos y `build.gradle`	4 en total.	6 o más, y uno por cada adaptador nuevo.
Aislamiento entre adaptadores	Lo vigilan dos reglas de ArchUnit.	Lo impone el compilador (`web` no puede ver `memory`).
Dependencias pesadas (JPA, cliente de mensajes)	Todas en el *classpath* de todos los adaptadores.	Cada una solo en el suyo (CRP).
Qué cambia junto (CCP)	Web y memoria cambian por razones distintas, juntos.	Cada módulo cambia por una sola razón.
Coste en el curso	Bajo.	Se multiplica por tres servicios y cuatro módulos en el capítulo 9.

La columna B gana en aislamiento real, y es un argumento serio: es lo que da la separación más fuerte con el compilador. La columna A gana en simplicidad, y en un proyecto de este tamaño el coste de B se paga todos los días y su beneficio solo se cobra el día que algo falla. El curso elige **A**, por tres razones medibles hoy: solo hay tres adaptadores; ninguno trae todavía una dependencia pesada; y el aislamiento entre `in` y `out` se puede vigilar con dos reglas de ArchUnit de pocas líneas. Elegir A no es definitivo. Convendría pasar a B si (a) un adaptador necesita una dependencia que no quieres en el *classpath* de los demás, (b) dos equipos distintos tocan adaptadores distintos con ritmos distintos, o (c) quieres reutilizar un adaptador en otra aplicación. El ejercicio 7.4 te pide hacer este mismo razonamiento por tu cuenta.

7.7 Lo que el compilador no ve: el aislamiento entre adaptadores

Con la opción A, los adaptadores de entrada y de salida conviven en el módulo `adapters`, y el compilador deja que se usen entre sí. Eso rompería la idea del hexágono: un controlador que llama a un repositorio concreto se salta el caso de uso, el puerto y, con ellos, todo el sentido de los capítulos anteriores. Se puede prohibir con dos reglas más en `ArchitectureTest`.

La pregunta es otra que en el capítulo 3. En vez de «¿qué capa depende de cuál?», es «¿qué lado de `adapters` depende del otro?». Para las clases de `adapter.in` se prohíbe depender de cualquiera de `adapter.out`, y al revés. Con el estilo de reglas explícitas del capítulo anterior, la primera se lee casi sola:

Regla: adapter.in no depende de adapter.out

```
noClasses().that().resideInAPackage("..adapter.in..")
    .should().dependOnClassesThat().resideInAPackage("..adapter.out..")
    .check(CLASSES);
```

Escribirla como prueba, y su inversa, es el ejercicio 7.3; el archivo completo, con las cuatro pruebas, está en la solución del ejercicio 7.5.

Hay una diferencia importante con lo que tenías antes de ArchUnit: la regla no lee texto, lee el *bytecode* de las clases ya compiladas. Una dependencia cuenta aunque se escriba con el nombre completo y sin `import`, y también cuando es un campo, un parámetro, una llamada o un tipo de retorno. Se ve en las dos direcciones: un `import` que no se usa no deja rastro en el bytecode y la regla no lo marca, porque no hay dependencia real. Lo que sigue sin ver es lo que no se escribe en Java: si alguien mete `adapters-memory` en el `build.gradle` de otro módulo, eso es un asunto de la revisión del `build.gradle`. Tampoco se ve una dependencia indirecta, a través de una clase intermedia que no esté en `adapter`, pero esa deja de ser un `import` directo entre lados y es más difícil de cometer sin querer.

7.8 En el proyecto: hito H3, parte 2

Hito 3 (parte 2): la estructura la impone el compilador

Al terminar el ejercicio 7.5, GuildBoard son cuatro módulos Gradle (`domain`, `application`, `adapters`, `bootstrap`) con el mismo comportamiento y las mismas pruebas que al final del capítulo 6: ningún archivo Java cambia. Cambian los archivos de Gradle: `settings.gradle`, el `build.gradle` raíz y los cuatro de los módulos (nuevos), y `ArchitectureTest` se muda a `bootstrap` y gana las dos reglas de aislamiento de adaptadores. Las comprobaciones de la verificación 3 se cumplen en los `build.gradle`, y el compilador impide romper la regla de dependencia entre capas.

7.8.1 Probar el hito

Desde la raíz, `./gradlew build` compila los cuatro módulos, ejecuta las pruebas (las del dominio y `ArchitectureTest`) y empaqueta. En la salida deben aparecer las tareas de cada módulo con su nombre completo:

```
$ ./gradlew build
> Task :domain:compileJava
> Task :domain:test
> Task :application:compileJava
> Task :adapters:compileJava
> Task :bootstrap:compileJava
> Task :bootstrap:test
> Task :bootstrap:bootJar
BUILD SUCCESSFUL
```

(Salida resumida.) Las cinco clases de prueba del dominio se ejecutan ahora en `domain`; puedes lanzarlas solas con `./gradlew :domain:test`, y la prueba de arquitectura con `./gradlew :bootstrap:test`. Para arrancar la aplicación, apunta al módulo que tiene el plugin de Boot:

```
$ ./gradlew :bootstrap:bootRun
```

La aplicación arranca en `http://localhost:8080` y el escenario de la sección 6.8.1 del capítulo 6 da exactamente las mismas respuestas: lo que has cambiado es dónde viven los archivos, no lo que hacen.

7.9 Ejercicios

Los ejercicios parten del proyecto tal como lo dejó el capítulo 6, salvo indicación en contra; el de proyecto lo corta en módulos y los demás, que trabajan sobre el resultado, suponen que ya lo has hecho. Cuando un ejercicio añada archivos de prueba, bórralos al terminar y no los incluyas en el *commit*. Empieza con el árbol de trabajo limpio y termina con un *commit* cuando el ejercicio lo pida.

Ejercicio 7.1: El grafo de los módulos ★★☆☆☆

Alguien ha escrito los `build.gradle` de los cuatro módulos de GuildBoard con una diferencia respecto a los tuyos. Supón que los plugins ya están aplicados, que las líneas de Lombok (`compileOnly` y `annotationProcessor`) y el starter de validación son idénticas a las tuyas y se omiten, y que todas las clases son las del proyecto (`BeanConfig` importa solo clases de `application` y de `adapters`):

Los bloques dependencies de otro autor

```
// domain
dependencies { /* solo Lombok */ }

// application
dependencies { implementation project(':domain') }

// adapters
dependencies {
    implementation project(':application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
}

// bootstrap
dependencies {
    implementation project(':adapters')
    implementation project(':application')
}
```

1. Dibuja el grafo de dependencias entre módulos, con una flecha por línea `project(...)` y la configuración escrita sobre ella.
2. Completa una tabla con una fila por módulo y estas columnas: ¿ve al compilar `Quest` (de `domain`)?, ¿ve `QuestRepository` (de `application`)?, ¿ve las clases de Spring?
3. Dos de los cuatro módulos no compilan. Nombra cuáles, la razón de cada uno y la línea mínima que lo arregla.
4. Con tus arreglos, ¿existe la clase `Quest` en el *classpath* de ejecución de `bootstrap`? ¿Y aunque ese módulo no la vea al compilar?

Está bien si: el grafo tiene cuatro flechas entre módulos, todas `implementation`; la tabla nombra a `adapters` como el módulo que no ve `Quest`; los dos módulos rotos son `adapters` y `bootstrap`; y distingues ver al compilar de tener en ejecución.

Pista: *pregunta por cada* `import` *de un módulo: ¿qué dependencia lo trae al* classpath *de compilación? Una dependencia* `implementation` *de un módulo no pasa a quien dependa de él.*

Lectura de código ▪ solución en la pág. 235

Ejercicio 7.2: Un import de Spring en el dominio ★★★☆☆

Un compañero ha escrito esta clase en `domain`, en `domain/src/main/java/com/guildboard/domain/QuestAnnouncer.java`. `ApplicationEventPublisher` es una interfaz de Spring que envía un evento a quien lo escuche dentro de la aplicación; no necesitas saber más:

domain/QuestAnnouncer.java

```java
package com.guildboard.domain;

import org.springframework.context.ApplicationEventPublisher;

public class QuestAnnouncer {

    private final ApplicationEventPublisher events;

    public QuestAnnouncer(ApplicationEventPublisher events) {
        this.events = events;
    }

    public void announce(Quest quest) {
        events.publishEvent("quest published: " + quest.getTitle());
    }
}
```

1. Crea el archivo y ejecuta `./gradlew :domain:compileJava`. Copia la primera línea de error.
2. Explica en dos frases por qué falla, y por qué *no* es buena idea arreglarlo añadiendo Spring al `build.gradle` de `domain`.
3. Decide dónde debe vivir cada parte. La necesidad («avisar de que una misión se ha publicado») y el mecanismo (los eventos de Spring) no pertenecen al mismo módulo. Borra la clase de `domain`; crea el puerto de salida `QuestAnnouncer` en `application` y su implementación `SpringQuestAnnouncer` en `adapter/out/notification` de `adapters`.
4. Ejecuta `./gradlew build`. Después borra los dos archivos (no los incluyas en ningún *commit*).

Está bien si: el error del paso 1 dice que el paquete `org.springframework.context` no existe; tu explicación menciona que el compilador ya no protege al dominio si le das Spring; tras el paso 3 `./gradlew build` termina con `BUILD SUCCESSFUL`; ningún `build.gradle` ha cambiado; y la única clase con un `import` de Spring es la del adaptador.

Pista: *el puerto es una interfaz con un método* ***`announce(Quest)`****, y el adaptador la implementa con el código de la clase original.* ***`adapters`*** *ve* ***`Quest`*** *porque* ***`application`*** *lo expone con* ***`api`****.*

Depuración ▪ solución en la pág. 236

Ejercicio 7.3: Aislar los adaptadores con ArchUnit ★★★☆☆

Añade a `ArchitectureTest`, ya en `bootstrap`, las dos pruebas que impiden que `adapter.in` y `adapter.out` dependan entre sí. Parte del archivo de la sección 7.5.5, con sus dos pruebas y la constante `CLASSES`.

1. Comprueba que el archivo está en `bootstrap/src/test/java/com/guildboard/` y que `bootstrap/build.gradle` declara `archunit`.
2. Escribe `adapterInDoesNotDependOnAdapterOut`: con `noClasses()`, las clases de `..adapter.in..` no deben depender de las de `..adapter.out..`.
3. Escribe `adapterOutDoesNotDependOnAdapterIn`, la regla inversa.
4. Ejecuta `./gradlew test`: debe pasar.

5. Rómpela a propósito: en `QuestController` añade el `import` de `com.guildboard.adapter.out.memory.InMemoryQuestRepository` y un campo `private InMemoryQuestRepository leak;`. Comprueba que `./gradlew test` falla y que el mensaje nombra las dos clases. Quita el campo y el `import`.

Está bien si: con las dos pruebas y sin la ruptura, `./gradlew test` pasa; con la ruptura, falla `adapterInDoesNotDependOnAdapterOut` y el informe nombra `QuestController` e `InMemoryQuestRepository`; y al quitar el campo vuelve a pasar. Prueba además a dejar solo el `import`, sin el campo: la prueba pasa, y piensa por qué.

Pista: *copia la forma de las pruebas del capítulo 3:* `noClasses().that().resideInAPackage(...).should().dependOnClassesThat().resideInAPackage(...).check(CLASSES)`. *Cambia los dos paquetes y el nombre del método. Los paquetes acaban en* `..`*, con puntos a los dos lados.*

Práctica ▪ solución en la pág. 237

Ejercicio 7.4: ¿Un módulo por adaptador? ★★★★☆

Este ejercicio parte del árbol de la verificación de estructura 3. Alguien propone, en lugar de ese árbol (opción A), este árbol B, con un módulo por adaptador:

Árbol B

```
guildboard/
|-- domain/                 igual que en A
|-- application/            igual que en A
|-- adapters-web/           controladores, dto, mapper, error
|-- adapters-memory/        repositorios en memoria
|-- adapters-notification/  LogNotificationSender
`-- bootstrap/              depende de los tres m^^c3^^b3dulos de
    ↪ adaptadores
```

1. Elige **tres** principios de componentes de la sección 7.3 y, para cada uno, di si favorece A o B en GuildBoard, con un ejemplo concreto del proyecto.
2. Aplica las cinco comprobaciones de la verificación de estructura 3 (sección 7.2) al árbol B. ¿Cambia la respuesta de alguna respecto a A? ¿Cuál?
3. ¿Qué regla de las que has escrito sobra en B, y qué nuevo riesgo aparece en los `build.gradle`?
4. Decide entre A y B para GuildBoard *hoy*, y escribe dos condiciones concretas bajo las que cambiarías de opinión.

Está bien si: los tres principios están bien aplicados (no basta con nombrarlos); la respuesta al apartado 3 distingue lo que el compilador impone en B de lo que sigue siendo revisión humana; la decisión es una sola y argumentada; y las condiciones de cambio son comprobables, no «si crece mucho».

Diseño ▪ solución en la pág. 239

Ejercicio 7.5: Hito H3 (parte 2): cuatro módulos ★★★☆☆

Parte del proyecto tal como lo dejó el capítulo 6 (H3, parte 1).

1. Haz el corte de la sección 7.5: crea las carpetas y mueve cada paquete con `git mv`, sin editar ningún archivo Java.

2. Escribe el `settings.gradle` y reemplaza el `build.gradle` de la raíz, con la configuración común de JUnit en `subprojects`.
3. Crea los cuatro `build.gradle` de los módulos; el de `bootstrap` lleva `archunit`.
4. Mueve `ArchitectureTest` a `bootstrap` y añade las dos reglas de aislamiento de adaptadores (ejercicio 7.3).
5. Ejecuta `./gradlew build` y `./gradlew test` y arranca con `./gradlew :bootstrap:bootRun`; repite el escenario del capítulo 6.
6. Haz los experimentos 1, 2 y 3 de la sección 7.5.6 y comprueba que fallan; deshazlos.
7. Comprueba el árbol con la lista de comprobaciones y haz un *commit* con el mensaje `H3b: cuatro modulos Gradle`.

Está bien si: `./gradlew build` y `./gradlew test` terminan con `BUILD SUCCESSFUL` y ejecutan las pruebas de `domain` y `ArchitectureTest` (en `bootstrap`); `git status` muestra los archivos Java como renombrados (`rename`) y ninguno modificado; `bootRun` arranca y el escenario da las mismas respuestas; los tres experimentos dan error de compilación; no queda ningún `src` en la raíz; y tu árbol coincide con el de la verificación 3.

Pista: *si Gradle no encuentra un módulo, mira que el nombre del* `include` *y el de la carpeta coinciden; si un módulo no compila, mira la tabla «Qué ve cada módulo» y pregunta qué* `import` *falla.*

Proyecto ▪ solución en la pág. 241

7.10 Resumen

Resumen

- Una convención (la lista, una prueba de ArchUnit que alguien puede saltarse) se puede ignorar; el compilador, no. Pasar de un módulo a cuatro convierte «no deberías importar Spring en la aplicación» en «no compila». El compilador no sustituye a la prueba ni a la revisión: es tan fuerte como cada `build.gradle`.
- Un módulo Gradle es un componente en el sentido de Martin. Sus principios guían el corte: cohesión (REP, CCP, CRP) y acoplamiento (ADP sin ciclos, SDP hacia lo estable, SAP). El grafo de GuildBoard apunta siempre hacia `domain`, lo más estable.
- Con `java-library`, `api` expone una dependencia a quien dependa del módulo e `implementation` la oculta al compilar (no en ejecución). Regla práctica: `implementation` por defecto, `api` cuando el tipo aparece en una firma pública que usan otros. `application` expone `domain` con `api`.
- El proyecto raíz declara el plugin de Spring Boot con `apply false`, aplica `java-library` y la gestión de dependencias a los subproyectos e importa el BOM de Boot; solo `bootstrap` aplica el plugin y es la raíz de composición. Lombok se declara en cada módulo que lo usa (`compileOnly` y `annotationProcessor` no se heredan); la configuración de JUnit va en `subprojects`.
- El corte no cambia ni una línea de Java: se mueven los paquetes a su módulo con `git mv`. Las pruebas del dominio pasan a `domain/src/test`; `application.yml` y `BeanConfig`, a `bootstrap`.
- `ArchitectureTest` se muda a `bootstrap`, el único módulo con `archunit` y con todos los

demás en su *classpath*. Sus reglas de `domain` y `application` quedan redundantes con el compilador (red de seguridad); las dos nuevas vigilan en `adapters` lo que el compilador no ve (`in` frente a `out`), leyendo bytecode, no texto.

- Un módulo por adaptador da más aislamiento con el compilador y cuesta más módulos; el curso elige uno solo (`adapters`) mientras no haya una dependencia pesada, equipos separados o reutilización. La verificación de estructura 3 recoge el árbol de cuatro módulos; el capítulo 8 lo usa para cambiar de adaptador.

Soluciones de los ejercicios

Solución del ejercicio 7.1 enunciado en la pág. 230

El grafo. Cuatro flechas entre módulos, todas `implementation` (`domain` no apunta a nadie):

```
application -> domain         (implementation)
adapters    -> application    (implementation)
bootstrap   -> adapters       (implementation)
bootstrap   -> application    (implementation)
```

Además, `adapters` depende del starter web de Spring (una dependencia externa, no un módulo) y `bootstrap` no depende de ninguna externa.

La tabla de visibilidad al compilar.

Módulo	`Quest`	`QuestRepository`	Spring
`domain`	sí (propio)	no	no
`application`	sí	sí (propio)	no
`adapters`	**no** (hay `implementation`)	sí	sí
`bootstrap`	no	sí	**no**

Los dos módulos rotos.

- `adapters`: los mapeadores importan `Quest` y `Adventurer`, y `application` declara `domain` como `implementation`, así que no sale de `application`. Arreglo mínimo, en `application/build.gradle`: cambiar `implementation` por `api` en la línea de `project(':domain')`. (Otro arreglo válido: añadir `implementation project(':domain')` a `adapters`; funciona, pero repite una dependencia que ya existe por la firma pública de `application`.)
- `bootstrap`: `GuildBoardApplication` y `BeanConfig` usan clases de Spring (`SpringApplication`, `@Bean`) y el starter está en `adapters` como `implementation`, que no pasa hacia fuera. Arreglo: añadir `implementation 'org.springframework.boot:spring-boot-starter-webmvc'` a `bootstrap/build.gradle`.

Compilar frente a ejecutar. Sí: en ejecución, el *classpath* de `bootstrap` incluye también las dependencias `implementation` de los módulos de los que depende (de forma transitiva), de modo que `Quest` está ahí. No la ve el compilador de `bootstrap` y por eso nadie en ese módulo puede escribir `import com.guildboard.domain.Quest` sin declararla (o sin que `application` la exponga con `api`). Esa diferencia es el motivo de ser de `api`.

Errores típicos: contar mal las flechas (recuerda que `bootstrap` tiene dos hacia dentro); creer que un módulo no tiene la clase si no la ve (la ve en ejecución); y arreglar el primer fallo creyendo que arregla también el segundo: son independientes.

Solución del ejercicio 7.2

enunciado en la pág. 231

Paso 1. El error es del estilo:

```
> Task :domain:compileJava FAILED
.../domain/QuestAnnouncer.java:3: error: package
org.springframework.context does not exist
import org.springframework.context.ApplicationEventPublisher;
```

(Y, a continuación, `cannot find symbol` por el tipo `ApplicationEventPublisher`.)
Paso 2. Falla porque `domain/build.gradle` no declara Spring, así que no está en el *classpath* de compilación del módulo; es justo lo que queremos. Añadir Spring a ese `build.gradle` haría compilar la clase, pero silenciaría la única protección que impone el compilador: desde ese momento, cualquiera podría importar Spring en el dominio sin ningún error. Lo que quedaría entre el dominio y Spring sería la prueba de ArchUnit (si se usa una clase de Spring) y la revisión.
Paso 3. El dominio sabe qué es una misión, no cómo se avisa de ella. La necesidad es un puerto de salida de `application`:

application/port/out/QuestAnnouncer.java

```java
package com.guildboard.application.port.out;

import com.guildboard.domain.Quest;

public interface QuestAnnouncer {

    void announce(Quest quest);
}
```

El mecanismo es un adaptador de salida, en `adapters`, donde Spring sí está:

adapter/out/notification/SpringQuestAnnouncer.java

```java
package com.guildboard.adapter.out.notification;

import com.guildboard.application.port.out.QuestAnnouncer;
import com.guildboard.domain.Quest;
import org.springframework.context.ApplicationEventPublisher;

public class SpringQuestAnnouncer implements QuestAnnouncer {

    private final ApplicationEventPublisher events;

    public SpringQuestAnnouncer(ApplicationEventPublisher events) {
        this.events = events;
    }

    @Override
    public void announce(Quest quest) {
        events.publishEvent("quest published: " + quest.getTitle());
    }
}
```

Paso 4 y por qué funciona. `adapters` ve `Quest` porque `application` lo expone con `api`, y ve `QuestAnnouncer` porque depende de `application`. `application` solo importa `Quest` (dominio). `domain` no vuelve a contener ninguna palabra de Spring. `./gradlew build` pasa con ambos archivos. No hace falta registrarlos como *bean*, porque ningún servicio los usa todavía; si los borras al terminar, el proyecto queda como estaba.
Errores típicos: dejar la interfaz en `domain` (compila, pero un puerto de salida es una necesidad del caso de uso y vive en `application/port/out`, como los demás); anotar el adaptador con `@Component` (el proyecto crea los objetos en `BeanConfig`); y confundir este error con una incompatibilidad de versiones y tocar la versión de Spring cuando el problema es un `build.gradle` que correctamente no tiene Spring.

Solución del ejercicio 7.3

enunciado en la pág. 232

Se completa `bootstrap/src/test/java/com/guildboard/ArchitectureTest.java` con las dos pruebas nuevas al final (el archivo entero):

bootstrap/src/test/.../ArchitectureTest.java

```
package com.guildboard;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;

import com.tngtech.archunit.core.domain.JavaClasses;
import com.tngtech.archunit.core.importer.ClassFileImporter;
import com.tngtech.archunit.core.importer.ImportOption;
import org.junit.jupiter.api.Test;

public class ArchitectureTest {

    private static final String BASE_PACKAGE = "com.guildboard";

    private static final JavaClasses CLASSES = new ClassFileImporter()
        .withImportOption(new ImportOption.DoNotIncludeTests())
        .importPackages(BASE_PACKAGE);

    @Test
    void domainDoesNotDependOnOuterLayers() {
        noClasses().that().resideInAPackage("..domain..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "..application..", "..adapter..", "..config..",
                "org.springframework..", "jakarta..")
            .check(CLASSES);
    }

    @Test
    void applicationOnlyDependsOnDomain() {
        classes().that().resideInAPackage("..application..")
            .should().onlyDependOnClassesThat().resideInAnyPackage(
                "..application..", "..domain..", "java..", "lombok..")
            .check(CLASSES);
    }

    @Test
    void adapterInDoesNotDependOnAdapterOut() {
        noClasses().that().resideInAPackage("..adapter.in..")
            .should().dependOnClassesThat().resideInAPackage("..adapter.out..")
            .check(CLASSES);
    }

    @Test
    void adapterOutDoesNotDependOnAdapterIn() {
        noClasses().that().resideInAPackage("..adapter.out..")
            .should().dependOnClassesThat().resideInAPackage("..adapter.in..")
            .check(CLASSES);
    }
}
```

Y `bootstrap/build.gradle`, con `archunit` (el resto de la configuración de pruebas llega desde el `subprojects` de la raíz):

`bootstrap/build.gradle`

```
plugins {
    id 'org.springframework.boot'
}

dependencies {
    implementation project(':adapters')
    implementation project(':application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

Cómo se prueba. Con el campo `private InMemoryQuestRepository leak;` (y su `import`) en `QuestController`, el código compila, porque `in` y `out` están en el mismo módulo, pero la prueba falla. El mensaje de ArchUnit, abreviado:

```
$ ./gradlew test
> Task :bootstrap:test FAILED
ArchitectureTest > adapterInDoesNotDependOnAdapterOut() FAILED
    java.lang.AssertionError: Architecture Violation [Priority: MEDIUM]
    - Rule 'no classes that reside in a package '..adapter.in..' should
    depend on classes that reside in a package '..adapter.out..'' was
    violated (1 times):
    Field <com.guildboard.adapter.in.web.QuestController.leak> has type
    <com.guildboard.adapter.out.memory.InMemoryQuestRepository> in
    (QuestController.java:...)
```

(El texto exacto y el número de línea pueden variar entre versiones.) Al quitar el campo y el `import`, pasa.

Por qué funciona. `CLASSES` contiene las clases compiladas de los cuatro módulos, porque están en el *classpath* de prueba de `bootstrap`. `resideInAPackage("..adapter.in..")` selecciona las del lado de entrada, y `dependOnClassesThat().resideInAPackage("..adapter.out..")` recoge cualquier uso del lado de salida; `noClasses()` exige que no haya ninguno. La segunda prueba es la misma con los lados cambiados. Si se deja solo el `import`, sin usar la clase, el compilador no lo guarda en el bytecode y no hay dependencia que marcar.

Errores típicos: escribir `"..adapter.in"` sin el `..` final (no selecciona los subpaquetes como `in.web`, y ArchUnit falla por selección vacía); poner el paquete del puerto (`..application.port.out..`) en vez del del adaptador (`..adapter.out..`), que también contiene `out`; declarar `archunit` en `adapters` o `domain` (en `domain` no están las clases de `adapter` y la selección queda vacía); dejar la prueba en el `src/test` de la raíz, donde nadie la compila; y probar la ruptura con un `import` sin uso, que no cambia el bytecode.

Solución del ejercicio 7.4 enunciado en la pág. 233

Solución de referencia: elegir A, el árbol del capítulo, hoy.
1. Principios. (Tres de varios posibles.)

- *CCP* favorece B, en parte: `LogNotificationSender` y los controladores cambian por razones distintas (un canal de avisos frente a la API), y B los separa. Pero hoy cambian poco y `adapters` los agrupa sin coste visible.
- *CRP* favorece B en cuanto aparezca una dependencia pesada: cuando haya JPA (capítulo 8), en A cada adaptador, también el web, tendrá JPA en el *classpath*; en B solo el módulo de persistencia. Hoy no hay ninguna dependencia así, así que el argumento todavía no se cobra.
- *ADP* no distingue: ambos grafos son acíclicos y Gradle lo impone en los dos.
- *SDP*: tampoco distingue; los adaptadores son lo más inestable y nadie los referencia salvo `bootstrap`, en A y en B.

2. Comprobaciones sobre B (las de la verificación 7.2). Cambia la 3 (¿cada adaptador está en `in` o en `out`?): en B se responde mirando el nombre del módulo y el compilador impide que `adapters-web` importe a `adapters-memory`. Las comprobaciones 1 y 2 no cambian. La 4 (las carpetas coinciden con el árbol) deja de ser literal: hay que comparar con el árbol B, que tiene tres módulos más y tres `build.gradle` más, y los nombres de módulo como `adapters-web` describen una tecnología. La 5 (`build` y `test` pasan) sigue valiendo, pero ahora recorre esos módulos adicionales.
3. Qué sobra y qué aparece. Sobrarían las dos reglas de aislamiento de adaptadores de `ArchitectureTest`, porque el compilador impone lo mismo. A cambio aparece un riesgo nuevo en los `build.gradle`: alguien puede añadir `implementation project(':adapters-memory')` a `adapters-web`, y entonces el compilador deja de proteger. Eso solo lo cubre la revisión del `build.gradle` (o una tarea que lea los `build.gradle`, que no tenemos): se pasa de vigilar `import` a vigilar archivos de build.
4. Decisión. A. Razones: solo hay tres adaptadores y ninguno trae una dependencia pesada; el coste de B se multiplica en el capítulo 9 (tres servicios por cuatro módulos son doce; con B, de seis módulos por servicio, serían dieciocho) y el beneficio de B (aislamiento por compilador) lo dan hoy dos reglas de pocas líneas. *Condiciones de cambio comprobables:* (a) cuando un adaptador exija una dependencia que no deba llegar al *classpath* de los demás (por ejemplo, JPA con su gestión de entidades, si el equipo ve que el controlador empieza a usarla); (b) cuando dos equipos mantengan adaptadores distintos con ritmos de entrega distintos. Otra válida: si las reglas de aislamiento dejan pasar por segunda vez un acoplamiento real.
Criterios para valorar otras soluciones. Es válida si (a) aplica los principios con ejemplos del proyecto y no solo los nombra; (b) reconoce que B gana en aislamiento real y que A gana en simplicidad; (c) identifica que en B cambia el tipo de vigilancia (de `import` a `build.gradle`) y no desaparece; (d) toma una decisión y da condiciones de cambio concretas; y (e) no usa argumentos falsos: por ejemplo, que B «es más hexagonal» (los dos respetan la regla de dependencia) o que A no permite cambiar de adaptador.
Alternativa válida. Elegir B alegando que el coste de módulos se amortiza con el aislamiento, siempre que (b), (c) y (d) se cumplan y se reconozca el coste del capítulo 9. También: A hoy, con el compromiso escrito de pasar a B al introducir la persistencia en el capítulo 8 si se añade una dependencia pesada; la verificación del capítulo 8 es el sitio natural para revisarlo.
Errores típicos: defender B por «un módulo por clase» sin ver el coste; olvidar que el compilador solo es tan fuerte como el `build.gradle`; y decidir sin condiciones de cambio, que convierte la decisión en una creencia.

Solución del ejercicio 7.5 enunciado en la pág. 233

Solución de referencia. El corte no modifica ningún archivo Java ni `docs/dominio.md` ni `.gitignore`. Se mueven, con `git mv`, los paquetes a su módulo, como en la sección 7.5. **Mover.**

```
$ mkdir -p domain/src/main/java/com/guildboard \
           domain/src/test/java/com/guildboard \
           application/src/main/java/com/guildboard \
           adapters/src/main/java/com/guildboard \
           bootstrap/src/main/java/com/guildboard \
           bootstrap/src/main/resources \
           bootstrap/src/test/java/com/guildboard
$ S=src/main/java/com/guildboard
$ git mv $S/domain domain/$S/domain
$ git mv src/test/java/com/guildboard/domain \
         domain/src/test/java/com/guildboard/domain
$ git mv src/test/java/com/guildboard/ArchitectureTest.java \
         bootstrap/src/test/java/com/guildboard/ArchitectureTest.java
$ git mv $S/application application/$S/application
$ git mv $S/adapter adapters/$S/adapter
$ git mv $S/config bootstrap/$S/config
$ git mv $S/GuildBoardApplication.java \
         bootstrap/$S/GuildBoardApplication.java
$ git mv src/main/resources/application.yml \
         bootstrap/src/main/resources/application.yml
$ find src -type d -empty -delete
```

Gradle: raíz.

settings.gradle

```
rootProject.name = 'guildboard'

include 'domain', 'application', 'adapters', 'bootstrap'
```

build.gradle (raíz)

```
plugins {
    id 'org.springframework.boot' version '4.1.1' apply false
}

subprojects {
    apply plugin: 'java-library'
    apply plugin: 'io.spring.dependency-management'

    group = 'com.guildboard'
    version = '0.0.1-SNAPSHOT'

    java {
        toolchain {
            languageVersion = JavaLanguageVersion.of(21)
        }
    }

    repositories {
        mavenCentral()
    }

    dependencyManagement {
        imports {
            mavenBom(org.springframework.boot.gradle.plugin
                    .SpringBootPlugin.BOM_COORDINATES)
        }
    }

    dependencies {
        testImplementation 'org.springframework.boot:spring-boot-starter-
    ↪ test'
        testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
    }

    tasks.named('test') {
        useJUnitPlatform()
    }
}
```

Gradle: módulos.

domain/build.gradle

```
dependencies {
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

application/build.gradle

```
dependencies {
    api project(':domain')

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

adapters/build.gradle

```
dependencies {
    implementation project(':application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencies {
    implementation project(':adapters')
    implementation project(':application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

Prueba de arquitectura. Se mueve con `git mv` (arriba) y queda con las cuatro pruebas:

bootstrap/src/test/.../ArchitectureTest.java

```
package com.guildboard;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;

import com.tngtech.archunit.core.domain.JavaClasses;
import com.tngtech.archunit.core.importer.ClassFileImporter;
import com.tngtech.archunit.core.importer.ImportOption;
import org.junit.jupiter.api.Test;

public class ArchitectureTest {

    private static final String BASE_PACKAGE = "com.guildboard";

    private static final JavaClasses CLASSES = new ClassFileImporter()
        .withImportOption(new ImportOption.DoNotIncludeTests())
        .importPackages(BASE_PACKAGE);

    @Test
    void domainDoesNotDependOnOuterLayers() {
        noClasses().that().resideInAPackage("..domain..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "..application..", "..adapter..", "..config..",
                "org.springframework..", "jakarta..")
            .check(CLASSES);
    }

    @Test
    void applicationOnlyDependsOnDomain() {
        classes().that().resideInAPackage("..application..")
            .should().onlyDependOnClassesThat().resideInAnyPackage(
                "..application..", "..domain..", "java..", "lombok..")
            .check(CLASSES);
    }

    @Test
    void adapterInDoesNotDependOnAdapterOut() {
        noClasses().that().resideInAPackage("..adapter.in..")
            .should().dependOnClassesThat().resideInAPackage("..adapter.out..")
            .check(CLASSES);
    }

    @Test
    void adapterOutDoesNotDependOnAdapterIn() {
        noClasses().that().resideInAPackage("..adapter.out..")
            .should().dependOnClassesThat().resideInAPackage("..adapter.in..")
            .check(CLASSES);
    }
}
```

Comprobar. `git status -short` muestra los archivos Java con `R` (renombrados; también `ArchitectureTest`, que además se edita), `settings.gradle` y `build.gradle` con `M`, y los cuatro `build.gradle` nuevos como no rastreados. Después:

```
$ ./gradlew build
BUILD SUCCESSFUL
$ ./gradlew test
BUILD SUCCESSFUL
$ ./gradlew :bootstrap:bootRun
```

El escenario del capítulo 6 da las mismas respuestas.

Por qué funciona. El paquete Java de cada clase no ha cambiado, así que los `import` siguen siendo los mismos; lo que ha cambiado es qué módulo contiene el archivo y qué ve ese módulo (matriz de la sección 7.5). `adapters` compila porque ve `domain` por el `api` de `application`; `bootstrap` compila porque ve `adapters`, `application` (y por él, `domain`) y Spring; `GuildBoardApplication` escanea `com.guildboard` en el *classpath* de ejecución, donde están los *jar* de los demás módulos.

Qué cambia para el capítulo siguiente. Nada de comportamiento. El capítulo 8 añade el adaptador de PostgreSQL dentro de `adapters`, sus migraciones y la configuración por perfiles en `bootstrap`, sin tocar `domain` ni `application`.

Criterios para valorar otras soluciones. Es válida si los `build.gradle` de `domain` y `application` no declaran Spring, si `application` expone `domain` con `api`, si solo `bootstrap` aplica el plugin de Boot, y si `build` pasa con las pruebas vigentes (también `ArchitectureTest`). Variante válida: aplicar la configuración común con un plugin de convenciones en `buildSrc` en lugar de `subprojects` (más ceremonia, recomendado por Gradle para builds grandes).

Errores típicos: dejar el `src` de la raíz (si queda algo dentro, Gradle lo ignora, y confunde a quien abra el proyecto); dejar `ArchitectureTest` en el `src/test` de la raíz (ya no se compila) o declarar `archunit` en un módulo que no ve todo (ArchUnit falla por selección vacía); declarar `api` en vez de `implementation` para Spring en `adapters` (filtra Spring a `bootstrap` sin necesidad); poner una versión al plugin de Boot en `bootstrap` (Gradle se queja de que ya está en el *classpath*); mover los archivos con `mv` en vez de `git mv` (Git acaba detectando el cambio, pero el historial es menos limpio); y olvidar Lombok en `adapters` (`@Slf4j` y los otros dejan de funcionar).

CAPÍTULO 8

Cambiar de adaptador: de memoria a PostgreSQL

Objetivos del capítulo

- Añadir un adaptador de salida con JPA (entidad separada del dominio y mapeo a mano) detrás de un puerto que ya existe, sin tocar `domain` ni `application`.
- Levantar PostgreSQL con Docker Compose y versionar el esquema con migraciones de Flyway.
- Elegir el adaptador con un perfil de Spring, sin que el modo en memoria intente conectar con una base de datos.
- Comprobar el árbol nuevo en la verificación de estructura 4, valorar si `adapters` sigue siendo un solo módulo y demostrar con `git diff --stat` que el núcleo no cambió (hito H4).

En el capítulo 4 escribiste un repositorio en memoria y prometí que, algún día, lo cambiaríamos por uno de PostgreSQL «sin tocar una línea de `application` ni de `domain`». Hoy se paga esa promesa. Es el experimento que justifica toda la arquitectura: si los puertos están bien puestos, cambiar de base de datos es añadir un adaptador y decidir cuál se usa, no reescribir el programa. Si en algún momento tienes que abrir un caso de uso para que funcione la base de datos, el diseño tenía un fallo y mejor descubrirlo ahora.

Dos avisos de alcance. Primero: este es un capítulo de *adaptadores*, no de bases de datos. PostgreSQL entra como herramienta (ya sabes Docker, y el SQL que necesitas cabe en una pantalla) y no se explican índices, planes de consulta ni administración. Segundo: como el árbol de carpetas cambia (un paquete, recursos, ficheros de configuración y un archivo nuevo en la raíz), el capítulo incluye la **verificación de estructura 4** y, con ella, un ejercicio de revisión. Además, la sección 8.8 retoma lo que quedó anotado en el capítulo 7: si un solo módulo `adapters` sigue siendo buena idea ahora que entra JPA.

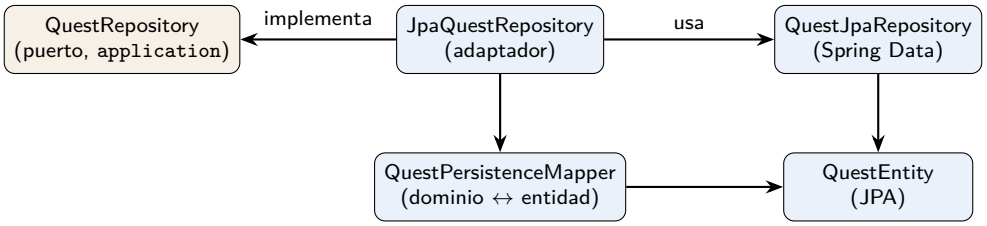
8.1 Qué cambia y qué no al cambiar de base de datos

Martin dedica un capítulo de [28] a la idea que sostiene este: el capítulo 30 se titula «The Database Is a Detail» (cito el título; el libro lo desarrolla). La base de datos es un mecanismo para guardar datos; lo que importa al negocio son los datos y las reglas, y por eso el mecanismo debe poder cambiarse sin tocar las reglas. Cockburn llegó a una conclusión parecida desde otro lado: el núcleo de la aplicación debe poder funcionar contra una base de datos real o contra un sustituto, y los adaptadores de salida son el sitio donde se elige [4]. En GuildBoard ya tienes el sustituto (la memoria) y el puerto (`QuestRepository` y `AdventurerRepository`). Falta el adaptador real.

Esta tabla es el contrato del capítulo. Lo que cambia está a la izquierda y lo que *no* debe cambiar a la derecha; al final lo comprobaremos con Git:

Cambia (o se añade)	No cambia
Un paquete nuevo, `adapter/out/persistence`, en `adapters`	`domain`: ni una clase.
Dependencias de `adapters` y `bootstrap`	`application`: ni los puertos ni los servicios.
Dos clases de configuración nuevas y `BeanConfig` más corta	Los controladores, los DTO y los mapeadores web.
Recursos: `application.yml`, `application-postgres.yml`, migraciones	`adapter/out/memory`: se queda, es el modo por defecto.
`compose.yaml` en la raíz	Las pruebas del dominio y `ArchitectureTest`.

El adaptador nuevo no es una sola clase. Son cuatro piezas, cada una con una razón de ser, que irás viendo en las secciones siguientes:



El puerto pertenece al núcleo; las otras cuatro cajas viven juntas en el paquete `adapter/out/persistence` de `adapters`. Para el resto de la aplicación, nada de eso existe: sigue pidiendo un `QuestRepository`.

8.2 Verificación de estructura 4: el árbol con persistencia

Nota

Este capítulo cambia la estructura de carpetas: un paquete nuevo, recursos, dos clases de configuración, dos `build.gradle` y un archivo nuevo en la raíz. Comprueba que tu proyecto coincide con el árbol de abajo antes de seguir. Compose, JPA, Flyway y los perfiles se explican en las secciones siguientes; aquí basta con leer los nombres como «lo que guarda datos» y «lo que elige cuál se usa».

8.2.1 El árbol anterior

Así dejó el proyecto el capítulo 7 (árbol T2, cuatro módulos):

Árbol anterior (T2, tras el capítulo 7)

```
guildboard/
|-- .git/, .gitignore
|-- settings.gradle
|-- build.gradle                configuraci^^c3^^b3n com^^c3^^ban (subprojects)
|-- gradlew, gradlew.bat
|-- docs/dominio.md
|-- gradle/wrapper/
|-- domain/                     build.gradle + src/{main,test}
|-- application/                build.gradle + src/main
|-- adapters/
```

```
|   |-- build.gradle            application, webmvc, validaci^^c3^^b3n, Lombok
|   `-- src/main/java/com/guildboard/adapter/
|       |-- in/web/             controladores + dto, mapper, error
|       `-- out/                memory, notification
`-- bootstrap/
    |-- build.gradle            plugin Boot; adapters, application, webmvc
    `-- src/main/
        |-- java/com/guildboard/
        |   |-- GuildBoardApplication.java
        |   `-- config/BeanConfig.java
        `-- resources/application.yml
```

8.2.2 El árbol nuevo

Lo nuevo va marcado con `[N]`, lo modificado con `[M]` y lo que no cambia, sin marca. Las dos migraciones y las dos clases de persistencia de aventureros las escribirás en los ejercicios, pero forman parte del árbol:

Árbol nuevo (T3, tras el capítulo 8)

```
guildboard/
|-- .git/, .gitignore
|-- settings.gradle
|-- build.gradle
|-- gradlew, gradlew.bat
|-- compose.yaml                    [N] PostgreSQL 18, solo desarrollo
|-- docs/dominio.md
|-- gradle/wrapper/
|-- domain/                         sin cambios
|-- application/                    sin cambios
|-- adapters/
|   |-- build.gradle                [M] + starter data-jpa
|   `-- src/main/java/com/guildboard/adapter/
|       |-- in/web/                 sin cambios
|       `-- out/
|           |-- memory/             sin cambios
|           |-- notification/        sin cambios
|           `-- persistence/        [N] ocho archivos:
|               QuestEntity, AdventurerEntity,
|               QuestJpaRepository, AdventurerJpaRepository,
|               QuestPersistenceMapper, AdventurerPersistenceMapper,
|               JpaQuestRepository, JpaAdventurerRepository
`-- bootstrap/
    |-- build.gradle                [M] + data-jpa, flyway, driver
    `-- src/
        |-- test/java/com/guildboard/
        |   `-- ArchitectureTest.java           [M] + regla JPA
        `-- main/
            |-- java/com/guildboard/
            |   |-- GuildBoardApplication.java
            |   `-- config/
            |       |-- BeanConfig.java                 [M] sin repositorios
            |       |-- MemoryPersistenceConfig.java    [N] "!postgres"
            |       `-- PostgresPersistenceConfig.java  [N] "postgres"
            `-- resources/
```

```
                |-- application.yml                      [M] + exclusiones
                |-- application-postgres.yml             [N]
                `-- db/migration/
                    |-- V1__create_guildboard_tables.sql   [N]
                    `-- V2__index_quests_by_adventurer.sql [N]
```

8.2.3 Cambios respecto al árbol anterior

- `domain` y `application`: sin cambios.
- `adapter/out/persistence`: paquete nuevo de `adapters` con ocho archivos, hermano de `memory` y `notification`.
- `adapters/build.gradle` y `bootstrap/build.gradle`: dependencias nuevas de JPA, Flyway y el driver.
- `ArchitectureTest` (en `bootstrap`): una prueba más, `adapterInDoesNotUseJpa` (sección 8.8).
- `MemoryPersistenceConfig` y `PostgresPersistenceConfig`: clases nuevas en `bootstrap`, que ya era la raíz de composición del capítulo 4; `BeanConfig` queda sin repositorios.
- `application.yml`, `application-postgres.yml` y las dos migraciones: recursos de `bootstrap`.
- `compose.yaml`: primer archivo de infraestructura en la raíz; en el capítulo 9 se le sumará `infra/`.

8.2.4 Comprobaciones

- [ ] `./gradlew test` pasa, `ArchitectureTest` incluido: `domain` sigue sin frameworks y `application` sigue dependiendo solo de `domain` (las entidades JPA son otras clases, en `adapters`, sección 8.4; `@Transactional` irá en el adaptador, sección 8.5).
- [ ] Cada adaptador está en `in` o en `out` según quién lo invoca: la persistencia está en `out`, porque la llama la aplicación.
- [ ] Las carpetas de tu proyecto coinciden con el árbol.
- [ ] `./gradlew build` pasa, y `git diff --stat` no muestra cambios en `domain` ni `application` (se comprueba al final del capítulo).

Si algo no coincide con el árbol, corrígelo antes de seguir. El ejercicio 8.4 practica estas comprobaciones sobre un cambio ajeno.

8.3 PostgreSQL con Docker Compose

Ya sabes Docker, así que el único concepto nuevo de esta sección es Compose. Un contenedor se arranca con un `docker run` largo, con sus variables, sus puertos y sus volúmenes. Funciona hasta que quieres repetirlo, compartirlo con el equipo o arrancar dos contenedores a la vez. **Docker Compose** es la herramienta de Docker que describe esos contenedores (en Compose se llaman *servicios*) en un archivo YAML y los arranca, para y destruye juntos con un solo comando [5]. En este capítulo describirá un único servicio, la base de datos; en el capítulo 15 describirá todo el sistema.

☞ Docker Compose

Compose ya viene con Docker en las instalaciones habituales, como un *plugin* del comando `docker`. Comprueba que lo tienes:

```
$ docker compose version
Docker Compose version v5.6.0
```

(La versión que veas puede ser otra; lo que importa es que el comando exista.) Fíjate en que se escribe `docker compose`, con un espacio. El comando antiguo con guion, `docker-compose`, era una versión anterior y separada; en este libro no se usa. Si `docker compose version` dice que no conoce el comando, instala el *plugin* de Compose siguiendo la documentación de Docker [5] y vuelve a comprobarlo. No hace falta instalar nada más: ni PostgreSQL en tu sistema ni el cliente `psql`, porque usaremos el que viene dentro del contenedor.

8.3.1 El archivo compose.yaml

Compose busca por defecto un archivo `compose.yaml` en la carpeta donde ejecutes el comando. Crea el de la raíz de GuildBoard, junto a `settings.gradle`:

compose.yaml

```
services:
  postgres:
    image: postgres:18
    environment:
      POSTGRES_DB: guildboard
      POSTGRES_USER: guildboard
      POSTGRES_PASSWORD: guildboard
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U guildboard -d guildboard"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  postgres-data:
```

Línea a línea:

- `services`: cada clave de este bloque es un servicio, es decir, un contenedor. El nombre (`postgres`) es el que usarás en los comandos y, dentro de una red de Compose, el nombre con el que otros contenedores lo encontrarán (lo aprovecharemos en el capítulo 15).
- `image: postgres:18`: la imagen oficial de PostgreSQL en su rama 18. La etiqueta `18` existe en Docker Hub (la comprobé el 8 de octubre de 2026) y apunta siempre a la última versión menor de la 18; la documentación de PostgreSQL de referencia del curso es la de esa versión [36].
- `environment`: la imagen lee estas tres variables solo la primera vez que arranca con

un directorio de datos vacío, y crea el usuario, su contraseña y la base de datos. Una contraseña igual que el usuario es aceptable en el portátil y un error en cualquier otro sitio.

- `ports: "5432:5432"`: publica el puerto del contenedor en tu máquina, *puerto de tu máquina:puerto del contenedor*. Las comillas no son decorativas: Compose las recomienda porque YAML puede leer algunos pares de números separados por dos puntos, como `22:22`, como un solo número. Si ya tienes un PostgreSQL local en el 5432, cambia el primero (por ejemplo, `"5433:5432"`) y acuérdate de cambiar el puerto de la URL en la sección 8.7.
- `volumes`: guarda los datos en un *volumen con nombre*, `postgres-data`, que Docker conserva aunque destruyas el contenedor. Sin él, cada `docker compose down` borraría la base. Se declara una vez en el bloque `volumes` del final. La ruta de dentro del contenedor es `/var/lib/postgresql`: a partir de la versión 18 de la imagen, el volumen definido y el directorio de datos cambiaron respecto a las anteriores (antes era `/var/lib/postgresql/data`), y la documentación de la imagen pide montar el volumen en `/var/lib/postgresql`. Si copias un `compose.yaml` de un tutorial antiguo, ese es el detalle que falla.
- `healthcheck`: un comando que Docker ejecuta cada 5 segundos para saber si el servicio *funciona*, no solo si el proceso existe. `pg_isready` es la utilidad de PostgreSQL que responde si el servidor acepta conexiones. Gracias a esto podremos esperar a que la base esté lista antes de arrancar la aplicación.

8.3.2 Los comandos que usarás

Todos se ejecutan en la carpeta del `compose.yaml`:

Comando	Qué hace
`docker compose up -d --wait`	Crea y arranca los servicios en segundo plano (`-d`) y espera a que su `healthcheck` pase (`--wait`).
`docker compose ps`	Lista los servicios y su estado (*healthy* cuando está listo).
`docker compose logs postgres`	Muestra los registros del servicio.
`docker compose exec postgres psql...`	Ejecuta un comando *dentro* del contenedor en marcha (abajo).
`docker compose down`	Para y elimina los contenedores y la red; *conserva* el volumen.
`docker compose down -v`	Lo mismo y además borra los volúmenes: la base vuelve a estar vacía.

Y para hablar con la base, el cliente de línea de comandos de PostgreSQL, `psql`, que ya está dentro de la imagen:

```
$ docker compose exec postgres psql -U guildboard -d guildboard
guildboard=# select current_user, current_database();
guildboard=# \dt
guildboard=# \q
```

`-U` es el usuario y `-d` la base. Las sentencias SQL terminan en punto y coma; los comandos que empiezan por barra invertida son de `psql`, no de SQL: `\dt` lista las tablas, `\d quests` describe una y `\q` sale. Practicarás todo esto en el ejercicio 8.1.

Spring Boot también sabe arrancar Compose

Spring Boot tiene una integración con Compose que arranca los servicios de un `compose.yaml` al iniciar la aplicación y deduce la URL de conexión de las etiquetas de la imagen [48]. En este curso no la usamos: queremos que arrancar la base y arrancar la aplicación sean dos pasos visibles, y que `compose.yaml` funcione igual cuando en el capítulo 15 metamos la aplicación dentro de Compose.

8.4 La entidad JPA no es el dominio

Guardar objetos Java en una base relacional se hace casi siempre con JPA (*Jakarta Persistence*), la especificación estándar de Java para eso. JPA solo describe qué anotaciones existen y cómo se comportan; quien lo cumple en nuestro proyecto es **Hibernate**. Encima, **Spring Data JPA** añade los repositorios: escribes una interfaz y Spring genera la clase que habla con la base [58]. Spring Boot configura todo a partir de lo que encuentre en el *classpath* y de las propiedades `spring.datasource.*` y `spring.jpa.*` [53].

☞ JPA y el driver de PostgreSQL

Hacen falta dos dependencias nuevas, cada una en el módulo que la usa. En `adapters` (que contendrá las entidades y el adaptador), el starter de Spring Data JPA; el resto del archivo es el del capítulo 7:

`adapters/build.gradle`

```
dependencies {
    implementation project(':application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

Y en `bootstrap`, que ensambla y elige, dos líneas más en `dependencies`: el mismo starter y el driver JDBC de PostgreSQL, que solo se necesita al ejecutar:

`bootstrap/build.gradle (líneas nuevas)`

```
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    runtimeOnly 'org.postgresql:postgresql'
```

El nombre del starter y el del driver los generó Spring Initializr para Boot 4.1.1 al pedir las dependencias `data-jpa` y `postgresql` (8 de octubre de 2026). El starter trae Hibernate, Spring Data JPA, el *pool* de conexiones HikariCP y el soporte de transacciones; no escribes ninguna versión porque las pone el catálogo de Boot (el de la 4.1.1 usa Hibernate 7.4.5 y el driver 42.7.13, según su `pom`).
¿Por qué `bootstrap` repite el starter si `adapters` ya lo tiene? Por lo que aprendiste en el capítulo 7: en `adapters` va con `implementation`, y eso lo oculta a quien dependa de `adapters` al compilar. `bootstrap` necesitará nombrar tipos de JPA y de Spring Data (en

la sección 8.7), así que los declara él mismo.
Comprueba que sigue compilando con `./gradlew compileJava`. **No arranques todavía la aplicación**: con JPA en el *classpath* y sin base de datos configurada, Boot fallaría al arrancar. Eso se arregla en la sección 8.7.

8.4.1 Por qué no se anota el dominio

La tentación es obvia: añadir `@Entity` y `@Id` a `Quest` y terminar. Pruébalo, y deshazlo con `git restore`, como vimos en el capítulo 7:

domain/Quest.java (líneas añadidas)

```
import jakarta.persistence.Entity;

@Entity
public final class Quest {
```

```
$ ./gradlew :domain:compileJava
> Task :domain:compileJava FAILED
.../domain/Quest.java:5: error: package jakarta.persistence
does not exist
```

(Mensaje abreviado.) El módulo `domain` no declara JPA, así que el compilador te frena, justo para lo que lo montamos en el capítulo anterior. Si le dieras JPA a `domain` para que compile, fallaría la primera pregunta de la lista de comprobación, y la regla de `domain` de `ArchitectureTest` marcaría el `import`. Pero incluso sin esas defensas hay tres razones de diseño para no hacerlo:

1. **JPA exige una forma que el dominio no quiere.** Hibernate necesita un constructor sin argumentos, una clase no final y campos modificables, porque crea el objeto vacío y luego rellena los campos. `Quest` es `final`, tiene un constructor privado y solo se construye con `draft`, que valida. Para dejar entrar a Hibernate habría que agujerear justo lo que protege las invariantes del capítulo 5.
2. **Cambian por razones distintas.** La forma de una tabla cambia por rendimiento, por una migración o por una consulta nueva; el dominio cambia cuando cambian las reglas. Atarlos significa que cada cambio de tabla puede tocar el núcleo. Es el principio de cohesión por razón de cambio (CCP) del capítulo 7, aplicado a las clases.
3. **Las filas de la base no deben cruzar el límite.** El artículo de Martin [27] lo dice así: por los límites deben cruzar estructuras de datos simples, no entidades de base de datos.

El precio es real y conviene decirlo: hay que escribir y mantener una segunda clase por agregado y un mapeador a mano (sin MapStruct, como en el resto del curso). El libro de Hombergs dedica espacio a ese coste y a cuándo compensa [19]. Aquí compensa por la razón 1: nuestro dominio es rico, no un saco de campos.

8.4.2 La entidad de misión

Una **entidad JPA** es una clase anotada que representa una fila de una tabla. Va en `adapter/out/persistence`, que es el paquete nuevo del árbol. Cada campo se corresponde con una columna:

Dominio (`Quest`)	Entidad (campo)	Columna de `quests`
`getId().value()`	`id`	`id` (texto)
`getTitle()`	`title`	`title` (texto)
`getReward().gold()`	`rewardGold`	`reward_gold` (entero)
`getRequiredRank()`	`requiredRank`	`required_rank` (texto)
`getStatus()`	`status`	`status` (texto)
`getAssignedTo()`	`adventurerId`	`adventurer_id` (opcional)
`getExpiresAt()`	`expiresAt`	`expires_at` (fecha con zona)

Los enumerados (`Rank`, `QuestStatus`) se guardan como texto con el nombre de la constante, y la entidad los declara como `String`. JPA tiene una anotación para guardar enumerados directamente, pero obligaría a la entidad a importar el `enum` del dominio; con `String`, el mapeador es la única clase que conoce los dos lados. Este es el código; guárdalo en `adapters`:

adapter/out/persistence/QuestEntity.java

```java
package com.guildboard.adapter.out.persistence;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.time.Instant;
import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "quests")
@Getter
@AllArgsConstructor
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class QuestEntity {

    @Id
    @Column(name = "id")
    private String id;

    @Column(name = "title", nullable = false)
    private String title;

    @Column(name = "reward_gold", nullable = false)
    private int rewardGold;

    @Column(name = "required_rank", nullable = false)
    private String requiredRank;

    @Column(name = "status", nullable = false)
    private String status;

    @Column(name = "adventurer_id")
    private String adventurerId;

    @Column(name = "expires_at")
    private Instant expiresAt;
```

```
41 }
```

Qué hace cada anotación. `@Entity` marca la clase como entidad; `@Table` le dice la tabla; `@Id` marca la clave primaria y `@Column` la columna de cada campo (con `nullable = false` para las obligatorias). Las de Lombok ya las conoces del capítulo 3: `@Getter` genera los lectores, `@AllArgsConstructor` un constructor con todos los campos, en el orden en que están escritos (el mapeador lo usa), y `@NoArgsConstructor(access = AccessLevel.PROTECTED)` el constructor vacío que exige Hibernate, pero oculto al resto de tu código. Los campos no son `final` y no hay *setters*: Hibernate rellena los campos directamente (como el `@Id` está en un campo, ese es el modo de acceso), y tu código solo construye la entidad con el constructor completo.

8.4.3 Del dominio a la entidad, y de vuelta

El mapeador es una clase `final` con métodos estáticos, igual que los mapeadores web del capítulo 6, y con visibilidad de paquete: solo el adaptador, que vive en el mismo paquete, lo necesita. La ida es directa: se leen los getters del `Quest`. La vuelta tiene un problema.

El problema de reconstruir un `Quest`. La única forma pública de crear una misión es `Quest.draft`, que siempre la deja en `DRAFT` y sin aventurero. Pero la fila de la base puede decir `ACCEPTED` por el aventurero `X`. ¿Cómo se construye ese objeto sin tocar el dominio? Hay dos caminos:

- **Añadir una fábrica de reconstrucción al dominio.** Es lo que hace Buckpal con `Account.withId(...)`, usada por su mapeador [20] (comprobado en el repositorio el 8 de octubre de 2026). Es limpio y robusto, pero modifica `domain` y abre una puerta pública que permite crear una misión en cualquier estado.
- **Repetir las transiciones.** Se crea el borrador y se le aplican las mismas operaciones públicas que tuvo en su vida (`publish`, `accept`, `complete`) hasta llegar al estado guardado. No añade nada al dominio.

El capítulo promete que `domain` no cambia, así que se elige el segundo. Tiene un truco que hay que contar: `accept` pide el rango del aventurero y el recuento de misiones activas, y aquí no son los de entonces, sino los más permisivos posibles (`Rank.VETERAN`, el más alto, y cero activas). No es una trampa: las reglas se comprobaron cuando la misión se aceptó de verdad; ahora solo se restaura el resultado.

adapter/out/persistence/QuestPersistenceMapper.java

```
1  package com.guildboard.adapter.out.persistence;
2
3  import com.guildboard.domain.AdventurerId;
4  import com.guildboard.domain.Quest;
5  import com.guildboard.domain.QuestId;
6  import com.guildboard.domain.QuestStatus;
7  import com.guildboard.domain.Rank;
8  import com.guildboard.domain.Reward;
9
10 final class QuestPersistenceMapper {
11
12     private QuestPersistenceMapper() {
13     }
14
15     static QuestEntity toEntity(Quest quest) {
```

```
16          return new QuestEntity(
17              quest.getId().value(),
18              quest.getTitle(),
19              quest.getReward().gold(),
20              quest.getRequiredRank().name(),
21              quest.getStatus().name(),
22              quest.getAssignedTo().map(AdventurerId::value).orElse(null),
23              quest.getExpiresAt().orElse(null));
24      }
25
26      static Quest toDomain(QuestEntity entity) {
27          Quest quest = Quest.draft(
28              new QuestId(entity.getId()),
29              entity.getTitle(),
30              new Reward(entity.getRewardGold()),
31              Rank.valueOf(entity.getRequiredRank()),
32              entity.getExpiresAt());
33          QuestStatus status = QuestStatus.valueOf(entity.getStatus());
34          switch (status) {
35              case DRAFT -> { }
36              case PUBLISHED -> quest.publish();
37              case ACCEPTED -> restoreAccepted(quest, entity);
38              case COMPLETED -> {
39                  restoreAccepted(quest, entity);
40                  quest.complete();
41              }
42          }
43          return quest;
44      }
45
46      private static void restoreAccepted(Quest quest, QuestEntity entity) {
47          if (entity.getAdventurerId() == null) {
48              throw new IllegalStateException("quest " + entity.getId()
49                  + " is " + entity.getStatus() + " without adventurer_id");
50          }
51          quest.publish();
52          quest.accept(new AdventurerId(entity.getAdventurerId()),
53                       Rank.VETERAN, 0);
54      }
55  }
```

Dos detalles. El `switch` cubre los cuatro estados y no tiene `default`: si algún día aparece un quinto estado, no habrá error de compilación (un `switch` sobre un `enum` que se usa como sentencia no lo exige), así que quien lo añada debe buscar los `switch` sobre `QuestStatus`. Y si la base contuviera un estado aceptado sin aventurero (algo que el dominio no permite crear), el mapeador lo dice con una excepción clara en lugar de seguir con un dato inconsistente.

La desventaja del camino elegido es el acoplamiento con las transiciones: si `accept` ganara algún día una regla que leyera algo más, esta restauración dejaría de valer, y entonces la solución sería la fábrica de reconstrucción, que sí aparecería en un `git diff` del dominio y se revisaría en la verificación de estructura correspondiente. Con el dominio actual, la repetición es suficiente.

El aventurero sigue el mismo patrón con menos trabajo: se registra con `Adventurer.register` y se le dan los puntos de experiencia guardados con `gainExperience`, que recalcula el rango (el rango deriva de la experiencia). Lo escribirás tú en el ejercicio 8.3.

8.5 El adaptador JPA y el puerto de salida

8.5.1 El repositorio de Spring Data

Spring Data JPA genera la implementación a partir de una interfaz que extiende `JpaRepository`, con el tipo de la entidad y el de su clave. Los métodos heredados (`findById`, `save`, `count`…) ya funcionan. Para una consulta propia basta con *nombrar* bien el método: Spring la deduce del nombre [58]. Aquí hace falta una, la de contar las misiones aceptadas por un aventurero:

adapter/out/persistence/QuestJpaRepository.java

```
package com.guildboard.adapter.out.persistence;

import org.springframework.data.jpa.repository.JpaRepository;

public interface QuestJpaRepository
        extends JpaRepository<QuestEntity, String> {

    long countByAdventurerIdAndStatus(String adventurerId, String status);
}
```

`countBy` significa «contar», y lo que sigue son nombres de *campos de la entidad* unidos por `And`: es el equivalente de `select count(*) from quests where adventurer_id = ? and status = ?`. Los nombres son los de los campos (`adventurerId`, no `adventurer_id`); si te equivocas en uno, la aplicación no arranca y el error nombra el campo que no encuentra. Fíjate en que la interfaz es *pública* y la entidad también: la configuración de `bootstrap` necesita nombrarlas.

8.5.2 El adaptador

El adaptador es la clase que el resto de la aplicación conoce, a través del puerto. Implementa `QuestRepository`, traduce con el mapeador y delega en el repositorio de Spring Data:

adapter/out/persistence/JpaQuestRepository.java

```
package com.guildboard.adapter.out.persistence;

import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import com.guildboard.domain.QuestStatus;
import java.util.Optional;
import lombok.RequiredArgsConstructor;
import org.springframework.transaction.annotation.Transactional;

@RequiredArgsConstructor
@Transactional
public class JpaQuestRepository implements QuestRepository {

    private final QuestJpaRepository jpa;

    @Override
    @Transactional(readOnly = true)
    public Optional<Quest> findById(QuestId id) {
```

```
        return jpa.findById(id.value())
            .map(QuestPersistenceMapper::toDomain);
    }

    @Override
    public void save(Quest quest) {
        jpa.save(QuestPersistenceMapper.toEntity(quest));
    }

    @Override
    @Transactional(readOnly = true)
    public int countActiveBy(AdventurerId adventurerId) {
        return Math.toIntExact(jpa.countByAdventurerIdAndStatus(
            adventurerId.value(), QuestStatus.ACCEPTED.name()));
    }
}
```

`countActiveBy` cuenta lo mismo que el repositorio en memoria: las misiones `ACCEPTED` del aventurero, que es lo que el capítulo 5 definió como «activa». No se anota con `@Component`: igual que los demás objetos del proyecto, se creará en la raíz de composición (sección 8.7).

Una nota sobre `save`. Spring Data decide si una entidad es nueva mirando su clave: si la clave es nula, inserta; si no, la *fusiona* (`merge`), lo que supone leer la fila y escribir lo que haya cambiado. Nuestras claves las genera la aplicación y nunca son nulas, así que cada `save` hace una lectura de más. Es el precio de conservar ids propios, aceptable aquí, y el mismo método sirve para crear y para actualizar.

8.5.3 Por qué `@Transactional` va aquí

Una **transacción** agrupa operaciones sobre la base para que ocurran todas o ninguna. En Spring se pide con `@Transactional`, que envuelve el método en una transacción (internamente, con un *proxy*). La decisión del curso, anunciada en el capítulo 5, es que esta anotación vive en el adaptador de persistencia y no en los casos de uso, por dos razones:

- La transacción es un mecanismo de la base de datos. El caso de uso dice *qué* hacer; *cómo* se garantiza que se guarde entero es asunto del adaptador. Con el repositorio en memoria no hay transacciones y el caso de uso no se entera.
- `application` no puede usarla: `@Transactional` es de Spring, y el módulo no tiene Spring en su *classpath*. Es la regla de dependencia otra vez.

En la clase, `@Transactional` sin más cubre todos los métodos; en las consultas se sobrescribe con `readOnly = true`, una indicación a Hibernate de que no va a haber escrituras. Con este adaptador concreto, la anotación no cambia el resultado (los métodos de `JpaRepository` ya son transaccionales por sí mismos), pero fija el límite donde debe estar, y se volverá imprescindible el día que un método del adaptador haga varias operaciones.

Y una advertencia que conviene decir en voz alta. **Cada método del adaptador es una transacción distinta; un caso de uso no lo es.** Los dos problemas que reconocimos en el capítulo 5 siguen donde estaban: en `AcceptQuest`, contar las misiones activas y guardar son dos transacciones, y dos peticiones simultáneas pueden pasar el límite de tres; y en `CompleteQuest`, guardar la misión y dar la experiencia son dos transacciones, y un fallo entre ambas deja el paso a medias. Con la memoria era posible que no lo notaras; con una base de datos compartida entre varias instancias, la carrera es más real. El curso no lo resuelve (necesitaría una transacción en la capa de aplicación, un bloqueo de filas o una saga), y lo dice aquí para que no lo descubras

en producción.

8.6 Migraciones con Flyway

Las tablas tienen que existir antes de que el adaptador las use. Hay una opción cómoda, dejar que Hibernate las cree al arrancar (`spring.jpa.hibernate.ddl-auto`), pero genera el esquema sin que nadie lo revise y no sabe evolucionarlo con cuidado cuando ya hay datos. La documentación de Boot recomienda usar un único mecanismo para generar el esquema [46]. Aquí es una herramienta de migraciones: **Flyway**.

Flyway funciona con tres ideas. (1) El esquema se describe con archivos SQL numerados, las *migraciones*, cuyo nombre sigue el patrón `V<versión>__<descripción>.sql` (la `V` mayúscula, el número y dos guiones bajos). (2) Al arrancar, Flyway compara esos archivos con una tabla que él mismo crea en la base, `flyway_schema_history`, y aplica en orden solo los que faltan. (3) Guarda una suma de comprobación de cada migración aplicada, y si el archivo cambia después, se niega a continuar. De ahí la regla de oro: **una migración aplicada no se edita jamás; los cambios nuevos van en una migración nueva.**

☞ Flyway

Dos dependencias en `bootstrap`, que es el módulo que elige y ejecuta las migraciones. Las dos líneas son las que Spring Initializr generó para Boot 4.1.1 con las dependencias `flyway` y `postgresql`; la documentación de Boot explica que el starter solo cubre bases en memoria o de archivo, y que para PostgreSQL hace falta además el módulo `flyway-database-postgresql` [46]:

bootstrap/build.gradle (líneas nuevas)

```
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
```

La versión de Flyway la pone el catálogo de Boot (12.4.0 en la 4.1.1). Comprueba con `./gradlew compileJava`. Flyway no hace nada hasta que arranques la aplicación con una base configurada (sección 8.7); a partir de ahí, cada arranque ejecuta las migraciones pendientes.

Por defecto Flyway busca en `classpath:db/migration`, es decir, en la carpeta `src/main/resources/db/migration` de cualquier módulo [46]. El curso coloca las migraciones en `bootstrap`. La razón: `bootstrap` es el módulo que decide con qué tecnología se ejecuta la aplicación y el que tiene el starter de Flyway, mientras que `adapters` solo conoce JPA. Hay un argumento contrario, y es bueno: el adaptador y el esquema cambian juntos (CCP otra vez), y quizá vivirían mejor uno junto a otro. Como Flyway busca en todo el *classpath*, mover las migraciones a `adapters` no obligaría a cambiar ninguna configuración si algún día lo prefieres.

La primera migración crea las dos tablas. Los ids son texto porque los genera la aplicación; los rangos y estados se guardan con el nombre de su constante; y `expires_at` existe desde ya porque, desde el capítulo 6, `Quest` tiene fecha de caducidad (junto a `adventurer_id`, es una de las dos columnas opcionales):

bootstrap/.../db/migration/V1__create_guildboard_tables.sql

```
-- V1: esquema inicial de GuildBoard.
-- Los ids son texto (los genera la aplicaci^^c3^^b3n). Rangos y estados se
```

```
-- guardan con el nombre de la constante del dominio.

CREATE TABLE adventurers (
    id               VARCHAR(64)  PRIMARY KEY,
    name             VARCHAR(100) NOT NULL,
    adventurer_rank  VARCHAR(20)  NOT NULL,
    experience       INTEGER      NOT NULL
);

CREATE TABLE quests (
    id             VARCHAR(64)  PRIMARY KEY,
    title          VARCHAR(255) NOT NULL,
    reward_gold    INTEGER      NOT NULL,
    required_rank  VARCHAR(20)  NOT NULL,
    status         VARCHAR(20)  NOT NULL,
    adventurer_id  VARCHAR(64),
    expires_at     TIMESTAMP WITH TIME ZONE
);
```

(Probé este SQL contra PostgreSQL 18.6 en un contenedor el 8 de octubre de 2026; lo que no ejecuté es la aplicación entera.) Tres decisiones, para que no parezcan descuidos:

- `TIMESTAMP WITH TIME ZONE` es el tipo de PostgreSQL para un instante absoluto, que es lo que representa un `Instant`. Guarda microsegundos: un `Instant` con nanosegundos pierde los tres últimos dígitos en la ida y vuelta.
- **No hay clave foránea** de `quests.adventurer_id` a `adventurers.id`. Una clave foránea es lo normal en una base única, pero en el capítulo 9 las misiones y los aventureros acabarán en servicios separados y en bases separadas, y ese vínculo no podrá existir. Es mejor acostumbrarse a que el dominio, y no la base, garantice la relación (el servicio ya comprueba que el aventurero existe).
- La columna se llama `adventurer_rank` y no `rank` para no chocar con una palabra que PostgreSQL usa como función.

Y la configuración de Hibernate se queda en modo pasivo: no crea ni valida nada (`ddl-auto` con valor `none`); el esquema es de Flyway. Hibernate podría validar que las entidades casan con las tablas al arrancar, y es una buena práctica; no lo activamos porque no he podido ejecutarlo para comprobar que mis tipos pasan esa validación.

8.7 Elegir el adaptador por perfil

Tienes dos adaptadores para cada puerto y falta decidir cuál se usa. Hay un obstáculo antes: cuando arranques la aplicación sin hacer nada más, Boot verá JPA, Flyway y un driver en el *classpath* y los configurará. Esta sección resuelve las dos cosas con un mismo mecanismo.

8.7.1 El problema: configuración automática sin base de datos

Spring Boot funciona con **autoconfiguración**: un conjunto de clases que se activan solas cuando detectan ciertas librerías en el *classpath* [52]. Al añadir `data-jpa`, Flyway y el driver, se activan las que crean la conexión a la base de datos (`DataSource`), la fábrica de Hibernate, los repositorios de Spring Data y las migraciones. Todas necesitan una URL de conexión. Si arrancas en modo memoria sin ninguna, la aplicación falla al iniciar con un error de que no

puede configurar el `DataSource` (el texto exacto varía con la versión), aunque nadie fuera a usar la base. El modo en memoria tiene que seguir siendo el modo por defecto y funcionar sin Docker ni PostgreSQL.

8.7.2 Perfiles de Spring

Un **perfil** es un nombre que activas al arrancar para cambiar qué parte de la aplicación se carga [52]. Dos usos nos interesan:

- **Configuración por perfil.** Si el perfil `postgres` está activo, Boot lee además `application-postgres.yml`, y sus valores ganan a los de `application.yml`. Si no hay ningún perfil activo, no se lee.
- **Clases por perfil.** Una clase `@Configuration` con `@Profile("postgres")` solo se carga con ese perfil activo. `@Profile("!postgres")` (con el signo de admiración, «no») se carga en todos los demás casos, incluido el de no activar ninguno.

Un perfil se activa con la propiedad `spring.profiles.active`, que puede llegar por la línea de comandos o por una variable de entorno, `SPRING_PROFILES_ACTIVE` [52]. Usaremos la variable de entorno.

8.7.3 Apagar la autoconfiguración de datos en el modo memoria

La propiedad `spring.autoconfigure.exclude` es una lista de clases de autoconfiguración que Boot debe ignorar [52]. La ponemos en el `application.yml` (el perfil por defecto, es decir, memoria) para las cuatro piezas que necesitan base de datos:

bootstrap/src/main/resources/application.yml

```yaml
spring:
  application:
    name: guildboard
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.
    ↪ HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.
    ↪ DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration

server:
  port: 8080
```

Los nombres no son los que verás en tutoriales anteriores a Boot 4. En Boot 4 la autoconfiguración se repartió en módulos más pequeños y las clases cambiaron de paquete: ahora cada una vive en el módulo de su tecnología (por ejemplo, `spring-boot-jdbc` para el `DataSource`). Si copias un nombre antiguo, la exclusión no surte efecto, porque esa clase ya no existe en Boot 4: el modo memoria vuelve a fallar por el `DataSource`, y ese es el síntoma que has de reconocer. Comprobé los cuatro nombres abriendo, el 8 de octubre de 2026, los *jar* de esos módulos en su versión 4.1.1 en Maven Central: cada uno trae un archivo `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` que lista sus clases. Es también la forma de comprobarlos tú si una versión posterior los vuelve a mover.

Ahora el perfil `postgres`. Dos cosas: vaciar esa lista y dar los datos de conexión. Una lista definida en un archivo de perfil *sustituye por completo* a la del archivo base, no se suma [52], y por eso vale una lista vacía:

bootstrap/src/main/resources/application-postgres.yml

```yaml
spring:
  autoconfigure:
    exclude: []
  datasource:
    url: jdbc:postgresql://localhost:5432/guildboard
    username: guildboard
    password: guildboard
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: "none"
```

Con la lista vacía, las cuatro piezas vuelven a activarse: Boot crea el `DataSource` con esa URL, ejecuta las migraciones de Flyway y arranca Hibernate. `open-in-view: false` apaga un comportamiento por defecto de Boot en aplicaciones web (mantener abierta la sesión de base de datos durante toda la petición HTTP, incluso en los controladores) [53]; en esta arquitectura los controladores no tocan la base, así que no hace falta. Y `ddl-auto: "none"` ya lo comentamos: el esquema es de Flyway. Las comillas en `"none"` evitan que alguien lea el valor como otra cosa.

Si la lista vacía no basta

Revisé la regla de sustitución en la documentación, pero no he ejecutado la aplicación. Si con el perfil `postgres` la aplicación sigue diciendo que no hay `DataSource`, comprueba primero la sangría de `exclude: []` y que el archivo se llama exactamente `application-postgres.yml`. Con `--debug` (más abajo) Boot imprime qué ha excluido.

8.7.4 Las dos clases de configuración

Los repositorios dejan de registrarse en `BeanConfig` y pasan a dos clases de `bootstrap/config`, una por perfil. La del modo memoria contiene lo que hasta ahora estaba en `BeanConfig`:

config/MemoryPersistenceConfig.java

```java
package com.guildboard.config;

import com.guildboard.adapter.out.memory.InMemoryAdventurerRepository;
import com.guildboard.adapter.out.memory.InMemoryQuestRepository;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.application.port.out.QuestRepository;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

@Configuration
@Profile("!postgres")
public class MemoryPersistenceConfig {

    @Bean
```

```
    public QuestRepository questRepository() {
        return new InMemoryQuestRepository();
    }

    @Bean
    public AdventurerRepository adventurerRepository() {
        return new InMemoryAdventurerRepository();
    }
}
```

La del perfil `postgres` registra los adaptadores JPA. Además lleva dos anotaciones que le dicen a Spring dónde buscar: `@EnableJpaRepositories` (dónde están las interfaces de Spring Data) y `@EntityScan` (dónde están las entidades). Con la configuración por defecto de Boot bastaría con la autoconfiguración, porque Boot busca en el paquete de la clase principal; las anotaciones explícitas hacen visible dónde vive la persistencia, y no fallan si algún día mueves el paquete:

config/PostgresPersistenceConfig.java

```
package com.guildboard.config;

import com.guildboard.adapter.out.persistence.AdventurerJpaRepository;
import com.guildboard.adapter.out.persistence.JpaAdventurerRepository;
import com.guildboard.adapter.out.persistence.JpaQuestRepository;
import com.guildboard.adapter.out.persistence.QuestEntity;
import com.guildboard.adapter.out.persistence.QuestJpaRepository;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.application.port.out.QuestRepository;
import org.springframework.boot.persistence.autoconfigure.EntityScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;

@Configuration
@Profile("postgres")
@EnableJpaRepositories(basePackageClasses = QuestJpaRepository.class)
@EntityScan(basePackageClasses = QuestEntity.class)
public class PostgresPersistenceConfig {

    @Bean
    public QuestRepository questRepository(QuestJpaRepository jpa) {
        return new JpaQuestRepository(jpa);
    }

    @Bean
    public AdventurerRepository adventurerRepository(
            AdventurerJpaRepository jpa) {
        return new JpaAdventurerRepository(jpa);
    }
}
```

`basePackageClasses` indica el paquete a través de una clase que vive en él, que sobrevive a un renombrado mejor que un texto. El paquete de `@EntityScan` en Boot 4 es `org.springframework.boot.persistence.autoconfigure`; lo comprobé en la documentación de Boot y en el *jar* `spring-boot-persistence` 4.1.1.

Falta el último paso: `BeanConfig` ya no debe registrar los repositorios, o habría dos *beans* del mismo tipo en el modo memoria. Borra de él los dos métodos siguientes y las dos líneas `import` de `InMemoryQuestRepository` y `InMemoryAdventurerRepository`; el resto del archivo queda como estaba (su versión completa está en la solución del ejercicio 8.5):

config/BeanConfig.java (métodos que se borran)

```
    @Bean
    public QuestRepository questRepository() {
        return new InMemoryQuestRepository();
    }

    @Bean
    public AdventurerRepository adventurerRepository() {
        return new InMemoryAdventurerRepository();
    }
```

Los casos de uso de `BeanConfig` piden un `QuestRepository` por tipo; Spring les entrega el que haya registrado el perfil activo. Fíjate en lo que *no* ha cambiado: ni un servicio ni un caso de uso saben que hay dos adaptadores. Elegir cuál se usa es lo único que esta capa de configuración hace.

8.7.5 Arrancar de las dos formas

Sin perfil, como siempre:

```
$ ./gradlew :bootstrap:bootRun
```

No debe aparecer ninguna línea de Hikari, de Hibernate ni de Flyway, y los datos viven solo mientras la aplicación. Si quieres ver por qué, arranca con el informe de condiciones de Boot, que incluye un bloque de exclusiones con las cuatro clases:

```
$ ./gradlew :bootstrap:bootRun --args='--debug'
```

Con PostgreSQL, primero la base (la primera vez descarga la imagen) y después la aplicación con el perfil:

```
$ docker compose up -d --wait
$ SPRING_PROFILES_ACTIVE=postgres ./gradlew :bootstrap:bootRun
```

En el registro de arranque debes ver a Flyway aplicar la migración, con un texto parecido a este (cambia con la versión):

```
Database: jdbc:postgresql://localhost:5432/guildboard (PostgreSQL 18.6)
Migrating schema "public" to version "1 - create guildboard tables"
Successfully applied 1 migration to schema "public", now at version v1
```

Si la base no está levantada, el arranque falla con un error de conexión rechazada: es la señal para ejecutar primero `docker compose up`. Y si aparece el aviso de que no puede de-

terminar el `DataSource` en modo memoria, es que se te ha caído alguna exclusión o hay otra autoconfiguración de datos activa.

8.8 ¿Sigue valiendo un solo módulo de adaptadores?

Ahora que hay JPA en `adapters`, esta decisión se puede razonar con datos y no solo con previsiones. El capítulo 7 eligió un solo módulo y dejó tres condiciones para cambiar: (a) un adaptador exige una dependencia que no quieres en el *classpath* de los demás, (b) dos equipos distintos tocan adaptadores distintos, (c) quieres reutilizar un adaptador.

Qué ha pasado con la condición (a). Se cumple, en parte. Antes, el módulo tenía Spring MVC y validación; ahora tiene además Hibernate, Spring Data, el *pool* de conexiones y las transacciones, y los ve el código de `adapter/in/web` igual que el de `adapter/out/persistence`. Ningún controlador usa JPA, pero nada del compilador lo impide: es la debilidad de la columna A de la tabla del capítulo anterior, ahora con un caso real. Las condiciones (b) y (c) no han cambiado.

Hay tres caminos, y los tres son legítimos:

	Qué es	Qué cuesta y qué da
A. Un módulo `adapters` (el árbol de la verificación de estructura 4)	Todo junto, como hasta ahora.	Ningún módulo nuevo. JPA visible desde los controladores; lo vigilan las reglas de `ArchitectureTest`, que solo miran JPA si añades su regla.
B. Un módulo por adaptador	`adapters-web`, `adapters-memory`, `adapters-notification`, `adapters-persistence`.	Cuatro `build.gradle` en vez de uno. El compilador separa todo.
C. Solo se aparta el pesado	`adapters` y un `adapters-persistence` con JPA.	Un módulo más. El compilador impide que un controlador vea JPA, y la regla de aislamiento de `ArchitectureTest` habría que revisarla.

La decisión: A, y se vuelve a mirar si se cumple una condición concreta. Las razones, por orden de peso:

1. **Es reversible a bajo coste.** Los paquetes Java no dependen del módulo que los contiene. Pasar de A a C o a B es un `git mv` de carpetas y un `build.gradle` nuevo, exactamente como el corte del capítulo 7: ninguna línea de Java cambia. Elegir A hoy no te encierra.
2. **El coste de B y de C se multiplica.** A partir del capítulo 9, GuildBoard son varios servicios, cada uno con sus cuatro módulos. Un módulo extra por servicio no es uno, es tantos como servicios.
3. **No hay evidencia de daño, y la debilidad se vigila con una línea.** El camino más probable para meter JPA donde no toca, usar una clase de `adapter.out.persistence` desde un controlador, lo caza la regla `adapterInDoesNotDependOnAdapterOut` del capítulo 7. Lo que esa regla no ve es un uso directo de `jakarta.persistence` (por ejemplo, un `EntityManager`) en `adapter/in`. Para eso basta una prueba más en el mismo `ArchitectureTest`, sin ninguna tarea de Gradle [66]:

Prueba nueva en bootstrap/src/test/java/com/guildboard/ArchitectureTest.java

```
@Test
void adapterInDoesNotUseJpa() {
```

```
    noClasses().that().resideInAPackage("..adapter.in..")
        .should().dependOnClassesThat().resideInAPackage("jakarta.persistence..
    ↪ ")
        .check(CLASSES);
}
```

Con esta prueba, la condición de cambio (1) de abajo deja de depender de la revisión humana: falla `./gradlew test`. La escribirás como parte del ejercicio 8.5.

Cuándo cambiar. Tres condiciones que se pueden comprobar en el repositorio y no son opiniones: (1) falla la prueba `adapterInDoesNotUseJpa` (o aparece en `adapter/in` una dependencia de `org.hibernate` o `org.springframework.data`, que la prueba se puede ampliar para cubrir); (2) entra una segunda dependencia pesada en `adapters` (el cliente de mensajería del capítulo 11 sería la candidata), y entonces el camino natural es C; (3) un segundo equipo empieza a mantener el adaptador de persistencia con otro ritmo de entregas. El curso mantiene un único módulo `adapters` por servicio; el capítulo 11 es el sitio para volver a mirarlo con la mensajería delante. El ejercicio 8.4 te pide aplicar la primera condición a un cambio ajeno.

8.9 En el proyecto: hito H4

Hito 4: GuildBoard guarda en PostgreSQL

Al terminar el ejercicio 8.5, GuildBoard funciona de dos formas con el mismo código de negocio: sin perfil, con los repositorios en memoria de siempre; con el perfil `postgres`, con PostgreSQL en Compose y el esquema creado por Flyway. Archivos nuevos:

Archivos nuevos de GuildBoard tras el capítulo 8

```
compose.yaml                                    (ra^^c3^^adz)
adapters/.../adapter/out/persistence/           ocho archivos
bootstrap/.../config/                           MemoryPersistenceConfig,
                                                PostgresPersistenceConfig
bootstrap/src/main/resources/                   application-postgres.yml
bootstrap/src/main/resources/db/migration/      V1 (cap^^c3^^adtulo), V2 (
    ↪ ejercicio)
```

Se modifican `adapters/build.gradle`, `bootstrap/build.gradle`, `BeanConfig` y `application.yml`. No se modifica ningún archivo de `domain` ni de `application`.

8.9.1 Probar el hito

Arranca con el perfil `postgres` (sección 8.7) y repite el escenario del capítulo 6: registra a Mara, publica un sótano de 150 de oro, acéptalo y complétalo. Las respuestas son las mismas. Comprueba que la base tiene los datos con el cliente que viene en el contenedor (los ids de los `curl` de ejemplo no aparecen: son largos y no aportan):

```
$ docker compose exec postgres psql -U guildboard -d guildboard \
    -c "select name, adventurer_rank, experience from adventurers"
 name | adventurer_rank | experience
------+-----------------+------------
 Mara | ADEPT           |        150
```

```
$ docker compose exec postgres psql -U guildboard -d guildboard \
    -c "select title, status, expires_at from quests"
       title        |  status   |          expires_at
--------------------+-----------+-------------------------------
 Limpiar el s^^c3^^b3tano  | COMPLETED | 2026-10-08 11:30:00.412903+00
```

(La fecha y el formato exacto dependen de cuándo lo hagas y de tu zona.) Esta es la prueba de que el adaptador funciona: la experiencia se dio a Mara (150 de oro, así que pasa de `NOVICE` a `ADEPT`) y la misión quedó completada. Ahora la prueba que la memoria no podía pasar: para la aplicación con `Ctrl+C`, vuelve a arrancarla con el mismo perfil y pide a Mara con el id que anotaste:

```
$ curl -s http://localhost:8080/adventurers/$MARA
{"id":"7d1c4e52-0b8a-4f3e-9c61-5a2b8e0d7f14","name":"Mara",
 "rank":"ADEPT","experience":150}
```

Los datos han sobrevivido. En el segundo arranque, Flyway no aplica nada (su registro dice algo como *«Schema "public" is up to date»*; el texto exacto varía). Si la arrancas sin perfil, la misma petición da `404`: los datos nuevos no están en la memoria. Es la mejor demostración de que son dos adaptadores intercambiables detrás del mismo puerto.

La demostración del capítulo. Haz un *commit* con todo (mensaje `H4: adaptador PostgreSQL con JPA y Flyway`) y pide a Git que compare con el anterior, solo las carpetas del núcleo:

```
$ git diff --stat HEAD~1 HEAD -- domain application
$
```

No imprime nada: ni una línea cambió en `domain` ni en `application`. Con la carpeta de adaptadores y la raíz sí:

```
$ git diff --stat HEAD~1 HEAD
 adapters/build.gradle                          |  1 +
 .../out/persistence/AdventurerEntity.java      | 29 ++++
 .../out/persistence/QuestEntity.java           | 41 +++++
 .../out/persistence/JpaQuestRepository.java    | 39 +++++
 ...
 bootstrap/build.gradle                         |  5 +
 .../com/guildboard/config/BeanConfig.java      | 12 --
 .../config/MemoryPersistenceConfig.java        | 25 +++
 .../config/PostgresPersistenceConfig.java      | 41 +++
 bootstrap/src/main/resources/application.yml   |  6 +
 compose.yaml                                   | 18 +++
 18 files changed, 520 insertions(+), 12 deletions(-)
```

(Salida resumida; los recuentos de líneas serán los de tu código.) Las únicas carpetas que aparecen son `adapters`, `bootstrap` y la raíz. Esa es la prueba de que el cambio de adaptador ha respetado la regla de dependencia: el núcleo no sabe que existe PostgreSQL.

8.10 Ejercicios

Los ejercicios parten del proyecto tal como lo dejó el capítulo 7 (cuatro módulos), salvo indicación en contra. El de proyecto es el último y junta todo el capítulo (incluye la base de datos y el adaptador de aventureros); el 8.2 y el 8.4 trabajan sobre su resultado. Haz primero el proyecto o, si prefieres, el ejercicio 8.1 y el 8.3, que son partes suyas. No arranques la aplicación con el perfil `postgres` hasta tener la base y los dos adaptadores.

Ejercicio 8.1: PostgreSQL con Compose ★★☆☆☆

Levanta la base de datos de GuildBoard con Compose y conéctate con el cliente que viene en el contenedor. No hay ningún cambio de código. Parte de la raíz del proyecto.

1. Comprueba que Compose existe: `docker compose version`.
2. Crea `compose.yaml` en la raíz con el contenido de la sección 8.3. Valídalo con `docker compose config`, que imprime el archivo ya interpretado y falla si el YAML no es correcto.
3. Arranca el servicio con `docker compose up -d --wait` y mira con `docker compose ps` su estado.
4. Conéctate con `docker compose exec postgres psql -U guildboard -d guildboard`. Ejecuta `select current_user, current_database();`. Después crea una tabla de prueba (`create table prueba (n integer);`), inserta una fila (`insert into prueba values (42);`) y consúltala. Sal con `\q`.
5. Ejecuta `docker compose down`, vuelve a levantar la base con el mismo comando del paso 3 y vuelve a consultar la tabla `prueba`.
6. Ejecuta ahora `docker compose down -v`, levanta otra vez y lista las tablas con `\dt`. Explica por qué la tabla ya no está.
7. Deja la base levantada para los demás ejercicios.

Está bien si: el paso 3 muestra el servicio en estado *healthy*; el paso 4 responde `guildboard` dos veces; el paso 5 devuelve la fila con 42; el paso 6 dice que no se encuentran relaciones; y no has instalado PostgreSQL ni `psql` en tu sistema.

Pista: *si el puerto 5432 está ocupado, cambia la parte izquierda de* `ports` *(por ejemplo,* `"5433:5432"`*) y recuerda ese número para la URL de conexión; si el servicio no llega a* healthy*, mira* `docker compose logs postgres`*.*

Guiado ▪ solución en la pág. 272

Ejercicio 8.2: Una migración nueva: V2 ★★★☆☆

La consulta que cuenta las misiones activas de un aventurero filtra `quests` por `adventurer_id` y `status`. Un *índice* sobre esas dos columnas hace que la base las encuentre sin recorrer toda la tabla, y se crea en SQL con `CREATE INDEX nombre ON tabla (columna1, columna2);`. Parte del proyecto con el hito H4 funcionando (ejercicio 8.5) y la base levantada con la migración `V1` ya aplicada.

1. Consulta el historial de migraciones: `select installed_rank, version, description, success from flyway_schema_history;` con `psql`. Debe haber una fila.
2. Crea `V2__index_quests_by_adventurer.sql` en la carpeta de migraciones de `bootstrap` con la sentencia que crea el índice `idx_quests_adventurer_status`.

3. Arranca la aplicación con el perfil `postgres` y repite la consulta del paso 1. Después ejecuta `\d quests` y busca el índice.
4. Añade al final de `V1` una línea de comentario SQL (`-- cambio`) y arranca de nuevo. Copia la primera línea del error y deshaz el cambio con `git restore`.
5. Escribe en dos frases por qué no se debe editar `V1` y qué se hace en su lugar.

Está bien si: el historial tiene dos filas, ambas con `success` verdadero; el índice aparece en `\d quests`; `V1` queda idéntica a como estaba; el error del paso 4 nombra un desajuste de suma de comprobación (*checksum*) de la versión 1; y la aplicación vuelve a arrancar tras deshacerlo.

Pista: *el nombre de la migración lleva una* `V` *mayúscula, el número y* dos *guiones bajos antes de la descripción.*

Práctica ▪ solución en la pág. 273

Ejercicio 8.3: JpaAdventurerRepository ★★★☆☆

Escribe el adaptador de persistencia de los aventureros, por analogía con el de misiones de las secciones 8.4 y 8.5. La tabla ya existe en `V1`: `adventurers (id, name, adventurer_rank, experience)`. Cuatro archivos en `adapter/out/persistence`:

1. `AdventurerEntity`: la entidad de la tabla, con los campos `id`, `name`, `rank` (columna `adventurer_rank`) y `experience`.
2. `AdventurerJpaRepository`: la interfaz de Spring Data, sin métodos propios.
3. `AdventurerPersistenceMapper`: `toEntity(Adventurer)` y `toDomain(AdventurerEntity)`. Al reconstruir, el rango debe quedar como estaba sin tocar el dominio.
4. `JpaAdventurerRepository`: implementa `AdventurerRepository` con `@Transactional`.

Está bien si: el proyecto compila; con el perfil `postgres`, registrar un aventurero crea una fila visible con `psql`; tras completar una misión de 150 de oro, la fila muestra `ADEPT` y 150; al reiniciar la aplicación, `GET /adventurers/{id}` devuelve el mismo rango y experiencia; y `grep -rn jakarta.persistence domain application` no encuentra nada.

Pista: *`Adventurer.gainExperience` rechaza los puntos que no sean positivos, y un aventurero recién registrado tiene 0.*

Práctica ▪ solución en la pág. 275

Ejercicio 8.4: Revisar el árbol: ¿ha tocado alguien el núcleo? ★★★☆☆

Es el ejercicio de revisión de la verificación de estructura 4, en tres partes. **Parte A, en tu proyecto** (necesita el hito H4 con su *commit*):

1. Ejecuta `git diff --stat HEAD~1 HEAD -- domain application`.
2. Ejecuta `./gradlew test`, que incluye `ArchitectureTest` con la regla de JPA.
3. Explica qué demuestran estas dos comprobaciones y qué *no* demuestran.

Parte B, un cambio ajeno. Un compañero propone esta rama sin tocar nada más. Son cuatro cambios, cada uno en el archivo que indica el comentario (en los dos últimos, la línea de `import` va acompañada de un uso real):

Diff de la rama (fragmentos)

```
// domain/.../domain/Quest.java
+import jakarta.persistence.Entity;
// application/build.gradle  (dentro de dependencies)
+    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
// adapters/.../adapter/in/web/QuestController.java
+import com.guildboard.adapter.out.persistence.QuestJpaRepository;
+private final QuestJpaRepository repository;
// adapters/.../adapter/in/web/AdventurerController.java
+import jakarta.persistence.EntityManager;
+private EntityManager em;
```

Para cada uno de los cuatro cambios, responde: ¿qué herramienta la detecta (el compilador, una prueba de `ArchitectureTest` (di cuál) o solo la revisión humana)?, ¿qué pregunta de la lista de comprobación rompe? y ¿la aceptarías? Por último, **Parte C**: ¿qué módulos ven JPA al compilar y por qué `bootstrap` lo declara por su cuenta? Está bien si: la parte A dice que el diff está vacío y que `./gradlew test` pasa, y distingue «el núcleo no cambió» de «el diseño es bueno» o «el comportamiento es el mismo»; la parte B tiene cuatro filas con herramienta, pregunta y decisión; reconoces que el cuarto cambio compila y solo la caza la prueba `adapterInDoesNotUseJpa`; y la parte C nombra a `adapters` y `bootstrap`.

Pista: *recuerda lo que no ve cada herramienta: el compilador solo conoce lo que declara el `build.gradle` del módulo, y ArchUnit solo ve el bytecode de las clases, no lo que declara un `build.gradle`.*

Lectura de código ▪ solución en la pág. 277

Ejercicio 8.5: Hito H4: GuildBoard con PostgreSQL ★★★★☆

Parte del proyecto tal como lo dejó el capítulo 7 (H3, parte 2). Este ejercicio es autocontenido: no necesitas haber hecho los ejercicios 8.1 y 8.3, porque los pasos 2 y 3 los incluyen. Su solución deja el estado final de todo el capítulo.

1. Añade las dependencias de las secciones 8.4 y 8.6 a `adapters/build.gradle` y `bootstrap/build.gradle`.
2. Crea `compose.yaml` en la raíz (sección 8.3) y los ocho archivos de `adapter/out/persistence`: entidad, repositorio de Spring Data, mapeador y adaptador, de misiones y de aventureros.
3. Crea la migración `V1`. La `V2` del ejercicio 8.2 es opcional.
4. Configura los perfiles: `application.yml`, `application-postgres.yml`, `MemoryPersistenceConfig`, `PostgresPersistenceConfig` y `BeanConfig` sin los repositorios.
5. Arranca sin perfil y haz el escenario del capítulo 6. Arranca con el perfil `postgres` y repítelo; comprueba con `psql` las filas; reinicia con el perfil y comprueba que los datos siguen.
6. Añade a `ArchitectureTest`, en `bootstrap`, la prueba `adapterInDoesNotUseJpa` de la sección 8.8.
7. Ejecuta `./gradlew build`, que incluye las pruebas de arquitectura.
8. Haz el *commit* `H4: adaptador PostgreSQL con JPA y Flyway` y comprueba con

`git diff --stat HEAD~1 HEAD -- domain application` que no cambió el núcleo.

Está bien si: sin perfil, la aplicación arranca sin Docker y sin líneas de Hikari ni de Flyway; con el perfil `postgres`, Flyway aplica las migraciones y los datos sobreviven a un reinicio; las respuestas de la API son las del capítulo 6 en los dos modos; `./gradlew build` termina con `BUILD SUCCESSFUL` con `adapterInDoesNotUseJpa` entre las pruebas; el diff de `domain` y `application` está vacío; y tu árbol coincide con el de la verificación de estructura 4.

Pista: *si el modo memoria falla diciendo que no encuentra un `DataSource`, repasa los cuatro nombres de `spring.autoconfigure.exclude`; si el modo `postgres` no encuentra el repositorio de Spring Data, repasa `@EnableJpaRepositories`.*

Proyecto ▪ solución en la pág. 278

8.11 Resumen

Resumen

- Cambiar de adaptador es añadir un adaptador de salida y decidir cuál se usa en la raíz de composición. Con los puertos bien puestos, `domain` y `application` no cambian; se comprueba con `git diff --stat`, que debe salir vacío para esas carpetas.
- Docker Compose describe servicios en un `compose.yaml`: `docker compose up -d --wait` los arranca y espera a su *healthcheck*, `exec postgres psql...` abre el cliente dentro del contenedor, `down` conserva los datos y `down -v` los borra. En la imagen de PostgreSQL 18 el volumen se monta en `/var/lib/postgresql`.
- La entidad JPA no es el dominio: es otra clase, en `adapter/out/persistence`, con mapeo a mano. Anotar el dominio no compila (no tiene JPA) y obligaría a abrir sus invariantes. Como el dominio no tiene fábrica de reconstrucción, el mapeador repite las transiciones públicas (`publish`, `accept`, `complete`); la alternativa es añadirla al dominio.
- El adaptador implementa el puerto, traduce con el mapeador y delega en un repositorio de Spring Data. `@Transactional` va en él, no en los casos de uso, que no pueden usar Spring. Cada método es una transacción; un caso de uso no, y las dos carreras del capítulo 5 siguen sin resolverse.
- Flyway versiona el esquema con `V<n>__descripcion.sql`, guarda lo aplicado y su suma de comprobación en `flyway_schema_history`, y no se edita nunca una migración aplicada: se añade otra. Hibernate queda en `ddl-auto: none`. `V1` incluye `expires_at` y no lleva clave foránea entre misiones y aventureros.
- El perfil por defecto es memoria y excluye cuatro autoconfiguraciones de datos (`DataSourceAutoConfiguration`, `HibernateJpaAutoConfiguration`, `DataJpaRepositoriesAutoConfiguration`, `FlywayAutoConfiguration`, con nombres de paquete de Boot 4). `application-postgres.yml` vacía la lista y da la conexión; `MemoryPersistenceConfig` (`"!postgres"`) y `PostgresPersistenceConfig` (`"postgres"`) registran los adaptadores.
- La verificación de estructura 4 fija el árbol T3. Con JPA se mantiene un solo módulo `adapters`, con condiciones comprobables para cambiar: un `import` de JPA en `adapter/in`, una segunda dependencia pesada o un segundo equipo.

Soluciones de los ejercicios

Solución del ejercicio 8.1

enunciado en la pág. 268

El archivo. El mismo de la sección 8.3:

compose.yaml

```yaml
services:
  postgres:
    image: postgres:18
    environment:
      POSTGRES_DB: guildboard
      POSTGRES_USER: guildboard
      POSTGRES_PASSWORD: guildboard
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U guildboard -d guildboard"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  postgres-data:
```

Pasos 1 a 3. `docker compose version` imprime una versión (`v5.6.0` cuando escribí el libro). `docker compose config` muestra el archivo normalizado: si falta una sangría o hay un tabulador, aquí falla con la línea. Después de `up -d --wait`, el servicio debe estar sano:

```
$ docker compose up -d --wait
$ docker compose ps
NAME                    SERVICE    STATUS
guildboard-postgres-1   postgres   Up 20 seconds (healthy)
```

(Salida recortada; el prefijo del nombre es el de la carpeta del proyecto.)
Paso 4. Una sesión completa:

```
$ docker compose exec postgres psql -U guildboard -d guildboard
guildboard=# select current_user, current_database();
 current_user | current_database
--------------+------------------
 guildboard   | guildboard
(1 row)

guildboard=# create table prueba (n integer);
CREATE TABLE
guildboard=# insert into prueba values (42);
INSERT 0 1
guildboard=# select * from prueba;
  n
----
 42
(1 row)

guildboard=# \q
```

Pasos 5 y 6. Tras `docker compose down` y un nuevo `up -d --wait`, la consulta vuelve a devolver el 42: `down` destruye el contenedor y la red, pero el volumen `postgres-data` sigue ahí y el contenedor nuevo lo monta. Tras `docker compose down -v`, el volumen se borra con todo lo que había; el contenedor siguiente arranca con un directorio de datos vacío, la imagen ejecuta su inicialización (crea la base y el usuario con las variables de `environment`) y `\dt` responde `Did not find any relations.` La tabla `prueba` ha desaparecido, y con ella cualquier cosa que hubieras guardado: `-v` es la forma de empezar de cero a propósito. **Errores típicos:** escribir el puerto sin comillas o con la parte de la izquierda ocupada (el error dice que el puerto ya está asignado: cámbialo); montar el volumen en `/var/lib/postgresql/data`, que era la ruta de las versiones anteriores (con la imagen 18 los datos no acaban en tu volumen con nombre y desaparecen al recrear el contenedor, o la imagen avisa del montaje); ejecutar los comandos desde otra carpeta (Compose dice que no encuentra ningún archivo de configuración); usar `docker-compose` con guion; y lanzar `down -v` creyendo que equivale a `down`.

Solución del ejercicio 8.2

enunciado en la pág. 268

Paso 1. Tras el primer arranque con el perfil `postgres`:

```
$ docker compose exec postgres psql -U guildboard -d guildboard \
    -c "select installed_rank, version, description, success
        from flyway_schema_history"
 installed_rank | version |       description        | success
----------------+---------+--------------------------+---------
              1 | 1       | create guildboard tables | t
```

Flyway guarda el nombre de la migración sin los guiones bajos.
Paso 2. La migración nueva:

bootstrap/.../db/migration/V2__index_quests_by_adventurer.sql

```
-- V2: acelera la cuenta de misiones aceptadas por aventurero.
CREATE INDEX idx_quests_adventurer_status
    ON quests (adventurer_id, status);
```

Paso 3. Al arrancar, Flyway ve que la versión 2 no está en su historial y la aplica; la versión 1, ya aplicada y con la misma suma de comprobación, no se toca. El historial tiene ahora dos filas y `\d quests` lista el índice (y el que PostgreSQL crea solo para la clave primaria):

```
$ docker compose exec postgres psql -U guildboard -d guildboard \
    -c "\d quests"
...
Indexes:
    "quests_pkey" PRIMARY KEY, btree (id)
    "idx_quests_adventurer_status" btree (adventurer_id, status)
```

(Probé el SQL de `V1` y de `V2` en PostgreSQL 18.6 y esa es su salida real; el resto, de Flyway, no lo ejecuté.)

Paso 4. Cambiar `V1`, aunque sea con un comentario, altera su suma de comprobación. En el arranque, Flyway compara la de la base con la del archivo, y la aplicación no arranca, con un error cuyo mensaje contiene algo como `Migration checksum mismatch for migration version 1`. Con `git restore` sobre el archivo, la aplicación arranca de nuevo.

Paso 5. `V1` es historia: otras bases (la de un compañero, la de producción) ya la aplicaron tal como estaba, y la suma de comprobación existe para que todas partan del mismo esquema. Cualquier cambio de esquema es una migración nueva, con el número siguiente; si te equivocaste en una migración que aún no ha salido de tu máquina, puedes borrar la base (`down -v`) y rehacer, pero no editarla con la base ya migrada.

Errores típicos: nombrar el archivo con un solo guion bajo (`V2_index...`) o con una `v` minúscula: Flyway no lo reconoce como migración y, por defecto, lo ignora sin avisar; repetir el número de versión de otro archivo (Flyway se queja de dos migraciones con la misma versión); guardar el archivo fuera de `db/migration`; y olvidar el punto y coma o escribir el nombre de una columna que no existe (el error de PostgreSQL sale en el arranque de la aplicación).

Solución del ejercicio 8.3 enunciado en la pág. 269

adapter/out/persistence/AdventurerEntity.java

```java
package com.guildboard.adapter.out.persistence;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "adventurers")
@Getter
@AllArgsConstructor
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class AdventurerEntity {

    @Id
    @Column(name = "id")
    private String id;

    @Column(name = "name", nullable = false)
    private String name;

    @Column(name = "adventurer_rank", nullable = false)
    private String rank;

    @Column(name = "experience", nullable = false)
    private int experience;
}
```

adapter/out/persistence/AdventurerJpaRepository.java

```java
package com.guildboard.adapter.out.persistence;

import org.springframework.data.jpa.repository.JpaRepository;

public interface AdventurerJpaRepository
        extends JpaRepository<AdventurerEntity, String> {
}
```

adapter/out/persistence/AdventurerPersistenceMapper.java

```java
package com.guildboard.adapter.out.persistence;

import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;

final class AdventurerPersistenceMapper {

    private AdventurerPersistenceMapper() {
    }

    static AdventurerEntity toEntity(Adventurer adventurer) {
        return new AdventurerEntity(
            adventurer.getId().value(),
            adventurer.getName(),
            adventurer.getRank().name(),
            adventurer.getExperience());
    }

    static Adventurer toDomain(AdventurerEntity entity) {
        Adventurer adventurer = Adventurer.register(
            new AdventurerId(entity.getId()), entity.getName());
        if (entity.getExperience() > 0) {
            adventurer.gainExperience(entity.getExperience());
        }
        return adventurer;
    }
}
```

adapter/out/persistence/JpaAdventurerRepository.java

```java
package com.guildboard.adapter.out.persistence;

import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;
import java.util.Optional;
import lombok.RequiredArgsConstructor;
import org.springframework.transaction.annotation.Transactional;

@RequiredArgsConstructor
@Transactional
public class JpaAdventurerRepository implements AdventurerRepository {

    private final AdventurerJpaRepository jpa;

    @Override
    @Transactional(readOnly = true)
    public Optional<Adventurer> findById(AdventurerId id) {
        return jpa.findById(id.value())
            .map(AdventurerPersistenceMapper::toDomain);
    }

    @Override
    public void save(Adventurer adventurer) {
        jpa.save(AdventurerPersistenceMapper.toEntity(adventurer));
    }
}
```

Por qué funciona. Es el mismo patrón que el de misiones. El rango se guarda (la columna `adventurer_rank` permite consultarlo con SQL), pero la fuente de verdad al reconstruir es la experiencia: `gainExperience` recalcula el rango a partir de ella con los umbrales del dominio (100 y 300), de modo que el aventurero sale con el mismo rango sin que el mapeador conozca esos umbrales. Registrar y guardar usan el mismo `save`: la primera vez la entidad no existe y se inserta; las siguientes se fusiona y se actualiza. `PostgresPersistenceConfig` ya registra `JpaAdventurerRepository` (sección 8.7); no hay nada más que enlazar.
Errores típicos: llamar a `gainExperience(0)` para un aventurero nuevo (el dominio lanza `IllegalArgumentException` porque los puntos deben ser positivos; de ahí el `if`); olvidar que la columna es `adventurer_rank` y dejar `rank` sin `@Column(name=...)` (Hibernate buscaría una columna `rank`); declarar la clase de la entidad sin `public` (la configuración de `bootstrap` no podría nombrarla); y copiar el rango de la entidad al dominio en lugar de reconstruirlo, con lo que se necesitaría una forma de fijar el rango que el dominio no tiene.

Solución del ejercicio 8.4

enunciado en la pág. 269

Parte A. El primer comando no imprime nada: ninguna línea de `domain` ni de `application` cambió entre el *commit* anterior y el del hito. `./gradlew test` termina con `BUILD SUCCESSFUL`, con las cinco pruebas de `ArchitectureTest` en verde. Qué demuestra: que cambiar el adaptador de persistencia no obligó a tocar ni los casos de uso ni las entidades de negocio, y que la regla de dependencia sigue cumpliéndose (las dos primeras pruebas), que `adapter/in` y `adapter/out` no se usan entre sí (las dos siguientes) y que `adapter/in` no usa JPA (la quinta). Qué *no* demuestra: que el comportamiento sea el mismo (eso se

comprueba con el escenario de `curl` y con los datos tras un reinicio); que el diseño de los adaptadores sea bueno (un adaptador puede estar mal hecho y no tocar el núcleo); y que las dependencias del `build.gradle` sean las debidas (ArchUnit lee bytecode, no ese archivo).
Parte B.

Línea	Qué la detecta	Pregunta	¿Aceptar?
`@Entity` en `Quest`	El compilador (`domain` no tiene `jakarta.persistence`); también, si se diera JPA a `domain`, la regla de `domain` de `ArchitectureTest`.	1 (`./gradlew test` pasa: `domain` sin frameworks).	No: la entidad es otra clase.
JPA en `application/build.gradle`	Ninguna herramienta: el compilador deja de proteger a `application` y ArchUnit solo ve bytecode, y aún no hay ninguna clase que use JPA. Solo la revisión del `build.gradle`.	1 (`application` depende solo de `domain`), aunque aquí ninguna prueba salta.	No: abre la puerta a que un servicio use JPA o transacciones.
Campo de tipo `QuestJpaRepository` (con su `import` de `adapter.out.persistence`) en un controlador	La prueba `adapterInDoesNotDependOnAdapterOut`. Compila, porque están en el mismo módulo.	2 (cada adaptador en su lado).	No: el controlador se saltaría el caso de uso y el puerto.
Campo `EntityManager` (con su `import`) en un controlador	La prueba `adapterInDoesNotUseJpa`: compila, y solo esa regla mira `jakarta.persistence` en `adapter/in`.	1 (la prueba deja de pasar), y la decisión de mantener un solo `adapters`.	No, y activa la condición (1) de la sección 8.8: se reabre la decisión.

La cuarta fila es la importante: es la debilidad que aceptamos al mantener un solo módulo `adapters`, y por eso la vigila una regla propia. Si la prueba falla una vez, la respuesta razonable es corregir el código; si falla de forma repetida, pasar a la opción C (apartar `adapters-persistence`).
Parte C. Ven JPA al compilar `adapters` (lo declara con `implementation`) y `bootstrap` (lo declara por su cuenta). `domain` y `application` no. `bootstrap` lo repite porque el `implementation` de `adapters` oculta JPA a quien dependa de él al compilar, y `PostgresPersistenceConfig` necesita nombrar `@EnableJpaRepositories`, `@EntityScan` y el tipo `QuestJpaRepository`, que extiende un tipo de Spring Data. Es la diferencia entre `api` e `implementation` del capítulo 7 en un caso real.
Errores típicos: dar por hecho que «el diff de domain y application está vacío» prueba que todo está bien; olvidar que ArchUnit no ve las dependencias del `build.gradle`, solo las que dejan rastro en el bytecode; y contestar «el compilador» en la segunda fila, cuando el compilador solo falla si hay un `import` que no resuelve.

Solución del ejercicio 8.5

enunciado en la pág. 270

Solución de referencia. Los archivos, en el orden en que los necesitas. Esta solución deja el estado final del capítulo, con todos los archivos completos; solo la `V2` opcional se remite al ejercicio 8.2.

Base de datos.

compose.yaml

```yaml
services:
  postgres:
    image: postgres:18
    environment:
      POSTGRES_DB: guildboard
      POSTGRES_USER: guildboard
      POSTGRES_PASSWORD: guildboard
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U guildboard -d guildboard"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  postgres-data:
```

Adaptador de aventureros (mismo paquete que el de misiones):

adapter/out/persistence/AdventurerEntity.java

```java
package com.guildboard.adapter.out.persistence;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "adventurers")
@Getter
@AllArgsConstructor
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class AdventurerEntity {

    @Id
    @Column(name = "id")
    private String id;

    @Column(name = "name", nullable = false)
    private String name;

    @Column(name = "adventurer_rank", nullable = false)
    private String rank;

    @Column(name = "experience", nullable = false)
    private int experience;
}
```

adapter/out/persistence/AdventurerJpaRepository.java

```java
package com.guildboard.adapter.out.persistence;

import org.springframework.data.jpa.repository.JpaRepository;

public interface AdventurerJpaRepository
        extends JpaRepository<AdventurerEntity, String> {
}
```

adapter/out/persistence/AdventurerPersistenceMapper.java

```java
package com.guildboard.adapter.out.persistence;

import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;

final class AdventurerPersistenceMapper {

    private AdventurerPersistenceMapper() {
    }

    static AdventurerEntity toEntity(Adventurer adventurer) {
        return new AdventurerEntity(
            adventurer.getId().value(),
            adventurer.getName(),
            adventurer.getRank().name(),
            adventurer.getExperience());
    }

    static Adventurer toDomain(AdventurerEntity entity) {
        Adventurer adventurer = Adventurer.register(
            new AdventurerId(entity.getId()), entity.getName());
        if (entity.getExperience() > 0) {
            adventurer.gainExperience(entity.getExperience());
        }
        return adventurer;
    }
}
```

adapter/out/persistence/JpaAdventurerRepository.java

```java
package com.guildboard.adapter.out.persistence;

import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.domain.Adventurer;
import com.guildboard.domain.AdventurerId;
import java.util.Optional;
import lombok.RequiredArgsConstructor;
import org.springframework.transaction.annotation.Transactional;

@RequiredArgsConstructor
@Transactional
public class JpaAdventurerRepository implements AdventurerRepository {

    private final AdventurerJpaRepository jpa;

    @Override
    @Transactional(readOnly = true)
    public Optional<Adventurer> findById(AdventurerId id) {
        return jpa.findById(id.value())
            .map(AdventurerPersistenceMapper::toDomain);
    }

    @Override
    public void save(Adventurer adventurer) {
        jpa.save(AdventurerPersistenceMapper.toEntity(adventurer));
    }
}
```

Gradle.

adapters/build.gradle

```
dependencies {
    implementation project(':application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencies {
    implementation project(':adapters')
    implementation project(':application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    runtimeOnly 'org.postgresql:postgresql'
    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

Adaptador de misiones (`adapters`, paquete `com.guildboard.adapter.out.persistence`): los mismos cuatro archivos de las secciones 8.4 y 8.5, completos aquí.

adapter/out/persistence/QuestEntity.java

```java
package com.guildboard.adapter.out.persistence;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.time.Instant;
import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "quests")
@Getter
@AllArgsConstructor
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class QuestEntity {

    @Id
    @Column(name = "id")
    private String id;

    @Column(name = "title", nullable = false)
    private String title;

    @Column(name = "reward_gold", nullable = false)
    private int rewardGold;

    @Column(name = "required_rank", nullable = false)
    private String requiredRank;

    @Column(name = "status", nullable = false)
    private String status;

    @Column(name = "adventurer_id")
    private String adventurerId;

    @Column(name = "expires_at")
    private Instant expiresAt;
}
```

adapter/out/persistence/QuestJpaRepository.java

```java
package com.guildboard.adapter.out.persistence;

import org.springframework.data.jpa.repository.JpaRepository;

public interface QuestJpaRepository
        extends JpaRepository<QuestEntity, String> {

    long countByAdventurerIdAndStatus(String adventurerId, String status);
}
```

adapter/out/persistence/QuestPersistenceMapper.java (1/2)

```java
package com.guildboard.adapter.out.persistence;

import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import com.guildboard.domain.QuestStatus;
import com.guildboard.domain.Rank;
import com.guildboard.domain.Reward;

final class QuestPersistenceMapper {

    private QuestPersistenceMapper() {
    }

    static QuestEntity toEntity(Quest quest) {
        return new QuestEntity(
            quest.getId().value(),
            quest.getTitle(),
            quest.getReward().gold(),
            quest.getRequiredRank().name(),
            quest.getStatus().name(),
            quest.getAssignedTo().map(AdventurerId::value).orElse(null),
            quest.getExpiresAt().orElse(null));
    }
```

Continuación del listado anterior:

adapter/out/persistence/QuestPersistenceMapper.java (2/2)

```
    static Quest toDomain(QuestEntity entity) {
        Quest quest = Quest.draft(
            new QuestId(entity.getId()),
            entity.getTitle(),
            new Reward(entity.getRewardGold()),
            Rank.valueOf(entity.getRequiredRank()),
            entity.getExpiresAt());
        QuestStatus status = QuestStatus.valueOf(entity.getStatus());
        switch (status) {
            case DRAFT -> { }
            case PUBLISHED -> quest.publish();
            case ACCEPTED -> restoreAccepted(quest, entity);
            case COMPLETED -> {
                restoreAccepted(quest, entity);
                quest.complete();
            }
        }
        return quest;
    }

    private static void restoreAccepted(Quest quest, QuestEntity entity) {
        if (entity.getAdventurerId() == null) {
            throw new IllegalStateException("quest " + entity.getId()
                + " is " + entity.getStatus() + " without adventurer_id");
        }
        quest.publish();
        quest.accept(new AdventurerId(entity.getAdventurerId()),
                     Rank.VETERAN, 0);
    }
}
```

adapter/out/persistence/JpaQuestRepository.java

```java
package com.guildboard.adapter.out.persistence;

import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.domain.AdventurerId;
import com.guildboard.domain.Quest;
import com.guildboard.domain.QuestId;
import com.guildboard.domain.QuestStatus;
import java.util.Optional;
import lombok.RequiredArgsConstructor;
import org.springframework.transaction.annotation.Transactional;

@RequiredArgsConstructor
@Transactional
public class JpaQuestRepository implements QuestRepository {

    private final QuestJpaRepository jpa;

    @Override
    @Transactional(readOnly = true)
    public Optional<Quest> findById(QuestId id) {
        return jpa.findById(id.value())
            .map(QuestPersistenceMapper::toDomain);
    }

    @Override
    public void save(Quest quest) {
        jpa.save(QuestPersistenceMapper.toEntity(quest));
    }

    @Override
    @Transactional(readOnly = true)
    public int countActiveBy(AdventurerId adventurerId) {
        return Math.toIntExact(jpa.countByAdventurerIdAndStatus(
            adventurerId.value(), QuestStatus.ACCEPTED.name()));
    }
}
```

Migración y recursos.

bootstrap/.../db/migration/V1__create_guildboard_tables.sql

```sql
-- V1: esquema inicial de GuildBoard.
-- Los ids son texto (los genera la aplicaci^^c3^^b3n). Rangos y estados se
-- guardan con el nombre de la constante del dominio.

CREATE TABLE adventurers (
    id              VARCHAR(64)  PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    adventurer_rank VARCHAR(20)  NOT NULL,
    experience      INTEGER      NOT NULL
);

CREATE TABLE quests (
    id             VARCHAR(64)  PRIMARY KEY,
    title          VARCHAR(255) NOT NULL,
    reward_gold    INTEGER      NOT NULL,
    required_rank  VARCHAR(20)  NOT NULL,
    status         VARCHAR(20)  NOT NULL,
    adventurer_id  VARCHAR(64),
    expires_at     TIMESTAMP WITH TIME ZONE
);
```

bootstrap/src/main/resources/application.yml

```yaml
spring:
  application:
    name: guildboard
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration

server:
  port: 8080
```

bootstrap/src/main/resources/application-postgres.yml

```yaml
spring:
  autoconfigure:
    exclude: []
  datasource:
    url: jdbc:postgresql://localhost:5432/guildboard
    username: guildboard
    password: guildboard
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: "none"
```

Configuración (`bootstrap`, paquete `com.guildboard.config`).

config/MemoryPersistenceConfig.java

```java
package com.guildboard.config;

import com.guildboard.adapter.out.memory.InMemoryAdventurerRepository;
import com.guildboard.adapter.out.memory.InMemoryQuestRepository;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.application.port.out.QuestRepository;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

@Configuration
@Profile("!postgres")
public class MemoryPersistenceConfig {

    @Bean
    public QuestRepository questRepository() {
        return new InMemoryQuestRepository();
    }

    @Bean
    public AdventurerRepository adventurerRepository() {
        return new InMemoryAdventurerRepository();
    }
}
```

config/PostgresPersistenceConfig.java

```
package com.guildboard.config;

import com.guildboard.adapter.out.persistence.AdventurerJpaRepository;
import com.guildboard.adapter.out.persistence.JpaAdventurerRepository;
import com.guildboard.adapter.out.persistence.JpaQuestRepository;
import com.guildboard.adapter.out.persistence.QuestEntity;
import com.guildboard.adapter.out.persistence.QuestJpaRepository;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.application.port.out.QuestRepository;
import org.springframework.boot.persistence.autoconfigure.EntityScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;

@Configuration
@Profile("postgres")
@EnableJpaRepositories(basePackageClasses = QuestJpaRepository.class)
@EntityScan(basePackageClasses = QuestEntity.class)
public class PostgresPersistenceConfig {

    @Bean
    public QuestRepository questRepository(QuestJpaRepository jpa) {
        return new JpaQuestRepository(jpa);
    }

    @Bean
    public AdventurerRepository adventurerRepository(
            AdventurerJpaRepository jpa) {
        return new JpaAdventurerRepository(jpa);
    }
}
```

El `BeanConfig` completo va en tres trozos, que se escriben seguidos en el mismo archivo (el segundo y el tercero continúan dentro de la clase):

config/BeanConfig.java (1 de 3)

```java
package com.guildboard.config;

import com.guildboard.adapter.out.notification.LogNotificationSender;
import com.guildboard.application.port.in.AcceptQuestUseCase;
import com.guildboard.application.port.in.CompleteQuestUseCase;
import com.guildboard.application.port.in.GetAdventurerUseCase;
import com.guildboard.application.port.in.GetQuestUseCase;
import com.guildboard.application.port.in.GrantExperienceUseCase;
import com.guildboard.application.port.in.PublishQuestUseCase;
import com.guildboard.application.port.in.RegisterAdventurerUseCase;
import com.guildboard.application.port.out.AdventurerRepository;
import com.guildboard.application.port.out.NotificationSender;
import com.guildboard.application.port.out.QuestRepository;
import com.guildboard.application.service.AcceptQuestService;
import com.guildboard.application.service.CompleteQuestService;
import com.guildboard.application.service.GetAdventurerService;
import com.guildboard.application.service.GetQuestService;
import com.guildboard.application.service.GrantExperienceService;
import com.guildboard.application.service.PublishQuestService;
import com.guildboard.application.service.RegisterAdventurerService;
import java.time.Clock;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public Clock clock() {
        return Clock.systemUTC();
    }

    @Bean
    public NotificationSender notificationSender() {
        return new LogNotificationSender();
    }
```

config/BeanConfig.java (2 de 3)

```
    @Bean
    public PublishQuestUseCase publishQuestUseCase(
            QuestRepository questRepository, Clock clock) {
        return new PublishQuestService(questRepository, clock);
    }

    @Bean
    public GetQuestUseCase getQuestUseCase(
            QuestRepository questRepository) {
        return new GetQuestService(questRepository);
    }

    @Bean
    public RegisterAdventurerUseCase registerAdventurerUseCase(
            AdventurerRepository adventurerRepository) {
        return new RegisterAdventurerService(adventurerRepository);
    }

    @Bean
    public GetAdventurerUseCase getAdventurerUseCase(
            AdventurerRepository adventurerRepository) {
        return new GetAdventurerService(adventurerRepository);
    }
```

config/BeanConfig.java (3 de 3)

```
    @Bean
    public GrantExperienceUseCase grantExperienceUseCase(
            AdventurerRepository adventurerRepository) {
        return new GrantExperienceService(adventurerRepository);
    }

    @Bean
    public AcceptQuestUseCase acceptQuestUseCase(
            QuestRepository questRepository,
            AdventurerRepository adventurerRepository, Clock clock) {
        return new AcceptQuestService(questRepository,
                                      adventurerRepository, clock);
    }

    @Bean
    public CompleteQuestUseCase completeQuestUseCase(
            QuestRepository questRepository,
            GrantExperienceUseCase grantExperienceUseCase,
            NotificationSender notificationSender) {
        return new CompleteQuestService(questRepository,
                                        grantExperienceUseCase,
                                        notificationSender);
    }
}
```

Pruebas de arquitectura. `ArchitectureTest` gana una prueba, la quinta; el resto del archivo es el del capítulo 7:

bootstrap/src/test/java/com/guildboard/ArchitectureTest.java

```java
package com.guildboard;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;

import com.tngtech.archunit.core.domain.JavaClasses;
import com.tngtech.archunit.core.importer.ClassFileImporter;
import com.tngtech.archunit.core.importer.ImportOption;
import org.junit.jupiter.api.Test;

public class ArchitectureTest {

    private static final String BASE_PACKAGE = "com.guildboard";

    private static final JavaClasses CLASSES = new ClassFileImporter()
        .withImportOption(new ImportOption.DoNotIncludeTests())
        .importPackages(BASE_PACKAGE);

    @Test
    void domainDoesNotDependOnOuterLayers() {
        noClasses().that().resideInAPackage("..domain..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "..application..", "..adapter..", "..config..",
                "org.springframework..", "jakarta..")
            .check(CLASSES);
    }

    @Test
    void applicationOnlyDependsOnDomain() {
        classes().that().resideInAPackage("..application..")
            .should().onlyDependOnClassesThat().resideInAnyPackage(
                "..application..", "..domain..", "java..", "lombok..")
            .check(CLASSES);
    }

    @Test
    void adapterInDoesNotDependOnAdapterOut() {
        noClasses().that().resideInAPackage("..adapter.in..")
            .should().dependOnClassesThat().resideInAPackage("..adapter.out..")
            .check(CLASSES);
    }

    @Test
    void adapterOutDoesNotDependOnAdapterIn() {
        noClasses().that().resideInAPackage("..adapter.out..")
            .should().dependOnClassesThat().resideInAPackage("..adapter.in..")
            .check(CLASSES);
    }

    @Test
    void adapterInDoesNotUseJpa() {
        noClasses().that().resideInAPackage("..adapter.in..")
            .should().dependOnClassesThat().resideInAPackage("jakarta.persistence..")
            .check(CLASSES);
    }
}
```

La regla no necesita dependencias nuevas: `bootstrap` ya declara `archunit` desde el capítulo 7 (se introdujo en el capítulo 3). Si `adapter/in` no usa JPA, la selección tiene clases y la prueba pasa; si un controlador usara `EntityManager`, fallaría con el nombre de la clase. **Comprobar.**

```
$ ./gradlew build
BUILD SUCCESSFUL
$ ./gradlew :bootstrap:bootRun
$ docker compose up -d --wait
$ SPRING_PROFILES_ACTIVE=postgres ./gradlew :bootstrap:bootRun
$ git commit -am "H4: adaptador PostgreSQL con JPA y Flyway"
$ git diff --stat HEAD~1 HEAD -- domain application
```

(Los archivos nuevos hay que añadirlos antes con `git add`; `-am` solo recoge los que Git ya conocía.) El primer `bootRun` es el modo memoria; el segundo, el de PostgreSQL. El último comando no imprime nada.

Por qué funciona. Los casos de uso piden un `QuestRepository` y un `AdventurerRepository` por tipo; con el perfil por defecto los entrega `MemoryPersistenceConfig` y con `postgres`, `PostgresPersistenceConfig`. En el modo memoria, la exclusión de las cuatro autoconfiguraciones evita que Boot busque un `DataSource`; en el modo `postgres`, la lista vacía las reactiva, Flyway migra antes de que se use ninguna tabla y los adaptadores JPA traducen en cada llamada entre dominio y entidades. `domain` y `application` no se han editado, y por eso su diff es vacío; las pruebas de `ArchitectureTest` siguen pasando porque `persistence` solo importa dominio y puertos. El `BeanConfig` de arriba es el del capítulo 6 sin los dos repositorios, ahora responsabilidad de las configuraciones por perfil.

Qué cambia para el capítulo siguiente. El comportamiento de la API es el mismo; ahora hay dos formas de ejecutarla y una base de datos que sobrevive. El capítulo 9 parte de este árbol (T3): cortará GuildBoard en dos servicios, cada uno con sus módulos, su configuración y su propia base de datos.

Criterios para valorar otras soluciones. Es válida si: (a) `domain` y `application` no tienen cambios en el *commit*; (b) el modo por defecto arranca sin base de datos; (c) con `postgres`, Flyway crea el esquema, los datos sobreviven al reinicio y la API responde igual; (d) ninguna clase de `domain` o `application` lleva anotaciones de JPA ni de transacciones; (e) la elección del adaptador está en `bootstrap`. *Alternativas válidas:* guardar los enumerados con `@Enumerated(EnumType.STRING)` (la entidad pasa a importar el `enum` del dominio, un acoplamiento aceptable pero que el curso evita); colocar las migraciones en `adapters` (Flyway las encuentra igual, aunque entonces `adapters` lleva SQL); o añadir una fábrica de reconstrucción a `Quest` en lugar de repetir las transiciones, aceptando que el diff de `domain` deja de estar vacío y justificándolo en la revisión.

Errores típicos: dejar los dos métodos de repositorio en `BeanConfig` (dos *beans* del mismo tipo en el modo memoria, y el arranque falla por ambigüedad); copiar de un tutorial los nombres de autoconfiguración de Boot 3 (la exclusión no tiene efecto, esas clases ya no existen, y el modo memoria falla por el `DataSource`); olvidar el starter de Spring Data en `bootstrap` (no compila `@EnableJpaRepositories`); poner el driver como `implementation` en `adapters` (funciona, pero el driver no es cosa de los adaptadores); activar el perfil sin haber levantado la base (error de conexión rechazada); y editar `V1` una vez aplicada.

Parte III

Del hexágono a los microservicios

CAPÍTULO 9

De un monolito a servicios: fronteras y primer corte

Objetivos del capítulo

- Decidir dónde cortar GuildBoard con contextos delimitados, y qué datos viven (y cuáles se copian) a cada lado de la frontera.
- Comprobar en la verificación de estructura 5 el árbol por servicios: carpetas, paquetes `com.guildboard.<servicio>`, nombres de proyecto Gradle únicos y una base de datos por servicio.
- Definir un contrato entre servicios con modelo propio en cada lado y llamar a otro servicio con `RestClient` desde un adaptador de salida.
- Separar Adventurers de Quests: dos aplicaciones, dos bases de datos y un cliente REST entre ellas (hito H5, parte 1).

Hasta ahora todo GuildBoard era un solo programa: un proceso, una base de datos y llamadas a método entre el caso de uso de aceptar una misión y el repositorio de aventureros. Este capítulo rompe eso. Es el salto más grande del curso, y conviene saber desde el principio qué te cuesta:

- **Un retroceso deliberado.** Cuando Adventurers sea otro servicio, `CompleteQuestService` ya no podrá llamar a `GrantExperienceUseCase`: ese caso de uso vive en otro proceso. Entre este capítulo y el capítulo 11 *completar una misión no da experiencia*. La experiencia se probará a mano con un `POST` provisional. El capítulo 11 conectará los servicios con un evento y devolverá la función.
- **Una herramienta adelantada.** `proyecto.md` situaba el cliente REST de Spring (`RestClient`) en el hito H6. Aquí hace falta ya: sin él, `AcceptQuest` no puede preguntar el rango del aventurero. Se adelanta; la resiliencia (tiempos máximos, reintentos, *circuit breaker*) sigue en el capítulo 12.
- **Un problema que no se resuelve.** La carrera entre contar las misiones activas y guardar (capítulo 5) sigue ahí: Quests cuenta y guarda en su propia base, sin saga. No se arregla en este curso; solo se recuerda.

Y como el árbol del repositorio cambia mucho (aparece `services/`, los paquetes cambian de nombre, hay una carpeta `infra/`), el capítulo incluye la **verificación de estructura 5**, con su ejercicio de revisión.

9.1 Qué es un microservicio y qué no

Lewis y Fowler describen el estilo como una aplicación hecha de servicios pequeños, cada uno en su propio proceso, que se comunican con mecanismos ligeros, a menudo una API HTTP; y entre las características que enumeran están la organización alrededor de capacidades de negocio y la

gestión descentralizada de los datos, cada servicio con los suyos [26]. El catálogo de Richardson recoge ese segundo punto como patrón con nombre, «base de datos por servicio» [41, 42].

Dos matices antes de seguir:

- **Es un coste, no una mejora gratuita.** Cada llamada que antes era un método ahora cruza una red que puede fallar, tardar o estar en otra versión; y lo que antes era una transacción de base de datos deja de serlo. Dragoni y sus coautores resumen el origen del estilo y sus problemas abiertos [6], y Newman dedica su libro a decidir cuándo compensa pagar ese precio [32]. Para un proyecto del tamaño de GuildBoard, un monolito bien ordenado sería lo sensato; lo cortamos *para aprender* a cortar.
- **Los servicios no son por sí mismos una arquitectura.** Martin dedica un capítulo entero a esto (el 27, «Services: Great and Small», en [28]); de ese capítulo cito solo el título, el argumento de este párrafo es del curso. Cortar un programa en procesos solo cambia la *forma de llamarse* sus partes: una llamada a método pasa a ser una petición HTTP. Si dentro de cada servicio las reglas de negocio dependen del framework, tienes un monolito enredado repartido por la red. Por eso este capítulo no tira nada de lo anterior: dentro de cada servicio sigue valiendo todo lo de los capítulos 3 a 8.

9.2 Dónde cortar: los contextos de GuildBoard

En el capítulo 2 identificaste tres contextos candidatos (Quests, Adventurers y Treasury) y dijiste que, mientras fuese un monolito, vivían en el mismo programa. Un *contexto delimitado* es la frontera dentro de la cual cada término tiene un único significado [10], y Khononov lo usa como guía para decidir las fronteras de los servicios [25]. Cuatro preguntas ayudan a trazarla:

1. **¿Cambia el significado de una palabra?** Para Quests, «aventurero» es un identificador y, como mucho, un rango; para Adventurers es una ficha con nombre, experiencia y rango calculado.
2. **¿Quién es el dueño del dato?** Un dato tiene un solo dueño y los demás lo *piden*. La experiencia es de Adventurers; el estado de la misión, de Quests.
3. **¿Qué cambia junto?** Los umbrales de rango (100 y 300 de experiencia) cambian con las reglas del gremio de aventureros, no con las de publicación de misiones.
4. **¿Una operación modifica un solo agregado?** Desde el capítulo 5 cada caso de uso modifica como máximo un agregado, así que el corte no parte ninguna transacción que ya existiera.

Con eso, el primer corte deja dos servicios (Treasury llega en el capítulo 10):

	Quests (puerto 8081)	**Adventurers** (puerto 8082)
Es dueño de	Misiones, su ciclo de vida, la caducidad, la regla de rango requerido y el límite de 3 misiones activas.	Aventureros, nombre, experiencia y rango (derivado de la experiencia).
Casos de uso	Publicar, consultar, aceptar y completar misión.	Registrar, consultar y dar experiencia.
Pregunta al otro	El rango de un aventurero, al aceptar.	Nada.
Base de datos	`quests`	`adventurers`

La regla «¿puede este aventurero aceptar esta misión?» no se mueve: sigue en `Quest.accept`, que recibe el rango y el número de misiones activas *como datos*. Lo que cambia es de dónde sale el rango: antes, de `AdventurerRepository`; ahora, de una llamada a otro servicio. Esa decisión del capítulo 5 (que el agregado reciba datos y no repositorios) es lo que hace barato este corte. Y el límite de 3 misiones activas se sigue contando en Quests, porque las misiones son suyas.

Hay datos que, tras el corte, existen en los dos lados. El identificador de aventurero está en Adventurers (donde es su clave) y en Quests (como `assignedTo`); el `Rank` lo define Adventurers (de él derivan los umbrales), pero Quests lo necesita para comparar con el rango que pide la misión. Son copias deliberadas, y qué datos copiar y cuáles no es el ejercicio 9.1.

9.3 Verificación de estructura 5: el árbol por servicios

Nota

Este capítulo cambia la estructura de carpetas: aparece `services/` con un subárbol por servicio, los paquetes y proyectos Gradle cambian de nombre, entra `infra/` y se modifica `compose.yaml`. Comprueba que tu proyecto coincide con el árbol de abajo antes de seguir. Las piezas nuevas (base por servicio, contrato, `RestClient`) se explican en las secciones siguientes; aquí basta con leer los nombres.

9.3.1 El árbol anterior

Así dejó el proyecto el capítulo 8 (árbol T3, un solo contexto, cuatro módulos en la raíz):

Árbol anterior (T3, tras el capítulo 8)

```
guildboard/
|-- .git/, .gitignore, gradlew, gradlew.bat
|-- settings.gradle        include 'domain', 'application', ...
|-- build.gradle           configuraci^^c3^^b3n com^^c3^^ban (subprojects)
|-- compose.yaml           PostgreSQL 18 (base guildboard)
|-- docs/dominio.md
|-- gradle/wrapper/
|-- domain/                Quest, Adventurer, Rank, errores
|-- application/           port/in, port/out, service
|-- adapters/
|   `-- src/main/java/com/guildboard/adapter/
|       |-- in/web/        controladores + dto, mapper, error
|       `-- out/           memory, notification, persistence
`-- bootstrap/
    `-- src/main/
        |-- java/com/guildboard/
        |   |-- GuildBoardApplication.java
        |   `-- config/
        |-- test/java/com/guildboard/
        |   `-- ArchitectureTest.java
        `-- resources/     application*.yml, db/migration (V1, V2)
```

9.3.2 El árbol nuevo

Primero la raíz. `[N]` es nuevo y `[M]` modificado; los cuatro módulos del monolito se mueven a `services/quests/` sin cambiar de forma (sus paquetes sí se renombran):

Árbol nuevo, raíz (T4, tras el capítulo 9)

```
guildboard/
|-- .git/, .gitignore, gradlew, gradlew.bat
|-- settings.gradle        [M] ocho proyectos, nombres ^^c3^^banicos
|-- build.gradle           sin cambios
|-- compose.yaml           [M] monta infra/postgres/init.sql
|-- docs/dominio.md        sin cambios
|-- gradle/wrapper/
|-- infra/                 [N]
|   `-- postgres/
|       `-- init.sql       [N] crea las bases quests y adventurers
`-- services/              [N]
    |-- quests/            los cuatro m^^c3^^b3dulos de la ra^^c3^^adz, movidos
    |   |-- domain/
    |   |-- application/
    |   |-- adapters/
    |   `-- bootstrap/
    `-- adventurers/       [N] con su propio hex^^c3^^a1gono
        |-- domain/
        |-- application/
        |-- adapters/
        `-- bootstrap/
```

Y dentro de cada servicio, el paquete base es `com.guildboard.<servicio>` y debajo de él, las mismas capas de siempre:

Dentro de cada servicio (paquetes)

```
services/quests/                      com.guildboard.quests
|-- domain/       Quest, QuestId, QuestStatus, Reward,
|                 Rank y AdventurerId [copias propias],
|                 DomainError, DomainException
|-- application/  port/in    los casos de uso de misiones
|                 port/out   QuestRepository, NotificationSender,
|                            AdventurerDirectory [N],
|                            AdventurerView [N]
|                 service    cuatro servicios
|-- adapters/     adapter/in/web  QuestController + dto, mapper, error
|                 adapter/out     memory, notification, persistence,
|                                 rest/RestAdventurerDirectory [N]
`-- bootstrap/    QuestsApplication, config/, application*.yml,
                  db/migration (V1 solo con la tabla quests, V2),
                  src/test/.../quests/ArchitectureTest [M]

services/adventurers/                 com.guildboard.adventurers
|-- domain/       Adventurer, AdventurerId, Rank [copias propias],
|                 DomainError (una variante), DomainException
|-- application/  port/in    registrar, consultar, dar experiencia
|                 port/out   AdventurerRepository
|                 service    tres servicios
```

```
|-- adapters/     adapter/in/web  AdventurerController + dto, mapper,
|                                 error
|                 adapter/out     memory, persistence
`-- bootstrap/    AdventurersApplication, config/, application*.yml,
                  db/migration (V1 solo con la tabla adventurers),
                  src/test/.../adventurers/ArchitectureTest [N]
```

9.3.3 Cambios respecto al árbol anterior

- `services/quests/`: los cuatro módulos del monolito, movidos enteros. Sus paquetes pasan de `com.guildboard.domain` a `com.guildboard.quests.domain`, y así con `application`, `adapter` y `config`.
- `services/adventurers/`: servicio nuevo con sus cuatro módulos. Recibe todo lo que solo trataba de aventureros: `Adventurer`, sus casos de uso y servicios, `AdventurerRepository`, el controlador, los DTO y el mapeador, el repositorio en memoria, los cuatro archivos de persistencia y la prueba `AdventurerTest`.
- Cada servicio tiene sus propias copias de `Rank` y `AdventurerId`, su `DomainError` y `DomainException`, su manejador de errores HTTP y sus clases de configuración, reducidas a lo suyo.
- `settings.gradle`: ocho proyectos, cada uno con un nombre único (`quests-domain`, `adventurers-domain`, etc.), cada uno con su carpeta mediante `projectDir` [14, 15]. `build.gradle` de la raíz no cambia.
- `ArchitectureTest`: el de Quests es el del capítulo anterior, movido con el servicio y con `com.guildboard.quests` como único dato; el de Adventurers es una copia suya con `com.guildboard.adventurers`. No hay ningún archivo de comprobación compartido (paso 5).
- `infra/postgres/init.sql`: crea las bases `quests` y `adventurers` en la misma instancia de PostgreSQL; `compose.yaml` lo monta en `docker-entrypoint-initdb.d`.
- No hay ningún módulo `common` ni `shared`: `Rank`, `AdventurerId` y los errores no se comparten entre servicios. Lo que ambos servicios comparten es el *contrato* (la forma del JSON de `GET /adventurers/{id}`, sección 9.5): cada lado escribe su clase para leerlo o producirlo.

9.3.4 Los nombres de proyecto Gradle

En el monolito, los módulos se llamaban `domain`, `application`, `adapters` y `bootstrap`. Con dos servicios habría dos módulos `domain`. Gradle distingue sus proyectos por la *ruta*, así que repetir nombres no es un error de Gradle; pero el nombre es lo que sale en el nombre del *jar*, en las listas del IDE y en muchos mensajes, y ahí no dices de cuál de los dos hablas. Por eso cada proyecto lleva el nombre del servicio como prefijo: `quests-domain`, `quests-application`, `quests-adapters`, `quests-bootstrap`, y los cuatro de `adventurers-`. Se declaran de forma plana (la ruta del proyecto es `:quests-domain`) y se les asigna la carpeta con `projectDir`. La alternativa, `include ':services:quests:domain'`, también crearía proyectos intermedios vacíos (`:services` y `:services:quests`) que el bloque `subprojects` de la raíz configuraría sin necesidad. Todo esto cambia los comandos: ahora es `./gradlew :quests-domain:test`, no `:domain:test`.

9.3.5 Comprobaciones

- [ ] `./gradlew test` pasa (incluye la prueba de arquitectura de cada servicio, con su paquete base; sección 9.7). El cliente HTTP no entra en `application`: lo que entra es el puerto `AdventurerDirectory`; `RestClient` vive en `adapters`.
- [ ] Cada adaptador está en `in` o en `out` según quién lo invoca: el que llama a Adventurers (`adapter/out/rest`) es de salida, igual que el repositorio; el `POST` provisional de experiencia es de entrada.
- [ ] Ningún `build.gradle` de un servicio nombra a un proyecto de otro. Hoy se comprueba leyendo; el capítulo 10 escribe la prueba que lo comprueba.
- [ ] Las carpetas de tu proyecto coinciden con el árbol.

Si algo no coincide con el árbol, corrígelo antes de seguir. El ejercicio 9.4 practica estas comprobaciones sobre un cambio ajeno.

9.4 Una base de datos por servicio

La regla del estilo es que *solo el dueño de un dato lo toca*: Adventurers es el único que lee y escribe la tabla `adventurers`, y Quests el único con `quests` [26, 42]. Si Quests leyera la tabla de aventureros directamente, cualquier cambio de esa tabla rompería a Quests y los dos servicios habrían vuelto a ser uno, solo que repartido. Por eso Quests pregunta por HTTP en lugar de hacer un `select`.

Lo que sí hace falta es que cada servicio tenga su base. En desarrollo no levantaremos dos PostgreSQL: una *instancia* sirve las dos bases, `quests` y `adventurers`, y cada servicio solo conoce la URL de la suya. La separación es lógica (otro nombre de base, otras tablas, otra migración de Flyway), no física; en producción irían en instancias distintas y con un usuario por base. Aquí hay un solo usuario, `guildboard`, porque es una herramienta de desarrollo.

La imagen de PostgreSQL ejecuta al arrancar cualquier archivo `.sql` que encuentre en `/docker-entrypoint-initdb.d`. Crea la carpeta `infra/postgres` en la raíz del proyecto con este archivo:

infra/postgres/init.sql

```
-- Una base de datos por servicio, en la misma instancia (solo
-- desarrollo). Se ejecuta UNA vez, al crear el volumen por primera vez.
CREATE DATABASE quests;
CREATE DATABASE adventurers;
```

y monta el archivo en el contenedor. Este es el `compose.yaml` completo; la única línea nueva es la segunda de `volumes`:

compose.yaml

```
services:
  postgres:
    image: postgres:18
    environment:
      POSTGRES_DB: guildboard
      POSTGRES_USER: guildboard
      POSTGRES_PASSWORD: guildboard
```

```
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql
      - ./infra/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U guildboard -d guildboard"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  postgres-data:
```

La base `guildboard`, que creaba el capítulo 8, sigue existiendo (la crea la variable `POSTGRES_DB`) pero ninguno de los dos servicios la usa.

El script solo se ejecuta una vez

Los scripts de `docker-entrypoint-initdb.d` se ejecutan únicamente cuando el volumen está vacío, es decir, la primera vez. Si ya tenías el volumen `postgres-data` del capítulo 8, las bases nuevas *no se crean* aunque el archivo exista (lo comprobé: añadir una base a `init.sql` y hacer `down` y `up` no la crea; con `down -v` sí). Haz esto una vez y pierdes los datos de prueba del capítulo anterior, que no importan:

```
$ docker compose down -v
$ docker compose up -d --wait
$ docker compose exec postgres psql -U guildboard -d guildboard -Atc \
    "select datname from pg_database \
     where datname in ('quests','adventurers') order by 1"
adventurers
quests
```

9.5 El contrato entre servicios

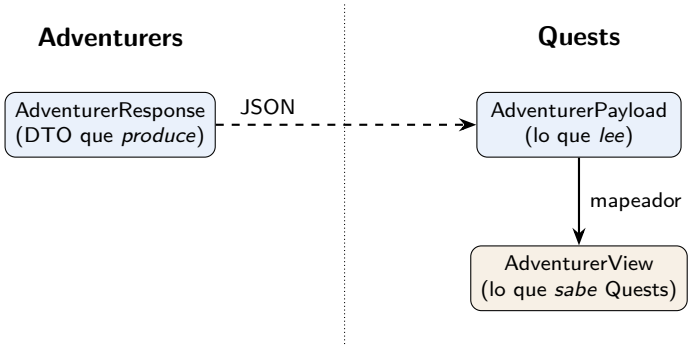
Quests y Adventurers solo comparten una cosa: lo que se dicen por HTTP. A eso se le llama *contrato*: una URL, un método, la forma del JSON y los códigos de estado. El de este capítulo es el de `GET /adventurers/{id}`, que Adventurers ya expone desde el capítulo 5:

`GET /adventurers/ID: las dos respuestas posibles`

```
200 OK
{"id":"7d1c4e52-0b8a-4f3e-9c61-5a2b8e0d7f14","name":"Mara",
 "rank":"ADEPT","experience":150}

404 Not Found       (sin cuerpo)
```

Cada lado escribe *su propio* modelo del contrato, y ninguna clase cruza la frontera:



- En Adventurers, `AdventurerResponse` (capítulo 6) es el DTO que produce el controlador.
- En Quests hay dos clases. `AdventurerPayload` es la forma del JSON tal como llega; vive dentro del adaptador y ninguna otra capa la ve. `AdventurerView` es lo que la aplicación de Quests necesita saber de un aventurero (su id y su rango) y es lo que devuelve el puerto `AdventurerDirectory`. El adaptador traduce de una a otra con un mapeador a mano, como los de los capítulos anteriores.

Esta traducción es una *capa anticorrupción* ligera (Khononov explica este y otros patrones de integración entre contextos [25]): si Adventurers cambia mañana el nombre del campo `rank`, el estropicio queda en un archivo del adaptador y no llega a `AcceptQuestService`. El puerto y la vista son del núcleo de Quests, y son estos dos archivos (en `application/port/out`):

application/port/out/AdventurerView.java

```java
package com.guildboard.quests.application.port.out;

import com.guildboard.quests.domain.AdventurerId;
import com.guildboard.quests.domain.Rank;

// Lo ^^c3^^banico que Quests necesita saber de un aventurero.
public record AdventurerView(AdventurerId id, Rank rank) {
}
```

application/port/out/AdventurerDirectory.java

```java
package com.guildboard.quests.application.port.out;

import com.guildboard.quests.domain.AdventurerId;
import java.util.Optional;

// Puerto de salida: "dame los datos de este aventurero".
public interface AdventurerDirectory {

    Optional<AdventurerView> findById(AdventurerId id);
}
```

Observa que `AdventurerView` usa el `Rank` *de Quests*, no el de Adventurers: es una de las copias que figuran en el árbol de la verificación de estructura 5. Tres reglas de buena vecindad para el contrato:

- **Lee solo lo que necesitas y tolera lo que no conoces.** Quests no lee `name` ni `experience`, y si Adventurers añade campos nuevos, Quests debe ignorarlos

(`@JsonIgnoreProperties(ignoreUnknown = true)` en el *payload*). Añadir campos no rompe a nadie; *renombrar o quitar* uno sí, y exige avisar al otro lado.

- **Un valor que no entiendes es un fallo, no un error del cliente.** Si Adventurers devolviera el rango `LEGEND` y Quests no lo conoce, `Rank.valueOf` lanza `IllegalArgumentException`. Pero el manejador de errores de Quests convierte esa excepción en un `400`, es decir, diría al *cliente* que se equivocó. No se equivocó: es un contrato roto entre servicios. Por eso el mapeador lo captura y lanza `IllegalStateException`, que acaba en un `500`.
- **Un 404 de la pregunta no es un fallo.** «No existe ese aventurero» es una respuesta normal: el adaptador la traduce a `Optional.empty()` y `AcceptQuestService` lanza `AdventurerNotFound`, como antes.

El ejercicio 9.2 te pide escribir el *payload* y el mapeador.

9.6 RestClient como adaptador de salida

`RestClient` es el cliente HTTP síncrono de Spring Framework 7, con una API encadenada («fluida»): construyes una petición, la envías y recibes el cuerpo ya convertido a un objeto. El anterior, `RestTemplate`, está desaconsejado en Spring Framework 7 [61].

☞ Instalar RestClient: no hay nada que instalar

`RestClient` está en `spring-web`, que ya llega con el starter `spring-boot-starter-webmvc` de los módulos `adapters` y `bootstrap`; y la conversión de JSON, con Jackson 3, llega con el mismo starter. No se añade ninguna dependencia a ningún `build.gradle`. Lo verifiqué leyendo Maven Central el 8 de octubre de 2026: Spring Boot 4.1.1 gestiona Spring Framework 7.0.9, el *jar* `spring-web-7.0.9` contiene `org.springframework.web.client.RestClient` y `HttpClientErrorException.NotFound`, y el POM del starter declara como dependencia de compilación al starter de Jackson. (No he ejecutado el código Java del libro, solo comprobado que esas clases y métodos existen.)

Lo básico cabe en tres trozos. Crear el cliente con una URL base:

Crear el cliente

```
RestClient client = RestClient.builder()
    .baseUrl("http://localhost:8082")
    .build();
```

Hacer una petición `GET`, con una variable en la ruta (`{id}` se sustituye y se codifica), y pedir el cuerpo convertido a una clase:

Una petición GET

```
AdventurerPayload payload = client.get()
    .uri("/adventurers/{id}", "7d1c4e52")
    .retrieve()
    .body(AdventurerPayload.class);
```

Y qué ocurre cuando no sale bien. `retrieve()` convierte por defecto las respuestas 4xx y 5xx en excepciones: un 404 es `HttpClientErrorException.NotFound`; otros 4xx, `HttpClientErrorException`; los 5xx, `HttpServerErrorException`; y una conexión que no se

puede abrir, `ResourceAccessException`. Todas son `RestClientException`. En este capítulo solo traducimos el 404:

Un 404 es una respuesta normal

```
try {
    // ... la petici^^c3^^b3n de arriba
} catch (HttpClientErrorException.NotFound e) {
    return Optional.empty();
}
```

Cualquier otra excepción sube hasta Quests y acaba en un `500`: lo puedes ver apagando Adventurers y aceptando una misión. Es incómodo y es correcto *por ahora*. No hay tiempos máximos configurados (si Adventurers no responde, Quests puede esperar), ni reintentos, ni *circuit breaker*: todo eso es el capítulo 12, y sería un error tapar ese hueco aquí con un `catch` genérico.

La clase del adaptador no usa anotaciones de Spring. La URL base la lee la configuración de `bootstrap` con `@Value` (capítulo 4) y se la pasa al constructor. Así, el adaptador se puede crear con `new` en un `@Bean`, como todos los demás. El esqueleto de la clase lo escribirás en el ejercicio 9.5.

9.7 El corte, paso a paso

Con el árbol de la verificación de estructura 5, estos son los pasos, en este orden. Parte de un *commit* limpio del hito H4 (`git status` sin cambios, `./gradlew build` en verde) y con la sección 9.4 ya hecha. Los comandos están pensados para *bash* desde la raíz del proyecto.

9.7.1 Paso 1: mover el monolito a Quests

```
$ mkdir -p services/quests
$ mkdir -p services/adventurers/{domain,application,adapters,bootstrap}
$ git mv domain application adapters bootstrap services/quests/
```

Las cuatro carpetas de Adventurers se crean ya, aunque estén vacías: Gradle se niega a configurar un proyecto cuya carpeta no existe, y el paso 4 declara los ocho.

9.7.2 Paso 2: repartir los archivos de aventureros

Todo lo que solo trata de aventureros se mueve a Adventurers (con `git mv`, para conservar el historial), y lo que los dos servicios necesitan (`AdventurerId` y `Rank`) se *copia*. Las rutas todavía llevan los paquetes antiguos: el cambio de nombre es el paso siguiente, y se aplicará a los dos servicios con el mismo comando.

```
$ Q=services/quests; A=services/adventurers
$ J=src/main/java/com/guildboard
$ mv_adv() { mkdir -p "$A/$(dirname "$1")"; git mv "$Q/$1" "$A/$1"; }
$ cp_adv() { mkdir -p "$A/$(dirname "$1")"; cp "$Q/$1" "$A/$1"; }
$ D=domain/$J/domain
$ mv_adv $D/Adventurer.java
```

```
$ cp_adv $D/AdventurerId.java
$ cp_adv $D/Rank.java
$ mv_adv domain/src/test/java/com/guildboard/domain/AdventurerTest.java
$ P=application/$J/application
$ for n in RegisterAdventurerCommand RegisterAdventurerUseCase \
>          GetAdventurerUseCase GrantExperienceCommand \
>          GrantExperienceUseCase; do mv_adv $P/port/in/$n.java; done
$ mv_adv $P/port/out/AdventurerRepository.java
$ for n in RegisterAdventurerService GetAdventurerService \
>          GrantExperienceService; do mv_adv $P/service/$n.java; done
$ W=adapters/$J/adapter/in/web
$ mv_adv $W/AdventurerController.java
$ mv_adv $W/dto/RegisterAdventurerRequest.java
$ mv_adv $W/dto/AdventurerResponse.java
$ mv_adv $W/mapper/AdventurerWebMapper.java
$ O=adapters/$J/adapter/out
$ mv_adv $O/memory/InMemoryAdventurerRepository.java
$ for n in AdventurerEntity AdventurerJpaRepository \
>          AdventurerPersistenceMapper JpaAdventurerRepository; do
>   mv_adv $O/persistence/$n.java; done
```

Las clases de configuración y la migración `V1` de Quests también hablan de aventureros; se quedan donde están y se reducen en el paso 6. Y `DomainError`, `DomainException` y el manejador de errores HTTP se *escriben de nuevo* para Adventurers en el ejercicio 9.5 (Quests conserva los suyos tal cual, incluida la variante `AdventurerNotFound`, que ahora lanza cuando el directorio no encuentra al aventurero).

9.7.3 Paso 3: renombrar los paquetes

Dos órdenes. La primera mueve los directorios de cada paquete a `com/guildboard/<servicio>/`; usa `mv` y no `git mv` porque en Adventurers hay archivos copiados que Git aún no conoce (al hacer `git add` reconocerá los renombrados por su contenido). La segunda reescribe los `package` e `import`:

```
$ for S in quests adventurers; do
>   for D in services/$S/*/src/*/java/com/guildboard; do
>     mkdir -p "$D/$S"
>     for P in domain application adapter config; do
>       [ -d "$D/$P" ] && mv "$D/$P" "$D/$S/$P"
>     done
>   done
> done
$ PKG='com\.guildboard\.(domain|application|adapter|config)'
$ for S in quests adventurers; do
>   grep -rl 'com\.guildboard' --include='*.java' services/$S |
>     xargs sed -i -E "s/$PKG/com.guildboard.$S.\1/g"
> done
$ grep -rEn "$PKG" services
```

El último `grep` no debe imprimir nada. Falta la clase principal de Quests, que estaba directamente en `com.guildboard`:

```
$ git mv services/quests/bootstrap/$J/GuildBoardApplication.java \
        services/quests/bootstrap/$J/quests/QuestsApplication.java
```

y se reescribe (el paquete nuevo y el nombre de la clase):

quests-bootstrap: QuestsApplication.java

```
1  package com.guildboard.quests;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6  @SpringBootApplication
7  public class QuestsApplication {
8
9      public static void main(String[] args) {
10         SpringApplication.run(QuestsApplication.class, args);
11     }
12 }
```

`@SpringBootApplication` escanea el paquete de su clase y los de abajo, es decir, `com.guildboard.quests.*`: el escaneo de un servicio nunca llega a las clases del otro.

9.7.4 Paso 4: Gradle

La raíz declara los ocho proyectos con nombres únicos y carpetas propias. `build.gradle` de la raíz no cambia.

settings.gradle

```
1  rootProject.name = 'guildboard'
2
3  // Un proyecto Gradle por m^^c3^^b3dulo y servicio, con nombre ^^c3^^banico
4  // (quests-domain, adventurers-domain...) y la carpeta de cada uno.
5  ['quests', 'adventurers'].each { service ->
6      ['domain', 'application', 'adapters', 'bootstrap'].each { module ->
7          def projectName = "${service}-${module}"
8          include projectName
9          project(":${projectName}").projectDir =
10             file("services/${service}/${module}")
11     }
12 }
```

Solo probé este `settings.gradle`, en un montaje Gradle 9.x de prueba (el resto del código está revisado, no ejecutado): `./gradlew projects` lista `:quests-domain`, etc., con sus carpetas. Y descubrí una trampa: Gradle *se niega a configurar un proyecto cuya carpeta no existe*. Por eso el paso 1 creó las cuatro carpetas de Adventurers. Los cuatro `build.gradle` de Quests cambian en una sola cosa: las dependencias entre módulos usan los nombres nuevos. Ya no declaran qué servicio son; `bootstrap` gana la dependencia de ArchUnit, que antes vivía en el `build.gradle` de la raíz del monolito.

services/quests/domain/build.gradle

```
dependencies {
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

services/quests/application/build.gradle

```
dependencies {
    api project(':quests-domain')

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

services/quests/adapters/build.gradle

```
dependencies {
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

services/quests/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencies {
    implementation project(':quests-adapters')
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

9.7.5 Paso 5: un ArchitectureTest por servicio

El `ArchitectureTest` del capítulo anterior vive en el `bootstrap` de la raíz, ahora `services/quests/bootstrap`, y sigue en `src/test/java/com/guildboard/`. Dos cosas lo dejan roto, y las dos son de nombres. Su paquete sigue siendo `com.guildboard`, y debe pasar a `com.guildboard.quests`; y su constante `BASE_PACKAGE` apunta a la raíz del monolito. Es el único dato del que depende la prueba, y es lo único que cambia:

```
$ T=services/quests/bootstrap/src/test/java/com/guildboard
$ git mv $T/ArchitectureTest.java $T/quests/ArchitectureTest.java
$ sed -i -e 's/^package com.guildboard;/package com.guildboard.quests;/' \
        -e 's/"com.guildboard";/"com.guildboard.quests";/' \
        $T/quests/ArchitectureTest.java
```

El archivo queda así (las cinco reglas del capítulo anterior, sin tocar):

quests-bootstrap: src/test/.../quests/ArchitectureTest.java

```java
package com.guildboard.quests;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;

import com.tngtech.archunit.core.domain.JavaClasses;
import com.tngtech.archunit.core.importer.ClassFileImporter;
import com.tngtech.archunit.core.importer.ImportOption;
import org.junit.jupiter.api.Test;

public class ArchitectureTest {

    private static final String BASE_PACKAGE = "com.guildboard.quests";

    private static final JavaClasses CLASSES = new ClassFileImporter()
        .withImportOption(new ImportOption.DoNotIncludeTests())
        .importPackages(BASE_PACKAGE);

    @Test
    void domainDoesNotDependOnOuterLayers() {
        noClasses().that().resideInAPackage("..domain..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "..application..", "..adapter..", "..config..",
                "org.springframework..", "jakarta..")
            .check(CLASSES);
    }

    @Test
    void applicationOnlyDependsOnDomain() {
        classes().that().resideInAPackage("..application..")
            .should().onlyDependOnClassesThat().resideInAnyPackage(
                "..application..", "..domain..", "java..", "lombok..")
            .check(CLASSES);
    }

    @Test
    void adapterInDoesNotDependOnAdapterOut() {
        noClasses().that().resideInAPackage("..adapter.in..")
            .should().dependOnClassesThat().resideInAPackage("..adapter.out..")
            .check(CLASSES);
    }

    @Test
    void adapterOutDoesNotDependOnAdapterIn() {
        noClasses().that().resideInAPackage("..adapter.out..")
            .should().dependOnClassesThat().resideInAPackage("..adapter.in..")
```

```
47              .check(CLASSES);
48      }
49
50      @Test
51      void adapterInDoesNotUseJpa() {
52          noClasses().that().resideInAPackage("..adapter.in..")
53              .should().dependOnClassesThat().resideInAPackage("jakarta.
        ↪ persistence..")
54              .check(CLASSES);
55      }
56  }
```

La prueba se *copia* para Adventurers, no se comparte. Es la misma decisión que con `Rank` y `AdventurerId`: un módulo común de pruebas ataría a los dos servicios a un artefacto compartido por veinte líneas de código, y cada servicio debe poder cambiar sus reglas sin pedir permiso al otro (el ejercicio 9.4 vuelve sobre este criterio). La copia cambia el nombre del paquete y el valor de la constante. Aún no puedes ejecutarla: Adventurers se rellena en el paso 6 y en el ejercicio 9.5. Haz la copia cuando exista su carpeta:

```
$ A=services/adventurers/bootstrap/src/test/java/com/guildboard/adventurers
$ mkdir -p $A
$ sed -e 's/com\.guildboard\.quests/com.guildboard.adventurers/g' \
      $T/quests/ArchitectureTest.java > $A/ArchitectureTest.java
```

Un paquete base mal escrito no pasa en silencio. Si dejaras `"com.guildboard.quest"` en la constante, ArchUnit no encontraría ninguna clase en `..domain..`, y una regla sobre una selección vacía *falla* (ArchUnit lo hace así a propósito [66]). Comprueba que la prueba sigue viendo. Con los pasos hechos hasta aquí, añade al índice (`git add -A`) y comete una infracción deliberada, inventada para el experimento y que el compilador sí acepta: un controlador que usa JPA. El adaptador de entrada declara un campo `jakarta.persistence.EntityManager` (`adapters` tiene `spring-boot-starter-data-jpa`, así que compila) y la regla `adapterInDoesNotUseJpa` lo prohíbe.

```
$ F=services/quests/adapters/$J/quests/adapter/in/web/QuestController.java
$ sed -i '/^public class QuestController/a\    private jakarta.persistence.
    ↪ EntityManager em;' $F
$ ./gradlew :quests-bootstrap:test
> Task :quests-bootstrap:test FAILED
ArchitectureTest > adapterInDoesNotUseJpa() FAILED
    java.lang.AssertionError: Architecture Violation [Priority: MEDIUM] ...
    Rule 'no classes that reside in a package '..adapter.in..' should depend
    on classes that reside in a package 'jakarta.persistence..'' was violated:
    Field <com.guildboard.quests.adapter.in.web.QuestController.em> has type
    <jakarta.persistence.EntityManager> ...
$ git restore $F
```

(Salida aproximada: el texto exacto varía; lo importante es que falle y que nombre la clase.) Con el controlador restaurado, `./gradlew :quests-bootstrap:test` vuelve a pasar. La lección del experimento es la de siempre: tras cualquier cambio de nombres, *provoca una infracción* y comprueba que la prueba la ve.

9.7.6 Paso 6: Quests deja de conocer los aventureros

Quedan siete cosas por hacer en Quests. Primero, el servicio de aceptar, que ahora pregunta al `AdventurerDirectory` (sección 9.5) en lugar de al repositorio de aventureros. Solo cambian los dos puertos de entrada del constructor y las líneas que usan la vista; sigue modificando *solo* `Quest`:

application/service/AcceptQuestService.java

```java
package com.guildboard.quests.application.service;

import com.guildboard.quests.application.port.in.AcceptQuestCommand;
import com.guildboard.quests.application.port.in.AcceptQuestUseCase;
import com.guildboard.quests.application.port.out.AdventurerDirectory;
import com.guildboard.quests.application.port.out.AdventurerView;
import com.guildboard.quests.application.port.out.QuestRepository;
import com.guildboard.quests.domain.DomainError;
import com.guildboard.quests.domain.DomainException;
import com.guildboard.quests.domain.Quest;
import java.time.Clock;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class AcceptQuestService implements AcceptQuestUseCase {

    private final QuestRepository quests;
    private final AdventurerDirectory adventurers;
    private final Clock clock;

    @Override
    public Quest accept(AcceptQuestCommand command) {
        Quest quest = quests.findById(command.questId())
            .orElseThrow(() -> new DomainException(
                new DomainError.QuestNotFound(command.questId())));
        AdventurerView adventurer = adventurers
            .findById(command.adventurerId())
            .orElseThrow(() -> new DomainException(
                new DomainError.AdventurerNotFound(
                    command.adventurerId())));
        quest.requireNotExpired(clock);
        int active = quests.countActiveBy(adventurer.id());
        quest.accept(adventurer.id(), adventurer.rank(), active);
        quests.save(quest);
        return quest;
    }
}
```

Segundo, el servicio de completar, que ya no da experiencia (retroceso declarado al principio del capítulo): pierde la dependencia de `GrantExperienceUseCase`, se queda con el repositorio y el aviso:

application/service/CompleteQuestService.java

```java
package com.guildboard.quests.application.service;

import com.guildboard.quests.application.port.in.CompleteQuestCommand;
```

```
import com.guildboard.quests.application.port.in.CompleteQuestUseCase;
import com.guildboard.quests.application.port.out.NotificationSender;
import com.guildboard.quests.application.port.out.QuestRepository;
import com.guildboard.quests.domain.AdventurerId;
import com.guildboard.quests.domain.DomainError;
import com.guildboard.quests.domain.DomainException;
import com.guildboard.quests.domain.Quest;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class CompleteQuestService implements CompleteQuestUseCase {

    private final QuestRepository quests;
    private final NotificationSender notifications;

    @Override
    public Quest complete(CompleteQuestCommand command) {
        Quest quest = quests.findById(command.questId())
            .orElseThrow(() -> new DomainException(
                new DomainError.QuestNotFound(command.questId())));
        quest.complete();
        AdventurerId assignee = quest.getAssignedTo().orElseThrow(
            () -> new IllegalStateException("accepted quest without "
                + "assignee: " + quest.getId().value()));
        quests.save(quest);
        notifications.send(assignee,
            "quest completed: " + quest.getTitle());
        return quest;
    }
}
```

Tercero, la configuración de Quests en `bootstrap` (paquete `com.guildboard.quests.config`). Los repositorios pierden el de aventureros:

config/MemoryPersistenceConfig.java

```
package com.guildboard.quests.config;

import com.guildboard.quests.adapter.out.memory.InMemoryQuestRepository;
import com.guildboard.quests.application.port.out.QuestRepository;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

@Configuration
@Profile("!postgres")
public class MemoryPersistenceConfig {

    @Bean
    public QuestRepository questRepository() {
        return new InMemoryQuestRepository();
    }
}
```

config/PostgresPersistenceConfig.java

```java
package com.guildboard.quests.config;

import com.guildboard.quests.adapter.out.persistence.JpaQuestRepository;
import com.guildboard.quests.adapter.out.persistence.QuestEntity;
import com.guildboard.quests.adapter.out.persistence.QuestJpaRepository;
import com.guildboard.quests.application.port.out.QuestRepository;
import org.springframework.boot.persistence.autoconfigure.EntityScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;

@Configuration
@Profile("postgres")
@EnableJpaRepositories(basePackageClasses = QuestJpaRepository.class)
@EntityScan(basePackageClasses = QuestEntity.class)
public class PostgresPersistenceConfig {

    @Bean
    public QuestRepository questRepository(QuestJpaRepository jpa) {
        return new JpaQuestRepository(jpa);
    }
}
```

Cuarto, `BeanConfig`: sin los casos de uso de aventureros, con el nuevo bean `AdventurerDirectory` (que lee la URL con `@Value`) y con `CompleteQuest` sin la experiencia. Va en dos trozos seguidos:

config/BeanConfig.java (1 de 2)

```java
package com.guildboard.quests.config;

import com.guildboard.quests.adapter.out.notification.LogNotificationSender;
import com.guildboard.quests.adapter.out.rest.RestAdventurerDirectory;
import com.guildboard.quests.application.port.in.AcceptQuestUseCase;
import com.guildboard.quests.application.port.in.CompleteQuestUseCase;
import com.guildboard.quests.application.port.in.GetQuestUseCase;
import com.guildboard.quests.application.port.in.PublishQuestUseCase;
import com.guildboard.quests.application.port.out.AdventurerDirectory;
import com.guildboard.quests.application.port.out.NotificationSender;
import com.guildboard.quests.application.port.out.QuestRepository;
import com.guildboard.quests.application.service.AcceptQuestService;
import com.guildboard.quests.application.service.CompleteQuestService;
import com.guildboard.quests.application.service.GetQuestService;
import com.guildboard.quests.application.service.PublishQuestService;
import java.time.Clock;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public Clock clock() {
```

```
        return Clock.systemUTC();
    }

    @Bean
    public NotificationSender notificationSender() {
        return new LogNotificationSender();
    }

    @Bean
    public AdventurerDirectory adventurerDirectory(
            @Value("${guildboard.adventurers.base-url}") String baseUrl) {
        return new RestAdventurerDirectory(baseUrl);
    }
```

config/BeanConfig.java (2 de 2)

```
    @Bean
    public PublishQuestUseCase publishQuestUseCase(
            QuestRepository questRepository, Clock clock) {
        return new PublishQuestService(questRepository, clock);
    }

    @Bean
    public GetQuestUseCase getQuestUseCase(
            QuestRepository questRepository) {
        return new GetQuestService(questRepository);
    }

    @Bean
    public AcceptQuestUseCase acceptQuestUseCase(
            QuestRepository questRepository,
            AdventurerDirectory adventurerDirectory, Clock clock) {
        return new AcceptQuestService(questRepository,
                                      adventurerDirectory, clock);
    }

    @Bean
    public CompleteQuestUseCase completeQuestUseCase(
            QuestRepository questRepository,
            NotificationSender notificationSender) {
        return new CompleteQuestService(questRepository,
                                        notificationSender);
    }
}
```

Quinto, la configuración de Quests: puerto 8081, su nombre, la URL de Adventurers y su base. Son los dos `yml` (el segundo solo cambia la URL de la base):

quests-bootstrap: application.yml

```
spring:
  application:
    name: quests
  autoconfigure:
    exclude:
```

```
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.
    ↪ HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.
    ↪ DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration

server:
  port: 8081

guildboard:
  adventurers:
    base-url: http://localhost:8082
```

`quests-bootstrap: application-postgres.yml`

```
spring:
  autoconfigure:
    exclude: []
  datasource:
    url: jdbc:postgresql://localhost:5432/quests
    username: guildboard
    password: guildboard
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: "none"
```

Sexto, la migración. `V1` deja de crear la tabla `adventurers`. Es la migración que se *edita*, y eso no contradice la regla del capítulo 8 («no se edita una migración ya aplicada»): la base `quests` es nueva y `V1` nunca se ha aplicado en ella; la historia de la base `guildboard` queda abandonada con la base. `V2` (el índice) no cambia.

`quests-bootstrap: V1__create_quests_table.sql`

```
-- V1 de Quests: solo la tabla de misiones. La tabla de aventureros es
-- de otro servicio y de otra base de datos; adventurer_id es un texto
-- sin clave for^^c3^^a1nea (no hay nada a lo que apuntar desde aqu^^c3^^ad).

CREATE TABLE quests (
    id              VARCHAR(64)  PRIMARY KEY,
    title           VARCHAR(255) NOT NULL,
    reward_gold     INTEGER      NOT NULL,
    required_rank   VARCHAR(20)  NOT NULL,
    status          VARCHAR(20)  NOT NULL,
    adventurer_id   VARCHAR(64),
    expires_at      TIMESTAMP WITH TIME ZONE
);
```

El archivo se renombra para que el nombre diga lo que hace (`git mv V1__create_guildboard_tables.sql V1__create_quests_table.sql` en `src/main/resources/db/migration`); Flyway solo mira el número y la descripción para ordenarlas. Y séptimo: crea la clase `RestAdventurerDirectory` en `adapter/out/rest` (la implementación del puerto); es parte del ejercicio 9.5, junto con todo Adventurers. Hasta que la escribas, `quests-adapters` y

`quests-bootstrap` no compilan, porque `BeanConfig` la nombra.

9.8 En el proyecto: hito H5 (parte 1)

Hito 5, parte 1: Quests y Adventurers separados

Al terminar el ejercicio 9.5, GuildBoard son dos aplicaciones que se hablan por REST:

- **Quests** (`services/quests`, puerto 8081, base `quests`): publicar, consultar, aceptar y completar misiones. Al aceptar, pregunta el rango a Adventurers con `RestClient`.
- **Adventurers** (`services/adventurers`, puerto 8082, base `adventurers`): registrar, consultar y dar experiencia, con un `POST /adventurers/{id}/experience` *provisional*.

Tres cosas que este hito deja dichas, para que no te sorprendan:

1. **Completar una misión ya no da experiencia** hasta el capítulo 11. Es un retroceso deliberado: el caso de uso que la daba vive ahora en otro proceso. Mientras tanto, la experiencia se da a mano con el `POST` provisional (que desaparecerá cuando llegue el evento).
2. **`RestClient` se adelanta** respecto a `proyecto.md`, que lo situaba en el hito H6: aceptar una misión necesita el rango del aventurero ya. Lo de H6 (resiliencia) sigue en su sitio.
3. **La carrera de `AcceptQuest` sigue**: Quests cuenta las misiones activas y guarda en su base, sin atomicidad entre ambas. No hay saga.

9.8.1 Probar el hito

Sin Docker, en modo memoria, con una terminal por servicio (Adventurers primero, porque Quests lo necesita al aceptar):

```
$ ./gradlew :adventurers-bootstrap:bootRun        # terminal 1
$ ./gradlew :quests-bootstrap:bootRun             # terminal 2
```

En una tercera, registra a Mara en Adventurers (8082) y anota su id; después publica, acepta y completa una misión en Quests (8081):

```
$ curl -s -X POST localhost:8082/adventurers \
    -H "Content-Type: application/json" -d '{"name":"Mara"}'
{"id":"7d1c4e52-...","name":"Mara","rank":"NOVICE","experience":0}
$ MARA=7d1c4e52-...
$ curl -s -X POST localhost:8081/quests \
    -H "Content-Type: application/json" \
    -d '{"title":"Limpiar el s^^c3^^b3tano","reward":150,
         "requiredRank":"NOVICE"}'
{"id":"3f9a...","title":"Limpiar el s^^c3^^b3tano","status":"PUBLISHED",...}
$ Q1=3f9a...
$ curl -s -X POST localhost:8081/quests/$Q1/accept \
    -H "Content-Type: application/json" \
    -d "{\"adventurerId\":\"$MARA\"}"
```

```
$ curl -s -X POST localhost:8081/quests/$Q1/complete
$ curl -s localhost:8082/adventurers/$MARA
{"id":"7d1c4e52-...","name":"Mara","rank":"NOVICE","experience":0}
```

(Salidas resumidas; los ids son los tuyos.) La última respuesta es el retroceso: **experiencia 0 tras completar una misión de 150 de oro**. Ahora comprueba que el rango cruza la frontera. Publica una misión que pida `ADEPT`, intenta aceptarla con Mara, da experiencia a mano y vuelve a intentarlo:

```
$ curl -s -X POST localhost:8081/quests \
    -H "Content-Type: application/json" \
    -d '{"title":"Escoltar la caravana","reward":300,
         "requiredRank":"ADEPT"}'
$ Q2=...
$ curl -s -X POST localhost:8081/quests/$Q2/accept \
    -H "Content-Type: application/json" \
    -d "{\"adventurerId\":\"$MARA\"}"
{"title":"Business rule violated","status":422,
 "detail":"rank NOVICE cannot take a quest that requires ADEPT",
 "code":"RANK_TOO_LOW"}
$ curl -s -X POST localhost:8082/adventurers/$MARA/experience \
    -H "Content-Type: application/json" -d '{"points":150}'
{"id":"7d1c4e52-...","name":"Mara","rank":"ADEPT","experience":150}
$ curl -s -X POST localhost:8081/quests/$Q2/accept \
    -H "Content-Type: application/json" \
    -d "{\"adventurerId\":\"$MARA\"}"
```

El primer intento da `422` y el segundo `200`: Quests no sabe nada de la experiencia, solo pregunta el rango cada vez. Dos comprobaciones más: un aventurero que no existe da `404` con el código `ADVENTURER_NOT_FOUND` (lo lanza Quests cuando el directorio devuelve vacío), y si paras Adventurers (`Ctrl+C`) y intentas aceptar, Quests responde `500`: es el comportamiento previsto hasta el capítulo 12.

Con Docker y el perfil `postgres`, cada servicio guarda en su base (tras el `down -v` de la sección 9.4):

```
$ docker compose up -d --wait
$ SPRING_PROFILES_ACTIVE=postgres \
    ./gradlew :adventurers-bootstrap:bootRun
$ SPRING_PROFILES_ACTIVE=postgres ./gradlew :quests-bootstrap:bootRun
$ docker compose exec postgres psql -U guildboard -d quests -c '\dt'
 Schema |         Name          | Type  |   Owner
--------+-----------------------+-------+------------
 public | flyway_schema_history | table | guildboard
 public | quests                | table | guildboard
```

(Salida aproximada.) En la base `quests` no existe la tabla `adventurers`, y en la base `adventurers` no existe `quests`: eso es «una base por servicio». Cada una tiene su propio `flyway_schema_history`.

Qué cambió y qué no. Haz el *commit* (mensaje `H5a: Quests y Adventurers como servicios`). Con `git diff -M --stat` contra el *commit* anterior, los archivos de dominio y de aplicación de Quests aparecen como *renombrados* con pocas líneas cambiadas (las de `package` e `import`); la lógica no se tocó, solo `AcceptQuestService` y `CompleteQuestService` cambian

de verdad. Eso es lo que cuesta cortar bien: moverlo todo y reescribir poco.

9.9 Ejercicios

Los ejercicios parten del proyecto tal como lo dejan los pasos 1 a 6 de la sección 9.7 (el de proyecto es el que lo termina). Los dos primeros no necesitan el proyecto abierto.

Ejercicio 9.1: Quién es el dueño: seis funcionalidades ★★☆☆☆

Tras el corte, GuildBoard tiene dos servicios: **Quests**, dueño de las misiones, y **Adventurers**, dueño de los aventureros (nombre, experiencia y rango). Para cada una de estas seis funcionalidades, copia y rellena la tabla: a qué servicio pertenece, en una frase *por qué*, y qué datos del otro servicio necesita (y si los pide por REST o los guarda copiados).

1. Publicar una misión con una recompensa de 150 de oro.
2. Registrar a un aventurero nuevo con su nombre.
3. Rechazar que un aventurero de rango `NOVICE` acepte una misión que pide `ADEPT`.
4. Subir a un aventurero de `NOVICE` a `ADEPT` al sumar 100 de experiencia.
5. Impedir que un aventurero tenga más de 3 misiones activas.
6. (Hipotética, no existe en GuildBoard.) Mostrar el *nombre* del aventurero en la lista de misiones aceptadas.

Después contesta: ¿qué dos valores existen en los dos servicios, y por qué esa copia es aceptable? Está bien si: cada fila tiene un servicio y un motivo basado en quién es dueño del dato o de la regla; las funcionalidades 3 y 5 se asignan a Quests y la 4 a Adventurers; para la 3 reconoces que Quests necesita el *rango* (dato de Adventurers, por REST) y el *rango requerido* (propio); para la 5, que la cuenta es de Quests; nombras `AdventurerId` y `Rank` como valores duplicados; y en la 6 ofreces al menos dos caminos con su coste.

Pista: *pregúntate de cada regla: ¿qué dato necesita para decidir y quién es su dueño? Una regla vive donde está el dato que más cambia con ella.*

Lectura de código ▪ solución en la pág. 324

Ejercicio 9.2: El contrato REST: payload y mapeador ★★★☆☆

Quests ya tiene el puerto `AdventurerDirectory` y la vista `AdventurerView` (sección 9.5). Escribe, en el paquete `com.guildboard.quests.adapter.out.rest` (módulo `quests-adapters`), las dos clases que faltan para leer el contrato de `GET /adventurers/{id}`:

1. Mira la respuesta de `GET /adventurers/{id}` que se muestra en la sección 9.5 y anota qué campos trae.
2. `AdventurerPayload`: un *record* público con *solo* los campos que Quests necesita, que ignora los demás.
3. `AdventurerPayloadMapper`: clase `final` con constructor privado y un método estático `AdventurerView toView(AdventurerPayload)`. Si falta el `id` o el `rank`, o el rango no es ninguna constante de `Rank`, lanza `IllegalStateException` con un mensaje que incluya el valor problemático. Nunca `IllegalArgumentException`.
4. Responde en dos frases: Adventurers renombra el campo `rank` a `level`, ¿qué archivo de Quests hay que tocar y qué ve el usuario de Quests mientras tanto?

Está bien si: el *payload* tiene dos campos y una anotación de Jackson para tolerar los desconocidos; `toView(new AdventurerPayload("a-1", "ADEPT"))` devuelve una vista con `Rank.ADEPT`; con `"LEGEND"` lanza `IllegalStateException` con `LEGEND` en el mensaje; con rango `null` también; el mapeador no importa nada de Spring; y la respuesta del paso 4 dice que solo hay que tocar el *payload* (y quizá el mapeador) y que, hasta entonces, aceptar misiones da un `500`.

Pista: *`Rank.valueOf` lanza `IllegalArgumentException` si no existe la constante: captúrala y relánzala como otra cosa, conservando la causa. La anotación es `@JsonIgnoreProperties` de `com.fasterxml.jackson.annotation`.*

Práctica ▪ solución en la pág. 324

Ejercicio 9.3: Dos servicios que se pisan ★★★☆☆

Un compañero ha creado Adventurers copiando la configuración de Quests y sin cambiar nada. Estos son los dos `application.yml` (los `exclude` de autoconfiguración, iguales en ambos, no se muestran) y los dos `application-postgres.yml`:

Configuración copiada (los dos servicios)

```
# quests-bootstrap: application.yml
spring: { application: { name: guildboard } }
server: { port: 8080 }
guildboard: { adventurers: { base-url: "http://localhost:8081" } }

# adventurers-bootstrap: application.yml
spring: { application: { name: guildboard } }
server: { port: 8080 }

# los dos application-postgres.yml: datasource.url
url: jdbc:postgresql://localhost:5432/guildboard
```

Estos son los cuatro síntomas que ha anotado, cada uno con su causa:

1. Arranca Quests y funciona. Arranca Adventurers y falla con *«Port 8080 was already in use»*.
2. Con el perfil `postgres` y los puertos ya arreglados, el segundo servicio que arranca falla en Flyway al validar la versión 1.
3. Con los dos en marcha, aceptar una misión con un aventurero que *sí* existe responde `404` con el código `ADVENTURER_NOT_FOUND`.
4. Tras corregir las URL a `.../quests` y `.../adventurers`, el arranque falla con *«FATAL: database "quests" does not exist»*.

Para cada síntoma, explica la causa y corrige lo que haga falta, dejando los cuatro archivos de configuración correctos y dando los comandos que arreglan el último. Está bien si: los servicios usan 8081 y 8082, `base-url` de Quests apunta a 8082, los nombres de aplicación son `quests` y `adventurers`, las URL acaban en `/quests` y `/adventurers`, el síntoma 2 se explica por un historial de Flyway compartido con dos `V1` distintas, el 3 por Quests llamándose a sí mismo (que no tiene `/adventurers/{id}`), y el 4 por un volumen anterior a `init.sql`, con `docker compose down -v` y `up`.

Pista: *para el síntoma 3, ¿a qué puerto llama el cliente de Quests y qué servicio responde allí?*

Depuración ▪ solución en la pág. 326

Ejercicio 9.4: Revisión de la verificación 5: ¿un módulo common? ★★★★☆

Es el ejercicio de revisión de la verificación de estructura 5. Un compañero propone evitar la duplicación de `Rank`, `AdventurerId` y los errores de dominio con un módulo compartido. Su árbol cambia solo esto:

Propuesta del compañero (diferencias con el árbol T4)

```
services/
|-- common/                     [N] proyecto Gradle :common
|   `-- src/main/java/com/guildboard/common/
|       |-- Rank.java
|       |-- AdventurerId.java
|       `-- DomainError.java
|-- quests/                     quests-application y quests-domain
|   `-- ...                     declaran api project(':common')
`-- adventurers/                igual que quests
```

Resuelve en tres partes. **Parte A.** Aplica al árbol propuesto la lista de cuatro comprobaciones de la verificación de estructura 5: ¿cuáles cambian de respuesta respecto al árbol T4? **Parte B.** El gremio añade el rango `LEGEND` (a partir de 1000 de experiencia). Describe, para el árbol T4 y para la propuesta, qué archivos hay que cambiar, qué hay que construir y desplegar y en qué orden, y qué ocurre mientras convive una versión vieja con una nueva de cada servicio. **Parte C.** Decide: ¿te quedas con el árbol T4 o con la propuesta? Escribe tres condiciones bajo las cuales sí compartirías algo entre servicios, y qué. Está bien si: la parte A señala al menos que la propuesta introduce una dependencia entre los dos servicios a través de `common` y que `application` ya no depende solo de su dominio; la parte B incluye en T4 una secuencia de despliegue con su motivo, y en la propuesta reconoce que los dos servicios cambian y se despliegan acoplados; y la parte C toma una decisión argumentada con tres condiciones concretas, no un «depende».

Pista: *piensa en qué pasa entre el momento en que se despliega el primer servicio y el segundo. ¿Qué servicio debe aprender primero a tolerar un valor nuevo: el que lo produce o el que lo lee?*

Diseño ▪ solución en la pág. 328

Ejercicio 9.5: Hito H5a: Adventurers y el cliente REST ★★★★☆

Parte del proyecto con los pasos 1 a 6 de la sección 9.7 hechos: los ocho proyectos declarados, Quests ya reducido y con `AdventurerDirectory`, y los archivos de aventureros movidos a `services/adventurers` con sus paquetes renombrados. Haz primero el ejercicio 9.2. Falta:

1. **Cliente REST de Quests.** Escribe `RestAdventurerDirectory` en `adapter/out/rest` a partir de este esqueleto; usa `AdventurerPayload` y `AdventurerPayloadMapper`:

Esqueleto: RestAdventurerDirectory

```java
public class RestAdventurerDirectory implements AdventurerDirectory {

    private final RestClient client;

    public RestAdventurerDirectory(String baseUrl) {
        // construir el RestClient con la URL base
    }

    @Override
    public Optional<AdventurerView> findById(AdventurerId id) {
        // GET /adventurers/{id}; un 404 es Optional.empty()
    }
}
```

2. **Gradle de Adventurers.** Los cuatro `build.gradle` (la copia de los de Quests con los nombres cambiados) y su `ArchitectureTest` (la copia de la de Quests con su paquete base).
3. **Núcleo de Adventurers.** `DomainError` (solo `AdventurerNotFound`) y `DomainException`, con *switch* exhaustivo y sin `default`.
4. **Adaptadores de Adventurers.** El manejador de errores HTTP, el DTO `GrantExperienceRequest` y el controlador con el nuevo `POST /adventurers/{id}/experience`.
5. **Arranque de Adventurers.** `AdventurersApplication`, `BeanConfig` (tres casos de uso), `MemoryPersistenceConfig`, `PostgresPersistenceConfig`, los dos `yml` (puerto 8082, base `adventurers`) y la migración `V1` solo con la tabla de aventureros.
6. Ejecuta `./gradlew build`, haz el escenario de la sección 9.8.1 en modo memoria y con el perfil `postgres`, y haz el *commit* del hito.

Está bien si: `./gradlew build` termina con `BUILD SUCCESSFUL` (corre las pruebas de los dos dominios y las pruebas de arquitectura de los dos servicios); el escenario da `422` y luego `200` al aceptar la misión de rango `ADEPT`; completar una misión deja la experiencia en 0; el `POST` de experiencia sube el rango; un aventurero inexistente da `404 ADVENTURER_NOT_FOUND`; en la base `quests` no existe la tabla `adventurers`; y ningún `build.gradle` de un servicio nombra un proyecto del otro.

Pista: *el 404 de `RestClient` es la excepción `HttpClientErrorException.NotFound`. Si la respuesta no trae cuerpo, `body()` devuelve `null`, y `Optional.ofNullable` lo convierte en un `Optional` vacío.*

Proyecto ▪ solución en la pág. 329

9.10 Resumen

Resumen

- Un microservicio es un servicio pequeño en su propio proceso, con sus propios datos, que se comunica con mecanismos ligeros; cuesta red, fallos parciales y consistencia, y no es por sí mismo una arquitectura: dentro de cada servicio sigue valiendo el hexágono.
- Los contextos delimitados trazan el corte: se corta donde cambia el significado de las

palabras, donde cambia el dueño del dato y donde cambian las cosas juntas. GuildBoard se corta en Quests y Adventurers (Treasury llega en el capítulo 10); la regla de aceptar sigue en `Quest.accept`, que recibe el rango como dato.

- Cada servicio tiene su base de datos: aquí una instancia de PostgreSQL con dos bases (`quests` y `adventurers`) creadas por `infra/postgres/init.sql`, que solo se ejecuta con el volumen vacío. Cada servicio tiene su `V1` de Flyway y no hay clave foránea entre ellos.
- El contrato se compone de URL, JSON y códigos de estado. Cada lado tiene su modelo: Adventurers produce `AdventurerResponse`; Quests lee `AdventurerPayload`, lo traduce en el adaptador a `AdventurerView` y su puerto es `AdventurerDirectory`. Un valor desconocido es un fallo del contrato (`IllegalStateException`, 500), no un error del cliente.
- `RestClient` (Spring Framework 7, sin dependencia nueva) vive en `adapter/out/rest`; `retrieve().body()` lanza `HttpClientErrorException.NotFound` en un 404, que el adaptador traduce a `Optional.empty()`. Sin tiempos máximos ni reintentos hasta el capítulo 12.
- La verificación de estructura 5 muestra el árbol T4: `services/<servicio>/<módulo>`, paquetes `com.guildboard.<servicio>`, ocho proyectos Gradle de nombre único con `projectDir`, `infra/` y sin módulo `common`: lo duplicado (`Rank`, `AdventurerId`) es el precio de no acoplar los servicios.
- `ArchitectureTest` se mueve con Quests y se copia, sin compartir, a Adventurers; el paquete base (`com.guildboard.quests` o `com.guildboard.adventurers`) es lo único que cambia. Un paquete base erróneo deja la selección vacía y la prueba falla; tras renombrar, provoca una infracción para comprobar que sigue viendo.
- Hasta el capítulo 11 completar una misión no da experiencia (retroceso declarado), `RestClient` se adelantó respecto a `proyecto.md`, y la carrera de `AcceptQuest` sigue sin resolver.

Soluciones de los ejercicios

Solución del ejercicio 9.1 enunciado en la pág. 319

Funcionalidad	Servicio	Por qué	Datos del otro
1. Publicar una misión	Quests	La misión, su recompensa y su rango requerido son de Quests.	Ninguno.
2. Registrar un aventurero	Adventurers	El nombre, la experiencia y el rango nacen allí.	Ninguno.
3. Rechazar a un `NOVICE` en una misión `ADEPT`	Quests	La regla compara dos rangos y es la misión quien la hace cumplir (`Quest.accept`).	El rango del aventurero, *por REST* cada vez; no se guarda en Quests. El rango requerido es propio.
4. Subir de `NOVICE` a `ADEPT` con 100 de experiencia	Adventurers	Los umbrales y la experiencia son suyos y cambian con sus reglas.	Ninguno (la experiencia entra por el `POST` provisional).
5. Máximo de 3 misiones activas	Quests	Se cuentan *misiones* (`countActiveBy`), que son de Quests.	Solo el identificador del aventurero, copiado como `assignedTo`.
6. Nombre del aventurero en la lista	Sin respuesta única	Ver abajo.	El nombre, que es de Adventurers.

La sexta tiene tres caminos. (a) Quests pide el nombre a Adventurers al construir la lista: no guarda nada, pero cada lectura depende de otro servicio y añade una llamada por aventurero. (b) Quests guarda una copia del nombre al aceptar: la lectura es local, pero la copia queda obsoleta si el nombre cambia y ahora hay dos sitios con un dato que tiene un solo dueño. (c) Quien llama (una interfaz o, en el capítulo 13, el *gateway*) pide las dos cosas y las junta: ninguno de los dos servicios cambia. Referencia: (c), o (a) si hace falta un solo punto de entrada; la (b) es la que más cuesta mantener.
Valores en los dos servicios: `AdventurerId` (la clave de un aventurero en Adventurers; `assignedTo` en una misión de Quests) y `Rank` (el rango de Adventurers que Quests compara con el requerido). La copia es aceptable porque son valores pequeños e inmutables, sin base de datos común, y lo que mantiene a los dos de acuerdo no es una clase compartida sino el contrato (el JSON de `GET /adventurers/{id}`) y el mapeador estricto del lado de Quests.
Errores típicos: asignar la 3 a Adventurers «porque el rango es suyo» (la regla necesita también el rango requerido, que es de Quests); guardar el rango en Quests para no hacer la llamada (el rango sube con la experiencia y la copia quedaría vieja); y olvidar que el recuento de la 5 es de Quests.

Solución del ejercicio 9.2 enunciado en la pág. 319

Paso 1. La respuesta del contrato (sección 9.5; la misma que da `curl localhost:8082/adventurers/$MARA` una vez montado el servicio) trae cuatro campos: `id`, `name`, `rank` y `experience`. Quests solo necesita los dos que no son nombre ni experiencia.

adapter/out/rest/AdventurerPayload.java

```java
package com.guildboard.quests.adapter.out.rest;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;

// La forma del JSON de GET /adventurers/{id} tal como la lee Quests:
// solo lo que Quests necesita; el resto de campos se ignora.
@JsonIgnoreProperties(ignoreUnknown = true)
public record AdventurerPayload(String id, String rank) {
}
```

adapter/out/rest/AdventurerPayloadMapper.java

```java
package com.guildboard.quests.adapter.out.rest;

import com.guildboard.quests.application.port.out.AdventurerView;
import com.guildboard.quests.domain.AdventurerId;
import com.guildboard.quests.domain.Rank;

final class AdventurerPayloadMapper {

    private AdventurerPayloadMapper() {
    }

    static AdventurerView toView(AdventurerPayload payload) {
        if (payload.id() == null || payload.rank() == null) {
            throw new IllegalStateException(
                "incomplete adventurer from adventurers service: id="
                + payload.id() + ", rank=" + payload.rank());
        }
        return new AdventurerView(new AdventurerId(payload.id()),
                                  parseRank(payload.rank()));
    }

    private static Rank parseRank(String value) {
        try {
            return Rank.valueOf(value);
        } catch (IllegalArgumentException e) {
            throw new IllegalStateException(
                "unknown rank from adventurers service: " + value, e);
        }
    }
}
```

Por qué funciona. El *record* tiene solo los dos campos y la anotación `ignoreUnknown` hace que `name` y `experience` (o un campo nuevo que añada Adventurers) se ignoren en lugar de fallar. El mapeador es de paquete (`final class`, método sin `public`): solo lo usa el adaptador. La excepción que lanza es `IllegalStateException`, y se conserva la causa. Con `null` en el rango, `Rank.valueOf(null)` lanzaría `NullPointerException`; por eso la comprobación de `null` va antes, con un mensaje que dice qué falta. Se puede probar con un `main` temporal en el mismo paquete (el mapeador es de paquete) o con un test sin Spring.
Paso 4. Hay que tocar el `AdventurerPayload`: renombrar su campo (o anotarlo con `@JsonProperty("level")`) y, si cambian los valores, el mapeador. Nada más: el puerto,

el servicio y el dominio ni se enteran. Mientras no se haga, `rank` llegaría como `null`, el mapeador lanza `IllegalStateException` y aceptar misiones da un `500`: es un contrato roto, y por eso no debe parecer un error del cliente.
Errores típicos: dejar que `Rank.valueOf` lance su `IllegalArgumentException` (el manejador de errores de Quests lo convierte en `400`); olvidar la anotación (si el JSON trae campos de más y el mapeador de Jackson está configurado para rechazarlos, falla); y poner el *payload* en `application`, donde no pinta nada: es un detalle del adaptador.

Solución del ejercicio 9.3 enunciado en la pág. 320

Causas.

1. Los dos servicios quieren el puerto 8080. El segundo en arrancar no puede abrirlo. Solución: 8081 para Quests y 8082 para Adventurers.
2. Las dos aplicaciones apuntan a la misma base, `guildboard`, y por tanto al mismo `flyway_schema_history`. Quests aplica su `V1` (que crea `quests`); cuando arranca Adventurers, Flyway ve que la versión 1 ya está aplicada con otro contenido y otra descripción, y su validación falla. Aunque no fallara, la tabla de uno no se crearía. Cada servicio necesita su base, y con ella su propio historial.
3. `base-url` de Quests apunta a 8081, es decir, a sí mismo. Quests no tiene `/adventurers/{id}`, así que responde `404`, el adaptador lo traduce a `Optional.empty()` y el servicio dice que el aventurero no existe. Es un 404 «correcto» a una pregunta equivocada. Solución: 8082.
4. El volumen de Compose es anterior a `init.sql`; los scripts de inicio solo se ejecutan con el volumen vacío. Solución: borrar el volumen y volver a levantar.

Configuración corregida.

quests-bootstrap: application.yml

```yaml
spring:
  application:
    name: quests
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration

server:
  port: 8081

guildboard:
  adventurers:
    base-url: http://localhost:8082
```

adventurers-bootstrap: application.yml

```yaml
spring:
  application:
    name: adventurers
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration

server:
  port: 8082
```

quests-bootstrap: application-postgres.yml

```yaml
spring:
  autoconfigure:
    exclude: []
  datasource:
    url: jdbc:postgresql://localhost:5432/quests
    username: guildboard
    password: guildboard
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: "none"
```

adventurers-bootstrap: application-postgres.yml

```yaml
spring:
  autoconfigure:
    exclude: []
  datasource:
    url: jdbc:postgresql://localhost:5432/adventurers
    username: guildboard
    password: guildboard
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: "none"
```

Para el síntoma 4:

```
$ docker compose down -v
$ docker compose up -d --wait
```

(Borra los datos de la base; en desarrollo no importa.) **Errores típicos:** corregir solo la

URL y no reiniciar el volumen; arreglar el puerto de Adventurers pero no el `base-url` de Quests (el síntoma 3 aparece justo cuando todo «parece» arrancado); y borrar el historial de Flyway a mano en la base común en lugar de separar las bases.

Solución del ejercicio 9.4 enunciado en la pág. 321

Parte A. Cambian de respuesta tres de las cuatro:

1. *¿`./gradlew test` pasa, con la prueba de arquitectura de cada servicio?* Deja de ser seguro: el `api project(':common')` hace que dominio y aplicación dependan de clases que no son de su servicio, y `applicationOnlyDependsOnDomain` solo admite `..application..`, `..domain..`, `java..` y `lombok..`; habría que ampliar las reglas o la prueba fallaría. Además, sus reglas miran el paquete base de cada servicio, no la carpeta `common`.
2. *¿Cada adaptador está en su lado?* Sin cambio.
3. *¿Algún `build.gradle` nombra a otro servicio?* Formalmente no, pero los dos nombran el mismo `:common`: el acoplamiento es indirecto y real (cambiar `common` cambia los dos).
4. *¿Las carpetas coinciden con el árbol?* Ya no: aparece `common/`, que no está en el árbol T4. Y su contenido es mixto: `Rank` es de Adventurers, pero un `DomainError` común tendría que conocer las variantes de misión (`QuestNotFound`, `QuestStatus`...), así que `common` acabaría conteniendo casi todo el dominio.

Parte B. Para el rango nuevo `LEGEND`:

	Árbol T4 (duplicado)	**Propuesta (`common`)**
Qué se toca	`Rank` y los umbrales de `Adventurer` en Adventurers (más su prueba); `Rank` en Quests, para que lo lea.	`Rank` una vez, en `common`; los umbrales en Adventurers.
Qué se construye	Cada servicio por su cuenta.	`common` y, como dependen de él, los dos servicios.
Orden de despliegue	Primero Quests (ya conoce `LEGEND`, aunque nadie lo emita), después Adventurers. Si fuese al revés, un `LEGEND` de Adventurers llegaría a un Quests que no lo conoce: su mapeador lanzaría `IllegalStateException` y aceptar daría `500`.	Los dos a la vez, o el mismo riesgo: el JSON lleva el valor `"LEGEND"` tanto si la clase es compartida como si no.
Versiones mezcladas	Seguras si se respeta el orden: el que *lee* aprende primero.	Son un riesgo igual, y el módulo común *anima* a desplegar juntos, es decir, a acoplar los ciclos de vida.

La conclusión importante: compartir la clase no elimina el problema de las versiones mezcladas, porque el contrato es el JSON, no la clase; solo elimina la señal de que existe. **Parte C. Decisión de referencia:** el árbol T4, duplicar. El coste (dos `Rank` que hay que mantener de acuerdo) lo cubren el mapeador estricto de Quests y el orden «primero el lector»; el beneficio es que cada servicio se construye y se despliega solo. Tres condiciones bajo las que sí compartiría algo:

1. Lo compartido es un *contrato publicado* (por ejemplo, un esquema JSON/OpenAPI o un cliente generado) y no una clase del dominio, con un dueño claro: el servicio que lo produce.
2. Es solo *forma* de datos, sin comportamiento de negocio (nada de `canTake` ni de

umbrales).

3. Está versionado de forma inmutable, con una política de compatibilidad (solo cambios aditivos, lectores tolerantes) para que cada consumidor actualice a su ritmo.

Quedan fuera de esas condiciones las utilidades técnicas sin dominio (por ejemplo, una librería de trazas), que sí pueden compartirse.
Criterios para valorar otras soluciones. Es válida si decide explícitamente con el caso de `LEGEND`, reconoce el acoplamiento de despliegue y no confunde *contrato* con *clase compartida*. Alternativa válida: compartir un artefacto de contrato versionado (condición 1) con su dueño y su política, siempre que la respuesta diga qué contiene y qué no.

Solución del ejercicio 9.5 enunciado en la pág. 321

Estado de partida. Tras los pasos 1 a 6 de la sección 9.7, `services/adventurers` contiene, ya con los paquetes `com.guildboard.adventurers.*`: `Adventurer`, `AdventurerId`, `Rank`, `AdventurerTest`; los cinco puertos de entrada y `AdventurerRepository`; los tres servicios; el controlador, los dos DTO y `AdventurerWebMapper`; `InMemoryAdventurerRepository` y los cuatro archivos de persistencia. Esos archivos *no cambian*, salvo el controlador (se reemplaza en el apartado 4, que además añade un DTO): son los de los capítulos 5, 6 y 8 con otro nombre de paquete. Lo que sigue son los archivos que hay que escribir o reescribir.
1. Cliente REST de Quests. Junto a `AdventurerPayload` y su mapeador:

quests-adapters: adapter/out/rest/RestAdventurerDirectory.java

```java
package com.guildboard.quests.adapter.out.rest;

import com.guildboard.quests.application.port.out.AdventurerDirectory;
import com.guildboard.quests.application.port.out.AdventurerView;
import com.guildboard.quests.domain.AdventurerId;
import java.util.Optional;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.RestClient;

public class RestAdventurerDirectory implements AdventurerDirectory {

    private final RestClient client;

    public RestAdventurerDirectory(String baseUrl) {
        this.client = RestClient.builder().baseUrl(baseUrl).build();
    }

    @Override
    public Optional<AdventurerView> findById(AdventurerId id) {
        try {
            AdventurerPayload payload = client.get()
                .uri("/adventurers/{id}", id.value())
                .retrieve()
                .body(AdventurerPayload.class);
            return Optional.ofNullable(payload)
                .map(AdventurerPayloadMapper::toView);
        } catch (HttpClientErrorException.NotFound e) {
            return Optional.empty();
        }
    }
}
```

2. Gradle y prueba de arquitectura de Adventurers. La copia de los de Quests con otros nombres, más la copia de `ArchitectureTest` con su paquete base.

services/adventurers/domain/build.gradle

```
dependencies {
    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

services/adventurers/application/build.gradle

```
dependencies {
    api project(':adventurers-domain')

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

services/adventurers/adapters/build.gradle

```
dependencies {
    implementation project(':adventurers-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

services/adventurers/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencies {
    implementation project(':adventurers-adapters')
    implementation project(':adventurers-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

adventurers-bootstrap: src/test/.../adventurers/ArchitectureTest.java

```java
package com.guildboard.adventurers;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;

import com.tngtech.archunit.core.domain.JavaClasses;
import com.tngtech.archunit.core.importer.ClassFileImporter;
import com.tngtech.archunit.core.importer.ImportOption;
import org.junit.jupiter.api.Test;

public class ArchitectureTest {

    private static final String BASE_PACKAGE = "com.guildboard.adventurers";

    private static final JavaClasses CLASSES = new ClassFileImporter()
        .withImportOption(new ImportOption.DoNotIncludeTests())
        .importPackages(BASE_PACKAGE);

    @Test
    void domainDoesNotDependOnOuterLayers() {
        noClasses().that().resideInAPackage("..domain..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "..application..", "..adapter..", "..config..",
                "org.springframework..", "jakarta..")
            .check(CLASSES);
    }

    @Test
    void applicationOnlyDependsOnDomain() {
        classes().that().resideInAPackage("..application..")
            .should().onlyDependOnClassesThat().resideInAnyPackage(
                "..application..", "..domain..", "java..", "lombok..")
            .check(CLASSES);
    }

    @Test
    void adapterInDoesNotDependOnAdapterOut() {
        noClasses().that().resideInAPackage("..adapter.in..")
            .should().dependOnClassesThat().resideInAPackage("..adapter.out..")
            .check(CLASSES);
    }

    @Test
    void adapterOutDoesNotDependOnAdapterIn() {
        noClasses().that().resideInAPackage("..adapter.out..")
            .should().dependOnClassesThat().resideInAPackage("..adapter.in..")
            .check(CLASSES);
    }

    @Test
    void adapterInDoesNotUseJpa() {
        noClasses().that().resideInAPackage("..adapter.in..")
            .should().dependOnClassesThat().resideInAPackage("jakarta.persistence..")
            .check(CLASSES);
    }
}
```

3. Núcleo de Adventurers. Un solo error: el aventurero no existe.

domain/DomainError.java (adventurers-domain)

```java
package com.guildboard.adventurers.domain;

public sealed interface DomainError {

    record AdventurerNotFound(AdventurerId adventurerId)
            implements DomainError { }

    default String code() {
        return switch (this) {
            case AdventurerNotFound e -> "ADVENTURER_NOT_FOUND";
        };
    }
}
```

domain/DomainException.java (adventurers-domain)

```java
package com.guildboard.adventurers.domain;

public final class DomainException extends RuntimeException {

    private final DomainError error;

    public DomainException(DomainError error) {
        super(describe(error));
        this.error = error;
    }

    public DomainError getError() {
        return error;
    }

    private static String describe(DomainError error) {
        return switch (error) {
            case DomainError.AdventurerNotFound(AdventurerId id) ->
                "adventurer not found: " + id.value();
        };
    }
}
```

4. Adaptadores de entrada. El manejador de errores (dos trozos seguidos):

adapter/in/web/error/DomainExceptionHandler.java (1/2)

```java
package com.guildboard.adventurers.adapter.in.web.error;

import com.guildboard.adventurers.domain.DomainError;
import com.guildboard.adventurers.domain.DomainException;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class DomainExceptionHandler {

    @ExceptionHandler(DomainException.class)
    public ProblemDetail handleDomain(DomainException exception) {
        DomainError error = exception.getError();
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            statusFor(error), exception.getMessage());
        problem.setTitle("Business rule violated");
        problem.setProperty("code", error.code());
        return problem;
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ProblemDetail handleMalformed(
            IllegalArgumentException exception) {
        return ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, exception.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail handleInvalidBody(
            MethodArgumentNotValidException exception) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, "request body is not valid");
        List<String> errors = exception.getBindingResult()
            .getFieldErrors().stream()
            .map(error -> error.getField() + ": "
                + error.getDefaultMessage())
            .toList();
        problem.setProperty("errors", errors);
        return problem;
    }
```

adapter/in/web/error/DomainExceptionHandler.java (2/2: sigue dentro de la clase)

```
    private static HttpStatusCode statusFor(DomainError error) {
        return switch (error) {
            case DomainError.AdventurerNotFound e -> HttpStatus.NOT_FOUND;
        };
    }
}
```

El DTO nuevo y el controlador con el `POST` provisional:

adapter/in/web/dto/GrantExperienceRequest.java

```
package com.guildboard.adventurers.adapter.in.web.dto;

import jakarta.validation.constraints.Positive;

public record GrantExperienceRequest(@Positive int points) {
}
```

adapter/in/web/AdventurerController.java (1/2)

```java
package com.guildboard.adventurers.adapter.in.web;

import com.guildboard.adventurers.adapter.in.web.dto.AdventurerResponse;
import com.guildboard.adventurers.adapter.in.web.dto.GrantExperienceRequest;
import com.guildboard.adventurers.adapter.in.web.dto.RegisterAdventurerRequest;
import com.guildboard.adventurers.adapter.in.web.mapper.AdventurerWebMapper;
import com.guildboard.adventurers.application.port.in.GetAdventurerUseCase;
import com.guildboard.adventurers.application.port.in.GrantExperienceCommand;
import com.guildboard.adventurers.application.port.in.GrantExperienceUseCase;
import com.guildboard.adventurers.application.port.in.RegisterAdventurerUseCase;
import com.guildboard.adventurers.domain.Adventurer;
import com.guildboard.adventurers.domain.AdventurerId;
import jakarta.validation.Valid;
import java.net.URI;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/adventurers")
@RequiredArgsConstructor
public class AdventurerController {

    private final RegisterAdventurerUseCase registerAdventurer;
    private final GetAdventurerUseCase getAdventurer;
    private final GrantExperienceUseCase grantExperience;

    @PostMapping
    public ResponseEntity<AdventurerResponse> register(
            @Valid @RequestBody RegisterAdventurerRequest request) {
        Adventurer adventurer = registerAdventurer.register(
            AdventurerWebMapper.toCommand(request));
        return ResponseEntity
            .created(URI.create(
                "/adventurers/" + adventurer.getId().value()))
            .body(AdventurerWebMapper.toResponse(adventurer));
    }
```

adapter/in/web/AdventurerController.java (2/2: sigue dentro de la clase)

```
    @GetMapping("/{id}")
    public ResponseEntity<AdventurerResponse> get(
            @PathVariable String id) {
        return getAdventurer.get(new AdventurerId(id))
            .map(AdventurerWebMapper::toResponse)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }

    // Provisional: lo sustituye el evento del cap^^c3^^adtulo 11.
    @PostMapping("/{id}/experience")
    public AdventurerResponse grantExperience(
            @PathVariable String id,
            @Valid @RequestBody GrantExperienceRequest request) {
        Adventurer adventurer = grantExperience.grant(
            new GrantExperienceCommand(
                new AdventurerId(id), request.points()));
        return AdventurerWebMapper.toResponse(adventurer);
    }
}
```

5. Arranque de Adventurers (paquete `com.guildboard.adventurers` y `.config` en `adventurers-bootstrap`).

AdventurersApplication.java

```
package com.guildboard.adventurers;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AdventurersApplication {

    public static void main(String[] args) {
        SpringApplication.run(AdventurersApplication.class, args);
    }
}
```

config/BeanConfig.java (adventurers)

```
package com.guildboard.adventurers.config;

import com.guildboard.adventurers.application.port.in.GetAdventurerUseCase;
import com.guildboard.adventurers.application.port.in.
    ↪ GrantExperienceUseCase;
import com.guildboard.adventurers.application.port.in.
    ↪ RegisterAdventurerUseCase;
import com.guildboard.adventurers.application.port.out.AdventurerRepository;
    ↪
import com.guildboard.adventurers.application.service.GetAdventurerService;
import com.guildboard.adventurers.application.service.
    ↪ GrantExperienceService;
import com.guildboard.adventurers.application.service.
    ↪ RegisterAdventurerService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public RegisterAdventurerUseCase registerAdventurerUseCase(
            AdventurerRepository adventurerRepository) {
        return new RegisterAdventurerService(adventurerRepository);
    }

    @Bean
    public GetAdventurerUseCase getAdventurerUseCase(
            AdventurerRepository adventurerRepository) {
        return new GetAdventurerService(adventurerRepository);
    }

    @Bean
    public GrantExperienceUseCase grantExperienceUseCase(
            AdventurerRepository adventurerRepository) {
        return new GrantExperienceService(adventurerRepository);
    }
}
```

config/MemoryPersistenceConfig.java (adventurers)

```
package com.guildboard.adventurers.config;

import com.guildboard.adventurers.adapter.out.memory.
    ↪ InMemoryAdventurerRepository;
import com.guildboard.adventurers.application.port.out.AdventurerRepository;
    ↪
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

@Configuration
@Profile("!postgres")
public class MemoryPersistenceConfig {

    @Bean
    public AdventurerRepository adventurerRepository() {
        return new InMemoryAdventurerRepository();
    }
}
```

config/PostgresPersistenceConfig.java (adventurers)

```
package com.guildboard.adventurers.config;

import com.guildboard.adventurers.adapter.out.persistence.AdventurerEntity;
import com.guildboard.adventurers.adapter.out.persistence.
    ↪ AdventurerJpaRepository;
import com.guildboard.adventurers.adapter.out.persistence.
    ↪ JpaAdventurerRepository;
import com.guildboard.adventurers.application.port.out.AdventurerRepository;
    ↪
import org.springframework.boot.persistence.autoconfigure.EntityScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
    ↪

@Configuration
@Profile("postgres")
@EnableJpaRepositories(basePackageClasses = AdventurerJpaRepository.class)
@EntityScan(basePackageClasses = AdventurerEntity.class)
public class PostgresPersistenceConfig {

    @Bean
    public AdventurerRepository adventurerRepository(
            AdventurerJpaRepository jpa) {
        return new JpaAdventurerRepository(jpa);
    }
}
```

La configuración, en `services/adventurers/bootstrap/src/main/resources` (puerto 8082 y base `adventurers`), y la migración:

`application.yml (adventurers)`

```yaml
spring:
  application:
    name: adventurers
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration

server:
  port: 8082
```

`application-postgres.yml (adventurers)`

```yaml
spring:
  autoconfigure:
    exclude: []
  datasource:
    url: jdbc:postgresql://localhost:5432/adventurers
    username: guildboard
    password: guildboard
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: "none"
```

`db/migration/V1__create_adventurers_table.sql`

```sql
-- V1 de Adventurers: solo la tabla de aventureros.

CREATE TABLE adventurers (
    id              VARCHAR(64)  PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    adventurer_rank VARCHAR(20)  NOT NULL,
    experience      INTEGER      NOT NULL
);
```

6. Comprobar.

```
$ ./gradlew build
BUILD SUCCESSFUL
$ grep -rn "adventurers-" services/quests --include=build.gradle
$ grep -rn "quests-" services/adventurers --include=build.gradle
$ git add -A
$ git commit -m "H5a: Quests y Adventurers como servicios"
```

Los dos `grep` no deben imprimir nada: ningún servicio nombra al otro. Después, el escenario de la sección 9.8.1.

Por qué funciona. El cliente construye el `RestClient` con la URL base que le da la configuración y pregunta por `/adventurers/{id}`. Un 404 se convierte en `Optional.empty()`; cualquier otro fallo sube. El servicio de aplicación no sabe que hay HTTP: depende del puerto. Adventurers no cambió en esencia: perdió lo de misiones, y ganó un endpoint de entrada.

Criterios para valorar otras soluciones. Es válida si cada servicio compila sin el otro (ningún `build.gradle` nombra al otro), `application` de Quests no importa `org.springframework.web.client`, el rango se pregunta a Adventurers en cada aceptación, y los errores de contrato no acaban en un `400`.

Errores típicos: dejar el `application.yml` de Adventurers con el puerto 8080; olvidar crear `services/adventurers/bootstrap` (Gradle se niega a configurar un proyecto sin carpeta); en `DomainExceptionHandler` de Adventurers dejar el *switch* con los casos de misiones (no compila: esas variantes no existen); registrar en `BeanConfig` un bean de `AdventurerRepository` además del de `MemoryPersistenceConfig` (Spring protesta por dos candidatos); y olvidar que, sin el perfil, los datos viven en memoria y se pierden al parar Adventurers.

CAPÍTULO 10

Tercer servicio y plantilla de servicio: Treasury

Objetivos del capítulo

- Comprobar en la verificación de estructura 6 el árbol con tres servicios, la tercera base de datos y las pruebas nuevas, y que los tres servicios tienen la misma forma.
- Modelar un libro de cuentas (*ledger*) con `Account`, `LedgerEntry` y `Gold`, y hacer idempotente el pago de una misión con dos defensas: una consulta previa y una restricción única en la base de datos.
- Reproducir la estructura de un servicio como plantilla y vigilar la de todos con dos pruebas: un `StructureTest` con `java.nio.file` y una regla de ArchUnit que impide que un servicio dependa de otro.
- Dejar completo el hito H5: Quests, Adventurers y Treasury, cada uno con su base, y un pago por REST provisional.

Hasta ahora, cada servicio nuevo era un corte de lo que ya existía. Treasury es el primero que se construye desde cero, y por eso sirve para comprobar si lo aprendido funciona como *molde*: si los capítulos anteriores están bien hechos, un servicio nuevo es repetir una forma, no inventarla. El capítulo tiene dos mitades. La primera es de estructura: la verificación de estructura 6, la plantilla y las pruebas que la vigilan. La segunda es de negocio, pequeño pero con una idea nueva: un pago que no debe ocurrir dos veces.

Tres cosas que el capítulo deja dichas desde el principio:

- **No hay dependencias nuevas.** Treasury usa lo que ya llevan Quests y Adventurers desde el capítulo 8: Spring Web, validación, Spring Data JPA, Flyway y el driver de PostgreSQL. Por eso no hay sección de instalación.
- **El adaptador REST es provisional.** Treasury ofrece `POST /payments` para poder probarlo con `curl`. En el capítulo 11 el pago lo dispara un evento de Quests y este endpoint desaparece.
- **Una regla del capítulo 5 se dobla.** Pagar escribe en dos sitios (el libro y el saldo de la cuenta), y ya no se puede decir «un caso de uso modifica un solo agregado». La sección 10.4 explica cómo se gestiona y qué riesgo queda.

10.1 Qué hace Treasury y por qué es un servicio aparte

En el capítulo 2 aparecieron tres contextos candidatos y en el capítulo 9 se cortaron dos. El tercero es el dinero. Treasury (la tesorería del gremio) guarda cuánto oro tiene cada aventurero y por qué: cada pago de una recompensa queda anotado una sola vez. Aplicamos las mismas preguntas del capítulo anterior:

1. **¿Cambia el significado de una palabra?** Para Treasury, un aventurero es una *cuenta* a la que acreditar oro, y una misión es solo un identificador (`questId`) que justifica un pago. No sabe nada de rangos ni de estados de misión.
2. **¿Quién es el dueño del dato?** El saldo es de Treasury: ni Quests ni Adventurers lo guardan.
3. **¿Qué cambia junto?** Si mañana el gremio cobra una comisión o paga en otra moneda, cambia Treasury y nada más. Los umbrales de rango no tienen nada que ver con eso.
4. **¿Una operación modifica un solo agregado?** Casi: la sección 10.4 es honesta con la excepción.

	Quests (8081)	**Adventurers** (8082)	**Treasury** (8083)
Es dueño de	Misiones y sus reglas.	Aventureros, experiencia y rango.	Cuentas de oro y libro de pagos.
Casos de uso	Publicar, consultar, aceptar y completar.	Registrar, consultar y dar experiencia.	Pagar una recompensa y consultar una cuenta.
Pregunta al otro	El rango, a Adventurers.	Nada.	Nada (por ahora).
Base de datos	`quests`	`adventurers`	`treasury`

La última fila repite la regla de la base por servicio [41, 42]. Newman insiste en que partir un sistema solo compensa si cada pieza gana autonomía real [32]: Treasury es el caso más claro de las tres, porque el dinero se audita y se conserva de otra forma que las misiones.

Una consecuencia que conviene aceptar ya: **Treasury confía en quien le manda la orden**. Recibe «paga 150 de oro al aventurero A por la misión Q» y no comprueba que A exista ni que Q se haya completado. Hoy quien manda la orden eres tú, con `curl`. Desde el capítulo 11 será el evento de Quests. Preguntar a los otros dos servicios añadiría llamadas REST que no necesitamos y crearía justo el acoplamiento que el corte quería evitar.

10.2 Verificación de estructura 6: tres servicios, un mismo molde

Nota

Este capítulo cambia la estructura de carpetas: aparece `services/treasury`, se añade una tercera base en `infra/postgres/init.sql` y la raíz gana su primera prueba, `StructureTest`. Comprueba que tu proyecto coincide con el árbol de abajo antes de seguir; las piezas se construyen en las secciones siguientes.

10.2.1 El árbol anterior

Así dejó el proyecto el capítulo 9 (árbol T4, dos servicios):

Árbol anterior (T4, tras el capítulo 9)

```
guildboard/
|-- .git/, .gitignore, gradlew, gradlew.bat
|-- settings.gradle        ocho proyectos, nombres ^^c3^^banicos
|-- build.gradle           configuraci^^c3^^b3n com^^c3^^ban (subprojects)
|-- compose.yaml           PostgreSQL 18; monta infra/postgres/init.sql
|-- docs/dominio.md
```

```
|-- gradle/wrapper/
|-- infra/
|   `-- postgres/init.sql  crea las bases quests y adventurers
`-- services/
    |-- quests/            domain/ application/ adapters/ bootstrap/
    `-- adventurers/       domain/ application/ adapters/ bootstrap/
```

10.2.2 El árbol nuevo

`[N]` es nuevo y `[M]` modificado. Los dos servicios existentes no se tocan:

Árbol nuevo, raíz (T5, tras el capítulo 10)

```
guildboard/
|-- .git/, .gitignore, gradlew, gradlew.bat
|-- settings.gradle        [M] 'treasury' en el bucle: doce proyectos
|-- build.gradle           [M] plugin java y JUnit, para alojar StructureTest
|-- src/test/java/com/guildboard/StructureTest.java   [N]
|-- compose.yaml           sin cambios
|-- docs/dominio.md        sin cambios
|-- gradle/wrapper/        sin cambios
|-- infra/
|   `-- postgres/
|       `-- init.sql       [M] crea tambi^^c3^^a9n la base treasury
`-- services/
    |-- quests/            sin cambios
    |-- adventurers/       sin cambios
    `-- treasury/          [N] la misma forma
        |-- domain/
        |-- application/
        |-- adapters/
        `-- bootstrap/
```

Y dentro de Treasury, paquete base `com.guildboard.treasury`. Es la forma final del capítulo: incluye lo que añaden los ejercicios.

Dentro de services/treasury (paquetes)

```
services/treasury/                    com.guildboard.treasury
|-- domain/       Account, LedgerEntry, Ledger, Gold, AdventurerId y
|                 QuestId [copias propias], DomainError (una variante),
|                 DomainException
|-- application/  port/in    PayReward*, GetAccountUseCase,
|                            GetLedgerBalanceUseCase
|                 port/out   AccountRepository, LedgerRepository
|                 service    PayRewardService, GetAccountService,
|                            GetLedgerBalanceService
|-- adapters/     adapter/in/web  TreasuryController,
|                                 LedgerBalanceController + dto, mapper,
|                                 error
|                 adapter/out     memory, persistence
`-- bootstrap/    TreasuryApplication, config/, application*.yml,
                  db/migration (V1 con account y ledger_entry,
                  V2 con la restricci^^c3^^b3n ^^c3^^banica sobre quest_id),
```

```
17                    src/test/.../treasury/ArchitectureTest
```

10.2.3 Cambios respecto al árbol anterior

- `services/treasury`: servicio nuevo con cuatro módulos, la misma separación del capítulo 7. El primer nivel del árbol sigue diciendo qué hace el sistema [28].
- Misma forma que Quests y Adventurers: es la *plantilla de servicio*, una convención del curso (ni una herramienta ni un generador) que una tarea comprueba.
- `settings.gradle`: doce proyectos Gradle; solo se añade `'treasury'` al bucle con `include` y `projectDir` [15].
- `infra/postgres/init.sql`: crea también la base `treasury`; una instancia de PostgreSQL y tres bases lógicas, la regla de una base por servicio [42].
- `build.gradle` de la raíz y `src/test/java/com/guildboard/StructureTest.java`: la raíz aplica el plugin `java` y las dependencias de JUnit para alojar la prueba, que comprueba que cada servicio tiene sus cuatro módulos.
- `ArchitectureTest`: Treasury lleva el suyo, copia de la plantilla con `com.guildboard.treasury` como único dato, y los tres ganan la regla de independencia entre servicios (sección 10.8).

10.2.4 Comprobaciones

- [ ] `./gradlew test` pasa: incluye el `StructureTest` de la raíz y el `ArchitectureTest` de cada servicio (el de Treasury, cuando tenga código, al final del capítulo).
- [ ] Cada adaptador está en `in` o en `out` según quién lo invoca: el controlador REST es de entrada; los repositorios (memoria y JPA) son de salida.
- [ ] Ningún `build.gradle` de un servicio nombra a un proyecto de otro. Con tres servicios hay seis direcciones posibles (cada uno puede nombrar a otros dos); la sección 10.8 lo convierte en una prueba.
- [ ] Las carpetas de tu proyecto coinciden con el árbol.

Si algo no coincide con el árbol, corrígelo antes de seguir. El ejercicio 10.1 practica estas comprobaciones comparando el interior de Quests con el de Treasury.

10.3 Reproducir la estructura de un servicio

Lo primero es la estructura, vacía, sin una línea de negocio. Parte de un *commit* limpio del hito H5a (`git status` sin cambios). Las carpetas primero, porque Gradle se niega a configurar un proyecto cuya carpeta no existe (lo viste en el capítulo 9):

```
$ T=services/treasury
$ JT=src/main/java/com/guildboard/treasury
$ mkdir -p $T/domain/$JT/domain
$ mkdir -p $T/domain/src/test/java/com/guildboard/treasury/domain
$ mkdir -p $T/application/$JT/application/{port/in,port/out,service}
$ mkdir -p $T/adapters/$JT/adapter/in/web/{dto,mapper,error}
```

```
$ mkdir -p $T/adapters/$JT/adapter/out/{memory,persistence}
$ mkdir -p $T/bootstrap/$JT/config
$ mkdir -p $T/bootstrap/src/main/resources/db/migration
```

Después, `settings.gradle`: el único cambio es añadir `'treasury'` a la lista del bucle. Este es el archivo completo:

settings.gradle

```
rootProject.name = 'guildboard'

// Un proyecto Gradle por m^^c3^^b3dulo y servicio, con nombre ^^c3^^banico
// (quests-domain, adventurers-domain, treasury-domain...).
['quests', 'adventurers', 'treasury'].each { service ->
    ['domain', 'application', 'adapters', 'bootstrap'].each { module ->
        def projectName = "${service}-${module}"
        include projectName
        project(":${projectName}").projectDir =
            file("services/${service}/${module}")
    }
}
```

Los cuatro `build.gradle` de Treasury son los de Adventurers con el nombre del servicio cambiado: ninguna línea es distinta salvo `adventurers` por `treasury`. Es exactamente el tipo de cambio que se hace con un comando:

```
$ for m in domain application adapters bootstrap; do
>   sed 's/adventurers/treasury/g' \
>     services/adventurers/$m/build.gradle \
>     > services/treasury/$m/build.gradle
> done
```

Este es el resultado en el módulo de adaptadores (los otros tres siguen el mismo patrón: las dependencias entre módulos con el prefijo `treasury-`; el `bootstrap` conserva además la línea de `archunit` que ya llevaba el de Adventurers):

services/treasury/adapters/build.gradle

```
dependencies {
    implementation project(':treasury-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

Comprueba a mano que ningún `build.gradle` nombra a otro servicio, con dos `grep` que no deben imprimir nada (la prueba de la sección 10.8 lo hará por ti), y que Gradle ve los doce proyectos:

```
$ grep -rnE "quests-|adventurers-" services/treasury --include=build.gradle
$ grep -rn "treasury-" services/quests services/adventurers \
    --include=build.gradle
$ ./gradlew projects | grep -c "Project ':"
12
```

Por último, la base. Añade una línea a `infra/postgres/init.sql`:

infra/postgres/init.sql

```
-- Una base de datos por servicio, en la misma instancia (solo
-- desarrollo). Se ejecuta UNA vez, al crear el volumen por primera vez.
CREATE DATABASE quests;
CREATE DATABASE adventurers;
CREATE DATABASE treasury;
```

Otra vez, el volumen

Como la vez anterior, el script solo se ejecuta con el volumen vacío. Añadir una base y hacer `down` y `up` no la crea; hace falta `down -v`, que borra los datos de prueba (en desarrollo no importa). Comprobado con `postgres:18` y este mismo `init.sql`:

```
$ docker compose down -v
$ docker compose up -d --wait
$ docker compose exec postgres psql -U guildboard -d guildboard -Atc \
    "select datname from pg_database \
     where datname in ('quests','adventurers','treasury') order by 1"
adventurers
quests
treasury
```

El `compose.yaml` no cambia: ya monta `init.sql`.

10.4 El libro de cuentas

Un *libro de cuentas* (en inglés, *ledger*) es una lista de *asientos*: cada uno anota un movimiento de dinero, con quién, cuánto, por qué y cuándo. La regla de oro de un libro es que **los asientos no se editan ni se borran**: si te equivocas, anotas un asiento nuevo que lo corrige. El *saldo* no es un dato independiente, es un resumen: la suma de los asientos. Esta idea contable es antigua y no figura en ninguna de las fuentes del curso; la explicación es mía.

En Treasury hay dos conceptos de dominio con identidad y uno sin ella. En los términos del capítulo 2 [8]:

- `Gold` es un *objeto de valor*: una cantidad de oro, inmutable y no negativa, que se compara por contenido, como `Reward`.
- `Account` es una *entidad* y la raíz de su agregado: la cuenta de un aventurero, identificada por el `AdventurerId` de su dueño. Tiene un saldo y una operación, `credit`.
- `LedgerEntry` es un asiento: un *record* inmutable con su propio identificador. Es una entidad que, por diseño, nunca cambia.

10.4.1 El dominio

Empieza por los identificadores. Son *copias propias*: Treasury no importa nada de los otros servicios, y su `AdventurerId` y su `QuestId` son sus propios tipos (como figura en el árbol de la verificación de estructura 5):

domain/AdventurerId.java

```
package com.guildboard.treasury.domain;

public record AdventurerId(String value) { }
```

domain/QuestId.java

```
package com.guildboard.treasury.domain;

public record QuestId(String value) { }
```

domain/Gold.java

```
package com.guildboard.treasury.domain;

// Una cantidad de oro: inmutable y nunca negativa.
public record Gold(int coins) {

    public static final Gold ZERO = new Gold(0);

    public Gold {
        if (coins < 0) {
            throw new IllegalArgumentException(
                "gold must not be negative: " + coins);
        }
    }

    public Gold plus(Gold other) {
        return new Gold(Math.addExact(coins, other.coins));
    }
}
```

El asiento. Su constructor rechaza datos mal formados (un pago de cero o negativo no es un pago) con `IllegalArgumentException`, como el resto del curso:

domain/LedgerEntry.java

```
package com.guildboard.treasury.domain;

import java.time.Instant;
import java.util.Objects;

// Un asiento del libro: el pago de la recompensa de una misi^^c3^^b3n.
public record LedgerEntry(
        String id,
        QuestId questId,
        AdventurerId adventurerId,
```

```
        Gold amount,
        Instant createdAt) {

    public LedgerEntry {
        if (id == null || id.isBlank()) {
            throw new IllegalArgumentException("id must not be blank");
        }
        Objects.requireNonNull(questId, "questId");
        Objects.requireNonNull(adventurerId, "adventurerId");
        Objects.requireNonNull(amount, "amount");
        Objects.requireNonNull(createdAt, "createdAt");
        if (amount.coins() <= 0) {
            throw new IllegalArgumentException(
                "a payment must be positive: " + amount.coins());
        }
    }
}
```

La cuenta. Como `Adventurer`, tiene constructor privado y una fábrica, y Lombok pone los *getters* (`getOwner()`, `getBalance()`):

domain/Account.java

```
package com.guildboard.treasury.domain;

import java.util.Objects;
import lombok.EqualsAndHashCode;
import lombok.Getter;

@Getter
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
public final class Account {

    @EqualsAndHashCode.Include
    private final AdventurerId owner;
    private Gold balance;

    private Account(AdventurerId owner) {
        this.owner = owner;
        this.balance = Gold.ZERO;
    }

    public static Account open(AdventurerId owner) {
        Objects.requireNonNull(owner, "owner");
        return new Account(owner);
    }

    public void credit(Gold amount) {
        if (amount.coins() <= 0) {
            throw new IllegalArgumentException(
                "credit must be positive: " + amount.coins());
        }
        balance = balance.plus(amount);
    }
}
```

Y los errores de dominio. Treasury solo tiene uno por ahora, el que usa la idempotencia: la misión ya se pagó. Es el mismo molde que en Adventurers (un *record* dentro de una interfaz sellada, *switch* exhaustivo y sin `default`):

domain/DomainError.java

```
package com.guildboard.treasury.domain;

public sealed interface DomainError {

    record QuestAlreadyPaid(QuestId questId) implements DomainError { }

    default String code() {
        return switch (this) {
            case QuestAlreadyPaid e -> "QUEST_ALREADY_PAID";
        };
    }
}
```

domain/DomainException.java

```
package com.guildboard.treasury.domain;

public final class DomainException extends RuntimeException {

    private final DomainError error;

    public DomainException(DomainError error) {
        super(describe(error));
        this.error = error;
    }

    public DomainError getError() {
        return error;
    }

    private static String describe(DomainError error) {
        return switch (error) {
            case DomainError.QuestAlreadyPaid(QuestId id) ->
                "quest already paid: " + id.value();
        };
    }
}
```

10.4.2 La aplicación: puertos y casos de uso

Los puertos de entrada son el comando y dos casos de uso. `PayRewardUseCase` devuelve el asiento que acaba de anotar:

application/port/in/PayRewardCommand.java

```
package com.guildboard.treasury.application.port.in;

import com.guildboard.treasury.domain.AdventurerId;
```

```
import com.guildboard.treasury.domain.Gold;
import com.guildboard.treasury.domain.QuestId;

public record PayRewardCommand(
        QuestId questId, AdventurerId adventurerId, Gold amount) { }
```

application/port/in/PayRewardUseCase.java

```
package com.guildboard.treasury.application.port.in;

import com.guildboard.treasury.domain.LedgerEntry;

public interface PayRewardUseCase {

    LedgerEntry pay(PayRewardCommand command);
}
```

application/port/in/GetAccountUseCase.java

```
package com.guildboard.treasury.application.port.in;

import com.guildboard.treasury.domain.Account;
import com.guildboard.treasury.domain.AdventurerId;
import java.util.Optional;

public interface GetAccountUseCase {

    Optional<Account> get(AdventurerId adventurerId);
}
```

Y los de salida. Fíjate en que `LedgerRepository` ofrece ya `existsByQuestId` (la pregunta «¿está pagada esta misión?», que usará la idempotencia) y `findByAdventurerId` (leer el libro de una persona, que usa el ejercicio 10.3). El servicio de esta sección no usa ninguna de las dos:

application/port/out/AccountRepository.java

```
package com.guildboard.treasury.application.port.out;

import com.guildboard.treasury.domain.Account;
import com.guildboard.treasury.domain.AdventurerId;
import java.util.Optional;

public interface AccountRepository {

    Optional<Account> findById(AdventurerId adventurerId);

    void save(Account account);
}
```

application/port/out/LedgerRepository.java

```
package com.guildboard.treasury.application.port.out;

```

```
import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.LedgerEntry;
import com.guildboard.treasury.domain.QuestId;
import java.util.List;

public interface LedgerRepository {

    boolean existsByQuestId(QuestId questId);

    void save(LedgerEntry entry);

    List<LedgerEntry> findByAdventurerId(AdventurerId adventurerId);
}
```

Ahora el caso de uso de pagar. Esta es la **versión ingenua**, deliberadamente: construye el asiento, lo guarda, abre la cuenta si no existía y la acredita. El identificador del asiento lo genera el servicio, como los demás identificadores del curso.

application/service/PayRewardService.java (versión ingenua)

```
package com.guildboard.treasury.application.service;

import com.guildboard.treasury.application.port.in.PayRewardCommand;
import com.guildboard.treasury.application.port.in.PayRewardUseCase;
import com.guildboard.treasury.application.port.out.AccountRepository;
import com.guildboard.treasury.application.port.out.LedgerRepository;
import com.guildboard.treasury.domain.Account;
import com.guildboard.treasury.domain.LedgerEntry;
import java.time.Clock;
import java.util.UUID;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class PayRewardService implements PayRewardUseCase {

    private final AccountRepository accounts;
    private final LedgerRepository ledger;
    private final Clock clock;

    @Override
    public LedgerEntry pay(PayRewardCommand command) {
        LedgerEntry entry = new LedgerEntry(
            UUID.randomUUID().toString(), command.questId(),
            command.adventurerId(), command.amount(), clock.instant());
        ledger.save(entry);
        Account account = accounts.findById(command.adventurerId())
            .orElseGet(() -> Account.open(command.adventurerId()));
        account.credit(command.amount());
        accounts.save(account);
        return entry;
    }
}
```

La consulta de la cuenta es de una línea de lógica:

application/service/GetAccountService.java

```java
package com.guildboard.treasury.application.service;

import com.guildboard.treasury.application.port.in.GetAccountUseCase;
import com.guildboard.treasury.application.port.out.AccountRepository;
import com.guildboard.treasury.domain.Account;
import com.guildboard.treasury.domain.AdventurerId;
import java.util.Optional;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class GetAccountService implements GetAccountUseCase {

    private final AccountRepository accounts;

    @Override
    public Optional<Account> get(AdventurerId adventurerId) {
        return accounts.findById(adventurerId);
    }
}
```

Una excepción a «un caso de uso, un agregado»

Pagar escribe en dos sitios: añade un asiento al libro y sube el saldo de la cuenta. El capítulo 5 fijó que cada caso de uso modifica como máximo un agregado, y el capítulo 9 se apoyó en esa regla para cortar sin sagas. Aquí no se cumple del todo, y es mejor decirlo que disimularlo. Lo que hace el curso:

- **El orden importa**: primero el asiento, después el saldo. Si el proceso muere entre las dos escrituras, queda un asiento sin reflejar en el saldo, y eso se puede reparar; lo contrario (saldo subido sin asiento) sería dinero sin explicación.
- **El libro es la verdad y el saldo, una copia**. El saldo guardado se puede recalcular sumando los asientos; el ejercicio 10.3 lo hace.
- **El riesgo queda reconocido y sin resolver.** Tras ese fallo, un reintento diría «ya pagado» (sección siguiente) y no repararía el saldo. Además, dos pagos *distintos* al mismo aventurero a la vez pueden pisarse: ambos leen el saldo 100 y ambos escriben 250 en lugar de 400. Es la misma clase de carrera que la del capítulo 5; resolverla pide transacciones entre dos escrituras o bloqueos, y queda fuera del curso.

Una alternativa válida sería no guardar el saldo y calcularlo al leer (un solo sitio de escritura, el libro), a cambio de leer todos los asientos en cada consulta. El curso guarda el saldo para tener una copia que el libro pueda contradecir.

10.5 Idempotencia: pagar una misión una sola vez

Una operación es *idempotente* si aplicarla una vez o varias deja el sistema en el mismo estado. Para Treasury: «pagar la misión Q» ocurre una vez aunque la orden llegue tres. Las órdenes duplicadas no son un caso raro:

- Un cliente no recibe la respuesta (se agotó el tiempo) y reenvía.
- El capítulo 12 añade reintentos automáticos a las llamadas entre servicios.

- Un *broker* de mensajería, como el del capítulo 11, entrega los mensajes *al menos una vez*: duplicar es parte del contrato.

El catálogo de Richardson tiene un patrón con este nombre para el lado que recibe los mensajes, «Idempotent Consumer» [42]. Aquí lo aplicamos al pago, con una clave natural: una misión, un pago. La clave es el `questId`.

Con la versión ingenua de la sección anterior, el problema se ve en cuanto la conectes a un endpoint (sección 10.7). Salida *esperada* de pagar dos veces la misma misión (no la he ejecutado; lo comprobarás tú en el ejercicio 10.2):

```
$ curl -s -X POST localhost:8083/payments ... (questId q-1, 150) -> 201
$ curl -s -X POST localhost:8083/payments ... (questId q-1, 150) -> 201
$ curl -s localhost:8083/accounts/$MARA
{"adventurerId":"7d1c4e52-...","balance":300}
```

Dos asientos y 300 de oro por una misión de 150. Hay dos defensas, y el capítulo usa las dos:

	Comprobación previa	**Restricción única**
Dónde	En `PayRewardService`: si `existsByQuestId` es cierto, lanza `QuestAlreadyPaid` sin escribir nada.	En la base de datos: una restricción `UNIQUE` sobre `quest_id`, creada por una migración `V2`.
Qué cubre	Duplicados que llegan uno detrás de otro. Da una respuesta limpia y barata.	También los que llegan *a la vez*: dos peticiones comprueban, las dos ven «no está pagada» y las dos insertan. La base de datos es la única que ve a todos los escritores y rechaza la segunda.
Qué cuesta	Una consulta más por pago.	Una migración y traducir el error de la base a un error de dominio.

La primera es la defensa de cortesía; la segunda, la garantía. Ninguna sobra.

Qué responde la segunda petición. Treasury responde `409 Conflict` con el código `QUEST_ALREADY_PAID` (el error de dominio de arriba). Otras APIs devuelven el `200` con el resultado original; son dos políticas válidas mientras estén documentadas. Elegimos el `409` porque hace visible la repetición, y el consumidor del capítulo 11 la tratará como «ya hecho» y dará el mensaje por bueno.

Qué hace falta en la base de datos. Hay una trampa de Flyway (capítulo 8) que evitar: la restricción *no* se añade editando `V1`. Una migración aplicada no se edita: Flyway guarda su *checksum* y rechazaría el cambio en cualquier base donde ya corrió. Se crea `V2__unique_quest_payment.sql`. Y una advertencia: si la tabla ya contiene dos asientos con el mismo `quest_id` (por ejemplo, del servicio ingenuo), `V2` falla al intentar crear la restricción. Hay que decidir qué hacer con los duplicados antes de migrar; no hay forma automática de saber cuál era el «bueno».

Qué hace falta saber de Spring para el segundo escudo. Dos detalles que el capítulo 8 no necesitó:

1. Spring traduce los errores de la base de datos a su propia jerarquía, que no depende de JPA ni del driver. Una violación de una restricción única llega al adaptador como `DataIntegrityViolationException`, del paquete `org.springframework.dao` (verificado en el *jar* `spring-tx` 7.0.9 de Maven Central, el 9 de octubre de 2026).

2. JPA no envía el `INSERT` cuando llamas a `save`: lo envía al *vaciar* (*flush*) la sesión, normalmente al confirmar la transacción, que ocurre *después* de que el método del adaptador termine. Un `catch` dentro del método no vería el error. Spring Data ofrece `saveAndFlush` (en `JpaRepository`, verificado en `spring-data-jpa` 4.1.1), que guarda y envía en el acto.

La forma del adaptador será, entonces, esta; escribirla completa y conectarla al servicio es el ejercicio 10.2:

Forma de la traducción (no es el archivo completo)

```
try {
    jpa.saveAndFlush(entity);
} catch (DataIntegrityViolationException e) {
    throw new DomainException(
        new DomainError.QuestAlreadyPaid(entry.questId()));
}
```

(El adaptador está en `adapters`, que ve el dominio; el dominio no ve nada de Spring. La traducción vive donde vive el detalle técnico.)

10.6 Persistencia de Treasury: memoria y JPA

Treasury sigue el patrón del capítulo 8: dos adaptadores por puerto de salida, uno en memoria (por defecto, arranca sin Docker) y otro con JPA (perfil `postgres`), y las dos clases de configuración que eligen entre ellos. No se añade ninguna dependencia ni se vuelve a explicar el mecanismo; si algo de lo que sigue te suena a nuevo, repasa las secciones 8.4 y 8.5 del capítulo 8.

La parte *del libro* la escribimos aquí, porque la tocan los ejercicios de idempotencia y de saldo. La parte *de la cuenta* y los adaptadores en memoria son por analogía y los escribes en el ejercicio 10.5. Primero el esquema. `V1` crea las dos tablas y **no** lleva restricción única sobre `quest_id`: es la versión ingenua. Un índice sobre `adventurer_id` acelera leer el libro de una persona. Sin clave foránea: los aventureros y las misiones son de otros servicios. SQL probado contra PostgreSQL 18 en un contenedor.

bootstrap/.../db/migration/V1__create_treasury_tables.sql

```
-- V1 de Treasury: la cuenta de cada aventurero y el libro de pagos.
-- Sin clave for^^c3^^a1nea: aventureros y misiones son de otros servicios.
-- Sin restricci^^c3^^b3n ^^c3^^banica sobre quest_id: la a^^c3^^b1ade V2.

CREATE TABLE account (
    adventurer_id  VARCHAR(64)  PRIMARY KEY,
    balance        INTEGER      NOT NULL
);

CREATE TABLE ledger_entry (
    id             VARCHAR(64)  PRIMARY KEY,
    quest_id       VARCHAR(64)  NOT NULL,
    adventurer_id  VARCHAR(64)  NOT NULL,
    amount         INTEGER      NOT NULL,
    created_at     TIMESTAMP WITH TIME ZONE NOT NULL
);

```

```
CREATE INDEX idx_ledger_entry_adventurer
    ON ledger_entry (adventurer_id);
```

Ahora el adaptador del libro, en `adapter/out/persistence` (con visibilidad de paquete para el mapeador, como en los otros servicios). La entidad:

adapter/out/persistence/LedgerEntryEntity.java

```
package com.guildboard.treasury.adapter.out.persistence;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.time.Instant;
import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "ledger_entry")
@Getter
@AllArgsConstructor
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class LedgerEntryEntity {

    @Id
    @Column(name = "id")
    private String id;

    @Column(name = "quest_id", nullable = false)
    private String questId;

    @Column(name = "adventurer_id", nullable = false)
    private String adventurerId;

    @Column(name = "amount", nullable = false)
    private int amount;

    @Column(name = "created_at", nullable = false)
    private Instant createdAt;
}
```

El repositorio de Spring Data, con dos consultas derivadas del nombre (`existsByQuestId` y `findByAdventurerId`: Spring Data las genera a partir del nombre de los campos `questId` y `adventurerId` de la entidad):

adapter/out/persistence/LedgerJpaRepository.java

```
package com.guildboard.treasury.adapter.out.persistence;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

```

```
public interface LedgerJpaRepository
        extends JpaRepository<LedgerEntryEntity, String> {

    boolean existsByQuestId(String questId);

    List<LedgerEntryEntity> findByAdventurerId(String adventurerId);
}
```

El mapeador, a mano. A diferencia de `Quest` y `Adventurer`, `LedgerEntry` es un *record* con constructor público, así que reconstruirlo no necesita repetir transiciones:

adapter/out/persistence/LedgerEntryPersistenceMapper.java

```
package com.guildboard.treasury.adapter.out.persistence;

import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.Gold;
import com.guildboard.treasury.domain.LedgerEntry;
import com.guildboard.treasury.domain.QuestId;

final class LedgerEntryPersistenceMapper {

    private LedgerEntryPersistenceMapper() {
    }

    static LedgerEntryEntity toEntity(LedgerEntry entry) {
        return new LedgerEntryEntity(
            entry.id(),
            entry.questId().value(),
            entry.adventurerId().value(),
            entry.amount().coins(),
            entry.createdAt());
    }

    static LedgerEntry toDomain(LedgerEntryEntity entity) {
        return new LedgerEntry(
            entity.getId(),
            new QuestId(entity.getQuestId()),
            new AdventurerId(entity.getAdventurerId()),
            new Gold(entity.getAmount()),
            entity.getCreatedAt());
    }
}
```

Y el adaptador, ingenuo como el servicio (el ejercicio lo endurece). Cada método es una transacción (`@Transactional` en la clase, de solo lectura en las consultas), y no lleva `@Component`: lo crea la configuración.

adapter/out/persistence/JpaLedgerRepository.java (ingenuo)

```
package com.guildboard.treasury.adapter.out.persistence;

import com.guildboard.treasury.application.port.out.LedgerRepository;
import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.LedgerEntry;
```

```
import com.guildboard.treasury.domain.QuestId;
import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.transaction.annotation.Transactional;

@RequiredArgsConstructor
@Transactional
public class JpaLedgerRepository implements LedgerRepository {

    private final LedgerJpaRepository jpa;

    @Override
    @Transactional(readOnly = true)
    public boolean existsByQuestId(QuestId questId) {
        return jpa.existsByQuestId(questId.value());
    }

    @Override
    public void save(LedgerEntry entry) {
        jpa.save(LedgerEntryPersistenceMapper.toEntity(entry));
    }

    @Override
    @Transactional(readOnly = true)
    public List<LedgerEntry> findByAdventurerId(
            AdventurerId adventurerId) {
        return jpa.findByAdventurerId(adventurerId.value()).stream()
            .map(LedgerEntryPersistenceMapper::toDomain)
            .toList();
    }
}
```

Ahora la cuenta tiene el mismo molde con tres diferencias: la clave es `adventurer_id`; el mapeador *repite transiciones* (abre la cuenta y la acredita con el saldo guardado, como `Adventurer` con la experiencia), y no hay consulta derivada. No la escribo aquí: es una parte del ejercicio 10.5.

10.7 El adaptador REST provisional

Treasury necesita una entrada para poder probarlo. Será un controlador REST con tres elementos del molde de siempre: DTO de entrada y salida (los *enum* ya no hacen falta; todo son cadenas y números), un mapeador estático a mano y un controlador sin lógica. El nombre del endpoint, `POST /payments`, dice qué hace Treasury, no qué framework usa. Es **provisional**: en el capítulo 11 el pago lo dispara un evento y este endpoint se borra. Los tres DTO:

adapter/in/web/dto/PayRewardRequest.java

```
package com.guildboard.treasury.adapter.in.web.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;

public record PayRewardRequest(
        @NotBlank String questId,
        @NotBlank String adventurerId,
```

```
        @Positive int amount) {
}
```

adapter/in/web/dto/PaymentResponse.java

```
package com.guildboard.treasury.adapter.in.web.dto;

public record PaymentResponse(
        String id,
        String questId,
        String adventurerId,
        int amount,
        String createdAt) {
}
```

adapter/in/web/dto/AccountResponse.java

```
package com.guildboard.treasury.adapter.in.web.dto;

public record AccountResponse(String adventurerId, int balance) {
}
```

adapter/in/web/mapper/TreasuryWebMapper.java

```
package com.guildboard.treasury.adapter.in.web.mapper;

import com.guildboard.treasury.adapter.in.web.dto.AccountResponse;
import com.guildboard.treasury.adapter.in.web.dto.PayRewardRequest;
import com.guildboard.treasury.adapter.in.web.dto.PaymentResponse;
import com.guildboard.treasury.application.port.in.PayRewardCommand;
import com.guildboard.treasury.domain.Account;
import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.Gold;
import com.guildboard.treasury.domain.LedgerEntry;
import com.guildboard.treasury.domain.QuestId;

public final class TreasuryWebMapper {

    private TreasuryWebMapper() {
    }

    public static PayRewardCommand toCommand(PayRewardRequest request) {
        return new PayRewardCommand(
            new QuestId(request.questId()),
            new AdventurerId(request.adventurerId()),
            new Gold(request.amount()));
    }

    public static PaymentResponse toResponse(LedgerEntry entry) {
        return new PaymentResponse(
            entry.id(),
            entry.questId().value(),
            entry.adventurerId().value(),
            entry.amount().coins(),
```

```
            entry.createdAt().toString());
    }

    public static AccountResponse toResponse(Account account) {
        return new AccountResponse(
            account.getOwner().value(), account.getBalance().coins());
    }
}
```

Y el controlador. `POST /payments` responde `201` con el asiento; `GET /accounts/{adventurerId}` responde `200` con el saldo o `404` sin cuerpo si esa persona nunca ha cobrado (la cuenta se abre en el primer pago). La repetición de un pago llegará como `DomainException` y el manejador de errores la traducirá a `409`; ese manejador es la copia reducida del de Adventurers que escribes en el ejercicio de proyecto.

adapter/in/web/TreasuryController.java

```
package com.guildboard.treasury.adapter.in.web;

import com.guildboard.treasury.adapter.in.web.dto.AccountResponse;
import com.guildboard.treasury.adapter.in.web.dto.PayRewardRequest;
import com.guildboard.treasury.adapter.in.web.dto.PaymentResponse;
import com.guildboard.treasury.adapter.in.web.mapper.TreasuryWebMapper;
import com.guildboard.treasury.application.port.in.GetAccountUseCase;
import com.guildboard.treasury.application.port.in.PayRewardUseCase;
import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.LedgerEntry;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequiredArgsConstructor
public class TreasuryController {

    private final PayRewardUseCase payReward;
    private final GetAccountUseCase getAccount;

    // Provisional: lo sustituye el evento del cap^^c3^^adtulo 11.
    @PostMapping("/payments")
    public ResponseEntity<PaymentResponse> pay(
            @Valid @RequestBody PayRewardRequest request) {
        LedgerEntry entry =
            payReward.pay(TreasuryWebMapper.toCommand(request));
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(TreasuryWebMapper.toResponse(entry));
    }

    @GetMapping("/accounts/{adventurerId}")
    public ResponseEntity<AccountResponse> account(
```

```
40              @PathVariable String adventurerId) {
41          return getAccount.get(new AdventurerId(adventurerId))
42              .map(TreasuryWebMapper::toResponse)
43              .map(ResponseEntity::ok)
44              .orElseGet(() -> ResponseEntity.notFound().build());
45      }
46  }
```

10.8 Pruebas que vigilan los servicios

Cuando había dos servicios, la comprobación «¿algún `build.gradle` nombra a otro servicio?» se hacía leyendo. Con tres, y con un cuarto a la vista, leer ya no es una comprobación: es una esperanza. Lo que se repite se vigila con una prueba, igual que el capítulo 3 hizo con la regla de dependencia. Hacen falta dos, porque vigilan cosas distintas:

- **Que cada servicio tiene su forma**: los cuatro módulos, cada uno con su `build.gradle`. Eso son archivos y carpetas, no código Java: lo mira un `StructureTest` con solo el JDK (`java.nio.file`).
- **Que un servicio no usa clases de otro**: eso sí es código, y lo mira una regla de ArchUnit con *slices*, dentro del `ArchitectureTest` de cada servicio.

10.8.1 java.nio.file mínimo

`java.nio.file` viene con el JDK; no hay nada que instalar. Cuatro piezas bastan para esta prueba:

- `Path.of("services")` crea una ruta (relativa al directorio de trabajo) y `ruta.resolve("domain")` le añade un trozo.
- `Files.isDirectory(ruta)` dice si es una carpeta.
- `Files.exists(ruta)` dice si existe un archivo o una carpeta.
- `Files.list(ruta)` devuelve un `Stream<Path>` con lo que cuelga de una carpeta. Mantiene la carpeta abierta, así que se cierra: se usa dentro de un *try-with-resources*. Lanza `IOException`, que el método de prueba declara con `throws`.

Gradle ejecuta las pruebas de un proyecto con su carpeta como directorio de trabajo. Para la raíz, esa carpeta es la raíz del repositorio, y por eso `Path.of("services")` basta [16].

10.8.2 La raíz gana el plugin java y JUnit

Hasta ahora la raíz no tenía código ni pruebas: solo configuraba a los subproyectos. Para alojar `StructureTest` necesita el plugin `java` y las dependencias de JUnit. Como la raíz no aplica el BOM de Boot (el bloque `subprojects` solo afecta a los módulos), JUnit lleva aquí la versión explícita, igual que en el capítulo 3 [23].

☞ JUnit en la raíz

No hay nada nuevo que descargar: son las dependencias del capítulo 3 puestas en `build.gradle` de la raíz, que gana un plugin y tres bloques. Añade `id 'java'` a `plugins` y, *fuera* de `subprojects`, el resto. Abajo tienes el archivo completo, con el `subprojects`

del capítulo 7 intacto.

build.gradle (raíz)

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.1' apply false
}

repositories {
    mavenCentral()
}

dependencies {
    testImplementation platform('org.junit:junit-bom:6.0.3')
    testImplementation 'org.junit.jupiter:junit-jupiter'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

tasks.named('test', Test) {
    useJUnitPlatform()
}

subprojects {
    apply plugin: 'java-library'
    apply plugin: 'io.spring.dependency-management'

    group = 'com.guildboard'
    version = '0.0.1-SNAPSHOT'

    java {
        toolchain {
            languageVersion = JavaLanguageVersion.of(21)
        }
    }

    repositories {
        mavenCentral()
    }

    dependencyManagement {
        imports {
            mavenBom(org.springframework.boot.gradle.plugin
                     .SpringBootPlugin.BOM_COORDINATES)
        }
    }

    dependencies {
        testImplementation 'org.springframework.boot:spring-boot-starter-test'
        testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
    }

    tasks.named('test') {
        useJUnitPlatform()
    }
}
```

10.8.3 StructureTest

Crea `src/test/java/com/guildboard/StructureTest.java` en la raíz. Hay un solo método. Recorre las carpetas de `services/`, acumula *todos* los problemas y falla una vez con la lista, para que veas todo de golpe:

src/test/java/com/guildboard/StructureTest.java

```
package com.guildboard;

import static org.junit.jupiter.api.Assertions.assertTrue;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Stream;
import org.junit.jupiter.api.Test;

public class StructureTest {

    private static final Path SERVICES = Path.of("services");

    private static final List<String> MODULES =
        List.of("domain", "application", "adapters", "bootstrap");

    @Test
    void everyServiceHasItsFourModules() throws IOException {
        assertTrue(Files.isDirectory(SERVICES),
            "no existe la carpeta " + SERVICES.toAbsolutePath());

        List<String> problems = new ArrayList<>();
        try (Stream<Path> children = Files.list(SERVICES)) {
            List<Path> services = children
                .filter(Files::isDirectory)
                .filter(p -> !p.getFileName().toString().startsWith("."))
                .sorted()
                .toList();
            if (services.isEmpty()) {
                problems.add("services/ no contiene ning^^c3^^ban servicio");
            }
            for (Path service : services) {
                for (String module : MODULES) {
                    Path buildFile =
                        service.resolve(module).resolve("build.gradle");
                    if (!Files.exists(buildFile)) {
                        problems.add("falta " + buildFile);
                    }
                }
            }
        }

        assertTrue(problems.isEmpty(), () ->
            "La estructura de servicios no se cumple:\n  "
            + String.join("\n  ", problems));
    }
```

```
50 }
```

Solo comprueba que *existen* los cuatro `build.gradle` de cada servicio, no lo que contienen. Un `bootstrap` sin carpeta de módulo no se confunde con uno vacío: falta el archivo y la prueba lo nombra.

10.8.4 Independencia entre servicios con slices

ArchUnit llama *slice* a un trozo del código que se agrupa por un patrón de paquete [66]. En `"com.guildboard.(*).."` el paréntesis captura el segundo nivel del paquete: `com.guildboard.quests.domain.Quest` está en el slice `quests`, y `com.guildboard.treasury.domain.Account`, en `treasury`. La regla dice que ningún slice depende de otro:

Regla de independencia (método nuevo de cada ArchitectureTest)

```
    @Test
    void servicesDoNotDependOnEachOther() {
        JavaClasses allClasses = new ClassFileImporter()
            .withImportOption(new ImportOption.DoNotIncludeTests())
            .importPackages("com.guildboard");

        slices().matching("com.guildboard.(*)..")
            .should().notDependOnEachOther()
            .check(allClasses);
    }
```

Y este `import` estático, junto a los del archivo:

Import nuevo

```
import static com.tngtech.archunit.library.dependencies.SlicesRuleDefinition.
    ↪ slices;
```

La regla importa `com.guildboard` entero, no la constante `BASE_PACKAGE` de la clase, porque necesita ver *todos* los servicios que haya en el *classpath*. Y aquí está la clave de por qué no hace falta filtrar nada: el *classpath* de pruebas del `bootstrap` de Quests contiene las clases de Quests y de sus módulos, y ninguna de Adventurers ni de Treasury, que son otros proyectos Gradle. Mientras nadie añada `implementation project(':treasury-domain')` a Quests, el único slice que ve la regla es `quests`, y pasa. En el momento en que alguien lo añada y use una clase de Treasury, el slice `treasury` aparece en el *classpath*, Quests lo usa y la prueba falla. Lo mismo vale para `gateway` y `config-server` (capítulo 13): no llevan `ArchitectureTest`, así que la regla nunca los ve.

La limitación, dicha claramente: **ArchUnit lee bytecode**. Una dependencia declarada en un `build.gradle` y que ninguna clase usa no deja rastro en el bytecode, y la regla no la ve. Es un acoplamiento latente que solo esta comprobación manual (o el `grep` de la sección anterior) atrapa. Y como siempre, una prueba que no has visto fallar no sabes si funciona: el ejercicio 10.4 te pide romperlas.

10.8.5 El ArchitectureTest de Treasury

El `ArchitectureTest` de Treasury forma parte de la plantilla. Es una copia del de Adventurers cuyo único dato propio es el paquete, igual que el de Adventurers lo era del de Quests (capítulo 9):

```
$ A=services/adventurers/bootstrap/src/test/java/com/guildboard/adventurers
$ T=services/treasury/bootstrap/src/test/java/com/guildboard/treasury
$ mkdir -p $T
$ sed -e 's/com\.guildboard\.adventurers/com.guildboard.treasury/g' \
    $A/ArchitectureTest.java > $T/ArchitectureTest.java
```

Añade antes la prueba de slices de arriba (con su `import`) a los `ArchitectureTest` de Quests y de Adventurers: así la copia ya la lleva. Aún no puedes ejecutar el de Treasury: mientras el servicio esté vacío, la selección de clases de las reglas queda vacía y ArchUnit falla a propósito. Pasará cuando exista el dominio, al final del capítulo. Con `./gradlew test` se ejecutan a la vez el `StructureTest` de la raíz y los tres `ArchitectureTest`.

Qué no se ha ejecutado

Como en el resto del libro, este código Java está revisado pero no compilado ni ejecutado: ni `StructureTest` ni la regla de slices. Lo que sí se ha tomado de la guía de ArchUnit [66] es la forma de `slices()`, `matching(...)` y `notDependOnEachOther()`.

10.9 En el proyecto: hito H5 (parte 2)

Hito 5, parte 2: tres servicios

Al terminar el ejercicio 10.5, GuildBoard son tres aplicaciones, cada una con su base de datos:

- **Quests** (`services/quests`, 8081, base `quests`) y **Adventurers** (`services/adventurers`, 8082, base `adventurers`): sin cambios.
- **Treasury** (`services/treasury`, 8083, base `treasury`): paga la recompensa de una misión una sola vez, guarda el libro y el saldo, y expone `POST /payments` (provisional) y `GET /accounts/{adventurerId}` (más `GET /accounts/{adventurerId}/ledger-balance` del ejercicio 10.3).

Y dos pruebas que vigilan que los tres tengan la misma forma (`StructureTest`) y no dependan unos de otros (la regla de slices de cada `ArchitectureTest`). Dos cosas siguen pendientes por diseño: nadie le manda aún a Treasury los pagos (lo hará el evento del capítulo 11; hoy los mandas tú con `curl`) y el pago sigue sin ser atómico (sección 10.4).

10.9.1 Probar el hito

Sin Docker, en modo memoria, una terminal por servicio (el orden no importa a Treasury, que no llama a nadie):

```
$ ./gradlew :adventurers-bootstrap:bootRun        # terminal 1
$ ./gradlew :quests-bootstrap:bootRun             # terminal 2
$ ./gradlew :treasury-bootstrap:bootRun           # terminal 3
```

En una cuarta, haz de «mensajero» entre servicios: registra a Mara, completa una misión y paga su recompensa a mano. Los ids son los tuyos:

```
$ curl -s -X POST localhost:8082/adventurers \
    -H "Content-Type: application/json" -d '{"name":"Mara"}'
$ MARA=7d1c4e52-...
$ curl -s -X POST localhost:8081/quests \
    -H "Content-Type: application/json" \
    -d '{"title":"Limpiar el s^^c3^^b3tano","reward":150,
         "requiredRank":"NOVICE"}'
$ Q1=3f9a...
$ curl -s -X POST localhost:8081/quests/$Q1/accept \
    -H "Content-Type: application/json" \
    -d "{\"adventurerId\":\"$MARA\"}"
$ curl -s -X POST localhost:8081/quests/$Q1/complete
```

Hasta aquí, lo del capítulo anterior. Ahora el nuevo trozo: pagar en Treasury, mirar la cuenta y repetir el pago.

```
$ curl -s -X POST localhost:8083/payments \
    -H "Content-Type: application/json" \
    -d "{\"questId\":\"$Q1\",\"adventurerId\":\"$MARA\",\"amount\":150}"
{"id":"b81e...","questId":"3f9a...","adventurerId":"7d1c...",
 "amount":150,"createdAt":"2026-10-09T10:15:00.123Z"}
$ curl -s localhost:8083/accounts/$MARA
{"adventurerId":"7d1c4e52-...","balance":150}
$ curl -s -X POST localhost:8083/payments \
    -H "Content-Type: application/json" \
    -d "{\"questId\":\"$Q1\",\"adventurerId\":\"$MARA\",\"amount\":150}"
{"title":"Business rule violated","status":409,
 "detail":"quest already paid: 3f9a...","code":"QUEST_ALREADY_PAID"}
$ curl -s localhost:8083/accounts/$MARA
{"adventurerId":"7d1c4e52-...","balance":150}
```

(Salidas resumidas y *esperadas*; no he ejecutado el código Java del libro.) El saldo sigue en 150 tras el segundo pago. Un aventurero que nunca ha cobrado da `404` en `GET /accounts/...`. Un pago con `"amount":0` o sin `questId` da `400`.

Con Docker y el perfil `postgres`, tras el `down -v` de la sección 10.3:

```
$ docker compose up -d --wait
$ SPRING_PROFILES_ACTIVE=postgres \
    ./gradlew :treasury-bootstrap:bootRun
$ docker compose exec postgres psql -U guildboard -d treasury -c '\dt'
 Schema |         Name          | Type  |   Owner
--------+-----------------------+-------+------------
 public | account               | table | guildboard
 public | flyway_schema_history | table | guildboard
 public | ledger_entry          | table | guildboard
$ docker compose exec postgres psql -U guildboard -d treasury -Atc \
    "select version, description from flyway_schema_history"
1|create treasury tables
2|unique quest payment
```

(Salida aproximada.) Tres bases, tres historiales de Flyway. Comprueba con `./gradlew test` y `./gradlew build` que todo está en verde y haz el *commit* (`H5b: Treasury y sus pruebas`). Con `git diff -M --stat` contra el *commit* anterior, solo `settings.gradle`, `build.gradle`, `init.sql` y los `ArchitectureTest` de Quests y Adventurers (la regla de slices) aparecen modificados; todo lo demás es nuevo y vive en `services/treasury`. Eso es lo que cuesta añadir un servicio cuando la plantilla está clara.

10.10 Ejercicios

Los ejercicios parten del proyecto tal como lo dejan las secciones anteriores y se hacen en este orden. Los cuatro primeros no necesitan Treasury en marcha: lo escribes o lo lees, y lo compruebas *en ejecución* cuando termines el último, el de proyecto, que lo junta todo.

Ejercicio 10.1: Comprobación de la estructura 6: molde y contenido ★★☆☆☆

Este ejercicio practica las comprobaciones de la verificación de estructura 6. Estos son los interiores de Quests y de Treasury, resumidos:

`Quests (com.guildboard.quests)`

```
domain/       Quest, QuestId, QuestStatus, Reward, Rank, AdventurerId,
              DomainError (7), DomainException
application/  port/in: 4 casos de uso
              port/out: QuestRepository, NotificationSender,
                        AdventurerDirectory, AdventurerView
              service: 4 servicios
adapters/     in/web: QuestController + dto, mapper, error
              out: memory, notification, persistence, rest
bootstrap/    QuestsApplication, config/ (BeanConfig y dos
              *PersistenceConfig), application.yml,
              application-postgres.yml
              db/migration: V1 (quests), V2 (^^c3^^adndice)
```

`Treasury (com.guildboard.treasury)`

```
domain/       Account, LedgerEntry, Ledger, Gold, AdventurerId, QuestId,
              DomainError (1), DomainException
application/  port/in: PayReward*, GetAccountUseCase, GetLedgerBalance*
              port/out: AccountRepository, LedgerRepository
              service: 3 servicios
adapters/     in/web: 2 controladores + dto, mapper, error
              out: memory, persistence
bootstrap/    TreasuryApplication, config/ (BeanConfig y dos
              *PersistenceConfig), application.yml,
              application-postgres.yml
              db/migration: V1 (account, ledger_entry), V2 (^^c3^^banica)
```

Haz tres cosas. **(1)** Copia y rellena una tabla con una fila por elemento, diciendo si es *molde* (existe con la misma forma en todos los servicios), *propio* (es contenido de ese servicio) o ambas cosas (la forma es molde y el contenido, propio), y por qué: los cuatro módulos y su grafo; los paquetes `com.guildboard.<servicio>` con sus capas; `DomainError` y `DomainException`; `Rank` y `AdventurerId` de Quests; `AdventurerDirectory` y `rest`; las dos clases `*PersistenceConfig`; los dos `application*.yml`; `V1`; `Account` y `LedgerEntry`;

el `ArchitectureTest` de cada `bootstrap`. **(2)** Para añadir un cuarto servicio, ¿qué archivos *fuera* de su carpeta hay que tocar? **(3)** Nombra dos cosas del molde que `StructureTest` y la regla de slices no comprueban. Está bien si: la tabla distingue la *forma* del *contenido* en las configuraciones, el `DomainError` y `V1`; marca como propios `Account`, `LedgerEntry`, `rest` y `AdventurerDirectory`; la respuesta a (2) son exactamente `settings.gradle` e `init.sql`; y la de (3) menciona al menos dos de: los nombres de paquete, la existencia de las clases de configuración o los `yml`, que el paquete base de cada `ArchitectureTest` coincida con el servicio, o los puertos.

Pista: *una forma que existe en los tres servicios es molde aunque su contenido sea distinto; un archivo que solo tiene sentido en un servicio es propio.*

Lectura de código ▪ solución en la pág. 371

Ejercicio 10.2: Pagar dos veces la misma misión ★★★☆☆

Parte del Treasury de las secciones 10.4 a 10.6: el servicio ingenuo, sin restricción única. Haz que pagar dos veces la misma misión no acredite dos veces, con las dos defensas de la sección 10.5:

1. Modifica `PayRewardService`: si `existsByQuestId` es cierto, lanza `DomainException` con `QuestAlreadyPaid` sin escribir nada.
2. Crea la migración `V2__unique_quest_payment.sql` con una restricción `UNIQUE` sobre `quest_id`. No toques `V1`.
3. Modifica `JpaLedgerRepository.save` para que, si la base rechaza el asiento por duplicado, lance el mismo error de dominio.
4. Contesta: ¿por qué hacen falta las dos defensas?, y ¿qué ocurre al aplicar `V2` en una base que ya tiene dos asientos con el mismo `quest_id`?

Está bien si: el servicio lanza `QuestAlreadyPaid` antes de llamar a `ledger.save` y a `accounts.save`; `V2` existe y `V1` no ha cambiado; el adaptador usa `saveAndFlush` y traduce `DataIntegrityViolationException`; `domain` y `application` no importan Spring; y, cuando tengas Treasury en marcha (ejercicio 10.5), el segundo `POST /payments` de una misma misión da `409 QUEST_ALREADY_PAID` y el saldo no cambia, y con el perfil `postgres` insertar a mano dos filas con el mismo `quest_id` da `duplicate key value violates unique constraint`.

Pista: *el `catch` solo ve el error si el `INSERT` se envía dentro del método; mira la sección 10.5.*

Práctica ▪ solución en la pág. 372

Ejercicio 10.3: El saldo, calculado desde el libro ★★★☆☆

La cuenta guarda un saldo, pero el libro es la verdad: el saldo debería ser la suma de los asientos. Añade una segunda forma de leerlo:

1. En `domain`, una clase `Ledger` (`final`, constructor privado) con `static Gold balanceOf(List<LedgerEntry> entries)`: la suma de los asientos, y `Gold.ZERO` si no hay ninguno.
2. En `application`, un caso de uso `GetLedgerBalanceUseCase` con `Gold get(AdventurerId adventurerId)` y su servicio, que lee el libro con `LedgerRepository.findByAdventurerId`.
3. Un endpoint `GET /accounts/{adventurerId}/ledger-balance` en un controlador nuevo, `LedgerBalanceController`, que responde `200` con un `AccountResponse` (un

aventurero sin asientos tiene saldo 0).

4. Contesta: ¿cuándo pueden diferir el saldo guardado y el calculado, y cuál crees?

Está bien si: `Ledger` no importa nada de Spring (el `ArchitectureTest` en verde); `Ledger.balanceOf(List.of())` es `Gold.ZERO`; tras dos pagos de 150 y 50 a la misma persona, `/accounts/{id}` y `/accounts/{id}/ledger-balance` devuelven ambos un `balance` de 200; y la respuesta a (4) habla del fallo entre las dos escrituras de la sección 10.4.

Pista: *`stream().map(LedgerEntry::amount).reduce(Gold.ZERO, Gold::plus)` suma objetos de valor sin pasar a `int`.*

Práctica ▪ solución en la pág. 375

Ejercicio 10.4: Que la plantilla se vigile sola ★★★★☆

Escribe las dos pruebas de la sección 10.8 y demuestra que detectan lo que dicen detectar. Parte del Treasury con sus carpetas y su `build.gradle`, y de los `ArchitectureTest` de Quests y Adventurers:

1. `StructureTest` en la raíz (`src/test/java/com/guildboard/`) con `java.nio.file`: cada carpeta de `services/` tiene `domain`, `application`, `adapters` y `bootstrap`, cada una con su `build.gradle`. Si falla, un solo mensaje con *todos* los problemas, uno por línea y con la ruta. Deja la raíz lista (plugin `java` y JUnit).

2. La regla de independencia entre servicios, con `slices()`, en el `ArchitectureTest` de cada uno de los tres servicios.

3. **Provoca las infracciones** para ver fallar las dos pruebas: renombra `services/treasury/adapters/build.gradle`; y haz que una clase de Quests use otra de Adventurers (declara `implementation project(':adventurers-domain')` en el `build.gradle` de `quests-application` e importa allí `AdventurerId` de Adventurers). Restaura ambas cosas.

Está bien si: `./gradlew test` pasa con los tres servicios; renombrar el `build.gradle` la hace fallar con un mensaje que nombra esa ruta; la dependencia de Quests hacia Adventurers hace fallar el `ArchitectureTest` de Quests, nombrando la clase y la línea de la dependencia; con las dos infracciones a la vez cada prueba muestra la suya; y, tras restaurar, vuelve a pasar.

Pista: *`Files.list` devuelve un `Stream` que hay que cerrar: usa* try-with-resources. *Si en la segunda infracción solo declaras la dependencia en `build.gradle` y no usas ninguna clase, la regla no falla: ArchUnit lee bytecode.*

Práctica ▪ solución en la pág. 378

Ejercicio 10.5: Hito H5b: Treasury en marcha, con tres bases ★★★★☆

Parte del proyecto con la estructura de la sección 10.3 (carpetas, `settings.gradle`, los cuatro `build.gradle`, `init.sql`), el dominio y los puertos de las secciones 10.4 y 10.6, el adaptador del libro, `V1` y el controlador, y con los ejercicios anteriores hechos (`V2`, la idempotencia, `Ledger` y las pruebas de estructura). Falta lo que convierte todo eso en un servicio:

1. **Adaptadores en memoria.** `InMemoryAccountRepository` y `InMemoryLedgerRepository` en `adapter/out/memory`.

2. **La cuenta en JPA.** En `adapter/out/persistence`: `AccountEntity` (tabla `account`), `AccountJpaRepository`, `AccountPersistenceMapper` (repite transiciones:

abre y acredita) y `JpaAccountRepository`.

3. **Manejador de errores.** `DomainExceptionHandler`, copia reducida del de Adventurers: `409` para `QuestAlreadyPaid`, `400` para `IllegalArgumentException` y para el cuerpo no válido.
4. **Arranque.** `TreasuryApplication`, `BeanConfig` (reloj y los tres casos de uso), `MemoryPersistenceConfig`, `PostgresPersistenceConfig` y los dos `yml` (puerto 8083, base `treasury`).
5. **Prueba de arquitectura.** `ArchitectureTest` de Treasury, copia de la de Adventurers con `com.guildboard.treasury` y con la regla de independencia entre servicios.
6. Ejecuta `./gradlew build`, el escenario de la sección 10.9.1 en modo memoria y con el perfil `postgres`, y haz el *commit* del hito.

Está bien si: `./gradlew build` termina con `BUILD SUCCESSFUL` y ejecuta `StructureTest` y los tres `ArchitectureTest` (reglas de capas y de independencia); el escenario da `201`, saldo 150, `409 QUEST_ALREADY_PAID` y saldo 150; un pago de cantidad 0 da `400`; un aventurero sin pagos da `404`; en la base `treasury` existen `account`, `ledger_entry` y los historiales `1` y `2` de Flyway; y ningún `build.gradle` de un servicio nombra a otro.

Pista: *el molde es el de Adventurers: copia sus archivos de configuración y cambia nombres, puerto, base y los repositorios que registra.*

Proyecto ▪ solución en la pág. 384

10.11 Resumen

Resumen

- Treasury es el tercer contexto: dueño de las cuentas de oro y del libro de pagos, con su propia base (`treasury`) y sin llamar a nadie. Confía en quien le manda la orden; hasta el capítulo 11 eres tú, con `curl`.
- La verificación de estructura 6 muestra el árbol T5: `services/treasury` con la misma forma que los otros dos, doce proyectos Gradle, tres bases en `init.sql` y `StructureTest` en la raíz. La *plantilla de servicio* es una convención del curso, no de una fuente, y lo que cambia entre servicios es el contenido, no la forma.
- Un libro de cuentas guarda asientos inmutables y el saldo es su suma. En el dominio: `Gold` (objeto de valor), `Account` (entidad) y `LedgerEntry` (record inmutable). El libro es la verdad; el saldo guardado, una copia que se puede recalcular.
- Pagar escribe en dos sitios (asiento primero, saldo después), una excepción reconocida a «un caso de uso, un agregado»: queda el riesgo de que un fallo entre las dos escrituras deje el saldo atrasado, y de que dos pagos distintos al mismo aventurero se pisen. No se resuelve.
- La idempotencia del pago tiene dos defensas: la consulta previa por `questId` en el servicio (cortesía) y una restricción única sobre `quest_id` en `V2` con traducción del error en el adaptador (garantía). La repetición responde `409 QUEST_ALREADY_PAID`.
- Para atrapar una violación de unicidad dentro del método del adaptador hace falta `saveAndFlush`; Spring la entrega como `DataIntegrityViolationException`. Las migraciones aplicadas no se editan: la restricción va en `V2`, y `V2` falla si ya hay duplicados.

- `StructureTest` es una prueba JUnit de la raíz con `java.nio.file` (`Path.of`, `Files.list`, `Files.isDirectory`, `Files.exists`): comprueba que cada servicio tiene sus cuatro módulos con `build.gradle`. La independencia entre servicios es una regla de slices (`slices().matching("com.guildboard.(*)..").should().notDependOnEachOther()`) en el `ArchitectureTest` de cada servicio, que solo ve su *classpath*. Limitación: una dependencia de `build.gradle` sin uso en el código no deja rastro en el bytecode. Hay que verlas fallar para fiarse de ellas.
- El adaptador REST `POST /payments` es provisional: lo sustituye el evento del capítulo 11. El hito H5 queda completo con tres servicios y tres bases de datos.

Soluciones de los ejercicios

Solución del ejercicio 10.1 enunciado en la pág. 367

(1) La tabla.

Elemento	Es	Por qué
Cuatro módulos y su grafo	Molde	Idénticos en los tres: `application` ve a `domain`, `adapters` a `application`, `bootstrap` a ambos. Solo cambian los nombres.
Paquetes `com.guildboard.<servicio>` y capas	Molde (forma)	Las capas `domain`, `application`, `adapter`, `config` son las mismas; el segundo nivel del nombre es el servicio.
`DomainError` y `DomainException`	Ambas	Molde: la técnica (interfaz sellada y *switch* exhaustivo). Propio: las variantes (siete en Quests, una en Treasury).
`Rank` y `AdventurerId` de Quests	Propio	`Rank` solo existe donde hace falta (Quests y Adventurers); Treasury no lo tiene. Lo que se repite es la práctica de copiar, no el archivo.
`AdventurerDirectory` y `rest`	Propio	Es la dependencia de Quests hacia otro servicio; Treasury no llama a nadie.
Las dos `*PersistenceConfig`	Ambas	Molde: existen en los tres, con los perfiles `!postgres` y `postgres`. Propio: qué repositorios registran.
Los dos `application*.yml`	Ambas	Molde: las cuatro exclusiones y el perfil. Propio: puerto, nombre, URL de la base y, en Quests, `base-url` de Adventurers.
`V1`	Ambas	Molde: dónde está y cómo se llama (`V1__create_...`). Propio: las tablas.
`Account` y `LedgerEntry`	Propio	Es el negocio de Treasury.
`ArchitectureTest`	Ambas	Molde: las reglas y la estructura de la clase. Propio: la constante `BASE_PACKAGE`.

(2) Para un cuarto servicio, fuera de su carpeta. Exactamente dos archivos: `settings.gradle` (su nombre en la lista del bucle) e `infra/postgres/init.sql` (su `CREATE DATABASE`, más un `down -v` la primera vez). `StructureTest` no cambia porque recorre las carpetas de `services/`, y `compose.yaml` tampoco.

(3) Lo que `StructureTest` y la regla de slices no comprueban. (a) Los nombres de paquete: un servicio cuyo código viva en `com.guildboard.domain` pasaría la tarea. (b) Que existan las clases de configuración, los `yml` o `V1`. (c) Que la constante `BASE_PACKAGE` del `ArchitectureTest` coincida con el servicio (un valor equivocado deja la selección vacía y

ArchUnit falla, pero un valor válido de otro servicio pasaría en silencio). (d) Que los puertos sean distintos. Cualquiera de ellas vale; las dos primeras son las más útiles de automatizar.
Las comprobaciones, aplicadas. Todas se cumplen: `./gradlew test` pasa, cada adaptador está en `in` o `out`, ningún `build.gradle` nombra a otro servicio (la regla de slices lo vigila si hay código que lo use) y las carpetas son iguales al árbol T5. El cambio total: un servicio, una línea en dos archivos y dos pruebas.
Errores típicos: clasificar por el nombre del archivo y no por su contenido (`DomainError` existe en los tres, pero no es «lo mismo»); marcar `Rank` como molde porque está en dos servicios; y creer que la plantilla es un generador: no lo es, es una forma que se repite y que una tarea comprueba.

Solución del ejercicio 10.2 enunciado en la pág. 368

Qué pasa hoy. Dos asientos para la misma misión y el saldo duplicado: nada impide el segundo pago.
1. El servicio. Se construye el asiento primero (así un importe inválido da `400` aunque la misión ya esté pagada) y se comprueba antes de escribir:

application/service/PayRewardService.java

```java
package com.guildboard.treasury.application.service;

import com.guildboard.treasury.application.port.in.PayRewardCommand;
import com.guildboard.treasury.application.port.in.PayRewardUseCase;
import com.guildboard.treasury.application.port.out.AccountRepository;
import com.guildboard.treasury.application.port.out.LedgerRepository;
import com.guildboard.treasury.domain.Account;
import com.guildboard.treasury.domain.DomainError;
import com.guildboard.treasury.domain.DomainException;
import com.guildboard.treasury.domain.LedgerEntry;
import java.time.Clock;
import java.util.UUID;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class PayRewardService implements PayRewardUseCase {

    private final AccountRepository accounts;
    private final LedgerRepository ledger;
    private final Clock clock;

    @Override
    public LedgerEntry pay(PayRewardCommand command) {
        LedgerEntry entry = new LedgerEntry(
            UUID.randomUUID().toString(), command.questId(),
            command.adventurerId(), command.amount(), clock.instant());
        if (ledger.existsByQuestId(command.questId())) {
            throw new DomainException(
                new DomainError.QuestAlreadyPaid(command.questId()));
        }
        ledger.save(entry);
        Account account = accounts.findById(command.adventurerId())
            .orElseGet(() -> Account.open(command.adventurerId()));
        account.credit(command.amount());
        accounts.save(account);
        return entry;
    }
}
```

2. La migración. Un archivo nuevo, junto a `V1`; `V1` no se toca:

bootstrap/.../db/migration/V2__unique_quest_payment.sql

```sql
-- V2: un pago por misi^^c3^^b3n. Red de seguridad de la base de datos: la
-- primera defensa es la comprobaci^^c3^^b3n previa de PayRewardService.

ALTER TABLE ledger_entry
    ADD CONSTRAINT uq_ledger_entry_quest UNIQUE (quest_id);
```

3. El adaptador. El mismo archivo de la sección 10.6 con `save` cambiado (dos trozos seguidos):

adapter/out/persistence/JpaLedgerRepository.java (1/2)

```java
package com.guildboard.treasury.adapter.out.persistence;

import com.guildboard.treasury.application.port.out.LedgerRepository;
import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.DomainError;
import com.guildboard.treasury.domain.DomainException;
import com.guildboard.treasury.domain.LedgerEntry;
import com.guildboard.treasury.domain.QuestId;
import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.transaction.annotation.Transactional;

@RequiredArgsConstructor
@Transactional
public class JpaLedgerRepository implements LedgerRepository {

    private final LedgerJpaRepository jpa;

    @Override
    @Transactional(readOnly = true)
    public boolean existsByQuestId(QuestId questId) {
        return jpa.existsByQuestId(questId.value());
    }

    // saveAndFlush: el INSERT se env^^c3^^ada aqu^^c3^^ad, dentro del try, y no al
    // confirmar la transacci^^c3^^b3n, cuando el catch ya no puede verlo.
    @Override
    public void save(LedgerEntry entry) {
        try {
            jpa.saveAndFlush(
                LedgerEntryPersistenceMapper.toEntity(entry));
        } catch (DataIntegrityViolationException e) {
            throw new DomainException(
                new DomainError.QuestAlreadyPaid(entry.questId()));
        }
    }
```

adapter/out/persistence/JpaLedgerRepository.java (2/2: sigue dentro de la clase)

```java
    @Override
    @Transactional(readOnly = true)
    public List<LedgerEntry> findByAdventurerId(
            AdventurerId adventurerId) {
        return jpa.findByAdventurerId(adventurerId.value()).stream()
            .map(LedgerEntryPersistenceMapper::toDomain)
            .toList();
    }
}
```

Comprobación. En modo memoria basta el servicio. Con el perfil `postgres`, comprueba la restricción directamente en la base:

```
$ docker compose exec postgres psql -U guildboard -d treasury -c \
    "insert into ledger_entry values ('x','q9','h1',10,now()),
                                     ('y','q9','h1',10,now())"
ERROR:  duplicate key value violates unique constraint
        "uq_ledger_entry_quest"
```

(Comprobado con `postgres:18`; el resto es la salida esperada.)

Respuestas. Hacen falta las dos porque la comprobación previa no ve a quien llega *a la vez*: dos peticiones preguntan, las dos reciben «no pagada» y las dos insertan. La restricción es la única que ve a todos los escritores; la comprobación previa evita llegar a ella en el caso normal y da una respuesta limpia sin lanzar una excepción de base de datos. Si la tabla ya tiene dos asientos con el mismo `quest_id`, `V2` falla al crear la restricción (`could not create unique index ... Key (quest_id)=(q1) is duplicated`; comprobado). Hay que localizar los duplicados con `select quest_id, count(*) from ledger_entry group by quest_id having count(*) > 1`, decidir cuál es el bueno, borrar los sobrantes y recalcular el saldo (ejercicio 10.3).

Errores típicos: añadir la restricción editando `V1` (Flyway rechaza la base donde ya corrió, por el *checksum*); usar `save` en lugar de `saveAndFlush`, con lo que el error sale del proxy transaccional al confirmar, ya fuera del `try`, y el cliente ve un `500`; capturar `Exception` en lugar de `DataIntegrityViolationException` (taparía otros fallos); dejar solo la defensa de la base y devolver siempre un `500` al duplicado; y olvidar que el adaptador en memoria no necesita cambios, porque la consulta previa le basta cuando las peticiones llegan una a una.

Solución del ejercicio 10.3

enunciado en la pág. 368

1. El dominio. Una clase sin estado, con una función pura:

domain/Ledger.java

```
1  package com.guildboard.treasury.domain;
2
3  import java.util.List;
4
5  public final class Ledger {
6
7      private Ledger() {
8      }
9
10     // El saldo es la suma de los asientos; un libro vac^^c3^^ado suma cero.
       ↪
11     public static Gold balanceOf(List<LedgerEntry> entries) {
12         return entries.stream()
13                 .map(LedgerEntry::amount)
14                 .reduce(Gold.ZERO, Gold::plus);
15     }
16 }
```

2. La aplicación.

application/port/in/GetLedgerBalanceUseCase.java

```java
package com.guildboard.treasury.application.port.in;

import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.Gold;

public interface GetLedgerBalanceUseCase {

    Gold get(AdventurerId adventurerId);
}
```

application/service/GetLedgerBalanceService.java

```java
package com.guildboard.treasury.application.service;

import com.guildboard.treasury.application.port.in.GetLedgerBalanceUseCase;
import com.guildboard.treasury.application.port.out.LedgerRepository;
import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.Gold;
import com.guildboard.treasury.domain.Ledger;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class GetLedgerBalanceService implements GetLedgerBalanceUseCase {

    private final LedgerRepository ledger;

    @Override
    public Gold get(AdventurerId adventurerId) {
        return Ledger.balanceOf(ledger.findByAdventurerId(adventurerId));
    }
}
```

3. El controlador. Es un controlador aparte para no reescribir el de la sección 10.7; dos controladores pueden compartir el prefijo `/accounts`. Reutiliza `AccountResponse`:

adapter/in/web/LedgerBalanceController.java

```java
package com.guildboard.treasury.adapter.in.web;

import com.guildboard.treasury.adapter.in.web.dto.AccountResponse;
import com.guildboard.treasury.application.port.in.GetLedgerBalanceUseCase;
import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.Gold;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/accounts")
@RequiredArgsConstructor
public class LedgerBalanceController {

    private final GetLedgerBalanceUseCase getLedgerBalance;

    @GetMapping("/{adventurerId}/ledger-balance")
    public AccountResponse ledgerBalance(
            @PathVariable String adventurerId) {
        Gold balance =
            getLedgerBalance.get(new AdventurerId(adventurerId));
        return new AccountResponse(adventurerId, balance.coins());
    }
}
```

4. El bean. En `BeanConfig` (la clase completa está en la solución del ejercicio 10.5) va un método más:

config/BeanConfig.java (método añadido)

```java
    @Bean
    public GetLedgerBalanceUseCase getLedgerBalanceUseCase(
            LedgerRepository ledgerRepository) {
        return new GetLedgerBalanceService(ledgerRepository);
    }
```

Probar. Con dos pagos de 150 y 50 a Mara, las dos lecturas dan 200. Para verlas *diferir*, simula el fallo entre las dos escrituras poniendo a cero el saldo guardado (con el perfil `postgres`):

```
$ docker compose exec postgres psql -U guildboard -d treasury -c \
    "update account set balance = 0"
$ curl -s localhost:8083/accounts/$MARA
{"adventurerId":"7d1c4e52-...","balance":0}
$ curl -s localhost:8083/accounts/$MARA/ledger-balance
{"adventurerId":"7d1c4e52-...","balance":200}
```

(Salida esperada, no ejecutada.)

Respuesta a (4). Difieren si el proceso muere entre guardar el asiento y guardar el saldo (el asiento está, el saldo no), o si dos pagos distintos se pisan al leer y escribir el saldo. Creo el calculado: el libro es de solo escritura y solo crece, así que no se puede quedar atrás. Con él se puede reparar el saldo guardado.
Errores típicos: sumar `int` a mano y perder el objeto de valor (`reduce(Gold.ZERO, Gold::plus)` lo conserva); devolver `404` para un aventurero sin asientos (un libro vacío suma cero, no es un error); poner la suma en el controlador o en el adaptador, fuera del dominio; y olvidar el bean, con lo que Spring no encuentra el `GetLedgerBalanceUseCase` al arrancar.

Solución del ejercicio 10.4

enunciado en la pág. 369

Solución de referencia. Tres piezas: la raíz, `StructureTest` y la regla de slices en cada `ArchitectureTest`. (Todo es Java y Gradle revisados, no ejecutados.)
1. La raíz. Plugin `java` y JUnit con versión explícita (la raíz no aplica el BOM de Boot); `subprojects` no cambia:

build.gradle (raíz)

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.1' apply false
}

repositories {
    mavenCentral()
}

dependencies {
    testImplementation platform('org.junit:junit-bom:6.0.3')
    testImplementation 'org.junit.jupiter:junit-jupiter'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

tasks.named('test', Test) {
    useJUnitPlatform()
}

subprojects {
    apply plugin: 'java-library'
    apply plugin: 'io.spring.dependency-management'

    group = 'com.guildboard'
    version = '0.0.1-SNAPSHOT'

    java {
        toolchain {
            languageVersion = JavaLanguageVersion.of(21)
        }
    }

    repositories {
        mavenCentral()
    }

    dependencyManagement {
        imports {
            mavenBom(org.springframework.boot.gradle.plugin
                    .SpringBootPlugin.BOM_COORDINATES)
        }
    }

    dependencies {
        testImplementation 'org.springframework.boot:spring-boot-starter-
    ↪ test'
        testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
    }

    tasks.named('test') {
        useJUnitPlatform()
    }
}
```

2. StructureTest.

src/test/java/com/guildboard/StructureTest.java

```java
package com.guildboard;

import static org.junit.jupiter.api.Assertions.assertTrue;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Stream;
import org.junit.jupiter.api.Test;

public class StructureTest {

    private static final Path SERVICES = Path.of("services");

    private static final List<String> MODULES =
        List.of("domain", "application", "adapters", "bootstrap");

    @Test
    void everyServiceHasItsFourModules() throws IOException {
        assertTrue(Files.isDirectory(SERVICES),
            "no existe la carpeta " + SERVICES.toAbsolutePath());

        List<String> problems = new ArrayList<>();
        try (Stream<Path> children = Files.list(SERVICES)) {
            List<Path> services = children
                .filter(Files::isDirectory)
                .filter(p -> !p.getFileName().toString().startsWith("."))
                .sorted()
                .toList();
            if (services.isEmpty()) {
                problems.add("services/ no contiene ning^^c3^^ban servicio");
            }
            for (Path service : services) {
                for (String module : MODULES) {
                    Path buildFile =
                        service.resolve(module).resolve("build.gradle");
                    if (!Files.exists(buildFile)) {
                        problems.add("falta " + buildFile);
                    }
                }
            }
        }

        assertTrue(problems.isEmpty(), () ->
            "La estructura de servicios no se cumple:\n  "
            + String.join("\n  ", problems));
    }
}
```

3. La regla de slices. El `ArchitectureTest` de Quests completo; el de Adventurers y el de Treasury son el mismo archivo con otro paquete y otro valor de `BASE_PACKAGE`:

quests-bootstrap: src/test/.../quests/ArchitectureTest.java

```java
package com.guildboard.quests;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;
import static com.tngtech.archunit.library.dependencies.SlicesRuleDefinition.slices;

import com.tngtech.archunit.core.domain.JavaClasses;
import com.tngtech.archunit.core.importer.ClassFileImporter;
import com.tngtech.archunit.core.importer.ImportOption;
import org.junit.jupiter.api.Test;

public class ArchitectureTest {

    private static final String BASE_PACKAGE = "com.guildboard.quests";

    private static final JavaClasses CLASSES = new ClassFileImporter()
        .withImportOption(new ImportOption.DoNotIncludeTests())
        .importPackages(BASE_PACKAGE);

    @Test
    void domainDoesNotDependOnOuterLayers() {
        noClasses().that().resideInAPackage("..domain..")
            .should().dependOnClassesThat().resideInAnyPackage(
                "..application..", "..adapter..", "..config..",
                "org.springframework..", "jakarta..")
            .check(CLASSES);
    }

    @Test
    void applicationOnlyDependsOnDomain() {
        classes().that().resideInAPackage("..application..")
            .should().onlyDependOnClassesThat().resideInAnyPackage(
                "..application..", "..domain..", "java..", "lombok..")
            .check(CLASSES);
    }

    @Test
    void adapterInDoesNotDependOnAdapterOut() {
        noClasses().that().resideInAPackage("..adapter.in..")
            .should().dependOnClassesThat().resideInAPackage("..adapter.out..")
            .check(CLASSES);
    }

    @Test
    void adapterOutDoesNotDependOnAdapterIn() {
        noClasses().that().resideInAPackage("..adapter.out..")
            .should().dependOnClassesThat().resideInAPackage("..adapter.in..")
            .check(CLASSES);
    }

    @Test
    void servicesDoNotDependOnEachOther() {
        JavaClasses allClasses = new ClassFileImporter()
            .withImportOption(new ImportOption.DoNotIncludeTests())
            .importPackages("com.guildboard");

        slices().matching("com.guildboard.(*)..")
```

```
$ B=services/adventurers/bootstrap/src/test/java/com/guildboard/adventurers
$ sed -e 's/com\.guildboard\.quests/com.guildboard.adventurers/g' \
    services/quests/bootstrap/src/test/java/com/guildboard/quests/
    ↪ ArchitectureTest.java \
    > $B/ArchitectureTest.java
$ T=services/treasury/bootstrap/src/test/java/com/guildboard/treasury
$ sed -e 's/com\.guildboard\.quests/com.guildboard.treasury/g' \
    services/quests/bootstrap/src/test/java/com/guildboard/quests/
    ↪ ArchitectureTest.java \
    > $T/ArchitectureTest.java
```

(Los dos `sed` sustituyen `com.guildboard.quests` también en la línea `package`. La regla de slices usa `"com.guildboard"` tal cual, que el `sed` no toca porque no lleva `.quests`.)
Las infracciones. Provócalas y restáuralas:

```
$ mv services/treasury/adapters/build.gradle /tmp/adapters.gradle
$ ./gradlew test
StructureTest > everyServiceHasItsFourModules() FAILED
    ... falta services/treasury/adapters/build.gradle
$ mv /tmp/adapters.gradle services/treasury/adapters/build.gradle
```

Para la segunda, en `services/quests/application/build.gradle` añade `implementation project(':adventurers-domain')` y crea una clase que use algo de ese módulo:

quests-application: Leak.java (temporal)

```java
package com.guildboard.quests.application.service;

public class Leak {

    private com.guildboard.adventurers.domain.AdventurerId other;
}
```

```
$ ./gradlew :quests-bootstrap:test
ArchitectureTest > servicesDoNotDependOnEachOther() FAILED
    ... Slice quests ... Slice adventurers ...
$ rm services/quests/application/src/main/java/com/guildboard/quests/
    ↪ application/service/Leak.java
$ git restore services/quests/application/build.gradle
$ ./gradlew test
```

(Las líneas con puntos suspensivos son las del mensaje real, que ArchUnit completa con la clase y la línea de la dependencia; aquí van resumidas.)
Por qué funciona. `Path.of("services")` es relativo al directorio de trabajo de la tarea `test` de la raíz, que es la raíz del repositorio. `Files.list` se cierra con *try-with-resources*. Todos los problemas se acumulan en una lista y se lanza un único fallo, con un `Supplier` para que el mensaje solo se construya si falla. La regla de slices importa `com.guildboard` entero, pero el *classpath* del `bootstrap` de Quests solo contiene las clases de Quests y de sus módulos: el slice `adventurers` no existe hasta que alguien añade esa dependencia y la

usa, y entonces la regla ve a Quests dependiendo de él. Por eso no hace falta filtrar `gateway` ni `config-server`, que no llevan esta prueba.
Criterios para valorar otras soluciones. Es válida si el `StructureTest` usa solo `java.nio.file` y JUnit, comprueba carpeta y `build.gradle` de cada módulo, acumula los fallos y nombra la ruta; si la regla de independencia está en los tres `ArchitectureTest`; y si has visto fallar las dos. Alternativa válida: una regla con `noClasses().that().resideInAPackage("com.guildboard.quests..") .should().dependOnClassesThat().resideInAnyPackage("com.guildboard.adventurers..", "com.guildboard.treasury..")`, que nombra los servicios prohibidos uno por uno; es más explícita, pero hay que actualizarla con cada servicio nuevo, mientras que los slices lo cubren solos.
Limitación. Si solo declaras `implementation project(':adventurers-domain')` y ninguna clase de Quests la usa, la regla pasa: ArchUnit lee bytecode, y una dependencia sin uso no deja rastro. La prueba tampoco mira el contenido de los `build.gradle`; `StructureTest` solo sabe que existen.
Errores típicos: abrir `Files.list` sin cerrarlo; lanzar el fallo en la primera carpeta incompleta en vez de acumular; olvidar el plugin `java` o JUnit en la raíz (Gradle no encuentra las pruebas o `@Test`); escribir la regla de slices con `BASE_PACKAGE` (solo ve un servicio y nunca falla); usar `"com.guildboard.(*)"` sin los dos puntos finales (el slice no incluye los subpaquetes); y probar solo con el servicio en limpio, sin romperlo.

Solución del ejercicio 10.5

enunciado en la pág. 369

Estado de partida. Tienes escritos, con paquetes `com.guildboard.treasury.*`: la estructura, `settings.gradle`, los cuatro `build.gradle` e `init.sql` (sección 10.3); el dominio, los puertos y los servicios (`PayRewardService` ya idempotente), `Ledger` y su caso de uso; el adaptador del libro (`LedgerEntryEntity`, `LedgerJpaRepository`, `LedgerEntryPersistenceMapper` y `JpaLedgerRepository` con `saveAndFlush`); `V1` y `V2`; los DTO, el mapeador y los dos controladores; y `StructureTest` con la raíz preparada y la regla de slices en los `ArchitectureTest` de Quests y Adventurers (ejercicio 10.4). Esos archivos *no cambian*. Lo que sigue es lo que falta, completo.
1. Adaptadores en memoria.

adapter/out/memory/InMemoryAccountRepository.java

```java
package com.guildboard.treasury.adapter.out.memory;

import com.guildboard.treasury.application.port.out.AccountRepository;
import com.guildboard.treasury.domain.Account;
import com.guildboard.treasury.domain.AdventurerId;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

public class InMemoryAccountRepository implements AccountRepository {

    private final Map<AdventurerId, Account> accounts =
        new ConcurrentHashMap<>();

    @Override
    public Optional<Account> findById(AdventurerId adventurerId) {
        return Optional.ofNullable(accounts.get(adventurerId));
    }

    @Override
    public void save(Account account) {
        accounts.put(account.getOwner(), account);
    }
}
```

adapter/out/memory/InMemoryLedgerRepository.java

```java
package com.guildboard.treasury.adapter.out.memory;

import com.guildboard.treasury.application.port.out.LedgerRepository;
import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.LedgerEntry;
import com.guildboard.treasury.domain.QuestId;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

public class InMemoryLedgerRepository implements LedgerRepository {

    private final List<LedgerEntry> entries =
        new CopyOnWriteArrayList<>();

    @Override
    public boolean existsByQuestId(QuestId questId) {
        return entries.stream()
            .anyMatch(entry -> entry.questId().equals(questId));
    }

    @Override
    public void save(LedgerEntry entry) {
        entries.add(entry);
    }

    @Override
    public List<LedgerEntry> findByAdventurerId(
            AdventurerId adventurerId) {
        return entries.stream()
            .filter(entry -> entry.adventurerId().equals(adventurerId))
            .toList();
    }
}
```

2. La cuenta en JPA.

adapter/out/persistence/AccountEntity.java

```java
package com.guildboard.treasury.adapter.out.persistence;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "account")
@Getter
@AllArgsConstructor
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class AccountEntity {

    @Id
    @Column(name = "adventurer_id")
    private String adventurerId;

    @Column(name = "balance", nullable = false)
    private int balance;
}
```

adapter/out/persistence/AccountJpaRepository.java

```java
package com.guildboard.treasury.adapter.out.persistence;

import org.springframework.data.jpa.repository.JpaRepository;

public interface AccountJpaRepository
        extends JpaRepository<AccountEntity, String> {
}
```

adapter/out/persistence/AccountPersistenceMapper.java

```
package com.guildboard.treasury.adapter.out.persistence;

import com.guildboard.treasury.domain.Account;
import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.Gold;

final class AccountPersistenceMapper {

    private AccountPersistenceMapper() {
    }

    static AccountEntity toEntity(Account account) {
        return new AccountEntity(
            account.getOwner().value(), account.getBalance().coins());
    }

    // Sin f^^c3^^a1brica de reconstrucci^^c3^^b3n: se repite la operaci^^
    ↪ c3^^b3n p^^c3^^blica.
    static Account toDomain(AccountEntity entity) {
        Account account =
            Account.open(new AdventurerId(entity.getAdventurerId()));
        if (entity.getBalance() > 0) {
            account.credit(new Gold(entity.getBalance()));
        }
        return account;
    }
}
```

adapter/out/persistence/JpaAccountRepository.java

```java
package com.guildboard.treasury.adapter.out.persistence;

import com.guildboard.treasury.application.port.out.AccountRepository;
import com.guildboard.treasury.domain.Account;
import com.guildboard.treasury.domain.AdventurerId;
import java.util.Optional;
import lombok.RequiredArgsConstructor;
import org.springframework.transaction.annotation.Transactional;

@RequiredArgsConstructor
@Transactional
public class JpaAccountRepository implements AccountRepository {

    private final AccountJpaRepository jpa;

    @Override
    @Transactional(readOnly = true)
    public Optional<Account> findById(AdventurerId adventurerId) {
        return jpa.findById(adventurerId.value())
            .map(AccountPersistenceMapper::toDomain);
    }

    @Override
    public void save(Account account) {
        jpa.save(AccountPersistenceMapper.toEntity(account));
    }
}
```

3. El manejador de errores (dos trozos seguidos):

adapter/in/web/error/DomainExceptionHandler.java (1/2)

```java
package com.guildboard.treasury.adapter.in.web.error;

import com.guildboard.treasury.domain.DomainError;
import com.guildboard.treasury.domain.DomainException;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class DomainExceptionHandler {

    @ExceptionHandler(DomainException.class)
    public ProblemDetail handleDomain(DomainException exception) {
        DomainError error = exception.getError();
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            statusFor(error), exception.getMessage());
        problem.setTitle("Business rule violated");
        problem.setProperty("code", error.code());
        return problem;
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ProblemDetail handleMalformed(
            IllegalArgumentException exception) {
        return ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, exception.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail handleInvalidBody(
            MethodArgumentNotValidException exception) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.BAD_REQUEST, "request body is not valid");
        List<String> errors = exception.getBindingResult()
            .getFieldErrors().stream()
            .map(error -> error.getField() + ": "
                + error.getDefaultMessage())
            .toList();
        problem.setProperty("errors", errors);
        return problem;
    }
```

adapter/in/web/error/DomainExceptionHandler.java (2/2: sigue dentro de la clase)

```
    private static HttpStatusCode statusFor(DomainError error) {
        return switch (error) {
            case DomainError.QuestAlreadyPaid e -> HttpStatus.CONFLICT;
        };
    }
}
```

4. Arranque (paquete `com.guildboard.treasury` y `.config` en `treasury-bootstrap`).

TreasuryApplication.java

```
package com.guildboard.treasury;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TreasuryApplication {

    public static void main(String[] args) {
        SpringApplication.run(TreasuryApplication.class, args);
    }
}
```

config/BeanConfig.java (treasury)

```java
package com.guildboard.treasury.config;

import com.guildboard.treasury.application.port.in.GetAccountUseCase;
import com.guildboard.treasury.application.port.in.GetLedgerBalanceUseCase;
import com.guildboard.treasury.application.port.in.PayRewardUseCase;
import com.guildboard.treasury.application.port.out.AccountRepository;
import com.guildboard.treasury.application.port.out.LedgerRepository;
import com.guildboard.treasury.application.service.GetAccountService;
import com.guildboard.treasury.application.service.GetLedgerBalanceService;
import com.guildboard.treasury.application.service.PayRewardService;
import java.time.Clock;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public Clock clock() {
        return Clock.systemUTC();
    }

    @Bean
    public PayRewardUseCase payRewardUseCase(
            AccountRepository accountRepository,
            LedgerRepository ledgerRepository, Clock clock) {
        return new PayRewardService(
            accountRepository, ledgerRepository, clock);
    }

    @Bean
    public GetAccountUseCase getAccountUseCase(
            AccountRepository accountRepository) {
        return new GetAccountService(accountRepository);
    }

    @Bean
    public GetLedgerBalanceUseCase getLedgerBalanceUseCase(
            LedgerRepository ledgerRepository) {
        return new GetLedgerBalanceService(ledgerRepository);
    }
}
```

config/MemoryPersistenceConfig.java (treasury)

```java
package com.guildboard.treasury.config;

import com.guildboard.treasury.adapter.out.memory.InMemoryAccountRepository;
import com.guildboard.treasury.adapter.out.memory.InMemoryLedgerRepository;
import com.guildboard.treasury.application.port.out.AccountRepository;
import com.guildboard.treasury.application.port.out.LedgerRepository;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

@Configuration
@Profile("!postgres")
public class MemoryPersistenceConfig {

    @Bean
    public AccountRepository accountRepository() {
        return new InMemoryAccountRepository();
    }

    @Bean
    public LedgerRepository ledgerRepository() {
        return new InMemoryLedgerRepository();
    }
}
```

config/PostgresPersistenceConfig.java (treasury)

```java
package com.guildboard.treasury.config;

import com.guildboard.treasury.adapter.out.persistence.AccountEntity;
import com.guildboard.treasury.adapter.out.persistence.AccountJpaRepository;
import com.guildboard.treasury.adapter.out.persistence.JpaAccountRepository;
import com.guildboard.treasury.adapter.out.persistence.JpaLedgerRepository;
import com.guildboard.treasury.adapter.out.persistence.LedgerJpaRepository;
import com.guildboard.treasury.application.port.out.AccountRepository;
import com.guildboard.treasury.application.port.out.LedgerRepository;
import org.springframework.boot.persistence.autoconfigure.EntityScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;

@Configuration
@Profile("postgres")
@EnableJpaRepositories(basePackageClasses = AccountJpaRepository.class)
@EntityScan(basePackageClasses = AccountEntity.class)
public class PostgresPersistenceConfig {

    @Bean
    public AccountRepository accountRepository(
            AccountJpaRepository jpa) {
        return new JpaAccountRepository(jpa);
    }

    @Bean
    public LedgerRepository ledgerRepository(LedgerJpaRepository jpa) {
        return new JpaLedgerRepository(jpa);
    }
}
```

La configuración, en `services/treasury/bootstrap/src/main/resources` (puerto 8083 y base `treasury`); las cuatro exclusiones de autoconfiguración son las de los otros dos servicios:

application.yml (treasury)

```yaml
spring:
  application:
    name: treasury
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration

server:
  port: 8083
```

application-postgres.yml (treasury)

```yaml
spring:
  autoconfigure:
    exclude: []
  datasource:
    url: jdbc:postgresql://localhost:5432/treasury
    username: guildboard
    password: guildboard
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: "none"
```

5. La prueba de arquitectura de Treasury. Es la copia del de Adventurers (que ya lleva la regla de slices), con el paquete de Treasury como único dato. Si no la hiciste en la sección 10.8:

```
$ A=services/adventurers/bootstrap/src/test/java/com/guildboard/adventurers
$ T=services/treasury/bootstrap/src/test/java/com/guildboard/treasury
$ mkdir -p $T
$ sed -e 's/com\.guildboard\.adventurers/com.guildboard.treasury/g' \
    $A/ArchitectureTest.java > $T/ArchitectureTest.java
```

El resultado es el archivo del ejercicio 10.4 con `package com.guildboard.treasury;` y `BASE_PACKAGE = "com.guildboard.treasury"`. Ahora ya puede pasar: Treasury tiene dominio (selecciones no vacías), su `application` solo importa `domain`, `java` y Lombok, y sus adaptadores no se cruzan entre `in` y `out`.

6. Comprobar.

```
$ ./gradlew build
BUILD SUCCESSFUL
$ ./gradlew test
$ git add -A
$ git commit -m "H5b: Treasury y sus pruebas"
```

Después, el escenario de la sección 10.9.1, en memoria y con `SPRING_PROFILES_ACTIVE=postgres` (con el `down -v` de la sección 10.3 hecho antes).

Por qué funciona. Treasury es el molde de Adventurers con otro contenido: mismos cuatro módulos, mismas dos clases de persistencia (`!postgres` y `postgres`), mismo arranque. `BeanConfig` solo registra el reloj y los casos de uso; los repositorios los pone la configuración de persistencia según el perfil. En modo `postgres`, Flyway aplica `V1` y `V2` a la base `treasury` y `ddl-auto: none` deja el esquema en manos de las migraciones.

Criterios para valorar otras soluciones. Es válida si Treasury arranca en los dos modos, no importa nada de Quests ni de Adventurers, los repositorios solo los registran las dos clases de configuración, y `domain` y `application` siguen sin importar Spring.

Errores típicos: dejar el puerto 8080 o la base `guildboard` en los `yml`; olvidar el bean `Clock` (Spring no puede crear `PayRewardUseCase`); registrar los repositorios también en `BeanConfig` (dos candidatos); que la base `treasury` no exista porque el volumen es anterior a `init.sql` (`down -v`); que `V2` falle en una base de pruebas con asientos duplicados del servicio ingenuo (límpiala o bórrala); olvidar `'treasury'` en `settings.gradle` (los proyectos no existen y `StructureTest` no lo nota, porque mira carpetas: comprueba `./gradlew projects`); y un `BASE_PACKAGE` del `ArchitectureTest` que no sea `com.guildboard.treasury`.

CAPÍTULO 11

Mensajería: eventos entre servicios con RabbitMQ

Objetivos del capítulo

- Decidir cuándo dos servicios deben hablar por REST y cuándo por mensajería, y explicar el recorrido de un evento: productor, *exchange*, enlace, cola, consumidor y *ack*.
- Ubicar en la verificación de estructura 7 los paquetes de mensajería y la configuración nueva que entran en cada servicio.
- Publicar `QuestCompleted` desde Quests con un puerto de salida y consumirlo en Treasury y Adventurers con adaptadores de entrada, sin que el núcleo conozca el *broker*.
- Hacer idempotente al consumidor de Treasury, mandar a una cola de mensajes muertos lo que no se puede procesar, y reconocer lo que esto no resuelve: el evento perdido.

Al terminar el capítulo 9, completar una misión dejó de dar experiencia: Quests y Adventurers son procesos distintos y el caso de uso ya no puede llamar a un objeto del otro. El capítulo 10 añadió un tercer servicio que paga recompensas, pero solo si tú le mandas la orden con `curl`. Hoy, completar una misión no hace nada más que cambiar su estado. Este capítulo cierra ese hueco con un *evento*: Quests anuncia «esta misión se ha completado» y los otros dos servicios reaccionan por su cuenta.

Tres cosas que el capítulo deja dichas desde el principio:

- **Hay una dependencia nueva y una herramienta nueva.** El *broker* RabbitMQ se instala en Compose y el starter `spring-boot-starter-amqp` se añade a los `build.gradle`. La sección 11.3 lo explica antes de usarlo.
- **Los `POST` provisionales desaparecen.** `POST /payments` en Treasury y `POST /adventurers/{id}/experience` en Adventurers eran andamios de los capítulos 9 y 10. Con el evento ya no hacen falta, y se borran.
- **El evento puede perderse.** El consumidor idempotente de la sección 11.6 protege contra mensajes *repetidos*, no contra mensajes *perdidos*. La sección 11.8 muestra cómo se pierde uno y por qué GuildBoard acepta ese riesgo en el curso. No se presenta como resuelto.

11.1 Por qué un evento y no una llamada

Al completar una misión pasan tres cosas: la misión cambia de estado (es de Quests), el aventurero gana experiencia (es de Adventurers) y se le paga la recompensa (es de Treasury). Con lo que sabes, la primera idea es que Quests llame por REST a los otros dos, como ya hace para preguntar un rango. Funciona, y tiene tres costes:

1. **Acoplamiento en el tiempo.** Si Treasury está parado, ¿falla completar la misión? Con una llamada síncrona hay que decidirlo: o falla todo, o se completa a medias.

2. **Quests conoce a sus interesados.** Si mañana un cuarto servicio quiere reaccionar (por ejemplo, un tablón de honor), hay que tocar Quests para llamarlo. El servicio que anuncia un hecho no debería saber quién escucha.
3. **Esperar lo que no hace falta.** Quests no necesita ninguna respuesta de Treasury para terminar su trabajo. Esperarla solo añade latencia y puntos de fallo.

Un *evento* resuelve las tres cosas. Es un mensaje que describe un hecho ya ocurrido, con el nombre en pasado (`QuestCompleted`), y se entrega a quien se haya apuntado. Quien lo publica no sabe cuántos lo reciben ni espera respuesta. Es la mensajería asíncrona que describen los catálogos de patrones de microservicios [41, 42], y es una de las dos formas de comunicación que Newman distingue entre servicios: la de petición y respuesta y la basada en eventos [32].

Pero no todo es un evento. La regla práctica del capítulo:

	REST (petición y respuesta)	**Mensajería (evento)**
Qué expresa	Una pregunta o una orden: «dame el rango de Mara».	Un hecho: «la misión Q se ha completado».
¿Hace falta la respuesta ya?	Sí: Quests no puede aceptar sin saber el rango.	No: Quests termina sin esperar a nadie.
Si el otro está parado	La llamada falla (o espera, capítulo 12).	El mensaje espera en una cola hasta que vuelva.
Quién conoce a quién	El que llama conoce al llamado.	El que publica no conoce a los que escuchan.
En GuildBoard	`AdventurerDirectory`: sigue siendo REST.	`QuestCompleted`: llega a Treasury y a Adventurers.

Hay un precio que conviene aceptar desde ya: la consistencia pasa a ser *eventual*. Tras completar una misión, el saldo y la experiencia se actualizan unos instantes después, no en la misma petición. Y quien consulta justo en medio puede ver la misión completada y el saldo sin cambiar.

11.2 Verificación de estructura 7: los paquetes de mensajería

Nota

Este capítulo cambia la estructura de carpetas: aparecen paquetes de mensajería dentro de módulos que ya existen, un servicio nuevo en `compose.yaml` y se borran los adaptadores web provisionales. Comprueba que tu proyecto coincide con el árbol de abajo antes de seguir; las piezas se construyen en las secciones siguientes.

11.2.1 El árbol anterior

Así dejó el proyecto el capítulo 10 (árbol T5). Solo cambian las partes marcadas en el árbol nuevo; aquí va lo relevante de cada servicio:

Árbol anterior (T5, tras el capítulo 10), resumido

```
guildboard/
|-- compose.yaml        solo PostgreSQL 18
|-- infra/postgres/init.sql, settings.gradle
`-- services/
    |-- quests/
    |   |-- application/  port/out: QuestRepository, NotificationSender,
```

```
    |   |                 AdventurerDirectory, AdventurerView
    |   |-- adapters/     adapter/in/web, adapter/out/{memory,
    |   |                 notification, persistence, rest}
    |   `-- bootstrap/    config/{BeanConfig, MemoryPersistenceConfig,
    |                     PostgresPersistenceConfig}
    |-- adventurers/
    |   |-- adapters/     adapter/in/web (con POST .../experience),
    |   |                 adapter/out/{memory, persistence}
    |   `-- bootstrap/    config/{BeanConfig, ...PersistenceConfig}
    `-- treasury/
        |-- adapters/     adapter/in/web (con POST /payments),
        |                 adapter/out/{memory, persistence}
        `-- bootstrap/    config/{BeanConfig, ...PersistenceConfig}
```

11.2.2 El árbol nuevo

`[N]` es nuevo, `[M]` modificado y `[D]` borrado. Es la forma final del capítulo: incluye lo que añaden los ejercicios.

Árbol nuevo, raíz (T6, tras el capítulo 11)

```
guildboard/
|-- settings.gradle, build.gradle, gradle/   sin cambios
|-- compose.yaml        [M] + servicio rabbitmq (5672 y 15672)
|-- infra/postgres/init.sql                  sin cambios
`-- services/
    |-- quests/         [M] publica el evento
    |-- adventurers/    [M] consume el evento
    `-- treasury/       [M] consume el evento
```

Árbol nuevo, Quests (T6)

```
services/quests/                     com.guildboard.quests
|-- application/
|   |-- port/out/QuestEventPublisher.java            [N]
|   `-- service/CompleteQuestService.java            [M]
|-- adapters/
|   |-- build.gradle                                 [M] + amqp
|   `-- .../adapter/out/messaging/                   [N]
|       |-- QuestCompleted.java
|       `-- RabbitQuestEventPublisher.java
`-- bootstrap/
    |-- build.gradle                                 [M] + amqp
    |-- resources/application.yml                    [M]
    `-- .../config/{BeanConfig [M], MessagingConfig [N]}
```

Árbol nuevo, Treasury (T6)

```
services/treasury/                   com.guildboard.treasury
|-- application/
|   |-- build.gradle                       [M] dependencias de prueba
|   `-- src/test/.../service/
|       `-- PayRewardServiceIdempotencyTest.java     [N] (ejercicio)
```

```
|-- adapters/
|   |-- build.gradle                                  [M] + amqp
|   `-- .../adapter/in/
|       |-- messaging/                                [N]
|       |   |-- QuestCompleted.java
|       |   `-- QuestCompletedListener.java
|       `-- web/  TreasuryController [M: sin POST], mapper/
|                 TreasuryWebMapper [M], dto/{PayRewardRequest,
|                 PaymentResponse} [D]
`-- bootstrap/
    |-- build.gradle                                  [M] + amqp
    |-- resources/application.yml                     [M]
    `-- .../config/MessagingConfig.java               [N]
```

Árbol nuevo, Adventurers (T6)

```
services/adventurers/                com.guildboard.adventurers
|-- adapters/
|   |-- build.gradle                                  [M] + amqp
|   `-- .../adapter/in/
|       |-- messaging/                                [N]
|       |   |-- QuestCompleted.java
|       |   `-- QuestCompletedListener.java
|       `-- web/  AdventurerController [M: sin POST],
|                 dto/GrantExperienceRequest [D]
`-- bootstrap/
    |-- build.gradle                                  [M] + amqp
    |-- resources/application.yml                     [M]
    `-- .../config/MessagingConfig.java               [N]
```

11.2.3 Cambios respecto al árbol anterior

- `adapter/out/messaging` (Quests) y `adapter/in/messaging` (Treasury y Adventurers): paquetes nuevos dentro del módulo `adapters`. Los subpaquetes `in` y `out` son los del capítulo 3 [19]: el listener es de entrada porque lo invoca el *broker*; el publicador es de salida porque lo invoca el caso de uso.
- `QuestEventPublisher` en `port/out`: un puerto de salida más, con nombre de rol como `NotificationSender`. El núcleo declara qué necesita («anunciar que se completó») y no conoce RabbitMQ.
- `MessagingConfig` en `bootstrap`: raíz de composición del capítulo 4; el conversor y la declaración de colas viven fuera de `adapters` y de `application`.
- `compose.yaml`: servicio `rabbitmq`, junto a PostgreSQL (capítulo 8).
- `adapters/build.gradle` y `bootstrap/build.gradle`: dependencia `spring-boot-starter-amqp`. Se mantiene un solo módulo `adapters`, como en el capítulo 8: ahora conviven Spring MVC, JPA y AMQP, y cada servicio usa AMQP en un único adaptador.
- Adaptadores web provisionales: se borran `POST /payments` en Treasury y `POST /adventurers/{id}/experience` en Adventurers, anunciados en los capítulos 9 y 10.

11.2.4 Comprobaciones

- [ ] `./gradlew test` pasa. Incluye los `ArchitectureTest` de cada servicio y `StructureTest`, que ya cubren los paquetes de mensajería sin ampliar ninguna regla: `adapter.in.messaging` no depende de `adapter.out.messaging` (ni al revés), y ni `domain` ni `application` ven `org.springframework.amqp`.
- [ ] Cada adaptador está en `in` o en `out` según quién lo invoca: el listener es de entrada y el publicador, de salida.
- [ ] `QuestEventPublisher` usa tipos del dominio (`QuestId`, `AdventurerId`, `Reward`) y nada de RabbitMQ; la dependencia nueva es el starter de AMQP, no un proyecto de otro servicio.
- [ ] Las carpetas de tu proyecto coinciden con el árbol.

Si algo no coincide con el árbol, corrígelo antes de seguir.

11.3 RabbitMQ en Compose: exchange, colas y enlaces

Un *broker* de mensajería es un servidor intermediario: quien publica le entrega el mensaje, el *broker* lo guarda y quien consume lo recoge cuando puede. GuildBoard usa RabbitMQ [38]. Su modelo tiene cinco piezas, y casi todos los errores de principiante vienen de confundirlas:

- **Productor y consumidor.** El primero publica mensajes, el segundo los recibe. Un mismo servicio puede ser ambas cosas; aquí Quests solo publica y Treasury y Adventurers solo consumen.
- **Exchange (intercambiador).** El productor *no* escribe en una cola: entrega el mensaje a un *exchange* y le dice una *clave de enrutado* (*routing key*). El *exchange* no guarda nada; solo decide a qué colas copiar el mensaje.
- **Cola (queue).** Es donde espera el mensaje hasta que alguien lo consume. Cada cola tiene un nombre.
- **Enlace (binding).** La regla que une un *exchange* con una cola: «las copias de los mensajes con esta clave van a esa cola». Hay varios tipos de *exchange*; el capítulo usa uno de tipo *topic*, que compara la clave del mensaje con la del enlace.
- **Ack (acuse de recibo).** Cuando el consumidor termina de procesar, avisa al *broker*, que entonces borra el mensaje de la cola. Si el consumidor se cae antes del *ack*, el *broker* lo entrega otra vez.

Así queda el recorrido de `QuestCompleted`. Observa que hay *una cola por consumidor*: si Treasury y Adventurers leyeran de la misma cola, cada mensaje lo recibiría solo uno de los dos (se lo repartirían). Para que lo reciban ambos, cada uno tiene su cola y el *exchange* copia el mensaje a las dos.

Recorrido del evento

```
Quests          RabbitMQ                          Consumidores

publica con     exchange (topic)
clave           guildboard.events
quest.completed    |
 ---------------->  |-- enlace quest.completed
```

```
                        |      v
                        |   cola treasury.quest-completed
                        |      v
                        |   Treasury     (PayRewardUseCase)
                        |
                        `-- enlace quest.completed
                               v
                            cola adventurers.quest-completed
                               v
                            Adventurers  (GrantExperienceUseCase)
```

Dos consecuencias que se ven en la práctica y que debes tener presentes:

- **Un mensaje sin cola de destino se descarta en silencio.** Si Quests publica cuando todavía no existe ningún enlace (porque ningún consumidor ha arrancado nunca), el *exchange* no tiene a quién copiar el mensaje y lo tira. Lo comprobé con la API de gestión: la respuesta de la publicación trae `routed` con el valor `false`. Por eso, en las pruebas del capítulo, los consumidores arrancan *antes* de la primera publicación; la sección 11.8 vuelve sobre ello.
- **Cada consumidor declara su propia cola.** El dueño de la cola es quien la lee. Quests solo declara el *exchange* que usa para publicar.

☞ RabbitMQ en Compose y el starter de AMQP

Tres pasos: el servicio en Compose, la dependencia en cada servicio y la conexión en cada `application.yml`. Lo he verificado así (9 de octubre de 2026):

- La etiqueta `rabbitmq:4-management` existe en Docker Hub (RabbitMQ 4.3.x en esa fecha) e incluye la consola web en el puerto 15672. El servicio de abajo arrancó con `docker compose up -d -wait` y pasó a *healthy*; con la API de gestión (`curl` con usuario y contraseña) declaré *exchanges*, colas y enlaces, publiqué un mensaje y lo leí.
- Initializr, con Boot 4.1.1, ofrece la dependencia con identificador `rabbitmq` (Spring for RabbitMQ) [62] y genera `implementation 'org.springframework.boot:spring-boot-starter-amqp'`. El BOM de Boot 4.1.1 gestiona Spring AMQP 4.1.1 [44], de modo que ninguna dependencia lleva versión.

1. El servicio. En `compose.yaml`, junto a `postgres`, y su volumen al final del archivo (el archivo completo está en la solución del ejercicio 11.1):

compose.yaml (servicio nuevo y volumen nuevo)

```
  rabbitmq:
    image: rabbitmq:4-management
    hostname: rabbitmq
    environment:
      RABBITMQ_DEFAULT_USER: guildboard
      RABBITMQ_DEFAULT_PASS: guildboard
    ports:
      - "5672:5672"
      - "15672:15672"
    volumes:
      - rabbitmq-data:/var/lib/rabbitmq
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 10s
      timeout: 5s
      retries: 10

volumes:
  postgres-data:
  rabbitmq-data:
```

El puerto 5672 es el que usan las aplicaciones (protocolo AMQP) y el 15672, la consola web. `hostname` fija el nombre del nodo: RabbitMQ guarda sus datos bajo ese nombre y, con un nombre aleatorio, no reconocería sus datos tras recrear el contenedor. El usuario es de desarrollo, como el de PostgreSQL: no es una configuración para producción. Arranca solo el *broker* con `docker compose up -d --wait rabbitmq`.

2. El starter. Una línea nueva en dos `build.gradle` de *cada* servicio: el de `adapters` (donde viven el publicador y los listeners) y el de `bootstrap` (donde vive `MessagingConfig`). Va justo debajo de la del starter web. Para Quests:

services/quests/adapters/build.gradle (dependencies)

```
dependencies {
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

services/quests/bootstrap/build.gradle (dependencies)

```
dependencies {
    implementation project(':quests-adapters')
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    runtimeOnly 'org.postgresql:postgresql'
}
```

En Adventurers y en Treasury los dos archivos son los mismos con sus nombres de proyecto (`adventurers-adapters`, `treasury-application`…): la única línea nueva es la del starter de AMQP. El bloque `plugins` de `bootstrap` no cambia.

3. La conexión. Boot lee la conexión con el RabbitMQ local de las propiedades `spring.rabbitmq.*`. Los valores por omisión son `localhost`, el puerto 5672 y el usuario `guest`; como hemos puesto otro usuario, se declara en cada `application.yml` (el archivo completo de cada servicio aparece en el ejercicio de proyecto):

application.yml (fragmento nuevo, en los tres servicios)

```
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: guildboard
    password: guildboard
```

Desde este capítulo, **los tres servicios necesitan el *broker* en marcha**, también en modo memoria; la parte de Docker es obligatoria para probarlos. Si RabbitMQ no está, la aplicación puede arrancar igualmente, pero Quests no podrá publicar al completar una misión y los otros dos registrarán errores de conexión mientras lo reintentan (no he ejecutado el arranque con el *broker* apagado). Para comprobar la consola, abre `http://localhost:15672` con `guildboard` y `guildboard`.

11.4 El núcleo sigue sin conocer el broker

La regla del libro no cambia: el caso de uso pide lo que necesita con una interfaz de su paquete, y un adaptador la implementa. Para publicar es un puerto de salida; para consumir, un adaptador de entrada que llama a un caso de uso que ya existe. Escribe estos archivos a medida que los lees.

11.4.1 Publicar: un puerto de salida y RabbitTemplate

El puerto va en `application` y habla en términos del negocio. No recibe la entidad `Quest` entera: recibe los tres datos que cuenta el hecho.

application/port/out/QuestEventPublisher.java (Quests)

```java
package com.guildboard.quests.application.port.out;

import com.guildboard.quests.domain.AdventurerId;
import com.guildboard.quests.domain.QuestId;
import com.guildboard.quests.domain.Reward;

public interface QuestEventPublisher {

    void questCompleted(
        QuestId questId, AdventurerId adventurerId, Reward reward);
}
```

`CompleteQuestService` gana un colaborador. El anuncio se hace justo después de guardar: si se hiciera antes, podría salir un evento de una misión que luego no se guarda. `NotificationSender` se queda: es otra cosa (avisar a la persona), y sustituirlo por el evento no tendría sentido. El orden de los campos fija el orden del constructor que genera Lombok:

application/service/CompleteQuestService.java (Quests)

```java
package com.guildboard.quests.application.service;

import com.guildboard.quests.application.port.in.CompleteQuestCommand;
import com.guildboard.quests.application.port.in.CompleteQuestUseCase;
import com.guildboard.quests.application.port.out.NotificationSender;
import com.guildboard.quests.application.port.out.QuestEventPublisher;
import com.guildboard.quests.application.port.out.QuestRepository;
import com.guildboard.quests.domain.AdventurerId;
import com.guildboard.quests.domain.DomainError;
import com.guildboard.quests.domain.DomainException;
import com.guildboard.quests.domain.Quest;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class CompleteQuestService implements CompleteQuestUseCase {

    private final QuestRepository quests;
    private final QuestEventPublisher events;
    private final NotificationSender notifications;

    @Override
    public Quest complete(CompleteQuestCommand command) {
        Quest quest = quests.findById(command.questId())
            .orElseThrow(() -> new DomainException(
                new DomainError.QuestNotFound(command.questId())));
        quest.complete();
        AdventurerId assignee = quest.getAssignedTo().orElseThrow(
            () -> new IllegalStateException("accepted quest without "
                + "assignee: " + quest.getId().value()));
        quests.save(quest);
        events.questCompleted(
            quest.getId(), assignee, quest.getReward());
        notifications.send(assignee,
            "quest completed: " + quest.getTitle());
        return quest;
```

```
    }
}
```

El adaptador de salida usa `RabbitTemplate`, la clase de Spring AMQP para enviar mensajes [43]. Boot crea un bean de ese tipo al ver el starter y lo conecta con las propiedades `spring.rabbitmq.*`. Su método `convertAndSend(exchange, routingKey, objeto)` convierte el objeto en un mensaje (la sección 11.5 explica cómo) y lo entrega al *exchange*. El mensaje es un *record* propio del adaptador, con los tres datos y nada más:

adapter/out/messaging/QuestCompleted.java (Quests)

```
package com.guildboard.quests.adapter.out.messaging;

public record QuestCompleted(
        String questId, String adventurerId, int reward) {
}
```

adapter/out/messaging/RabbitQuestEventPublisher.java

```
package com.guildboard.quests.adapter.out.messaging;

import com.guildboard.quests.application.port.out.QuestEventPublisher;
import com.guildboard.quests.domain.AdventurerId;
import com.guildboard.quests.domain.QuestId;
import com.guildboard.quests.domain.Reward;
import lombok.RequiredArgsConstructor;
import org.springframework.amqp.rabbit.core.RabbitTemplate;

@RequiredArgsConstructor
public class RabbitQuestEventPublisher implements QuestEventPublisher {

    private final RabbitTemplate rabbit;
    private final String exchange;
    private final String routingKey;

    @Override
    public void questCompleted(
            QuestId questId, AdventurerId adventurerId, Reward reward) {
        QuestCompleted message = new QuestCompleted(
            questId.value(), adventurerId.value(), reward.gold());
        rabbit.convertAndSend(exchange, routingKey, message);
    }
}
```

El nombre del *exchange* y la clave no están escritos aquí: los recibe el constructor, y los pone la raíz de composición desde `application.yml`, igual que la URL de Adventurers en el capítulo 9. El adaptador no tiene anotaciones de Spring; lo crea un `@Bean`.

11.4.2 Declarar exchange, colas y enlaces: Declarables

Antes de publicar o de consumir, el *exchange*, la cola y el enlace tienen que existir en el *broker*. Spring AMQP los crea por ti: Boot configura un `RabbitAdmin` que, al abrir la primera conexión, declara todos los objetos de tipo `Exchange`, `Queue` y `Binding` que encuentre como

beans. Declarar algo que ya existe con las mismas propiedades no hace nada. Para agruparlos se usa `Declarables`, un bean que contiene varios de esos objetos juntos [43]. Esta es la declaración de Treasury, en `bootstrap`. La clase se completará en las secciones siguientes (conversor, cola de mensajes muertos):

config/MessagingConfig.java (Treasury, primera versión)

```
package com.guildboard.treasury.config;

import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Declarables;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class MessagingConfig {

    @Bean
    public Declarables questCompletedDeclarables(
            @Value("${guildboard.messaging.exchange}") String exchange,
            @Value("${guildboard.messaging.queue}") String queue,
            @Value("${guildboard.messaging.routing-key}") String key) {
        TopicExchange events = new TopicExchange(exchange);
        Queue questCompleted = QueueBuilder.durable(queue).build();
        return new Declarables(events, questCompleted,
            BindingBuilder.bind(questCompleted).to(events).with(key));
    }
}
```

`TopicExchange` es duradero por omisión (sobrevive a un reinicio del *broker*), y `QueueBuilder.durable` crea una cola duradera. Los nombres salen del `application.yml` de Treasury:

application.yml (Treasury, fragmento)

```
guildboard:
  messaging:
    exchange: guildboard.events
    routing-key: quest.completed
    queue: treasury.quest-completed
```

11.4.3 Consumir: un adaptador de entrada con @RabbitListener

Un *listener* es el equivalente de un controlador: lo invoca algo de fuera (aquí el *broker*), traduce lo que llega a una orden del caso de uso y lo llama. No lleva reglas de negocio. Con la anotación `@RabbitListener` sobre un método de un bean, Spring crea un consumidor de la cola indicada y le pasa cada mensaje recibido [44]. El nombre de la cola puede llevar un marcador `${...}` que Spring sustituye por la propiedad, así que el nombre sigue viviendo solo en el `yml`.

El mensaje que llega a Treasury es un *record* propio, distinto del de Quests aunque tenga los mismos campos (la sección 11.5 explica por qué):

adapter/in/messaging/QuestCompleted.java (Treasury)

```
package com.guildboard.treasury.adapter.in.messaging;

public record QuestCompleted(
        String questId, String adventurerId, int reward) {
}
```

Y la primera versión del *listener*. Traduce el mensaje a `PayRewardCommand`, con los objetos de valor de Treasury, y llama al caso de uso de siempre. Es un `@Component` porque nadie más lo crea; los controladores web ya se encuentran de la misma forma:

adapter/in/messaging/QuestCompletedListener.java (primera versión)

```
package com.guildboard.treasury.adapter.in.messaging;

import com.guildboard.treasury.application.port.in.PayRewardCommand;
import com.guildboard.treasury.application.port.in.PayRewardUseCase;
import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.Gold;
import com.guildboard.treasury.domain.QuestId;
import lombok.RequiredArgsConstructor;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;

@Component
@RequiredArgsConstructor
public class QuestCompletedListener {

    private final PayRewardUseCase payReward;

    @RabbitListener(queues = "${guildboard.messaging.queue}")
    public void onQuestCompleted(QuestCompleted message) {
        payReward.pay(new PayRewardCommand(
            new QuestId(message.questId()),
            new AdventurerId(message.adventurerId()),
            new Gold(message.reward())));
    }
}
```

Cuándo se da el *ack*: por omisión, Spring AMQP lo envía cuando el método *termina sin excepción*. Si lanza una excepción, el mensaje no se da por procesado. Esa es la clave de las secciones 11.6 y 11.7. Y esta primera versión tiene un fallo a propósito: si el mismo mensaje llega dos veces, la segunda llamada a `pay` lanza `DomainException` (`QuestAlreadyPaid`) y el mensaje se trata como fallido.

11.5 El evento como mensaje JSON

Un mensaje de RabbitMQ no es un objeto Java: son bytes con unas cuantas propiedades (entre ellas `content_type`, que dice cómo leerlos). Entre `convertAndSend` y el *broker* hay un *conversor de mensajes* que transforma el objeto en bytes, y entre el *broker* y el *listener* hay otro que hace lo contrario. Si no configuras ninguno, Spring AMQP usa la serialización nativa de Java: exige que la clase sea `Serializable`, ata a quien publica y a quien consume a la *misma clase* y no se

puede leer en la consola. Justo lo que el capítulo 9 nos enseñó a evitar con los contratos.

Elegimos **JSON**: legible, independiente del lenguaje, y el contrato es el formato del texto y no una clase compartida. Para `QuestCompleted` el mensaje es:

Cuerpo del mensaje (JSON)

```
{"questId": "3f9a...", "adventurerId": "7d1c...", "reward": 150}
```

Ese JSON es el contrato entre Quests y sus consumidores. Cada lado tiene su *record*: el de Quests (sección 11.4) para escribirlo y el de cada consumidor para leerlo, sin proyecto común ni clase compartida, con la misma regla que el `AdventurerPayload` del capítulo 9. Si Quests añade un campo mañana, los consumidores no tienen que recompilarse a la vez; y si cada uno tolera los campos que no conoce, ni siquiera notan el cambio. Por eso el *record* de cada consumidor lleva `@JsonIgnoreProperties(ignoreUnknown = true)`, la misma anotación del capítulo 9. El de Treasury queda así (el de Adventurers es el ejercicio 11.2):

adapter/in/messaging/QuestCompleted.java (Treasury, versión final)

```
package com.guildboard.treasury.adapter.in.messaging;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;

@JsonIgnoreProperties(ignoreUnknown = true)
public record QuestCompleted(
        String questId, String adventurerId, int reward) {
}
```

11.5.1 El conversor: qué se verificó y qué no

Spring AMQP trae conversores a JSON basados en Jackson. En Boot 4 hay dos generaciones, y conviene elegir la correcta. Lo que comprobé el 9 de octubre de 2026, leyendo los *jars* de Maven Central de la versión que gestiona Boot 4.1.1 (`spring-amqp` 4.1.1):

- `org.springframework.amqp.support.converter.JacksonJsonMessageConverter` existe y está construido sobre **Jackson 3**: su constructor recibe un `tools.jackson.databind.json.JsonMapper`. Es el conversor que usamos.
- `Jackson2JsonMessageConverter`, el de Jackson 2 (el nombre que verás en tutoriales antiguos), sigue en el *jar* pero está marcado como obsoleto (`@Deprecated`). No lo uses aquí.
- Boot busca un bean de tipo `MessageConverter` y, si hay uno solo, lo asocia a la `RabbitTemplate` y a la fábrica de contenedores de los `@RabbitListener`. Lo leí en el código de `RabbitAutoConfiguration` y lo dice también la documentación [44]: si defines un bean `MessageConverter`, se asocia automáticamente a la fábrica por omisión. Por eso basta con declarar el bean.
- Para leer, el tipo que se construye sale por omisión del *parámetro del método* del *listener* (precedencia «inferida»), y no de la cabecera `__TypeId__` que el publicador añade con el nombre de *su* clase. Comprobado en el código de `DefaultJacksonJavaTypeMapper`, que arranca con esa precedencia. Si no fuera así, Treasury intentaría cargar `com.guildboard.quests...QuestCompleted` y fallaría.

Lo que no he podido comprobar: que el mensaje se escriba y se lea de verdad con estas clases en ejecución (el Java del curso no se compila). Lo verificarás tú en el escenario de la sección 11.10.1. Una señal de que algo falla es un mensaje en la cola de mensajes muertos de la sección 11.7.

El bean es de dos líneas. Va en `MessagingConfig` de cada servicio. La de Quests queda completa; en Treasury (y en Adventurers) se añade el método `messageConverter` a la clase de la sección anterior:

config/MessagingConfig.java (Quests)

```java
package com.guildboard.quests.config;

import org.springframework.amqp.core.Declarables;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.amqp.support.converter.JacksonJsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class MessagingConfig {

    @Bean
    public MessageConverter messageConverter() {
        return new JacksonJsonMessageConverter();
    }

    @Bean
    public Declarables eventsExchange(
            @Value("${guildboard.messaging.exchange}") String exchange) {
        return new Declarables(new TopicExchange(exchange));
    }
}
```

config/MessagingConfig.java (Treasury): método que se añade

```java
    @Bean
    public MessageConverter messageConverter() {
        return new JacksonJsonMessageConverter();
    }
```

(Con los `import` de `JacksonJsonMessageConverter` y `MessageConverter` de arriba, del paquete `org.springframework.amqp.support.converter`. Las dos clases de Treasury y Adventurers completas, con la cola de mensajes muertos, son las de la solución del ejercicio 11.4.)

Un detalle que ahorra horas: el cuerpo de un mensaje publicado a mano desde la consola de gestión necesita la propiedad `content_type` con el valor `application/json`; sin ella, no es seguro que el conversor lo trate como JSON.

11.6 Mensajes duplicados: consumidor idempotente

RabbitMQ ofrece entrega *al menos una vez*: garantiza que el mensaje llega, no que llega una sola. El motivo es el *ack*. El consumidor procesa, y justo antes de avisar al *broker*, se cae o se corta la conexión. El *broker* no sabe si el mensaje se procesó y, para no perderlo, lo entrega otra vez. Es el reverso de la garantía: perder no, pero repetir sí. Otros dos casos habituales de repetición: un reintento (sección 11.7) tras un fallo a medias, y un publicador que reenvía porque no recibió la confirmación.

La salida es diseñar al consumidor para que repetir no cambie nada: ser *idempotente*. Es el patrón «Idempotent Consumer» del catálogo de Richardson [42], y Treasury ya tiene la mitad hecha: el capítulo 10 protegió el pago con la comprobación previa y la restricción única, y repetir una misión da `QuestAlreadyPaid`. Falta la otra mitad, que es del adaptador: **una repetición no es un fallo**. Si el *listener* dejara escapar la excepción, el mensaje duplicado se reintentaría y acabaría en la cola de mensajes muertos como si hubiera algo roto. El *listener* la captura, registra que el mensaje ya estaba procesado y termina con normalidad, que provoca el *ack*. Esta es su versión final:

adapter/in/messaging/QuestCompletedListener.java (final)

```java
package com.guildboard.treasury.adapter.in.messaging;

import com.guildboard.treasury.application.port.in.PayRewardCommand;
import com.guildboard.treasury.application.port.in.PayRewardUseCase;
import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.DomainError;
import com.guildboard.treasury.domain.DomainException;
import com.guildboard.treasury.domain.Gold;
import com.guildboard.treasury.domain.QuestId;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;

@Slf4j
@Component
@RequiredArgsConstructor
public class QuestCompletedListener {

    private final PayRewardUseCase payReward;

    @RabbitListener(queues = "${guildboard.messaging.queue}")
    public void onQuestCompleted(QuestCompleted message) {
        PayRewardCommand command = new PayRewardCommand(
            new QuestId(message.questId()),
            new AdventurerId(message.adventurerId()),
            new Gold(message.reward()));
        try {
            payReward.pay(command);
        } catch (DomainException e) {
            if (!(e.getError() instanceof DomainError.QuestAlreadyPaid)) {
                throw e;
            }
            log.info("quest {} already paid, message ignored",
                message.questId());
```

```
        }
    }
}
```

Observa lo que *no* hace: no decide nada de negocio. «Una misión se paga una vez» sigue siendo del caso de uso y de la base de datos; el *listener* solo decide qué hacer con la respuesta del núcleo (para mensajería, ya hecho es éxito). El ejercicio 11.3 prueba la regla en el caso de uso.

Adventurers no es idempotente

Dar experiencia *suma* puntos: si el mismo `QuestCompleted` llega dos veces, el aventurero cobra la experiencia dos veces, y nada lo impide. Para evitarlo hace falta guardar qué mensajes se han procesado (el patrón de Richardson lo hace con una tabla de identificadores de mensaje) y que el mensaje lleve un identificador único. GuildBoard no lo implementa: el evento no tiene identificador, y el curso lo deja como riesgo conocido. A diferencia de Treasury, que lo tiene resuelto con su restricción única, aquí nada detiene la repetición. Lo verás en el escenario.

11.7 Mensajes que fallan: reintentos y cola de mensajes muertos

Un *listener* falla por dos razones muy distintas:

- **Fallo transitorio**: la base de datos no responde un segundo. Si lo intentas otra vez, funciona.
- **Mensaje venenoso** (*poison message*): el contenido es inválido (`reward` negativo, un aventurero que no existe, un JSON roto). Por muchas veces que lo intentes, falla siempre.

Qué hace Spring AMQP depende de la configuración, y el comportamiento por omisión es el peor. Según la documentación de Boot [44], con los reintentos desactivados (que es el valor por omisión), si el *listener* lanza una excepción, **el mensaje se vuelve a entregar indefinidamente**: un mensaje venenoso se procesa en bucle, llena el registro y bloquea al consumidor.

La receta tiene dos mitades:

1. Reintentos con espera creciente, en el consumidor. Se activan por configuración, en el `application.yml` del servicio que consume. Una trampa de Boot 4 que verás en tutoriales antiguos: la propiedad que contaba los intentos se llamaba `max-attempts`; en Boot 4 está marcada como error y se sustituye por `max-retries` (lo leí en los metadatos de configuración de Boot 4.1.1).

`application.yml (fragmento, consumidor)`

```yaml
spring:
  rabbitmq:
    listener:
      simple:
        retry:
          enabled: true
          max-retries: 3
          initial-interval: 1s
          multiplier: 2
          max-interval: 10s
```

Con esto, tras un fallo el *listener* espera 1 segundo y reintenta, luego 2, luego 4, hasta tres reintentos (el valor por omisión de `max-retries` es también 3; `multiplier` por omisión es 1, sin crecimiento). Cuántas veces se ejecuta exactamente tu método en total, cuéntalo en el registro cuando hagas el ejercicio 11.4: la descripción de la propiedad habla de «reintentos» y no lo he ejecutado. Cuando se agotan, Boot rechaza el mensaje *sin reponerlo en la cola* (usa por omisión un `RejectAndDontRequeueRecoverer`). Y entonces, o se tira, o va a donde le diga la cola.

2. Una cola de mensajes muertos (*dead-letter queue*, DLQ). Es una cola normal, con un *exchange* normal delante (el *dead-letter exchange*, DLX), a la que RabbitMQ envía los mensajes rechazados sin reposición. Se activa con dos argumentos en la cola *de origen*: a qué *exchange* se reenvía el mensaje muerto y con qué clave. En Spring AMQP se escriben con `QueueBuilder`; esta es la forma:

Forma de una cola con DLQ (no es el archivo completo)

```
Queue origen = QueueBuilder.durable("cola.origen")
    .deadLetterExchange("mi.dlx")      // x-dead-letter-exchange
    .deadLetterRoutingKey("mi.dlq")    // x-dead-letter-routing-key
    .build();
```

Y hay que declarar el DLX, la DLQ y el enlace entre ellos, como cualquier otro *exchange*, cola y enlace (con `Declarables`). Lo comprobé a mano contra RabbitMQ 4 con la API de gestión: declaré una cola con esos dos argumentos, rechacé un mensaje sin reponerlo y apareció en la cola de mensajes muertos.

Los argumentos de una cola no se pueden cambiar

Una cola ya creada tiene fijados sus argumentos. Si añades la DLQ a una cola que existe, RabbitMQ rechaza la declaración: lo probé declarando de nuevo una cola con DLX sin los argumentos, y la respuesta fue un error `inequivalent arg 'x-dead-letter-exchange'`. La solución, en desarrollo, es **borrar la cola vieja** (desde la consola, en la pestaña de colas, o con `docker compose down -v` si no te importan los datos de RabbitMQ) y arrancar de nuevo el servicio para que la declare con los argumentos nuevos. En producción esto es una migración con cuidado, no un borrado.

Otras tres cosas que saber:

- Un cuerpo que no es JSON válido no llega ni a la zona de reintentos: el conversor lanza `MessageConversionException`, que Spring AMQP trata como un fallo que no tiene sentido repetir y rechaza el mensaje de inmediato (no lo he ejecutado).
- La DLQ no se vacía sola. Alguien tiene que mirarla (hay métricas, capítulo 15) y decidir: corregir el dato y republicar, o descartar. Reponer automáticamente los mensajes muertos reproduce el bucle de antes.
- Reintentar es seguro aquí porque el consumidor de Treasury es idempotente. Con Adventurers, un reintento tras un fallo a medias puede dar la experiencia dos veces: el riesgo anterior tiene también esta puerta.

11.8 Lo que esto no resuelve: el evento perdido

Un consumidor idempotente protege de una cosa: recibir el mismo mensaje más de una vez. No protege de no recibirlo. Y hay un hueco en el diseño de Quests que lo deja a la vista: dentro de `CompleteQuestService`, la misión se guarda (`quests.save`) y *después* se publica

(`events.questCompleted`). Son dos sistemas distintos, la base de datos de Quests y el *broker*, y no hay nada que los haga atómicos. Entre las dos líneas, o durante la segunda, pueden pasar tres cosas:

1. **El proceso muere entre guardar y publicar.** La misión queda completada y no sale ningún evento.
2. **El *broker* está caído o no confirma.** `convertAndSend` lanza una excepción de conexión; el servicio responde `500`, pero la misión ya está guardada como completada.
3. **El mensaje se publica pero nadie lo recibe.** No hay ninguna cola enlazada (nadie ha arrancado nunca), y el *exchange* lo descarta, como vimos en la sección 11.3.

En los tres casos, la misión está completada y **ni se paga ni da experiencia**, sin ninguna señal de error útil. Y no hay forma de repararlo desde la API: completar de nuevo da `409` (ya está completada), así que el evento no se puede volver a publicar.

La segunda se puede provocar. Con RabbitMQ parado, completar una misión debería dar esto (salida *esperada*; no la he ejecutado):

```
$ docker compose stop rabbitmq
$ curl -s -o /dev/null -w "%{http_code}\n" -X POST \
    localhost:8081/quests/$Q1/complete
500
$ curl -s localhost:8081/quests/$Q1
{"id":"3f9a...","title":"Limpiar el s^^c3^^b3tano",...,"status":"COMPLETED",...}
```

La misión figura como completada (el `500` llegó después de guardar) y Treasury no ha recibido nada.

Qué haría falta en un proyecto real. Dos patrones, ambos fuera de alcance de este curso:

- **Bandeja de salida transaccional (*transactional outbox*).** En lugar de publicar, Quests guarda el evento en una tabla `outbox` de su propia base *en la misma transacción* que la misión. Un proceso aparte lee la tabla y publica, y marca como enviado lo que el *broker* confirma. Si se cae, retoma desde la tabla. La publicación pasa a ser «al menos una vez» (puede repetir), que es justo lo que los consumidores idempotentes ya toleran: los dos patrones se complementan. El patrón está en el catálogo de Richardson [42].
- **Sagas.** Cuando una operación abarca varios servicios y alguno puede fallar *después* de que otro haya actuado, se coordinan los pasos y se deshacen con acciones compensatorias («devolver el oro»). El término viene del artículo de Garcia-Molina y Salem de 1987 [12], pensado para transacciones largas en una base de datos. GuildBoard no las necesita, porque ningún consumidor de `QuestCompleted` puede «tener que deshacer» la misión: pagar y dar experiencia son consecuencias, no condiciones.

Hay mitigaciones parciales que no cierran el hueco: Boot permite activar las confirmaciones del publicador y los retornos de mensajes no enrutables (`spring.rabbitmq.publisher-confirm-type`, `spring.rabbitmq.publisher-returns` y `spring.rabbitmq.template.mandatory`, propiedades comprobadas en los metadatos de Boot 4.1.1), que convierten la pérdida silenciosa en un error visible; y un proceso periódico que compare misiones completadas con pagos detectaría los huérfanos. Ninguna se implementa aquí.

Riesgo aceptado

GuildBoard acepta esta pérdida en el curso. No está resuelta: el consumidor idempotente cubre duplicados y no cubre ausencias. Quien lleve algo así a producción debe resolverlo antes (con una bandeja de salida) o saber que puede haber misiones cobradas sin pagar.

11.9 Por qué RabbitMQ y no Kafka (contexto)

Esta sección es lectura de contexto: no hay ejercicio ni se instala nada. Apache Kafka es la otra opción habitual y Spring también la integra [45]. La diferencia de fondo está en qué es un mensaje después de leerlo:

	RabbitMQ	**Kafka**
Modelo	Colas: el *broker* reparte los mensajes y los borra tras el *ack*.	Registro de eventos: cada tema guarda los mensajes en orden durante un tiempo y cada grupo de consumidores lleva su posición.
Reprocesar el pasado	No: lo consumido ya no está.	Sí: se puede volver a leer desde una posición antigua.
Enrutado	Flexible: tipos de *exchange*, claves, enlaces.	Por tema y partición.
Volumen típico	Mensajes sueltos y tareas.	Grandes flujos de datos.
En Spring	`RabbitTemplate`, `@RabbitListener`.	`KafkaTemplate`, `@KafkaListener`, reintentos, lectura desde una posición y semántica de «exactamente una vez» documentados en Spring for Apache Kafka [59].

GuildBoard necesita un evento de uno a muchos, con una cola por consumidor, sin reprocesar el histórico y con poco volumen: el caso para el que RabbitMQ es más simple de entender, con el modelo *exchange*–cola. Kafka empezaría a compensar si Treasury quisiera reconstruir sus saldos releyendo todo el historial de misiones o si el volumen creciera mucho. La documentación del *broker* está en la web de Kafka [2]. Un dato verificado en esta sesión: Boot 4.1.1 gestiona Spring for Apache Kafka 4.1.1 y Initializr genera `spring-boot-starter-kafka` para la dependencia `kafka`, así que la integración con Boot 4 existe. La elección es de ajuste, no de calidad.

11.10 En el proyecto: hito H6 (parte 1)

Hito 6, parte 1: completar una misión mueve el oro y la experiencia

Al terminar el ejercicio 11.5, GuildBoard tiene un *broker* y tres servicios que se hablan por eventos:

- **Quests** publica `QuestCompleted` en el *exchange* `guildboard.events` cada vez que se completa una misión, después de guardarla.
- **Treasury** consume el evento desde la cola `treasury.quest-completed` y paga la recompensa (una vez por misión), con reintentos y una cola de mensajes muertos.
- **Adventurers** lo consume desde `adventurers.quest-completed` y suma la recompensa como experiencia, con los mismos reintentos y su propia cola de mensajes

muertos.

- Desaparecen `POST /payments` y `POST /adventurers/{id}/experience`. La experiencia vuelve a darse al completar una misión, como antes del capítulo 9.

Queda un riesgo por diseño: el evento puede perderse (sección 11.8) y Adventurers no es idempotente (sección 11.6). La llamada REST de Quests a Adventurers para el rango no cambia.

11.10.1 Probar el hito

Necesitas el *broker* y los tres servicios. Los consumidores arrancan primero para que existan sus colas antes de la primera publicación (sección 11.3), cada uno en su terminal y en modo memoria:

```
$ docker compose up -d --wait rabbitmq
$ ./gradlew :adventurers-bootstrap:bootRun        # terminal 1
$ ./gradlew :treasury-bootstrap:bootRun           # terminal 2
$ ./gradlew :quests-bootstrap:bootRun             # terminal 3
```

En la consola (`http://localhost:15672`, pestaña de colas) deben aparecer `treasury.quest-completed`, `treasury.quest-completed.dlq`, `adventurers.quest-completed` y `adventurers.quest-completed.dlq`. En una cuarta terminal, el recorrido de siempre (los ids son los tuyos), pero sin ningún paso manual en Treasury:

```
$ curl -s -X POST localhost:8082/adventurers \
    -H "Content-Type: application/json" -d '{"name":"Mara"}'
$ MARA=7d1c4e52-...
$ curl -s -X POST localhost:8081/quests \
    -H "Content-Type: application/json" \
    -d '{"title":"Limpiar el s^^c3^^b3tano","reward":150,
         "requiredRank":"NOVICE"}'
$ Q1=3f9a...
$ curl -s -X POST localhost:8081/quests/$Q1/accept \
    -H "Content-Type: application/json" \
    -d "{\"adventurerId\":\"$MARA\"}"
$ curl -s -X POST localhost:8081/quests/$Q1/complete
```

La respuesta es la de siempre (la misión, con estado `COMPLETED`). Un instante después, comprueba los dos efectos (salidas *esperadas*; no he ejecutado el código Java del libro):

```
$ curl -s localhost:8083/accounts/$MARA
{"adventurerId":"7d1c4e52-...","balance":150}
$ curl -s localhost:8082/adventurers/$MARA
{"id":"7d1c4e52-...","name":"Mara","rank":"ADEPT","experience":150}
```

Con 150 de experiencia, Mara ya es `ADEPT` (umbral 100). Ahora un duplicado, a propósito. Escribe un archivo `publish.json` con el mensaje dentro del cuerpo que espera la API de gestión de RabbitMQ (el `sed` cambia `QID` y `AID` por tus ids) y publícalo otra vez en el *exchange*, como si el *broker* repitiera la entrega:

```
$ cat > publish.json <<'EOF'
{"properties": {"content_type": "application/json"},
 "routing_key": "quest.completed",
 "payload_encoding": "string",
 "payload": "{\"questId\":\"QID\",\"adventurerId\":\"AID\",\"reward\":150}"}
EOF
$ sed -i "s/QID/$Q1/;s/AID/$MARA/" publish.json
$ curl -s -u guildboard:guildboard -X POST \
    -H "content-type: application/json" -d @publish.json \
    localhost:15672/api/exchanges/%2F/guildboard.events/publish
{"routed":true}
$ curl -s localhost:8083/accounts/$MARA
{"adventurerId":"7d1c4e52-...","balance":150}
$ curl -s localhost:8082/adventurers/$MARA
{"id":"7d1c4e52-...","name":"Mara","rank":"VETERAN","experience":300}
```

Esto enseña los dos lados de la sección 11.6 con una sola prueba: Treasury sigue en 150 (el duplicado se reconoce y se ignora) y Adventurers pasó a 300 (lo cobró dos veces). El primero es el diseño; el segundo, el riesgo reconocido. Para ver el hueco de la sección 11.8, para RabbitMQ (`docker compose stop rabbitmq`), completa otra misión y comprueba que da `500` pero queda completada.

Comprueba también con `./gradlew test` y `./gradlew build` que todo está en verde y haz el *commit* (`H6a: QuestCompleted por RabbitMQ`). Los dos `POST` provisionales ya no responden: Treasury y Adventurers solo se modifican por eventos.

11.11 Ejercicios

Los ejercicios parten del proyecto tal como lo dejan las secciones anteriores y se hacen en este orden. Los cuatro primeros son piezas sueltas; el último las junta con el resto del hito.

Ejercicio 11.1: RabbitMQ en Compose y un mensaje a mano ★★☆☆☆

Parte de `compose.yaml` con solo `postgres`. Vas a levantar el *broker* y hacer pasar un mensaje por él sin escribir una línea de Java:

1. Añade el servicio `rabbitmq` y el volumen `rabbitmq-data` (sección 11.3).
2. Arranca solo el *broker* con `docker compose up -d -wait rabbitmq` y comprueba con `docker compose ps` que está *healthy*.
3. Abre `http://localhost:15672` e identifícate con `guildboard` y `guildboard`.
4. Crea una cola duradera llamada `demo` (pestaña de colas, formulario «Add a new queue»).
5. En la página de la cola, publica con el panel «Publish message» el texto `hola` y comprueba que la cola muestra un mensaje listo.
6. Con el panel «Get messages», lee el mensaje dejando «Ack Mode» en «Nack message requeue true» (lo lee y lo repone). Después léelo con «Ack message requeue false» y comprueba que la cola queda vacía.

Está bien si: `docker compose ps` muestra `rabbitmq` como *healthy* con los puertos 5672 y 15672; la cola `demo` se ve en la consola; la primera lectura devuelve `hola` y deja el mensaje en la cola; la segunda lo devuelve y la deja vacía; y puedes contestar por qué el primer panel

de publicación no te pidió ningún *exchange*.

Pista: *la consola publica por el* exchange *por omisión, llamado «(AMQP default)», que enruta cada mensaje a la cola cuyo nombre coincide con la clave.*

Guiado ▪ solución en la pág. 421

Ejercicio 11.2: El listener de Adventurers ★★★☆☆

Parte de Adventurers con los puertos de entrada del capítulo 5. Escribe el adaptador de entrada de mensajería, sin tocar `domain` ni `application`:

1. Añade el starter de AMQP al `build.gradle` de `adventurers-adapters`.
2. En `com.guildboard.adventurers.adapter.in.messaging`, crea el *record* `QuestCompleted(String questId, String adventurerId, int reward)` con su propia copia (sin importar nada de Quests) y tolerante a campos desconocidos.
3. En el mismo paquete, crea el *listener* `QuestCompletedListener`, un bean que escucha la cola cuyo nombre da la propiedad `guildboard.messaging.queue` y, por cada mensaje, llama a `GrantExperienceUseCase.grant(new GrantExperienceCommand(adventurerId, points))`. Usa la recompensa como puntos. Las firmas de partida son `GrantExperienceCommand(AdventurerId adventurerId, int points)` (ambos en `com.guildboard.adventurers.application.port.in`, y `AdventurerId` en `com.guildboard.adventurers.domain`) y `Adventurer grant(GrantExperienceCommand)`.
4. Compila con `./gradlew :adventurers-adapters:compileJava` y pasa `./gradlew :adventurers-adapters:check`.
5. Contesta: ¿qué pasa si el mismo mensaje llega dos veces?

Está bien si: el módulo compila y `check` pasa (incluida la comprobación de aislamiento de `adapter.in` y `adapter.out`); el *listener* no contiene ninguna regla de negocio (ni `if` sobre puntos o rangos); ningún archivo importa nada de `com.guildboard.quests`; el nombre de la cola no está escrito en el código; y la clase es un bean de Spring.

Pista: *si olvidas la anotación que registra la clase como bean, la aplicación arranca sin errores y nadie consume la cola.*

Práctica ▪ solución en la pág. 423

Ejercicio 11.3: Pagar dos veces la misma misión, probado ★★★☆☆

Parte de `PayRewardService` del capítulo 10, ya idempotente. Escribe una prueba JUnit que fije esa garantía, en el módulo `treasury-application`:

1. No cambies su `build.gradle`: JUnit, AssertJ y el lanzador de pruebas ya llegan a todos los módulos desde `subprojects {}` (JUnit, capítulo 3).
2. Crea `PayRewardServiceIdempotencyTest` en el paquete `com.guildboard.treasury.application.service` de `src/test/java`.
3. Como `application` no ve los adaptadores, define en la propia prueba dos repositorios en memoria de unas líneas (clases anidadas privadas) que implementen `AccountRepository` y `LedgerRepository`.
4. Escribe tres pruebas: pagar dos veces la misma misión lanza `DomainException` con `QuestAlreadyPaid`, deja un solo asiento y un saldo de 150; la repetición con otro

importe se rechaza igual; y dos misiones distintas al mismo aventurero se pagan las dos.

5. Ejecuta `./gradlew :treasury-application:test`.

Está bien si: las tres pruebas pasan; ninguna importa Spring ni clases de `adapters`; el reloj es `Clock.fixed`; y, si comentas la comprobación `ledger.existsByQuestId` de `PayRewardService`, la primera prueba falla (la vuelves a dejar como estaba).

Pista: *para comprobar el tipo del error dentro de la excepción, mira `assertThatThrownBy(...).isInstanceOfSatisfying(...)` de AssertJ.*

Test ▪ solución en la pág. 425

Ejercicio 11.4: Reintentos y cola de mensajes muertos en Treasury ★★★☆☆

Parte de Treasury tal como queda tras el texto del capítulo: el starter de AMQP en sus dos `build.gradle`, la conexión y los nombres de *exchange*, clave y cola en su `application.yml`, el *listener* final de la sección 11.6 y la `MessagingConfig` con el conversor de la sección 11.5, pero sin reintentos ni cola de mensajes muertos. Añádelos:

1. En el `application.yml` de Treasury, activa los reintentos (3, con espera inicial de 1 segundo, que se duplica hasta un máximo de 10) y añade las propiedades `guildboard.messaging.dead-letter-exchange` (valor `guildboard.events.dlx`) y `guildboard.messaging.dead-letter-queue` (valor `treasury.quest-completed.dlq`).
2. En `MessagingConfig`, declara el *exchange* de mensajes muertos (de tipo *direct*), la cola de mensajes muertos, su enlace con la clave igual al nombre de esa cola, y haz que la cola `treasury.quest-completed` apunte a ellos.
3. Borra la cola `treasury.quest-completed` si ya existía (consola de gestión) y arranca Treasury.
4. Publica un mensaje que Treasury no pueda procesar nunca: `{"questId":"q-9","adventurerId":"mara","reward":-5}` (con `content_type application/json`).
5. Observa el registro de Treasury y la consola, y publica a continuación un mensaje válido (por ejemplo, con `reward` 150 y un `questId` distinto de los anteriores, con el aventurero que quieras).

Está bien si: tras unos segundos, `treasury.quest-completed.dlq` tiene 1 mensaje con el mismo cuerpo y `treasury.quest-completed` tiene 0; el registro muestra el fallo repetido con esperas crecientes; el mensaje válido se paga con normalidad (el consumidor no se quedó bloqueado); y no has tocado ni `adapters` ni `application`. Contesta además: ¿qué error da RabbitMQ si no borras la cola antigua, y por qué?

Pista: *`Gold` rechaza importes negativos, por eso el mensaje falla siempre.*

Práctica ▪ solución en la pág. 428

Ejercicio 11.5: Hito H6a: QuestCompleted de Quests a Treasury y Adventurers ★★★★☆

Parte del proyecto con Quests, Adventurers y Treasury del capítulo 10, RabbitMQ en `compose.yaml` (ejercicio 11.1), y con el *listener* de Adventurers (ejercicio 11.2), la prueba de idempotencia y la cola de mensajes muertos de Treasury (ejercicios 11.3 y 11.4) hechos. Falta lo que lo convierte en un hito:

1. **Quests publica.** Puerto `QuestEventPublisher`, `CompleteQuestService` con el pu-

blicador, *record* y `RabbitQuestEventPublisher` (sección 11.4), `MessagingConfig` (sección 11.5), el bean del publicador en `BeanConfig`, el starter en los dos `build.gradle` y el `application.yml` (conexión, *exchange* y clave).

2. **Adventurers consume.** `MessagingConfig` con el conversor, el *exchange*, su cola `adventurers.quest-completed` y su cola de mensajes muertos; el starter en `bootstrap`; el `application.yml` con la conexión, los nombres y los reintentos.
3. **Treasury:** no hay nada nuevo que escribir. El starter en los dos `build.gradle` y la conexión del `application.yml` vienen de la instalación del capítulo, los nombres de la sección 11.5, y los reintentos y la cola de mensajes muertos del ejercicio 11.4. Comprueba que los tres están y que el `application.yml` coincide con el de la solución.
4. **Borra los andamios.** `POST /payments` y sus DTO en Treasury (deja `GET /accounts/{adventurerId}`) y `POST /adventurers/{id}/experience` con su DTO en Adventurers.
5. Ejecuta `./gradlew build`, el escenario de la sección 11.10.1 y haz el *commit*.

Está bien si: `./gradlew build` termina con `BUILD SUCCESSFUL` (incluidas las pruebas de arquitectura y la prueba del ejercicio 11.3); al completar una misión de 150, Treasury muestra saldo 150 y Adventurers experiencia 150; un duplicado deja el saldo en 150; con RabbitMQ parado, completar da `500` y la misión queda completada; `POST /payments` y `POST /adventurers/{id}/experience` ya no funcionan; `domain` y `application` de los tres servicios no importan nada de `org.springframework.amqp`; y ningún `build.gradle` de un servicio nombra a otro.

Pista: *cada archivo nuevo de Adventurers es la versión con otros nombres de uno de Treasury; los cambios de Quests son los de las secciones 11.4 y 11.5.*

Proyecto ▪ solución en la pág. 431

11.12 Resumen

Resumen

- Un evento es un hecho en pasado que se publica sin esperar respuesta; REST es para preguntas y órdenes cuya respuesta se necesita ya. A cambio, la consistencia es eventual. `AdventurerDirectory` sigue siendo REST; `QuestCompleted`, un evento.
- En RabbitMQ el productor entrega al *exchange*, los enlaces lo copian a las colas y cada consumidor lee la suya y da el *ack*. Un mensaje sin cola de destino se descarta en silencio, y cada consumidor tiene su propia cola.
- La verificación de estructura 7 muestra el árbol T6: paquetes `adapter/out/messaging` en Quests y `adapter/in/messaging` en Treasury y Adventurers, una `MessagingConfig` en cada `bootstrap`, RabbitMQ en Compose y los `POST` provisionales borrados. Se mantiene un solo módulo `adapters` aunque ya conviven web, JPA y AMQP.
- El núcleo no conoce el *broker*: el caso de uso pide un puerto de salida (`QuestEventPublisher`) y el publicador lo implementa con `RabbitTemplate`; el *listener* con `@RabbitListener` es un adaptador de entrada sin lógica de negocio. `Declarables` declara *exchange*, colas y enlaces como beans.
- El evento viaja como JSON con `JacksonJsonMessageConverter` (Jackson 3 en Boot 4; el de Jackson 2 está obsoleto). El contrato es el JSON y cada lado tiene su *record* propio,

tolerante a campos desconocidos; el tipo se infiere del parámetro del *listener*.

- La entrega es «al menos una vez»: hay duplicados. Treasury es idempotente (consulta previa, restricción única, y el *listener* trata «ya pagado» como éxito). Adventurers no lo es y el curso lo deja como riesgo.
- Con los reintentos desactivados, un mensaje que falla se reentrega sin fin. Con `max-retries` (no `max-attempts`) y una cola de mensajes muertos, lo que se agota va a la DLQ, que hay que vigilar. Los argumentos de una cola no se pueden cambiar sin borrarla.
- El consumidor idempotente cubre duplicados, no el evento perdido: si Quests guarda y cae antes de publicar, la misión queda completada sin pago ni experiencia. La solución (bandeja de salida; sagas si hubiera compensación) está fuera del curso, y GuildBoard acepta el riesgo.
- RabbitMQ encaja por ser un evento de uno a muchos sin reprocesar el pasado; Kafka (registro reproducible) sería la opción si hiciera falta releer el histórico.

Soluciones de los ejercicios

Solución del ejercicio 11.1 enunciado en la pág. 417

1. El archivo completo. `postgres` no cambia; el servicio nuevo y el volumen nuevo se añaden:

compose.yaml

```yaml
services:
  postgres:
    image: postgres:18
    environment:
      POSTGRES_DB: guildboard
      POSTGRES_USER: guildboard
      POSTGRES_PASSWORD: guildboard
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql
      - ./infra/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U guildboard -d guildboard"]
      interval: 5s
      timeout: 3s
      retries: 10

  rabbitmq:
    image: rabbitmq:4-management
    hostname: rabbitmq
    environment:
      RABBITMQ_DEFAULT_USER: guildboard
      RABBITMQ_DEFAULT_PASS: guildboard
    ports:
      - "5672:5672"
      - "15672:15672"
    volumes:
      - rabbitmq-data:/var/lib/rabbitmq
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 10s
      timeout: 5s
      retries: 10

volumes:
  postgres-data:
  rabbitmq-data:
```

2. Arrancar y comprobar.

```
$ docker compose up -d --wait rabbitmq
$ docker compose ps
NAME                  IMAGE                    ...  STATUS
guildboard-rabbitmq-1 rabbitmq:4-management    ...  Up (healthy)
```

(Salida resumida; el nombre del contenedor depende de la carpeta del proyecto.)

3 a 6. La consola. En `http://localhost:15672`: la pestaña de colas (*Queues*) tiene el formulario «Add a new queue»; pon `demo` y deja el tipo y la durabilidad por omisión. Dentro de la cola, «Publish message» con el texto `hola`, y en «Get messages», «Nack message requeue true» devuelve `hola` y deja el mensaje (`Redelivered: true` en la siguiente lectura), mientras que «Ack message requeue false» lo devuelve y lo borra. (Los nombres de los

paneles están escritos de memoria; el equivalente por API, que sí he ejecutado, es este.)

```
$ H='content-type: application/json'
$ U=guildboard:guildboard
$ B=localhost:15672/api
$ curl -s -u $U -H "$H" -X PUT $B/queues/%2F/demo -d '{"durable":true}'
$ curl -s -u $U -H "$H" -X POST $B/exchanges/%2F/amq.default/publish \
    -d '{"properties":{},"routing_key":"demo","payload":"hola",
         "payload_encoding":"string"}'
{"routed":true}
$ curl -s -u $U -H "$H" -X POST $B/queues/%2F/demo/get \
    -d '{"count":1,"ackmode":"ack_requeue_true","encoding":"auto"}'
$ curl -s -u $U -H "$H" -X POST $B/queues/%2F/demo/get \
    -d '{"count":1,"ackmode":"ack_requeue_false","encoding":"auto"}'
```

La primera lectura devuelve el mensaje y lo deja en la cola; la segunda lo devuelve (con `redelivered` a `true`) y la cola queda vacía.

Por qué funciona. La publicación del panel va al *exchange* por omisión (en la API se llama `amq.default`), que tiene un enlace implícito con cada cola, con el nombre de la cola como clave. Por eso no pide un *exchange*: solo una clave, que es el nombre de la cola. Un mensaje queda en la cola hasta que se confirma con *ack* o se rechaza sin reposición; «requeue true» equivale a decir «no lo he procesado, devuélvelo», y por eso queda disponible para otro.

Errores típicos: olvidar la entrada `rabbitmq-data` bajo el `volumes` de primer nivel (Compose responde que el volumen no está definido); tener ya algo escuchando en el puerto 5672 o 15672; entrar con `guest` en lugar del usuario del archivo; y levantar con `docker compose up -d` sin `-wait` y abrir la consola antes de que el *broker* esté listo.

Solución del ejercicio 11.2

enunciado en la pág. 418

1. La dependencia. Una línea en `services/adventurers/adapters/build.gradle`:

services/adventurers/adapters/build.gradle

```
dependencies {
    implementation project(':adventurers-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

2. El mensaje. Una copia propia, que no importa nada de Quests:

adapter/in/messaging/QuestCompleted.java (Adventurers)

```java
package com.guildboard.adventurers.adapter.in.messaging;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;

@JsonIgnoreProperties(ignoreUnknown = true)
public record QuestCompleted(
        String questId, String adventurerId, int reward) {
}
```

3. El *listener*. Traduce y delega; ni una regla de negocio:

adapter/in/messaging/QuestCompletedListener.java (Adventurers)

```java
package com.guildboard.adventurers.adapter.in.messaging;

import com.guildboard.adventurers.application.port.in.
    ↪ GrantExperienceCommand;
import com.guildboard.adventurers.application.port.in.
    ↪ GrantExperienceUseCase;
import com.guildboard.adventurers.domain.AdventurerId;
import lombok.RequiredArgsConstructor;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;

@Component
@RequiredArgsConstructor
public class QuestCompletedListener {

    private final GrantExperienceUseCase grantExperience;

    @RabbitListener(queues = "${guildboard.messaging.queue}")
    public void onQuestCompleted(QuestCompleted message) {
        grantExperience.grant(new GrantExperienceCommand(
            new AdventurerId(message.adventurerId()),
            message.reward()));
    }
}
```

4. Comprobar.

```
$ ./gradlew :adventurers-adapters:compileJava
$ ./gradlew :adventurers-adapters:check
BUILD SUCCESSFUL
```

Respuesta. Si el mismo mensaje llega dos veces, `grant` se ejecuta dos veces y el aventurero recibe la experiencia doble: este consumidor no es idempotente (el aviso de la sección 11.6). Corregirlo exigiría un identificador de mensaje y guardar los procesados.

Por qué funciona. El *listener* es un adaptador de entrada: lo invoca el *broker*, convierte el mensaje al comando del núcleo y llama al puerto. Spring AMQP construye el *record* desde el JSON porque el conversor de la sección 11.5 lee el tipo del parámetro del método, y la expresión `${guildboard.messaging.queue}` se resuelve al arrancar.

Criterios para valorar otras soluciones. Es válida si el *listener* solo traduce y delega, el *record* es propio, el nombre de la cola viene de la configuración y el módulo pasa `check`. Una variante válida pone la traducción a comando en una clase de mapeo, como en el lado web.
Errores típicos: importar el *record* de Quests (comparte clase entre servicios); escribir el nombre de la cola en la anotación (al cambiar de entorno, hay que recompilar); olvidar `@Component` (la aplicación arranca y no consume nada: la cola crece); capturar y tragarse la excepción de `grant`, con lo que el mensaje se da por procesado aunque fallara (se pierde); y llamar a un repositorio en lugar del caso de uso.

Solución del ejercicio 11.3

enunciado en la pág. 418

1. Dependencias de prueba. No hay que tocar nada: JUnit, AssertJ y el lanzador de pruebas llegan a `treasury-application` desde `subprojects {}`. Su `build.gradle` se queda como está.
2. La prueba. En `services/treasury/application/src/test/java/com/guildboard/treasury/application/service/`. Primera parte: la clase, el servicio con un reloj fijo y los dos auxiliares; la segunda, las tres pruebas; la tercera, los repositorios en memoria.

PayRewardServiceIdempotencyTest.java (1/3)

```
package com.guildboard.treasury.application.service;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

import com.guildboard.treasury.application.port.in.PayRewardCommand;
import com.guildboard.treasury.application.port.out.AccountRepository;
import com.guildboard.treasury.application.port.out.LedgerRepository;
import com.guildboard.treasury.domain.Account;
import com.guildboard.treasury.domain.AdventurerId;
import com.guildboard.treasury.domain.DomainError;
import com.guildboard.treasury.domain.DomainException;
import com.guildboard.treasury.domain.Gold;
import com.guildboard.treasury.domain.LedgerEntry;
import com.guildboard.treasury.domain.QuestId;
import java.time.Clock;
import java.time.Instant;
import java.time.ZoneOffset;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import org.junit.jupiter.api.Test;

class PayRewardServiceIdempotencyTest {

    private final MemoryAccounts accounts = new MemoryAccounts();
    private final MemoryLedger ledger = new MemoryLedger();
    private final PayRewardService service = new PayRewardService(
        accounts, ledger, Clock.fixed(
            Instant.parse("2026-10-09T10:00:00Z"), ZoneOffset.UTC));
```

PayRewardServiceIdempotencyTest.java (2/3: sigue en la clase)

```
    private static PayRewardCommand pay(String quest, int coins) {
        return new PayRewardCommand(new QuestId(quest),
            new AdventurerId("mara"), new Gold(coins));
    }

    private Gold balance() {
        return accounts.findById(new AdventurerId("mara"))
            .orElseThrow().getBalance();
    }

    @Test
    void secondPaymentOfTheSameQuestIsRejected() {
        service.pay(pay("q-1", 150));

        assertThatThrownBy(() -> service.pay(pay("q-1", 150)))
            .isInstanceOfSatisfying(DomainException.class, e ->
                assertThat(e.getError())
                    .isInstanceOf(DomainError.QuestAlreadyPaid.class));
        assertThat(ledger.entries).hasSize(1);
        assertThat(balance()).isEqualTo(new Gold(150));
    }

    @Test
    void repetitionWithAnotherAmountIsRejectedToo() {
        service.pay(pay("q-1", 150));

        assertThatThrownBy(() -> service.pay(pay("q-1", 999)))
            .isInstanceOf(DomainException.class);
        assertThat(balance()).isEqualTo(new Gold(150));
    }

    @Test
    void differentQuestsArePaidBoth() {
        service.pay(pay("q-1", 150));
        service.pay(pay("q-2", 80));

        assertThat(ledger.entries).hasSize(2);
        assertThat(balance()).isEqualTo(new Gold(230));
    }
```

PayRewardServiceIdempotencyTest.java (3/3: repositorios en memoria)

```
    private static class MemoryAccounts implements AccountRepository {

        private final Map<AdventurerId, Account> accounts =
            new HashMap<>();

        @Override
        public Optional<Account> findById(AdventurerId adventurerId) {
            return Optional.ofNullable(accounts.get(adventurerId));
        }

        @Override
        public void save(Account account) {
            accounts.put(account.getOwner(), account);
        }
    }

    private static class MemoryLedger implements LedgerRepository {

        private final List<LedgerEntry> entries = new ArrayList<>();

        @Override
        public boolean existsByQuestId(QuestId questId) {
            return entries.stream()
                .anyMatch(entry -> entry.questId().equals(questId));
        }

        @Override
        public void save(LedgerEntry entry) {
            entries.add(entry);
        }

        @Override
        public List<LedgerEntry> findByAdventurerId(
                AdventurerId adventurerId) {
            return entries.stream().filter(entry ->
                entry.adventurerId().equals(adventurerId)).toList();
        }
    }
}
```

(Las tres partes son un único archivo: la 2 y la 3 continúan dentro de la clase de la 1, y la última llave cierra la clase.) Ejecuta:

```
$ ./gradlew :treasury-application:test
BUILD SUCCESSFUL
```

Por qué funciona. La prueba vive en `application`, que no ve los adaptadores, así que los repositorios en memoria se definen dentro de la propia prueba: cumplen los puertos con un mapa y una lista. `PayRewardService` se construye a mano, sin Spring, como en producción. Con un reloj fijo, el resultado no depende de la hora. La clave de idempotencia es la misión, no el importe: la segunda prueba lo fija.

Qué no prueba. La restricción única de la base de datos (necesitaría PostgreSQL) ni dos pagos simultáneos: esas garantías son del adaptador JPA y de `V2`. Esta prueba fija la primera defensa, la consulta previa.
Comprobación extra. Comenta el bloque `if` de `PayRewardService` que consulta `ledger.existsByQuestId`: la primera prueba debe fallar (el segundo pago se acepta). Si no falla, la prueba no prueba nada. Vuelve a dejar el servicio como estaba.
Errores típicos: olvidar las dos líneas de `build.gradle` (el compilador no encuentra `org.junit`); comparar `Gold` con un `int` en lugar de con otro `Gold`; usar `@SpringBootTest` para algo que no necesita Spring; probar solo que no lanza excepción, sin mirar el saldo; y escribir los repositorios en memoria como clases públicas de `adapters`, que `application` no puede importar.

Solución del ejercicio 11.4 enunciado en la pág. 419

1. Configuración. Las propiedades nuevas de Treasury (el `application.yml` completo está en la solución del ejercicio de proyecto):

`application.yml (Treasury): lo que se añade`

```yaml
spring:
  rabbitmq:
    listener:
      simple:
        retry:
          enabled: true
          max-retries: 3
          initial-interval: 1s
          multiplier: 2
          max-interval: 10s

guildboard:
  messaging:
    dead-letter-exchange: guildboard.events.dlx
    dead-letter-queue: treasury.quest-completed.dlq
```

2. Declaraciones. El *exchange* de mensajes muertos, la cola, su enlace y los dos argumentos de la cola de origen. Es la clase final de Treasury:

config/MessagingConfig.java (Treasury, final)

```java
package com.guildboard.treasury.config;

import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Declarables;
import org.springframework.amqp.core.DirectExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.amqp.support.converter.
    ↪ JacksonJsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class MessagingConfig {

    @Bean
    public MessageConverter messageConverter() {
        return new JacksonJsonMessageConverter();
    }

    @Bean
    public Declarables questCompletedDeclarables(
            @Value("${guildboard.messaging.exchange}") String exchange,
            @Value("${guildboard.messaging.routing-key}") String key,
            @Value("${guildboard.messaging.queue}") String queue,
            @Value("${guildboard.messaging.dead-letter-exchange}")
            String deadExchange,
            @Value("${guildboard.messaging.dead-letter-queue}")
            String deadQueue) {
        TopicExchange events = new TopicExchange(exchange);
        DirectExchange dead = new DirectExchange(deadExchange);
        Queue questCompleted = QueueBuilder.durable(queue)
            .deadLetterExchange(deadExchange)
            .deadLetterRoutingKey(deadQueue)
            .build();
        Queue deadLetters = QueueBuilder.durable(deadQueue).build();
        return new Declarables(events, dead, questCompleted, deadLetters,
            BindingBuilder.bind(questCompleted).to(events).with(key),
            BindingBuilder.bind(deadLetters).to(dead).with(deadQueue));
    }
}
```

3 a 5. Borrar la cola, arrancar y publicar. Borra la cola vieja desde la consola de gestión (pestaña de colas, la cola, «Delete») o por API, y arranca Treasury. Después, el mensaje que siempre falla (con el `sed` no hace falta: no lleva ids):

```
$ U=guildboard:guildboard
$ B=localhost:15672/api
$ curl -s -u $U -X DELETE $B/queues/%2F/treasury.quest-completed
$ ./gradlew :treasury-bootstrap:bootRun     # otra terminal
$ cat > poison.json <<'EOF'
{"properties": {"content_type": "application/json"},
 "routing_key": "quest.completed",
 "payload_encoding": "string",
 "payload": "{\"questId\":\"q-9\",\"adventurerId\":\"mara\",\"reward\":-5}"}
EOF
$ curl -s -u $U -H "content-type: application/json" -X POST \
    -d @poison.json $B/exchanges/%2F/guildboard.events/publish
{"routed":true}
```

Unos segundos después, mira la cola de mensajes muertos (las cifras de la API tardan unos segundos en refrescarse):

```
$ curl -s -u $U $B/queues/%2F/treasury.quest-completed.dlq \
    | grep -o '"messages_ready":[0-9]*'
"messages_ready":1
```

(Salida esperada; el efecto de rechazar un mensaje hacia una cola con DLX lo comprobé a mano contra RabbitMQ 4, pero no con este *listener*.) En el registro de Treasury verás el mismo fallo (`IllegalArgumentException` de `Gold`) varias veces, separado por esperas de 1, 2 y 4 segundos. Cuenta cuántas veces sale: es la forma de saber cuántas ejecuciones implica `max-retries: 3`.

5. Un mensaje válido después. Publica uno con otro `questId` y comprueba que se paga (es la prueba de que el consumidor no se quedó bloqueado):

```
$ cat > ok.json <<'EOF'
{"properties": {"content_type": "application/json"},
 "routing_key": "quest.completed",
 "payload_encoding": "string",
 "payload": "{\"questId\":\"q-10\",\"adventurerId\":\"mara\",\"reward\":150}"}
EOF
$ curl -s -u $U -H "content-type: application/json" -X POST \
    -d @ok.json $B/exchanges/%2F/guildboard.events/publish
{"routed":true}
$ curl -s localhost:8083/accounts/mara
```

(La respuesta de la cuenta debe mostrar un saldo de 150 y la cola de mensajes muertos sigue con 1 mensaje.)

Respuestas. Si no borras la cola, RabbitMQ rechaza la declaración con un error `inequivalent arg 'x-dead-letter-exchange'`, porque la cola existente ya tiene fijados sus argumentos (sin DLX) y la nueva declaración pide otros distintos. Un mensaje válido posterior se paga con normalidad: mientras el mensaje venenoso espera sus reintentos el consumidor está ocupado, pero al agotarlos se libera.

Por qué funciona. Los reintentos están en la configuración del contenedor de listeners, no en el código. Al agotarlos, Boot rechaza el mensaje sin reponerlo, y RabbitMQ, que ve en la cola de origen `x-dead-letter-exchange`, lo reenvía a ese *exchange* con la clave

`x-dead-letter-routing-key`; el enlace del *exchange* de mensajes muertos con su cola hace que aterrice allí.
Errores típicos: escribir `max-attempts` (en Boot 4 está marcada como error); no borrar la cola vieja; que la clave de enrutado de la cola de origen no sea la del enlace de la cola de mensajes muertos (el mensaje muerto se descarta al no encontrar cola); olvidar el `content_type`; creer que los reintentos se hacen en RabbitMQ (son del cliente, dentro del servicio); y reponer a mano y sin mirar los mensajes de la cola de mensajes muertos: vuelven al bucle.

Solución del ejercicio 11.5 enunciado en la pág. 419

Estado de partida. Tienes escritos, con paquetes `com.guildboard.*`: el servicio `rabbitmq` en `compose.yaml`; en Quests, el puerto `QuestEventPublisher`, `CompleteQuestService`, el *record* `QuestCompleted`, `RabbitQuestEventPublisher` y `MessagingConfig` (secciones 11.4 y 11.5) y el starter en los dos `build.gradle` de Quests (sección 11.3); en Treasury, el starter en sus dos `build.gradle`, el `application.yml` con conexión, nombres, reintentos y cola de mensajes muertos, el *record*, el *listener* final (sección 11.6), la `MessagingConfig` final y la prueba (ejercicios 11.4 y 11.3); y en Adventurers, el *record* y el *listener* del ejercicio 11.2, con el starter en `adventurers-adapters`. Esos archivos *no cambian*. Lo que sigue es lo que falta, completo.
1. Quests: el bean del publicador. `BeanConfig` gana el bean `QuestEventPublisher` (que lee el *exchange* y la clave) y `CompleteQuest` se construye con tres colaboradores. En dos trozos seguidos:

config/BeanConfig.java (Quests, 1 de 2)

```java
package com.guildboard.quests.config;

import com.guildboard.quests.adapter.out.messaging.
    ↪ RabbitQuestEventPublisher;
import com.guildboard.quests.adapter.out.notification.LogNotificationSender;
    ↪
import com.guildboard.quests.adapter.out.rest.RestAdventurerDirectory;
import com.guildboard.quests.application.port.in.AcceptQuestUseCase;
import com.guildboard.quests.application.port.in.CompleteQuestUseCase;
import com.guildboard.quests.application.port.in.GetQuestUseCase;
import com.guildboard.quests.application.port.in.PublishQuestUseCase;
import com.guildboard.quests.application.port.out.AdventurerDirectory;
import com.guildboard.quests.application.port.out.NotificationSender;
import com.guildboard.quests.application.port.out.QuestEventPublisher;
import com.guildboard.quests.application.port.out.QuestRepository;
import com.guildboard.quests.application.service.AcceptQuestService;
import com.guildboard.quests.application.service.CompleteQuestService;
import com.guildboard.quests.application.service.GetQuestService;
import com.guildboard.quests.application.service.PublishQuestService;
import java.time.Clock;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public Clock clock() {
        return Clock.systemUTC();
    }

    @Bean
    public NotificationSender notificationSender() {
        return new LogNotificationSender();
    }
```

config/BeanConfig.java (Quests, 2 de 2: sigue dentro de la clase)

```
    @Bean
    public AdventurerDirectory adventurerDirectory(
            @Value("${guildboard.adventurers.base-url}") String baseUrl) {
        return new RestAdventurerDirectory(baseUrl);
    }

    @Bean
    public QuestEventPublisher questEventPublisher(
            RabbitTemplate rabbitTemplate,
            @Value("${guildboard.messaging.exchange}") String exchange,
            @Value("${guildboard.messaging.routing-key}") String key) {
        return new RabbitQuestEventPublisher(
            rabbitTemplate, exchange, key);
    }

    @Bean
    public PublishQuestUseCase publishQuestUseCase(
            QuestRepository questRepository, Clock clock) {
        return new PublishQuestService(questRepository, clock);
    }

    @Bean
    public GetQuestUseCase getQuestUseCase(
            QuestRepository questRepository) {
        return new GetQuestService(questRepository);
    }

    @Bean
    public AcceptQuestUseCase acceptQuestUseCase(
            QuestRepository questRepository,
            AdventurerDirectory adventurerDirectory, Clock clock) {
        return new AcceptQuestService(questRepository,
                                      adventurerDirectory, clock);
    }

    @Bean
    public CompleteQuestUseCase completeQuestUseCase(
            QuestRepository questRepository,
            QuestEventPublisher questEventPublisher,
            NotificationSender notificationSender) {
        return new CompleteQuestService(questRepository,
            questEventPublisher, notificationSender);
    }
}
```

2. Quests: configuración.

quests-bootstrap: application.yml

```yaml
spring:
  application:
    name: quests
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration
  rabbitmq:
    host: localhost
    port: 5672
    username: guildboard
    password: guildboard

server:
  port: 8081

guildboard:
  adventurers:
    base-url: http://localhost:8082
  messaging:
    exchange: guildboard.events
    routing-key: quest.completed
```

3. Treasury: quitar el andamio. Borra los dos DTO y reescribe el controlador y el mapeador, que pierden todo lo del pago:

```
$ cd services/treasury/adapters/src/main/java/com/guildboard/treasury
$ git rm adapter/in/web/dto/PayRewardRequest.java \
        adapter/in/web/dto/PaymentResponse.java
$ cd -
```

adapter/in/web/mapper/TreasuryWebMapper.java (Treasury)

```java
package com.guildboard.treasury.adapter.in.web.mapper;

import com.guildboard.treasury.adapter.in.web.dto.AccountResponse;
import com.guildboard.treasury.domain.Account;

public final class TreasuryWebMapper {

    private TreasuryWebMapper() {
    }

    public static AccountResponse toResponse(Account account) {
        return new AccountResponse(
            account.getOwner().value(), account.getBalance().coins());
    }
}
```

adapter/in/web/TreasuryController.java (Treasury)

```java
package com.guildboard.treasury.adapter.in.web;

import com.guildboard.treasury.adapter.in.web.dto.AccountResponse;
import com.guildboard.treasury.adapter.in.web.mapper.TreasuryWebMapper;
import com.guildboard.treasury.application.port.in.GetAccountUseCase;
import com.guildboard.treasury.domain.AdventurerId;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequiredArgsConstructor
public class TreasuryController {

    private final GetAccountUseCase getAccount;

    @GetMapping("/accounts/{adventurerId}")
    public ResponseEntity<AccountResponse> account(
            @PathVariable String adventurerId) {
        return getAccount.get(new AdventurerId(adventurerId))
            .map(TreasuryWebMapper::toResponse)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }
}
```

`DomainExceptionHandler`, `LedgerBalanceController`, `AccountResponse` y los servicios no cambian (el manejador conserva el `409`; ahora ningún endpoint llega a lanzarlo). **Treasury: Gradle y configuración** (ya los tienes; se muestran completos para que compares y no cambias nada).

services/treasury/adapters/build.gradle

```
dependencies {
    implementation project(':treasury-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

services/treasury/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencies {
    implementation project(':treasury-adapters')
    implementation project(':treasury-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

treasury-bootstrap: application.yml

```yaml
spring:
  application:
    name: treasury
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration
  rabbitmq:
    host: localhost
    port: 5672
    username: guildboard
    password: guildboard
    listener:
      simple:
        retry:
          enabled: true
          max-retries: 3
          initial-interval: 1s
          multiplier: 2
          max-interval: 10s

server:
  port: 8083

guildboard:
  messaging:
    exchange: guildboard.events
    routing-key: quest.completed
    queue: treasury.quest-completed
    dead-letter-exchange: guildboard.events.dlx
    dead-letter-queue: treasury.quest-completed.dlq
```

4. Adventurers: quitar el andamio.

```
$ git rm services/adventurers/adapters/src/main/java/com/guildboard/\
adventurers/adapter/in/web/dto/GrantExperienceRequest.java
```

El controlador pierde el caso de uso de experiencia y el `POST`. En dos trozos:

adapter/in/web/AdventurerController.java (1/2)

```java
package com.guildboard.adventurers.adapter.in.web;

import com.guildboard.adventurers.adapter.in.web.dto.AdventurerResponse;
import com.guildboard.adventurers.adapter.in.web.dto.
    ↪ RegisterAdventurerRequest;
import com.guildboard.adventurers.adapter.in.web.mapper.AdventurerWebMapper;
    ↪
import com.guildboard.adventurers.application.port.in.GetAdventurerUseCase;
import com.guildboard.adventurers.application.port.in.
    ↪ RegisterAdventurerUseCase;
import com.guildboard.adventurers.domain.Adventurer;
import com.guildboard.adventurers.domain.AdventurerId;
import jakarta.validation.Valid;
import java.net.URI;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/adventurers")
@RequiredArgsConstructor
public class AdventurerController {

    private final RegisterAdventurerUseCase registerAdventurer;
    private final GetAdventurerUseCase getAdventurer;

    @PostMapping
    public ResponseEntity<AdventurerResponse> register(
            @Valid @RequestBody RegisterAdventurerRequest request) {
        Adventurer adventurer = registerAdventurer.register(
            AdventurerWebMapper.toCommand(request));
        return ResponseEntity
            .created(URI.create(
                "/adventurers/" + adventurer.getId().value()))
            .body(AdventurerWebMapper.toResponse(adventurer));
    }
```

adapter/in/web/AdventurerController.java (2/2: sigue dentro de la clase)

```

    @GetMapping("/{id}")
    public ResponseEntity<AdventurerResponse> get(
            @PathVariable String id) {
        return getAdventurer.get(new AdventurerId(id))
            .map(AdventurerWebMapper::toResponse)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }
}
```

Adventurers: Gradle, mensajería y configuración.

services/adventurers/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencies {
    implementation project(':adventurers-adapters')
    implementation project(':adventurers-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

config/MessagingConfig.java (Adventurers)

```java
package com.guildboard.adventurers.config;

import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Declarables;
import org.springframework.amqp.core.DirectExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.amqp.support.converter.
    ↪ JacksonJsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class MessagingConfig {

    @Bean
    public MessageConverter messageConverter() {
        return new JacksonJsonMessageConverter();
    }

    @Bean
    public Declarables questCompletedDeclarables(
            @Value("${guildboard.messaging.exchange}") String exchange,
            @Value("${guildboard.messaging.routing-key}") String key,
            @Value("${guildboard.messaging.queue}") String queue,
            @Value("${guildboard.messaging.dead-letter-exchange}")
            String deadExchange,
            @Value("${guildboard.messaging.dead-letter-queue}")
            String deadQueue) {
        TopicExchange events = new TopicExchange(exchange);
        DirectExchange dead = new DirectExchange(deadExchange);
        Queue questCompleted = QueueBuilder.durable(queue)
            .deadLetterExchange(deadExchange)
            .deadLetterRoutingKey(deadQueue)
            .build();
        Queue deadLetters = QueueBuilder.durable(deadQueue).build();
        return new Declarables(events, dead, questCompleted, deadLetters,
            BindingBuilder.bind(questCompleted).to(events).with(key),
            BindingBuilder.bind(deadLetters).to(dead).with(deadQueue));
    }
}
```

adventurers-bootstrap: application.yml

```yaml
spring:
  application:
    name: adventurers
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration
  rabbitmq:
    host: localhost
    port: 5672
    username: guildboard
    password: guildboard
    listener:
      simple:
        retry:
          enabled: true
          max-retries: 3
          initial-interval: 1s
          multiplier: 2
          max-interval: 10s

server:
  port: 8082

guildboard:
  messaging:
    exchange: guildboard.events
    routing-key: quest.completed
    queue: adventurers.quest-completed
    dead-letter-exchange: guildboard.events.dlx
    dead-letter-queue: adventurers.quest-completed.dlq
```

5. Comprobar.

```
$ ./gradlew build
BUILD SUCCESSFUL
$ ./gradlew test
$ git add -A
$ git commit -m "H6a: QuestCompleted por RabbitMQ"
```

Después, el escenario de la sección 11.10.1, en modo memoria y con `SPRING_PROFILES_ACTIVE=postgres`.

Por qué funciona. Los tres servicios declaran el *exchange* `guildboard.events` con las mismas propiedades (declarar dos veces lo mismo no hace nada), y cada consumidor es dueño de su cola y de su cola de mensajes muertos. Los dos consumidores usan el mismo *exchange* de mensajes muertos (`guildboard.events.dlx`), con la clave igual al nombre de

cada cola de mensajes muertos, así que cada una recibe solo lo suyo. Quests solo conoce el puerto; el núcleo de los tres servicios no cambió salvo `CompleteQuestService`, y el único `import` de `org.springframework.amqp` está en `adapters` y `bootstrap`.

Criterios para valorar otras soluciones. Es válida si el evento sale después de guardar, cada servicio tiene su *record*, los nombres vienen de la configuración, los `POST` provisionales no existen y `./gradlew test` (con los `ArchitectureTest`) pasa. Una alternativa válida declara también la cola de cada consumidor desde Quests: así el mensaje no se pierde por no haber arrancado nadie, pero Quests pasa a conocer la existencia de sus consumidores, justo lo que el evento quería evitar.

Errores típicos: arrancar Quests y publicar antes de que exista alguna cola (el evento se descarta; arranca primero los consumidores); olvidar el starter en `bootstrap` (`MessagingConfig` no compila); olvidar el starter en `adapters` (no compila el *listener*); poner en `CompleteQuestService` el anuncio antes de `quests.save`; escribir `Jackson2JsonMessageConverter` (el obsoleto) o `max-attempts`; no borrar la cola antigua al añadir la cola de mensajes muertos; olvidar `DirectExchange` para el *exchange* de mensajes muertos (con `TopicExchange` el enlace usaría comodines); e intentar dar solución al evento perdido con un `try/catch` que traga la excepción de `convertAndSend`: la misión quedaría completada y nadie se enteraría.

CAPÍTULO 12

Resiliencia básica: timeouts, reintentos y circuit breaker

Objetivos del capítulo

- Explicar qué puede fallar en una llamada de red entre servicios y por qué una llamada sin tiempo máximo convierte la caída de un servicio en la caída de otro.
- Fijar tiempos máximos, un reintento con espera creciente (solo en operaciones seguras) y un *circuit breaker*, y razonar los valores elegidos.
- Instalar Resilience4j en Quests, aplicarlo solo en el adaptador de salida y distinguir sus dos formas de uso: anotaciones y API programática.
- Decidir, como decisión de negocio, qué hace `AcceptQuest` cuando Adventurers no responde, y cerrar el hito H6.

Al terminar el capítulo 9, Quests dejó de tener a los aventureros en casa: para aceptar una misión pregunta su rango a Adventurers por REST. Esa pregunta es una llamada de red, y las llamadas de red fallan. Hoy, si Adventurers está parado, aceptar una misión da un `500` con un error de conexión en el registro; si está vivo pero lento, la petición de Mara se queda esperando y, mientras espera, ocupa un hilo de Quests. Este capítulo protege esa única llamada con tres defensas (tiempo máximo, reintento y *circuit breaker*) y obliga a contestar la pregunta que ninguna librería contesta por ti: cuando no hay respuesta, ¿qué hace el negocio?

Tres cosas que el capítulo deja dichas desde el principio:

- **Hay una dependencia nueva.** Resilience4j 2.4.0 se instala en los dos `build.gradle` que lo necesitan. La sección 12.6 explica antes de usarla qué artefacto es, qué se verificó y qué no.
- **No cambia la estructura.** Ni módulos ni paquetes nuevos de adaptadores: solo archivos dentro de lo que ya existe. Por eso este capítulo no tiene verificación de estructura propia (sección 12.2).
- **Una parte no se ha podido ejecutar.** Como en todo el curso, el código Java no se compila. Además, no he podido comprobar que las *anotaciones* de Resilience4j intercepten las llamadas en Boot 4.1.1. Por eso el proyecto usa la API programática, y el libro lo dice donde corresponde.

12.1 Qué falla en una llamada de red

Una llamada dentro de un mismo proceso o devuelve un resultado o lanza una excepción, y lo hace enseguida. Una llamada de red tiene más finales posibles, y cada uno pide una respuesta distinta. Estos son los que puede ver `RestAdventurerDirectory`:

Qué ocurre	Qué ve Quests	¿Reintentar?
Nadie escucha (Adventurers está parado)	Conexión rechazada, casi al instante. Spring la envuelve en `ResourceAccessException`.	Sí: puede volver en un momento.
Alguien escucha, pero no contesta (proceso colgado, red perdida)	Sin tiempo máximo, nada: la petición espera. Con tiempo máximo, un `ResourceAccessException` cuando vence.	Con cuidado: cada intento cuesta un tiempo máximo entero.
Contesta con un error de servidor (5xx)	`HttpServerErrorException`.	Sí, si la operación es segura (sección 12.4).
Contesta con un error del cliente (4xx)	`HttpClientErrorException`. El 404 de «ese aventurero no existe» es una respuesta de negocio, no un fallo.	No: la petición está mal o la respuesta es definitiva.

Lo peligroso no es el fallo rápido, sino el lento. Cuando Adventurers deja de contestar y Quests espera sin límite, cada petición de aceptar retiene un hilo. Un servidor web como el que trae Boot tiene un número finito de hilos de petición (por omisión, unos 200). Cuando se acaban, Quests no puede atender ni siquiera `GET /quests/{id}`, que no necesita a Adventurers para nada. Es el fallo en cascada que describe Nygard: un problema en un sistema se propaga al que depende de él [33]. Los patrones que siguen son su remedio, y son los que también recoge el catálogo de patrones de microservicios [42].

Lo que Quests tiene hoy

`RestAdventurerDirectory` (capítulo 9) construye su cliente con `RestClient.builder().baseUrl(baseUrl).build()`: no fija ningún tiempo máximo (el valor por omisión depende de la fábrica de peticiones que Spring elija, y no es una decisión tuya), no reintenta nada y no se acuerda de que Adventurers falló hace un segundo. Cualquier fallo que no sea el 404 sube como excepción hasta el `500`, como dejó anotado el capítulo.

12.2 Este capítulo no cambia la estructura

Los capítulos con verificación de estructura (3, 6, 7, 8, 9, 10 y 11) cambiaron el árbol de carpetas. Este no lo hace, y conviene comprobarlo en lugar de suponerlo. Esto es lo que entra, sobre el árbol T6 del capítulo 11 (que sigue vigente): `[N]` es nuevo y `[M]` modificado.

Qué toca el capítulo 12 (árbol T6 sin cambios de carpetas)

```
services/quests/
|-- application/  src/main/java/.../quests/application/
|                   port/out/AdventurerUnavailableException.java [N]
|-- adapters/     build.gradle [M] + resilience4j y dependencias de prueba
|                 src/main/java/.../quests/adapter/
|                   in/web/error/
|                     AdventurerUnavailableExceptionHandler.java [N]
|                   out/rest/AdventurerClientFactory.java [N]
|                   out/rest/RestAdventurerDirectory.java [M]
|                 src/test/java/.../quests/adapter/out/rest/
|                   RestAdventurerDirectoryBreakerTest.java [N]
`-- bootstrap/    build.gradle [M] + resilience4j
                  src/main/java/.../quests/config/BeanConfig.java [M]
                  src/main/resources/application.yml [M]
```

La comprobación, con las preguntas de siempre de las verificaciones:

- *¿Módulo Gradle nuevo?* No. Siguen siendo los doce proyectos.
- *¿Paquete nuevo?* No: `port.out`, `adapter.in.web.error` y `adapter.out.rest` ya existían. La única carpeta nueva es `src/test` de `quests-adapters`, que es la convención de Gradle para las pruebas (los módulos `domain` y, desde el capítulo 11, `treasury-application` ya tienen la suya), no una decisión de estructura.
- *¿Un `build.gradle` nombra a otro servicio?* No.
- *¿Se cumple alguna de las condiciones para partir `adapters`?* No: Resilience4j es una librería que usa el adaptador `rest` que ya existe, no un adaptador de una tecnología nueva.

Compruébalo tú al final del capítulo con `./gradlew test`, que ejecuta las pruebas de arquitectura (sección 12.9). Si en algún momento tuvieras que crear un módulo o un paquete de adaptadores para que algo de este capítulo funcione, ese cambio de estructura tendría que figurar en el árbol de una verificación, y este capítulo no la tiene.

12.3 Timeouts: siempre

Un *timeout* es el tiempo máximo que aceptas esperar. Hay dos y no son lo mismo:

- El de **conexión**: cuánto esperas a que el otro lado acepte la conexión. Si el servicio está parado en otra máquina, sin respuesta de red, sin este límite esperarías al valor del sistema operativo.
- El de **lectura**: cuánto esperas, ya conectado, a que llegue la respuesta. Cubre el caso «escucha, pero no contesta».

Regla del capítulo: **toda llamada de red lleva los dos**, aunque no uses nada más de lo que sigue. Es la defensa más barata (un par de líneas) y la que más daño evita. Los valores salen de un presupuesto, no de la intuición: Adventurers contesta en milisegundos, de modo que un segundo de lectura es muy generoso; y todo lo que esperes aquí se suma a lo que espera quien llamó a Quests. Para GuildBoard: 1 segundo de conexión y 1 de lectura.

En Spring, el tiempo máximo no está en el `RestClient`, sino en la fábrica de peticiones que éste usa [61]. Con la del JDK (`JdkClientHttpRequestFactory`, de `spring-web`, que ya tienes) se configuran así: el de conexión en el `HttpClient` de Java, el de lectura en la fábrica. Comprobé en `spring-web` 7.0.9 (el que gestiona Boot 4.1.1) que existen `JdkClientHttpRequestFactory(HttpClient)`, `setReadTimeout(Duration)` y `RestClient.Builder.requestFactory(...)`:

Dos tiempos máximos en un RestClient (esquema)

```
HttpClient http = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(1))
    .build();
JdkClientHttpRequestFactory factory =
    new JdkClientHttpRequestFactory(http);
factory.setReadTimeout(Duration.ofSeconds(1));
RestClient client = RestClient.builder()
    .baseUrl("http://localhost:8082")
    .requestFactory(factory)
```

```
10      .build();
```

Cuando uno de los dos vence, la llamada falla con `ResourceAccessException`, la misma excepción que una conexión rechazada. El ejercicio 12.1 lo lleva al proyecto y te hace ver las dos situaciones.

Un *timeout* no arregla nada por sí solo: convierte una espera infinita en un fallo rápido. Lo que hagas con ese fallo es el resto del capítulo.

12.4 Reintentos: solo en operaciones seguras

Muchos fallos de red duran un instante: un reinicio, un paquete perdido. Reintentar la llamada una o dos veces los absorbe sin que nadie lo note. Pero reintentar tiene tres reglas, y saltarse una causa más daño que no reintentar:

1. **Solo si repetir la operación es seguro.** Un `GET` que lee un rango lo es: leerlo dos veces no cambia nada. Un `POST` que crea algo no: si la primera llamada sí llegó y solo se perdió la respuesta, el reintento lo crea dos veces. Treasury aguanta esa repetición porque es idempotente por `questId` (capítulo 10), pero esa es una propiedad del *servidor*; el cliente no puede suponerla. La única llamada de Quests a otro servicio es un `GET`, y por eso es la única que reintentamos.
2. **Solo los fallos transitorios.** Conexión rechazada, tiempo agotado y 5xx pueden mejorar. Un 400 o un 404 no mejoran por repetirse.
3. **Pocos intentos y con espera creciente.** Si Adventurers está agobiado, cien clientes reintentando de golpe lo hunden del todo. Se espera un poco antes del segundo intento, más antes del tercero (*backoff* exponencial) y se para pronto.

Con 3 intentos, una espera inicial de 200 ms y un multiplicador de 2, las esperas son 200 ms y 400 ms: 0,6 s de espera en total. Hay que sumarlas al presupuesto: con un servidor que acepta y no contesta, el peor caso de una consulta es $3 \times 1\,\text{s}$ de lectura más 0,6 s de esperas, 3,6 s, y con conexión rechazada, solo los 0,6 s. En Resilience4j, el reintento se describe con `RetryConfig`; por omisión son 3 intentos y 500 ms de espera (comprobado en la versión 2.4.0):

Un reintento con espera creciente (esquema)

```
RetryConfig config = RetryConfig.custom()
    .maxAttempts(3)
    .intervalFunction(IntervalFunction.ofExponentialBackoff(
        Duration.ofMillis(200), 2.0))
    .retryExceptions(ResourceAccessException.class,
                     HttpServerErrorException.class)
    .build();
Retry retry = Retry.of("adventurers", config);
```

La lista de excepciones no es opcional

Sin `retryExceptions`, Resilience4j reintenta ante *cualquier* excepción, y eso incluye la que lanza el *circuit breaker* abierto al rechazar una llamada. Reintentar contra un interruptor abierto es absurdo: gasta las esperas para nada. Lista siempre las dos únicas excepciones transitorias.

12.5 Circuit breaker: cerrado, abierto y semiabierto

Con timeouts y reintentos, un Adventurers caído aún le cuesta a cada petición de Quests seis décimas de segundo o más, y le sigue llegando tráfico que no puede atender. Un *circuit breaker* (cortacircuitos) añade memoria: si las últimas llamadas han fallado, deja de llamar durante un rato y falla al instante. Así Quests se libera de esperar y Adventurers tiene tiempo de recuperarse. Es el patrón que Nygard llama *circuit breaker* y que Fowler resume como un objeto que envuelve la llamada, cuenta los fallos y, pasado un umbral, corta la conexión [33, 11].

Tiene tres estados normales [39]:

Estado	Qué hace con las llamadas	Cuándo cambia
`CLOSED` (cerrado)	Las deja pasar todas y anota en una ventana si salieron bien o mal.	Pasa a `OPEN` cuando, con un mínimo de llamadas anotadas, el porcentaje de fallos llega al umbral.
`OPEN` (abierto)	No llama: rechaza al instante con `CallNotPermittedException`.	Pasado el tiempo de espera, pasa a `HALF_OPEN`.
`HALF_OPEN` (semiabierto)	Deja pasar un número pequeño de llamadas de prueba; rechaza el resto.	Si el porcentaje de fallos de las pruebas llega al umbral, vuelve a `OPEN`; si no, pasa a `CLOSED`.

Los valores por omisión de Resilience4j (ventana de 100 llamadas, mínimo de 100, umbral del 50 %, 60 s de espera y 10 llamadas de prueba; comprobados en la clase `CircuitBreakerConfig` de la 2.4.0) están pensados para un servicio con mucho tráfico. GuildBoard, en una prueba con `curl`, no hace cien llamadas: con ellos el interruptor no se abriría nunca mientras lo pruebas. Estos son los valores del proyecto:

- `sliding-window-size: 10`: la ventana recuerda las 10 últimas llamadas.
- `minimum-number-of-calls: 5`: no se calcula el porcentaje hasta tener 5.
- `failure-rate-threshold: 50`: se abre si fallan la mitad o más.
- `wait-duration-in-open-state: 10s`: abierto, rechaza durante 10 segundos.
- `permitted-number-of-calls-in-half-open-state: 2`: dos llamadas de prueba.

Falta decir *qué cuenta como fallo*. La documentación es explícita: si indicas una lista de excepciones (`recordExceptions`), **todas las demás cuentan como éxito** [39]. Eso es lo que queremos: `ResourceAccessException` y `HttpServerErrorException` son fallos de Adventurers; el 404 ni siquiera llega como excepción (el adaptador lo convierte en `Optional.empty()`); y una excepción del mapeador del capítulo 9 (`IllegalStateException`, un contrato roto) no debe abrir el interruptor porque Adventurers esté «caído»: contestó, aunque contestara algo raro.

Con esos valores, la secuencia de un Adventurers parado es: las cinco primeras llamadas fallan por conexión (con el reintento, cada *intento* cuenta como una llamada para el interruptor); a la quinta se abre; durante 10 s toda llamada falla al instante; pasados los 10 s, la siguiente llamada pasa a `HALF_OPEN` y las dos de prueba deciden: si Adventurers sigue caído, se abre otra vez; si volvió, se cierra. Para observarlo en un programa, el interruptor ofrece `getState()` y `getMetrics()`; las pruebas del ejercicio 12.3 se apoyan en eso.

12.6 Resilience4j en el adaptador de salida, nunca en el dominio

Resilience4j es una librería de resiliencia para Java con un módulo por patrón (`CircuitBreaker`, `Retry`, y también *bulkhead*, limitador de frecuencia y límite de tiempo, que este curso no usa).

Dónde se usa es la decisión de arquitectura del capítulo: **solo en el adaptador de salida**. `RestAdventurerDirectory` sabe que habla por HTTP, que HTTP falla y que hay un interruptor. El puerto `AdventurerDirectory` no sabe nada de eso, y `AcceptQuestService` tampoco: solo sabe que, a veces, el puerto le dice «no puedo responder» con una excepción en su propio idioma, `AdventurerUnavailableException`. Ni `domain` ni `application` nombran Resilience4j: sus `build.gradle` no lo tienen y, aunque alguien escribiera el *import*, no compilarían.

☞ Resilience4j 2.4.0 en Quests

Qué se instala, qué se verificó el 9 de octubre de 2026 y qué no:

- **El artefacto.** Para Boot 4 es `io.github.resilience4j:resilience4j-spring-boot4`. En Maven Central su última versión es la 2.4.0 (`latest` y `release` del `maven-metadata.xml`), la misma que anuncia la versión de GitHub [40]. Arrastra `resilience4j-spring6` y, con él, los módulos de circuit breaker y reintento. El BOM de Boot 4.1.1 *no* gestiona Resilience4j (su `pom` no lo menciona): por eso la versión se escribe en la dependencia.
- **Compatibilidad con Boot 4.1.1.** El artefacto trae una comprobación (`SpringBoot4Verifier`) que solo rechaza versiones de Boot anteriores a la 4; su `pom` está construido contra Boot 4.0.0 y Spring 7.0.2, y el resto de versiones las pone Gradle según el BOM. No he arrancado Quests con él: es un punto a vigilar si algo falla al iniciar.
- **AOP.** Las anotaciones necesitan AspectJ. La autoconfiguración solo crea los *aspectos* si `org.aspectj.lang.ProceedingJoinPoint` está en el *classpath* (lo hace la clase `AspectJOnClasspathCondition` de `resilience4j-spring6` 2.4.0). Para Boot 4.1.1 la dependencia es `org.springframework.boot:spring-boot-starter-aspectj`, que existe en Maven Central y trae `spring-aop` y `aspectjweaver`; es la dependencia estándar para usar AOP con Boot. **El proyecto no la añade**: usa la API programática, que no necesita AOP.
- **Alternativa de Initializr.** Initializr, con Boot 4.1.1, ofrece la dependencia `cloud-resilience4j` [62], que genera `org.springframework.cloud:spring-cloud-starter-circuitbreaker-resilience4j` y añade el BOM `spring-cloud-dependencies` 2025.1.3. No la usamos: el BOM de Spring Cloud no entra en el curso hasta el capítulo 13, y ese starter expone la abstracción de Spring Cloud CircuitBreaker, no los objetos `CircuitBreaker` y `Retry` que se explican aquí. Es una opción razonable si tu proyecto ya usa Spring Cloud.

Añade la misma línea a `dependencies` de `services/quests/adapters/build.gradle` (el adaptador usa `CircuitBreaker` y `Retry`) y de `services/quests/bootstrap/build.gradle` (la composición pide los registros que crea la autoconfiguración):

Línea nueva en los dos build.gradle de Quests

```
    implementation 'io.github.resilience4j:resilience4j-spring-boot4:2.4.0'
```

No se ha comprobado por ejecución: que Quests arranque con la autoconfiguración de Resilience4j y que Gradle resuelva el *classpath* sin conflictos de versión.

12.6.1 La configuración: un mismo YAML para las dos formas

La autoconfiguración lee el bloque `resilience4j` del `application.yml` y crea dos beans, `CircuitBreakerRegistry` y `RetryRegistry`, con una instancia por cada nombre declarado (aquí, `adventurers`). Con las anotaciones, los aspectos las buscan por nombre; con la API programática, la composición las pide por nombre. El bloque completo para Quests:

quests-bootstrap: application.yml (bloques nuevos)

```yaml
guildboard:
  adventurers:
    base-url: http://localhost:8082
    connect-timeout: 1s
    read-timeout: 1s

resilience4j:
  circuitbreaker:
    instances:
      adventurers:
        sliding-window-size: 10
        minimum-number-of-calls: 5
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 2
        record-exceptions:
          - org.springframework.web.client.ResourceAccessException
          - org.springframework.web.client.HttpServerErrorException
  retry:
    instances:
      adventurers:
        max-attempts: 3
        wait-duration: 200ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        retry-exceptions:
          - org.springframework.web.client.ResourceAccessException
          - org.springframework.web.client.HttpServerErrorException
```

Los nombres de propiedad (por ejemplo `enable-exponential-backoff` o `record-exceptions`) los comprobé en las clases de propiedades de `resilience4j-framework-common` 2.4.0 y en los metadatos de `resilience4j-spring-boot4`, que declaran el prefijo `resilience4j.circuitbreaker`. Dos trampas de este archivo:

- *El nombre de la instancia tiene que coincidir.* Si la composición pide `breakers.circuitBreaker("adventurer")` (sin la «s»), no falla: te da un interruptor con la configuración por omisión, es decir, ventana de 100 llamadas, y jamás se abre en tus pruebas.
- *Las excepciones van con el nombre completo de la clase*, con su paquete. Una lista distinta de la que esperas cambia qué cuenta como fallo, y eso se nota en el interruptor, no en un error de arranque.

12.6.2 Con anotaciones

La forma más corta. Sobre el método del adaptador se escribe qué protección quieres y la autoconfiguración hace el resto:

Versión con anotaciones (no es la del proyecto)

```
@Retry(name = "adventurers", fallbackMethod = "unavailable")
@CircuitBreaker(name = "adventurers")
@Override
public Optional<AdventurerView> findById(AdventurerId id) {
    return fetch(id);   // el cuerpo de siempre, con el 404 a vac^^c3^^ado
}

private Optional<AdventurerView> unavailable(
        AdventurerId id, Throwable cause) {
    throw new AdventurerUnavailableException(
        "adventurers did not answer", cause);
}
```

`@Retry` es de `io.github.resilience4j.retry.annotation` y `@CircuitBreaker` de `io.github.resilience4j.circuitbreaker.annotation`; las dos están en el artefacto de anotaciones que arrastra el de Boot. Lo que hay que saber de ellas, según la guía de arranque de Resilience4j (escrita para Boot 2 y 3) y lo que comprobé en la 2.4.0:

- Funcionan con *proxies* de Spring: solo intercepta una llamada que entra *desde fuera* del bean. Si anotas `fetch` y lo llama `findById`, no pasa nada, y no avisa.
- Necesitan AspectJ en el *classpath* (más arriba): sin él, los aspectos no se crean y las anotaciones no hacen nada, también sin avisar.
- El orden por omisión es `Retry(CircuitBreaker(llamada))`: el reintento envuelve al interruptor. Es el que queremos, y es el orden que dice la guía; en la 2.4.0 la propiedad de orden del `Retry` tiene prioridad sobre la del interruptor.
- El `fallbackMethod` funciona como un `catch`: mismos parámetros más una excepción al final. Aquí se ha puesto en el `@Retry`, el más externo, para que traduzca la excepción *después* de todos los intentos. Si lo pusieras en `@CircuitBreaker`, el reintento recibiría ya la excepción traducida, que no está en su lista, y no reintentaría.

No he podido ejecutar nada de esto. Verifiqué el artefacto, la condición de AspectJ, el orden y los nombres de las anotaciones; no que Quests arranque con AspectJ y que los *proxies* intercepten. Si lo pruebas y funciona, es una alternativa válida; por eso el libro no las descarta, solo no se apoya en ellas.

12.6.3 Con la API programática

La alternativa no necesita proxies ni AOP: el adaptador recibe el interruptor y el reintento y los aplica a mano. Es la forma que usa el proyecto. Primero, el estado de la clase y sus dos constructores:

RestAdventurerDirectory.java (1 de 2)

```
package com.guildboard.quests.adapter.out.rest;

```

```
import com.guildboard.quests.application.port.out.AdventurerDirectory;
import com.guildboard.quests.application.port.out.
    ↪ AdventurerUnavailableException;
import com.guildboard.quests.application.port.out.AdventurerView;
import com.guildboard.quests.domain.AdventurerId;
import io.github.resilience4j.circuitbreaker.CallNotPermittedException;
import io.github.resilience4j.circuitbreaker.CircuitBreaker;
import io.github.resilience4j.retry.Retry;
import java.time.Duration;
import java.util.Optional;
import java.util.function.Supplier;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.HttpServerErrorException;
import org.springframework.web.client.ResourceAccessException;
import org.springframework.web.client.RestClient;

public class RestAdventurerDirectory implements AdventurerDirectory {

    private final RestClient client;
    private final CircuitBreaker breaker;
    private final Retry retry;

    public RestAdventurerDirectory(String baseUrl,
            Duration connectTimeout, Duration readTimeout,
            CircuitBreaker breaker, Retry retry) {
        this(AdventurerClientFactory.create(
                baseUrl, connectTimeout, readTimeout),
            breaker, retry);
    }

    public RestAdventurerDirectory(RestClient client,
            CircuitBreaker breaker, Retry retry) {
        this.client = client;
        this.breaker = breaker;
        this.retry = retry;
    }
```

Después, los métodos. `findById` compone las dos protecciones y traduce; `fetch` es el cuerpo de siempre:

RestAdventurerDirectory.java (2 de 2: sigue dentro de la clase)

```
    @Override
    public Optional<AdventurerView> findById(AdventurerId id) {
        Supplier<Optional<AdventurerView>> call = () -> fetch(id);
        Supplier<Optional<AdventurerView>> protectedCall =
            Retry.decorateSupplier(retry,
                CircuitBreaker.decorateSupplier(breaker, call));
        try {
            return protectedCall.get();
        } catch (CallNotPermittedException e) {
            throw new AdventurerUnavailableException(
                "adventurers circuit breaker is open", e);
        } catch (ResourceAccessException
                 | HttpServerErrorException e) {
            throw new AdventurerUnavailableException(
```

```
                "adventurers did not answer", e);
        }
    }

    private Optional<AdventurerView> fetch(AdventurerId id) {
        try {
            AdventurerPayload payload = client.get()
                .uri("/adventurers/{id}", id.value())
                .retrieve()
                .body(AdventurerPayload.class);
            return Optional.ofNullable(payload)
                .map(AdventurerPayloadMapper::toView);
        } catch (HttpClientErrorException.NotFound e) {
            return Optional.empty();
        }
    }
}
```

Se lee de dentro hacia fuera. `call` es una llamada HTTP. El interruptor la envuelve: anota si salió bien o mal y, con el circuito abierto, la rechaza sin ejecutarla. El reintento envuelve al interruptor (el mismo orden que las anotaciones): repite la llamada protegida si falla con una de las dos excepciones de su lista. Por último, el `try/catch` de `findById` está *fuera de los dos*: es el único sitio donde ocurre la traducción, así que el reintento y el interruptor siguen viendo las excepciones originales. `CallNotPermittedException` (circuito abierto) y las dos excepciones transitorias acaban igual para quien llama: «no puedo responder». Los métodos estáticos `Retry.decorateSupplier` y `CircuitBreaker.decorateSupplier` existen en la 2.4.0 (comprobado con `javap` en los *jar*).

Qué ganas y qué pierdes frente a las anotaciones. Ganas que no hay *proxies* (no hay llamadas internas que no se interceptan), que la traducción y el orden están a la vista en un único método, que no hace falta AOP y que la clase se prueba con un `new` (el ejercicio 12.3). Pierdes brevedad: son unas líneas más por método protegido.

Queda la excepción del puerto. Es una excepción de *aplicación*: está en `port.out`, junto a `AdventurerDirectory`, y no sabe nada de HTTP.

quests-application: port/out/AdventurerUnavailableException.java

```
package com.guildboard.quests.application.port.out;

/**
 * Adventurers no ha podido responder: no hay conexi^^c3^^b3n, ha agotado el
 * tiempo, ha contestado con un error de servidor o el circuit breaker
 * est^^c3^^a1 abierto. No dice nada sobre si el aventurero existe.
 */
public class AdventurerUnavailableException extends RuntimeException {

    public AdventurerUnavailableException(String message,
                                          Throwable cause) {
        super(message, cause);
    }
}
```

Es no comprobada (`RuntimeException`) como `DomainException`: el puerto `AdventurerDirectory` no cambia de firma. Contratos del puerto desde este capítulo:

`Optional.empty()` significa «ese aventurero no existe»; `AdventurerUnavailableException`, «no sé si existe».

La fábrica del cliente (con los dos tiempos máximos) es la del esquema de la sección 12.3 metida en una clase, y es parte del ejercicio 12.1.

12.7 Cuando no hay respuesta: ¿se rechaza o se degrada?

Los tres patrones anteriores no deciden qué hace Quests cuando, aun así, no hay respuesta. Eso lo decide el negocio, y las opciones tienen consecuencias distintas sobre las reglas del dominio:

Opción	Qué hace `AcceptQuest`	Qué pasa con las reglas
Rechazar	Responde 503: «ahora no puedo comprobarlo, inténtalo luego».	Ninguna regla se arriesga: no se acepta nada sin comprobar el rango (R3). Se pierde disponibilidad.
Usar un rango en caché	Si Adventurers no contesta, usa el último rango visto de ese aventurero.	La experiencia solo sube (R6), así que un rango en caché es igual o menor que el real: puede rechazar de más, nunca aceptar de más. Un aventurero nunca visto no se puede resolver.
Aceptar condicionalmente	Acepta ya y comprueba el rango después.	Puede aceptar a quien no cumple R3. Exige deshacerlo luego (una compensación): es una saga, fuera de este curso.

Dos ideas que importan más que la tabla:

- **Degradar es una decisión de negocio, no un detalle técnico.** Un *fallback* escondido dentro del adaptador («si falla, devuelve un rango por omisión») convierte en secreto técnico una regla de producto. Si se degrada, que sea visible en el núcleo o en un decorador con nombre propio.
- **El valor por omisión de GuildBoard es rechazar.** Es lo que hace el proyecto en el hito: la excepción sube y se convierte en 503. Es lo seguro; las otras dos opciones se estudian en el ejercicio 12.5.

12.8 Por qué no `@Retryable` de Spring Framework 7

Spring Framework 7 trae reintentos propios: `@Retryable`, `@ConcurrencyLimit` y `RetryTemplate`, que se activan con `@EnableResilientMethods` [60]. No necesitan ninguna dependencia, y si lo único que quisieras fuera reintentar, serían una opción excelente. Aquí no las usamos por una razón: **no incluyen un *circuit breaker***, que es la tercera defensa que queremos. Usar `@Retryable` para el reintento y otra librería para el interruptor serían dos mecanismos distintos, con configuración distinta, para un único problema. Con Resilience4j aprendes los dos con una sola librería. No hemos comparado a fondo el resto de sus características; la decisión es de alcance, no de calidad.

12.9 En el proyecto: hito H6 (parte 2)

Hito 6, parte 2: Quests protegido frente a Adventurers

Al terminar el ejercicio 12.4, la llamada de Quests a Adventurers tiene:

- **Tiempos máximos** de 1 s de conexión y 1 s de lectura.
- **Un reintento con espera creciente**: 3 intentos, 200 ms y 400 ms de espera, solo ante conexión rechazada, tiempo agotado y 5xx.
- **Un *circuit breaker*** de 10 llamadas con mínimo de 5, umbral del 50 %, 10 s abierto y 2 llamadas de prueba.
- **Una traducción**: todo lo anterior acaba en `AdventurerUnavailableException` (puerto) y el adaptador web la convierte en un `503` con `ProblemDetail`. Rechazar es la política.

H6 queda completo: mensajería asíncrona para el evento (capítulo 11) y REST protegido para la pregunta. No cambian `domain`, `application` (salvo la excepción nueva) ni la estructura.

12.9.1 Probar el hito

Preparación común de los ejercicios, en modo memoria. Quests necesita RabbitMQ (capítulo 11); Adventurers, no es necesario que esté en marcha para la mayoría de las pruebas:

```
$ docker compose up -d --wait rabbitmq
$ ./gradlew :quests-bootstrap:bootRun                # terminal 1
$ QUEST=$(curl -s -X POST localhost:8081/quests \
    -H 'Content-Type: application/json' \
    -d '{"title":"Rats","reward":50,"requiredRank":"NOVICE"}' \
    | grep -o '"id":"[^"]*"' | cut -d'"' -f4)
$ echo $QUEST
```

(Publicar una misión no llama a Adventurers.) Aceptarla sí. Con Adventurers parado, mide el tiempo de la respuesta:

```
$ curl -s -o /dev/null -w '%{http_code} %{time_total}s\n' \
    -X POST localhost:8081/quests/$QUEST/accept \
    -H 'Content-Type: application/json' -d '{"adventurerId":"mara"}'
```

Con el hito completo, la secuencia esperada (no la he ejecutado) es:

1. Primera petición: `503` en unos 0,6 s (tres intentos con conexión rechazada y dos esperas de 200 y 400 ms).
2. Segunda petición: `503` igual de lenta. En su segundo intento el interruptor llega a cinco fallos y se abre; el tercero, tras su espera, ya es rechazado sin salir a la red.
3. Terceras y siguientes, durante 10 s: `503` en milisegundos, sin tocar la red. El registro de Quests lo dice con «circuit breaker is open».
4. Pasados los 10 s con Adventurers aún parado, una petición de prueba falla y el interruptor vuelve a abrirse. Arranca Adventurers (`./gradlew :adventurers-bootstrap:bootRun`), registra un aventurero y espera otros 10 s: las dos peticiones de prueba salen bien y

el interruptor se cierra. Entonces aceptar vuelve a dar `200` o el error de negocio que corresponda.

Y la comprobación de que nada estructural se ha movido:

```
$ ./gradlew test
$ ./gradlew build
$ git status --short
```

12.10 Ejercicios

Los ejercicios parten del proyecto tal como lo dejó el capítulo 11 y se hacen en este orden. Los tres primeros preparan las piezas; el cuarto las junta en el hito; el último pide una decisión.

Ejercicio 12.1: Tiempos máximos en el cliente de Adventurers ★★☆☆☆

Parte de `RestAdventurerDirectory(String baseUrl)` del capítulo 9 y del `BeanConfig` del capítulo 11. Vas a ponerle los dos tiempos máximos y a ver qué pasa con Adventurers parado y con un servidor que no contesta:

1. En `quests-adapters`, paquete `adapter.out.rest`, crea la clase `AdventurerClientFactory` con un método estático público `RestClient create(String baseUrl, Duration connectTimeout, Duration readTimeout)` (sección 12.3).
2. Cambia el constructor de `RestAdventurerDirectory` a `(String baseUrl, Duration connectTimeout, Duration readTimeout)`, que usa la fábrica. El resto de la clase no cambia.
3. En el `application.yml` de Quests añade, bajo `guildboard.adventurers`, `connect-timeout: 1s` y `read-timeout: 1s`. En `BeanConfig`, léelos con `@Value` como `Duration` y pásalos.
4. Con Quests arrancado y la misión publicada (sección 12.9.1), sin Adventurers en marcha, acepta la misión con `curl` y mide el tiempo.
5. Arranca en el puerto 8082 un servidor que acepta conexiones y no contesta nunca (el programa Java de la pista) y repite la petición.

Está bien si: con Adventurers parado, la respuesta es `500` en menos de medio segundo y el registro de Quests muestra un `ResourceAccessException` con «Connection refused» (o el texto equivalente de tu sistema); con el servidor mudo, la respuesta es `500` y `time_total` está entre 1,0 y 1,5 s, y el registro muestra un `ResourceAccessException` por tiempo agotado; y `./gradlew :quests-adapters:compileJava` compila. (Sigue siendo 500: la traducción a 503 llega en el ejercicio 12.4.)

Pista: *el servidor mudo es un programa Java de un solo archivo: un `ServerSocket` en el puerto 8082 que hace `accept` en bucle y `Thread.sleep(30000)` tras cada uno. Se ejecuta con `java Mudo.java`. La solución lo trae entero.*

Guiado ▪ solución en la pág. 459

Ejercicio 12.2: Reintento con espera creciente, solo en el GET ★★★☆☆

Parte del ejercicio anterior. Añade un reintento al `GET` del rango, sin *circuit breaker* todavía:

1. Añade la dependencia de la sección 12.6 a `quests-adapters` y a `quests-bootstrap`.
2. En `application.yml`, bloque `resilience4j.retry.instances.adventurers`: 3 intentos, espera inicial de 200 ms, multiplicador 2 y solo `ResourceAccessException` y `HttpServerErrorException`.
3. `RestAdventurerDirectory` recibe un `Retry` (constructor `(String, Duration, Duration, Retry)`) y lo aplica a la llamada con `Retry.decorateSupplier`.
4. `BeanConfig` pide el `RetryRegistry` y pasa `retries.retry("adventurers")`.
5. Con Adventurers parado, mide la petición de aceptar.
6. Contesta: ¿por qué no deberías aplicar esto mismo a un `POST`?

Está bien si: con Adventurers parado, la petición tarda entre 0,6 y 1,2 s (dos esperas, de 200 y 400 ms, y conexiones que se rechazan al instante) y sigue respondiendo `500`; con Adventurers arrancado y un aventurero que no existe, la respuesta es `404` y tarda lo de siempre (el 404 no se reintenta); `retry-exceptions` tiene exactamente las dos clases pedidas; y sabes decir qué ocurriría con una lista vacía de excepciones.

Pista: *la clase de la configuración es `RetryRegistry` (de `io.github.resilience4j.retry`); el registro lo crea la autoconfiguración, no tú.*

Práctica ▪ solución en la pág. 462

Ejercicio 12.3: El interruptor se abre, probado sin servidor ★★★☆☆

Parte de la clase `RestAdventurerDirectory` completa de la sección 12.6 (la versión programática, con el constructor `(RestClient, CircuitBreaker, Retry)`). Escribe una prueba JUnit en `quests-adapters` que fije el comportamiento del interruptor sin levantar ningún servidor:

1. Si aún no los tienes, copia del texto la excepción `AdventurerUnavailableException` y la clase `RestAdventurerDirectory` completa. `quests-bootstrap` no compilará hasta el ejercicio 12.4 (su `BeanConfig` sigue con el constructor del ejercicio 12.2); aquí solo necesitas `quests-adapters`.
2. No toques `services/quests/adapters/build.gradle` para las pruebas: JUnit, AssertJ y el lanzador de pruebas llegan a `quests-adapters` desde `subprojects {}` de la raíz, como en `quests-domain`.
3. Crea `RestAdventurerDirectoryBreakerTest` en el paquete `com.guildboard.quests.adapter.out.rest` de `src/test/java`.
4. Construye en la prueba un `CircuitBreaker` con la API programática (ventana de 4, mínimo de 4, umbral 50, 1 minuto abierto, 2 pruebas en semiabierto, y como fallos `ResourceAccessException`), un `Retry` de un solo intento y un `RestClient` que apunte a `http://localhost:1` (un puerto cerrado: la conexión rechazada es el fallo).
5. Escribe tres pruebas: con tres fallos el interruptor sigue `CLOSED`; con cuatro se abre y la quinta llamada se rechaza con `AdventurerUnavailableException` cuya causa es `CallNotPermittedException`; y pasado a `HALF_OPEN` a mano, dos fallos más lo vuelven a abrir.
6. Ejecuta `./gradlew :quests-adapters:test`.

Está bien si: las tres pruebas pasan; ninguna usa un servidor simulado, `Thread.sleep` ni Spring (es una prueba de clase pura con `new`); toda llamada fallida se comprueba con `AdventurerUnavailableException`; la prueba de la apertura comprueba tanto el estado (`getState()`) como que hubo una llamada no permitida (`getMetrics().getNumberOfNotPermittedCalls()`); y, si en la clase quitas `CircuitBreaker.decorateSupplier`, la segunda prueba falla (la vuelves a dejar como estaba).

Pista: *para pasar a semiabierto sin esperar un minuto, el interruptor tiene `transitionToHalfOpenState()`.*

Test ▪ solución en la pág. 465

Ejercicio 12.4: Hito H6b: Quests protegido y 503 sin Adventurers ★★★★☆

Parte del proyecto con los tres ejercicios anteriores hechos (fábrica con tiempos máximos, reintento y la prueba del interruptor). Falta lo que lo convierte en hito:

1. **El puerto y el adaptador.** `AdventurerUnavailableException` en `quests-application`, `port.out`, y `RestAdventurerDirectory` con el constructor de cinco parámetros y la traducción (sección 12.6; si los copiaste en el ejercicio anterior, comprueba que coinciden).
2. **La composición.** `BeanConfig` pide también el `CircuitBreakerRegistry` y pasa `breakers.circuitBreaker("adventurers")`; el bloque `circuitbreaker` de `application.yml`.
3. **El 503.** Un `@RestControllerAdvice` nuevo, `AdventurerUnavailableExceptionHandler`, que devuelve un `ProblemDetail` 503 con `code ADVENTURERS_UNAVAILABLE` y deja una línea en el registro.
4. Ejecuta `./gradlew build`, el escenario de la sección 12.9.1 y haz el *commit*.

Está bien si: `./gradlew build` termina con `BUILD SUCCESSFUL` (incluidas `test`, con la prueba de arquitectura, y la prueba del ejercicio 12.3); con Adventurers parado, aceptar da `503` con `code ADVENTURERS_UNAVAILABLE`, la primera petición tarda unos 0,6 s y las que llegan con el interruptor abierto, milisegundos; con Adventurers arrancado, aceptar vuelve a funcionar tras la espera de 10 s; `git diff -stat` sobre `quests-domain` no muestra ningún archivo, y en `quests-application` solo aparece el archivo nuevo; ni `domain` ni `application` contienen `io.github.resilience4j` ni `org.springframework.web.client`; y `build.gradle` de ningún servicio nombra a otro.

Pista: *el 404 de un aventurero que no existe sigue siendo un 404 (`AdventurerNotFound`): no es un fallo del interruptor.*

Proyecto ▪ solución en la pág. 472

Ejercicio 12.5: Política de degradación de AcceptQuest ★★★★★

Quests protegido responde 503 cuando Adventurers cae. El gremio quiere que los aventureros puedan seguir aceptando misiones durante las caídas de Adventurers, que duran minutos. Decide qué hace `AcceptQuest` y entrégalo como un documento de una página más el código que haga falta:

- Elige entre rechazar, usar un rango en caché y aceptar condicionalmente (o propón una variante), y justifica la decisión con las reglas del dominio que arriesga (R3, R4 y R6 de `docs/dominio.md`).

- Di qué ve el cliente HTTP en cada caso (estado, cabeceras) y qué pasa si la caída dura una hora.
- Di dónde vive el código de la decisión (¿dominio, aplicación, adaptador?) y por qué ahí; esboza o escribe el código.
- Di qué mirarías en producción para saber que la política funciona, y qué coste tienen las dos opciones que no elegiste.

Criterios de valoración (no hay una única respuesta correcta): (1) cada opción se relaciona con al menos una regla concreta (R3, R4 o R6) y se dice qué puede salir mal; (2) ninguna opción deja una regla del dominio sin cumplir sin decirlo y sin una compensación; (3) la decisión vive en una capa coherente con la arquitectura (el dominio y `application` no conocen HTTP ni Resilience4j); (4) el comportamiento visible para el cliente está definido, incluida la caída larga; (5) se nombran los costes de lo descartado.

Diseño ▪ solución en la pág. 481

12.11 Resumen

Resumen

- Una llamada de red falla de más formas que una local, y la peligrosa es la lenta: sin tiempo máximo, un servicio caído retiene hilos del que lo llama y lo arrastra (fallo en cascada).
- Un *timeout* de conexión y otro de lectura en toda llamada de red. En Spring van en la fábrica de peticiones del `RestClient`; al vencer, `ResourceAccessException`. Se eligen con un presupuesto, no a ojo.
- Se reintenta solo lo que es seguro repetir (un `GET`, o un `POST` contra un servidor idempotente), solo ante fallos transitorios, con pocos intentos y espera creciente. Sin lista de excepciones, Resilience4j reintenta hasta contra un interruptor abierto.
- Un *circuit breaker* tiene tres estados: cerrado (anota resultados), abierto (rechaza al instante) y semiabierto (deja pasar unas pruebas). Con `recordExceptions`, lo que no está en la lista cuenta como éxito. Los valores por omisión no sirven para pruebas con pocas llamadas.
- Resilience4j 2.4.0 se instala con `resilience4j-spring-boot4` (versión explícita, no está en el BOM de Boot); las anotaciones necesitan además AspectJ (`spring-boot-starter-aspectj`) y proxies; la API programática no. El proyecto usa la programática; las anotaciones no se han podido ejecutar.
- Resilience4j vive solo en el adaptador de salida. El puerto expone `AdventurerUnavailableException` (“no sé si existe”), distinta de `Optional.empty()` (“no existe”); el adaptador web la traduce a 503.
- Qué hacer sin respuesta (rechazar, cachear, aceptar condicionalmente) es una decisión de negocio con consecuencias sobre las reglas del dominio. El valor por omisión de GuildBoard es rechazar con 503. `@Retryable` de Spring Framework 7 reintenta pero no tiene *circuit breaker*.
- El capítulo no cambia la estructura: solo archivos dentro de módulos y paquetes que ya existen, y `./gradlew test` lo comprueba.

Soluciones de los ejercicios

Solución del ejercicio 12.1 enunciado en la pág. 455

1. La fábrica. Junto a `RestAdventurerDirectory`, en `quests-adapters`:

adapter/out/rest/AdventurerClientFactory.java

```
package com.guildboard.quests.adapter.out.rest;

import java.net.http.HttpClient;
import java.time.Duration;
import org.springframework.http.client.JdkClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

public final class AdventurerClientFactory {

    private AdventurerClientFactory() {
    }

    public static RestClient create(String baseUrl,
            Duration connectTimeout, Duration readTimeout) {
        HttpClient http = HttpClient.newBuilder()
            .connectTimeout(connectTimeout)
            .build();
        JdkClientHttpRequestFactory factory =
            new JdkClientHttpRequestFactory(http);
        factory.setReadTimeout(readTimeout);
        return RestClient.builder()
            .baseUrl(baseUrl)
            .requestFactory(factory)
            .build();
    }
}
```

2. El adaptador. Solo cambia el constructor; `findById` es el del capítulo 9:

adapter/out/rest/RestAdventurerDirectory.java

```java
package com.guildboard.quests.adapter.out.rest;

import com.guildboard.quests.application.port.out.AdventurerDirectory;
import com.guildboard.quests.application.port.out.AdventurerView;
import com.guildboard.quests.domain.AdventurerId;
import java.time.Duration;
import java.util.Optional;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.RestClient;

public class RestAdventurerDirectory implements AdventurerDirectory {

    private final RestClient client;

    public RestAdventurerDirectory(String baseUrl,
            Duration connectTimeout, Duration readTimeout) {
        this.client = AdventurerClientFactory.create(
            baseUrl, connectTimeout, readTimeout);
    }

    @Override
    public Optional<AdventurerView> findById(AdventurerId id) {
        try {
            AdventurerPayload payload = client.get()
                .uri("/adventurers/{id}", id.value())
                .retrieve()
                .body(AdventurerPayload.class);
            return Optional.ofNullable(payload)
                .map(AdventurerPayloadMapper::toView);
        } catch (HttpClientErrorException.NotFound e) {
            return Optional.empty();
        }
    }
}
```

3. La composición. En `BeanConfig` (módulo `quests-bootstrap`) añade `import java.time.Duration;` y sustituye el método del bean `AdventurerDirectory`; el resto de la clase no cambia:

config/BeanConfig.java (solo el bean AdventurerDirectory)

```java
    @Bean
    public AdventurerDirectory adventurerDirectory(
            @Value("${guildboard.adventurers.base-url}") String baseUrl,
            @Value("${guildboard.adventurers.connect-timeout}")
            Duration connectTimeout,
            @Value("${guildboard.adventurers.read-timeout}")
            Duration readTimeout) {
        return new RestAdventurerDirectory(
            baseUrl, connectTimeout, readTimeout);
    }
```

quests-bootstrap: application.yml (bloque guildboard)

```yaml
guildboard:
  adventurers:
    base-url: http://localhost:8082
    connect-timeout: 1s
    read-timeout: 1s
  messaging:
    exchange: guildboard.events
    routing-key: quest.completed
```

4. Con Adventurers parado. Con Quests arrancado y `$QUEST` definida como en la sección 12.9.1:

```
$ curl -s -o /dev/null -w '%{http_code} %{time_total}s\n' \
    -X POST localhost:8081/quests/$QUEST/accept \
    -H 'Content-Type: application/json' -d '{"adventurerId":"mara"}'
500 0.04s
```

El tiempo es solo orientativo (unos milisegundos). En el registro de Quests aparece algo como esto (el texto exacto depende del sistema):

```
ResourceAccessException: I/O error on GET request for
"http://localhost:8082/adventurers/mara": Connection refused
```

La conexión se rechaza al instante: el tiempo máximo de conexión no llega a actuar.

5. Con un servidor mudo. Este programa Java acepta conexiones en el 8082 y no contesta nunca (los segundos que duerme son para que la conexión siga abierta). Es un archivo suelto, fuera del proyecto, y el JDK lo ejecuta directamente sin compilarlo antes:

Mudo.java

```java
import java.net.ServerSocket;
import java.net.Socket;

public class Mudo {
    public static void main(String[] args) throws Exception {
        try (ServerSocket server = new ServerSocket(8082)) {
            System.out.println("escuchando en 8082 sin contestar");
            while (true) {
                Socket connection = server.accept();
                Thread.sleep(30_000);
                connection.close();
            }
        }
    }
}
```

```
$ java Mudo.java
escuchando en 8082 sin contestar
```

En otra terminal, la misma petición:

```
$ curl -s -o /dev/null -w '%{http_code} %{time_total}s\n' \
    -X POST localhost:8081/quests/$QUEST/accept \
    -H 'Content-Type: application/json' -d '{"adventurerId":"mara"}'
500 1.03s
```

El tiempo es el del tiempo máximo de lectura (algo más de 1 s), y el registro muestra un `ResourceAccessException` cuyo texto indica que la petición agotó el tiempo (el JDK dice algo como «request timed out»). Termina el programa con Ctrl+C.

Por qué funciona. El de conexión va en el `HttpClient` del JDK; el de lectura, en la fábrica de Spring, porque el `HttpClient` no tiene una noción de «tiempo máximo de lectura» global. Ambos acaban en la misma excepción. Los valores llegan por `@Value` porque el adaptador no debe saber de dónde salen.

Errores típicos. (1) Poner solo el tiempo de conexión y creer que ya está: el servidor mudo seguiría colgando la petición. (2) Un `Duration` que no se convierte: si al arrancar ves un error de conversión de `String` a `Duration` en `@Value`, escribe los valores en formato ISO (`PT1S`); no lo he podido ejecutar. (3) Dejar un servidor mudo vivo en el 8082 y preguntarte luego por qué Adventurers no arranca.

Solución del ejercicio 12.2

enunciado en la pág. 456

1. Dependencia. Una línea en cada uno de los dos `build.gradle`:

services/quests/adapters/build.gradle (dependencies)

```
dependencies {
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'io.github.resilience4j:resilience4j-spring-boot4:2.4.0'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

services/quests/bootstrap/build.gradle (dependencies)

```
dependencies {
    implementation project(':quests-adapters')
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'io.github.resilience4j:resilience4j-spring-boot4:2.4.0'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

2. Configuración. Añade al `application.yml` de Quests:

quests-bootstrap: application.yml (bloque nuevo)

```
resilience4j:
  retry:
    instances:
      adventurers:
        max-attempts: 3
        wait-duration: 200ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        retry-exceptions:
          - org.springframework.web.client.ResourceAccessException
          - org.springframework.web.client.HttpServerErrorException
```

3. El adaptador. Constructor con `Retry` y llamada decorada:

adapter/out/rest/RestAdventurerDirectory.java

```java
package com.guildboard.quests.adapter.out.rest;

import com.guildboard.quests.application.port.out.AdventurerDirectory;
import com.guildboard.quests.application.port.out.AdventurerView;
import com.guildboard.quests.domain.AdventurerId;
import io.github.resilience4j.retry.Retry;
import java.time.Duration;
import java.util.Optional;
import java.util.function.Supplier;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.RestClient;

public class RestAdventurerDirectory implements AdventurerDirectory {

    private final RestClient client;
    private final Retry retry;

    public RestAdventurerDirectory(String baseUrl,
            Duration connectTimeout, Duration readTimeout,
            Retry retry) {
        this.client = AdventurerClientFactory.create(
            baseUrl, connectTimeout, readTimeout);
        this.retry = retry;
    }

    @Override
    public Optional<AdventurerView> findById(AdventurerId id) {
        Supplier<Optional<AdventurerView>> call = () -> fetch(id);
        return Retry.decorateSupplier(retry, call).get();
    }

    private Optional<AdventurerView> fetch(AdventurerId id) {
        try {
            AdventurerPayload payload = client.get()
                .uri("/adventurers/{id}", id.value())
                .retrieve()
                .body(AdventurerPayload.class);
            return Optional.ofNullable(payload)
                .map(AdventurerPayloadMapper::toView);
        } catch (HttpClientErrorException.NotFound e) {
            return Optional.empty();
        }
    }
}
```

4. La composición. En `BeanConfig` añade `import io.github.resilience4j.retry.RetryRegistry;` y sustituye el bean:

config/BeanConfig.java (solo el bean AdventurerDirectory)

```
    @Bean
    public AdventurerDirectory adventurerDirectory(
            @Value("${guildboard.adventurers.base-url}") String baseUrl,
            @Value("${guildboard.adventurers.connect-timeout}")
            Duration connectTimeout,
            @Value("${guildboard.adventurers.read-timeout}")
            Duration readTimeout,
            RetryRegistry retries) {
        return new RestAdventurerDirectory(baseUrl,
            connectTimeout, readTimeout, retries.retry("adventurers"));
    }
```

5. Medir. Con Adventurers parado:

```
$ curl -s -o /dev/null -w '%{http_code} %{time_total}s\n' \
    -X POST localhost:8081/quests/$QUEST/accept \
    -H 'Content-Type: application/json' -d '{"adventurerId":"mara"}'
500 0.62s
```

Tres intentos que se rechazan al instante más las dos esperas de 200 y 400 ms: unos 0,6 s (valor orientativo). Sigue siendo `500`: el reintento relanza la última excepción cuando se agotan los intentos.

6. ¿Por qué no un `POST`? Un reintento repite la petición sin saber si la primera llegó. En un `GET` da igual; en un `POST` que crea algo, si lo que se perdió fue la *respuesta*, el reintento crea el recurso dos veces. Solo sería seguro si el servidor fuera idempotente (como Treasury con `questId`), y eso lo garantiza el servidor, no el cliente. Quests solo hace un `GET` a otro servicio.

Qué pasa con la lista vacía. Resilience4j reintenta ante cualquier excepción: también un error 4xx (que no mejora) o un `IllegalStateException` del mapeador, y, cuando añadas el interruptor, el rechazo por circuito abierto. Es lo que hace que la lista no sea opcional.

Errores típicos. (1) Pedir a la composición una instancia con otro nombre (`"adventurer"`): no falla, pero usa la configuración por omisión (500 ms de espera, sin crecimiento). (2) Poner el reintento en un método llamado desde el mismo bean con anotaciones; con la API programática no ocurre. (3) Olvidar la dependencia en `bootstrap`: no compila `BeanConfig`, que usa `RetryRegistry`.

Solución del ejercicio 12.3

enunciado en la pág. 456

1. Dependencias de prueba. No hay que tocar nada: JUnit, AssertJ y el lanzador de pruebas llegan a `quests-adapters` desde `subprojects {}`. Su `build.gradle` queda así, con solo las dependencias de producción:

services/quests/adapters/build.gradle

```
dependencies {
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'io.github.resilience4j:resilience4j-spring-boot4:2.4.0'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

2. El código que se prueba. La excepción del puerto y el adaptador completo (los de la sección 12.6); la fábrica es la del ejercicio 12.1:

application/port/out/AdventurerUnavailableException.java

```
package com.guildboard.quests.application.port.out;

public class AdventurerUnavailableException extends RuntimeException {

    public AdventurerUnavailableException(String message,
                                          Throwable cause) {
        super(message, cause);
    }
}
```

adapter/out/rest/RestAdventurerDirectory.java (1 de 2)

```java
package com.guildboard.quests.adapter.out.rest;

import com.guildboard.quests.application.port.out.AdventurerDirectory;
import com.guildboard.quests.application.port.out.
    ↪ AdventurerUnavailableException;
import com.guildboard.quests.application.port.out.AdventurerView;
import com.guildboard.quests.domain.AdventurerId;
import io.github.resilience4j.circuitbreaker.CallNotPermittedException;
import io.github.resilience4j.circuitbreaker.CircuitBreaker;
import io.github.resilience4j.retry.Retry;
import java.time.Duration;
import java.util.Optional;
import java.util.function.Supplier;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.HttpServerErrorException;
import org.springframework.web.client.ResourceAccessException;
import org.springframework.web.client.RestClient;

public class RestAdventurerDirectory implements AdventurerDirectory {

    private final RestClient client;
    private final CircuitBreaker breaker;
    private final Retry retry;

    public RestAdventurerDirectory(String baseUrl,
            Duration connectTimeout, Duration readTimeout,
            CircuitBreaker breaker, Retry retry) {
        this(AdventurerClientFactory.create(
                baseUrl, connectTimeout, readTimeout),
            breaker, retry);
    }

    public RestAdventurerDirectory(RestClient client,
            CircuitBreaker breaker, Retry retry) {
        this.client = client;
        this.breaker = breaker;
        this.retry = retry;
    }
```

adapter/out/rest/RestAdventurerDirectory.java (2 de 2: sigue dentro de la clase)

```
    @Override
    public Optional<AdventurerView> findById(AdventurerId id) {
        Supplier<Optional<AdventurerView>> call = () -> fetch(id);
        Supplier<Optional<AdventurerView>> protectedCall =
            Retry.decorateSupplier(retry,
                CircuitBreaker.decorateSupplier(breaker, call));
        try {
            return protectedCall.get();
        } catch (CallNotPermittedException e) {
            throw new AdventurerUnavailableException(
                "adventurers circuit breaker is open", e);
        } catch (ResourceAccessException
                 | HttpServerErrorException e) {
            throw new AdventurerUnavailableException(
                "adventurers did not answer", e);
        }
    }

    private Optional<AdventurerView> fetch(AdventurerId id) {
        try {
            AdventurerPayload payload = client.get()
                .uri("/adventurers/{id}", id.value())
                .retrieve()
                .body(AdventurerPayload.class);
            return Optional.ofNullable(payload)
                .map(AdventurerPayloadMapper::toView);
        } catch (HttpClientErrorException.NotFound e) {
            return Optional.empty();
        }
    }
}
```

3. La prueba. En `services/quests/adapters/src/test/java/com/guildboard/quests/adapter/out/rest/`. Primera parte: la configuración y un auxiliar; la segunda, las tres pruebas.

RestAdventurerDirectoryBreakerTest.java (1 de 3)

```
package com.guildboard.quests.adapter.out.rest;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

import com.guildboard.quests.application.port.out.
    ↪ AdventurerUnavailableException;
import com.guildboard.quests.domain.AdventurerId;
import io.github.resilience4j.circuitbreaker.CallNotPermittedException;
import io.github.resilience4j.circuitbreaker.CircuitBreaker;
import io.github.resilience4j.circuitbreaker.CircuitBreakerConfig;
import io.github.resilience4j.circuitbreaker.CircuitBreakerConfig.
    ↪ SlidingWindowType;
import io.github.resilience4j.retry.Retry;
import io.github.resilience4j.retry.RetryConfig;
import java.time.Duration;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.web.client.ResourceAccessException;
import org.springframework.web.client.RestClient;
```

Sigue el mismo archivo (*continuación*):

RestAdventurerDirectoryBreakerTest.java (2 de 3: sigue dentro de la clase)

```
class RestAdventurerDirectoryBreakerTest {

    private static final AdventurerId MARA = new AdventurerId("mara");

    private CircuitBreaker breaker;
    private RestAdventurerDirectory directory;

    @BeforeEach
    void setUp() {
        CircuitBreakerConfig config = CircuitBreakerConfig.custom()
            .slidingWindowType(SlidingWindowType.COUNT_BASED)
            .slidingWindowSize(4)
            .minimumNumberOfCalls(4)
            .failureRateThreshold(50)
            .waitDurationInOpenState(Duration.ofMinutes(1))
            .permittedNumberOfCallsInHalfOpenState(2)
            .recordExceptions(ResourceAccessException.class)
            .build();
        breaker = CircuitBreaker.of("adventurers-test", config);
        Retry oneAttempt = Retry.of("adventurers-test",
            RetryConfig.custom().maxAttempts(1).build());
        RestClient closedPort = AdventurerClientFactory.create(
            "http://localhost:1",
            Duration.ofMillis(500), Duration.ofMillis(500));
        directory = new RestAdventurerDirectory(
            closedPort, breaker, oneAttempt);
    }

    private void callAndFail() {
        assertThatThrownBy(() -> directory.findById(MARA))
            .isInstanceOf(AdventurerUnavailableException.class)
            .hasCauseInstanceOf(ResourceAccessException.class);
    }
```

RestAdventurerDirectoryBreakerTest.java (3 de 3: sigue dentro de la clase)

```
    @Test
    void staysClosedBeforeTheMinimumNumberOfCalls() {
        callAndFail();
        callAndFail();
        callAndFail();

        assertThat(breaker.getState())
            .isEqualTo(CircuitBreaker.State.CLOSED);
        assertThat(breaker.getMetrics().getNumberOfFailedCalls())
            .isEqualTo(3);
    }

    @Test
    void opensAfterFourFailuresAndRejectsTheNextCall() {
        for (int i = 0; i < 4; i++) {
            callAndFail();
        }

        assertThat(breaker.getState())
            .isEqualTo(CircuitBreaker.State.OPEN);
        assertThatThrownBy(() -> directory.findById(MARA))
            .isInstanceOf(AdventurerUnavailableException.class)
            .hasCauseInstanceOf(CallNotPermittedException.class);
        assertThat(breaker.getMetrics().getNumberOfNotPermittedCalls())
            .isEqualTo(1);
    }

    @Test
    void opensAgainWhenTheTrialCallsFail() {
        for (int i = 0; i < 4; i++) {
            callAndFail();
        }
        breaker.transitionToHalfOpenState();
        assertThat(breaker.getState())
            .isEqualTo(CircuitBreaker.State.HALF_OPEN);

        callAndFail();
        callAndFail();

        assertThat(breaker.getState())
            .isEqualTo(CircuitBreaker.State.OPEN);
    }
}
```

Por qué funciona. El puerto 1 está cerrado, así que cada llamada falla con una conexión rechazada, que Spring envuelve en `ResourceAccessException`: es un fallo real de red, sin simular nada. Con una ventana de 4 y un mínimo de 4, el interruptor no calcula el porcentaje hasta la cuarta llamada (primera prueba: `CLOSED` con tres fallos anotados); a la cuarta, el 100 % de fallos supera el 50 % y se abre (segunda prueba); la siguiente llamada ni siquiera sale a la red: el interruptor lanza `CallNotPermittedException` y el adaptador la traduce. La tercera prueba usa `transitionToHalfOpenState()` para no esperar un minuto: con dos llamadas de prueba permitidas y las dos fallando, vuelve a abrirse. El `Retry` de un

solo intento evita que el reintento multiplique las llamadas y descuadre las cuentas. Los tiempos máximos de 500 ms hacen que, si en tu máquina algo filtra el puerto 1 en lugar de rechazarlo, la prueba siga terminando.

Errores típicos. (1) Olvidar `recordExceptions`: con la lista vacía, la excepción de red igualmente cuenta como fallo, pero el comportamiento ya no corresponde al del proyecto. (2) Dejar el `Retry` con tres intentos: cada llamada del adaptador pasa a contar tres veces en el interruptor y la primera prueba falla (se abriría antes). (3) Usar un puerto en el que sí escucha algo.

Solución del ejercicio 12.4

enunciado en la pág. 457

1. Los dos `build.gradle`. El de `adapters` es el de la solución del ejercicio 12.3; el de `bootstrap`:

services/quests/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencies {
    implementation project(':quests-adapters')
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'io.github.resilience4j:resilience4j-spring-boot4:2.4.0'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

services/quests/adapters/build.gradle

```
dependencies {
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'io.github.resilience4j:resilience4j-spring-boot4:2.4.0'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

2. El puerto y el adaptador. La excepción, la fábrica y `RestAdventurerDirectory` son exactamente los de las soluciones de los ejercicios 12.1 y 12.3 (el adaptador, con el constructor de cinco parámetros). Para que el ejercicio quede entero aquí, el adaptador:

adapter/out/rest/RestAdventurerDirectory.java (1 de 2)

```java
package com.guildboard.quests.adapter.out.rest;

import com.guildboard.quests.application.port.out.AdventurerDirectory;
import com.guildboard.quests.application.port.out.
    ↪ AdventurerUnavailableException;
import com.guildboard.quests.application.port.out.AdventurerView;
import com.guildboard.quests.domain.AdventurerId;
import io.github.resilience4j.circuitbreaker.CallNotPermittedException;
import io.github.resilience4j.circuitbreaker.CircuitBreaker;
import io.github.resilience4j.retry.Retry;
import java.time.Duration;
import java.util.Optional;
import java.util.function.Supplier;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.HttpServerErrorException;
import org.springframework.web.client.ResourceAccessException;
import org.springframework.web.client.RestClient;

public class RestAdventurerDirectory implements AdventurerDirectory {

    private final RestClient client;
    private final CircuitBreaker breaker;
    private final Retry retry;

    public RestAdventurerDirectory(String baseUrl,
            Duration connectTimeout, Duration readTimeout,
            CircuitBreaker breaker, Retry retry) {
        this(AdventurerClientFactory.create(
                baseUrl, connectTimeout, readTimeout),
            breaker, retry);
    }

    public RestAdventurerDirectory(RestClient client,
            CircuitBreaker breaker, Retry retry) {
        this.client = client;
        this.breaker = breaker;
        this.retry = retry;
    }
```

adapter/out/rest/RestAdventurerDirectory.java (2 de 2: sigue dentro de la clase)

```
    @Override
    public Optional<AdventurerView> findById(AdventurerId id) {
        Supplier<Optional<AdventurerView>> call = () -> fetch(id);
        Supplier<Optional<AdventurerView>> protectedCall =
            Retry.decorateSupplier(retry,
                CircuitBreaker.decorateSupplier(breaker, call));
        try {
            return protectedCall.get();
        } catch (CallNotPermittedException e) {
            throw new AdventurerUnavailableException(
                "adventurers circuit breaker is open", e);
        } catch (ResourceAccessException
                 | HttpServerErrorException e) {
            throw new AdventurerUnavailableException(
                "adventurers did not answer", e);
        }
    }

    private Optional<AdventurerView> fetch(AdventurerId id) {
        try {
            AdventurerPayload payload = client.get()
                .uri("/adventurers/{id}", id.value())
                .retrieve()
                .body(AdventurerPayload.class);
            return Optional.ofNullable(payload)
                .map(AdventurerPayloadMapper::toView);
        } catch (HttpClientErrorException.NotFound e) {
            return Optional.empty();
        }
    }
}
```

La excepción del puerto (`quests-application`) y la fábrica (`quests-adapters`) son las de las soluciones anteriores, sin cambios:

application/port/out/AdventurerUnavailableException.java

```
package com.guildboard.quests.application.port.out;

public class AdventurerUnavailableException extends RuntimeException {

    public AdventurerUnavailableException(String message,
                                          Throwable cause) {
        super(message, cause);
    }
}
```

adapter/out/rest/AdventurerClientFactory.java

```java
package com.guildboard.quests.adapter.out.rest;

import java.net.http.HttpClient;
import java.time.Duration;
import org.springframework.http.client.JdkClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

public final class AdventurerClientFactory {

    private AdventurerClientFactory() {
    }

    public static RestClient create(String baseUrl,
            Duration connectTimeout, Duration readTimeout) {
        HttpClient http = HttpClient.newBuilder()
            .connectTimeout(connectTimeout)
            .build();
        JdkClientHttpRequestFactory factory =
            new JdkClientHttpRequestFactory(http);
        factory.setReadTimeout(readTimeout);
        return RestClient.builder()
            .baseUrl(baseUrl)
            .requestFactory(factory)
            .build();
    }
}
```

3. El 503. Un manejador propio, separado de `DomainExceptionHandler`: la excepción no es del dominio, y así ese archivo no cambia.

adapter/in/web/error/AdventurerUnavailableExceptionHandler.java

```java
package com.guildboard.quests.adapter.in.web.error;

import com.guildboard.quests.application.port.out.
    ↪ AdventurerUnavailableException;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@Slf4j
@RestControllerAdvice
public class AdventurerUnavailableExceptionHandler {

    @ExceptionHandler(AdventurerUnavailableException.class)
    public ProblemDetail handleUnavailable(
            AdventurerUnavailableException exception) {
        log.warn("adventurers unavailable: {} ({})",
            exception.getMessage(), exception.getCause());
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.SERVICE_UNAVAILABLE,
            "the adventurers service cannot answer right now");
        problem.setTitle("Dependency unavailable");
        problem.setProperty("code", "ADVENTURERS_UNAVAILABLE");
        return problem;
    }
}
```

4. La composición. `BeanConfig` completo de Quests (dos trozos seguidos); cambia el bean `AdventurerDirectory` y los *imports*:

config/BeanConfig.java (Quests, 1 de 2)

```java
package com.guildboard.quests.config;

import com.guildboard.quests.adapter.out.messaging.
    ↪ RabbitQuestEventPublisher;
import com.guildboard.quests.adapter.out.notification.LogNotificationSender;
    ↪
import com.guildboard.quests.adapter.out.rest.RestAdventurerDirectory;
import com.guildboard.quests.application.port.in.AcceptQuestUseCase;
import com.guildboard.quests.application.port.in.CompleteQuestUseCase;
import com.guildboard.quests.application.port.in.GetQuestUseCase;
import com.guildboard.quests.application.port.in.PublishQuestUseCase;
import com.guildboard.quests.application.port.out.AdventurerDirectory;
import com.guildboard.quests.application.port.out.NotificationSender;
import com.guildboard.quests.application.port.out.QuestEventPublisher;
import com.guildboard.quests.application.port.out.QuestRepository;
import com.guildboard.quests.application.service.AcceptQuestService;
import com.guildboard.quests.application.service.CompleteQuestService;
import com.guildboard.quests.application.service.GetQuestService;
import com.guildboard.quests.application.service.PublishQuestService;
import io.github.resilience4j.circuitbreaker.CircuitBreakerRegistry;
import io.github.resilience4j.retry.RetryRegistry;
import java.time.Clock;
import java.time.Duration;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public Clock clock() {
        return Clock.systemUTC();
    }

    @Bean
    public NotificationSender notificationSender() {
        return new LogNotificationSender();
    }
```

config/BeanConfig.java (Quests, 2 de 2: sigue dentro de la clase)

```
    @Bean
    public AdventurerDirectory adventurerDirectory(
            @Value("${guildboard.adventurers.base-url}") String baseUrl,
            @Value("${guildboard.adventurers.connect-timeout}")
            Duration connectTimeout,
            @Value("${guildboard.adventurers.read-timeout}")
            Duration readTimeout,
            CircuitBreakerRegistry breakers, RetryRegistry retries) {
        return new RestAdventurerDirectory(baseUrl,
            connectTimeout, readTimeout,
            breakers.circuitBreaker("adventurers"),
            retries.retry("adventurers"));
    }

    @Bean
    public QuestEventPublisher questEventPublisher(
            RabbitTemplate rabbitTemplate,
            @Value("${guildboard.messaging.exchange}") String exchange,
            @Value("${guildboard.messaging.routing-key}") String key) {
        return new RabbitQuestEventPublisher(
            rabbitTemplate, exchange, key);
    }

    @Bean
    public PublishQuestUseCase publishQuestUseCase(
            QuestRepository questRepository, Clock clock) {
        return new PublishQuestService(questRepository, clock);
    }

    @Bean
    public GetQuestUseCase getQuestUseCase(
            QuestRepository questRepository) {
        return new GetQuestService(questRepository);
    }

    @Bean
    public AcceptQuestUseCase acceptQuestUseCase(
            QuestRepository questRepository,
            AdventurerDirectory adventurerDirectory, Clock clock) {
        return new AcceptQuestService(questRepository,
                                      adventurerDirectory, clock);
    }

    @Bean
    public CompleteQuestUseCase completeQuestUseCase(
            QuestRepository questRepository,
            QuestEventPublisher questEventPublisher,
            NotificationSender notificationSender) {
        return new CompleteQuestService(questRepository,
            questEventPublisher, notificationSender);
    }
}
```

5. La configuración. El `application.yml` completo de Quests:

quests-bootstrap: application.yml (1 de 2)

```yaml
spring:
  application:
    name: quests
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration
  rabbitmq:
    host: localhost
    port: 5672
    username: guildboard
    password: guildboard

server:
  port: 8081

guildboard:
  adventurers:
    base-url: http://localhost:8082
    connect-timeout: 1s
    read-timeout: 1s
  messaging:
    exchange: guildboard.events
    routing-key: quest.completed
```

quests-bootstrap: application.yml (2 de 2: sigue a nivel raíz)

```yaml
resilience4j:
  circuitbreaker:
    instances:
      adventurers:
        sliding-window-size: 10
        minimum-number-of-calls: 5
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 2
        record-exceptions:
          - org.springframework.web.client.ResourceAccessException
          - org.springframework.web.client.HttpServerErrorException
  retry:
    instances:
      adventurers:
        max-attempts: 3
        wait-duration: 200ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        retry-exceptions:
          - org.springframework.web.client.ResourceAccessException
          - org.springframework.web.client.HttpServerErrorException
```

6. Comprobar. Con Quests arrancado, la misión publicada y Adventurers parado (la secuencia de la sección 12.9.1), lo esperado es:

```
$ for i in 1 2 3 4; do curl -s -o /dev/null \
    -w '%{http_code} %{time_total}s\n' \
    -X POST localhost:8081/quests/$QUEST/accept \
    -H 'Content-Type: application/json' \
    -d '{"adventurerId":"mara"}'; done
503 0.62s
503 0.62s
503 0.01s
503 0.01s
```

Las dos primeras peticiones agotan los tres intentos (la segunda ve cómo se abre el interruptor y su tercer intento ya es rechazado); las siguientes se rechazan sin salir a la red. Los números son orientativos y no los he podido ejecutar. Con `curl -s` sin `-o` se ve el cuerpo: un `ProblemDetail` con `status` 503, `title` «Dependency unavailable» y `code` `ADVENTURERS_UNAVAILABLE`. Por último, `./gradlew test` y `./gradlew build`.

Por qué funciona. Cada capa hace lo suyo: el adaptador de salida protege y traduce, el servicio de aplicación no cambia (la excepción lo atraviesa porque ocurre antes de modificar nada: la misión no se guarda), y el adaptador web convierte una excepción de aplicación en un código HTTP. `git diff -stat` sobre `domain` sale vacío y sobre `application` muestra solo el archivo nuevo.

Errores típicos. (1) Olvidar la dependencia en `bootstrap`: los registros no compilan en `BeanConfig`. (2) El nombre de la instancia distinto en el YAML y en la composición: sin error, con la configuración por omisión (el interruptor no se abre con cinco llamadas). (3) Meter el `@ExceptionHandler` en `DomainExceptionHandler` con un `switch` exhaustivo:

la excepción no es un `DomainError`. (4) Capturar `AdventurerUnavailableException` en `AcceptQuestService` y devolver `Optional.empty()`: se confundiría «no sé» con «no existe» y se mentiría con un 404.

Solución del ejercicio 12.5 enunciado en la pág. 457

Solución de referencia: rechazar (503) con `Retry-After`.
Decisión. Mientras Adventurers no responde, `AcceptQuest` rechaza con 503. No arriesga ninguna regla: R3 (el rango basta) no se puede comprobar, así que no se acepta; R4 (máximo 3 misiones activas) es un dato de Quests y no se ve afectado; R6 (la experiencia solo sube) no interviene. El coste es de disponibilidad: aceptar vale lo que valga Adventurers.
Qué ve el cliente. `503 Service Unavailable`, cuerpo `ProblemDetail` con `code ADVENTURERS_UNAVAILABLE` y cabecera `Retry-After: 10`, que coincide con `wait-duration-in-open-state`. Si la caída dura una hora, el cliente recibe el 503 durante una hora; Quests solo prueba a Adventurers dos veces cada 10 s (las llamadas semiabiertas), no con cada petición.
Dónde vive. Nada nuevo en el dominio ni en `application`: la decisión es «dejar subir la excepción», y la traducción a HTTP es del adaptador web. Cambiar el 503 por otra política sí tocaría `application`, porque entonces el caso de uso tendría que saber algo que hoy ignora.
Qué mirar. Tasa de respuestas 503, estado del interruptor y llamadas rechazadas (`getMetrics()`; con Actuator, capítulo 15) y, en el registro, la línea de `warn` del manejador. Alarma si el interruptor lleva abierto más de unos minutos.
Costes de lo descartado. El rango en caché ofrece más disponibilidad, pero añade estado (memoria y caducidad) y la posibilidad de rechazar de más; no resuelve a quien Quests nunca ha visto. Aceptar condicionalmente puede violar R3 y exige compensar (deshacer la aceptación), es decir, una saga, fuera del curso.
El único cambio de código de la referencia: la cabecera `Retry-After` en el manejador.

adapter/in/web/error/AdventurerUnavailableExceptionHandler.java

```java
package com.guildboard.quests.adapter.in.web.error;

import com.guildboard.quests.application.port.out.
    ↪ AdventurerUnavailableException;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@Slf4j
@RestControllerAdvice
public class AdventurerUnavailableExceptionHandler {

    // Igual que wait-duration-in-open-state del circuit breaker.
    private static final String RETRY_AFTER_SECONDS = "10";

    @ExceptionHandler(AdventurerUnavailableException.class)
    public ResponseEntity<ProblemDetail> handleUnavailable(
            AdventurerUnavailableException exception) {
        log.warn("adventurers unavailable: {} ({})",
            exception.getMessage(), exception.getCause());
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
            HttpStatus.SERVICE_UNAVAILABLE,
            "the adventurers service cannot answer right now");
        problem.setTitle("Dependency unavailable");
        problem.setProperty("code", "ADVENTURERS_UNAVAILABLE");
        return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE)
            .header(HttpHeaders.RETRY_AFTER, RETRY_AFTER_SECONDS)
            .body(problem);
    }
}
```

Criterios para valorar otras soluciones. (1) Cada opción se razona con las reglas que arriesga (R3, R4, R6) y sale algo concreto de cada una; (2) ninguna deja una regla sin cumplir sin decirlo y sin una compensación; (3) la decisión vive en una capa coherente (el dominio y `application` no conocen HTTP ni Resilience4j); (4) el comportamiento visible está definido, incluida la caída de una hora; (5) se nombran los costes de lo descartado.
Alternativa válida: rango en caché. Es válida *porque* el rango solo sube (R6): un valor antiguo es igual o menor que el real, así que la caché puede rechazar de más (el aventurero ascendió mientras tanto) pero nunca aceptar de más. Se implementa como un decorador del puerto en el adaptador, con nombre propio para que la política se vea. No cachea el «no existe» y, sin dato cacheado, deja subir la excepción, es decir, rechaza.

adapter/out/rest/CachedAdventurerDirectory.java (1 de 2)

```
package com.guildboard.quests.adapter.out.rest;

import com.guildboard.quests.application.port.out.AdventurerDirectory;
import com.guildboard.quests.application.port.out.
    ↪ AdventurerUnavailableException;
import com.guildboard.quests.application.port.out.AdventurerView;
import com.guildboard.quests.domain.AdventurerId;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

public class CachedAdventurerDirectory implements AdventurerDirectory {

    private record Entry(AdventurerView view, Instant storedAt) {
    }

    private final AdventurerDirectory delegate;
    private final Clock clock;
    private final Duration maxAge;
    private final Map<AdventurerId, Entry> cache =
        new ConcurrentHashMap<>();

    public CachedAdventurerDirectory(AdventurerDirectory delegate,
            Clock clock, Duration maxAge) {
        this.delegate = delegate;
        this.clock = clock;
        this.maxAge = maxAge;
    }
```

adapter/out/rest/CachedAdventurerDirectory.java (2 de 2: sigue dentro de la clase)

```
    @Override
    public Optional<AdventurerView> findById(AdventurerId id) {
        try {
            Optional<AdventurerView> fresh = delegate.findById(id);
            fresh.ifPresent(view ->
                cache.put(id, new Entry(view, clock.instant())));
            return fresh;
        } catch (AdventurerUnavailableException e) {
            Entry entry = cache.get(id);
            if (entry != null && clock.instant()
                    .isBefore(entry.storedAt().plus(maxAge))) {
                return Optional.of(entry.view());
            }
            throw e;
        }
    }
}
```

Y en `BeanConfig`, el bean envuelve al adaptador (con `import java.time.Duration;` ya presente y los *imports* de la clase nueva):

config/BeanConfig.java (bean AdventurerDirectory con caché)

```
    @Bean
    public AdventurerDirectory adventurerDirectory(
            @Value("${guildboard.adventurers.base-url}") String baseUrl,
            @Value("${guildboard.adventurers.connect-timeout}")
            Duration connectTimeout,
            @Value("${guildboard.adventurers.read-timeout}")
            Duration readTimeout,
            CircuitBreakerRegistry breakers, RetryRegistry retries,
            Clock clock) {
        AdventurerDirectory rest = new RestAdventurerDirectory(baseUrl,
            connectTimeout, readTimeout,
            breakers.circuitBreaker("adventurers"),
            retries.retry("adventurers"));
        return new CachedAdventurerDirectory(
            rest, clock, Duration.ofMinutes(10));
    }
```

La consecuencia que la caché obliga a asumir: con una caída de más de 10 minutos, la caché caduca y vuelve el 503 (la caída de una hora cuesta 50 minutos de rechazos). Y si algún día hubiera una regla que bajara el rango, esta opción dejaría de ser segura.

Errores típicos. (1) Cachear siempre, no solo cuando hay fallo: se sirve un rango viejo con Adventurers sano. (2) Esconder la caché dentro del `catch` del adaptador de REST: la política queda invisible. (3) Cachear `Optional.empty()`: un aventurero recién registrado se vería como inexistente. (4) Elegir «aceptar condicionalmente» sin proponer cómo se deshace una aceptación errónea.

CAPÍTULO 13

API gateway y configuración centralizada

Objetivos del capítulo

- Explicar qué problema resuelve un API gateway y cuáles crea, y configurar rutas con un predicado y un filtro (`Path` y `StripPrefix`) hacia Quests, Adventurers y Treasury.
- Sacar la configuración de despliegue de los servicios a un Config Server con backend nativo, leerla con `spring.config.import` y distinguir qué se mueve y qué no.
- Comprobar en la verificación de estructura 8 el árbol con `gateway/`, `config-server/` y `config-repo/` en la raíz, y justificar por qué esos proyectos no tienen hexágono.
- Dejar la primera mitad del hito H7: los tres servicios configurados desde un solo sitio y accesibles por el puerto 8080.

Hoy, para usar GuildBoard, un cliente tiene que saber que las misiones viven en el puerto 8081, los aventureros en el 8082 y la tesorería en el 8083. Si mañana Treasury cambia de puerto, o se parte en dos servicios, cambian todos los clientes. Y por dentro hay un problema gemelo: los puertos, la dirección de RabbitMQ, los tiempos máximos y las colas están escritos en el `application.yml` de cada servicio, es decir, dentro de su *jar*. Cambiar un valor es tocar el proyecto, y la misma compilación no se puede llevar tal cual a otro entorno. Este capítulo añade dos piezas de infraestructura: un *gateway* que da una sola entrada a los tres servicios, y un servidor de configuración que sirve a todos sus valores desde una carpeta.

Cuatro cosas que el capítulo deja dichas desde el principio:

- **Hay dependencias nuevas.** Spring Cloud: un BOM, el *starter* del gateway, el servidor de configuración y el cliente. Cada una se explica justo antes de usarla (secciones 13.4, 13.5 y 13.6), con lo que se verificó y lo que no.
- **La estructura cambia.** Entran tres carpetas en la raíz. La sección 13.3 muestra el árbol antes de crear nada, y el ejercicio 13.4 te pide revisarlo.
- **Los dos proyectos nuevos no tienen hexágono.** No es un descuido: la sección 13.2 explica por qué, y qué tendría que pasar para que lo necesitaran.
- **El código Java no se ha compilado ni ejecutado.** Como en todo el curso. Lo que sí se comprobó (nombres de artefactos, versiones, propiedades, la documentación) está marcado con su fecha; el resto es lo esperado.

13.1 Qué problema resuelve el gateway (y cuáles crea)

El patrón *API gateway* pone un único punto de entrada delante de los servicios. En su forma más simple reenvía cada petición al servicio que corresponde; en otras formas también combina respuestas de varios servicios o aplica la seguridad [41, 42]. Richardson cuenta como beneficios que aísla a los clientes de cómo está partida la aplicación y de dónde está cada instancia, y que

simplifica al cliente. También cuenta los costes: es una pieza más que desarrollar, desplegar y operar, y añade un salto de red a cada petición.

El recorrido con gateway

```
cliente --> gateway (8080) --/api/quests/**-------> quests      (8081)
                           --/api/adventurers/**--> adventurers (8082)
                           --/api/treasury/**-----> treasury    (8083)

quests ------------------------------------------> adventurers
        (la llamada del rango sigue siendo directa)
```

Para GuildBoard, el beneficio es concreto: el cliente conoce *una* dirección y un prefijo por servicio (`/api/quests`, `/api/adventurers`, `/api/treasury`), y los puertos pasan a ser un detalle interno. Los costes también lo son, y conviene tenerlos delante antes de construirlo:

- **Un punto único de fallo.** Si el gateway cae, no entra nadie, aunque los tres servicios estén vivos.
- **Un salto más.** Cada petición cruza dos procesos en lugar de uno.
- **Una tentación.** Como todo pasa por ahí, es fácil escribir en el gateway "solo una regla pequeña". Esta advertencia es mía, no de Richardson; la sección 13.2 la convierte en un criterio.
- **No es un cortafuegos.** Los servicios siguen escuchando en sus puertos; quien los alcance directamente se salta el gateway. La seguridad llega en el capítulo 14, y ni siquiera allí el gateway será la única barrera.

Este gateway **solo enruta**: no combina respuestas de varios servicios y no valida a nadie. Spring Cloud Gateway tiene dos variantes, una reactiva (WebFlux) y otra sobre Spring MVC con Servlets [57]. El curso usa la segunda: los tres servicios son aplicaciones Spring MVC y el modelo mental (un hilo por petición) es el que ya conoces.

13.2 Dos proyectos sin hexágono: el límite parcial

Hasta ahora, cada pieza nueva copiaba el molde de cuatro módulos. Estas dos no, y esa diferencia tiene una razón. La pregunta que decide si un proyecto necesita hexágono es una sola: **¿hay un núcleo que proteger?** El hexágono existe para que las reglas de negocio no dependan de HTTP, de la base de datos ni del *broker*. Mira qué hace cada pieza nueva:

- El **gateway** decide a qué dirección mandar una petición según su ruta. Eso es una tabla de rutas, no una regla del gremio. No hay estados, ni invariantes, ni casos de uso.
- El **Config Server** lee archivos de una carpeta y los sirve por HTTP. Tampoco hay negocio.

Un `domain` y un `application` vacíos serían ocho carpetas para una clase de diez líneas y un archivo YAML. Martin llama *límites parciales* (*partial boundaries*) a los casos en que pagar el coste completo de un límite no compensa y se deja a medias o se prepara sin construirlo [28]; aquí el caso es aún más sencillo, porque no hay nada al otro lado del límite. No es un argumento contra la regla de dependencia: es la comprobación de que la regla no tiene nada que proteger en estos dos proyectos.

Lo que sí se mantiene del molde:

- Un proyecto Gradle por pieza, con el plugin de Spring Boot y su clase principal mínima.
- Nada de lógica de negocio en esa clase: es el *main*, como en `bootstrap` (capítulo 3).
- Ningún `build.gradle` nombra un proyecto de un servicio: el gateway conoce a los servicios por URL, no por código.
- El servicio no sabe que existe un gateway: sus rutas internas no cambian.

La condición para reconsiderarlo es igual de corta que la pregunta: **en el momento en que alguno de los dos tenga que decidir algo que tenga significado para el negocio, ya hay un núcleo.** Por ejemplo, un gateway que rechazara a los aventureros de rango `NOVICE` a partir de cierto número de peticiones por minuto no estaría enrutando: estaría aplicando una regla del gremio. El ejercicio 13.4 te pide más señales, y qué cambiaría en el árbol.

13.3 Verificación de estructura 8: infraestructura en la raíz

Nota

Este capítulo cambia la estructura de carpetas: aparecen `gateway/`, `config-server/` y `config-repo/` en la raíz y cambia la configuración de los tres `bootstrap`. Comprueba que tu proyecto coincide con el árbol de abajo antes de seguir; las piezas se construyen en las secciones siguientes.

13.3.1 El árbol anterior

Así quedó el proyecto tras el capítulo 11 (árbol T6; el capítulo 12 no cambió carpetas):

Árbol anterior (T6, tras los capítulos 11 y 12)

```
guildboard/
|-- .git/, .gitignore, gradlew, gradlew.bat
|-- settings.gradle        doce proyectos (tres servicios x cuatro)
|-- build.gradle           configuraci^^c3^^b3n com^^c3^^ban (subprojects)
|-- compose.yaml           PostgreSQL 18 y RabbitMQ 4
|-- docs/dominio.md
|-- gradle/wrapper/
|-- infra/
|   `-- postgres/init.sql  crea las bases quests, adventurers, treasury
`-- services/
    |-- quests/            domain/ application/ adapters/ bootstrap/
    |-- adventurers/       domain/ application/ adapters/ bootstrap/
    `-- treasury/          domain/ application/ adapters/ bootstrap/
```

13.3.2 El árbol nuevo

`[N]` es nuevo y `[M]` modificado:

Árbol nuevo, raíz (T7, tras el capítulo 13)

```
guildboard/
|-- .git/, .gitignore, gradlew, gradlew.bat
|-- settings.gradle        [M] + include 'config-server', 'gateway'
```

```
|-- build.gradle           sin cambios
|-- compose.yaml           sin cambios
|-- docs/dominio.md        sin cambios
|-- gradle/wrapper/        sin cambios
|-- src/test/java/com/guildboard/StructureTest.java   sin cambios
|-- infra/postgres/        sin cambios
|-- gateway/               [N] proyecto 'gateway', puerto 8080
|   |-- build.gradle
|   `-- src/main/
|       |-- java/com/guildboard/gateway/GatewayApplication.java
|       `-- resources/application.yml
|-- config-server/         [N] proyecto 'config-server', puerto 8888
|   |-- build.gradle
|   `-- src/main/
|       |-- java/com/guildboard/configserver/
|       |   `-- ConfigServerApplication.java
|       `-- resources/application.yml
|-- config-repo/           [N] datos, no un proyecto Gradle
|   |-- application.yml    lo com^^c3^^ban a todos los clientes
|   |-- quests.yml
|   |-- adventurers.yml
|   |-- treasury.yml
|   `-- gateway.yml        las rutas
`-- services/              (ver abajo)
```

Dentro de `services/` no entra ninguna carpeta ni ningún paquete. Lo único que cambia, en cada uno de los tres servicios, son dos archivos de configuración de su `bootstrap`:

services/<servicio>/bootstrap (solo lo que cambia)

```
services/quests/bootstrap/           (igual en adventurers y treasury)
|-- build.gradle                     [M] + cliente de configuraci^^c3^^b3n
|-- src/test/java/com/guildboard/<servicio>/ArchitectureTest.java
|                                    sin cambios
`-- src/main/resources/application.yml
                                     [M] se vac^^c3^^ada: nombre, import y
                                     exclusiones; el resto, a config-repo
```

`domain`, `application` y `adapters` de los tres servicios quedan exactamente como estaban; lo comprobarás con `git diff` al final.

13.3.3 Cambios respecto al árbol anterior

- `gateway/` y `config-server/`: dos proyectos Gradle nuevos en la raíz, no bajo `services/`. Las fuentes describen qué hacen [57, 56]; `StructureTest` exige los cuatro módulos a todo lo que cuelga de `services/`, y estos no son servicios de negocio. Tampoco llevan `ArchitectureTest`: sin `domain` ni `application` no hay núcleo que proteger.
- `config-repo/`: carpeta de datos (YAML), no un proyecto Gradle; el servidor es código y el repositorio de configuración son datos.
- `settings.gradle`: dos `include` más (catorce proyectos en total); la carpeta se llama como el proyecto, así que no hace falta `projectDir`.
- `services/*/bootstrap`: `build.gradle` y `application.yml` cambian; no cambia ningún

paquete Java.

- `gateway` y `config-server` no llevan `domain` ni `application`: no contienen reglas de negocio, como explica la sección 13.2 [28].

13.3.4 Comprobaciones

- [ ] `./gradlew test` pasa (incluye las pruebas de arquitectura de los tres servicios y `StructureTest`). `gateway`, `config-server` y `config-repo` no llevan `ArchitectureTest`: no tienen núcleo que proteger. Ningún `domain` ni `application` cambia.
- [ ] Cada adaptador está en `in` o en `out` según quién lo invoca: sin cambios en los servicios; los proyectos nuevos no tienen adaptadores propios (el gateway lo aporta entero la biblioteca).
- [ ] Los nombres de carpeta dicen el rol (`gateway`, `config-server`), no el producto.
- [ ] Las carpetas de tu proyecto coinciden con el árbol: tres carpetas nuevas en la raíz, una línea en `settings.gradle` y dos archivos de configuración por servicio.
- [ ] Ningún `build.gradle` de un servicio nombra a un proyecto de otro: lo nuevo es una biblioteca, y `gateway` conoce a los servicios por URL en `config-repo`.
- [ ] Ningún proyecto de la raíz contiene lógica de negocio. Si alguno la tuviera, adoptaría el molde de cuatro módulos (ejercicio 13.4).

Si algo no coincide con el árbol, corrígelo antes de seguir.

13.4 Rutas, predicados y filtros

Una ruta del gateway tiene cuatro piezas:

- un `id`, que es solo su nombre;
- una `uri`, la dirección del servicio al que se reenvía;
- uno o varios **predicados**: condiciones sobre la petición (su ruta, su método, una cabecera…). La ruta se aplica si se cumplen;
- cero o más **filtros**: pequeñas transformaciones de la petición antes de reenviarla (o de la respuesta al volver).

En Spring Cloud Gateway Web MVC las rutas se pueden declarar en YAML o en Java [57]. El curso usa YAML por una razón práctica: las rutas son datos de despliegue (qué hay en qué dirección), y así viven en `config-repo` junto al resto de la configuración. Esta es una ruta completa:

Una ruta (YAML)

```yaml
spring:
  cloud:
    gateway:
      server:
        webmvc:
          routes:
            - id: quests
              uri: http://localhost:8081
              predicates:
```

```
10                - Path=/api/quests/**
11              filters:
12                - StripPrefix=1
```

Tres detalles que conviene fijar antes de leer más rutas:

- **El predicado `Path`** recibe patrones de Spring. En un patrón, `**` al final significa "cero o más segmentos de la ruta hasta el final", según la documentación de `PathPattern`. Por eso `/api/quests/**` casa con `/api/quests` (publicar una misión) y con `/api/quests/abc/accept`.
- **El filtro `StripPrefix`** recibe un número, `parts`: cuántos segmentos del principio de la ruta se quitan antes de reenviar. La documentación lo ilustra con `StripPrefix=2`: una petición a `/name/blue/red` llega al destino como `/red` [57].
- **La `uri` es solo el servidor.** Cualquier ruta que escribas en la `uri` de una ruta se ignora (lo dice la documentación): el destino es el servidor más la ruta que queda tras los filtros.

Las tres rutas de GuildBoard salen de una decisión: **cada servicio conserva sus rutas internas** (`/quests/...`, `/adventurers/...`, `/accounts/...`) y no sabe que hay un gateway delante. Todo el trabajo de traducción recae en el gateway:

Petición al gateway (8080)	Ruta	Le llega al servicio
GET /api/quests/7	quests, `StripPrefix=1`	GET :8081/quests/7
POST /api/quests	quests, `StripPrefix=1`	POST :8081/quests
GET /api/adventurers/7	adventurers, `StripPrefix=1`	GET :8082/adventurers/7
GET /api/treasury/accounts/7	treasury, `StripPrefix=2`	GET :8083/accounts/7
GET /api/other/x	ninguna	no llega a ningún servicio

Fíjate en la asimetría. En Quests y Adventurers, el segundo segmento público (`quests`, `adventurers`) coincide con el primero de la ruta interna, así que basta quitar `api`. En Treasury no: su ruta interna empieza por `accounts`, y el prefijo público `/api/treasury` son dos segmentos que hay que quitar (`StripPrefix=2`). Es el precio de una convención pública uniforme (`/api/<servicio>/...`) sobre servicios que no se escribieron con ella en mente. Había dos alternativas, y se descartaron: renombrar las rutas de Treasury a `/treasury/accounts/...` (el servicio cambiaría por culpa del gateway, justo lo que quisimos evitar) o usar un filtro de reescritura más general. Con tres rutas, contar segmentos es lo más simple que funciona; con treinta, habría que replantearlo.

☞ El gateway: Spring Cloud y su starter

El gateway es un proyecto Gradle nuevo, `gateway`, en la raíz. Necesita dos cosas que no tenías:

- **El BOM de Spring Cloud.** Spring Cloud se publica por *trenes de versiones* (*release trains*), no por versiones sueltas. El tren `2025.1.x` es el compatible con Spring Boot 4.0 y 4.1 (con 4.1 a partir de `2025.1.2`) [55]. El BOM `spring-cloud-dependencies` fija las versiones de todas sus bibliotecas y se importa con el mismo mecanismo que ya usas para el de Boot (capítulo 7): una importación más, sin quitar la de la raíz.

- **El *starter* del gateway**, variante Web MVC.

El `build.gradle` de `gateway`:

gateway/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencyManagement {
    imports {
        mavenBom 'org.springframework.cloud:' +
            'spring-cloud-dependencies:2025.1.3'
    }
}

dependencies {
    implementation 'org.springframework.cloud:' +
        'spring-cloud-starter-gateway-server-webmvc'
    implementation 'org.springframework.cloud:' +
        'spring-cloud-starter-config'
}
```

La segunda dependencia es el cliente de configuración (sección 13.6): el gateway también lee sus rutas del Config Server. No hay `project(...)` de ningún servicio.
Qué se verificó (9 de octubre de 2026). En Spring Initializr con Boot 4.1.1: la dependencia `cloud-gateway` ("Gateway") genera `spring-cloud-starter-gateway-server-webmvc` y la reactiva, `cloud-gateway-reactive`, genera la variante `webflux`; ambos usan el BOM `spring-cloud-dependencies` `2025.1.3`. En Maven Central existen las versiones `2025.1.0` a `2025.1.3`; el tren `2025.1.3` trae Gateway `5.0.3` y Config `5.0.5`, el mismo par de las referencias del curso. El *starter* del gateway depende de `spring-boot-starter-web`, que en Boot 4.1.1 existe y se describe como obsoleto a favor de `spring-boot-starter-webmvc`: el proyecto funciona igual, solo cambia el nombre. La propiedad `spring.cloud.gateway.server.webmvc.routes` figura en los metadatos del *jar* de Gateway `5.0.3`. La página de documentación del filtro `StripPrefix` escribe un prefijo distinto (`gateway.mvc`) en uno de sus ejemplos de otra sección: vale el de los metadatos, `gateway.server.webmvc`. **Qué no se verificó:** que el proyecto arranque, que el BOM `2025.1.3` (construido contra Boot 4.0.8, según sus propiedades) se lleve bien con Boot 4.1.1 al resolver las dependencias en Gradle (Initializr lo ofrece para 4.1.1, pero no lo he ejecutado) y el comportamiento exacto de las rutas.

Con esto el gateway se escribe casi entero en YAML. Las rutas de GuildBoard, que vivirán en `config-repo/gateway.yml`, son las de la tabla. El ejercicio 13.1 las practica sobre otra configuración y el ejercicio 13.3 te enseña a diagnosticar un prefijo mal quitado.

13.5 Config Server: la configuración fuera del jar

El patrón *externalized configuration* dice que un servicio debe poder ejecutarse en varios entornos sin modificarlo ni recompilarlo, y que para eso sus datos de conexión y de entorno se leen de fuera [42]. Spring ya lo hace con variables de entorno, argumentos y archivos. *Spring Cloud Config* da un paso más: un **servidor** que sirve esa configuración por HTTP a todos los servicios [56]. Lo que ganas: un solo lugar donde ver y cambiar los puertos, las URLs y las credenciales

de desarrollo de todo el sistema, y una sola fuente para lo que dos servicios deben acordar (el nombre del *exchange* del capítulo 11 lo escribían tres archivos).

Dónde guarda el servidor los datos. El respaldo por omisión es Git; el curso usa el **backend nativo**, que lee los archivos de una carpeta del sistema de archivos [56]. Es una decisión práctica: no exige un repositorio Git ni credenciales para arrancar. La documentación lo presenta como adecuado para empezar y para probar, y advierte que en producción hay que asegurarse de que el sistema de archivos sea fiable y esté compartido entre las instancias del servidor. El cambio a Git sería de configuración del servidor, no de los clientes.

Cómo encuentra un cliente su configuración. El servidor responde a `/{aplicación}/{perfil}`, y también a `/{aplicación}-{perfil}.yml`:

- `{aplicación}` es el `spring.application.name` del cliente (`quests`, `adventurers`, `treasury`, `gateway`);
- `{perfil}` son sus perfiles activos, o `default` si no hay ninguno.

Con un repositorio de archivos, el servidor monta la configuración de un cliente a partir de `application.yml` (compartido por todos) y de `<aplicación>.yml` (específico), y **el específico gana** [56]. Es el esquema que usarás:

`config-repo/: un archivo común y uno por cliente`

```
config-repo/
|-- application.yml    para todos: RabbitMQ y el contrato del evento
|-- quests.yml         para spring.application.name = quests
|-- adventurers.yml
|-- treasury.yml
`-- gateway.yml        puerto 8080 y las rutas
```

☞ El servidor de configuración

Otro proyecto Gradle nuevo, `config-server`. Necesita el mismo BOM de Spring Cloud y una dependencia, `spring-cloud-config-server` (nota: no se llama *starter*, aunque se use como uno):

`config-server/build.gradle`

```
plugins {
    id 'org.springframework.boot'
}

dependencyManagement {
    imports {
        mavenBom 'org.springframework.cloud:' +
            'spring-cloud-dependencies:2025.1.3'
    }
}

dependencies {
    implementation 'org.springframework.cloud:' +
        'spring-cloud-config-server'
}
```

Qué se verificó (9 de octubre de 2026). Con Boot 4.1.1, Initializr ofrece

`cloud-config-server` como `spring-cloud-config-server` y `cloud-config-client` como `spring-cloud-starter-config`, ambos con el BOM `2025.1.3`. En el *jar* `5.0.5` del servidor existen la anotación `org.springframework.cloud.config.server.EnableConfigServer` y la propiedad `spring.cloud.config.server.native.search-locations`. La documentación exige activar el perfil `native` y escribir el prefijo `file:` en las rutas de archivos. **Qué no se verificó:** que el servidor arranque, que la ruta relativa de abajo resuelva donde dice, y que no haga falta reiniciarlo tras editar un archivo.

Paso 1. Carpetas y `settings.gradle`. Gradle se niega a configurar un proyecto cuya carpeta no existe, así que primero las carpetas, desde la raíz:

```
$ mkdir -p config-repo
$ mkdir -p config-server/src/main/java/com/guildboard/configserver
$ mkdir -p config-server/src/main/resources
```

Después, `settings.gradle`: una línea al final (el bucle de los servicios no se toca). La carpeta se llama igual que el proyecto, así que no hace falta `projectDir`:

settings.gradle (solo el final)

```
include 'config-server'
```

Paso 2. La clase principal. La anotación `@EnableConfigServer` es todo el código del servidor:

config-server: ConfigServerApplication.java

```
package com.guildboard.configserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;

@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}
```

Paso 3. Su configuración. El servidor no es cliente de sí mismo: su propio `application.yml` sí vive en el *jar*. Activa el perfil `native` y apunta a la carpeta:

config-server: application.yml

```
spring:
  application:
    name: config-server
  profiles:
    active: native
  cloud:
```

```
    config:
      server:
        native:
          search-locations: file:../config-repo/

server:
  port: 8888
```

La ruta es relativa a la carpeta desde la que arranca el proceso. Con `./gradlew :config-server:bootRun`, Gradle ejecuta el proceso en la carpeta del proyecto (`config-server/`), de modo que `../config-repo/` es la de la raíz. (Es lo esperado; no lo he ejecutado. Si lo lanzas de otra forma, usa una ruta absoluta.)

Paso 4. Un primer archivo y la primera consulta. Todavía ningún servicio es cliente. Crea `config-repo/quests.yml` con el puerto que hoy está en el `application.yml` de Quests:

config-repo/quests.yml (primera versión)

```
server:
  port: 8081
```

Arranca el servidor y pregúntale, como lo haría Quests, por su configuración. La respuesta es un JSON en una sola línea; `grep -o` imprime solo los trozos que casan con el patrón (el nombre de cada fuente y el puerto):

```
$ ./gradlew :config-server:bootRun                    # terminal 1
$ curl -s localhost:8888/quests/default |
    grep -o -e '"name":"[^"]*"' -e '"server.port":[0-9]*'
```

Lo esperado: un `"name"` que acaba en `quests.yml` y, después, `"server.port":8081`. Prueba también `curl localhost:8888/quests-default.yml`, que devuelve la misma información ya como YAML. Dos trampas conocidas:

- **Un nombre mal escrito no da error.** Si pides `/queststs/default`, el servidor contesta sin quejarse y con lo único que encuentra, el `application.yml` común. Por omisión acepta aplicaciones que no existen (`spring.cloud.config.server.accept-empty` es `true` [56]). Si un servicio arranca con la configuración "a medias", empieza por mirar que `quests.yml` se llame como `spring.application.name`.
- **Lo que ya existía en el servidor no se filtra.** El propio `application.yml` del servidor no se envía a los clientes: según la documentación, las fuentes que tiene el servidor se quitan antes de responder.

13.6 Los servicios como clientes de configuración

Un servicio se convierte en cliente con tres cambios: una dependencia, una línea en su `application.yml` local y mover valores al `config-repo`. La línea es `spring.config.import`, el mecanismo de Spring Boot para importar más configuración, y en Spring Cloud Config es la forma actual de conectar con el servidor [56]:

Lo que queda en el application.yml local (fragmento)

```
spring:
  application:
    name: quests
  config:
    import: optional:configserver:http://localhost:8888
```

Dos cosas importan de esas cuatro líneas:

- `spring.application.name` **tiene que seguir en el archivo local**: es la clave con la que el cliente pide su configuración, y no puede venir de la configuración que todavía no ha pedido. Lo mismo vale para el propio `import`.
- **Los valores importados ganan.** Según la documentación de Spring Boot, los valores de lo importado tienen prioridad sobre el archivo que lo importó. Si una clave está a la vez en el `application.yml` local y en el `config-repo`, vale la del `config-repo`. Lo normal, mientras migras, es que no esté en los dos.

☞ El cliente de configuración

En cada `bootstrap` de servicio, el BOM de Spring Cloud y `spring-cloud-starter-config`. Con Quests como ejemplo, el `build.gradle` completo de `services/quests/bootstrap` (las líneas nuevas son el bloque `dependencyManagement` y la última dependencia):

services/quests/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencyManagement {
    imports {
        mavenBom 'org.springframework.cloud:' +
            'spring-cloud-dependencies:2025.1.3'
    }
}

dependencies {
    implementation project(':quests-adapters')
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'io.github.resilience4j:' +
        'resilience4j-spring-boot4:2.4.0'
    implementation 'org.springframework.cloud:' +
        'spring-cloud-starter-config'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

No hace falta ningún archivo `bootstrap.yml`: la documentación aclara que con `spring.config.import` no es necesario. El *starter* es el mismo que en el gateway; su nombre y su versión están verificados en el recuadro de la sección 13.4.

Para ver el cliente funcionando con un solo cambio, en Quests quita `server.port` de su `application.yml` (se queda en `quests.yml`) y añade el bloque `config.import` de arriba. Arranca el servidor y después Quests:

```
$ ./gradlew :config-server:bootRun                    # terminal 1
$ ./gradlew :quests-bootstrap:bootRun                 # terminal 2
$ curl -s -o /dev/null -w '%{http_code}\n' localhost:8081/quests/x
```

Si Quests escucha en el 8081 sin tener `server.port` en su archivo local, ese valor ha llegado del servidor. (La respuesta del `curl` será 404, porque la misión `x` no existe: lo que importa es que alguien conteste en ese puerto.)

13.6.1 Qué se mueve y qué no

La regla es una pregunta: ¿este valor cambia de un entorno a otro sin que cambie el comportamiento del negocio? Si sí, es configuración de despliegue y se puede mover.

Se mueve a `config-repo`	No se mueve
Puertos de los servicios.	El límite de 3 misiones activas (`MAX_ACTIVE_QUESTS`, regla R4).
Direcciones de otros servicios (`guildboard.adventurers.base-url`).	Los umbrales de rango (100 y 300 de experiencia).
Tiempos máximos e interruptor (capítulo 12).	Las exclusiones de autoconfiguración y `application-postgres.yml`: dicen qué adaptadores usa el código, y viven con él.
Conexión y credenciales de desarrollo de RabbitMQ; nombres del *exchange*, la clave y las colas.	Cualquier secreto de producción.

La columna de la derecha merece una explicación. El límite de tres misiones es una **regla de dominio** (capítulo 5): vive en `Quest`, tiene pruebas y se cambia con una revisión de código. Si estuviera en un YAML externo, cualquiera podría cambiar una regla del gremio sin pruebas ni revisión, y el dominio tendría que leer configuración, es decir, depender de Spring. Además el dominio no tiene por qué saber de dónde vienen los valores: `BeanConfig` sigue leyendo `guildboard.adventurers.base-url` con `@Value` (capítulo 9), y **no cambia ni una línea de Java** cuando ese valor pasa del archivo local al servidor. La composición es el único sitio que lee configuración, y por eso mover su origen no toca el núcleo.

13.6.2 Orden de arranque y lo que no hace esta configuración

- **Primero el Config Server.** Hay que arrancarlo antes que cualquier cliente. El `optional:` de la línea del `import` hace que, si el servidor no responde, el cliente arranque igualmente con lo que tiene en local, dejando solo un aviso en el registro. Es cómodo mientras migras servicio a servicio y peligroso después: un Quests que arranca antes que el servidor, ya sin `server.port` local, escucharía en el 8080 (el valor por omisión de Spring Boot, el del gateway) y, sin `base-url`, fallaría al construir `BeanConfig` porque no puede resolver el

marcador. Sin `optional:`, el cliente se niega a arrancar si no hay servidor; así falla pronto y a la vista [56]. El capítulo 15 usará esa forma.

- **La configuración se lee al arrancar.** Cambiar un archivo del `config-repo` no cambia a los servicios que ya están en marcha: hay que reiniciarlos. Recargar en caliente es posible con otras piezas de Spring Cloud y queda fuera del curso.
- **Los secretos en claro no son configuración segura.** Las credenciales `guildboard/guildboard` que hay en el `config-repo` son de desarrollo, y están en un archivo de texto. Config Server tiene otros *backends* (Git, Vault, y más) y formas de cifrar [56]; elegirlos es una decisión de producción que el curso no toma.
- **Hay propiedades que no pueden venir de fuera.** La documentación avisa de que `spring.main.*` no se puede poner en un repositorio remoto, porque se usa al iniciar la aplicación.

13.7 Crear el gateway

Ya tienes las piezas (rutas, el starter y un servidor de configuración que sirve una carpeta). Ahora se juntan en el proyecto `gateway`. El `build.gradle` es el de la sección 13.4; falta lo demás.

Paso 1. Carpetas y `settings.gradle`. Desde la raíz:

```
$ mkdir -p gateway/src/main/java/com/guildboard/gateway
$ mkdir -p gateway/src/main/resources
```

En `settings.gradle`, la línea del paso anterior pasa a incluir los dos proyectos:

settings.gradle (solo el final)

```
include 'config-server', 'gateway'
```

Paso 2. La clase principal y el `application.yml` local. Como en el Config Server, solo el *main*; y en el archivo local, el nombre y el `import`:

gateway: GatewayApplication.java

```
package com.guildboard.gateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class GatewayApplication {

    public static void main(String[] args) {
        SpringApplication.run(GatewayApplication.class, args);
    }
}
```

gateway: application.yml (el local)

```
spring:
  application:
    name: gateway
  config:
    import: optional:configserver:http://localhost:8888
```

Paso 3. Puerto y rutas, en el `config-repo`. Es el `gateway.yml` con las tres rutas de la tabla de la sección 13.4:

config-repo/gateway.yml

```
server:
  port: 8080

spring:
  cloud:
    gateway:
      server:
        webmvc:
          routes:
            - id: quests
              uri: http://localhost:8081
              predicates:
                - Path=/api/quests/**
              filters:
                - StripPrefix=1
            - id: adventurers
              uri: http://localhost:8082
              predicates:
                - Path=/api/adventurers/**
              filters:
                - StripPrefix=1
            - id: treasury
              uri: http://localhost:8083
              predicates:
                - Path=/api/treasury/**
              filters:
                - StripPrefix=2
```

Paso 4. Probarlo. Los servicios, por ahora, siguen como antes (Adventurers y Treasury con su puerto en su archivo local; Quests ya cliente). Con RabbitMQ y el servidor de configuración en marcha, arranca los tres servicios y, el último, el gateway, cada uno en su terminal:

```
$ docker compose up -d --wait rabbitmq
$ ./gradlew :config-server:bootRun           # terminal 1
$ ./gradlew :adventurers-bootstrap:bootRun   # terminal 2
$ ./gradlew :treasury-bootstrap:bootRun      # terminal 3
$ ./gradlew :quests-bootstrap:bootRun        # terminal 4
$ ./gradlew :gateway:bootRun                 # terminal 5
```

En una sexta, registra un aventurero por el 8080 y pídelo otra vez por la misma puerta. La función `jid` extrae el primer campo `"id"` de una respuesta JSON compacta (es lo esperado; no

lo he ejecutado):

```
$ jid() { grep -o '"id":"[^"]*"' | head -1 | cut -d'"' -f4; }
$ MARA=$(curl -s -X POST localhost:8080/api/adventurers \
    -H 'Content-Type: application/json' -d '{"name":"Mara"}' | jid)
$ curl -s -o /dev/null -w '%{http_code}\n' \
    localhost:8080/api/adventurers/$MARA
```

Lo esperado es 200. Las tres rutas están ya declaradas; el ejercicio 13.3 parte de este gateway.

13.8 En el proyecto: hito H7 (parte 1)

Hito 7, parte 1: una puerta y una fuente de configuración

Al terminar el ejercicio 13.5, GuildBoard tiene:

- **Un gateway** (`gateway`, puerto 8080) con tres rutas declaradas en `config-repo/gateway.yml`: `/api/quests/**` y `/api/adventurers/**` con `StripPrefix=1`, y `/api/treasury/**` con `StripPrefix=2`.
- **Un Config Server** (`config-server`, puerto 8888) con backend nativo sobre `config-repo/`.
- **Tres servicios que son clientes.** En el `application.yml` local de cada uno quedan el nombre, el `import` y las exclusiones de autoconfiguración. Puertos, direcciones, tiempos, interruptor, RabbitMQ y colas están en `config-repo`.
- **La estructura T7**: las tres carpetas raíz del árbol de la verificación 8 y nada nuevo dentro de `services/`, salvo configuración.

No cambia ninguna línea de Java, ni de `domain`, ni de `application`, ni de `adapters`. La otra mitad de H7, la seguridad, llega en el capítulo 14.

13.8.1 Probar el hito

El orden de arranque importa (sección 13.6): primero el servidor de configuración, y solo después todo lo demás. Cada comando, en su terminal y en modo memoria:

```
$ docker compose up -d --wait rabbitmq
$ ./gradlew :config-server:bootRun           # terminal 1
$ ./gradlew :adventurers-bootstrap:bootRun   # terminal 2
$ ./gradlew :treasury-bootstrap:bootRun      # terminal 3
$ ./gradlew :quests-bootstrap:bootRun        # terminal 4
$ ./gradlew :gateway:bootRun                 # terminal 5
```

En una sexta terminal, el recorrido de siempre, ahora todo por el 8080. La función `jid` extrae el primer campo `"id"` de una respuesta JSON compacta (con `grep -o` se queda con el trozo `"id":"..."` y `cut` con el valor):

```
$ jid() { grep -o '"id":"[^"]*"' | head -1 | cut -d'"' -f4; }
$ MARA=$(curl -s -X POST localhost:8080/api/adventurers \
    -H 'Content-Type: application/json' -d '{"name":"Mara"}' | jid)
```

```
$ Q1=$(curl -s -X POST localhost:8080/api/quests \
    -H 'Content-Type: application/json' \
    -d '{"title":"Limpiar el s^^c3^^b3tano","reward":150,
         "requiredRank":"NOVICE"}' | jid)
$ curl -s -X POST localhost:8080/api/quests/$Q1/accept \
    -H 'Content-Type: application/json' \
    -d "{\"adventurerId\":\"$MARA\"}"
$ curl -s -X POST localhost:8080/api/quests/$Q1/complete
```

Un instante después, el pago y la experiencia, también por el gateway (salidas *esperadas*; no he ejecutado el código):

```
$ curl -s localhost:8080/api/treasury/accounts/$MARA
$ curl -s localhost:8080/api/adventurers/$MARA
```

La primera respuesta es `{"adventurerId":"<id>","balance":150}`. La segunda, el aventurero con `"rank":"ADEPT"` y `"experience":150`. Dos pruebas más, las que enseñan para qué sirve cada pieza:

- **El servidor manda.** Pregúntale por Treasury y compara con lo que ve el propio servicio: `curl localhost:8888/treasury/default` debe mostrar `server.port = 8083`, y `treasury` no tiene ya ese puerto en su archivo local.
- **El prefijo se quita.** Las dos llamadas siguientes deben dar el mismo código, porque el gateway reenvía la primera como la segunda.

```
$ curl -s -o /dev/null -w '%{http_code}\n' \
    localhost:8080/api/treasury/accounts/$MARA
$ curl -s -o /dev/null -w '%{http_code}\n' \
    localhost:8083/accounts/$MARA
```

Y la comprobación de que la estructura es la que mostró el árbol de la verificación 8:

```
$ ./gradlew test
$ ./gradlew build
$ git status --short
$ git diff --stat -- 'services/*/domain' 'services/*/application' \
    'services/*/adapters'
```

El último comando no debe mostrar nada: ni `domain`, ni `application`, ni `adapters` de ningún servicio han cambiado. En `git status` deben aparecer solo los archivos de las carpetas nuevas, `settings.gradle` y, en cada servicio, el `build.gradle` y el `application.yml` de `bootstrap`.

13.9 Ejercicios

Los ejercicios parten del proyecto tal como lo dejó el capítulo 12, más lo que has hecho en las secciones 13.5, 13.6 y 13.7 (el Config Server, Quests como primer cliente con su puerto en `config-repo/quests.yml`, y el gateway con sus tres rutas). El primero se resuelve leyendo; el segundo mueve un valor; el tercero diagnostica una ruta; el cuarto revisa la estructura de la verificación 8; el último junta todo en el hito.

Ejercicio 13.1: Leer las rutas de un gateway ★★☆☆☆

Un equipo vecino tiene delante de sus servicios un gateway con estas tres rutas (la lista que cuelga de `spring.cloud.gateway.server.webmvc.routes`):

Rutas del equipo vecino

```
- id: tienda
  uri: http://localhost:9001
  predicates:
    - Path=/shop/**
  filters:
    - StripPrefix=1
- id: pagos
  uri: http://localhost:9002
  predicates:
    - Path=/v1/pay/**
  filters:
    - StripPrefix=2
- id: informes
  uri: http://localhost:9003
  predicates:
    - Path=/reports/**
```

Para cada una de estas siete peticiones al gateway, di si alguna ruta la recibe, a qué puerto va y con qué ruta llega al servicio. No arranques nada:

1. `GET /shop/items/7`
2. `POST /v1/pay/invoices/3`
3. `GET /reports/2026/q1`
4. `DELETE /reports/2026`
5. `GET /pay/invoices/3`
6. `GET /v1/shop/items/7`
7. `GET /shop/v1/pay/x`

Está bien si: contestas las siete con la forma «puerto y ruta» o «ninguna ruta»; sabes decir por qué las peticiones 5 y 6 no llegan a ningún servicio; y distingues la ruta `informes`, que no tiene filtros, de las otras dos, que quitan segmentos.

Lectura de código ▪ solución en la pág. 505

Ejercicio 13.2: Mover la dirección de Adventurers al Config Server ★★★☆☆

Parte de Quests como cliente (sección 13.6): su `server.port` ya está en `config-repo/quests.yml` y el servidor de configuración funciona. Ahora mueves la dirección de Adventurers:

1. En el `application.yml` local de Quests, quita `guildboard.adventurers.base-url`. Deja los dos tiempos máximos donde están.
2. Añade la misma clave, con el mismo valor, a `config-repo/quests.yml`.
3. Pregunta al servidor por `/quests/default` y localiza la clave en la respuesta.
4. Con RabbitMQ, el servidor, Adventurers y Quests en marcha, registra a un aventurero,

publica una misión y acéptala llamando directamente a Quests (`localhost:8081`).

5. Para Quests, cambia en `config-repo/quests.yml` el valor a `http://localhost:9999` (un puerto donde no escucha nadie), reinícialo y repite la aceptación. Quests guarda en memoria: tras reiniciarlo, publica una misión nueva antes de cada aceptación. Devuelve el valor a `http://localhost:8082`, reinicia y comprueba que vuelve a aceptar.

6. Contesta en dos frases: ¿por qué no moverías también el límite de tres misiones activas?

Está bien si: el `application.yml` local de Quests ya no contiene `base-url`; la respuesta del servidor para `quests/default` muestra `guildboard.adventurers.base-url` y `server.port`; el paso 4 acepta la misión (200); el paso 5 devuelve `503` con `code ADVENTURERS_UNAVAILABLE` sin que se haya modificado ni recompilado ningún archivo Java; y `git status` muestra solo archivos `.yml`.

Pista: *el servidor lee los archivos al contestar, pero Quests lee el valor solo al arrancar: tras editar `config-repo/quests.yml` hay que reiniciar Quests.*

Práctica ▪ solución en la pág. 505

Ejercicio 13.3: Un 404 detrás del gateway ★★★☆☆

Parte del gateway de la sección 13.7, funcionando, con Adventurers, Quests y Treasury en marcha (por su puerto, como hasta ahora) y Mara con una misión completada, es decir, 150 de oro en Treasury. Alguien cambió la ruta de Treasury en `config-repo/gateway.yml` (sustituye la tuya por esta y reinicia el gateway; las otras dos rutas son las del capítulo y funcionan):

`config-repo/gateway.yml (solo la ruta de Treasury)`

```
            - id: treasury
              uri: http://localhost:8083
              predicates:
                - Path=/api/treasury/**
              filters:
                - StripPrefix=1
```

Mara ya tiene una cuenta con 150 de oro. Esto es lo que se ve:

```
$ curl -s -o /dev/null -w '%{http_code}\n' \
    localhost:8083/accounts/$MARA
200
$ curl -s -o /dev/null -w '%{http_code}\n' \
    localhost:8080/api/treasury/accounts/$MARA
404
```

Haz lo siguiente:

1. Di qué ruta recibe Treasury cuando la petición pasa por esta ruta, y por qué eso da 404 si el servicio funciona.

2. Compruébalo sin tocar el gateway: llama directamente a Treasury (8083) con esa ruta y mira el código.

3. Corrige la ruta y haz que el gateway use el cambio.

4. Explica por qué no arreglarías esto cambiando las rutas del controlador de Treasury.

Está bien si: la causa que das es que se quita un solo segmento (`api`) y el servicio recibe `/treasury/accounts/...`; el paso 2 devuelve 404 con la ruta que dedujiste; tras el arreglo, la llamada por el gateway devuelve 200 con `balance` 150; el único archivo modificado es `config-repo/gateway.yml`; y no has tocado ningún archivo de `services/treasury`.

Pista: *el gateway lee sus rutas al arrancar, así que hay que reiniciarlo después de editar* `gateway.yml`*.*

Depuración ▪ solución en la pág. 507

Ejercicio 13.4: Revisar la estructura de la verificación 8 ★★★★☆

Haz la revisión de la verificación de estructura 8 como si no hubieras escrito este capítulo. Entrega una página con:

1. La lista de comprobación de la sección 13.3 aplicada a `gateway` y a `config-server`, con "sí", "no" o "parcial" y una frase por punto.
2. El argumento por el que ninguno de los dos lleva `domain` ni `application`, en cuatro frases como máximo.
3. Al menos tres **señales** distintas (cosas que se verían en el código o en los requisitos) que obligarían a darle hexágono a uno de los dos, cada una con un ejemplo de GuildBoard.
4. Para una de esas señales, el árbol resultante: qué carpetas y módulos Gradle aparecerían y si el proyecto seguiría en la raíz.
5. Tu decisión sobre la ubicación de `gateway/`, `config-server/` y `config-repo/`: la aceptas o propones otra, y qué se rompería con tu alternativa.

Criterios de valoración (no hay una única respuesta correcta): (1) la comprobación distingue lo que no se aplica de lo que se cumple; (2) el argumento se apoya en la pregunta «¿hay un núcleo que proteger?» y no en «son proyectos pequeños»; (3) cada señal es distinta de las demás, observable y con un ejemplo que implique una regla del gremio, no solo más código; (4) el árbol del punto 4 respeta el molde del capítulo 10 y la regla de dependencia, y dice qué hacen `ArchitectureTest` y `StructureTest` con él; (5) la ubicación se defiende con una consecuencia concreta de moverla.

Diseño ▪ solución en la pág. 508

Ejercicio 13.5: Hito H7a: gateway, configuración centralizada y acceso por el 8080 ★★★★☆

Parte del proyecto con los ejercicios anteriores hechos: Config Server funcionando, Quests como cliente con el puerto y la dirección de Adventurers en `config-repo/quests.yml`, y el gateway de la sección 13.7 (`gateway/`, su `include` y `config-repo/gateway.yml` con las tres rutas correctas). Falta lo que lo convierte en hito:

1. **El gateway, comprobado.** Verifica que `gateway.yml` tiene `StripPrefix` 1, 1 y 2 (si en el ejercicio 13.3 lo dejaste bien, no cambia) y que `settings.gradle` incluye `gateway`.
2. **La configuración común.** `config-repo/application.yml` con la conexión a RabbitMQ y el *exchange* y la clave del evento.
3. **Adventurers y Treasury, clientes.** En cada `bootstrap`: el BOM y el cliente en el `build.gradle`, y un `application.yml` local que se queda con nombre, `import` y exclusiones. Su puerto, los reintentos del *listener* y los nombres de cola y de cola de mensajes muertos van a `adventurers.yml` y `treasury.yml`; la conexión a RabbitMQ

y el *exchange* salen de su archivo local, porque ya están en el común.

4. **El resto de Quests.** Los tiempos máximos y el bloque `resilience4j` van a `quests.yml`; la conexión a RabbitMQ y el *exchange*, que ya están en el común, salen de su archivo local. Ese archivo queda con nombre, `import` y exclusiones.

5. Arranca todo en el orden de la sección 13.8.1, repite el recorrido por el 8080 y haz el *commit*.

Está bien si: `./gradlew build` termina con `BUILD SUCCESSFUL` (incluidas las pruebas de arquitectura); el recorrido de la sección 13.8.1 funciona por el 8080, con 150 de saldo para Mara al final; para cada uno de los cuatro `spring.application.name`, `curl localhost:8888/<nombre>/default` devuelve sus claves; el `application.yml` local de cada servicio tiene solo `name`, `import` y exclusiones, y el del gateway solo `name` e `import`; el `git diff` del final de la sección 13.8.1 sobre `domain`, `application` y `adapters` no muestra nada; `MAX_ACTIVE_QUESTS` sigue en `Quest`; y ningún `build.gradle` nombra a un proyecto de otro servicio.

Pista: *si un servicio escucha en el 8080 o no arranca por un marcador sin resolver, arrancó antes que el servidor de configuración.*

Proyecto ▪ solución en la pág. 510

13.10 Resumen

Resumen

- Un API gateway da una sola entrada a los servicios y aísla a los clientes de cómo está partido el sistema; a cambio es una pieza más, un salto de red más y un punto único de fallo. No es un cortafuegos y no debe acumular lógica de negocio.
- Una ruta tiene `id`, `uri`, predicados y filtros. `Path=/api/quests/**` casa con cero o más segmentos tras el prefijo; `StripPrefix=n` quita `n` segmentos al principio. Los servicios conservan sus rutas internas: Quests y Adventurers quitan uno, Treasury quita dos.
- Spring Cloud se instala con su BOM (`2025.1.3`, compatible con Boot 4.1) y artefactos propios: `spring-cloud-starter-gateway-server-webmvc`, `spring-cloud-config-server` y `spring-cloud-starter-config`. No pude ejecutar nada: lo verificado fueron nombres, versiones y propiedades.
- Config Server con backend nativo sirve una carpeta (`config-repo`) por HTTP: `application.yml` para todos y `<aplicación>.yml` para cada cliente, que gana al común. Un nombre de aplicación mal escrito no falla: devuelve solo lo común.
- Un cliente lleva en su archivo local el nombre, la línea `spring.config.import` y poco más. Lo importado gana al archivo local. La configuración se lee al arrancar. Con `optional:` el cliente arranca aunque no haya servidor, con valores equivocados; sin él, falla pronto.
- Se mueve lo que cambia entre entornos sin cambiar el negocio (puertos, direcciones, tiempos, conexión al *broker*); no se mueve una regla de dominio como el límite de tres misiones, ni los secretos reales. El código no cambia porque la composición ya leía los valores con `@Value`.
- `gateway` y `config-server` no tienen hexágono porque no hay núcleo que proteger: es un límite parcial. Esa respuesta cambia en cuanto uno de los dos tenga que decidir algo con significado de negocio.

- La estructura cambió (verificación de estructura 8): tres carpetas en la raíz y dos archivos de configuración por servicio. `domain`, `application` y `adapters` no se tocaron.

Soluciones de los ejercicios

Solución del ejercicio 13.1

enunciado en la pág. 501

Se evalúa cada petición contra los patrones de `Path` *tal como llega al gateway*; solo después, con la ruta ya elegida, se aplican los filtros:

Petición	Ruta	Destino
GET /shop/items/7	tienda	puerto 9001, ruta `/items/7`
POST /v1/pay/invoices/3	pagos	puerto 9002, ruta `/invoices/3`
GET /reports/2026/q1	informes	puerto 9003, ruta `/reports/2026/q1` (sin filtro, no cambia)
DELETE /reports/2026	informes	puerto 9003, ruta `/reports/2026` (la ruta no limita el método)
GET /pay/invoices/3	ninguna	no llega a ningún servicio
GET /v1/shop/items/7	ninguna	no llega a ningún servicio
GET /shop/v1/pay/x	tienda	puerto 9001, ruta `/v1/pay/x`

Por qué. Los patrones de `Path` se comparan desde el principio de la ruta. La petición 5 le falta el `/v1` que exige `pagos`; la 6 trae un `/v1` delante que `tienda` no espera. Ninguna ruta las casa y, por tanto, ningún servicio las recibe (lo esperado es que conteste el propio gateway con un 404). La petición 7 es la trampa: parece de `pagos` por su final, pero empieza por `/shop`, así que la elige `tienda`, que quita un solo segmento (`shop`) y reenvía lo demás tal cual, `/v1/pay/x`. La ruta se elige antes de quitar nada.
Errores típicos: aplicar los filtros antes de elegir la ruta (como si `StripPrefix` pudiera cambiar qué ruta casa); olvidar que `informes` no tiene `filters` y no toca la ruta; y contar mal los segmentos de `pagos`: `/v1/pay/invoices/3` tiene cuatro, se quitan los dos primeros y quedan `invoices/3`.

Solución del ejercicio 13.2

enunciado en la pág. 501

1 y 2. Mover la clave. El bloque `guildboard` del `application.yml` local de Quests pierde la dirección:

quests-bootstrap: application.yml (bloque guildboard)

```
guildboard:
  adventurers:
    connect-timeout: 1s
    read-timeout: 1s
  messaging:
    exchange: guildboard.events
    routing-key: quest.completed
```

Y `config-repo/quests.yml` la gana:

config-repo/quests.yml

```
server:
  port: 8081

guildboard:
  adventurers:
    base-url: http://localhost:8082
```

Las dos fuentes se mezclan clave a clave: `connect-timeout` y `read-timeout` siguen viniendo del archivo local, y `base-url` y el puerto, del servidor.

3. Preguntar al servidor.

```
$ curl -s localhost:8888/quests/default |
    grep -o -e '"server.port":[0-9]*' \
            -e '"guildboard.adventurers.base-url":"[^"]*"'
"server.port":8081
"guildboard.adventurers.base-url":"http://localhost:8082"
```

(El orden de las dos líneas puede variar.)

4. La aceptación directa. Con RabbitMQ, el servidor, Adventurers y Quests en marcha:

```
$ jid() { grep -o '"id":"[^"]*"' | head -1 | cut -d'"' -f4; }
$ MARA=$(curl -s -X POST localhost:8082/adventurers \
    -H 'Content-Type: application/json' -d '{"name":"Mara"}' | jid)
$ Q1=$(curl -s -X POST localhost:8081/quests \
    -H 'Content-Type: application/json' \
    -d '{"title":"Ratas","reward":50,"requiredRank":"NOVICE"}' | jid)
$ curl -s -o /dev/null -w '%{http_code}\n' \
    -X POST localhost:8081/quests/$Q1/accept \
    -H 'Content-Type: application/json' \
    -d "{\"adventurerId\":\"$MARA\"}"
200
```

5. Cambiar el valor sin tocar el código. Edita `config-repo/quests.yml` (`base-url: http://localhost:9999`), detén Quests y arráncalo otra vez. Como guarda en memoria, la misión `Q1` ya no existe: publica una nueva y acéptala.

```
$ Q2=$(curl -s -X POST localhost:8081/quests \
    -H 'Content-Type: application/json' \
    -d '{"title":"Ratas","reward":50,"requiredRank":"NOVICE"}' | jid)
$ curl -s -X POST localhost:8081/quests/$Q2/accept \
    -H 'Content-Type: application/json' \
    -d "{\"adventurerId\":\"$MARA\"}"
```

La respuesta es un `503` con `ProblemDetail` y `"code":"ADVENTURERS_UNAVAILABLE"` (el comportamiento del capítulo 12 ante una conexión rechazada; la primera respuesta tarda unos 0,6 s por los reintentos). Devuelve el valor a `http://localhost:8082`, reinicia Quests, publica otra misión y acéptala: vuelve el `200`. No se recompiló nada: solo cambió un archivo `.yml` de fuera de los *jar*.

6. El límite de tres misiones. Es una regla de dominio: tiene pruebas y se cambia con una revisión; si estuviera en un YAML externo, cualquiera podría cambiar una regla del gremio sin pruebas, y `domain` tendría que leer configuración, es decir, depender de Spring. **Errores típicos:** editar `quests.yml` y no reiniciar Quests (el valor se lee al arrancar); escribir `quest.yml` en lugar de `quests.yml` (el servidor no se queja y el cliente se queda sin su archivo); dejar la clave también en el archivo local (la del servidor gana y no entenderás por qué un cambio local no hace nada); y tras reiniciar Quests intentar aceptar la misión de antes (en memoria ya no existe y da 404, no 503).

Solución del ejercicio 13.3 enunciado en la pág. 502

1. Qué recibe Treasury. `StripPrefix=1` quita un segmento. La petición `/api/treasury/accounts/<id>` llega a Treasury como `/treasury/accounts/<id>`. Treasury solo tiene `/accounts/...`, así que contesta 404: el servicio funciona, la ruta que recibe no existe.

2. Comprobarlo sin el gateway. Pidiendo a Treasury la ruta que le llega:

```
$ curl -s -o /dev/null -w '%{http_code}\n' \
    localhost:8083/treasury/accounts/$MARA
404
```

Es el mismo código que vio el gateway, y el 200 de la llamada directa a `/accounts/$MARA` descarta que la cuenta no exista. Esa comparación ("la ruta buena da 200, la que cree el gateway da 404") es la técnica general: reproduce a mano, contra el servicio, la ruta que debería estar recibiendo.

3. La corrección. Hay que quitar dos segmentos, `api` y `treasury`. El archivo completo, con las tres rutas del capítulo:

config-repo/gateway.yml

```yaml
server:
  port: 8080

spring:
  cloud:
    gateway:
      server:
        webmvc:
          routes:
            - id: quests
              uri: http://localhost:8081
              predicates:
                - Path=/api/quests/**
              filters:
                - StripPrefix=1
            - id: adventurers
              uri: http://localhost:8082
              predicates:
                - Path=/api/adventurers/**
              filters:
                - StripPrefix=1
            - id: treasury
              uri: http://localhost:8083
              predicates:
                - Path=/api/treasury/**
              filters:
                - StripPrefix=2
```

Después, reinicia el gateway (lee sus rutas al arrancar): `curl localhost:8080/api/treasury/accounts/$MARA` devuelve 200 con `"balance":150`.

4. Por qué no cambiar Treasury. Porque el defecto está en la ruta del gateway, que es quien introduce el prefijo público. Si Treasury aceptara `/treasury/accounts`, un servicio sabría de la existencia del gateway, tendría dos conjuntos de rutas (las que ve el resto del sistema y las del gateway) y dejaría de servir igual si el prefijo público cambia. La dirección de la dependencia sería la contraria a la de todo el curso: el borde conoce al núcleo, no al revés.

Errores típicos: editar una copia del archivo (por ejemplo, un `gateway.yml` dentro de `gateway/src/main/resources`) en lugar del de `config-repo`; olvidar reiniciar; ponerse a modificar rutas en Treasury; o pasarse a `StripPrefix=3`, que deja `/<id>` y sigue dando 404.

Solución del ejercicio 13.4

enunciado en la pág. 503

Solución de referencia.

1. La lista aplicada.

Comprobación	gateway	config-server
1. Dominio sin frameworks	No se aplica: no hay dominio.	No se aplica.
2. Aplicación solo depende del dominio	No se aplica.	No se aplica.
3. Adaptadores en `in/out`	Parcial: es un adaptador entero, aportado por la biblioteca.	Parcial: igual, sirve archivos por HTTP.
4. Nombres que dicen qué hace	Parcial: dice el rol, no el negocio.	Parcial: igual.
5. Las carpetas coinciden con el árbol	Sí: carpeta nueva y una línea en `settings.gradle`.	Sí: igual, más `config-repo/`.
6. `build.gradle` que nombra otro servicio	Cumple: ninguno.	Cumple: ninguno.
7. Lógica de negocio en la raíz	Cumple: ninguna.	Cumple: ninguna.

2. El argumento. El hexágono protege reglas de negocio de HTTP, la base de datos y el *broker*. Ni el gateway ni el Config Server tienen reglas: uno traduce rutas con una tabla y el otro sirve archivos. Sin núcleo no hay nada que proteger, y el molde de cuatro módulos sería código que no defiende nada. Es un límite parcial en el sentido de Martin [28], llevado al extremo en que no hay límite que construir.
3. Señales (cada una, con su ejemplo):

1. **Una regla del glosario en el gateway.** "Los `NOVICE` no pueden aceptar más de cinco misiones por minuto" mezcla un concepto del dominio (`Rank`) con estado (un contador): hay una invariante que probar y que cambiará.
2. **Composición con criterio.** Un `GET /api/board` que junta misiones abiertas (Quests), el rango (Adventurers) y el saldo (Treasury) y decide qué mostrar según el rango. Pasa de enrutar a decidir.
3. **Una decisión que quieres probar sin Spring.** Si necesitas una prueba JUnit que cree una clase con ramas solo para el gateway, esa clase es un núcleo en potencia.
4. **Política en el Config Server.** Que decida qué servicio ve qué claves según un permiso del gremio, o que lea de una base de datos propia, añade reglas (quién ve qué) y un adaptador de salida.

4. El árbol, para la señal 1 si el gateway adopta el molde. Pasa de infraestructura a servicio, así que se mueve a `services/` y `StructureTest` lo exige entero y `ArchitectureTest` vigilaría su núcleo:

Árbol si el gateway adopta el molde

```
services/gateway/
|-- domain/       RateRule, Rank [copia]; tests
|-- application/  port/in  CheckQuotaUseCase
|                 port/out QuotaStore
|-- adapters/     adapter/in/web   filtro del gateway que llama al caso
|                 adapter/out/memory  QuotaStore en memoria
`-- bootstrap/    GatewayApplication, config/, application.yml
settings.gradle   'gateway' entra en la lista del bucle de servicios
```

Desaparece el proyecto `gateway` de la raíz, y las rutas siguen en `config-repo/gateway.yml`.

La alternativa que suele ser mejor es no dárselo: esa regla pertenece a quien ya tiene núcleo (aceptar misiones es de Quests) y el gateway sigue siendo una tabla.
5. La ubicación. Se acepta. Las alternativas: `services/gateway` haría fallar `StructureTest` (exige los cuatro módulos a todo lo que cuelga de `services/`); `infra/` mezclaría proyectos con código con archivos de apoyo; meter `config-repo` dentro de `config-server` mezclaría datos y código, y el día de usar Git el cambio de un puerto se vería como un cambio del proyecto.
Criterios de valoración. (1) La tabla distingue lo que no se aplica de lo que se cumple. (2) El argumento usa "núcleo que proteger". (3) Hay al menos tres señales distintas, observables y con una regla del gremio. (4) El árbol respeta el molde y la regla de dependencia, y se dice qué comprobarían `ArchitectureTest` y `StructureTest`. (5) La ubicación se defiende con una consecuencia concreta.
Alternativa válida. Dejar el gateway en la raíz como proyecto de un solo módulo y, al aparecer la primera regla, crear solo un paquete `rules` con la clase sin dependencias de Spring, y sacar el molde completo cuando haya una segunda regla o un segundo adaptador. Es el "no dar el último paso" de los límites parciales: más barato hoy, a costa de que nada del compilador impida mezclar la regla con el filtro.

Solución del ejercicio 13.5

enunciado en la pág. 503

1. Punto de partida. Los proyectos `config-server` y `gateway` ya existen (secciones 13.5 y 13.7); no hay carpetas que crear. Lo que falta son los clientes y la configuración común.
2. `settings.gradle` completo.

settings.gradle

```
rootProject.name = 'guildboard'

// Un proyecto Gradle por m^^c3^^b3dulo y servicio, con nombre ^^c3^^banico
// (quests-domain, adventurers-domain, treasury-domain...).
['quests', 'adventurers', 'treasury'].each { service ->
    ['domain', 'application', 'adapters', 'bootstrap'].each { module ->
        def projectName = "${service}-${module}"
        include projectName
        project(":${projectName}").projectDir =
            file("services/${service}/${module}")
    }
}

// Infraestructura sin hex^^c3^^a1gono: la carpeta se llama como el
    ↪ proyecto.
include 'config-server', 'gateway'
```

3. El gateway ya existe. Lo creaste en la sección 13.7: `gateway/build.gradle`, `GatewayApplication`, su `application.yml` local (solo nombre e `import`) y `config-repo/gateway.yml` con el puerto 8080 y las tres rutas (`StripPrefix` 1, 1 y 2). Si en el ejercicio 13.3 dejaste la ruta de Treasury con `StripPrefix=2`, no hay nada que añadir: comprueba solo que `gateway.yml` tiene esa forma y que `git status` lo lista como archivo nuevo. **4. El `config-repo` común y el de cada servicio.** El gateway también recibe el archivo común, que habla de RabbitMQ y que el gateway ignora porque no tiene cliente de RabbitMQ.

config-repo/application.yml

```yaml
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: guildboard
    password: guildboard

guildboard:
  messaging:
    exchange: guildboard.events
    routing-key: quest.completed
```

config-repo/quests.yml

```yaml
server:
  port: 8081

guildboard:
  adventurers:
    base-url: http://localhost:8082
    connect-timeout: 1s
    read-timeout: 1s

resilience4j:
  circuitbreaker:
    instances:
      adventurers:
        sliding-window-size: 10
        minimum-number-of-calls: 5
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 2
        record-exceptions:
          - org.springframework.web.client.ResourceAccessException
          - org.springframework.web.client.HttpServerErrorException
  retry:
    instances:
      adventurers:
        max-attempts: 3
        wait-duration: 200ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        retry-exceptions:
          - org.springframework.web.client.ResourceAccessException
          - org.springframework.web.client.HttpServerErrorException
```

config-repo/adventurers.yml

```yaml
server:
  port: 8082

spring:
  rabbitmq:
    listener:
      simple:
        retry:
          enabled: true
          max-retries: 3
          initial-interval: 1s
          multiplier: 2
          max-interval: 10s

guildboard:
  messaging:
    queue: adventurers.quest-completed
    dead-letter-exchange: guildboard.events.dlx
    dead-letter-queue: adventurers.quest-completed.dlq
```

config-repo/treasury.yml

```yaml
server:
  port: 8083

spring:
  rabbitmq:
    listener:
      simple:
        retry:
          enabled: true
          max-retries: 3
          initial-interval: 1s
          multiplier: 2
          max-interval: 10s

guildboard:
  messaging:
    queue: treasury.quest-completed
    dead-letter-exchange: guildboard.events.dlx
    dead-letter-queue: treasury.quest-completed.dlq
```

5. Los tres `build.gradle` de `bootstrap`. Cada uno es el de antes más el BOM y el cliente:

services/quests/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencyManagement {
    imports {
        mavenBom 'org.springframework.cloud:' +
            'spring-cloud-dependencies:2025.1.3'
    }
}

dependencies {
    implementation project(':quests-adapters')
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'io.github.resilience4j:' +
        'resilience4j-spring-boot4:2.4.0'
    implementation 'org.springframework.cloud:' +
        'spring-cloud-starter-config'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

services/adventurers/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencyManagement {
    imports {
        mavenBom 'org.springframework.cloud:' +
            'spring-cloud-dependencies:2025.1.3'
    }
}

dependencies {
    implementation project(':adventurers-adapters')
    implementation project(':adventurers-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'org.springframework.cloud:' +
        'spring-cloud-starter-config'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

services/treasury/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencyManagement {
    imports {
        mavenBom 'org.springframework.cloud:' +
            'spring-cloud-dependencies:2025.1.3'
    }
}

dependencies {
    implementation project(':treasury-adapters')
    implementation project(':treasury-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'org.springframework.cloud:' +
        'spring-cloud-starter-config'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

6. Los tres `application.yml` locales. Idénticos salvo el nombre. Se quedan con tres cosas: el nombre, el `import` y las exclusiones, que dicen qué adaptadores usa el código (el perfil `postgres` sigue en su archivo local).

quests-bootstrap: application.yml

```
spring:
  application:
    name: quests
  config:
    import: optional:configserver:http://localhost:8888
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.
    ↪ DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.
    ↪ HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.
    ↪ DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.
    ↪ FlywayAutoConfiguration
```

adventurers-bootstrap: application.yml

```yaml
spring:
  application:
    name: adventurers
  config:
    import: optional:configserver:http://localhost:8888
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration
```

treasury-bootstrap: application.yml

```yaml
spring:
  application:
    name: treasury
  config:
    import: optional:configserver:http://localhost:8888
  autoconfigure:
    exclude:
      - org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration
      - org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
      - org.springframework.boot.data.jpa.autoconfigure.DataJpaRepositoriesAutoConfiguration
      - org.springframework.boot.flyway.autoconfigure.FlywayAutoConfiguration
```

7. Comprobar. Qué claves sirve el servidor a cada uno de los cuatro nombres:

```
$ for app in quests adventurers treasury gateway; do
>   echo "== $app"
>   curl -s localhost:8888/$app/default |
>     grep -o '"[a-zA-Z0-9_-]*\.[^"]*":' | sort -u
> done
```

(El `grep` muestra solo las claves con punto, como `server.port`.) Después, el orden de la sección 13.8.1 (servidor primero, el gateway el último), el recorrido por el 8080, `./gradlew build`, y el *commit* (`H7a: gateway y Config Server`).

Por qué funciona. Cada cliente pide al servidor `/<nombre>/default`, y recibe el `application.yml` común más su archivo (que gana). Las claves siguen llamándose igual, así que `BeanConfig`, `MessagingConfig` y los `@RabbitListener` de las colas (que leen `${guildboard.messaging.queue}`) las resuelven como antes y **no cambia ninguna línea de Java**. El gateway lee sus rutas del mismo sitio.

Errores típicos: olvidar `include 'gateway'` (`:gateway:bootRun` no existe); arrancar un servicio antes que el servidor (por el `optional:`, arranca sin su puerto, con el 8080, o falla por un marcador sin resolver); dejar una clave en el archivo local y en el `config-repo` (gana la remota); poner `adventurers.yml` con otro nombre; cambiar el `config-repo` y no reiniciar el cliente; y escribir mal la sangría de `routes`, que deja al gateway sin rutas sin error en el arranque.

CAPÍTULO 14

Seguridad: JWT validado en cada servicio

Objetivos del capítulo

- Leer un JWT (cabecera, claims y firma), decodificarlo con la shell y distinguir decodificar de validar.
- Configurar cada servicio como *resource server* que valida firma, caducidad y emisor del token, y convertir la claim `roles` en autoridades para autorizar por método y ruta.
- Mantener la autorización en el borde: el adaptador convierte el `sub` del token en un `AdventurerId` del comando y ni `domain` ni `application` conocen JWT.
- Reenviar el token de Quests a Adventurers desde el adaptador de salida, y situar en el árbol de la verificación de estructura 9 la herramienta `tools/token-issuer` y el paquete de seguridad.

Desde el capítulo anterior, quien llame al puerto 8081, 8082 u 8083 puede hacer lo que quiera: publicar misiones, registrar aventureros, leer cuentas. El gateway no lo impide ni lo pretendía (el capítulo 13 ya avisó de que no es un cortafuegos). Este capítulo cierra esa puerta de la forma que pide el briefing: **cada servicio exige un token y lo valida él mismo**, sin fiarse de quien lo llama. Además, el token dice quién es la persona, y eso permite una regla que el dominio no puede comprobar solo: un aventurero solo acepta misiones *a su nombre*.

Cuatro cosas que el capítulo deja dichas desde el principio:

- **Hay una herramienta nueva y una dependencia nueva.** La herramienta es un emisor de tokens de desarrollo (`tools/token-issuer`, sección 14.3), que se da escrito entero: es un andamio para poder probar, no el objeto de estudio, y **no es seguridad de producción**. La dependencia es el *starter* de resource server de Spring Boot (sección 14.4).
- **La estructura cambia.** Entra una carpeta nueva en la raíz y un paquete de seguridad en cada servicio. La sección 14.1 muestra el árbol antes de crear nada.
- **El núcleo no se entera.** `domain` y `application` no importan nada de seguridad. Lo comprueban las pruebas de arquitectura con ArchUnit (`./gradlew test`), que ya existen desde el capítulo 3 y no cambian en este capítulo.
- **El código Java no se ha compilado ni ejecutado.** Como en todo el curso. Lo que sí se comprobó (nombres de artefactos, versiones, firmas de las clases, propiedades, documentación) está marcado con su fecha; el resto es lo esperado. Las pruebas de autorización se hacen con `curl`, no con pruebas automáticas.

14.1 Verificación de estructura 9: el emisor y el paquete de seguridad

Nota

Este capítulo cambia la estructura de carpetas. Comprueba que tu proyecto coincide con el árbol de abajo antes de seguir.

14.1.1 El árbol anterior

Así quedó el proyecto tras el capítulo 13 (árbol T7):

Árbol anterior (T7, tras el capítulo 13)

```
guildboard/
|-- .git/, .gitignore, gradlew, gradlew.bat
|-- settings.gradle        catorce proyectos (doce de servicios,
|                          gateway y config-server)
|-- build.gradle           configuraci^^c3^^b3n com^^c3^^ban (subprojects)
|-- compose.yaml           PostgreSQL 18 y RabbitMQ 4
|-- docs/dominio.md
|-- gradle/wrapper/
|-- infra/postgres/init.sql
|-- gateway/               puerto 8080
|-- config-server/         puerto 8888
|-- config-repo/           application.yml, quests.yml, adventurers.yml,
|                          treasury.yml, gateway.yml
`-- services/
    |-- quests/            domain/ application/ adapters/ bootstrap/
    |-- adventurers/       domain/ application/ adapters/ bootstrap/
    `-- treasury/          domain/ application/ adapters/ bootstrap/
```

14.1.2 El árbol nuevo

`[N]` es nuevo y `[M]` modificado. Primero la raíz:

Árbol nuevo, raíz (T8, tras el capítulo 14)

```
guildboard/
|-- settings.gradle        [M] + proyecto 'token-issuer' (con projectDir)
|-- build.gradle, compose.yaml, docs/, gradle/, infra/   sin cambios
|-- gateway/, config-server/                              sin cambios
|-- config-repo/
|   `-- application.yml    [M] + issuer-uri y jwk-set-uri (el resto
|                          de archivos, sin cambios)
|-- tools/                 [N] herramientas, no servicios
|   `-- token-issuer/      [N] proyecto 'token-issuer', puerto 9000
|       |-- build.gradle
|       `-- src/main/
|           |-- java/com/guildboard/tokenissuer/
|           |   |-- TokenIssuerApplication.java
|           |   |-- IssuerProperties.java
|           |   |-- KeyConfig.java
|           |   |-- TokenController.java
```

```
|           |   `-- JwksController.java
|           `-- resources/application.yml
`-- services/              (ver abajo)
```

Y lo que cambia dentro de cada servicio (el ejemplo es Quests; Adventurers y Treasury llevan lo mismo salvo las marcas de Quests):

services/quests (solo lo que cambia)

```
services/quests/
|-- domain/, application/                       sin cambios
|-- adapters/
|   |-- build.gradle                            [M] + starter de
|   |                                           resource server
|   `-- src/main/java/com/guildboard/quests/
|       |-- adapter/in/web/
|       |   |-- security/SecurityConfig.java    [N]
|       |   |-- QuestController.java            [M] solo en Quests
|       |   `-- dto/AcceptQuestRequest.java     [M] solo en Quests
|       `-- adapter/out/rest/                   solo en Quests
|           |-- BearerTokenForwardingInterceptor.java   [N]
|           `-- AdventurerClientFactory.java    [M]
`-- bootstrap/
    |-- build.gradle                            [M] + el mismo starter
    `-- src/main/java/com/guildboard/quests/config/
        `-- SecurityImportConfig.java           [N]
```

Ningún archivo de `domain` ni de `application` cambia en ningún servicio.

14.1.3 Cambios respecto al árbol anterior

- `tools/token-issuer/` [N]: proyecto Gradle nuevo en la raíz, con un solo módulo y sin hexágono. Está fuera de `services/` porque `StructureTest` exige los cuatro módulos a todo lo que cuelga de ahí; no forma parte del sistema que se despliega. Tampoco lleva `ArchitectureTest`: no tiene núcleo que proteger.
- `settings.gradle` [M]: un `include` y un `projectDir` (la carpeta ya no se llama como el proyecto). Quince proyectos en total.
- `adapter/in/web/security` [N]: la seguridad de HTTP es un detalle del adaptador de entrada web [28].
- `SecurityConfig` [N]: sin `@Configuration`; `bootstrap` la activa con `@Import` (sección 14.4), como raíz de composición (capítulo 3).
- `SecurityConfig` está repetida en los tres servicios, sin módulo común, como `Rank` y `AdventurerId` (verificación de estructura 5): las reglas de cada servicio no son iguales.
- `config-repo/application.yml` [M]: la dirección del emisor, la misma para los tres clientes (capítulo 13).
- `SecurityConfig` cuelga de `adapter/in/web/security`: está dentro de `adapter` y fuera de `domain` y `application`, así que no rompe ninguna regla de `ArchitectureTest`.
- `domain` y `application`: sin cambios en ningún servicio.

14.1.4 Comprobaciones

- [ ] `./gradlew test` pasa (incluye la prueba de arquitectura de cada servicio, que vigila `domain`, `application` y el aislamiento entre `in` y `out`).
- [ ] Cada adaptador está en `in` o en `out`: `SecurityConfig` y el controlador reciben la petición (`in`); el interceptor que copia el token hacia Adventurers sale del servicio (`out`).
- [ ] Ningún `build.gradle` de un servicio nombra a un proyecto de otro: los servicios conocen el emisor por URL en `config-repo`.
- [ ] Las carpetas coinciden con el árbol de arriba.

La primera casilla cubre también que `domain` y `application` no importen `org.springframework.security`: lo vigilan las reglas de ArchUnit. Lo demás (en qué lado está cada adaptador, los `build.gradle` y las carpetas) se revisa a mano.

Si algo no coincide con el árbol, corrígelo antes de seguir.

14.2 Qué es un JWT: cabecera, claims y firma

Un *JSON Web Token* (JWT) es una cadena que lleva afirmaciones sobre alguien (quién es, qué rol tiene, hasta cuándo vale) y una firma que permite comprobar que no se han alterado [22]. Son tres partes separadas por puntos, cada una en base64url:

Un JWT (cortado para que quepa)

```
eyJhbGciOiJSUzI1NiIs...   .   eyJpc3MiOiJodHRw...   .   WfJ3k0aB...
cabecera                      claims (payload)           firma
```

- La **cabecera** dice cómo está firmado: `alg` (el algoritmo; el curso usa `RS256`, RSA con SHA-256), `typ` y, a veces, `kid`, el identificador de la clave que firmó.
- Los **claims** son los datos. El estándar define algunos con nombre fijo [22]: `iss` (quién emitió el token), `sub` (de quién habla), `exp` (instante a partir del cual ya no vale), `nbf` (antes del cual aún no vale), `iat` (cuándo se emitió) y `aud` (para quién es). Los tiempos son segundos desde el 1 de enero de 1970 UTC. Cada emisor puede añadir claims propios; GuildBoard añade `roles`.
- La **firma** se calcula con la clave *privada* del emisor sobre las dos primeras partes. Cualquiera con la clave *pública* comprueba que es auténtica; nadie sin la privada puede fabricar una igual.

14.2.1 Decodificar con la shell

base64url es base64 con dos símbolos cambiados (`-` por `+` y `_` por `/`) y sin el relleno de signos `=` al final. Para leer un token no hace falta ninguna herramienta nueva: `cut` separa las partes por el punto, `tr` deshace el cambio de símbolos, un bucle reconstruye el relleno hasta que la longitud sea múltiplo de cuatro y `base64 -d` decodifica. Esta función (de bash) lo hace:

```
$ b64url() {
>   local s
>   s=$(printf '%s' "$1" | tr '_-' '/+')
```

```
>   while [ $(( ${#s} % 4 )) -ne 0 ]; do s="$s="; done
>   printf '%s' "$s" | base64 -d
> }
$ b64url "$(echo "$TOKEN" | cut -d. -f1)"; echo
$ b64url "$(echo "$TOKEN" | cut -d. -f2)"; echo
```

La primera orden extrae la cabecera (primer campo, `-f1`) y la segunda los claims (segundo campo). El `echo` final solo añade el salto de línea que `base64` no escribe. El resultado esperado, para un token del emisor del apartado siguiente, es JSON como este:

Cabecera y claims

```
{"alg":"RS256","kid":"...","typ":"JWT"}
{"sub":"master","roles":["GUILD_MASTER"],"iss":"http://localhost:9000",
"exp":1760000900,"iat":1760000000}
```

El orden de las claves y los valores concretos varían. El ejercicio 14.1 te da un token para decodificar.

Decodificar no es validar

Lo que acabas de hacer solo *lee*; cualquiera puede leer un JWT, porque el contenido no va cifrado. No pongas contraseñas ni datos personales en los claims. Y el mero hecho de que el JSON se vea bien no demuestra nada: para saber si el token es auténtico hay que verificar la firma con la clave pública, y eso no se hace desde la shell. Lo hace el servicio.

14.2.2 Qué comprueba un servicio al recibir un token

Un *resource server* de Spring Security hace, con cada petición que trae `Authorization: Bearer <token>` [63]:

1. Verifica la **firma** con una clave pública que obtiene de una URL (el `jwk-set-uri`, un JSON con las claves públicas del emisor).
2. Valida las marcas de tiempo, `exp` y `nbf`.
3. Valida el **emisor** (`iss`) si está configurado el `issuer-uri`.
4. Convierte los claims en una *autenticación* con autoridades (los permisos del usuario). Por omisión, cada *scope* se convierte en una autoridad con el prefijo `SCOPE_`; la sección 14.5 cambia eso.

No valida por su cuenta `aud` ni ninguna regla de negocio, y no consulta al emisor en cada petición: la validación es local, con la clave pública que ya tiene. Es la razón de que cada servicio pueda validar sin que el emisor esté siempre disponible. El patrón que describe Richardson, *access token* [42], hace que sea el gateway quien autentica y reenvía a los servicios un token con la identidad del llamante. GuildBoard hace la variante más exigente que pide el briefing: el gateway **no valida**, solo reenvía la cabecera `Authorization`, y **cada servicio valida por su cuenta**. Así, quien llegue a un servicio saltándose el gateway tampoco pasa.

14.3 El emisor de tokens de desarrollo

Un resource server necesita a alguien que emita tokens. En producción sería un servidor de autorización (Keycloak, Auth0, el de tu empresa). Para probar en local sin cuentas externas, el curso trae un emisor mínimo: dos *endpoints* y unos usuarios en un archivo YAML.

No es seguridad de producción

Las contraseñas están en texto plano en un YAML, se comparan sin protección, la clave RSA se genera de nuevo cada vez que arranca el programa y no hay limitación de intentos ni caducidad de sesión. Sirve para desarrollar, nada más. No lo expongas a una red, y no copies este patrón a un emisor real.

- `POST /token`: recibe `{"username":"...","password":"..."}` y devuelve un JSON con `access_token`, `token_type` (`Bearer`) y `expires_in` (segundos). Si los datos no coinciden, 401.
- `GET /.well-known/jwks.json`: devuelve las claves *públicas* en el formato *JSON Web Key Set* (JWKS). Es lo que leerán los servicios.
- Dos usuarios fijos: `master` (`sub master`, rol `GUILD_MASTER`) y `hero` (rol `ADVENTURER`, con un `sub` que será el id de un aventurero ya registrado; se rellena en la sección 14.8). `master` no es aventurero y solo publica.

☞ El emisor: `tools/token-issuer`

Qué se instala. Un proyecto Gradle nuevo que se arranca con `./gradlew :token-issuer:bootRun`. Tiene dos dependencias, ya gestionadas por el BOM de Spring Boot, así que sin versión:

- `spring-boot-starter-webmvc`, el mismo que ya usan los servicios.
- `org.springframework.security:spring-security-oauth2-jose`, que aporta el codificador `NimbusJwtEncoder`, las clases `JwtClaimsSet` y `JwsHeader` y, de forma transitiva, la biblioteca Nimbus JOSE+JWT con la que se construyen las claves (`RSAKey`, `JWKSet`).

Por qué no el *starter* de resource server también aquí. Los servicios lo tendrán (sección 14.4) y trae estas mismas clases. Pero ese *starter* activa también la configuración de seguridad por omisión de Spring Boot, que protegería el propio emisor con una contraseña generada; para este programa no queremos eso. Con solo `spring-security-oauth2-jose` no se activa ninguna.

Qué se verificó (9 de octubre de 2026). En Maven Central, el *pom* de `spring-security-oauth2-jose 7.1.1` (la versión que gestiona el BOM de Spring Boot `4.1.1`) declara `nimbus-jose-jwt 10.9.1` con ámbito *compile*: llega sin añadir ninguna librería. El *pom* del *starter* de resource server de Boot `4.1.1` arrastra `spring-security-oauth2-jose` (también en *compile*) y `spring-security-oauth2-resource-server`. En el *jar* `7.1.1`: `NimbusJwtEncoder` (constructor con un `JWKSource<SecurityContext>`), `JwtEncoderParameters.from(JwsHeader, JwtClaimsSet)`, `JwtClaimsSet.Builder` (`issuer`, `subject`, `issuedAt`, `expiresAt`, `claim`), `JwsHeader.with(...)` y `SignatureAlgorithm.RS256`. En Nimbus `10.9.1`: `RSAKey.Builder` (`privateKey`, `keyUse`, `algorithm`, `keyIDFromThumbprint`), `JWKSet`, `toPublicJWKSet()`, `toJSONObject()` e `ImmutableJWKSet`. **Qué no se verificó:** que el programa compile y arranque, y la salida de los *endpoints*. Si alguna de estas clases faltase, el plan B es firmar con `java.security.Signature` (`SHA256withRSA`) y construir a

mano el JSON del token y del JWKS: tampoco necesita librerías, pero el código es bastante más largo.

Paso 1. Carpetas y `settings.gradle`. Desde la raíz:

```
$ mkdir -p tools/token-issuer/src/main/resources
$ mkdir -p tools/token-issuer/src/main/java/com/guildboard/tokenissuer
```

En `settings.gradle`, al final, el proyecto y su carpeta. A diferencia de `gateway` y `config-server`, aquí la carpeta no se llama como el proyecto, así que hace falta `projectDir`:

settings.gradle (solo el final)

```
include 'token-issuer'
project(':token-issuer').projectDir = file('tools/token-issuer')
```

Paso 2. El `build.gradle`. Como el de `gateway`: el plugin de Spring Boot y dos dependencias.

tools/token-issuer/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.security:' +
        'spring-security-oauth2-jose'
}
```

Paso 3. Los archivos Java. Se copian tal cual; las explicaciones van detrás. La clase principal activa la lectura de la configuración, y la configuración se recibe en un *record*:

tokenissuer/TokenIssuerApplication.java

```
package com.guildboard.tokenissuer;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;

@SpringBootApplication
@ConfigurationPropertiesScan
public class TokenIssuerApplication {

    public static void main(String[] args) {
        SpringApplication.run(TokenIssuerApplication.class, args);
    }
}
```

tokenissuer/IssuerProperties.java

```
package com.guildboard.tokenissuer;

import java.time.Duration;
import java.util.List;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.context.properties.bind.DefaultValue;

@ConfigurationProperties("guildboard.token-issuer")
public record IssuerProperties(String issuer,
                               @DefaultValue("15m") Duration ttl,
                               List<User> users) {

    public record User(String username, String password,
                       String subject, String role) {
    }
}
```

La clave RSA se genera al arrancar. Se construye una `RSAKey` con su parte privada y un identificador (`kid`) calculado a partir de la propia clave; con él, si reinicias el emisor y cambia la clave, los servicios ven un `kid` que no conocen y vuelven a pedir el JWKS:

tokenissuer/KeyConfig.java

```
package com.guildboard.tokenissuer;

import com.nimbusds.jose.JOSEException;
import com.nimbusds.jose.JWSAlgorithm;
import com.nimbusds.jose.jwk.JWKSet;
import com.nimbusds.jose.jwk.KeyUse;
import com.nimbusds.jose.jwk.RSAKey;
import com.nimbusds.jose.jwk.source.ImmutableJWKSet;
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.NoSuchAlgorithmException;
import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.NimbusJwtEncoder;

@Configuration
public class KeyConfig {

    @Bean
    public JWKSet jwkSet()
            throws NoSuchAlgorithmException, JOSEException {
        KeyPairGenerator generator = KeyPairGenerator.getInstance("RSA");
        generator.initialize(2048);
        KeyPair pair = generator.generateKeyPair();
        RSAKey key = new RSAKey.Builder((RSAPublicKey) pair.getPublic())
            .privateKey((RSAPrivateKey) pair.getPrivate())
            .keyUse(KeyUse.SIGNATURE)
            .algorithm(JWSAlgorithm.RS256)
```

```
            .keyIDFromThumbprint()
            .build();
        return new JWKSet(key);
    }

    @Bean
    public JwtEncoder jwtEncoder(JWKSet jwkSet) {
        return new NimbusJwtEncoder(new ImmutableJWKSet<>(jwkSet));
    }
}
```

El controlador que emite. Busca al usuario, construye los claims, firma con `RS256` y responde. Los claims son los de la sección 14.2: `iss`, `sub`, `iat`, `exp` y la claim propia `roles`, una lista:

tokenissuer/TokenController.java (1 de 2)

```
package com.guildboard.tokenissuer;

import com.nimbusds.jose.jwk.JWKSet;
import java.time.Instant;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.oauth2.jose.jws.SignatureAlgorithm;
import org.springframework.security.oauth2.jwt.JwsHeader;
import org.springframework.security.oauth2.jwt.JwtClaimsSet;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.JwtEncoderParameters;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class TokenController {

    public record TokenRequest(String username, String password) {
    }

    private final JwtEncoder encoder;
    private final JWKSet jwkSet;
    private final IssuerProperties properties;

    public TokenController(JwtEncoder encoder, JWKSet jwkSet,
                           IssuerProperties properties) {
        this.encoder = encoder;
        this.jwkSet = jwkSet;
        this.properties = properties;
    }
```

tokenissuer/TokenController.java (2 de 2: sigue dentro de la clase)

```
    @PostMapping("/token")
    public ResponseEntity<Map<String, Object>> token(
```

```
            @RequestBody TokenRequest request) {
        Optional<IssuerProperties.User> found = properties.users()
            .stream()
            .filter(u -> u.username().equals(request.username())
                && u.password().equals(request.password()))
            .findFirst();
        if (found.isEmpty()) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
        }
        IssuerProperties.User user = found.get();
        Instant now = Instant.now();
        JwtClaimsSet claims = JwtClaimsSet.builder()
            .issuer(properties.issuer())
            .subject(user.subject())
            .issuedAt(now)
            .expiresAt(now.plus(properties.ttl()))
            .claim("roles", List.of(user.role()))
            .build();
        JwsHeader header = JwsHeader.with(SignatureAlgorithm.RS256)
            .keyId(jwkSet.getKeys().get(0).getKeyID())
            .build();
        String token = encoder
            .encode(JwtEncoderParameters.from(header, claims))
            .getTokenValue();
        return ResponseEntity.ok(Map.of(
            "access_token", token,
            "token_type", "Bearer",
            "expires_in", properties.ttl().toSeconds()));
    }
}
```

Y el *endpoint* de las claves públicas. `toPublicJWKSet()` quita la parte privada: es la línea que impide publicar por error la clave con la que se firma.

tokenissuer/JwksController.java

```
package com.guildboard.tokenissuer;

import com.nimbusds.jose.jwk.JWKSet;
import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class JwksController {

    private final JWKSet jwkSet;

    public JwksController(JWKSet jwkSet) {
        this.jwkSet = jwkSet;
    }

    @GetMapping("/.well-known/jwks.json")
    public Map<String, Object> jwks() {
        return jwkSet.toPublicJWKSet().toJSONObject();
    }
```

```
}
```

Paso 4. La configuración. Puerto 9000, el emisor (el valor que irá en el claim `iss`), la duración del token y los dos usuarios. La contraseña de cada uno es su nombre de usuario. El `subject` de `hero` es un marcador que sustituirás en la sección 14.8:

tools/token-issuer/src/main/resources/application.yml

```
spring:
  application:
    name: token-issuer

server:
  port: 9000

guildboard:
  token-issuer:
    issuer: http://localhost:9000
    ttl: 15m
    users:
      - username: master
        password: master
        subject: master
        role: GUILD_MASTER
      - username: hero
        password: hero
        subject: id-del-aventurero-registrado
        role: ADVENTURER
```

Qué hace cada pieza, sin entrar en Nimbus. `KeyConfig` crea un par de claves RSA de 2048 bits y las guarda en memoria; `JwtEncoder` es el objeto que, dados unos claims y una cabecera, produce la cadena firmada. `TokenController` es el único código con «lógica»: comprueba usuario y contraseña, pone la fecha de caducidad a `ttl` desde ahora, y escribe el rol en la claim `roles` como lista. `JwksController` publica la mitad pública de la clave. Este programa no tiene dominio, ni puertos, ni adaptadores: no hay nada que decidir con significado de negocio.

Errores típicos: olvidar el `projectDir` (`./gradlew :token-issuer:bootRun` falla porque Gradle busca la carpeta `token-issuer` en la raíz); poner en el `issuer` un valor distinto al que escribirás en `config-repo` (todos los tokens fallan con 401 en los servicios); y reiniciar el emisor y seguir usando un token antiguo: la clave nueva no lo firmó, así que se rechaza hasta que pidas uno nuevo.

14.4 Resource server: cada servicio valida el token

Ahora, los servicios. Un servicio se convierte en *resource server* con una dependencia, dos propiedades y una clase de configuración. Empiezo por Quests; Adventurers y Treasury lo repiten en el ejercicio 14.5.

☞ Starter de OAuth2 resource server de Spring Boot

Qué se instala. El *starter* `org.springframework.boot:spring-boot-starter-security-oauth2-resource-server`, sin versión (la gestiona el BOM). Se añade a dos módulos de cada servicio: `adapters` (donde estarán `SecurityConfig` y, en Quests, el con-

trolador y el interceptor que usan clases de Spring Security) y `bootstrap` (donde se activa la autoconfiguración). Es el mismo reparto que ya sigue `spring-boot-starter-amqp`.
Qué se verificó (9 de octubre de 2026). En Spring Initializr con Boot `4.1.1`, la dependencia `oauth2-resource-server` genera exactamente ese artefacto (y `security` genera `spring-boot-starter-security`). El *pom* del *starter* depende de `spring-boot-starter-security` y de `spring-boot-security-oauth2-resource-server`, que a su vez arrastra Spring Security `7.1.1` (`oauth2-jose` y `oauth2-resource-server`); es decir, Nimbus llega transitivo, como en el emisor. En el *jar* de `spring-boot-security-oauth2-resource-server` `4.1.1`, los metadatos de configuración incluyen `spring.security.oauth2.resourceserver.jwt.jwk-set-uri`, `...issuer-uri`, `...jws-algorithms` (por omisión `RS256`), `...audiences`, `...authorities-claim-name` y `...authority-prefix`; la autoconfiguración del decodificador con `jwk-set-uri` añade un `JwtIssuerValidator` cuando también hay `issuer-uri`. La documentación de Spring Security `7.1.1` [63] dice lo mismo con palabras: con las dos propiedades, el servicio no consulta al emisor al arrancar y aun así valida el `iss`. **Qué no se verificó:** que los servicios arranquen con el *starter*, y que la prioridad entre propiedades locales y de `config-repo` sea la descrita.

Las dependencias. En `services/quests/adapters/build.gradle` y en `services/quests/bootstrap/build.gradle`, una línea más dentro de `dependencies`:

Línea nueva en los dos build.gradle de Quests

```
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-security-oauth2-resource-server'
```

La dirección del emisor. Va en el archivo común de `config-repo`, porque la comparten los tres servicios. Añade un bloque `security` bajo `spring` (el resto del archivo, sin cambios):

config-repo/application.yml (bloque nuevo, dentro de spring)

```
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: guildboard
    password: guildboard
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:9000
          jwk-set-uri: http://localhost:9000/.well-known/jwks.json
```

- `jwk-set-uri` es dónde está el JWKS (sección 14.3).
- `issuer-uri` es el valor que debe traer el claim `iss`. Con las dos puestas, Spring no intenta descubrir el emisor por su cuenta (el emisor de desarrollo no ofrece ese descubrimiento) y sí comprueba el `iss`.
- El servicio descarga el JWKS al validar el primer token, no al arrancar. Por eso los servicios arrancan aunque el emisor no esté levantado; solo rechazarán los tokens hasta que lo esté.

El gateway también recibe este archivo común, pero no lleva el *starter*: las propiedades le son indiferentes (el Config Server no es cliente de sí mismo). Y como todos los cambios de `config-repo`, hay que reiniciar los clientes para que las lean.

La clase de configuración. Va en el adaptador de entrada web, en el paquete `security`. Es el primer borrador, que solo exige un token válido: la sección 14.5 añadirá los roles.

adapter/in/web/security/SecurityConfig.java (Quests, primera versión)

```
package com.guildboard.quests.adapter.in.web.security;

import org.springframework.context.annotation.Bean;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity
    ↪ ;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;

public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth -> oauth
                .jwt(Customizer.withDefaults()));
        return http.build();
    }
}
```

Una cadena de filtros (`SecurityFilterChain`) es la lista de filtros que cada petición HTTP cruza antes de llegar al controlador. Qué hace esta:

- `csrf(...disable)`: CSRF es un ataque contra sesiones con *cookies*. Aquí no hay sesiones ni *cookies*: la identidad viaja en una cabecera que el navegador no añade solo. Y si lo dejas activado, un `POST` sin token devuelve 403 en lugar de 401.
- `STATELESS`: no se crea sesión; cada petición se valida desde cero.
- `anyRequest().authenticated()`: toda petición necesita un token válido.
- `oauth2ResourceServer(...jwt(...))`: activa la validación de JWT con los valores por omisión: el decodificador que Spring Boot crea con las propiedades de `config-repo`.

Fíjate en que la clase **no lleva `@Configuration`**. Si la llevara, el *escaneo de componentes* de Spring Boot (que recorre el paquete `com.guildboard.quests`) la activaría sola en cualquier arranque, incluida una futura prueba que solo quisiera los controladores. En su lugar, la raíz de composición la activa de forma explícita, con una clase de configuración mínima en `bootstrap`:

config/SecurityImportConfig.java (Quests)

```java
package com.guildboard.quests.config;

import com.guildboard.quests.adapter.in.web.security.SecurityConfig;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Import;

@Configuration
@Import(SecurityConfig.class)
public class SecurityImportConfig {
}
```

`@Import` acepta también una clase corriente con métodos `@Bean`, y registra sus beans. Este es el reparto de siempre: `adapters` ofrece una pieza, `bootstrap` las enchufa.

14.4.1 Qué verá el cliente

Con esto, y antes de hablar de roles, Quests responde así (los códigos son los esperados; no los he ejecutado):

- Sin cabecera `Authorization`: **401**, con `WWW-Authenticate: Bearer`.
- Con un token mal firmado, caducado o con otro `iss`: **401**, con `WWW-Authenticate: Bearer error="invalid_token"....`
- Con un token válido: lo que el controlador conteste.

La diferencia importa: 401 significa «no sé quién eres» y 403 significa «sé quién eres, pero no puedes». Esa distinción es lo que hace útil el rol.

14.5 Roles: de la claim a las autoridades

Spring Security autoriza con *autoridades* (`GrantedAuthority`), cadenas como `ROLE_GUILD_MASTER`. Un *rol* es una autoridad con el prefijo `ROLE_`, y `hasRole("GUILD_MASTER")` comprueba la autoridad `ROLE_GUILD_MASTER`. El JWT trae los roles en la claim `roles` (`["GUILD_MASTER"]`); falta traducirlos. Por omisión, Spring Security solo convierte la claim `scope` y con prefijo `SCOPE_` [63], así que hay que decirle otra cosa. El objeto que lo hace se llama `JwtGrantedAuthoritiesConverter`:

Esquema: la claim roles como autoridades

```java
JwtGrantedAuthoritiesConverter authorities =
    new JwtGrantedAuthoritiesConverter();
authorities.setAuthoritiesClaimName("roles");
authorities.setAuthorityPrefix("ROLE_");

JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
converter.setJwtGrantedAuthoritiesConverter(authorities);
```

La claim se llama `roles`, cada valor recibe el prefijo `ROLE_`, y el `JwtAuthenticationConverter` resultante se le pasa a `.jwt(...)` con `jwt.jwtAuthenticationConverter(converter)` en lugar de `Customizer.withDefaults()`. (Los metadatos de Spring Boot `4.1.1` incluyen las

propiedades `authorities-claim-name` y `authority-prefix` para esto mismo, sin código; no las he probado y el curso prefiere tenerlo escrito y visible.)

Con las autoridades en su sitio, las reglas se escriben dentro de `authorizeHttpRequests`, de arriba abajo; **gana la primera que casa**, así que lo específico va antes y `anyRequest()` es siempre la última:

Esquema: reglas por método y ruta

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.POST, "/quests").hasRole("GUILD_MASTER")
    .requestMatchers(HttpMethod.GET, "/quests/*").authenticated()
    .anyRequest().authenticated())
```

El asterisco casa con un solo segmento de ruta. Este es un esquema y no la regla de GuildBoard, que decide el ejercicio 14.3: publicar solo `GUILD_MASTER`, aceptar solo `ADVENTURER`.

401, 403 y quién responde

Las reglas de arriba se evalúan *antes* de llegar al controlador. Un rol que falta no produce un 404 ni una excepción tuya: produce un 403 que Spring Security responde sin entrar en tu código. Y por eso el dominio no necesita saber nada: cuando un caso de uso se ejecuta, la petición ya ha pasado por el filtro.

14.6 Dónde autorizar: el borde, no el dominio

Hay dos clases de reglas que suenan parecido y no son lo mismo:

- Una **regla de seguridad**: «solo un `GUILD_MASTER` publica». Habla de quién puede usar el sistema y se puede comprobar con solo la petición. Vive en el borde.
- Una **regla de negocio**: «un aventurero de rango `NOVICE` no puede aceptar una misión `VETERAN`». Habla del gremio y no depende del protocolo. Vive en el dominio (capítulo 5).

Entre las dos hay una tercera, «un aventurero solo acepta misiones *a su nombre*», que pide mirar el token y el comando a la vez. ¿Dónde va? Si va al dominio, tendríamos que pasarle el `Jwt`, y el núcleo dependería del framework. La salida es la que ya conoces del capítulo 4: **el adaptador traduce**. El controlador toma el `sub` del token, que el emisor garantiza que es un `AdventurerId` registrado, y lo pone en el `AcceptQuestCommand`. El dominio sigue viendo un `AdventurerId` corriente. Es el límite de Martin [28], capítulo 17: el detalle (aquí, el token) se queda al otro lado y por la frontera cruza una estructura de datos simple.

Hasta ahora, el cuerpo de `POST /quests/{id}/accept` traía el id del aventurero (`{"adventurerId":"..."}`) y el servicio se fiaba. Con JWT sobra: lo dice el token. El controlador queda así; los *imports* nuevos son `HttpStatus`, `AuthenticationPrincipal`, `Jwt` y `ResponseStatusException` (el archivo entero está en la solución del ejercicio 14.5). `ResponseStatusException` es la excepción de Spring que, lanzada desde un controlador, hace que la respuesta salga con el código HTTP indicado (aquí 403 o 400) y un mensaje:

adapter/in/web/QuestController.java (solo el método accept)

```
@PostMapping("/{id}/accept")
public QuestResponse accept(
        @PathVariable String id,
```

```
        @AuthenticationPrincipal Jwt jwt,
        @RequestBody(required = false) AcceptQuestRequest request) {
    String subject = jwt.getSubject();
    if (request != null && request.adventurerId() != null
            && !request.adventurerId().equals(subject)) {
        throw new ResponseStatusException(HttpStatus.FORBIDDEN,
            "the token does not belong to that adventurer");
    }
    Quest quest = acceptQuest.accept(new AcceptQuestCommand(
        new QuestId(id), new AdventurerId(subject)));
    return QuestWebMapper.toResponse(quest);
}
```

Tres detalles:

- `@AuthenticationPrincipal Jwt jwt` entrega el token ya validado a la petición actual; no hay que leerlo de ningún sitio.
- El cuerpo pasa a ser opcional. Si alguien aún lo manda, un id distinto del `sub` da **403**: el token es válido pero no es el de ese aventurero. `AcceptQuestRequest` pierde su `@NotBlank`.
- `AcceptQuestUseCase`, `AcceptQuestCommand` y `AcceptQuestService` **no cambian**: siguen recibiendo un `AdventurerId`.

Si el `sub` no es un aventurero registrado, el servicio responde 404 por la regla de siempre (`AdventurerNotFound`); el token no basta para ser aventurero. El ejercicio 14.4 te pide defender esta decisión frente a otras, como una claim propia (`adventurer_id`).

14.7 Pasar el token de Quests a Adventurers

Hay una llamada entre servicios que ya existe: al aceptar una misión, Quests pregunta a Adventurers por el rango del aventurero (`GET /adventurers/{id}`, capítulos 9 y 12). Cuando Adventurers valide tokens, esa llamada llegará sin token y recibirá un 401. Hay dos formas de resolverlo [32]:

- **Reenviar el token de la persona.** Quests copia la cabecera `Authorization` de la petición que está atendiendo. Adventurers ve a la misma persona. Es lo que hace el patrón *access token* [42] y lo más simple.
- **Que Quests tenga su propia identidad** y pida un token para sí mismo al emisor. Es más limpio cuando el servicio actúa por su cuenta, pero exige que el emisor sepa de servicios, no solo de personas, y queda fuera del alcance de este curso.

El curso reenvía el token de la persona. Y lo hace en el sitio que le toca: el **adaptador de salida**. Mira cómo queda el puerto, que no cambia:

application/port/out/AdventurerDirectory.java (sin cambios)

```
package com.guildboard.quests.application.port.out;

import com.guildboard.quests.domain.AdventurerId;
import java.util.Optional;

public interface AdventurerDirectory {
```

```
7
8       Optional<AdventurerView> findById(AdventurerId id);
9   }
```

Ni el puerto ni `AcceptQuestService` saben que existe un token. «Para llamar a Adventurers hay que ir identificado» es un detalle del transporte HTTP, y por tanto del adaptador `RestAdventurerDirectory`.

14.7.1 Un interceptor del RestClient

Un *interceptor* de `RestClient` es un objeto al que el cliente pasa cada petición saliente antes de enviarla; puede cambiarla. Aquí solo añade una cabecera. Su única fuente de información es el `SecurityContextHolder`, el sitio donde Spring Security guarda la autenticación de la petición que se está atendiendo:

adapter/out/rest/BearerTokenForwardingInterceptor.java

```
1   package com.guildboard.quests.adapter.out.rest;
2
3   import java.io.IOException;
4   import org.springframework.http.HttpRequest;
5   import org.springframework.http.client.ClientHttpRequestExecution;
6   import org.springframework.http.client.ClientHttpRequestInterceptor;
7   import org.springframework.http.client.ClientHttpResponse;
8   import org.springframework.security.core.Authentication;
9   import org.springframework.security.core.context.SecurityContextHolder;
10  import org.springframework.security.oauth2.server.resource.authentication.
        ↪ JwtAuthenticationToken;
11
12  public class BearerTokenForwardingInterceptor
13          implements ClientHttpRequestInterceptor {
14
15      @Override
16      public ClientHttpResponse intercept(HttpRequest request, byte[] body,
17              ClientHttpRequestExecution execution) throws IOException {
18          Authentication current =
19              SecurityContextHolder.getContext().getAuthentication();
20          if (current instanceof JwtAuthenticationToken jwt) {
21              request.getHeaders()
22                  .setBearerAuth(jwt.getToken().getTokenValue());
23          }
24          return execution.execute(request, body);
25      }
26  }
```

Si no hay autenticación (por ejemplo, en la prueba del capítulo 12, que llama al adaptador sin Spring Security), no añade nada y la llamada sale como antes. Se registra en el único sitio donde se construye el cliente, la fábrica del capítulo 12; una línea más:

adapter/out/rest/AdventurerClientFactory.java (solo el método create)

```
1   public static RestClient create(String baseUrl,
2           Duration connectTimeout, Duration readTimeout) {
3       HttpClient http = HttpClient.newBuilder()
```

```
        .connectTimeout(connectTimeout)
        .build();
    JdkClientHttpRequestFactory factory =
        new JdkClientHttpRequestFactory(http);
    factory.setReadTimeout(readTimeout);
    return RestClient.builder()
        .baseUrl(baseUrl)
        .requestFactory(factory)
        .requestInterceptor(new BearerTokenForwardingInterceptor())
        .build();
}
```

Ni `RestAdventurerDirectory` ni `BeanConfig` cambian: la clase ya construía el cliente con la fábrica.

El contexto de seguridad vive en el hilo de la petición

`SecurityContextHolder` guarda la autenticación en una variable ligada al hilo. El interceptor la encuentra porque la llamada a Adventurers se hace *en el mismo hilo* que atiende la petición de Quests: los decoradores de Resilience4j del capítulo 12 son síncronos, así que el reintento y el interruptor corren ahí también. Si algún día esa llamada se pasara a otro hilo (una tarea asíncrona, un *executor*), el contexto no viajaría solo y el interceptor no encontraría el token.

14.7.2 El gateway y los mensajes

- **El gateway** no necesita cambios: reenvía la cabecera `Authorization` al servicio y, como se dijo, no la valida. Compruebo en el *jar* de Gateway `5.0.3` la lista de cabeceras que su filtro `RemoveHopByHopRequestHeadersFilter` elimina de la petición (`connection`, `keep-alive`, `transfer-encoding`, `te`, `trailer`, `proxy-authorization`, `proxy-authenticate`, `x-application-context` y `upgrade`): `authorization` no está. No lo he ejecutado; el recorrido de la sección 14.8 lo comprueba.
- **RabbitMQ** no lleva JWT. El evento `QuestCompleted` del capítulo 11 sigue siendo solo ids y recompensa. Los consumidores no saben quién completó la misión; confían en que solo los servicios con la contraseña del *broker* publican en ese *exchange*. Es una garantía más débil que la de HTTP, y se reconoce como tal al final del capítulo.

14.8 En el proyecto: hito H7 (parte 2)

Hito 7, parte 2: cada servicio exige y valida un token

Al terminar el ejercicio 14.5, GuildBoard tiene:

- **Un emisor de desarrollo** (`tools/token-issuer`, puerto 9000) con `POST /token`, `GET /.well-known/jwks.json`, firma `RS256` y dos usuarios: `master` (rol `GUILD_MASTER`) y `hero` (rol `ADVENTURER`, con `sub` igual a un aventurero registrado).
- **Tres resource servers.** Quests, Adventurers y Treasury validan firma, caducidad y emisor con la configuración común de `config-repo`, y cada uno tiene su `SecurityConfig` en `adapter/in/web/security`.
- **Roles.** Publicar misiones y registrar aventureros solo con `GUILD_MASTER`; aceptar solo

con `ADVENTURER`; completar con cualquiera de los dos; leer, con cualquier token válido.

- **El `sub` manda.** Quests construye el `AcceptQuestCommand` con el `sub` del token. Un id distinto en el cuerpo da 403.
- **Reenvío.** El token llega de Quests a Adventurers con un interceptor del `RestClient`, en el adaptador de salida; el gateway reenvía `Authorization` sin validarlo.
- **La estructura T8** del árbol de la verificación de estructura 9. `domain` y `application` de los tres servicios, sin tocar.

Con esto, H7 está completo: gateway, configuración centralizada y seguridad.

14.8.1 Probar el hito

Arranca todo en el orden del capítulo 13, con el emisor justo después del Config Server (en el `application.yml` de `config-repo` ya está su dirección). Cada comando, en su terminal y en modo memoria:

```
$ docker compose up -d --wait rabbitmq
$ ./gradlew :config-server:bootRun           # terminal 1
$ ./gradlew :token-issuer:bootRun            # terminal 2
$ ./gradlew :adventurers-bootstrap:bootRun   # terminal 3
$ ./gradlew :treasury-bootstrap:bootRun      # terminal 4
$ ./gradlew :quests-bootstrap:bootRun        # terminal 5
$ ./gradlew :gateway:bootRun                 # terminal 6
```

En una séptima terminal, dos funciones de ayuda. `jid` extrae el campo `id` de una respuesta; `code` imprime solo el código HTTP; `tok` pide un token al emisor (la contraseña de cada usuario es su nombre) y escribe solo el `access_token` (el emisor devuelve JSON compacto, que es lo que espera el `grep`):

```
$ jid() { grep -o '"id":"[^"]*"' | head -1 | cut -d'"' -f4; }
$ code() { curl -s -o /dev/null -w '%{http_code}\n' "$@"; }
$ tok() {
>   curl -s -X POST localhost:9000/token \
>     -H 'Content-Type: application/json' \
>     -d "{\"username\":\"$1\",\"password\":\"$1\"}" |
>   grep -o '"access_token" *: *"[^"]*"' | cut -d'"' -f4
> }
$ MASTER=$(tok master)
```

Un huevo y una gallina. El token de `hero` debe llevar como `sub` el id de un aventurero registrado, pero registrar a un aventurero exige el token de `master` (cuando Adventurers esté protegido por el ejercicio 14.5). Por eso se hace en dos pasos: primero con `master`, que no necesita ningún aventurero, se registra a Mara y se copia su id.

```
$ MARA=$(curl -s -X POST localhost:8080/api/adventurers \
    -H "Authorization: Bearer $MASTER" \
    -H 'Content-Type: application/json' -d '{"name":"Mara"}' | jid)
$ echo $MARA
```

Escribe ese id en `subject` del usuario `hero` en `tools/token-issuer/src/main/resources/application.yml`, para y vuelve a lanzar el emisor (terminal 2). Al reiniciar, el emisor genera una clave nueva: los tokens anteriores dejan de valer. Los servicios lo notan por el `kid` desconocido y vuelven a descargar el JWKS (lo esperado; no lo he ejecutado). Ahora los dos tokens y el recorrido de siempre, esta vez con cabeceras:

```
$ MASTER=$(tok master)
$ HERO=$(tok hero)
$ Q1=$(curl -s -X POST localhost:8080/api/quests \
    -H "Authorization: Bearer $MASTER" \
    -H 'Content-Type: application/json' \
    -d '{"title":"Limpiar el s^^c3^^b3tano","reward":150,
         "requiredRank":"NOVICE"}' | jid)
$ curl -s -X POST localhost:8080/api/quests/$Q1/accept \
    -H "Authorization: Bearer $HERO"
$ curl -s -X POST localhost:8080/api/quests/$Q1/complete \
    -H "Authorization: Bearer $HERO"
```

La aceptación no lleva cuerpo: el aventurero es el `sub` del token. Un instante después, el pago y la experiencia, que ahora también necesitan token (salidas *esperadas*; no he ejecutado el código):

```
$ curl -s localhost:8080/api/treasury/accounts/$MARA \
    -H "Authorization: Bearer $HERO"
$ curl -s localhost:8080/api/adventurers/$MARA \
    -H "Authorization: Bearer $HERO"
```

La primera respuesta es `{"adventurerId":"<id>","balance":150}`. La segunda, el aventurero con `"rank":"ADEPT"` y `"experience":150`. Y las pruebas que enseñan para qué sirve cada pieza:

```
$ code localhost:8080/api/quests/$Q1
$ code -X POST localhost:8080/api/quests \
    -H "Authorization: Bearer $HERO" \
    -H 'Content-Type: application/json' \
    -d '{"title":"x","reward":1,"requiredRank":"NOVICE"}'
$ code -X POST localhost:8080/api/quests/$Q1/accept \
    -H "Authorization: Bearer $MASTER"
$ code localhost:8082/adventurers/$MARA
$ BAD="${HERO:0:${#HERO}-4}AAAA"
$ code localhost:8080/api/quests/$Q1 -H "Authorization: Bearer $BAD"
```

Los cinco códigos esperados, en orden: **401** (sin token), **403** (un aventurero no publica), **403** (el maestro no es aventurero), **401** (Adventurers, llamado directamente en su puerto, sin pasar por el gateway: también exige token) y **401** (firma alterada: se cambian los cuatro últimos caracteres de la firma). Para probar la caducidad, pon `ttl: 10s` en el emisor, reinícialo, pide un token, espera quince segundos y úsalo: 401 con `error="invalid_token"`. Y la comprobación de que el núcleo no se enteró:

```
$ ./gradlew test
$ ./gradlew build
$ git status --short
```

```
$ git diff --stat -- 'services/*/domain' 'services/*/application'
$ grep -rln springframework services/*/domain/src/main \
    services/*/application/src/main
```

Los dos últimos comandos no deben mostrar nada. En `git status` deben aparecer `tools/`, `settings.gradle`, `config-repo/application.yml`, y en cada servicio los dos `build.gradle`, `SecurityConfig.java` y `SecurityImportConfig.java`; en Quests, además, el controlador, el DTO de aceptar, la fábrica y el interceptor.

14.8.2 Lo que este capítulo no resuelve

- **No es seguridad de producción.** El emisor es un andamio. Un emisor real gestiona usuarios y contraseñas con *hash*, rota las claves sin perder los tokens vivos, y permite revocar.
- **Todo va por HTTP sin cifrar.** Un token interceptado vale hasta que caduque. En producción, TLS en cada salto.
- **Un token reenviado vale en todos los servicios.** No se valida `aud`: el token de Quests sirve en Treasury. Hacerlo bien exige que cada servicio tenga su propia audiencia, y de ello trata el emisor real, no este.
- **Si el token caduca a mitad de cadena**, Adventurers responde 401 a Quests. `RestAdventurerDirectory` solo traduce el 404, los fallos de red y los 5xx: un 401 o 403 de Adventurers llegaría al cliente como 500. Es un hueco a cerrar con un manejador propio, como el del 503.
- **Los mensajes de RabbitMQ no están autenticados** más allá de la contraseña del *broker*.
- **El Config Server sigue abierto**, y con él las credenciales de desarrollo del `config-repo`.

14.9 Ejercicios

Los ejercicios parten del proyecto tal como lo dejó el capítulo 13, más lo que has hecho en las secciones 14.3 a 14.7: el emisor funcionando, `config-repo/application.yml` con el bloque `security`, y Quests con el *starter*, su `SecurityConfig` de la primera versión (solo `authenticated()`), `SecurityImportConfig`, el método `accept` del controlador y el interceptor. Adventurers y Treasury todavía no validan nada. El primero se resuelve leyendo; el segundo es un recorrido guiado de la herramienta; el tercero escribe las reglas de roles; el cuarto defiende el diseño; el último junta todo en el hito.

Ejercicio 14.1: Leer un JWT con la shell ★★☆☆☆

Un compañero te pasa este token de prueba y te dice que «el servicio lo rechaza». La firma es un texto cualquiera: este ejercicio no verifica firmas. Pega estas líneas en una terminal de `bash`:

```
H=eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCIsImtpZCI6ImRldi1rZXktMSJ9
P=eyJpc3MiOiJodHRwOi8vbG9jYWxob3N0OjkwMDEiLCJzdWIiOiJoZXJvLTci
P=${P}LCJyb2xlcyI6WyJBRFZFTlRVUkVSIl0sImlhdCI6MTc2NzIyNTYwMCwi
P=${P}ZXhwIjoxNzY3MjI2NTAwfQ
S=ZmlybWEtZGUtZWplbXBsby1xdWUtbm8tc2UtcHVlZGUtdmVyaWZpY2Fy
TOKEN="$H.$P.$S"
```

Las líneas que empiezan por `P=${P}` añaden texto al final de `P`; así el token cabe en la página. Haz lo siguiente, sin arrancar ningún servicio:

1. Define la función `b64url` de la sección 14.2 y decodifica la cabecera y los claims con `cut -d.`, `tr` y `base64 -d`.
2. Convierte `iat` y `exp` a fechas legibles con `date -u -d @<segundos>`.
3. Los servicios de GuildBoard esperan `iss` igual a `http://localhost:9000`. Asume que hoy es posterior al 1 de enero de 2026. Para cada comprobación de la sección 14.2 (firma, `exp`, `nbf` y `iss`) di si el token la supera, la falla o no se puede decidir desde la shell, y por qué.
4. Si el token fuese válido, ¿qué autoridad tendría el usuario tras el convertidor de la sección 14.5? ¿Pasaría `hasRole("ADVENTURER")`?
5. Alguien cambia a mano `roles` a `["GUILD_MASTER"]` en el segundo fragmento y deja los otros dos como están. ¿Qué ocurre al validarlo y por qué?

Está bien si: la cabecera sale con `RS256` y `kid dev-key-1`; los claims con `iss`, `sub`, `roles`, `iat` y `exp`; `exp` es `2026-01-01T00:15:00Z`; el veredicto señala `exp` e `iss` como fallos, `nbf` como «no aplica» (no hay esa claim) y la firma como «no decidible en la shell»; y la respuesta al último punto dice que la firma ya no corresponde al contenido.

Pista: *el segundo fragmento mide 138 caracteres, que no es múltiplo de cuatro: la función añade dos signos `=` antes de decodificar.*

Lectura de código ▪ solución en la pág. 542

Ejercicio 14.2: Pedir tokens al emisor y llamar a Quests ★★☆☆☆

Vas a recorrer la herramienta de punta a punta, con Quests ya protegido y los demás servicios aún abiertos. Todo se hace en modo memoria y sin gateway.

1. Arranca RabbitMQ, el Config Server, Adventurers y Quests (cuatro terminales, en ese orden; el emisor, todavía no).
2. Registra a Mara llamando directamente a Adventurers (aún no pide token): `POST localhost:8082/adventurers` con `{"name":"Mara"}`. Guarda su id en la variable `MARA`.
3. Escribe ese id en `subject` del usuario `hero` del `application.yml` del emisor y arranca el emisor.
4. Define `jid`, `code` y `tok` (sección 14.8) y pide `MASTER=$(tok master)` y `HERO=$(tok hero)`. Decodifica los claims de `HERO` con `b64url`.
5. Con `MASTER`, publica una misión directamente en Quests (`localhost:8081`). Guarda su id en `Q1`. Con la configuración actual publica cualquiera con un token válido: los roles llegan en el ejercicio 14.3.
6. Pide `GET /quests/$Q1` dos veces y muestra solo el código HTTP: una vez sin cabecera

y otra con `HERO`.

7. Acepta `Q1` con `HERO`, sin cuerpo.
8. Publica una segunda misión `Q2` y acéptala con `HERO` pero con el cuerpo `{"adventurerId":"otro"}`.

Está bien si: el payload de `HERO` muestra `sub` igual a `MARA`, `iss` `http://localhost:9000` y `roles` con `ADVENTURER`; el paso 5 devuelve 201; el paso 6 da 401 sin cabecera y 200 con ella; el paso 7 devuelve 200 y la misión queda asignada a `MARA`; y el paso 8 devuelve 403.

Pista: *para imprimir solo el código HTTP usa la función* `code` *de la sección 14.8; la cabecera es* `-H "Authorization: Bearer $HERO"`.

Pista: *para publicar, el cuerpo es el de siempre:* `title`, `reward` *y* `requiredRank` `NOVICE`.

Guiado ▪ solución en la pág. 544

Ejercicio 14.3: Roles: quién publica, quién acepta ★★★☆☆

Parte de Quests con la primera versión de `SecurityConfig` y el flujo del ejercicio anterior (tienes `MASTER`, `HERO` y `MARA`). Cambia solo `SecurityConfig` para que:

1. las autoridades salgan de la claim `roles`, con el prefijo `ROLE_`;
2. `POST /quests` lo pueda hacer solo `GUILD_MASTER`;
3. `POST /quests/{id}/accept` lo pueda hacer solo `ADVENTURER`;
4. `POST /quests/{id}/complete` lo puedan hacer `GUILD_MASTER` o `ADVENTURER`;
5. cualquier otra petición, con cualquier token válido.

Reinicia Quests (guarda en memoria: publica de nuevo lo que necesites) y rellena esta tabla con los códigos que obtengas de verdad, en este orden, con una misma misión: (a) `MASTER` publica; (b) `HERO` publica; (c) `HERO` acepta la misión de (a); (d) `MASTER` acepta otra misión publicada; (e) publicar sin token; (f) `HERO` completa la misión de (c); (g) `MASTER` lee la misión con `GET`. Explica en dos frases por qué (e) y (b) no devuelven el mismo código.
Está bien si: la tabla da 201, 403, 200, 403, 401, 200 y 200; `anyRequest()` es la última regla; solo has modificado `SecurityConfig.java`; y no hay ningún *import* de Spring Security en `domain` ni en `application`.

Pista: `hasRole("GUILD_MASTER")` *se escribe sin* `ROLE_`*; si todo devuelve 403, mira el prefijo y el nombre de la claim, no las reglas.*

Práctica ▪ solución en la pág. 545

Ejercicio 14.4: Un aventurero solo acepta a su nombre ★★★★☆

La regla es: «un aventurero solo puede aceptar misiones a su nombre». Antes del capítulo, el servicio se fiaba del id que venía en el cuerpo. Entrega una página con:

1. **Dónde se comprueba.** Qué capa decide y por qué no `Quest.accept` ni `AcceptQuestService`: qué tendrían que importar o recibir para poder hacerlo.
2. **Cómo llega la identidad al caso de uso.** La firma de `AcceptQuestCommand` antes y después de tu diseño, y qué hace el controlador con el token.
3. **Qué responde el servicio** cuando el cuerpo trae otro id: código HTTP y por qué ese y no otro.
4. **Una alternativa**: una claim propia (por ejemplo `adventurer_id`) en lugar de usar el `sub`. Qué cambia en el emisor, en el controlador y en el dominio, y cuándo la

preferirías.

5. **Un caso nuevo**: el soporte del gremio necesita que un `GUILD_MASTER` acepte una misión *en nombre de* un aventurero. Dónde pondrías esa excepción y qué cambia en la firma del caso de uso.

6. **Tres pruebas** (comandos o frases) de que el núcleo no conoce JWT.

Criterios de valoración (no hay una única respuesta correcta): (1) la regla se ubica en el adaptador de entrada con el argumento del límite, no con «es más cómodo»; (2) la firma del comando no incorpora ningún tipo de seguridad (solo `QuestId` y `AdventurerId`); (3) el código elegido para el id distinto es 403 y el razonamiento distingue «no sé quién eres» (401) de «sé quién eres y no puedes» (403); (4) la alternativa nombra las tres piezas que cambian y un caso en el que compensa (por ejemplo, el `sub` identifica a una persona y no a un aventurero); (5) la excepción del soporte se resuelve sin tocar el dominio y deja claro quién queda registrado como actor; (6) al menos una de las pruebas es mecánica (un `grep` o una compilación del módulo sin Spring Security).

Diseño ▪ solución en la pág. 547

Ejercicio 14.5: Hito H7b: emisor, tres servicios protegidos y reenvío del token★★★★☆

Parte del proyecto con los ejercicios anteriores hechos: emisor, Quests con roles, `sub` como `AdventurerId` e interceptor. Falta lo que lo convierte en hito:

1. **Adventurers.** En `adapters` y `bootstrap`, el *starter* de resource server. Un `SecurityConfig` en `adapter/in/web/security` con el convertidor de la claim `roles`, `POST /adventurers` solo para `GUILD_MASTER` y el resto con cualquier token válido, y su `SecurityImportConfig` en `config`.

2. **Treasury.** Lo mismo, con todas sus rutas abiertas a cualquier token válido.

3. **Revisión de Quests.** Comprueba que están las siete piezas: las dos dependencias, el `SecurityConfig` del ejercicio 14.3, su `SecurityImportConfig`, el método `accept`, el interceptor y la fábrica con `requestInterceptor`.

4. **El recorrido.** Arranca en el orden de la sección 14.8.1, resuelve el paso del huevo y la gallina con `master` y repite todo el recorrido por el 8080 y las cinco pruebas de códigos.

5. Haz el *commit*.

Está bien si: `./gradlew build` termina con `BUILD SUCCESSFUL` (incluidas las pruebas de arquitectura); el recorrido da 150 de saldo y `ADEPT` para Mara con el token de `hero`; los cinco códigos de las pruebas de la sección 14.8.1 son 401, 403, 403, 401 y 401; Adventurers llamado en su puerto sin token da 401; un token de `hero` no puede registrar aventureros (403); el `grep` y el `git diff` del final de la sección 14.8.1 no muestran nada; y ningún `build.gradle` nombra a un proyecto de otro servicio.

Pista: *si el aceptar da 401 en lugar de 200 con* `hero` *y Adventurers ya está protegido, mira primero que el interceptor esté registrado en la fábrica.*

Pista: *si todos los tokens dan 401 en un solo servicio, ese servicio arrancó antes de que el Config Server tuviera el bloque* `security`*: reinícialo.*

Proyecto ▪ solución en la pág. 548

14.10 Resumen

Resumen

- Un JWT son tres partes en base64url (cabecera, claims, firma). Se lee con `cut`, `tr` y `base64 -d`, pero leer no es validar: el contenido no va cifrado y solo la firma, comprobada con la clave pública del emisor, demuestra que es auténtico.
- Un *resource server* valida firma, `exp`, `nbf` y, con `issuer-uri`, el `iss`. Con `jwk-set-uri` y `issuer-uri` a la vez no consulta al emisor al arrancar. Cada servicio valida por su cuenta; el gateway solo reenvía `Authorization`.
- Los roles llegan en una claim propia (`roles`); un `JwtGrantedAuthoritiesConverter` la traduce a autoridades `ROLE_*` y `hasRole` las comprueba. Las reglas se leen de arriba abajo, gana la primera que casa y `anyRequest()` va la última. Sin token es 401; con token y sin permiso, 403.
- La autorización va en el borde. Una regla de seguridad («solo el maestro publica») la resuelve el filtro; una regla de negocio («rango suficiente») sigue en el dominio; y «solo acepta a su nombre» la resuelve el adaptador, que toma el `sub` del token y lo pone en el `AcceptQuestCommand`. Dominio y aplicación no importan nada de seguridad.
- El token se reenvía de Quests a Adventurers con un `ClientHttpRequestInterceptor` del `RestClient` dentro del adaptador de salida. El puerto `AdventurerDirectory` no cambia. Funciona porque la llamada ocurre en el hilo de la petición. Los mensajes de RabbitMQ no llevan JWT.
- El emisor de desarrollo (`tools/token-issuer`) firma con `RS256` y publica un JWKS, pero no es seguridad de producción. Usa Nimbus, que llega con `spring-security-oauth2-jose`; no hizo falta añadir ninguna librería.
- La estructura cambió (verificación de estructura 9): `tools/token-issuer` en la raíz y un paquete `security` por servicio, activado desde `bootstrap`. `domain` y `application` no se tocaron.

Soluciones de los ejercicios

Solución del ejercicio 14.1 enunciado en la pág. 538

1. Decodificar. Con la función `b64url` de la sección 14.2 (`tr` deshace el cambio de símbolos, el bucle rellena con `=` y `base64 -d` decodifica):

```
$ b64url() {
>   local s
>   s=$(printf '%s' "$1" | tr '_-' '/+')
>   while [ $(( ${#s} % 4 )) -ne 0 ]; do s="$s="; done
>   printf '%s' "$s" | base64 -d
> }
$ b64url "$(echo "$TOKEN" | cut -d. -f1)"; echo
{"alg":"RS256","typ":"JWT","kid":"dev-key-1"}
$ b64url "$(echo "$TOKEN" | cut -d. -f2)"; echo
{"iss":"http://localhost:9001","sub":"hero-7","roles":["ADVENTURER"],
"iat":1767225600,"exp":1767226500}
```

(Los claims salen en una sola línea; aquí están partidos.)

2. Fechas.

```
$ date -u -d @1767225600 +%FT%TZ
2026-01-01T00:00:00Z
$ date -u -d @1767226500 +%FT%TZ
2026-01-01T00:15:00Z
```

El token se emitió a medianoche del 1 de enero de 2026 y valía quince minutos.

3. Veredicto por comprobación.

Comprobación	Resultado	Por qué
Firma	no decidible en la shell	Verificarla exige la clave pública del emisor y una librería de criptografía; `base64` solo lee. En este token la firma es un texto cualquiera, así que un servicio real también la rechazaría.
`exp`	falla	`2026-01-01T00:15:00Z` es anterior a hoy.
`nbf`	no aplica	El token no tiene esa claim: no hay «antes del cual no vale».
`iss`	falla	Trae `http://localhost:9001`; el servicio espera `http://localhost:9000`. La comparación es de texto exacto.

4. Autoridad. Con el convertidor de la sección 14.5, la claim `roles ["ADVENTURER"]` se convierte en la autoridad `ROLE_ADVENTURER`, y `hasRole("ADVENTURER")` comprueba justo esa. Pasaría la regla de rol *si* el token fuese válido; pero las reglas de rol se evalúan después de validar el token, y este no lo supera.

5. Cambiar `roles` a mano. Se puede: es solo base64url de un JSON, y un compañero puede codificar otro payload con `GUILD_MASTER`. El token resultante sigue decodificando bien, pero la firma se calculó sobre la cabecera y el payload *originales*. El servicio recalcula la verificación sobre lo que recibe, no coincide, y responde 401. La firma no impide leer, impide modificar.

Errores típicos: olvidar el relleno (`base64: invalid input`); olvidar el `tr` (la cadena tiene `-` o `_`); hacer `cut -f2` sin `-d.` (el separador por omisión es el tabulador); creer que, si el JSON sale bien, el token es auténtico.

Solución del ejercicio 14.2 enunciado en la pág. 539

Arranque. Cuatro terminales, en este orden (el emisor irá al final del paso 3):

```
$ docker compose up -d --wait rabbitmq
$ ./gradlew :config-server:bootRun           # terminal 1
$ ./gradlew :adventurers-bootstrap:bootRun   # terminal 2
$ ./gradlew :quests-bootstrap:bootRun        # terminal 3
```

En una quinta terminal, las funciones de ayuda y el paso 2. La función `code` imprime solo el código HTTP:

```
$ jid() { grep -o '"id":"[^"]*"' | head -1 | cut -d'"' -f4; }
$ code() { curl -s -o /dev/null -w '%{http_code}\n' "$@"; }
$ MARA=$(curl -s -X POST localhost:8082/adventurers \
    -H 'Content-Type: application/json' -d '{"name":"Mara"}' | jid)
$ echo $MARA
```

Paso 3. Edita `tools/token-issuer/src/main/resources/application.yml`: el valor de `subject` del usuario `hero` pasa a ser lo que imprimió `echo`. Después, el emisor (terminal 4):

```
$ ./gradlew :token-issuer:bootRun            # terminal 4
```

Paso 4. Los tokens, con la función `tok` de la sección 14.8, y los claims de `HERO` con `b64url` (sección 14.2):

```
$ MASTER=$(tok master)
$ HERO=$(tok hero)
$ b64url "$(echo "$HERO" | cut -d. -f2)"; echo
{"sub":"<id de Mara>","roles":["ADVENTURER"],
"iss":"http://localhost:9000","exp":...,"iat":...}
```

Pasos 5 a 8. Todo contra Quests (8081):

```
$ Q1=$(curl -s -X POST localhost:8081/quests \
    -H "Authorization: Bearer $MASTER" \
    -H 'Content-Type: application/json' \
    -d '{"title":"Limpiar el s^^c3^^b3tano","reward":150,
         "requiredRank":"NOVICE"}' | jid)
$ code localhost:8081/quests/$Q1
401
$ code localhost:8081/quests/$Q1 -H "Authorization: Bearer $HERO"
200
$ curl -s -X POST localhost:8081/quests/$Q1/accept \
    -H "Authorization: Bearer $HERO"
$ Q2=$(curl -s -X POST localhost:8081/quests \
    -H "Authorization: Bearer $MASTER" \
    -H 'Content-Type: application/json' \
    -d '{"title":"Cazar ratas","reward":50,
         "requiredRank":"NOVICE"}' | jid)
$ code -X POST localhost:8081/quests/$Q2/accept \
    -H "Authorization: Bearer $HERO" \
    -H 'Content-Type: application/json' \
    -d '{"adventurerId":"otro"}'
403
```

El paso 5 (publicar) responde 201, que no ves porque `jid` consume la respuesta; el paso 7 devuelve la misión en JSON con `"status":"ACCEPTED"` y `"assignedTo"` igual a `MARA`.
Por qué sale cada código. Sin cabecera, ningún filtro autentica y Spring responde 401. Con `HERO` el token es válido (firma, `exp` e `iss`) y el controlador contesta. Aceptar no lleva cuerpo: el controlador toma el `sub`, que es `MARA`, y el caso de uso se ejecuta como Mara. En el paso 8, el cuerpo trae `otro`, que no coincide con el `sub`: el controlador responde 403 antes de llamar al caso de uso.
Errores típicos: arrancar Quests antes que el Config Server (sin las propiedades de `security`, no hay `JwtDecoder` y el servicio ni siquiera arranca: reiníciala); olvidar reiniciar el emisor tras editar `subject` (el `sub` sigue siendo el marcador y el paso 7 da 404 por `AdventurerNotFound`); usar un token después de reiniciar el emisor (clave nueva, 401); y escribir mal el prefijo `Bearer` (con otra palabra, el servicio ve una petición sin token).

Solución del ejercicio 14.3

enunciado en la pág. 540

El archivo completo. Las reglas van antes de `anyRequest()` y el convertidor se pasa al decodificador:

adapter/in/web/security/SecurityConfig.java (Quests, 1 de 2)

```java
package com.guildboard.quests.adapter.in.web.security;

import org.springframework.context.annotation.Bean;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.
    ↪ HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.oauth2.server.resource.authentication.
    ↪ JwtAuthenticationConverter;
import org.springframework.security.oauth2.server.resource.authentication.
    ↪ JwtGrantedAuthoritiesConverter;
import org.springframework.security.web.SecurityFilterChain;

public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.POST, "/quests")
                    .hasRole("GUILD_MASTER")
                .requestMatchers(HttpMethod.POST, "/quests/*/accept")
                    .hasRole("ADVENTURER")
                .requestMatchers(HttpMethod.POST, "/quests/*/complete")
                    .hasAnyRole("GUILD_MASTER", "ADVENTURER")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth -> oauth
                .jwt(jwt -> jwt
                    .jwtAuthenticationConverter(rolesConverter())));
        return http.build();
    }
```

adapter/in/web/security/SecurityConfig.java (Quests, 2 de 2: sigue dentro de la clase)

```java
    private static JwtAuthenticationConverter rolesConverter() {
        JwtGrantedAuthoritiesConverter authorities =
            new JwtGrantedAuthoritiesConverter();
        authorities.setAuthoritiesClaimName("roles");
        authorities.setAuthorityPrefix("ROLE_");
        JwtAuthenticationConverter converter =
            new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(authorities);
        return converter;
    }
}
```

La tabla. Con `code` (solución del ejercicio 14.2), una misión `QA` publicada en (a) y una `QB` publicada también por `MASTER`:

Petición	Código	Quién responde
(a) `MASTER` publica	201	el controlador
(b) `HERO` publica	403	la regla de rol
(c) `HERO` acepta `QA`, sin cuerpo	200	el controlador
(d) `MASTER` acepta `QB`	403	la regla de rol
(e) publicar sin token	401	el filtro de autenticación
(f) `HERO` completa `QA`	200	el controlador
(g) `MASTER` hace `GET` de `QA`	200	`anyRequest()`

Por qué (e) y (b) difieren. En (e) no hay identidad: Spring no sabe quién llama y responde 401 («identifícate»). En (b) el token es válido y la identidad es `HERO`, pero su autoridad `ROLE_ADVENTURER` no cumple la regla, y responde 403 («sé quién eres, y no puedes»).
Por qué funciona. `JwtGrantedAuthoritiesConverter` lee la lista de la claim `roles` y antepone `ROLE_` a cada valor: `GUILD_MASTER` pasa a ser `ROLE_GUILD_MASTER`, que es lo que `hasRole("GUILD_MASTER")` busca. El patrón `/quests/*/accept` casa con un solo segmento en medio, así que no acepta rutas más largas. Las reglas se comprueban de arriba abajo y gana la primera que casa.
Errores típicos: no pasar el convertidor a `.jwt(...)` (queda `Customizer.withDefaults()` y las autoridades son `SCOPE_*`: todo da 403); escribir `hasRole("ROLE_ADVENTURER")` (Spring lo rechaza al arrancar porque añade el prefijo por su cuenta); poner `anyRequest()` antes de las reglas específicas (las siguientes ya no se pueden configurar: error al arrancar); olvidar reiniciar Quests; y escribir el patrón con `{id}` en lugar del asterisco.

Solución del ejercicio 14.4 enunciado en la pág. 540

Referencia.
1. Dónde. En el adaptador de entrada. La regla compara dos datos, el `sub` del token y el id que quiere actuar; el primero solo existe en HTTP. Para comprobarlo en `Quest.accept` o en `AcceptQuestService`, el dominio o la aplicación tendrían que recibir un `Jwt` (o un «usuario actual») y, por tanto, importar Spring Security: la regla de dependencia se rompe y el dominio ya no se puede probar sin un token. Es la frontera de Martin [28], capítulo 17: el detalle queda fuera.
2. La identidad. `AcceptQuestCommand(QuestId questId, AdventurerId adventurerId)` no cambia. Antes, el `AdventurerId` lo construía el controlador con el id del cuerpo; ahora lo construye con `jwt.getSubject()`. El caso de uso ve lo mismo y no sabe de dónde salió.
3. Otro id en el cuerpo. 403. El token es válido (no es un 401: Spring sí sabe quién llama), y esa identidad no tiene permiso para actuar como otro. No es un 400, porque el cuerpo está bien formado, ni un 404, porque el aventurero puede existir.
4. Alternativa: claim `adventurer_id`. Se añade al emisor (`.claim("adventurer_id", ...)` en `TokenController`, con un campo más en `IssuerProperties.User`), y el controlador lee `jwt.getClaimAsString("adventurer_id")` en lugar del `sub`; el dominio no cambia. Se prefiere cuando el `sub` identifica a una persona de otro sistema (un UUID de Keycloak, un correo) que no es un `AdventurerId`: la claim hace explícita la relación y permite que haya personas sin aventurero (si falta, 403). Cuesta una claim más que el emisor debe garantizar. En GuildBoard, que el `sub` *sea* el `AdventurerId` evita el emparejamiento; es válido porque emisor y servicios los controla el mismo equipo.
5. Un `GUILD_MASTER` en nombre de un aventurero. En el mismo adaptador, sin tocar el caso de uso. La regla de seguridad pasa a `hasAnyRole("ADVENTURER", "GUILD_MASTER")`

en `SecurityConfig`, y el controlador decide quién es el aventurero que actúa:

`QuestController.java (variante: el maestro actúa en nombre de otro)`

```java
@PostMapping("/{id}/accept")
public QuestResponse accept(
        @PathVariable String id,
        @AuthenticationPrincipal Jwt jwt,
        Authentication authentication,
        @RequestBody(required = false) AcceptQuestRequest request) {
    boolean master = authentication.getAuthorities().stream()
        .anyMatch(a -> a.getAuthority().equals("ROLE_GUILD_MASTER"));
    String requested = request == null ? null : request.adventurerId();
    String actor = jwt.getSubject();
    if (master) {
        if (requested == null || requested.isBlank()) {
            throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
                "a master must say which adventurer");
        }
        actor = requested;
    } else if (requested != null && !requested.equals(actor)) {
        throw new ResponseStatusException(HttpStatus.FORBIDDEN,
            "the token does not belong to that adventurer");
    }
    Quest quest = acceptQuest.accept(new AcceptQuestCommand(
        new QuestId(id), new AdventurerId(actor)));
    return QuestWebMapper.toResponse(quest);
}
```

(Imports extra: `org.springframework.security.core.Authentication`.) La firma del caso de uso sigue igual. Lo que se pierde es que el servicio ya no sabe quién *ordenó* la aceptación: si el negocio necesita auditarlo, es un dato nuevo del comando (por ejemplo, un `requestedBy`) y entonces sí cambia la firma; habría que decidirlo con el negocio, no por comodidad.

6. Tres pruebas. (i) `grep -rln springframework services/*/domain/src/main services/*/application/src/main` no muestra nada. (ii) `./gradlew :quests-application:compileJava` compila, aunque se quite el *starter* de seguridad de `quests-adapters`: `quests-application` nunca lo tuvo. (iii) Las pruebas de dominio y de aplicación (`:quests-domain:test`) corren sin Spring, y `AcceptQuestCommand` sigue teniendo dos campos, ninguno de seguridad.

Criterios para valorar otras soluciones. Los seis de arriba: ubicar la regla en el adaptador con el argumento del límite; un comando sin tipos de seguridad; 403 razonado frente a 401; una alternativa con las tres piezas (emisor, controlador, dominio) y un caso de uso; una excepción resuelta sin tocar el dominio, con el coste de perder al actor; y al menos una prueba mecánica. **Una alternativa válida** que cambia la ubicación: un filtro o un `HandlerInterceptor` que compare el id de la ruta con el `sub` antes del controlador. Es válida si sigue dentro del adaptador de entrada; es peor si la comprobación se reparte entre filtro y controlador sin que ninguno de los dos la nombre.

Solución del ejercicio 14.5

enunciado en la pág. 541

0. Lo que no cambia. El emisor son los archivos de la sección 14.3 (`build.gradle`, las cinco clases Java y `application.yml`); solo se edita el `subject` de `hero` cuando tienes el

id de Mara. `domain` y `application` de los tres servicios no se tocan.

1. `settings.gradle` completo.

settings.gradle

```
rootProject.name = 'guildboard'

// Un proyecto Gradle por m^^c3^^b3dulo y servicio, con nombre ^^c3^^banico
// (quests-domain, adventurers-domain, treasury-domain...).
['quests', 'adventurers', 'treasury'].each { service ->
    ['domain', 'application', 'adapters', 'bootstrap'].each { module ->
        def projectName = "${service}-${module}"
        include projectName
        project(":${projectName}").projectDir =
            file("services/${service}/${module}")
    }
}

// Infraestructura sin hex^^c3^^a1gono: la carpeta se llama como el
    ↪ proyecto.
include 'config-server', 'gateway'

// Herramienta de desarrollo, fuera de services/.
include 'token-issuer'
project(':token-issuer').projectDir = file('tools/token-issuer')
```

2. La dirección del emisor en el archivo común.

config-repo/application.yml

```
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: guildboard
    password: guildboard
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:9000
          jwk-set-uri: http://localhost:9000/.well-known/jwks.json

guildboard:
  messaging:
    exchange: guildboard.events
    routing-key: quest.completed
```

3. Quests: los dos `build.gradle`.

services/quests/adapters/build.gradle

```
dependencies {
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-security-oauth2-resource-server'
    implementation 'io.github.resilience4j:' +
        'resilience4j-spring-boot4:2.4.0'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

services/quests/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencyManagement {
    imports {
        mavenBom 'org.springframework.cloud:' +
            'spring-cloud-dependencies:2025.1.3'
    }
}

dependencies {
    implementation project(':quests-adapters')
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-security-oauth2-resource-server'
    implementation 'io.github.resilience4j:' +
        'resilience4j-spring-boot4:2.4.0'
    implementation 'org.springframework.cloud:' +
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
    runtimeOnly 'org.postgresql:postgresql'
}
```

4. Quests: seguridad. El `SecurityConfig` es el de la solución del ejercicio 14.3, que repito para que el ejercicio quede entero:

adapter/in/web/security/SecurityConfig.java (Quests, 1 de 2)

```java
package com.guildboard.quests.adapter.in.web.security;

import org.springframework.context.annotation.Bean;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.
    ↪ HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.oauth2.server.resource.authentication.
    ↪ JwtAuthenticationConverter;
import org.springframework.security.oauth2.server.resource.authentication.
    ↪ JwtGrantedAuthoritiesConverter;
import org.springframework.security.web.SecurityFilterChain;

public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.POST, "/quests")
                    .hasRole("GUILD_MASTER")
                .requestMatchers(HttpMethod.POST, "/quests/*/accept")
                    .hasRole("ADVENTURER")
                .requestMatchers(HttpMethod.POST, "/quests/*/complete")
                    .hasAnyRole("GUILD_MASTER", "ADVENTURER")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth -> oauth
                .jwt(jwt -> jwt
                    .jwtAuthenticationConverter(rolesConverter())));
        return http.build();
    }
```

adapter/in/web/security/SecurityConfig.java (Quests, 2 de 2: sigue dentro de la clase)

```java
    private static JwtAuthenticationConverter rolesConverter() {
        JwtGrantedAuthoritiesConverter authorities =
            new JwtGrantedAuthoritiesConverter();
        authorities.setAuthoritiesClaimName("roles");
        authorities.setAuthorityPrefix("ROLE_");
        JwtAuthenticationConverter converter =
            new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(authorities);
        return converter;
    }
}
```

config/SecurityImportConfig.java (Quests)

```java
package com.guildboard.quests.config;

import com.guildboard.quests.adapter.in.web.security.SecurityConfig;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Import;

@Configuration
@Import(SecurityConfig.class)
public class SecurityImportConfig {
}
```

5. Quests: el DTO y el controlador.

adapter/in/web/dto/AcceptQuestRequest.java

```java
package com.guildboard.quests.adapter.in.web.dto;

public record AcceptQuestRequest(String adventurerId) {
}
```

adapter/in/web/QuestController.java (1 de 2)

```java
package com.guildboard.quests.adapter.in.web;

import com.guildboard.quests.adapter.in.web.dto.AcceptQuestRequest;
import com.guildboard.quests.adapter.in.web.dto.PublishQuestRequest;
import com.guildboard.quests.adapter.in.web.dto.QuestResponse;
import com.guildboard.quests.adapter.in.web.mapper.QuestWebMapper;
import com.guildboard.quests.application.port.in.AcceptQuestCommand;
import com.guildboard.quests.application.port.in.AcceptQuestUseCase;
import com.guildboard.quests.application.port.in.CompleteQuestCommand;
import com.guildboard.quests.application.port.in.CompleteQuestUseCase;
import com.guildboard.quests.application.port.in.GetQuestUseCase;
import com.guildboard.quests.application.port.in.PublishQuestUseCase;
import com.guildboard.quests.domain.AdventurerId;
import com.guildboard.quests.domain.Quest;
import com.guildboard.quests.domain.QuestId;
import jakarta.validation.Valid;
import java.net.URI;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

@RestController
@RequestMapping("/quests")
@RequiredArgsConstructor
public class QuestController {

    private final PublishQuestUseCase publishQuest;
    private final GetQuestUseCase getQuest;
    private final AcceptQuestUseCase acceptQuest;
    private final CompleteQuestUseCase completeQuest;

    @PostMapping
    public ResponseEntity<QuestResponse> publish(
            @Valid @RequestBody PublishQuestRequest request) {
        Quest quest = publishQuest.publish(
            QuestWebMapper.toCommand(request));
        return ResponseEntity
            .created(URI.create("/quests/" + quest.getId().value()))
            .body(QuestWebMapper.toResponse(quest));
    }
```

adapter/in/web/QuestController.java (2 de 2: sigue dentro de la clase)

```
    @GetMapping("/{id}")
    public ResponseEntity<QuestResponse> get(@PathVariable String id) {
        return getQuest.get(new QuestId(id))
            .map(QuestWebMapper::toResponse)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping("/{id}/accept")
    public QuestResponse accept(
            @PathVariable String id,
            @AuthenticationPrincipal Jwt jwt,
            @RequestBody(required = false) AcceptQuestRequest request) {
        String subject = jwt.getSubject();
        if (request != null && request.adventurerId() != null
                && !request.adventurerId().equals(subject)) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN,
                "the token does not belong to that adventurer");
        }
        Quest quest = acceptQuest.accept(new AcceptQuestCommand(
            new QuestId(id), new AdventurerId(subject)));
        return QuestWebMapper.toResponse(quest);
    }

    @PostMapping("/{id}/complete")
    public QuestResponse complete(@PathVariable String id) {
        Quest quest = completeQuest.complete(
            new CompleteQuestCommand(new QuestId(id)));
        return QuestWebMapper.toResponse(quest);
    }
}
```

6. Quests: el reenvío. El interceptor y la fábrica, enteros:

adapter/out/rest/BearerTokenForwardingInterceptor.java

```java
package com.guildboard.quests.adapter.out.rest;

import java.io.IOException;
import org.springframework.http.HttpRequest;
import org.springframework.http.client.ClientHttpRequestExecution;
import org.springframework.http.client.ClientHttpRequestInterceptor;
import org.springframework.http.client.ClientHttpResponse;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.oauth2.server.resource.authentication.
    ↪ JwtAuthenticationToken;

public class BearerTokenForwardingInterceptor
        implements ClientHttpRequestInterceptor {

    @Override
    public ClientHttpResponse intercept(HttpRequest request, byte[] body,
            ClientHttpRequestExecution execution) throws IOException {
        Authentication current =
            SecurityContextHolder.getContext().getAuthentication();
        if (current instanceof JwtAuthenticationToken jwt) {
            request.getHeaders()
                .setBearerAuth(jwt.getToken().getTokenValue());
        }
        return execution.execute(request, body);
    }
}
```

adapter/out/rest/AdventurerClientFactory.java

```java
package com.guildboard.quests.adapter.out.rest;

import java.net.http.HttpClient;
import java.time.Duration;
import org.springframework.http.client.JdkClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

public final class AdventurerClientFactory {

    private AdventurerClientFactory() {
    }

    public static RestClient create(String baseUrl,
            Duration connectTimeout, Duration readTimeout) {
        HttpClient http = HttpClient.newBuilder()
            .connectTimeout(connectTimeout)
            .build();
        JdkClientHttpRequestFactory factory =
            new JdkClientHttpRequestFactory(http);
        factory.setReadTimeout(readTimeout);
        return RestClient.builder()
            .baseUrl(baseUrl)
            .requestFactory(factory)
            .requestInterceptor(new BearerTokenForwardingInterceptor())
            .build();
    }
}
```

7. Adventurers. Los dos `build.gradle`, `SecurityConfig` (solo el registro exige el maestro) y su importación:

services/adventurers/adapters/build.gradle

```
dependencies {
    implementation project(':adventurers-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-security-oauth2-resource-server'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

services/adventurers/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencyManagement {
    imports {
        mavenBom 'org.springframework.cloud:' +
            'spring-cloud-dependencies:2025.1.3'
    }
}

dependencies {
    implementation project(':adventurers-adapters')
    implementation project(':adventurers-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-security-oauth2-resource-server'
    implementation 'org.springframework.cloud:' +
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
    runtimeOnly 'org.postgresql:postgresql'
}
```

adapter/in/web/security/SecurityConfig.java (Adventurers, 1 de 2)

```
package com.guildboard.adventurers.adapter.in.web.security;

import org.springframework.context.annotation.Bean;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.
    ↪ HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.oauth2.server.resource.authentication.
    ↪ JwtAuthenticationConverter;
import org.springframework.security.oauth2.server.resource.authentication.
    ↪ JwtGrantedAuthoritiesConverter;
import org.springframework.security.web.SecurityFilterChain;

public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.POST, "/adventurers")
                    .hasRole("GUILD_MASTER")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth -> oauth
                .jwt(jwt -> jwt
                    .jwtAuthenticationConverter(rolesConverter())));
        return http.build();
    }
```

adapter/in/web/security/SecurityConfig.java (Adventurers, 2 de 2: sigue dentro de la clase)

```
    private static JwtAuthenticationConverter rolesConverter() {
        JwtGrantedAuthoritiesConverter authorities =
            new JwtGrantedAuthoritiesConverter();
        authorities.setAuthoritiesClaimName("roles");
        authorities.setAuthorityPrefix("ROLE_");
        JwtAuthenticationConverter converter =
            new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(authorities);
        return converter;
    }
}
```

config/SecurityImportConfig.java (Adventurers)

```java
package com.guildboard.adventurers.config;

import com.guildboard.adventurers.adapter.in.web.security.SecurityConfig;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Import;

@Configuration
@Import(SecurityConfig.class)
public class SecurityImportConfig {
}
```

8. Treasury. Lo mismo, con todas las rutas abiertas a cualquier token válido:

services/treasury/adapters/build.gradle

```
dependencies {
    implementation project(':treasury-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-validation
    ↪ '
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-security-oauth2-resource-server'

    compileOnly 'org.projectlombok:lombok:1.18.48'
    annotationProcessor 'org.projectlombok:lombok:1.18.48'
}
```

services/treasury/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencyManagement {
    imports {
        mavenBom 'org.springframework.cloud:' +
            'spring-cloud-dependencies:2025.1.3'
    }
}

dependencies {
    implementation project(':treasury-adapters')
    implementation project(':treasury-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-security-oauth2-resource-server'
    implementation 'org.springframework.cloud:' +
        'spring-cloud-starter-config'
    runtimeOnly 'org.postgresql:postgresql'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
```

adapter/in/web/security/SecurityConfig.java (Treasury, 1 de 2)

```java
package com.guildboard.treasury.adapter.in.web.security;

import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.
    ↪ HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.oauth2.server.resource.authentication.
    ↪ JwtAuthenticationConverter;
import org.springframework.security.oauth2.server.resource.authentication.
    ↪ JwtGrantedAuthoritiesConverter;
import org.springframework.security.web.SecurityFilterChain;

public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth -> oauth
                .jwt(jwt -> jwt
                    .jwtAuthenticationConverter(rolesConverter())));
        return http.build();
    }
```

adapter/in/web/security/SecurityConfig.java (Treasury, 2 de 2: sigue dentro de la clase)

```java
    private static JwtAuthenticationConverter rolesConverter() {
        JwtGrantedAuthoritiesConverter authorities =
            new JwtGrantedAuthoritiesConverter();
        authorities.setAuthoritiesClaimName("roles");
        authorities.setAuthorityPrefix("ROLE_");
        JwtAuthenticationConverter converter =
            new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(authorities);
        return converter;
    }
}
```

config/SecurityImportConfig.java (Treasury)

```java
package com.guildboard.treasury.config;

import com.guildboard.treasury.adapter.in.web.security.SecurityConfig;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Import;

@Configuration
@Import(SecurityConfig.class)
public class SecurityImportConfig {
}
```

9. Comprobar. El orden de arranque y el recorrido son los de la sección 14.8.1: servidor de configuración, emisor, los tres servicios y el gateway; token de `master`; registrar a Mara; editar `subject` de `hero`, reiniciar el emisor; publicar con `master`; aceptar y completar con `hero`; leer el saldo (150) y el rango (`ADEPT`) con `hero`. Las pruebas de códigos dan 401, 403, 403, 401 y 401; un registro con el token de `hero` da 403. Después, `./gradlew build`, el `git diff` y el `grep`, y el *commit* (`H7b: seguridad con JWT`).

Por qué funciona. Cada servicio resuelve el `JwtDecoder` con las dos propiedades del archivo común y valida por sí mismo; el gateway nunca interviene. En la aceptación, el `sub` de `hero` es el id de Mara, que Adventurers sí conoce: el interceptor copia el token de la petición actual, y Adventurers lo valida con la misma clave pública. El evento `QuestCompleted` no lleva token y Treasury paga igual, porque la validación afecta a HTTP, no al *broker*.

Errores típicos: olvidar `SecurityImportConfig` (no hay cadena de seguridad propia y Spring Boot aplica la suya por omisión: todo pide una contraseña generada); olvidar `POST /adventurers` del maestro y que `hero` se registre a sí mismo; arrancar un servicio antes que el Config Server (no arranca por falta de `JwtDecoder`); y escribir el `subject` de `hero` y no reiniciar el emisor (el `sub` sigue siendo el marcador y aceptar da 404).

CAPÍTULO 15

Observabilidad y todo el sistema en Docker Compose

Objetivos del capítulo

- Distinguir métricas, trazas y logs, y activar las dos primeras con Actuator, Micrometer y un puente de trazas hacia Zipkin sin tocar `domain` ni `application`.
- Seguir una petición por varios servicios con un `traceId` que aparece en los logs y en Zipkin, y saber qué hace falta para que el contexto cruce una llamada REST y un mensaje de RabbitMQ.
- Empaquetar cada servicio con un `Dockerfile` y levantar todo GuildBoard con un solo comando, entendiendo la red de Compose, los nombres de servicio frente a `localhost`, los *healthchecks* y el perfil `docker`.
- Comprobar en la verificación de estructura 10 el árbol final del proyecto contra la lista de comprobación y la regla de dependencia.

Desde el capítulo anterior arrancas el sistema a mano: RabbitMQ en Compose y seis procesos de Gradle en seis terminales, cada uno con su puerto y su orden. Si algo falla (la experiencia de Mara no llega, una aceptación tarda tres segundos), no sabes por dónde empezar a mirar. Este capítulo cierra el proyecto con dos capacidades que cualquier sistema distribuido necesita antes de salir de tu máquina: **poder ver lo que hace** y **poder levantarlo entero con una orden**. Lo primero es observabilidad; lo segundo, empaquetado y despliegue local.

Cinco cosas que el capítulo deja dichas desde el principio:

- **Es la última verificación de estructura.** Entran un `Dockerfile` en cada proyecto ejecutable, un archivo de perfil en `config-repo` y el `compose.yaml` completo. No se añade ningún proyecto Gradle (siguen siendo quince), pero se fija el **árbol final** del curso. La sección 15.1 lo comprueba antes de construir nada, y el ejercicio 15.4 te lo hace comprobar a ti.
- **El núcleo no se entera.** La observabilidad se añade con dependencias y configuración en `bootstrap` y en `config-repo`. Fuera de ahí hay solo dos cambios de código, pequeños y con razón: la fábrica del `RestClient` de Quests (sección 15.4) y una regla pública para `/actuator/health/**` en cada `SecurityConfig` (sección 15.3). El `application-postgres.yml` de cada servicio gana además un valor por omisión del *host*.
- **El código Java no se ha compilado ni ejecutado**, como en todo el curso. Lo que sí se comprobó (artefactos, propiedades, etiquetas de imagen, la sintaxis del `Dockerfile` y de Compose, y el arranque real de PostgreSQL, RabbitMQ y Zipkin en Compose) está marcado con su fecha en la sección 15.7.3.
- **Hay una pieza diseñada pero no verificada:** que el `traceId` cruce RabbitMQ. Si en tu ejecución no lo hace, el capítulo se queda en REST y logs, y la sección 15.4 dice cómo reconocerlo.

- **No hay métricas de negocio propias.** Se recortó para acotar la carga: medirás peticiones, JVM y mensajes, no «misiones completadas».

15.1 Verificación de estructura 10: el árbol final

Verificación de estructura 10

Este capítulo no añade proyectos Gradle ni carpetas de servicio, pero toca casi todos los proyectos ejecutables de la raíz y deja el árbol final del curso. Se comprueba aquí, antes de crear nada.

Para leer el árbol basta saber, por ahora, qué hace cada pieza nueva. Un **Dockerfile** describe cómo meter un programa ya construido en una imagen de contenedor. Un **perfil** `docker` es un conjunto de valores de configuración que solo se aplica cuando se activa ese nombre. El **compose.yaml** lista los contenedores del sistema y cómo se conectan. Las secciones siguientes explican cómo funciona cada una; aquí solo importa dónde van.

15.1.1 El árbol anterior

Así quedó el proyecto tras el capítulo 14 (árbol T8):

Árbol anterior (T8, tras el capítulo 14)

```
guildboard/
|-- .git/, .gitignore, gradlew, gradlew.bat
|-- settings.gradle        quince proyectos Gradle
|-- build.gradle           configuraci^^c3^^b3n com^^c3^^ban (subprojects)
|-- compose.yaml           PostgreSQL 18 y RabbitMQ 4
|-- docs/dominio.md
|-- gradle/wrapper/
|-- infra/postgres/init.sql
|-- config-repo/           application.yml, quests.yml, adventurers.yml,
|                          treasury.yml, gateway.yml
|-- config-server/         puerto 8888
|-- gateway/               puerto 8080
|-- tools/token-issuer/    puerto 9000
`-- services/
    |-- quests/            domain/ application/ adapters/ bootstrap/
    |-- adventurers/       domain/ application/ adapters/ bootstrap/
    `-- treasury/          domain/ application/ adapters/ bootstrap/
(m^^c3^^a1s StructureTest en src/test/java de la ra^^c3^^adz y un
    ↪ ArchitectureTest en
 el bootstrap de cada servicio)
```

15.1.2 El árbol final

`[N]` es nuevo y `[M]` modificado. Primero la raíz:

Árbol final, raíz (T9, tras el capítulo 15)

```
guildboard/
|-- .git/, .gitignore, gradlew, gradlew.bat      sin cambios
```

```
|-- settings.gradle        sin cambios: siguen siendo quince proyectos
|-- build.gradle, docs/, gradle/, infra/         sin cambios
|-- src/test/java/com/guildboard/StructureTest.java   sin cambios
|-- compose.yaml           [M] completo: nueve servicios
|-- config-repo/
|   |-- application.yml    [M] + management, trazas y observaci^^c3^^b3n de
|   |                      RabbitMQ
|   |-- application-docker.yml   [N] perfil docker: nombres de servicio
|   `-- quests.yml, adventurers.yml, treasury.yml, gateway.yml
|                          sin cambios
|-- config-server/         [M] build.gradle (bootJar); Dockerfile [N]
|-- gateway/               [M] build.gradle (actuator, zipkin, bootJar);
|                          Dockerfile [N]
|-- tools/token-issuer/    [M] build.gradle (bootJar); Dockerfile [N]
`-- services/              (ver abajo)
```

Los tres `ArchitectureTest` (uno en el `bootstrap` de cada servicio, en `src/test/java`) y el `StructureTest` de la raíz **no cambian**: son el árbol final de pruebas de arquitectura. Y lo que cambia dentro de cada servicio (el ejemplo es Quests; Adventurers y Treasury llevan lo mismo salvo las marcas de Quests):

services/quests (solo lo que cambia)

```
services/quests/
|-- domain/, application/                      sin cambios
|-- adapters/
|   `-- src/main/java/com/guildboard/quests/adapter/
|       |-- in/web/security/SecurityConfig.java       [M] regla de
|       |                                      /actuator/health
|       `-- out/rest/                          solo en Quests
|           |-- AdventurerClientFactory.java   [M] acepta un builder
|           `-- RestAdventurerDirectory.java   [M] un constructor m^^c3^^a1s
`-- bootstrap/
    |-- Dockerfile                             [N]
    |-- build.gradle                           [M] actuator, zipkin,
    |                                          bootJar (y restclient)
    `-- src/main/
        |-- java/com/guildboard/quests/config/
        |   `-- BeanConfig.java                [M] solo en Quests
        `-- resources/application-postgres.yml [M] host con valor
                                               por omisi^^c3^^b3n
```

Ningún archivo de `domain` ni de `application` cambia en ningún servicio, y en ningún `bootstrap` hay un `application-docker.yml`: la configuración del perfil `docker` vive en un único sitio, `config-repo`.

15.1.3 Cambios respecto al árbol anterior

- `Dockerfile` [N] en cada proyecto ejecutable (seis, idénticos), junto al `build.gradle`, que es el contexto de construcción de la imagen (sección 15.5). Copia el `bootJar` de [51].
- `config-repo/application-docker.yml` [N]: un único archivo de perfil que el Config Server sirve a quien pida ese perfil [56]. Contiene las rutas del gateway y el resto de claves.
- `compose.yaml` [M]: completo, con nueve servicios; formato y *healthchecks* de [5].

- `SPRING_PROFILES_ACTIVE` y `SPRING_CONFIG_IMPORT` como variables de entorno: una variable de entorno gana a `application.yml` [52], así que no hace falta ningún archivo nuevo en `bootstrap`.
- `adapters` [M] (`SecurityConfig`, fábrica del `RestClient`): sección 15.3 y sección 15.4; ambos están en el lado de los detalles ([28], capítulo 17 «Boundaries: Drawing Lines»).
- Ningún proyecto Gradle nuevo: `settings.gradle`, los tres `ArchitectureTest` y el `StructureTest` no cambian.

15.1.4 Comprobaciones

- [ ] `./gradlew test` pasa: incluye los tres `ArchitectureTest` y el `StructureTest`, que cubren que `domain` y `application` no importan frameworks, que `application` solo depende de `domain`, que `in` y `out` no se miran entre sí. Con el *classpath* de un solo servicio, la regla de que los servicios no se nombran pasa de forma trivial: la independencia real la garantiza que su `build.gradle` no declare al otro servicio, y Gradle impide compilar sin declararlo.
- [ ] Cada adaptador está en `in` o en `out`: la regla de `/actuator/health` en `SecurityConfig` recibe la petición (`in`); la fábrica del `RestClient` sale hacia Adventurers (`out`).
- [ ] Los nombres de carpeta dicen qué hace el sistema y no qué framework usa (`Dockerfile` y `compose.yaml` son los nombres que exige su herramienta; `infra`, `tools` y `config-repo` dicen la función).
- [ ] Los cambios respecto al árbol anterior son los de la lista de arriba: seis `Dockerfile`, un archivo de perfil, la ampliación de `compose.yaml`, dependencias de operación en seis `build.gradle`, una regla de seguridad por servicio y la fábrica del cliente en Quests.
- [ ] Ningún `build.gradle` de un servicio nombra a un proyecto de otro (el `depends_on` de Compose ordena contenedores, no es dependencia de código).
- [ ] Ningún proyecto de la raíz (gateway, Config Server, emisor) contiene lógica de negocio.
- [ ] Ningún módulo de `domain` ni de `application` menciona Micrometer, Actuator, Zipkin ni Docker.

Las pruebas no vigilan los nombres de Micrometer, Actuator, Zipkin ni Docker: esa séptima casilla se comprueba con una orden, desde la raíz, que no debe imprimir nada:

```
$ grep -rliE 'micrometer|actuator|zipkin|docker' \
    services/*/domain/src services/*/application/src
```

Las casillas segunda a sexta se revisan a mano, como las de las nueve verificaciones anteriores.

La regla de dependencia de [27] (el código fuente apunta solo hacia dentro) se cumple por construcción en el despliegue: el `Dockerfile` y el `compose.yaml` dependen del *jar* que construye Gradle, y ningún código depende de ellos. Docker y Compose son detalles del anillo exterior.

15.2 Tres señales: métricas, trazas y logs

Un sistema repartido en procesos se vigila con tres tipos de datos. Ninguno sustituye a los otros; cada uno responde a una pregunta distinta:

- Las **métricas** son números agregados en el tiempo: cuántas peticiones por segundo, cuánto tarda la más lenta, cuánta memoria usa la JVM, cuántos mensajes procesó un consumidor. Responden a «¿cómo va el sistema?». Son baratas y sirven para alertar, pero no te dicen qué le pasó a *una* petición.
- Los **logs** son líneas de texto que un proceso escribe cuando ocurre algo. Responden a «¿qué pasó en este servicio y por qué?». Los has usado desde el capítulo 4. Su límite es que cada servicio escribe en su propio flujo, sin saber qué línea de otro servicio pertenece a la misma petición.
- Las **trazas** siguen *una* petición por todos los procesos que toca. Responden a «¿por dónde pasó y cuánto tardó en cada sitio?». Una traza es un conjunto de *spans*; cada *span* es un tramo (una petición HTTP recibida, una llamada saliente, el procesamiento de un mensaje) con su servicio, su inicio y su duración.

Un ejemplo de GuildBoard. El maestro se queja de que Mara completó una misión y su experiencia sigue en 100. Las métricas te dicen que el consumidor de Adventurers no ha procesado ningún mensaje. La traza te enseña que la petición de completar llegó al gateway, entró en Quests y publicó el mensaje, y que ahí se acaba el recorrido. El log de Adventurers, buscado por el identificador de esa traza, te da la excepción. [32] y [33] defienden la misma idea de fondo: un sistema distribuido que no puedes inspeccionar mientras funciona es un sistema que no puedes operar.

15.2.1 Micrometer: una fachada para medir y trazar

Spring Boot no mide por su cuenta. Usa **Micrometer**, una fachada neutral al proveedor de métricas: tú (o el framework) registras contadores y temporizadores, y el proveedor que elijas los recoge [30]. Desde Spring Boot 3 la pieza central es la *Observation API*: una sola instrumentación (por ejemplo, «una petición HTTP») genera a la vez una métrica y un *span*. Boot registra el manejador que convierte cada observación terminada en un *span* [49], así que no escribes el código de medir. Las trazas, a su vez, pasan por otra fachada, **Micrometer Tracing**, que se conecta a una biblioteca concreta mediante un *puente* (Brave u OpenTelemetry) [31].

Dos consecuencias para la arquitectura. La primera: como la medición ocurre en el framework y en los adaptadores que ya usan Spring, `domain` y `application` no cambian. La segunda: si algún día quieres medir algo de negocio (misiones completadas), el contador vive en un adaptador o en un decorador de caso de uso, nunca en una entidad. Este capítulo no lo implementa.

15.3 Actuator: salud y métricas de cada servicio

Actuator es el conjunto de *endpoints* operativos de Spring Boot: cuelgan de `/actuator` y exponen la salud del servicio, sus métricas y otros datos de funcionamiento. Micrometer mide; Actuator publica lo medido por HTTP.

☞ Actuator en cada servicio y en el gateway

Se añade un *starter* a los `build.gradle` de los tres `bootstrap` y del gateway. Va en `bootstrap` y no en `adapters` porque observar el proceso es una decisión de ensamblado, no de un adaptador.

bootstrap/build.gradle (línea nueva, sin versión)

```
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-actuator'
```

Initializr, para Boot 4.1.1, ofrece la dependencia `actuator` como `org.springframework.boot:spring-boot-starter-actuator` (verificado el 9 de octubre de 2026, sección 15.7.3). La versión la pone el BOM de Boot.
Por omisión, Boot publica por HTTP solo el *endpoint* `health` (los metadatos de la propiedad lo confirman). Para publicar también `metrics`, una propiedad común para todos los clientes, en `config-repo/application.yml`:

config-repo/application.yml (bloque nuevo)

```
management:
  endpoints:
    web:
      exposure:
        include: health,metrics
```

Como cada cliente lee la configuración al arrancar, reinicia después los servicios.

15.3.1 Health: ¿estás vivo?

Con el *starter* y la propiedad, `GET /actuator/health` devuelve el estado agregado de los *indicadores de salud* que Boot activa solos según lo que hay en el *classpath*: la base de datos si hay JDBC, RabbitMQ si hay el *starter* de AMQP, el disco. La respuesta es de este estilo (forma esperada):

```
$ curl -s localhost:8081/actuator/health
{"status":"UP"}
```

Si un indicador falla (por ejemplo, RabbitMQ parado), el estado general pasa a `DOWN` y el código HTTP es 503. Es una buena noticia para un *healthcheck* de contenedor: «sano» significa «puedo hacer mi trabajo», no solo «el proceso existe». Es también una trampa: una caída de PostgreSQL pone en rojo a tres servicios a la vez, y Compose los marcará a todos *unhealthy*.

15.3.2 El conflicto con la seguridad

Desde el capítulo 14, los tres servicios exigen un token a todo lo que no tenga una regla propia (`anyRequest().authenticated()`). Un *healthcheck* de Compose es un `curl` sin token: sin cambios, daría 401 y el contenedor nunca estaría sano. La decisión del curso: **`health` es público y `metrics` exige token.** La salud no revela datos sensibles (no se muestran detalles); las métricas sí cuentan cosas del servicio. En `SecurityConfig`, una regla más, la primera de la lista (gana la primera que casa, capítulo 14):

SecurityConfig.java de Quests (solo las reglas)

```
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**").permitAll()
```

```
                .requestMatchers(HttpMethod.POST, "/quests")
                    .hasRole("GUILD_MASTER")
                .requestMatchers(HttpMethod.POST, "/quests/*/accept")
                    .hasRole("ADVENTURER")
                .requestMatchers(HttpMethod.POST, "/quests/*/complete")
                    .hasAnyRole("GUILD_MASTER", "ADVENTURER")
                .anyRequest().authenticated())
```

La línea nueva es la segunda; el resto es el capítulo anterior. El patrón termina en `/**`, que casa con `/actuator/health` y con sus subrutas (cero o más segmentos, como en el gateway). En Adventurers y Treasury, la misma línea va delante de sus reglas; las soluciones del ejercicio 15.1 y del ejercicio 15.5 las muestran enteras. La regla está en `adapter/in/web/security` porque es un detalle de la entrada web.

Error típico

Olvidar la regla en un servicio. Síntoma: `docker compose up --wait` termina con un error de contenedor *unhealthy* y, dentro, el `curl` del *healthcheck* recibe 401. Se comprueba sin Compose, con la función `code` del capítulo 14: `code localhost:8081/actuator/health` debe dar 200.

15.3.3 Metrics: números de lo que ha pasado

`GET /actuator/metrics` lista los nombres de las métricas disponibles. Una de las más útiles es `http.server.requests`, un *temporizador* de las peticiones HTTP que el servicio ha recibido. Con un token, y tras unas peticiones, se pide por su nombre (forma esperada, no ejecutada):

```
$ curl -s localhost:8081/actuator/metrics/http.server.requests \
    -H "Authorization: Bearer $MASTER"
{"name":"http.server.requests","baseUnit":"seconds",
 "measurements":[{"statistic":"COUNT","value":5.0},
 {"statistic":"TOTAL_TIME","value":0.31},
 {"statistic":"MAX","value":0.12}],
 "availableTags":[{"tag":"uri","values":["/quests","/quests/{id}"]},
 {"tag":"method","values":["GET","POST"]},
 {"tag":"status","values":["200","201"]}, ...]}
```

Un temporizador da tres cifras: `COUNT` (cuántas veces), `TOTAL_TIME` (segundos acumulados) y `MAX` (la más lenta). Las `availableTags` son las dimensiones por las que puedes filtrar: se añaden como `?tag=nombre:valor`. La etiqueta `uri` guarda la *plantilla* (`/quests/{id}`), no la URL concreta; si guardase cada URL, habría una serie de métricas por cada misión y la memoria crecería sin límite (alta cardinalidad).

```
$ B=localhost:8081/actuator/metrics/http.server.requests
$ curl -s "$B?tag=uri:/quests&tag=method:POST" \
    -H "Authorization: Bearer $MASTER"
```

En Adventurers y Treasury, además, el contenedor de `@RabbitListener` registra un temporizador `spring.rabbitmq.listener` con las etiquetas `queue`, `result` (`success` o `failure`) y `exception` [43]. Es la métrica que habría respondido al caso de la experiencia perdida: sin ningún `COUNT` para la cola `adventurers.quest-completed`, el mensaje no llegó.

Fuera de este capítulo

Para ver las métricas en gráficas hace falta alguien que las recoja cada pocos segundos (por ejemplo, Prometheus, con el registro `micrometer-registry-prometheus`, que Initializr ofrece como dependencia `prometheus`) y alguien que las dibuje. No se instalan aquí: el curso se queda en leer los *endpoints*.

15.4 Trazas entre servicios

Una traza empieza donde entra la petición (aquí, el gateway) y cada proceso que la atiende añade sus *spans* con el mismo identificador de traza, el `traceId`. Cada *span* tiene además su `spanId`. Para que el segundo proceso sepa a qué traza pertenece, el primero le pasa el contexto en una cabecera de la propia petición: Micrometer Tracing escribe por omisión la cabecera W3C `traceparent` y acepta también las de Brave (`B3`) al recibir. Cuando la comunicación es un mensaje, el contexto viaja como cabecera del mensaje.

15.4.1 Qué se instala y por qué Brave con Zipkin

Micrometer Tracing necesita un puente y un sitio al que enviar los *spans*. Boot trae dos combinaciones con *starter* propio: OpenTelemetry con OTLP, y Brave con Zipkin [49]. El curso usa la segunda por una razón práctica: un solo *starter*, `spring-boot-starter-zipkin`, trae el puente de Brave y el envío a Zipkin, y Zipkin es un solo contenedor con interfaz web. Un dato de la documentación para no equivocarse: el exportador de OpenTelemetry a Zipkin está *deprecated* y desaparecerá en Spring Boot 4.2, así que no se mezclan OpenTelemetry y Zipkin.

☞ Trazas con Brave y Zipkin

Una línea en cada uno de los mismos cuatro `build.gradle` (los tres `bootstrap` y el gateway):

`build.gradle (línea nueva, sin versión)`

```
1     implementation 'org.springframework.boot:spring-boot-starter-zipkin'
```

Initializr ofrece la dependencia `zipkin` como ese artefacto. En Maven Central, su *pom* depende de `spring-boot-micrometer-tracing-brave`, de `spring-boot-zipkin` (que arrastra `zipkin-reporter-brave`) y, en tiempo de ejecución, de `micrometer-tracing-bridge-brave`.
Tres propiedades, también comunes, en `config-repo/application.yml`. La primera es imprescindible para este capítulo: por omisión Boot muestrea solo el 10 % de las peticiones y verías una de cada diez. Las otras dos activan la observación de RabbitMQ, desactivada por omisión (los metadatos dan `false`):

config-repo/application.yml (bloques nuevos)

```yaml
management:
  tracing:
    sampling:
      probability: 1.0

spring:
  rabbitmq:
    template:
      observation-enabled: true
    listener:
      simple:
        observation-enabled: true
```

(Cada bloque se fusiona con el que ya tiene el archivo bajo `spring.rabbitmq`. El archivo completo está en la solución del ejercicio 15.5.) El destino de los *spans* es, por omisión, `http://localhost:9411/api/v2/spans`: el puerto de Zipkin. La propiedad se llama `management.tracing.export.zipkin.endpoint`; la antigua `management.zipkin.tracing.endpoint` sigue como alias obsoleto.
Y Zipkin mismo, un contenedor más en `compose.yaml` (imagen `openzipkin/zipkin:3`, que existe en Docker Hub y trae su propio *healthcheck*):

compose.yaml (servicio nuevo)

```yaml
  zipkin:
    image: openzipkin/zipkin:3
    ports:
      - "9411:9411"
```

```
$ docker compose up -d --wait zipkin
$ curl -s localhost:9411/health
```

La segunda orden debe devolver un JSON con `"status" : "UP"`. La interfaz está en `http://localhost:9411`. Esta imagen guarda las trazas en memoria: al reiniciar el contenedor se pierden.

Con solo esto, cada servicio, si lo arrancas a mano con `bootRun` y tienes Zipkin en el 9411, ya envía sus *spans*. Falta que el contexto *cruce* entre procesos.

15.4.2 Los logs, con el identificador

Con Micrometer Tracing en el *classpath*, Spring Boot añade a cada línea de log un identificador de correlación formado por el `traceId` y el `spanId` [49]. No configuras nada. Una línea de Quests mientras completa una misión se ve (forma aproximada; el formato exacto depende de la versión) así:

```
2026-10-09T10:15:07.543Z INFO 1 --- [quests] [nio-8081-exec-2]
[5d08e6b2c94a10f7e3b7a1d9066c52f8-30aa41c7b8d29e05] ... notification
```

Lo importante es el bloque entre corchetes tras el hilo: los primeros caracteres son el `traceId`, común a toda la traza; los últimos, el `spanId`. Con él puedes buscar *la misma petición* en los logs de todos los servicios. Es el uso más barato de las trazas y el que sigue valiendo si Zipkin no está.

15.4.3 La trampa del RestClient

El servidor es automático: cuando una petición llega a un servicio con Actuator, Boot extrae el contexto de la cabecera. El *cliente* no lo es. La documentación de Boot lo dice sin rodeos: para propagar la traza por la red hay que construir el `RestClient`, el `RestTemplate` o el `WebClient` con el *builder autoconfigurado*; si lo creas por tu cuenta, «la propagación automática no funciona» [49].

Y la fábrica de Quests (capítulo 12) hace justo lo que no debe: empieza con `RestClient.builder()`, el método estático, que da un *builder* sin instrumentar. La traza de una aceptación se rompería en la llamada de Quests a Adventurers: Adventurers abriría una traza nueva. Hay que arreglarlo, y es el único cambio de código de esta parte, en tres sitios:

- **Una dependencia en el `bootstrap` de Quests.** El *builder* autoconfigurado (el bean `RestClient.Builder`, de ámbito *prototype*, con el personalizador que añade la observación) lo crea la autoconfiguración `RestClientAutoConfiguration`, que vive en `spring-boot-restclient`. El *starter* `webmvc` no lo trae (se comprobó en su *pom*), así que se añade el *starter* `spring-boot-starter-restclient`.
- **La fábrica acepta un *builder*.** Un método más; el antiguo, con tres parámetros, se queda y delega con `RestClient.builder()`: lo usa la prueba del capítulo 12, que no arranca Spring.
- **El bean lo pasa.** `BeanConfig` recibe el *builder* de Spring y se lo da a `RestAdventurerDirectory`, que gana un constructor.

La fábrica, con sus dos métodos:

adapter/out/rest/AdventurerClientFactory.java

```
1  package com.guildboard.quests.adapter.out.rest;
2
3  import java.net.http.HttpClient;
4  import java.time.Duration;
5  import org.springframework.http.client.JdkClientHttpRequestFactory;
6  import org.springframework.web.client.RestClient;
7
8  public final class AdventurerClientFactory {
9
10     private AdventurerClientFactory() {
11     }
12
13     public static RestClient create(String baseUrl,
14             Duration connectTimeout, Duration readTimeout) {
15         return create(RestClient.builder(), baseUrl,
16             connectTimeout, readTimeout);
17     }
18
19     public static RestClient create(RestClient.Builder builder,
20             String baseUrl, Duration connectTimeout,
```

```
            Duration readTimeout) {
        HttpClient http = HttpClient.newBuilder()
            .connectTimeout(connectTimeout)
            .build();
        JdkClientHttpRequestFactory factory =
            new JdkClientHttpRequestFactory(http);
        factory.setReadTimeout(readTimeout);
        return builder
            .baseUrl(baseUrl)
            .requestFactory(factory)
            .requestInterceptor(new BearerTokenForwardingInterceptor())
            .build();
    }
}
```

El constructor nuevo de `RestAdventurerDirectory` (el resto de la clase y sus otros dos constructores no cambian; `RestClient` ya está importado):

adapter/out/rest/RestAdventurerDirectory.java (constructor nuevo)

```
    public RestAdventurerDirectory(RestClient.Builder builder,
            String baseUrl, Duration connectTimeout,
            Duration readTimeout, CircuitBreaker breaker, Retry retry) {
        this(AdventurerClientFactory.create(
                builder, baseUrl, connectTimeout, readTimeout),
            breaker, retry);
    }
```

Y el bean, con el *builder* como primer parámetro (añade en `BeanConfig` el *import* `org.springframework.web.client.RestClient`):

config/BeanConfig.java (Quests, solo el bean AdventurerDirectory)

```
    @Bean
    public AdventurerDirectory adventurerDirectory(
            RestClient.Builder restClientBuilder,
            @Value("${guildboard.adventurers.base-url}") String baseUrl,
            @Value("${guildboard.adventurers.connect-timeout}")
            Duration connectTimeout,
            @Value("${guildboard.adventurers.read-timeout}")
            Duration readTimeout,
            CircuitBreakerRegistry breakers, RetryRegistry retries) {
        return new RestAdventurerDirectory(restClientBuilder, baseUrl,
            connectTimeout, readTimeout,
            breakers.circuitBreaker("adventurers"),
            retries.retry("adventurers"));
    }
```

El *builder* de Boot ya trae un `requestFactory`, y la fábrica pone el suyo después: el que cuenta es el de los tiempos máximos del capítulo anterior. El interceptor del token y la observación se suman: son cosas distintas.

El gateway no necesita nada: su proxy se construye con el `RestClient.Builder` autoconfigurado (se comprobó en el *bytecode* del *jar*), así que reenvía la cabecera `traceparent` que recibe.

15.4.4 La mensajería

Los mensajes de RabbitMQ no son HTTP, y el contexto tiene que viajar en una cabecera del mensaje. Spring AMQP lo permite con la API de observación: con la propiedad del `RabbitTemplate` activada, el emisor crea un contexto de envío cuyo *setter* escribe en las propiedades del mensaje (`RabbitMessageSenderContext` extiende el `SenderContext` de Micrometer), y con la del contenedor de escucha activada, el receptor lo lee. Boot registra los manejadores de propagación de emisor y de receptor [49]. Eso es lo que activan las dos propiedades de la instalación.

Diseñado, no verificado

Todo lo anterior se ha comprobado leyendo *jars* y documentación; **no se ha ejecutado**. No he podido confirmar que, con `JacksonJsonMessageConverter` y nuestras colas, el listener de Treasury y el de Adventurers reciban el mismo `traceId` que Quests. Cómo reconocer un fallo: tras completar una misión, busca en los logs el `traceId` de Quests. Si el listener lo repite, el contexto cruzó RabbitMQ. Si sus líneas llevan otro, el contexto no cruzó y ese tramo es una traza distinta. En ese caso, la lectura honesta del capítulo es: la traza cubre la petición REST (gateway, Quests, Adventurers) y los logs de los listeners se correlacionan por la misión (su `questId`), no por el `traceId`. Las trazas de mensajería quedan como tarea pendiente de verificación.

Otro error típico: el servicio no aparece en Zipkin. Mira por orden: que el contenedor (o el proceso) alcance el 9411 (dentro de Compose, `zipkin`, no `localhost`; sección 15.6), que `probability` sea 1.0 y que Zipkin no se haya reiniciado (guarda en memoria).

15.5 Empaquetar cada servicio: el Dockerfile

Para que Compose arranque un servicio necesita una *imagen*: el programa y el entorno mínimo para ejecutarlo. Un `Dockerfile` es la receta de esa imagen. Hay dos maneras de prepararla:

- **Construir dentro de Docker** (*multi-stage*): una primera etapa con JDK y Gradle compila el proyecto y una segunda copia el resultado a una imagen ligera. Es reproducible (no depende de lo que tengas instalado), pero cada construcción repite las descargas de Gradle si no hay caché y el `Dockerfile` necesita ver todo el repositorio, no solo una carpeta.
- **Copiar el *jar* ya construido en tu equipo.** Gradle compila, `bootJar` empaqueta y el `Dockerfile` solo copia un archivo. Es mucho más rápido y cada proyecto es autónomo, pero depende de que hayas construido antes y de tu JDK.

El curso usa la segunda, la que fija el plan: es un entorno de aprendizaje y lo que importa es entender la red y la configuración, no optimizar la construcción. En un proyecto real la construcción la haría un sistema de integración continua con la primera estrategia, y la imagen se publicaría en un registro.

☞ Docker: lo que ya tienes y el nombre del *jar*

Docker y su plugin de Compose ya están instalados (briefing). Comprueba que se ven:

```
$ docker --version
$ docker compose version
```

La imagen base es `eclipse-temurin:21-jre`: un JRE 21 sin compilador, de Eclipse Temurin. La etiqueta existe en Docker Hub (actualizada el 2 de octubre de 2026, para `amd64` y `arm64`); es una distribución de Ubuntu y trae `curl`, que necesitarán los *healthchecks* (comprobado al ejecutarla; una imagen base sin `curl` los rompería).
Falta un detalle de Gradle. El plugin de Boot configura la tarea `jar` para que genere un *jar* «plano» con el sufijo `-plain` [51], así que `build/libs` contiene dos archivos por proyecto, y un `COPY` con comodín fallaría. Se arregla fijando el nombre del *jar* ejecutable, en cada uno de los seis `build.gradle` con el plugin de Boot (los tres `bootstrap`, `gateway`, `config-server` y `token-issuer`):

`build.gradle` (bloque nuevo, al final del archivo)

```
tasks.named('bootJar') {
    archiveFileName = 'app.jar'
}
```

Luego `./gradlew bootJar` (sin proyecto) construye los seis y deja `build/libs/app.jar` en cada carpeta.

El `Dockerfile`, igual para los seis proyectos:

`Dockerfile`

```
FROM eclipse-temurin:21-jre
RUN useradd --system --no-create-home guildboard
WORKDIR /app
COPY build/libs/app.jar app.jar
USER guildboard
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- `FROM` elige la imagen base.
- `RUN useradd` crea un usuario sin privilegios, y `USER` hace que el proceso no corra como `root`: si alguien vulnera el servicio, no es administrador del contenedor.
- `WORKDIR /app` fija el directorio de trabajo. Importa: el Config Server lee sus archivos con una ruta relativa a ese directorio (sección 15.6).
- `COPY build/libs/app.jar app.jar` copia el *jar*. La ruta de origen es relativa al *contexto de construcción*, la carpeta que Compose pasa a Docker.
- `ENTRYPOINT` arranca la aplicación con la forma de lista, que ejecuta `java` directamente y le deja recibir las señales de parada.

Como todo lo que cambia entre servicios (puerto, perfil, dirección del Config Server) llega por configuración y por variables de entorno, **los seis archivos son idénticos**. Se copian a mano o con un bucle:

```
$ for d in services/adventurers/bootstrap services/treasury/bootstrap \
    gateway config-server tools/token-issuer
> do cp services/quests/bootstrap/Dockerfile "$d/Dockerfile"; done
```

(Escribe el primero en `services/quests/bootstrap` y cópialo desde ahí.) El contexto de construcción de cada imagen es la carpeta de su proyecto. En `compose.yaml` se escribe como

`build: ./services/quests/bootstrap`, y Compose busca ahí el `Dockerfile`.

Error típico

Olvidar `./gradlew bootJar` antes de `docker compose build`. Docker falla con un error de `COPY` porque no encuentra `build/libs/app.jar`. Y lo contrario: cambiar el código, no reconstruir el *jar* y arrancar la imagen vieja. Siempre, en este orden: `bootJar`, luego `docker compose up --build`.

15.6 Compose completo: red, nombres de servicio y orden de arranque

Compose ya te levanta PostgreSQL y RabbitMQ. Ahora tiene que levantar nueve contenedores que se hablan entre sí, y el cambio de modelo mental es uno solo: **dentro de un contenedor, `localhost` es el propio contenedor.**

15.6.1 Una red, un DNS

Compose crea una red para el proyecto y conecta a ella todos los servicios. Dentro de esa red, el *nombre del servicio* del `compose.yaml` se resuelve como nombre de máquina [5]. Lo comprobé arrancando PostgreSQL, RabbitMQ y Zipkin y resolviendo `rabbitmq` y `postgres` desde el contenedor de Zipkin. Y hay dos formas de llegar a un puerto, según desde dónde mires:

Quién llama	Dirección	Por qué
Tú, con `curl` o el navegador	`localhost:8080`	El puerto está *publicado* en tu máquina con `ports: "8080:8080"`
Un contenedor a otro	`quests:8081`, `rabbitmq:5672`	Nombre del servicio y puerto *interno*
Un contenedor a su propio puerto	`localhost:8081`	Es el mismo contenedor: sirve para el *healthcheck*
Un contenedor a un servicio que corre en tu máquina (no en Compose)	no hay un nombre fijo	Fuera de este capítulo

Por eso la configuración del capítulo 13 no vale tal cual: todas sus direcciones son `localhost`. Dentro de Compose, un Quests con `base-url: http://localhost:8082` buscaría a Adventurers *dentro de sí mismo* y fallaría con «conexión rechazada». Hay que cambiar cada dirección, y hacerlo sin romper el modo de antes (todo en tu máquina con `bootRun`), que tiene que seguir valiendo.

15.6.2 El perfil docker

Un perfil de Spring es un nombre que, al activarse, añade configuración. El Config Server entrega al cliente, además de `application.yml` y de `<nombre>.yml`, los archivos `application-<perfil>.yml` del perfil que el cliente pida [56]. Basta un archivo nuevo en `config-repo` con todo lo que cambia dentro de Compose:

config-repo/application-docker.yml (1 de 2)

```
spring:
  rabbitmq:
    host: rabbitmq
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: http://token-issuer:9000/.well-known/jwks.json
  cloud:
    gateway:
      server:
        webmvc:
          routes:
            - id: quests
              uri: http://quests:8081
              predicates:
                - Path=/api/quests/**
              filters:
                - StripPrefix=1
            - id: adventurers
              uri: http://adventurers:8082
              predicates:
                - Path=/api/adventurers/**
              filters:
                - StripPrefix=1
            - id: treasury
              uri: http://treasury:8083
              predicates:
                - Path=/api/treasury/**
              filters:
                - StripPrefix=2
```

config-repo/application-docker.yml (2 de 2: sigue en la raíz del archivo)

```
guildboard:
  db:
    host: postgres
  adventurers:
    base-url: http://adventurers:8082

management:
  tracing:
    export:
      zipkin:
        endpoint: http://zipkin:9411/api/v2/spans
```

Cada bloque se explica con una regla:

- **Lo que cambia es lo que se *llama*.** RabbitMQ, el JWKS del emisor, las rutas del gateway, el `base-url` de Adventurers y Zipkin: todo es una dirección a la que un proceso conecta, y desde dentro de Compose es un nombre de servicio.
- **Lo que no cambia es lo que se *compara*.** `issuer-uri` sigue siendo `http://localhost:9000`. No es una dirección a la que el servicio conecte (con

`jwk-set-uri` presente no consulta al emisor al arrancar, capítulo 14): es la cadena que debe coincidir con la claim `iss`, y el emisor sigue escribiendo `http://localhost:9000` en sus tokens. Cambiar una sola de las dos cosas haría que todos los tokens fallasen; por eso el emisor no necesita tocarse. (Se puede pedir al emisor desde fuera de Compose con `localhost`, como hasta ahora.)

- **Las listas se reemplazan, no se fusionan.** Las rutas del gateway son una lista: si el perfil solo cambiase la `uri` de la primera, la lista del perfil sustituiría a la de `gateway.yml` y quedaría una ruta sin predicado. Por eso las tres rutas se repiten enteras. Es la decisión que el capítulo 13 dejó pendiente para este.
- **Una clave, un dueño.** PostgreSQL es el caso delicado. Su URL vive en `application-postgres.yml`, un archivo *local* de cada `bootstrap` (capítulo 9). Si el perfil `docker` definiese también `spring.datasource.url`, la misma clave tendría dos valores en dos orígenes (uno local de perfil y otro importado del Config Server), y cuál gana depende de un orden que este libro no ha podido comprobar. Mejor evitar el empate: se crea una clave propia, `guildboard.db.host`, que solo define el perfil `docker`, y el archivo local la usa con un valor por omisión.

El archivo local, en Quests (los de Adventurers y Treasury cambian igual, con su base de datos):

`quests-bootstrap: application-postgres.yml`

```yaml
spring:
  autoconfigure:
    exclude: []
  datasource:
    url: "jdbc:postgresql://${guildboard.db.host:localhost}:5432/quests"
    username: guildboard
    password: guildboard
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: "none"
```

Sin el perfil `docker`, la clave no existe y se usa `localhost`: el modo de antes no cambia. Con él, vale `postgres`. El marcador `${clave:valor}` usa el valor tras los dos puntos si la clave no está. Esa resolución la hace el cliente con todas sus fuentes juntas, también con las que vienen del Config Server (la documentación de Config describe ese mecanismo para sustituir valores del servidor [56]).

15.6.3 Dos variables de entorno por cliente

¿Cómo sabe un contenedor que debe usar el perfil `docker` y dónde está el Config Server? Por dos variables de entorno en `compose.yaml`, para cada cliente de configuración (los tres servicios y el gateway):

`compose.yaml (fragmento de un cliente)`

```yaml
    environment:
      SPRING_PROFILES_ACTIVE: docker,postgres
      SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
```

- `SPRING_PROFILES_ACTIVE` activa los perfiles. Los servicios necesitan `docker` y también `postgres` (si no, usarían los repositorios en memoria y la base de datos sobraría). El gateway solo `docker`.
- `SPRING_CONFIG_IMPORT` sustituye el `optional:configserver:http://localhost:8888` del `application.yml` local: Boot da prioridad a las variables de entorno sobre los archivos [52]. Se escribe **sin** `optional:` a propósito: si el Config Server no está, el cliente debe fallar al arrancar y decirlo, no seguir con valores por omisión que lo dejarían escuchando en el puerto equivocado.
- **Por qué no están en el `config-repo`:** la dirección del Config Server no puede venir del propio Config Server. Y por qué no en un `application-docker.yml` local en cada `bootstrap`: sería un archivo nuevo por servicio y un cambio de estructura respecto al árbol de la verificación 10.

El Config Server no es cliente de sí mismo y no lleva estas variables; usa `native` desde su propio `application.yml`.

15.6.4 El Config Server dentro de un contenedor

El servidor busca su repositorio en `file:../config-repo/`, relativo al directorio de trabajo (capítulo 13). En el contenedor, el directorio de trabajo es `/app` (el `WORKDIR`), y `../config-repo/` es `/config-repo/`. Basta *montar* la carpeta del repositorio ahí, de solo lectura, sin cambiar su configuración:

compose.yaml (servicio config-server)

```yaml
  config-server:
    build: ./config-server
    ports:
      - "8888:8888"
    volumes:
      - ./config-repo:/config-repo:ro
    healthcheck:
      test: ["CMD", "curl", "-fs",
             "http://localhost:8888/application/default"]
      interval: 5s
      timeout: 3s
      retries: 20
      start_period: 20s
```

Como los archivos se leen del disco montado, editar `config-repo` en tu máquina cambia lo que sirve el servidor sin reconstruir nada. Los clientes siguen leyendo la configuración al arrancar: hay que reiniciarlos (`docker compose restart quests`).

15.6.5 Healthchecks y orden de arranque

Un contenedor está *arrancado* en cuanto su proceso existe; eso no dice que esté *listo*. Un `healthcheck` es una orden que Compose ejecuta cada pocos segundos dentro del contenedor: si termina con éxito, el servicio está `healthy`. Y `depends_on` con `condition: service_healthy` hace que un servicio no arranque hasta que sus dependencias estén sanas. El mapa de GuildBoard:

Servicio	Healthcheck	Espera a
`postgres`	`pg_isready`	nada
`rabbitmq`	`rabbitmq-diagnostics ping`	nada
`zipkin`	el de la imagen	nada
`config-server`	`GET /application/default`	nada
`token-issuer`	`GET /.well-known/jwks.json`	nada
`adventurers`, `treasury`	`GET /actuator/health`	`postgres`, `rabbitmq`, `zipkin`, `config-server`
`quests`	`GET /actuator/health`	lo anterior más `adventurers` y `treasury`
`gateway`	`GET /actuator/health`	`config-server`, `zipkin`, `quests`

Tres razones del orden. Adventurers y Treasury declaran sus colas al arrancar y Quests publica el primer mensaje: si Quests fuera antes, un mensaje sin cola enlazada se descartaría (capítulo 11). Los servicios necesitan la base de datos *lista* para que Flyway migre. Y todos necesitan el Config Server sano, porque sin `optional:` no arrancan sin él. El emisor no está en la lista de nadie: los servicios piden el JWKS al validar el primer token, no al arrancar.

`start_period` da un margen inicial en el que los fallos del *healthcheck* no cuentan (un Spring Boot tarda varios segundos en arrancar). Y un recordatorio del apartado de Actuator: `/actuator/health` depende de los indicadores, así que un servicio con RabbitMQ caído es `unhealthy`. Es lo que quieres para decidir el orden de arranque y algo que debes recordar al diagnosticar.

Un servicio de ejemplo, Adventurers, entero (Treasury es igual con el puerto 8083; Quests añade a Adventurers y Treasury en `depends_on`; el archivo completo está en la solución del ejercicio 15.5):

compose.yaml (servicio adventurers)

```yaml
  adventurers:
    build: ./services/adventurers/bootstrap
    environment:
      SPRING_PROFILES_ACTIVE: docker,postgres
      SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    ports:
      - "8082:8082"
    depends_on:
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
      zipkin:
        condition: service_healthy
      config-server:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-fs",
             "http://localhost:8082/actuator/health"]
      interval: 5s
      timeout: 3s
      retries: 30
      start_period: 30s
```

Los puertos de los servicios se publican para que puedas llamarlos directamente (como en el capítulo 14, para ver el 401 sin pasar por el gateway). En un despliegue real solo el gateway

estaría expuesto.

Dos formas de usar Compose con Spring

Spring Boot tiene un módulo propio de soporte de Compose [48], que arranca los contenedores de *infraestructura* cuando arranca tu aplicación y configura las conexiones. Resuelve otro problema: desarrollar con la aplicación en tu máquina. Aquí Compose ejecuta *también* las aplicaciones, y el módulo no se usa.

15.7 En el proyecto: hito H8

Hito 8: GuildBoard completo en Compose, con observabilidad

Al terminar el ejercicio 15.5, GuildBoard tiene:

- **Observabilidad.** Actuator y Brave/Zipkin en los tres servicios y en el gateway; `health` público y `metrics` con token; muestreo del 100 %; `traceId` en los logs.
- **Trazas que cruzan REST** (gateway, Quests, Adventurers) gracias al `RestClient.Builder` de Boot en la fábrica de Quests, y observación de RabbitMQ activada (diseñada, no verificada).
- **Seis `Dockerfile` idénticos** y `archiveFileName = 'app.jar'` en los seis proyectos.
- **El perfil `docker`** en `config-repo/application-docker.yml`, con las variables `SPRING_PROFILES_ACTIVE` y `SPRING_CONFIG_IMPORT` en cada cliente, y la URL de PostgreSQL con un *host* por omisión.
- **Un `compose.yaml` de nueve servicios** con *healthchecks* y orden de arranque: `docker compose up -d --build --wait` levanta todo.
- **La estructura T9** de la verificación de estructura 10 y el núcleo sin tocar.

Con esto, el proyecto está terminado.

15.7.1 Probar el hito

Construye, levanta y mira el estado. La primera vez tarda, porque Docker descarga las imágenes:

```
$ ./gradlew bootJar
$ docker compose up -d --build --wait
$ docker compose ps
```

Deben aparecer los nueve servicios, con `healthy` en todos los que tienen *healthcheck*. Si alguno falla, `docker compose logs <servicio>` lo cuenta. Define de nuevo `jid`, `code` y `tok` (sección 14.8), pide `MASTER=$(tok master)` y repite el huevo y la gallina del capítulo anterior, ahora con una vuelta de más: el emisor es un *jar* dentro de una imagen.

```
$ MARA=$(curl -s -X POST localhost:8080/api/adventurers \
    -H "Authorization: Bearer $MASTER" \
    -H 'Content-Type: application/json' -d '{"name":"Mara"}' | jid)
$ echo $MARA
```

Escribe ese id en `subject` del usuario `hero` del `tools/token-issuer/src/main/resources/application.yml` y reconstruye solo el emisor. Al reiniciar genera claves nuevas: los tokens anteriores dejan de valer.

```
$ ./gradlew :token-issuer:bootJar
$ docker compose up -d --build --wait token-issuer
$ MASTER=$(tok master); HERO=$(tok hero)
```

Ahora el recorrido por el gateway: publicar con `MASTER`, aceptar y completar con `HERO`, y leer el saldo y el rango. Es el del capítulo 14 (sección 14.8.1), con las mismas salidas esperadas: 150 de saldo y `ADEPT`. Lo nuevo es lo que se puede *ver*:

```
$ code localhost:8081/actuator/health
$ code localhost:8081/actuator/metrics
$ docker compose logs quests | grep 'notification to'
```

Los códigos esperados son **200** (la salud es pública) y **401** (las métricas piden token). La tercera orden imprime la línea de log de Quests al completar la misión; en ella, entre corchetes, está el identificador de correlación. Copia el `traceId` y búscalo en los tres servicios:

```
$ TRACE=$(docker compose logs quests | grep 'notification to' | tail -1 \
    | grep -oE '[0-9a-f]{16,32}' | head -1)
$ docker compose logs quests adventurers treasury | grep "$TRACE"
```

Con el puente de mensajería funcionando (es lo que *debería* pasar), verás líneas de los tres servicios con el mismo `traceId`; si solo salen las de Quests, el contexto no cruzó RabbitMQ y estás en el caso descrito en la sección 15.4. En Zipkin (`http://localhost:9411`), elige `gateway` en *serviceName* y pulsa *Run Query*: la traza de la aceptación debe mostrar *spans* de `gateway`, `quests` y `adventurers`. Para las métricas, con token:

```
$ curl -s localhost:8081/actuator/metrics/http.server.requests \
    -H "Authorization: Bearer $MASTER"
```

Y una prueba de que la resiliencia del capítulo 12 sigue funcionando dentro de Compose, con la red por medio:

```
$ docker compose stop adventurers
$ code -X POST localhost:8080/api/quests/$Q2/accept \
    -H "Authorization: Bearer $HERO"
$ docker compose start adventurers
```

con `Q2` una segunda misión publicada: la aceptación debe dar **503** (el nombre `adventurers` deja de resolverse y Quests lo traduce como indisponibilidad). Para terminar, que el núcleo no se enteró:

```
$ ./gradlew test
$ ./gradlew build
$ git status --short
$ grep -rliE 'micrometer|actuator|zipkin|docker' \
```

```
    services/*/domain/src services/*/application/src
```

El `grep` no debe mostrar nada; `./gradlew test` ya vigila `springframework` en el núcleo. En `git status` debes ver: el `compose.yaml`, `config-repo/application.yml` y `config-repo/application-docker.yml`; los seis `Dockerfile` y los seis `build.gradle`; en cada servicio, `SecurityConfig.java` y `application-postgres.yml`; y, en Quests, la fábrica, `RestAdventurerDirectory` y `BeanConfig`.

15.7.2 Lo que este capítulo no resuelve

- **Las trazas por RabbitMQ no están verificadas**, y de ellas depende el recorrido completo de «completar una misión».
- **Credenciales en texto plano** (PostgreSQL, RabbitMQ) en `compose.yaml` y en `config-repo`; sin TLS entre contenedores. Los puertos de los servicios están publicados en tu máquina.
- **Zipkin guarda en memoria** y el muestreo es del 100 %: sirve para aprender, no para un sistema con tráfico (`probability: 1.0` multiplica el coste de cada petición).
- **La imagen depende de tu *jar*.** Dos personas con el mismo código pueden generar imágenes distintas.
- **Una sola instancia de PostgreSQL** con tres bases (capítulo 9); sin límites de memoria ni política de reinicio en los contenedores.
- **`depends_on` ordena el arranque, no la vida.** Si PostgreSQL cae más tarde, los servicios no se enteran hasta que fallan; la resiliencia es la del capítulo 12.
- **Sin métricas de negocio, sin alertas y sin gráficas.**
- **Los tokens** siguen siendo los del emisor de juguete del capítulo anterior, que regenera la clave en cada arranque.

15.7.3 Qué se verificó y qué no (9 de octubre de 2026)

Verificado. Initializr, con Boot 4.1.1: `actuator` es `spring-boot-starter-actuator`, `zipkin` es `spring-boot-starter-zipkin`, `distributed-tracing` es `micrometer-tracing-bridge-brave`, `opentelemetry` es `spring-boot-starter-opentelemetry`, `prometheus` es `micrometer-registry-prometheus` (tiempo de ejecución) y `spring-restclient` es `spring-boot-starter-restclient`. En Maven Central: el *pom* del *starter* de Actuator depende de `spring-boot-starter-micrometer-metrics`, `spring-boot-actuator-autoconfigure`, `spring-boot-health`, `micrometer-observation` y `micrometer-jakarta9`; el de Zipkin, de lo que se dijo arriba; el de `webmvc` no incluye `spring-boot-restclient`, que sí trae el *starter* de `restclient`. En los *jars* 4.1.1: las propiedades `management.tracing.export.zipkin.endpoint` (por omisión `http://localhost:9411/api/v2/spans`), `management.tracing.sampling.probability` (0.1), `management.tracing.propagation.produce` (W3C) y `.consume` (W3C, B3, B3_MULTI), `management.endpoints.web.exposure.include` (por omisión `health`), `spring.rabbitmq.template.observation-enabled` y `spring.rabbitmq.listener.simple.observation-enabled` (por omisión `false`); `MicrometerTracingAutoConfiguration` registra los manejadores de propagación de emisor y de receptor; `RestClientAutoConfiguration` define el bean `RestClient.Builder` de ámbito *prototype* y existe el personalizador de observación; en `spring-web` 7.0.9, `RestClient.Builder.observationRegistry`; en `spring-rabbit` 4.1.1, `RabbitMessageSenderContext` extiende

`SenderContext`; en el *jar* del gateway 5.0.3, el proxy se construye con `RestClient.Builder`. Documentación: la página de trazas de Boot (muestreo del 10 % por omisión, identificadores de correlación por omisión, la advertencia de propagación, OpenTelemetry con Zipkin obsoleto), la del plugin de Gradle (el archivo `-plain`), la de Spring AMQP (los temporizadores `spring.rabbitmq.listener`) y la de Config (los marcadores se sobrescriben en el cliente). Docker Hub: `eclipse-temurin:21-jre`, `postgres:18`, `rabbitmq:4-management` y `openzipkin/zipkin:3` existen. **Ejecutado:** construir un `Dockerfile` como el del capítulo con un *jar* de relleno; `docker compose config` sobre el `compose.yaml` de la solución; levantar PostgreSQL, RabbitMQ y Zipkin en Compose (los tres `healthy`), resolver sus nombres desde otro contenedor, enviar un *span* a Zipkin y verlo listado.

No verificado por ejecución (el Java no se compila): el arranque de cualquiera de los seis procesos, de la imagen y de todo el sistema; que el Config Server entregue el perfil `docker` con prioridad sobre `quests.yml` y `gateway.yml`; que `../config-repo/` se resuelva a `/config-repo/`; que la propagación del `traceId` funcione por REST y, sobre todo, por RabbitMQ; la forma exacta de las líneas de log y de las salidas de `curl`; que `permitAll` sobre `/actuator/health/**` deje pasar el *healthcheck*; y todos los tiempos.

15.8 Cierre del curso: qué cambiaría en un proyecto real

GuildBoard es un proyecto para aprender, y se nota en lo que se dejó fuera. Este apartado mira atrás y adelante con el árbol final delante.

15.8.1 Lo que no se movió

En los capítulos de microservicios cambiaron la forma del repositorio, la comunicación y la operación. Lo que cambió muy poco fue el **núcleo**: desde el corte del capítulo 9, `domain` y `application` de cada servicio solo ganaron *puertos* (el de eventos en el capítulo 11, la excepción de indisponibilidad en el capítulo 12) y nada en los capítulos 13 a 15: ni el gateway, ni la seguridad, ni las trazas, ni Docker les han tocado una línea. Esa es la promesa de la arquitectura hexagonal medida con el árbol: los detalles (frameworks, red, despliegue) cambian alrededor de un centro estable [28], capítulos 15 «What Is Architecture?» y 16 «Independence».

15.8.2 Las diez verificaciones de estructura, de un vistazo

N.º	Cap.	Qué cambia (árbol)	Comprobación principal
1	3	Árbol inicial de un solo módulo	`./gradlew test` pasa (`ArchitectureTest`)
2	6	Paquetes `dto`, `mapper`, `error`, `notification`	Cada paquete en el anillo correcto
3	7	De un módulo a cuatro módulos Gradle	Dependencias entre módulos
4	8	Persistencia, migraciones, perfiles y Compose	`./gradlew test` pasa; persistencia fuera del núcleo
5	9	Disposición `services/` y un módulo por servicio	`./gradlew test` pasa; un `ArchitectureTest` por servicio
6	10	Tercer servicio, tercera base, `StructureTest` y slices	`./gradlew test` pasa
7	11	Paquetes de mensajería y RabbitMQ	Mensajería solo en adaptadores
8	13	`gateway`, `config-server`, `config-repo`	Sin lógica de negocio en la raíz
9	14	`tools/token-issuer` y paquete de seguridad	Dominio sin seguridad
10	15	Dockerfiles, perfil `docker`, Compose completo y árbol final	`./gradlew test` pasa; núcleo sin observabilidad ni despliegue

Desde la tercera verificación, los tres `ArchitectureTest` y el `StructureTest` se ejecutan con `./gradlew test`. Los capítulos 4, 5 y 12 no cambiaron la estructura y lo declararon. En cada verificación hubo un árbol anterior, un árbol nuevo, la lista de cambios y las comprobaciones.

15.8.3 Qué cambiaría en un proyecto real

- **La construcción.** Imágenes construidas en integración continua, con la estrategia *multi-stage*, versionadas y subidas a un registro. Nada de `bootJar` en el portátil de alguien.
- **Configuración y secretos.** Un repositorio Git de configuración (o la del orquestador) con cifrado, y secretos fuera de él. Las contraseñas de `compose.yaml` y de `config-repo` son de desarrollo.
- **Identidad.** Un servidor de autorización real, TLS en cada salto, validación de `aud` y tokens con una audiencia por servicio.
- **Datos y mensajes.** Bases de datos en instancias separadas; *outbox* o sagas donde haga falta consistencia entre servicios; contratos de eventos versionados y pruebas de contrato en lugar de un *record* copiado por cada lado.
- **Observabilidad.** Un colector y un almacén persistentes, muestreo bajo en vez del 100 %, métricas recogidas y dibujadas por un sistema externo, alertas, métricas de negocio en adaptadores y logs centralizados. [33] trata la transparencia del sistema en producción como parte del diseño, no como un extra.
- **Despliegue.** Un orquestador (Kubernetes está fuera del alcance de este curso) y *health-checks* que distingan «el proceso vive» de «puedo atender peticiones».
- **La pregunta previa.** Si el sistema necesita microservicios. Un monolito hexagonal (capítulos 3 a 8) ya separa el núcleo de los detalles; los servicios añaden despliegue independiente a cambio de red, consistencia eventual y operación. [32] dedica el libro a decidir cuándo compensa.

15.9 Ejercicios

Los ejercicios parten del proyecto tal como lo dejó el capítulo 14, más lo que has hecho en las secciones 15.3 a 15.6. El primero activa Actuator en un servicio, paso a paso; el segundo se resuelve leyendo logs; el tercero arregla un Compose que no arranca; el cuarto es la comprobación del árbol de la verificación de estructura 10; el quinto junta todo en el hito; y el último diseña un servicio que no se llega a construir.

Ejercicio 15.1: Actuator en Quests: salud pública, métricas con token ★★☆☆☆

Vas a activar Actuator solo en Quests, en modo memoria y sin Compose ni trazas. Parte del estado del capítulo 14, con el emisor y `hero` ya configurados.

1. Añade el *starter* de Actuator a `services/quests/bootstrap/build.gradle`.
2. En `config-repo/application.yml`, publica por HTTP `health` y `metrics`.
3. En el `SecurityConfig` de Quests, añade como *primera* regla una que deje pasar sin token `/actuator/health/**`.
4. Arranca RabbitMQ, el Config Server, el emisor, Adventurers y Quests (reinicia Quests si ya estaba en marcha: la configuración se lee al arrancar).
5. Sin token, pide `/actuator/health` y `/actuator/metrics` de Quests (puerto 8081) mostrando solo el código HTTP.
6. Con `MASTER`, publica tres misiones y lee una de ellas con `GET /quests/<id>` dos veces.
7. Con `MASTER`, pide `/actuator/metrics`, busca `http.server.requests` en la lista y pídela por su nombre. Después filtra por `uri:/quests` y por `uri:/quests/{id}`.

Está bien si: el paso 5 da 200 para la salud y 401 para las métricas; el listado del paso 7 incluye `http.server.requests`; el filtro por `uri:/quests` da `COUNT` igual a `3.0` (reinicia Quests antes de contar, para que el contador parta de cero; las tres publicaciones, y solo ellas, porque son las únicas con esa plantilla) y el de `uri:/quests/{id}`, `2.0`.

Pista: *las etiquetas se añaden a la URL como `?tag=uri:/quests`; con dos, se unen con `&`, y la URL va entre comillas.*

Pista: *las llaves de `{id}` son un patrón para `curl`; con el parámetro `-g` (globoff) las trata como texto.*

Pista: *la respuesta de `/actuator/metrics` es un JSON con una lista llamada `names`; busca `http.server.requests` en ella.*

Guiado ▪ solución en la pág. 592

Ejercicio 15.2: Reconstruir una misión completada con su traceId ★★★☆☆

Un compañero te pasa estas líneas de `docker compose logs quests adventurers treasury`, ya ordenadas por hora. Los `traceId` están acortados a 16 caracteres y los mensajes son ilustrativos (en el proyecto, solo `LogNotificationSender` escribe «notification to»); sirven para practicar la lectura. El formato es `servicio | hora [traceId-spanId] mensaje`.

Logs

```
quests      | 10:15:01.120 [a91f3c0d7e52b846-1c0d] accepting quest 7c1e
adventurers | 10:15:01.131 [a91f3c0d7e52b846-77e1] GET /adventurers/42ab
quests      | 10:15:01.160 [a91f3c0d7e52b846-1c0d] quest 7c1e accepted
quests      | 10:15:07.502 [5d08e6b2c94a10f7-30aa] completing quest 7c1e
quests      | 10:15:07.540 [5d08e6b2c94a10f7-30aa] quest.completed sent
quests      | 10:15:07.543 [5d08e6b2c94a10f7-30aa] notification to 42ab
treasury    | 10:15:07.571 [5d08e6b2c94a10f7-8f2b] payment 150 for 7c1e
adventurers | 10:15:07.575 [e3b7a1d9066c52f8-2d44] granting 150 xp to 42ab
adventurers | 10:15:07.576 [e3b7a1d9066c52f8-2d44] 42ab is now ADEPT
treasury    | 10:15:09.001 [b2c4d6e8f0a1b3c5-0e91] GET /accounts/42ab
```

Responde:

1. ¿Cuántas trazas distintas hay? Para cada una, di qué operación es (petición del usuario o consumo de un mensaje) y qué servicios intervienen.
2. Reconstruye, en orden, el flujo de completar la misión `7c1e`: qué hace cada servicio y con qué cabecera o mensaje se enteran los siguientes.
3. ¿Qué línea demuestra que el contexto cruzó una llamada REST? ¿Y por qué su `spanId` es distinto al de la línea anterior?
4. Algo no cuadra con la experiencia de Mara: ¿qué ves exactamente, qué significa y qué dos cosas comprobarías?
5. La última línea ocurre dos segundos después de completar. ¿Es parte de ese flujo? ¿Por qué no basta la hora para saberlo?

Está bien si: cuentas cuatro trazas (aceptar, completar, la de Adventurers por su cuenta y la lectura del saldo); describes el flujo de completar como Quests, mensaje a Treasury y Adventurers, con el `traceId` `5d08...` en Quests y Treasury; señalas la línea 2 (mismo `traceId`, otro `spanId` porque es otro tramo); identificas que las líneas de Adventurers llevan otro `traceId` y que eso significa que su consumidor no usó el contexto del mensaje; y dices que la última línea es otra traza.

Pista: *agrupa primero las líneas por el trozo anterior al guion de cada corchete.*

Lectura de código ▪ solución en la pág. 596

Ejercicio 15.3: Compose: Quests no encuentra su base de datos ★★★☆☆

Alguien te pasa esta versión del Compose y del perfil, con el resto del sistema (RabbitMQ, Config Server, los otros servicios) como en el capítulo. Al hacer `docker compose up -d --wait quests`, Quests se reinicia sin parar.

compose.yaml (fragmento)

```
  postgres:
    image: postgres:18
    environment:
      POSTGRES_DB: guildboard
      POSTGRES_USER: guildboard
      POSTGRES_PASSWORD: guildboard
    volumes:
      - postgres-data:/var/lib/postgresql
      - ./infra/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro

  quests:
    build: ./services/quests/bootstrap
    environment:
      SPRING_PROFILES_ACTIVE: docker,postgres
      SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    depends_on:
      - postgres
      - rabbitmq
      - config-server
```

config-repo/application-docker.yml (fragmento)

```
guildboard:
  db:
    host: localhost
```

Los logs de Quests muestran, con el fragmento tal cual, el primer fallo. El segundo es lo que verías en algunos arranques después de arreglar solo la dirección:

```
Caused by: org.postgresql.util.PSQLException: Connection to
localhost:5432 refused. Check that the hostname and port are correct
and that the postmaster is accepting TCP/IP connections.
...
Caused by: org.postgresql.util.PSQLException: FATAL: the database
system is starting up
```

Encuentra y arregla los tres defectos del fragmento (uno por cada problema de arranque: una dirección y dos relativos al orden) y explica por qué un `depends_on` en forma de lista no basta.

Está bien si: tras `docker compose down -v` y `docker compose up -d --wait quests`, Quests queda en ejecución sin reiniciarse (`docker compose ps quests` no muestra reinicios); `docker compose exec postgres psql -U guildboard -d quests -c '\dt'` lista la tabla `quests`; y `docker compose logs quests` ya no contiene `localhost:5432`.

Pista: *Compose no sabe si PostgreSQL está listo hasta que alguien se lo dice: mira cómo lo hacen* `rabbitmq` *y el* `postgres` *del capítulo.*

Depuración ▪ solución en la pág. 597

Ejercicio 15.4: La lista de comprobación sobre el árbol final ★★★☆☆

Verificación de estructura 10. Tienes el árbol final de la sección 15.1 y estos datos del repositorio, sacados con `grep` y con las líneas `project(...)` de los `build.gradle`:

Dependencias entre proyectos Gradle

```
<s>-domain        ninguna (solo Lombok)
<s>-application   api <s>-domain
<s>-adapters      implementation <s>-application
<s>-bootstrap     implementation <s>-adapters, <s>-application
gateway           ninguna
config-server     ninguna
token-issuer      ninguna
(<s> = quests, adventurers, treasury: doce proyectos de servicios)
```

Imports que importan

```
<s>-domain, <s>-application: ning^^c3^^ban import de org.springframework,
    jakarta ni io.micrometer
quests-adapters:   org.springframework.web.client.RestClient,
                   io.github.resilience4j.*
quests-bootstrap:  org.springframework.web.client.RestClient
                   (BeanConfig), org.springframework.amqp.*
gateway, config-server, token-issuer: sin carpeta domain ni application
```

Haz esto:

1. Ejecuta `./gradlew test` y pasa la lista de comprobación reducida (siete preguntas). Entrega una tabla con las siete, cada una con «sí», «no» o «parcial» y una razón en una línea; en cada fila di si la cubren las pruebas o se revisa a mano.
2. Di qué proyectos, si alguno, violan la regla de dependencia. Si no hay ninguno, escribe «ninguno» y dile al lector de qué dato lo sacas.
3. `bootstrap` depende de `adapters` y de `application` a la vez y es el único que ve «todo». ¿Es una violación? Razónalo con la regla.
4. Imagina que alguien añade a una entidad de `quests-domain` un `io.micrometer.core.instrument.Counter` para contar misiones completadas. ¿Qué pregunta de la lista lo detecta, con qué orden y dónde debería vivir el contador?
5. Nombra tres cosas del árbol final que siguen siendo «parcial» o decisión propia sin fuente que las prescriba.

Está bien si: la tabla tiene siete filas, con la primera apoyada en `./gradlew test` y la distinción entre lo que cubren las pruebas y lo que se revisa a mano y marca como «parcial» al menos la tercera (nombres de carpeta, a mano); ningún proyecto viola la regla (lo prueban `./gradlew test`, la tabla de dependencias, que solo apunta hacia dentro, y el `grep` vacío del núcleo); el razonamiento del punto 3 dice que `bootstrap` es la raíz de composición y que la regla se refiere a las dependencias del *código fuente* hacia dentro, no a quién ensambla; el punto 4 señala la séptima pregunta, el `grep` de `micrometer` y un adaptador o decorador como sitio del contador; y las tres cosas del punto 5 salen de la lista: `gateway`, `config-server` y `token-issuer` sin hexágono; los seis `Dockerfile` idénticos; el archivo `application-docker.yml` único y común.

Pista: *la regla se mira siguiendo las flechas de la primera tabla: ¿alguna sale de un círculo interior hacia uno exterior?*

Lectura de código ▪ solución en la pág. 599

Ejercicio 15.5: Hito H8: GuildBoard completo en Compose, con observabilidad ★★★★☆

Parte del proyecto con el capítulo 14 terminado: los archivos de los capítulos anteriores existen y aquí solo se pide el cambio indicado en cada uno (constructor, bean, cadena de reglas, líneas de `build.gradle`, URL de PostgreSQL); la solución da cada delta con su contexto. Vas a dejarlo como el árbol final de la verificación de estructura 10:

1. **Actuator y trazas.** En los tres `bootstrap` y en el gateway, los *starters* de Actuator y de Zipkin. En el `bootstrap` de Quests, además, el de `restclient`. En los tres `SecurityConfig`, la regla pública de `/actuator/health/**`.
2. **Quests, el cliente.** La fábrica con el método que acepta un `RestClient.Builder`, el constructor nuevo de `RestAdventurerDirectory` y el bean de `BeanConfig`.
3. **Empaquetado.** `archiveFileName = 'app.jar'` en los seis proyectos con el plugin de Boot y los seis `Dockerfile`.
4. **Configuración.** `config-repo/application.yml` con `management`, muestreo y observación de RabbitMQ; `config-repo/application-docker.yml`; y `application-postgres.yml` de los tres servicios con el *host* por omisión.
5. **Compose.** `compose.yaml` con los nueve servicios, sus *healthchecks* y sus `depends_on`, y las dos variables de entorno en cada cliente de configuración.
6. **El recorrido.** `docker compose up -d --build --wait`, el recorrido completo por el 8080 con el paso del huevo y la gallina, la prueba de las trazas y del 503 con Adventurers parado.
7. Haz el *commit* (por ejemplo, «H8: observabilidad y Compose completo»).

Está bien si: `docker compose ps` muestra los nueve servicios con `healthy` los que tienen *healthcheck*; el recorrido da 150 de saldo y `ADEPT` para Mara con el token de `hero`; `/actuator/health` da 200 sin token y `/actuator/metrics` da 401; Zipkin lista `gateway`, `quests` y `adventurers` y la traza de aceptar contiene *spans* de los tres; `./gradlew test` pasa; el `grep` de la sección 15.7.1 no muestra nada; `./gradlew build` termina con `BUILD SUCCESSFUL`; y no hay ningún `application-docker.yml` dentro de un `bootstrap`.

Pista: *si un servicio queda en* `unhealthy` *con 401 en su* healthcheck*, falta la regla pública de* `/actuator/health/**` *en su* `SecurityConfig`*.*

Pista: *si un cliente falla al arrancar diciendo que no encuentra el Config Server, mira que* `SPRING_CONFIG_IMPORT` *use* `config-server`*, no* `localhost`*.*

Pista: *si el gateway devuelve 502 al llamar a un servicio, revisa que las rutas de* `application-docker.yml` *usen el nombre de servicio y que el cliente haya reiniciado tras cambiar* `config-repo`*.*

Proyecto ▪ solución en la pág. 599

Ejercicio 15.6: Un cuarto servicio en papel: Notifications ★★★★★

Hoy Quests notifica con un adaptador de log (`LogNotificationSender`) que escribe una línea. El gremio quiere avisos reales (correo, mensajes) sin cargar al resto. Diseña **en papel** un cuarto servicio, `notifications`. **No se implementa y no ejecuta ninguna**

verificación de estructura (aunque tu diseño cambiaría el árbol, como lo haría cualquier servicio nuevo); el ejercicio es pensar. Entrega:

1. **Por qué un servicio y no una clase o un adaptador más.** Qué argumentos del curso usas (fronteras, datos propios, ritmo de cambio) y qué argumentos hay en contra.
2. **Estructura.** El árbol de `services/notifications` y los cambios en la raíz del repositorio.
3. **Contratos.** Qué mensaje consume, de qué *exchange*, con qué cola y qué cola de mensajes muertos; qué expone por REST, si algo.
4. **Cambios en el resto del sistema**, uno por uno, y cuáles no hacen falta.
5. **El listener y su caso de uso.** El código de la entrada del mensaje y del puerto de salida, en el molde de Treasury.
6. **Riesgos.** Qué pasa si Notifications está caído, si recibe el mensaje dos veces y si el canal de envío falla.

Criterios de valoración (no hay una única respuesta correcta): (1) el argumento principal es que Notifications cambia a otro ritmo y con otras dependencias (canales externos), no «porque es un servicio más»; (2) el árbol repite el molde de cuatro módulos y nombra la raíz (`settings.gradle`, `config-repo/notifications.yml`, `compose.yaml`); (3) consume el evento existente con su propia cola, sin que Quests cambie de código; (4) la lista de cambios distingue lo que hace falta (Compose, Config Server, Gradle) de lo que no (el dominio de los otros servicios, el contrato del evento); (5) el código mantiene la regla de dependencia: el listener es un adaptador de entrada, el canal de envío un puerto de salida y el caso de uso no importa Spring; (6) los riesgos incluyen la idempotencia (un duplicado envía dos avisos) y el mensaje perdido, que el curso reconoce y no resuelve.

Diseño ▪ solución en la pág. 612

15.10 Resumen

Resumen

- Tres señales: las métricas son números agregados, los logs son texto de un proceso y las trazas siguen una petición por varios procesos con un `traceId`. Se complementan. Micrometer es la fachada; la Observation API genera a la vez métrica y *span*, y `domain` y `application` no cambian.
- Actuator publica la salud y las métricas por HTTP. Por omisión solo `health`; `management.endpoints.web.exposure.include` añade más. `health` se deja público (el *healthcheck* no lleva token) y `metrics` exige token. `http.server.requests` es un temporizador con `COUNT`, `TOTAL_TIME` y `MAX` y etiquetas como `uri`, que guarda la plantilla.
- Trazas con Brave y Zipkin (`spring-boot-starter-zipkin`) y muestreo del 100 % en el curso (el 10 % por omisión). Boot añade el `traceId` a los logs solo. El contexto cruza HTTP con el cliente construido desde el *builder* autoconfigurado (el `RestClient.builder()` estático no propaga: se arregló la fábrica de Quests) y por RabbitMQ con la observación del `RabbitTemplate` y del listener, que está diseñada pero no verificada.
- Un `Dockerfile` copia el *jar* de `bootJar` (con `archiveFileName = 'app.jar'`, porque el plugin deja también el *jar* `-plain`) sobre `eclipse-temurin:21-jre`. Los seis son

idénticos: todo lo que varía entra por configuración.

- En Compose, el nombre del servicio es el nombre de máquina y `localhost` dentro de un contenedor es él mismo. El perfil `docker` (`config-repo/application-docker.yml`) cambia las direcciones que se *llaman*; no cambia `issuer-uri`, que se *compara*. Las listas (rutas del gateway) se reemplazan, y para PostgreSQL se usa una clave propia con valor por omisión («una clave, un dueño»).
- `SPRING_PROFILES_ACTIVE` y `SPRING_CONFIG_IMPORT` (sin `optional`) se definen en `compose.yaml` de cada cliente, porque la dirección del Config Server no puede salir del Config Server. `healthcheck` más `depends_on` con `service_healthy` ordenan el arranque; no sustituyen a la resiliencia.
- La estructura final (verificación de estructura 10) añade los `Dockerfile`, el archivo de perfil y el Compose completo sin tocar el núcleo. El curso termina con diez verificaciones de estructura y un proyecto cuya regla de dependencia se comprueba con `grep` y con Gradle.

Soluciones de los ejercicios

Solución del ejercicio 15.1 enunciado en la pág. 586

1. El *starter*. En `services/quests/bootstrap/build.gradle`, el archivo entero con la línea nueva:

services/quests/bootstrap/build.gradle

```
plugins {
    id 'org.springframework.boot'
}

dependencyManagement {
    imports {
        mavenBom 'org.springframework.cloud:' +
            'spring-cloud-dependencies:2025.1.3'
    }
}

dependencies {
    implementation project(':quests-adapters')
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-security-oauth2-resource-server'
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-actuator'
    implementation 'io.github.resilience4j:' +
        'resilience4j-spring-boot4:2.4.0'
    implementation 'org.springframework.cloud:' +
        'spring-cloud-starter-config'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}
```

2. La publicación. `config-repo/application.yml`, entero, con el bloque `management` al final:

config-repo/application.yml

```yaml
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: guildboard
    password: guildboard
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:9000
          jwk-set-uri: http://localhost:9000/.well-known/jwks.json

guildboard:
  messaging:
    exchange: guildboard.events
    routing-key: quest.completed

management:
  endpoints:
    web:
      exposure:
        include: health,metrics
```

3. La regla pública. `SecurityConfig` de Quests, entera; la línea nueva es la primera de la cadena de reglas:

adapter/in/web/security/SecurityConfig.java (Quests)

```java
package com.guildboard.quests.adapter.in.web.security;

import org.springframework.context.annotation.Bean;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.
    ↪ HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.oauth2.server.resource.authentication.
    ↪ JwtAuthenticationConverter;
import org.springframework.security.oauth2.server.resource.authentication.
    ↪ JwtGrantedAuthoritiesConverter;
import org.springframework.security.web.SecurityFilterChain;

public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**").permitAll()
                .requestMatchers(HttpMethod.POST, "/quests")
                    .hasRole("GUILD_MASTER")
                .requestMatchers(HttpMethod.POST, "/quests/*/accept")
                    .hasRole("ADVENTURER")
                .requestMatchers(HttpMethod.POST, "/quests/*/complete")
                    .hasAnyRole("GUILD_MASTER", "ADVENTURER")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth -> oauth
                .jwt(jwt -> jwt
                    .jwtAuthenticationConverter(rolesConverter())));
        return http.build();
    }

    private static JwtAuthenticationConverter rolesConverter() {
        JwtGrantedAuthoritiesConverter authorities =
            new JwtGrantedAuthoritiesConverter();
        authorities.setAuthoritiesClaimName("roles");
        authorities.setAuthorityPrefix("ROLE_");
        JwtAuthenticationConverter converter =
            new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(authorities);
        return converter;
    }
}
```

4. Probar. Con todo arrancado, `MASTER` definido y las funciones `jid` y `code` del capítulo 14 (reinicia Quests antes, para que los recuentos partan de cero):

```
$ code localhost:8081/actuator/health
200
$ code localhost:8081/actuator/metrics
401
$ P='{"title":"Misi^^c3^^b3n","reward":10,"requiredRank":"NOVICE"}'
$ Q1=$(curl -s -X POST localhost:8081/quests \
    -H "Authorization: Bearer $MASTER" \
    -H 'Content-Type: application/json' -d "$P" | jid)
$ for i in 1 2; do curl -s -o /dev/null -X POST localhost:8081/quests \
>   -H "Authorization: Bearer $MASTER" \
>   -H 'Content-Type: application/json' -d "$P"; done
$ for i in 1 2; do curl -s -o /dev/null localhost:8081/quests/$Q1 \
>   -H "Authorization: Bearer $MASTER"; done
$ B=localhost:8081/actuator/metrics/http.server.requests
$ curl -g -s "$B?tag=uri:/quests" -H "Authorization: Bearer $MASTER"
$ curl -g -s "$B?tag=uri:/quests/{id}" -H "Authorization: Bearer $MASTER"
```

Son tres publicaciones y dos lecturas. Las dos últimas respuestas llevan `"statistic":"COUNT","value":3.0` y `"value":2.0`, respectivamente (forma esperada, no ejecutada).

Por qué funciona. El *starter* registra los *endpoints*; la propiedad publica `metrics` (por omisión solo se publica `health`); la regla `permitAll` va primera porque gana la primera que casa y `anyRequest().authenticated()` habría exigido token a la salud. Las métricas piden token porque caen en esa última regla. La etiqueta `uri` es la plantilla del controlador: las tres publicaciones usan `/quests` y las dos lecturas usan `/quests/{id}`, que son series distintas.

Errores típicos. (1) `curl` sin `-g` falla con «bad range in URL» o interpreta `{id}` como un patrón. (2) La salud da 401: falta la regla o está después de `anyRequest()`, o Quests no se reinició. (3) `/actuator/metrics` da 404 en vez de 401: la propiedad de `config-repo` no se publicó o Quests no se reinició (la configuración se lee al arrancar). (4) Los recuentos salen más altos: Quests no se reinició y trae peticiones de antes.

Solución del ejercicio 15.2

enunciado en la pág. 586

1. Las trazas. Agrupando por el trozo antes del guion de cada corchete hay **cuatro**:

traceId	Petición	Servicios
`a91f3c0d...`	Aceptar la misión `7c1e`	Quests y Adventurers (líneas 1 a 3)
`5d08e6b2...`	Completar la misión	Quests y Treasury (líneas 4 a 7)
`e3b7a1d9...`	Experiencia de Mara	Solo Adventurers (líneas 8 y 9)
`b2c4d6e8...`	Leer el saldo	Solo Treasury (línea 10)

2. El flujo de completar. Quests recibe la petición y la procesa (línea 4), publica `quest.completed` en el *exchange* (línea 5) y avisa a Mara con `NotificationSender` (línea 6). El mensaje llega a la cola de Treasury, cuyo listener paga 150 (línea 7) y comparte el `traceId` de Quests: el contexto viajó en una cabecera del mensaje y el listener abrió un *span* hijo (otro `spanId`, `8f2b`). Hasta aquí, es el flujo esperado. El mismo mensaje llega a la cola de Adventurers, que da la experiencia y asciende a Mara (líneas 8 y 9).

3. La llamada REST. La línea 2: es de Adventurers y lleva el `traceId` de Quests (`a91f...`). Su `spanId` es otro (`77e1`) porque cada servicio abre su propio *span* (su propio tramo) dentro de la misma traza. El `traceId` no cambia; el `spanId` sí.

4. La anomalía. Las líneas 8 y 9 llevan un `traceId` distinto (`e3b7...`) al de Quests y Treasury, con lo que **el listener de Adventurers no usó el contexto del mensaje**: su trabajo quedó en una traza propia que ni la UI de Zipkin ni el `grep` del `traceId` de Quests conectan con la petición. El mismo mensaje llegó a Treasury con el contexto bien puesto, así que el fallo es del lado de Adventurers y no del emisor. Dos comprobaciones: (1) qué configuración tiene realmente Adventurers: `curl localhost:8888/adventurers/default` muestra las propiedades que sirve el Config Server y debe incluir las de observación del listener; (2) si ya las tiene, reinícialo, porque lee la configuración al arrancar y pudo arrancar antes de que las añadieras. Si tras ambas cosas sigue igual, el puente de mensajería no está funcionando en ese lado, y se aplica la salida honesta del capítulo: la traza cubre REST, y los listeners se correlacionan por el `questId`.
5. La última línea. No: es la traza `b2c4...`, una lectura que no tiene nada que ver con la anterior. La hora no basta porque los servicios atienden muchas peticiones a la vez y en un sistema real las líneas de dos peticiones se entrelazan al milisegundo. Para eso existe el identificador.
Error típico. Agrupar por servicio o por hora: el `traceId` es el único criterio que separa peticiones concurrentes.

Solución del ejercicio 15.3 enunciado en la pág. 587

Hay tres defectos, que se descubren en este orden:

1. **La dirección.** Con `host: localhost`, Quests busca PostgreSQL en su propio contenedor. Por eso el primer fallo sale siempre, y tapa a los demás.
2. **PostgreSQL no tiene *healthcheck*.** Compose solo sabe que el proceso existe, no que acepta conexiones.
3. **`depends_on` en forma de lista.** Equivale a «arrancar después de», sin esperar a que esté sano. Hace falta la forma larga con `condition: service_healthy`.

El Compose arreglado (PostgreSQL con la comprobación que ya usaba el proyecto; RabbitMQ y el Config Server deben tenerla también, como en el capítulo):

compose.yaml (fragmento corregido)

```yaml
  postgres:
    image: postgres:18
    environment:
      POSTGRES_DB: guildboard
      POSTGRES_USER: guildboard
      POSTGRES_PASSWORD: guildboard
    volumes:
      - postgres-data:/var/lib/postgresql
      - ./infra/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U guildboard -d guildboard"]
      interval: 5s
      timeout: 3s
      retries: 10

  quests:
    build: ./services/quests/bootstrap
    environment:
      SPRING_PROFILES_ACTIVE: docker,postgres
      SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    depends_on:
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
      config-server:
        condition: service_healthy
```

config-repo/application-docker.yml (fragmento corregido)

```yaml
guildboard:
  db:
    host: postgres
```

Por qué funciona. El nombre del servicio, `postgres`, es el nombre de máquina dentro de la red de Compose; `localhost` no lo es. El *healthcheck* da a Compose una señal de «listo» (`pg_isready` termina bien cuando PostgreSQL acepta conexiones) y `service_healthy` hace que Quests espere a esa señal. Fíjate en que `pg_isready` corre *dentro* del contenedor de PostgreSQL, donde `localhost` sí es PostgreSQL: el `localhost` no es un error en sí, depende de quién lo mire. Tras cambiarlo, hay que reiniciar Quests para que relea la configuración: `docker compose restart quests`.

Errores típicos. (1) Arreglar solo la dirección: el arranque funciona casi siempre y falla a veces con «the database system is starting up». (2) Poner el *healthcheck* y dejar la lista simple: el *healthcheck* existe pero nadie lo espera. (3) Probar sin `down -v`: el volumen conserva datos de una prueba anterior y no se ejecuta de nuevo `init.sql`; para ver el arranque desde cero se borra el volumen. (4) Cambiar `application-docker.yml` y no reiniciar Quests.

Solución del ejercicio 15.4 enunciado en la pág. 589

1. La tabla. Primero, desde la raíz, `./gradlew test`: debe terminar con `BUILD SUCCESSFUL`. Las pruebas cubren la primera fila; el resto se revisa a mano.

N.º	Veredicto	Razón
1	sí	`./gradlew test` pasa: `ArchitectureTest` (núcleo sin Spring ni dependencias hacia fuera) y `StructureTest`. Lo cubren las pruebas.
2	sí	El `healthcheck` de `/actuator/health` y la regla de seguridad están en `in`; la fábrica del `RestClient`, en `out`. A mano.
3	parcial	`Dockerfile`, `compose.yaml` y `tools` son nombres de herramienta o genéricos; los de negocio (`services/quests`) sí gritan. A mano.
4	sí	Seis `Dockerfile`, un perfil, el Compose, dependencias de operación y la fábrica de Quests; el porqué es observar y arrancar todo. A mano.
5	sí	Ninguna fila de la tabla nombra a otro servicio; `depends_on` es del Compose. A mano (las pruebas vigilan las clases, no el `build.gradle`).
6	sí	`gateway`, `config-server` y `token-issuer` no contienen reglas de negocio. A mano.
7	sí	Ningún `micrometer`, `actuator`, `zipkin` ni `docker` en el núcleo; lo comprueba el `grep`, no las pruebas.

2. Violaciones. Ninguna. Los datos: en la tabla de dependencias todas las flechas van de fuera a dentro (`bootstrap` a `adapters` y `application`; `adapters` a `application`; `application` a `domain`); `domain` no depende de nada; y los `import` sospechosos (`RestClient`, Resilience4j, AMQP) están en `adapters` y `bootstrap`, no en el núcleo.

3. `bootstrap`. No es una violación. La regla de dependencia [27] dice que las dependencias del código fuente apuntan hacia dentro; `bootstrap` es el anillo más exterior y la raíz de composición, el sitio donde se crean los objetos concretos y se enchufan a los puertos. Que dependa de todo no lo convierte en un problema: lo es que algo interior dependa de él, y nada lo hace.

4. El contador. La detecta la **séptima pregunta**, con `grep -rliE 'micrometer|actuator|zipkin|docker' services/*/domain/src services/*/application/src`, que ya no saldría vacío. El contador debe vivir en un **adaptador** (o en un decorador del caso de uso, definido en `adapters` o `bootstrap`) que cuente cuando el caso de uso termina; el dominio ni sabe que se mide.

5. Tres «parciales» o decisiones propias. (a) `gateway`, `config-server` y `token-issuer` no tienen hexágono (límite parcial, capítulo 13). (b) Los seis `Dockerfile` idénticos: duplicación aceptada para no inventar un módulo común de despliegue. (c) Un único `application-docker.yml` común, que mezcla las rutas del gateway con claves que solo leen los servicios. (También valen: `tools` y `infra` fuera de `services`, o `SecurityConfig` repetida en tres servicios.)

Error típico. Marcar `bootstrap` como violación por «ver todo» o confundir el `depends_on` de Compose con una dependencia de proyecto Gradle.

Solución del ejercicio 15.5 enunciado en la pág. 590

Se hace por pasos, de dentro afuera. Los pasos 1 a 4 dejan los servicios listos para observarse; los pasos 5 y 6, el Compose.

1. Starters y nombre del *jar*. `services/quests/bootstrap/build.gradle`, entero:

`services/quests/bootstrap/build.gradle`

```
plugins {
    id 'org.springframework.boot'
}

dependencyManagement {
    imports {
        mavenBom 'org.springframework.cloud:' +
            'spring-cloud-dependencies:2025.1.3'
    }
}

dependencies {
    implementation project(':quests-adapters')
    implementation project(':quests-application')
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    implementation 'org.springframework.boot:spring-boot-starter-amqp'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-flyway'
    implementation 'org.flywaydb:flyway-database-postgresql'
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-security-oauth2-resource-server'
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-actuator'
    implementation 'org.springframework.boot:spring-boot-starter-zipkin'
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-restclient'
    implementation 'io.github.resilience4j:' +
        'resilience4j-spring-boot4:2.4.0'
    implementation 'org.springframework.cloud:' +
        'spring-cloud-starter-config'
    runtimeOnly 'org.postgresql:postgresql'

    testImplementation 'com.tngtech.archunit:archunit:1.5.1'
}

tasks.named('bootJar') {
    archiveFileName = 'app.jar'
}
```

En `adventurers/bootstrap/build.gradle`, `treasury/bootstrap/build.gradle` y `gateway/build.gradle`: dentro de `dependencies`, estas líneas (sin la de `restclient`, que es solo de Quests; la línea de `archunit` se queda en los `bootstrap` y no va en el `gateway`), y, al final del archivo, el bloque `bootJar` de arriba. En `config-server/build.gradle` y `tools/token-issuer/build.gradle`, solo el bloque `bootJar`:

`Líneas nuevas en adventurers, treasury y gateway`

```
    implementation 'org.springframework.boot:' +
        'spring-boot-starter-actuator'
    implementation 'org.springframework.boot:spring-boot-starter-zipkin'
```

2. Seguridad: la salud pública. En Quests, la clase del ejercicio 15.1. En Adventurers y Treasury, solo cambia la cadena de reglas (el resto de cada clase es el del capítulo 14):

`SecurityConfig.java (Adventurers y Treasury, solo la cadena)`

```
// Adventurers
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**").permitAll()
                .requestMatchers(HttpMethod.POST, "/adventurers")
                    .hasRole("GUILD_MASTER")
                .anyRequest().authenticated())

// Treasury (no importa HttpMethod; sin esa regla)
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**").permitAll()
                .anyRequest().authenticated())
```

3. Quests: el cliente instrumentado. La fábrica entera, el constructor nuevo de `RestAdventurerDirectory` y el bean (añade el *import* `org.springframework.web.client.RestClient` a `BeanConfig`):

adapter/out/rest/AdventurerClientFactory.java

```
package com.guildboard.quests.adapter.out.rest;

import java.net.http.HttpClient;
import java.time.Duration;
import org.springframework.http.client.JdkClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

public final class AdventurerClientFactory {

    private AdventurerClientFactory() {
    }

    public static RestClient create(String baseUrl,
            Duration connectTimeout, Duration readTimeout) {
        return create(RestClient.builder(), baseUrl,
            connectTimeout, readTimeout);
    }

    public static RestClient create(RestClient.Builder builder,
            String baseUrl, Duration connectTimeout,
            Duration readTimeout) {
        HttpClient http = HttpClient.newBuilder()
            .connectTimeout(connectTimeout)
            .build();
        JdkClientHttpRequestFactory factory =
            new JdkClientHttpRequestFactory(http);
        factory.setReadTimeout(readTimeout);
        return builder
            .baseUrl(baseUrl)
            .requestFactory(factory)
            .requestInterceptor(new BearerTokenForwardingInterceptor())
            .build();
    }
}
```

RestAdventurerDirectory.java (constructor nuevo) y BeanConfig (bean)

```
    // RestAdventurerDirectory: junto a los otros dos constructores
    public RestAdventurerDirectory(RestClient.Builder builder,
            String baseUrl, Duration connectTimeout,
            Duration readTimeout, CircuitBreaker breaker, Retry retry) {
        this(AdventurerClientFactory.create(
                builder, baseUrl, connectTimeout, readTimeout),
            breaker, retry);
    }

    // BeanConfig: sustituye al bean adventurerDirectory
    @Bean
    public AdventurerDirectory adventurerDirectory(
            RestClient.Builder restClientBuilder,
            @Value("${guildboard.adventurers.base-url}") String baseUrl,
            @Value("${guildboard.adventurers.connect-timeout}")
            Duration connectTimeout,
            @Value("${guildboard.adventurers.read-timeout}")
            Duration readTimeout,
            CircuitBreakerRegistry breakers, RetryRegistry retries) {
        return new RestAdventurerDirectory(restClientBuilder, baseUrl,
            connectTimeout, readTimeout,
            breakers.circuitBreaker("adventurers"),
            retries.retry("adventurers"));
    }
```

4. Los seis `Dockerfile`. Crea el de Quests y cópialo:

services/quests/bootstrap/Dockerfile

```
FROM eclipse-temurin:21-jre
RUN useradd --system --no-create-home guildboard
WORKDIR /app
COPY build/libs/app.jar app.jar
USER guildboard
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
$ for d in services/adventurers/bootstrap services/treasury/bootstrap \
    gateway config-server tools/token-issuer
> do cp services/quests/bootstrap/Dockerfile "$d/Dockerfile"; done
```

5. La configuración. `config-repo/application.yml`, entero (los tres bloques nuevos son `template`, `listener` y `management`):

config-repo/application.yml

```yaml
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: guildboard
    password: guildboard
    template:
      observation-enabled: true
    listener:
      simple:
        observation-enabled: true
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:9000
          jwk-set-uri: http://localhost:9000/.well-known/jwks.json

guildboard:
  messaging:
    exchange: guildboard.events
    routing-key: quest.completed

management:
  endpoints:
    web:
      exposure:
        include: health,metrics
  tracing:
    sampling:
      probability: 1.0
```

`config-repo/application-docker.yml`, entero:

config-repo/application-docker.yml (1 de 2)

```yaml
spring:
  rabbitmq:
    host: rabbitmq
  security:
    oauth2:
      resourceserver:
        jwt:
          jwk-set-uri: http://token-issuer:9000/.well-known/jwks.json
  cloud:
    gateway:
      server:
        webmvc:
          routes:
            - id: quests
              uri: http://quests:8081
              predicates:
                - Path=/api/quests/**
              filters:
                - StripPrefix=1
            - id: adventurers
              uri: http://adventurers:8082
              predicates:
                - Path=/api/adventurers/**
              filters:
                - StripPrefix=1
            - id: treasury
              uri: http://treasury:8083
              predicates:
                - Path=/api/treasury/**
              filters:
                - StripPrefix=2
```

config-repo/application-docker.yml (2 de 2: sigue en la raíz del archivo)

```yaml
guildboard:
  db:
    host: postgres
  adventurers:
    base-url: http://adventurers:8082

management:
  tracing:
    export:
      zipkin:
        endpoint: http://zipkin:9411/api/v2/spans
```

Y el `application-postgres.yml` de cada servicio, con la clave nueva; el de Quests, entero (en Adventurers y Treasury cambia solo la línea `url`: la misma, con `adventurers` y `treasury` al final en lugar de `quests`):

`quests-bootstrap: application-postgres.yml`

```yaml
spring:
  autoconfigure:
    exclude: []
  datasource:
    url: "jdbc:postgresql://${guildboard.db.host:localhost}:5432/quests"
    username: guildboard
    password: guildboard
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: "none"
```

6. El `compose.yaml` (cinco trozos que van seguidos, bajo `services:`; el archivo empieza por esa línea). PostgreSQL y RabbitMQ son los de siempre; Zipkin es nuevo:

compose.yaml (1 de 5)

```yaml
services:
  postgres:
    image: postgres:18
    environment:
      POSTGRES_DB: guildboard
      POSTGRES_USER: guildboard
      POSTGRES_PASSWORD: guildboard
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql
      - ./infra/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U guildboard -d guildboard"]
      interval: 5s
      timeout: 3s
      retries: 10

  rabbitmq:
    image: rabbitmq:4-management
    hostname: rabbitmq
    environment:
      RABBITMQ_DEFAULT_USER: guildboard
      RABBITMQ_DEFAULT_PASS: guildboard
    ports:
      - "5672:5672"
      - "15672:15672"
    volumes:
      - rabbitmq-data:/var/lib/rabbitmq
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 10s
      timeout: 5s
      retries: 10

  zipkin:
    image: openzipkin/zipkin:3
    ports:
      - "9411:9411"
```

compose.yaml (2 de 5)

```yaml
  config-server:
    build: ./config-server
    ports:
      - "8888:8888"
    volumes:
      - ./config-repo:/config-repo:ro
    healthcheck:
      test: ["CMD", "curl", "-fs",
             "http://localhost:8888/application/default"]
      interval: 5s
      timeout: 3s
      retries: 20
      start_period: 20s

  token-issuer:
    build: ./tools/token-issuer
    ports:
      - "9000:9000"
    healthcheck:
      test: ["CMD", "curl", "-fs",
             "http://localhost:9000/.well-known/jwks.json"]
      interval: 5s
      timeout: 3s
      retries: 20
      start_period: 20s
```

compose.yaml (3 de 5)

```yaml
  adventurers:
    build: ./services/adventurers/bootstrap
    environment:
      SPRING_PROFILES_ACTIVE: docker,postgres
      SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    ports:
      - "8082:8082"
    depends_on:
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
      zipkin:
        condition: service_healthy
      config-server:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-fs",
             "http://localhost:8082/actuator/health"]
      interval: 5s
      timeout: 3s
      retries: 30
      start_period: 30s

  treasury:
    build: ./services/treasury/bootstrap
    environment:
      SPRING_PROFILES_ACTIVE: docker,postgres
      SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    ports:
      - "8083:8083"
    depends_on:
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
      zipkin:
        condition: service_healthy
      config-server:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-fs",
             "http://localhost:8083/actuator/health"]
      interval: 5s
      timeout: 3s
      retries: 30
      start_period: 30s
```

compose.yaml (4 de 5)

```
  quests:
    build: ./services/quests/bootstrap
    environment:
      SPRING_PROFILES_ACTIVE: docker,postgres
      SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    ports:
      - "8081:8081"
    depends_on:
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
      zipkin:
        condition: service_healthy
      config-server:
        condition: service_healthy
      adventurers:
        condition: service_healthy
      treasury:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-fs",
             "http://localhost:8081/actuator/health"]
      interval: 5s
      timeout: 3s
      retries: 30
      start_period: 30s
```

compose.yaml (5 de 5)

```yaml
  gateway:
    build: ./gateway
    environment:
      SPRING_PROFILES_ACTIVE: docker
      SPRING_CONFIG_IMPORT: configserver:http://config-server:8888
    ports:
      - "8080:8080"
    depends_on:
      config-server:
        condition: service_healthy
      zipkin:
        condition: service_healthy
      quests:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-fs",
             "http://localhost:8080/actuator/health"]
      interval: 5s
      timeout: 3s
      retries: 30
      start_period: 30s

volumes:
  postgres-data:
  rabbitmq-data:
```

7. El recorrido. Las tres funciones de ayuda, los pasos de la sección 15.7.1 y el *commit*:

```
$ jid() { grep -o '"id":"[^"]*"' | head -1 | cut -d'"' -f4; }
$ code() { curl -s -o /dev/null -w '%{http_code}\n' "$@"; }
$ tok() { curl -s -X POST localhost:9000/token \
>   -H 'Content-Type: application/json' \
>   -d "{\"username\":\"$1\",\"password\":\"$1\"}" |
>   grep -o '"access_token":"[^"]*"' | cut -d'"' -f4; }
$ ./gradlew bootJar
$ docker compose up -d --build --wait
$ docker compose ps
$ git add -A && git commit -m "H8: observabilidad y Compose completo"
```

Por qué funciona. Cada servicio recibe de Compose solo dos variables; el Config Server le sirve el `application.yml` común, el de su nombre y, por el perfil, el `application-docker.yml`, que cambia las direcciones de `localhost` a nombres de servicio. La URL de PostgreSQL usa un valor por omisión que solo el perfil sustituye. La traza REST cruza Quests porque su cliente sale del *builder* de Boot. Y el núcleo no cambia: el `git diff` sobre `domain` y `application` sale vacío.

Errores típicos. (1) No reconstruir con `bootJar`: se arranca el *jar* viejo. (2) La regla de salud en dos servicios y no en el tercero: ese queda *unhealthy*. (3) Olvidar `ports` en el gateway y no poder llamarlo desde tu máquina. (4) Copiar la URL de PostgreSQL con `quests` en los tres servicios: Adventurers y Treasury migrarían la base equivocada. (5) Dejar la lista de rutas del perfil a medias: sustituye entera a la de `gateway.yml`, no se fusiona.

Solución del ejercicio 15.6 enunciado en la pág. 590

Es una propuesta de referencia; no forma parte del proyecto y no se construye.

1. Por qué un servicio. A favor: las notificaciones cambian a otro ritmo (nuevos canales, plantillas), dependen de sistemas externos (correo, mensajería) que tienen sus propias caídas, y no deben retrasar ni romper la finalización de misiones: el servicio consume el evento que ya existe. Es el argumento de fronteras del capítulo 9. En contra: un proceso más que operar, otra imagen, otra cola, y a un equipo pequeño le sale más barato un segundo adaptador de salida de `NotificationSender` en Quests. La decisión depende del equipo y del número de canales.

2. Estructura. El molde de los demás servicios, sin base de datos:

Árbol propuesto

```
services/notifications/
|-- domain/       Notification, AdventurerId [copia]
|-- application/  port/in/{SendNotificationCommand,
|                          SendNotificationUseCase}
|                 port/out/NotificationChannel
|                 service/SendNotificationService
|-- adapters/     adapter/in/messaging/{QuestCompleted,
|                                       QuestCompletedListener}
|                 adapter/out/log/LogNotificationChannel
`-- bootstrap/    NotificationsApplication,
                  config/{BeanConfig, MessagingConfig},
                  resources/application.yml, Dockerfile
```

En la raíz: `notifications` en el bucle de `settings.gradle`; un `config-repo/notifications.yml` (puerto 8084, nombres de cola y `management.endpoints.web.exposure.include: health`); un servicio `notifications` en `compose.yaml` (depende de RabbitMQ, Zipkin y Config Server, no de PostgreSQL). `StructureTest` lo vigilaría porque recorre `services/`, y llevaría su propio `ArchitectureTest`. No se ejecuta ninguna verificación ni prueba de estructura, pero el diseño cambiaría el árbol: nuevo servicio y nueva carpeta bajo `services/`.

3. Contratos. Consume el *exchange* `guildboard.events` (tipo topic) con la clave `quest.completed`; su cola es `notifications.quest-completed`, con mensajes muertos en `guildboard.events.dlx` y la cola `notifications.quest-completed.dlq`. El cuerpo es el JSON del capítulo 11: `questId`, `adventurerId` y `reward`, con un *record* propio e ignorando campos desconocidos. No expone REST (solo la salud).

4. Cambios en el resto. *Hacen falta:* Gradle (`settings.gradle`), Config Server (`notifications.yml`) y Compose (el servicio). *No hacen falta:* ningún cambio de código en Quests, Adventurers ni Treasury (el *exchange* entrega una copia a cada cola enlazada), ni en el contrato del evento, ni en el gateway (sin ruta), ni en `application-docker.yml` (ya tiene `rabbitmq` y Zipkin). Decisión: `LogNotificationSender` de Quests se queda hasta que Notifications lo sustituya del todo; retirarlo cambiaría un puerto de `application`.

5. El código.

domain y application (notifications)

```java
// domain: AdventurerId.java (copia propia, como en los otros servicios)
package com.guildboard.notifications.domain;

public record AdventurerId(String value) {
}

// domain: Notification.java
package com.guildboard.notifications.domain;

public record Notification(AdventurerId recipient, String text) {
    public Notification {
        if (recipient == null || text == null || text.isBlank()) {
            throw new IllegalArgumentException("invalid notification");
        }
    }
}

// application/port/in: SendNotificationCommand.java
package com.guildboard.notifications.application.port.in;

import com.guildboard.notifications.domain.AdventurerId;

public record SendNotificationCommand(
        AdventurerId recipient, String text) {
}

// application/port/in: SendNotificationUseCase.java
package com.guildboard.notifications.application.port.in;

public interface SendNotificationUseCase {
    void send(SendNotificationCommand command);
}

// application/port/out: NotificationChannel.java
package com.guildboard.notifications.application.port.out;

import com.guildboard.notifications.domain.Notification;

public interface NotificationChannel {
    void deliver(Notification notification);
}
```

application, adapters y bootstrap (notifications)

```java
// application/service: SendNotificationService.java
package com.guildboard.notifications.application.service;

import com.guildboard.notifications.application.port.in.
    ↪ SendNotificationCommand;
import com.guildboard.notifications.application.port.in.
    ↪ SendNotificationUseCase;
import com.guildboard.notifications.application.port.out.
    ↪ NotificationChannel;
import com.guildboard.notifications.domain.Notification;
import lombok.RequiredArgsConstructor;

@RequiredArgsConstructor
public class SendNotificationService implements SendNotificationUseCase {

    private final NotificationChannel channel;

    @Override
    public void send(SendNotificationCommand command) {
        channel.deliver(
            new Notification(command.recipient(), command.text()));
    }
}

// adapters/adapter/out/log: LogNotificationChannel.java
package com.guildboard.notifications.adapter.out.log;

import com.guildboard.notifications.application.port.out.
    ↪ NotificationChannel;
import com.guildboard.notifications.domain.Notification;
import lombok.extern.slf4j.Slf4j;

@Slf4j
public class LogNotificationChannel implements NotificationChannel {

    @Override
    public void deliver(Notification notification) {
        log.info("notification to {}: {}",
            notification.recipient().value(), notification.text());
    }
}
```

adapter/in/messaging (notifications)

```java
// QuestCompleted.java
package com.guildboard.notifications.adapter.in.messaging;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;

@JsonIgnoreProperties(ignoreUnknown = true)
public record QuestCompleted(String questId, String adventurerId,
        int reward) {
}

// QuestCompletedListener.java
package com.guildboard.notifications.adapter.in.messaging;

import com.guildboard.notifications.application.port.in.
    ↪ SendNotificationCommand;
import com.guildboard.notifications.application.port.in.
    ↪ SendNotificationUseCase;
import com.guildboard.notifications.domain.AdventurerId;
import lombok.RequiredArgsConstructor;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;

@Component
@RequiredArgsConstructor
public class QuestCompletedListener {

    private final SendNotificationUseCase notifications;

    @RabbitListener(queues = "${guildboard.messaging.queue}")
    public void onQuestCompleted(QuestCompleted event) {
        notifications.send(new SendNotificationCommand(
            new AdventurerId(event.adventurerId()),
            "Quest " + event.questId() + " completed: "
                + event.reward() + " gold"));
    }
}
```

config (notifications), 1 de 2

```java
// BeanConfig.java
package com.guildboard.notifications.config;

import com.guildboard.notifications.adapter.out.log.LogNotificationChannel;
import com.guildboard.notifications.application.port.in.
    ↪ SendNotificationUseCase;
import com.guildboard.notifications.application.port.out.
    ↪ NotificationChannel;
import com.guildboard.notifications.application.service.
    ↪ SendNotificationService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class BeanConfig {

    @Bean
    public NotificationChannel notificationChannel() {
        return new LogNotificationChannel();
    }

    @Bean
    public SendNotificationUseCase sendNotification(
            NotificationChannel channel) {
        return new SendNotificationService(channel);
    }
}
```

config (notifications), 2 de 2

```java
// MessagingConfig.java
package com.guildboard.notifications.config;

import org.springframework.amqp.core.BindingBuilder;
import org.springframework.amqp.core.Declarables;
import org.springframework.amqp.core.DirectExchange;
import org.springframework.amqp.core.Queue;
import org.springframework.amqp.core.QueueBuilder;
import org.springframework.amqp.core.TopicExchange;
import org.springframework.amqp.support.converter.
    ↪ JacksonJsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class MessagingConfig {

    @Bean
    public MessageConverter messageConverter() {
        return new JacksonJsonMessageConverter();
    }

    @Bean
    public Declarables messagingDeclarables(
            @Value("${guildboard.messaging.exchange}") String exchange,
            @Value("${guildboard.messaging.routing-key}") String key,
            @Value("${guildboard.messaging.queue}") String queueName,
            @Value("${guildboard.messaging.dead-letter-exchange}")
            String dlx,
            @Value("${guildboard.messaging.dead-letter-queue}")
            String dlq) {
        TopicExchange events = new TopicExchange(exchange);
        DirectExchange deadLetters = new DirectExchange(dlx);
        Queue queue = QueueBuilder.durable(queueName)
            .deadLetterExchange(dlx)
            .deadLetterRoutingKey(dlq)
            .build();
        Queue deadQueue = QueueBuilder.durable(dlq).build();
        return new Declarables(events, deadLetters, queue, deadQueue,
            BindingBuilder.bind(queue).to(events).with(key),
            BindingBuilder.bind(deadQueue).to(deadLetters).with(dlq));
    }
}
```

6. Riesgos. *Caído:* el *broker* retiene los mensajes en su cola (durable), y los procesa al volver; nadie más se entera, que es la ventaja del acoplamiento por evento. *Duplicado:* un mensaje repetido envía dos avisos; a diferencia de Treasury no hay clave natural de idempotencia (¿se notifica dos veces la misma misión?), pero se puede guardar el `questId` ya notificado. *El canal falla:* el listener lanza la excepción, los reintentos de Spring AMQP se agotan y el mensaje va a la cola de mensajes muertos, a la espera de que alguien lo reencole. Y sigue sin resolverse lo que reconoció el capítulo 11: si Quests guarda y cae antes

de publicar, nadie avisa.
Alternativa válida. Un segundo adaptador de `NotificationSender` dentro de Quests (correo en lugar de log), sin servicio nuevo. Gana en sencillez y pierde en aislamiento: Quests conoce el canal y se cae con él.
Criterios de valoración. Una propuesta es buena si el argumento a favor del servicio es de fronteras y ritmo de cambio; si el árbol sigue el molde y nombra los tres sitios de la raíz; si los contratos reutilizan el evento sin tocar a los demás; si el código mantiene la regla de dependencia (el caso de uso no importa Spring, el listener es un adaptador de entrada y el canal un puerto de salida); y si los riesgos nombran idempotencia y mensaje perdido.

Resumen final

Has recorrido el libro de un monolito en memoria a tres servicios que se hablan por REST y por mensajería. Esto es lo que has aprendido en cada parte.

Parte I, Bases. Repasaste Java 21 moderno y SOLID, y los fundamentos de DDD. Después viste la arquitectura hexagonal con su estructura de carpetas [4] y construiste el primer caso de uso con su endpoint REST y su repositorio en memoria.

Parte II, Hexagonal en un monolito. Pusiste las reglas de negocio en un dominio rico con invariantes, dibujaste los límites (DTO, mapeo manual y errores HTTP), hiciste que Gradle multimódulo impida las dependencias prohibidas y cambiaste la persistencia a PostgreSQL sin tocar dominio ni aplicación.

Parte III, Del hexágono a los microservicios. Cortaste el monolito por sus fronteras, añadiste un tercer servicio con una plantilla reutilizable, comunicaste los servicios con RabbitMQ, los protegiste con timeouts, reintentos y circuit breaker, los pusiste detrás de un API gateway con configuración centralizada, validaste JWT en cada servicio y cerraste con observabilidad y todo el sistema en Docker Compose.

Estructura y código. A lo largo del libro comprobaste la estructura del proyecto en diez verificaciones de estructura, para ver que cada carpeta figuraba en el árbol de su verificación y que el proyecto seguía respetando el hexágono, con pruebas de arquitectura con ArchUnit y `./gradlew test`. Recuerda también que el código Java del libro está revisado a mano, no ejecutado, como se explica en el capítulo 0.

El proyecto final. GuildBoard es ahora un tablón de misiones distribuido en los servicios Quests, Adventurers y Treasury, con gateway, Config Server, emisor de tokens, PostgreSQL, RabbitMQ y Zipkin, arrancable con Docker Compose. Los servicios conservan el mismo hexágono interno que construiste en el monolito. Esa es la idea que más conviene que te lleves: los microservicios cambian dónde viven los hexágonos y cómo se hablan, no cómo se diseña cada uno por dentro.

Referencias

[1] Amazon Web Services. *Hexagonal architecture pattern (AWS Prescriptive Guidance)*. URL: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html (visitado 08-10-2026).

[2] Apache Kafka. *Apache Kafka Documentation*. URL: https://kafka.apache.org/documentation/ (visitado 08-10-2026).

[3] Joshua Bloch. *Effective Java*. 3.ª ed. Addison-Wesley Professional, 2018. ISBN: 978-0-13-468599-1.

[4] Alistair Cockburn. *The Hexagonal (Ports & Adapters) Architecture*. HaT Technical Report 2005.02. 4 de sep. de 2005. URL: https://alistair.cockburn.us/hexagonal-architecture/ (visitado 08-10-2026).

[5] Docker. *Docker Compose*. URL: https://docs.docker.com/compose/ (visitado 08-10-2026).

[6] Nicola Dragoni et al. «Microservices: yesterday, today, and tomorrow». En: *arXiv preprint* (2016). DOI: 10.48550/arXiv.1606.04036. arXiv: 1606.04036 [cs.SE]. URL: https://arxiv.org/abs/1606.04036.

[7] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003. ISBN: 978-0-321-12521-7.

[8] Eric Evans. *Domain-Driven Design Reference: Definitions and Pattern Summaries*. Licencia CC BY 4.0; PDF fechado 2015-03. URL: https://www.domainlanguage.com/ddd/reference/ (visitado 08-10-2026).

[9] Martin Fowler. *Anemic Domain Model*. 25 de nov. de 2003. URL: https://martinfowler.com/bliki/AnemicDomainModel.html (visitado 08-10-2026).

[10] Martin Fowler. *Bounded Context*. 15 de ene. de 2014. URL: https://martinfowler.com/bliki/BoundedContext.html (visitado 08-10-2026).

[11] Martin Fowler. *Circuit Breaker*. 6 de mar. de 2014. URL: https://martinfowler.com/bliki/CircuitBreaker.html (visitado 08-10-2026).

[12] Hector Garcia-Molina y Kenneth Salem. «Sagas». En: *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data (SIGMOD '87)*. ACM, 1987. DOI: 10.1145/38713.38742.

[13] Gradle. *Gradle Build Language Reference (Groovy DSL) 9.8.1*. URL: https://docs.gradle.org/current/dsl/index.html (visitado 08-10-2026).

[14] Gradle. *Multi-Project Builds (Gradle User Manual 9.8.1)*. URL: https://docs.gradle.org/current/userguide/multi_project_builds.html (visitado 08-10-2026).

[15] Gradle. *Organizing Gradle Projects (Gradle User Manual 9.8.1)*. URL: https://docs.gradle.org/current/userguide/organizing_gradle_projects.html (visitado 08-10-2026).

[16] Gradle. *Testing in Java & JVM projects (Gradle User Manual 9.8.1)*. URL: https://docs.gradle.org/current/userguide/java_testing.html (visitado 10-10-2026).

[17] Gradle. *The Java Library Plugin (Gradle User Manual 9.8.1)*. URL: https://docs.gradle.org/current/userguide/java_library_plugin.html (visitado 08-10-2026).

[18] Tom Hombergs. *Hexagonal Architecture with Java and Spring*. 3 de nov. de 2019. URL: https://reflectoring.io/spring-hexagonal/ (visitado 08-10-2026).

[19] Tom Hombergs. *Get Your Hands Dirty on Clean Architecture*. 2.ª ed. Packt Publishing, 2023. ISBN: 978-1-80512-837-3.

[20] Tom Hombergs. *buckpal: An example approach for implementing a Clean/Hexagonal Architecture*. Repositorio de ejemplo del libro (2.a edicion). URL: https://github.com/thombergs/buckpal (visitado 08-10-2026).

[21] Tom Hombergs. *DependencyRuleTests.java (buckpal)*. URL: https://raw.githubusercontent.com/thombergs/buckpal/master/src/test/java/io/reflectoring/buckpal/DependencyRuleTests.java (visitado 10-10-2026).

[22] Michael B. Jones, John Bradley y Nat Sakimura. *JSON Web Token (JWT), RFC 7519*. Mayo de 2015. URL: https://www.rfc-editor.org/info/rfc7519 (visitado 08-10-2026).

[23] JUnit. *Build Support: Gradle*. URL: https://docs.junit.org/current/running-tests/build-support.html (visitado 10-10-2026).

[24] JUnit. *JUnit User Guide (6.1.3)*. URL: https://docs.junit.org/current/user-guide/ (visitado 10-10-2026).

[25] Vlad Khononov. *Learning Domain-Driven Design: Aligning Software Architecture and Business Strategy*. O'Reilly Media, 2021. ISBN: 978-1-098-10013-1.

[26] James Lewis y Martin Fowler. *Microservices: a definition of this new architectural term*. 25 de mar. de 2014. URL: https://martinfowler.com/articles/microservices.html (visitado 08-10-2026).

[27] Robert C. Martin. *The Clean Architecture*. 13 de ago. de 2012. URL: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html (visitado 08-10-2026).

[28] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Pearson, 2017. ISBN: 978-0-13-449416-6.

[29] Maven Central. *com.tngtech.archunit:archunit-junit6, metadatos de versiones*. URL: https://repo1.maven.org/maven2/com/tngtech/archunit/archunit-junit6/maven-metadata.xml (visitado 10-10-2026).

[30] Micrometer. *Micrometer Documentation 1.17.1*. URL: https://docs.micrometer.io/micrometer/reference/ (visitado 08-10-2026).

[31] Micrometer. *Micrometer Tracing 1.7.1*. URL: https://docs.micrometer.io/tracing/reference/ (visitado 08-10-2026).

[32] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. 2.ª ed. O'Reilly Media, 2021. ISBN: 978-1-492-03402-5.

[33] Michael T. Nygard. *Release It! Design and Deploy Production-Ready Software*. 2.ª ed. Pragmatic Bookshelf, 2018. ISBN: 978-1-68050-239-8.

[34] OpenJDK. *JDK 21*. 19 de sep. de 2023. URL: https://openjdk.org/projects/jdk/21/ (visitado 08-10-2026).

[35] Jeffrey Palermo. *The Onion Architecture: part 1*. 29 de jul. de 2008. URL: https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/ (visitado 08-10-2026).

[36] PostgreSQL Global Development Group. *PostgreSQL 18 Documentation*. URL: https://www.postgresql.org/docs/current/index.html (visitado 08-10-2026).

[37] Project Lombok. *Using Lombok with Gradle*. URL: https://projectlombok.org/setup/gradle (visitado 08-10-2026).

[38] RabbitMQ. *RabbitMQ Tutorials*. URL: https://www.rabbitmq.com/tutorials (visitado 08-10-2026).

[39] Resilience4j. *CircuitBreaker (documentacion de Resilience4j)*. URL: https://resilience4j.readme.io/docs/circuitbreaker (visitado 08-10-2026).

[40] Resilience4j. *Resilience4j v2.4.0 (soporte para Spring Boot 4 y Spring Cloud 5)*. URL: https://github.com/resilience4j/resilience4j/releases/tag/v2.4.0 (visitado 08-10-2026).

[41] Chris Richardson. *Microservices Patterns: With Examples in Java*. Manning Publications, 2018. ISBN: 978-1-61729-454-9.

[42] Chris Richardson. *Microservices.io: Microservice Architecture pattern language*. URL: https://microservices.io/patterns/index.html (visitado 08-10-2026).

[43] Spring AMQP. *Spring AMQP 4.1.1 Reference*. URL: https://docs.spring.io/spring-amqp/reference/index.html (visitado 08-10-2026).

[44] Spring Boot. *AMQP / RabbitMQ Support (Spring Boot 4.1.1)*. URL: https://docs.spring.io/spring-boot/reference/messaging/amqp.html (visitado 08-10-2026).

[45] Spring Boot. *Apache Kafka Support (Spring Boot 4.1.1)*. URL: https://docs.spring.io/spring-boot/reference/messaging/kafka.html (visitado 08-10-2026).

[46] Spring Boot. *Database Initialization: Flyway and Liquibase (Spring Boot 4.1.1)*. URL: https://docs.spring.io/spring-boot/how-to/data-initialization.html (visitado 08-10-2026).

[47] Spring Boot. *Dependency Versions (Spring Boot 4.1.1)*. URL: https://docs.spring.io/spring-boot/appendix/dependency-versions/coordinates.html (visitado 10-10-2026).

[48] Spring Boot. *Docker Compose (How-to, Spring Boot 4.1.1)*. URL: https://docs.spring.io/spring-boot/how-to/docker-compose.html (visitado 08-10-2026).

[49] Spring Boot. *Observability (Spring Boot 4.1.1)*. URL: https://docs.spring.io/spring-boot/reference/actuator/observability.html (visitado 08-10-2026).

[50] Spring Boot. *Spring Boot 4.0 Migration Guide*. URL: https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide (visitado 08-10-2026).

[51] Spring Boot. *Spring Boot Gradle Plugin Reference Guide 4.1.1*. URL: https://docs.spring.io/spring-boot/gradle-plugin/index.html (visitado 08-10-2026).

[52] Spring Boot. *Spring Boot Reference Documentation 4.1.1*. URL: https://docs.spring.io/spring-boot/index.html (visitado 08-10-2026).

[53] Spring Boot. *SQL Databases (Spring Boot 4.1.1)*. URL: https://docs.spring.io/spring-boot/reference/data/sql.html (visitado 08-10-2026).

[54] Spring Boot. *System Requirements (Spring Boot 4.1.1)*. URL: https://docs.spring.io/spring-boot/system-requirements.html (visitado 08-10-2026).

[55] Spring Cloud. *Spring Cloud (proyecto y tabla de compatibilidad de release trains)*. URL: https://spring.io/projects/spring-cloud (visitado 08-10-2026).

[56] Spring Cloud Config. *Spring Cloud Config 5.0.5 Reference*. URL: https://docs.spring.io/spring-cloud-config/reference/index.html (visitado 08-10-2026).

[57] Spring Cloud Gateway. *Spring Cloud Gateway 5.0.3 Reference*. URL: https://docs.spring.io/spring-cloud-gateway/reference/index.html (visitado 08-10-2026).

[58] Spring Data JPA. *Spring Data JPA 4.1.1 Reference*. URL: https://docs.spring.io/spring-data/jpa/reference/index.html (visitado 08-10-2026).

[59] Spring for Apache Kafka. *Spring for Apache Kafka 4.1.1 Reference*. URL: https://docs.spring.io/spring-kafka/reference/index.html (visitado 08-10-2026).

[60] Spring Framework. *Resilience Features (Spring Framework 7.0.9 Reference)*. URL: https://docs.spring.io/spring-framework/reference/core/resilience.html (visitado 08-10-2026).

[61] Spring Framework. *REST Clients (Spring Framework 7.0.9 Reference)*. URL: https://docs.spring.io/spring-framework/reference/integration/rest-clients.html (visitado 08-10-2026).

[62] Spring Initializr. *Spring Initializr: metadata del cliente (versiones de Boot, Java y dependencias)*. URL: https://start.spring.io/metadata/client (visitado 08-10-2026).

[63] Spring Security. *OAuth 2.0 Resource Server JWT (Spring Security 7.1.1)*. URL: https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html (visitado 08-10-2026).

[64] TNG Technology Consulting. *ArchUnit Javadoc (1.5.0)*. URL: https://javadoc.io/doc/com.tngtech.archunit/archunit/1.5.0 (visitado 10-10-2026).

[65] TNG Technology Consulting. *ArchUnit Releases*. URL: https://github.com/TNG/ArchUnit/releases (visitado 10-10-2026).

[66] TNG Technology Consulting. *ArchUnit User Guide (1.5.1)*. URL: https://www.archunit.org/userguide/html/000_Index.html (visitado 10-10-2026).

[67] TNG Technology Consulting. *ArchUnit: a Java architecture test library*. Licencia Apache 2.0. URL: https://github.com/TNG/ArchUnit (visitado 10-10-2026).

[68] Sebastian Gabriel Trzpiot. *spring-gradle-hexagonal-example*. Repositorio archivado el 2025-02-04; licencia MIT. URL: https://github.com/trzpiot/spring-gradle-hexagonal-example (visitado 08-10-2026).